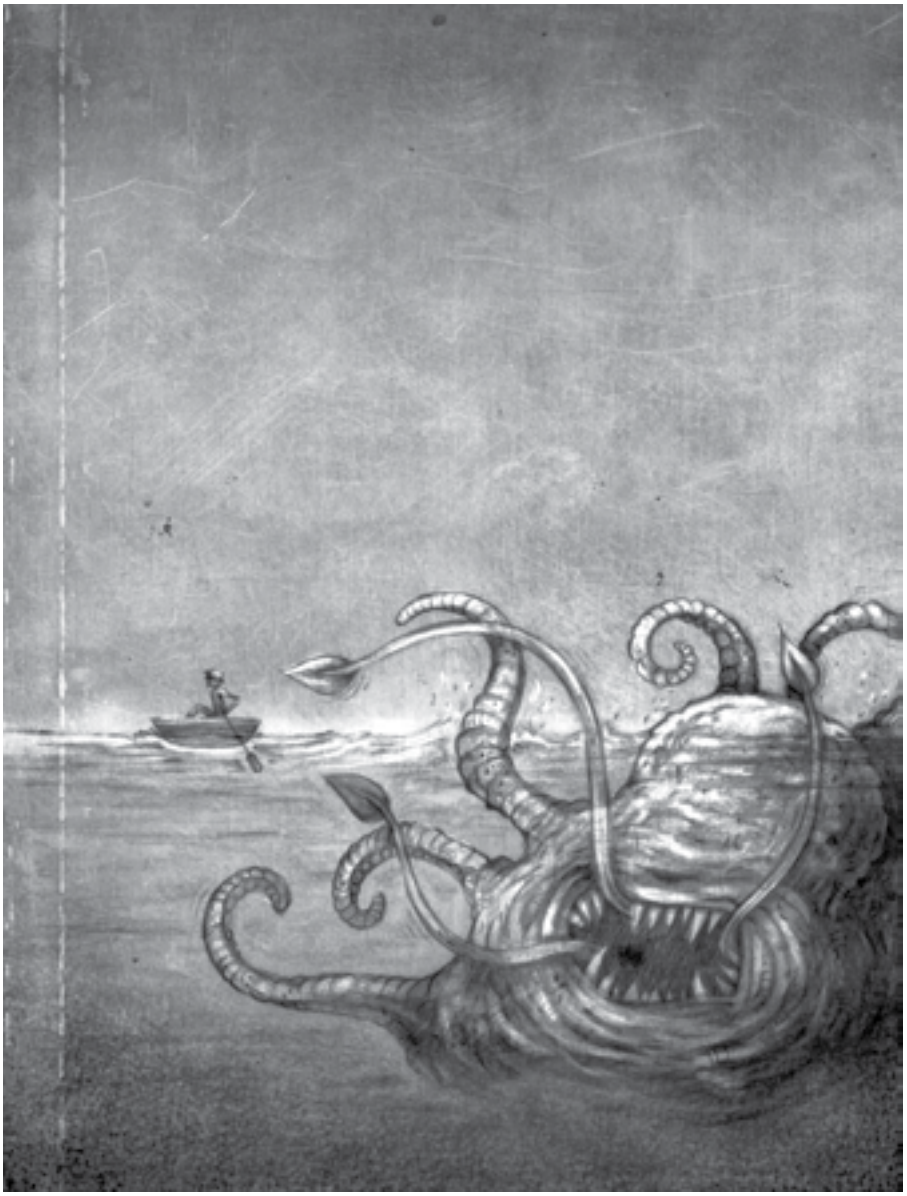
Руткиты и буткиты. Реверс-инжиниринг современных вредоносных программ и угроз нового поколения

Русский перевод практического руководства по исследованию руткитов, буткитов, механизмов загрузки Windows и UEFI, методов реверс-инжиниринга и компьютерной криминалистики.

Больше крутых материалов в моём телеграм-канале @grogrigs

Глава 1. Что внутри руткита: исследование TDL3



В этой главе мы познакомимся с руткитами на примере TDL3. Этот руткит для Windows хорошо иллюстрирует передовые методы перехвата потока управления и потока данных, использующие нижние уровни архитектуры ОС. Мы рассмотрим, как TDL3 заражает систему и как он подменяет отдельные интерфейсы и механизмы ОС, чтобы сохраняться в ней и оставаться незамеченным.

TDL3 применяет механизм заражения, который загружает его код непосредственно в ядро Windows, поэтому меры защиты целостности ядра, введённые Microsoft в 64-разрядных системах Windows, лишили его эффективности. Однако методы, с помощью которых TDL3 встраивает код в ядро, по-прежнему ценны как пример того, каким образом после обхода подобных механизмов контроля целостности можно надёжно и эффективно перехватить выполнение ядра.

Как и у многих других руткитов, перехват путей выполнения кода ядра в TDL3 опирается на ключевые закономерности собственной архитектуры ядра. В некотором смысле перехваты руткита могут служить более точным путеводителем по фактической структуре ядра, чем официальная документация, и уж совершенно точно являются лучшим пособием для понимания недокументированных системных структур и алгоритмов.

И действительно, на смену TDL3 пришёл TDL4, унаследовавший значительную часть функций TDL3 для уклонения от обнаружения и противодействия криминалистическому анализу, но перешедший к методам буткитов, чтобы обходить механизм подписывания кода режима ядра Windows в 64-разрядных системах (эти методы будут описаны в главе 7).

На протяжении всей главы мы будем отмечать конкретные интерфейсы и механизмы ОС, которые подменяет TDL3. Мы объясним, как устроены и работают TDL3 и сходные с ним руткиты, а затем, в части II, обсудим методы и инструменты, позволяющие обнаруживать, наблюдать и анализировать их.

История распространения TDL3

Руткит TDL3, впервые обнаруженный в 2010 году,[1] был одним из наиболее сложных образцов вредоносного ПО, созданных к тому времени. Его механизмы сокрытия стали серьёзным испытанием для всей антивирусной отрасли (как впоследствии и его преемник-буткит TDL4, ставший первым широко распространённым буткитом для платформы x64).

> **Примечание.** Это семейство вредоносного ПО также известно под названиями TDSS, Olmarik и Alureon. Такое обилие названий одного семейства не редкость, поскольку производители антивирусов склонны давать ему разные имена в своих отчётах. Кроме того, исследовательские группы нередко присваивают разные названия отдельным компонентам одной атаки, особенно на ранних этапах анализа.

TDL3 распространялся по бизнес-модели Pay-Per-Install (PPI, оплата за установку) через партнёрские программы DogmaMillions и GangstaBucks (обе впоследствии были закрыты). Популярная среди киберпреступных групп схема PPI похожа на схемы, широко применяемые при распространении панелей инструментов для браузеров. Распространители таких панелей отслеживают их использование, создавая специальные сборки со встроенным уникальным идентификатором (UID) для каждого пакета или комплекта, доступного для загрузки через разные каналы распространения. Это позволяет разработчику подсчитать число установок (число пользователей), связанных с конкретным UID, и тем самым определить доход от каждого канала распространения. Подобным образом сведения о распространителе встраивались в исполняемый файл руткита TDL3, а специальные серверы подсчитывали число установок, связанных с распространителем и начисленных на его счёт.

Партнёры киберпреступных групп получали уникальные имя пользователя и пароль, с помощью которых учитывалось число установок для каждого ресурса. Кроме того, у каждого партнёра был личный менеджер, к которому можно было обратиться при возникновении технических проблем.

Чтобы снизить риск обнаружения антивирусными программами, партнёры часто переупаковывали распространяемое вредоносное ПО и применяли сложные защитные методы для выявления отладчиков и виртуальных машин, затрудняя анализ исследователям вредоносного ПО.[2]

Партнёрам также запрещалось пользоваться такими ресурсами, как VirusTotal, чтобы проверить, способны ли защитные программы обнаружить текущие версии вредоносного ПО; за подобные действия им даже грозили штрафами. Причина заключалась в том, что образцы, отправленные в VirusTotal, с большой вероятностью привлекли бы внимание лабораторий, занимающихся исследованиями в области безопасности, а следовательно, подверглись бы анализу, что фактически сократило бы срок полезной жизни вредоносного ПО. Если распространителей беспокоила скрытность продукта, их направляли к сервисам, которыми управляли разработчики вредоносного ПО: эти сервисы напоминали VirusTotal, но могли гарантировать, что переданные образцы не попадут в руки производителей защитных программ.

Процедура заражения

После того как инфектор TDL3 загружается в систему пользователя по одному из каналов распространения, он начинает процесс заражения. Чтобы пережить перезагрузку системы, TDL3 заражает один из драйверов, запускаемых при загрузке и необходимых для загрузки ОС, внедряя вредоносный код в двоичный файл этого драйвера. Такие драйверы загружаются вместе с образом ядра на раннем этапе инициализации ОС. В результате при запуске заражённой машины изменённый драйвер загружается, а вредоносный код получает контроль над процессом запуска.

Итак, выполняясь в адресном пространстве режима ядра, процедура заражения просматривает список загружаемых при запуске драйверов, которые обеспечивают работу основных компонентов операционной системы, и случайным образом выбирает один из них в качестве цели. Каждая запись списка описывается недокументированной структурой `KLDR_DATA_TABLE_ENTRY`, приведённой в листинге 1-1; ссылка на неё находится в поле `DriverSection` структуры `DRIVER_OBJECT`. Каждому загруженному драйверу режима ядра соответствует структура `DRIVER_OBJECT`.

```
typedef struct _KLDR_DATA_TABLE_ENTRY {
   LIST_ENTRY InLoadOrderLinks;
   LIST_ENTRY InMemoryOrderLinks;
   LIST_ENTRY InInitializationOrderLinks;
   PVOID ExceptionTable;
   ULONG ExceptionTableSize;
   PVOID GpValue;
   PNON_PAGED_DEBUG_INFO NonPagedDebugInfo;
   PVOID ImageBase;
   PVOID EntryPoint;
   ULONG SizeOfImage;
   UNICODE_STRING FullImageName;
   UNICODE_STRING BaseImageName;

   ULONG Flags;
   USHORT LoadCount;
   USHORT Reserved1;
   PVOID SectionPointer;
   ULONG CheckSum;
   PVOID LoadedImports;
   PVOID PatchInformation;
} KLDR_DATA_TABLE_ENTRY, *PKLDR_DATA_TABLE_ENTRY;
```

Листинг 1-1. Структура `KLDR_DATA_TABLE_ENTRY`*, на которую ссылается поле* `DriverSection`

Выбрав целевой драйвер, инфектор TDL3 изменяет его образ в памяти: вредоносный загрузчик записывается поверх первых нескольких сотен байтов раздела ресурсов `.rsrc`. Этот загрузчик устроен довольно просто: во время запуска он всего лишь загружает с жёсткого диска остальной необходимый код вредоносного ПО.

Перезаписанные исходные байты раздела `.rsrc`, которые по-прежнему нужны для правильной работы драйвера, сохраняются в файле `rsrc.dat` внутри скрытой файловой системы, поддерживаемой вредоносным ПО. (Обратите внимание: размер заражаемого файла драйвера при заражении не изменяется.) После этой модификации TDL3 меняет поле точки входа в заголовке Portable Executable (PE) драйвера, чтобы оно указывало на вредоносный загрузчик. Таким образом, адрес точки входа заражённого TDL3 драйвера указывает на раздел ресурсов, что в нормальных условиях недопустимо. На рисунке 1-1 драйвер, запускаемый при загрузке, показан до и после заражения; схема демонстрирует изменение образа драйвера, причём метка `Header` обозначает заголовок PE вместе с таблицей разделов.

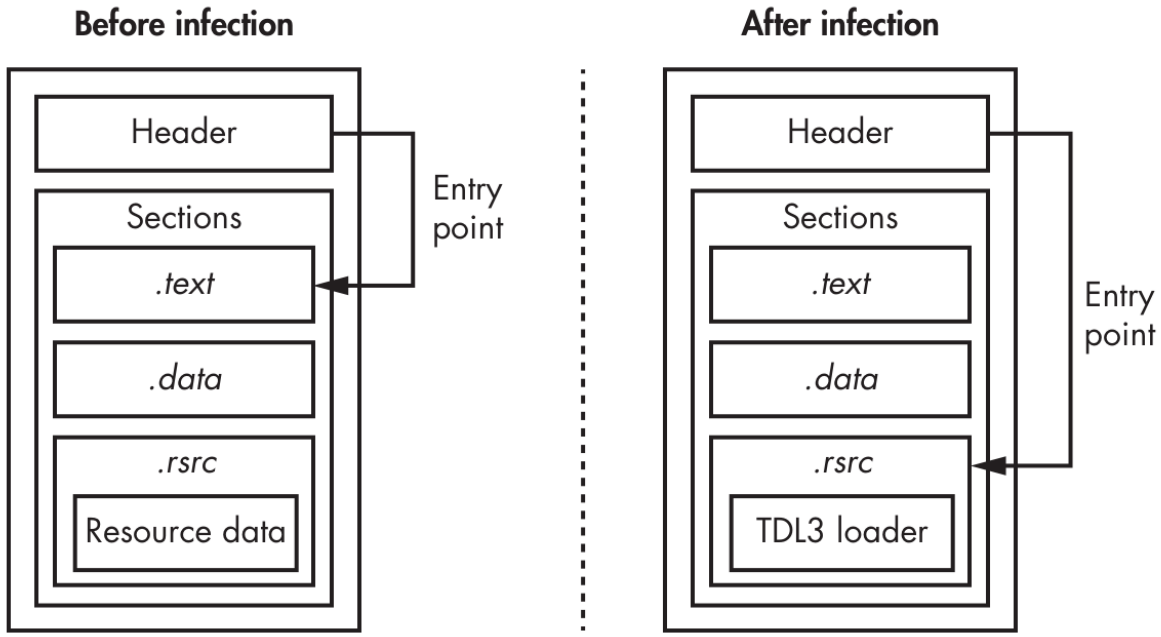


Рисунок 1-1. Изменения драйвера режима ядра, запускаемого при загрузке, при заражении системы. Обозначения: Before infection - до заражения; After infection - после заражения; Header - заголовок; Sections - разделы; Entry point - точка входа; Resource data - данные ресурсов; TDL3 loader - загрузчик TDL3

Такая схема заражения исполняемых файлов в формате PE, основном двоичном формате исполняемых файлов и динамических библиотек (DLL) Windows, типична для вирусных инфекторов, но не столь распространена среди руткитов. И заголовок PE, и таблица разделов незаменимы для любого PE-файла. Заголовок PE содержит важнейшие сведения о расположении кода и данных, системные метаданные, размер стека и прочую информацию, а таблица разделов содержит сведения о разделах исполняемого файла и их расположении.

Чтобы завершить процесс заражения, вредоносное ПО записывает в элемент каталога метаданных .NET заголовка PE те же значения, которые содержит элемент каталога данных безопасности. Вероятно, этот шаг предназначен для противодействия статическому анализу заражённых образов, поскольку при разборе заголовка PE распространёнными средствами анализа вредоносного ПО он может вызвать ошибку. И действительно, попытка загрузить такие образы приводила к аварийному завершению IDA Pro версии 5.6; впоследствии эту ошибку исправили. Согласно спецификации PE/COFF компании Microsoft, каталог метаданных .NET содержит данные, которые среда Common Language Runtime (CLR) использует для загрузки и выполнения приложений .NET. Однако для запускаемых при загрузке драйверов режима ядра этот элемент каталога не имеет значения, поскольку все они являются нативными двоичными файлами и не содержат управляемого системой кода. Поэтому загрузчик ОС не проверяет данный элемент каталога, и заражённый драйвер успешно загружается даже при недопустимом содержимом элемента.

Обратите внимание, что этот метод заражения TDL3 имеет ограничение: он работает только на 32-разрядных платформах из-за политики Microsoft по подписыванию кода режима ядра, которая предусматривает обязательную проверку целостности кода в 64-разрядных системах. Поскольку при заражении системы содержимое драйвера изменяется, его цифровая подпись утрачивает действительность, и ОС уже не может загрузить драйвер в 64-разрядной системе. В ответ разработчики вредоносного ПО создали TDL4. И саму политику, и её обход мы подробно обсудим в главе 6.

Управление потоком данных

Чтобы выполнить свою задачу и остаться незаметными, руткиты ядра должны изменять поток управления или поток данных (либо оба потока) системных вызовов ядра во всех случаях, когда исходный поток управления или данных ОС способен выдать присутствие неактивных компонентов вредоносного ПО (например, файлов), его выполняющихся задач или артефактов (например, структур данных ядра). Для этого руткиты обычно внедряют свой код в одном из мест на пути выполнения реализации системного вызова; расположение таких перехватчиков кода относится к числу наиболее поучительных особенностей руткитов.

Собственный компоновщик

По сути, перехват является компоновкой. Современные руткиты приносят с собой собственные компоновщики, чтобы связать свой код с системой; этот шаблон проектирования мы называем Bring Your Own Linker, то есть «принеси собственный компоновщик». Чтобы незаметно встроить такие «компоновщики», TDL3 следует нескольким общим принципам проектирования вредоносного ПО.

Во-первых, несмотря на внедрение дополнительного кода, целевая система должна сохранять устойчивость: злоумышленник ничего не выиграет и многое потеряет, если атакуемое программное обеспечение аварийно завершит работу. С точки зрения программной инженерии перехват представляет собой форму композиции программного обеспечения и требует осторожного подхода. Злоумышленник должен обеспечить переход системы к новому коду только в предсказуемом состоянии, которое этот код сможет правильно обработать, иначе возможны сбой или необычное поведение, способные привлечь внимание пользователя.

Может показаться, что выбор мест для перехватов ограничен лишь воображением автора руткита, однако в действительности автор вынужден придерживаться устойчивых границ и интерфейсов программного обеспечения, которые он очень хорошо понимает. Поэтому неудивительно, что мишенями для перехвата обычно становятся те же структуры, которые используются собственными средствами динамической компоновки системы, независимо от того, документированы они открыто или нет. Таблицы обратных вызовов, методов и других указателей на функции, связывающих уровни абстракции или программные модули, являются наиболее безопасными местами для перехватов; хорошо работает и перехват прологов функций.

Во-вторых, место перехвата не должно быть слишком очевидным. Ранние руткиты перехватывали таблицу системных вызовов верхнего уровня ядра, но этот метод быстро устарел именно из-за своей заметности. Когда в 2005 году его использовал руткит Sony,[3] такое размещение уже считалось несовременным и потому вызвало немало удивления. По мере усложнения руткитов их перехватчики перемещались всё ниже по стеку: сначала из основных таблиц диспетчеризации системных вызовов в подсистемы ОС, предоставлявшие единые уровни API для различных реализаций, например в виртуальную файловую систему (Virtual File System, VFS), а затем в

методы и обратные вызовы конкретных драйверов. TDL3 представляет собой особенно наглядный пример такой миграции.

Как работают перехваты режима ядра в TDL3

Чтобы не попадаться на глаза, TDL3 применял довольно сложный и прежде не встречавшийся в реальных атаках метод перехвата: он перехватывал запросы ввода-вывода на чтение и запись, отправляемые жёсткому диску, на уровне драйвера порта хранения или минипорт-драйвера, то есть драйвера аппаратного носителя, находящегося в самом низу стека драйверов хранения. Драйверы портов являются системными модулями, предоставляющими программный интерфейс минипорт-драйверам, которые поставляют производители соответствующих устройств хранения. На рисунке 1-2 показана архитектура стека драйверов устройств хранения в Microsoft Windows.

Обработка структуры пакета запроса ввода-вывода (I/O request packet, IRP), адресованного объекту на устройстве хранения, начинается на уровне драйвера файловой системы. Соответствующий драйвер файловой системы определяет конкретное устройство, на котором хранится объект, например раздел диска и дисковый экстент, то есть непрерывную область хранения, изначально зарезервированную для файловой системы, после чего выдаёт ещё один IRP объекту устройства драйвера класса. Тот, в свою очередь, преобразует запрос ввода-вывода в запрос к соответствующему объекту устройства минипорта.

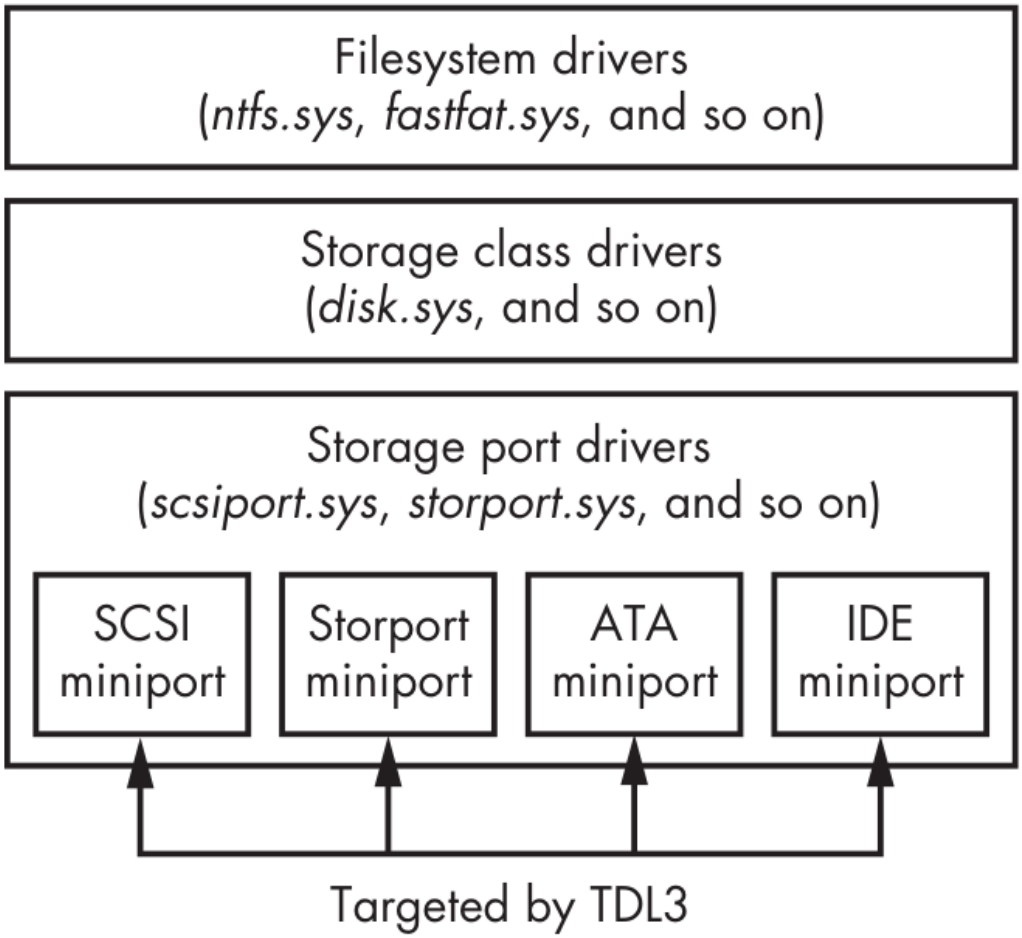


Рисунок 1-2. Архитектура стека драйверов устройств хранения в Microsoft Windows. Обозначения: Filesystem drivers - драйверы файловой системы; Storage class drivers - драйверы класса хранения; Storage port drivers - драйверы портов хранения; SCSI/Storport/ATA/IDE miniport - соответствующие минипорты; Targeted by TDL3 - цели TDL3

Согласно документации Windows Driver Kit (WDK), драйверы портов хранения предоставляют интерфейс между независимым от оборудования драйвером класса и минипорт-драйвером, зависящим от HBA (host-based architecture, архитектуры на основе узла). Получив доступ к этому интерфейсу, TDL3 устанавливает перехватчики режима ядра на самом нижнем независимом от оборудования уровне стека драйверов устройств хранения и тем самым обходит все средства наблюдения и защиты, работающие на уровне файловой системы или драйвера класса хранения. Обнаружить такие перехватчики способны лишь средства, которым известен нормальный состав этих таблиц для конкретного набора устройств или заведомо исправная конфигурация конкретной машины.

Чтобы реализовать этот метод перехвата, TDL3 сначала получает указатель на объект минипорт-драйвера соответствующего объекта устройства. В частности, код перехвата пытается открыть дескриптор `\??\PhysicalDriveXX`, где `XX` соответствует номеру жёсткого диска, однако эта строка в действительности является символической ссылкой на объект устройства `\Device\HardDisk0\DR0`, созданный драйвером класса хранения. Спускаясь по стеку устройств от `\Device\HardDisk0\DR0`, в самом низу мы находим объект устройства минипорта хранения. После его обнаружения получить указатель на объект его драйвера несложно: достаточно обратиться к полю `DriverObject` документированной структуры `DEVICE_OBJECT`. Теперь у вредоносного ПО есть все сведения, необходимые для перехвата стека драйверов хранения.

Затем TDL3 создаёт новый вредоносный объект драйвера и заменяет в объекте минипорт-драйвера поле `DriverObject` указателем на только что созданный объект, как показано на рисунке 1-3. Благодаря этому вредоносное ПО может перехватывать запросы чтения и записи, направленные нижележащему жёсткому диску, поскольку адреса всех обработчиков указаны в связанной с ним структуре объекта драйвера, а именно в массиве `MajorFunction` структуры `DRIVER_OBJECT`.

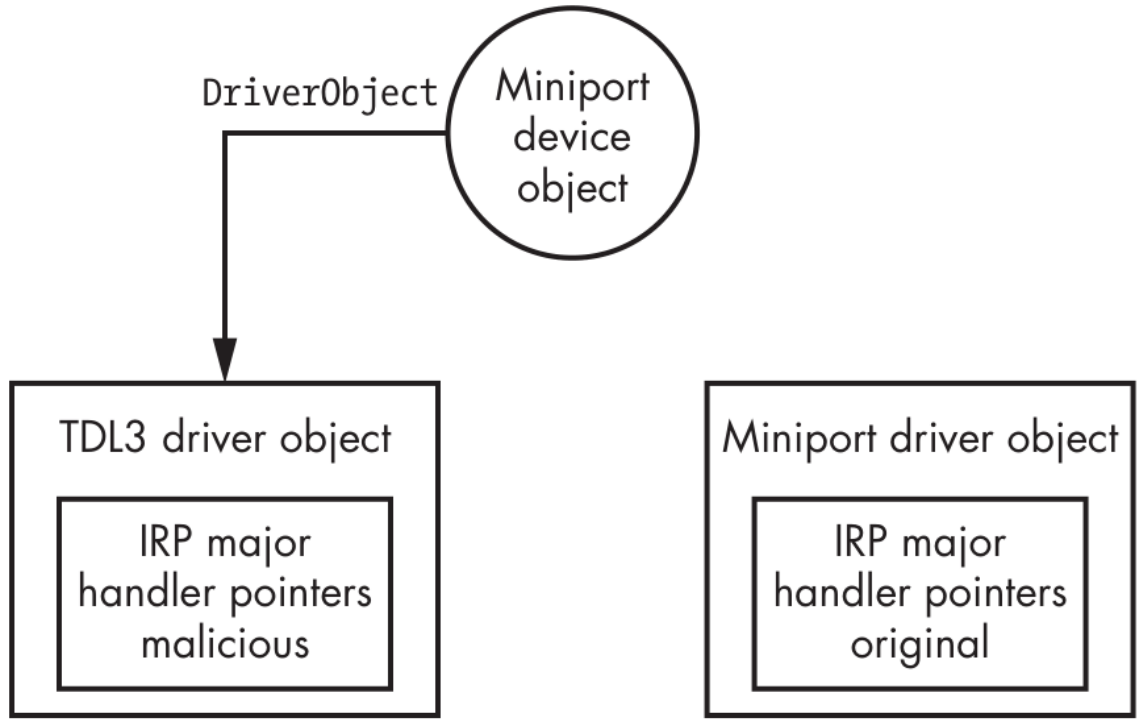


Рисунок 1-3. Перехват объекта минипорт-драйвера хранения. Обозначения: Miniport device object - объект устройства минипорта; TDL3 driver object - объект драйвера TDL3; Miniport driver object - объект минипорт-драйвера; IRP major handler pointers malicious - указатели на вредоносные основные обработчики IRP; IRP major handler pointers original - указатели на исходные основные обработчики IRP

Показанные на рисунке 1-3 вредоносные основные обработчики перехватывают `IRP_MJ_INTERNAL_CONTROL` и `IRP_MJ_DEVICE_CONTROL` для следующих кодов управления вводом-выводом (Input/Output Control, IOCTL), чтобы отслеживать и изменять запросы чтения и записи на жёстком диске, где хранятся заражённый драйвер и образ скрытой файловой системы, реализованной вредоносным ПО:

• `IOCTL_ATA_PASS_THROUGH_DIRECT`
• `IOCTL_ATA_PASS_THROUGH`

TDL3 не позволяет средствам Windows читать секторы жёсткого диска, содержащие защищаемые данные, а файловой системе Windows случайно перезаписывать их, сохраняя тем самым и скрытность, и целостность руткита. При операции чтения TDL3 после завершения операции ввода-вывода заполняет возвращаемый буфер нулями, а при запросе на запись данных полностью пропускает операцию чтения. Метод перехвата TDL3 позволяет обходить некоторые средства обнаружения исправлений ядра: изменения TDL3 не затрагивают ни одну из часто защищаемых и контролируемых областей, включая системные модули, таблицу описателей системных служб (System Service Descriptor Table, SSDT), глобальную таблицу дескрипторов (Global Descriptor Table, GDT) и таблицу дескрипторов прерываний (Interrupt Descriptor Table, IDT). Его преемник TDL4 применяет тот же подход для обхода имеющейся в 64-разрядных операционных системах Windows защиты ядра от исправлений PatchGuard, поскольку наследует от TDL3 значительную часть функций режима ядра, включая перехваты минипорт-драйвера хранения.

Скрытая файловая система

TDL3 стал первой вредоносной системой, которая хранила свои файлы конфигурации и полезную нагрузку в скрытой зашифрованной области на целевой системе, а не полагалась на службу файловой системы, предоставляемую операционной системой. В наши дни подход TDL3 переняли и приспособили другие сложные угрозы, в том числе буткит Rovnix, ZeroAccess, Avatar и Gapz.

Этот метод скрытого хранения значительно затрудняет криминалистический анализ, поскольку вредоносные данные находятся в зашифрованном контейнере, размещённом где-то на жёстком диске, но за пределами области, зарезервированной собственной файловой системой ОС. При этом вредоносное ПО может обращаться к содержимому скрытой файловой системы через обычные функции Win32 API, такие как `CreateFile`, `ReadFile`, `WriteFile` и `CloseHandle`. Это упрощает разработку вредоносной полезной нагрузки, поскольку её создатели могут использовать стандартные интерфейсы Windows для чтения полезной нагрузки из области хранения и записи в неё, не разрабатывая и не сопровождая специальные интерфейсы. Это архитектурное решение имеет важное значение: вместе с применением стандартных интерфейсов для перехвата оно повышает общую надёжность руткита; с точки зрения программной инженерии перед нами правильный и удачный пример повторного использования кода! Формулой успеха собственного генерального директора Microsoft было: «Разработчики, разработчики, разработчики, разработчики!», иначе говоря, имеющиеся навыки разработчиков следует считать ценным капиталом. TDL3 сходным образом решил воспользоваться существующими навыками программирования для Windows у разработчиков, перешедших на тёмную сторону, возможно, одновременно для облегчения перехода и повышения надёжности вредоносного кода.

TDL3 размещает образ своей скрытой файловой системы в секторах жёсткого диска, не занятых собственной файловой системой ОС. Образ растёт от конца диска к его началу, а значит, если он станет достаточно велик, то в конце концов может перезаписать данные файловой системы пользователя. Образ разделён на блоки по 1024 байта. Первый блок, расположенный в конце жёсткого диска, содержит таблицу файлов, записи которой описывают файлы внутри файловой системы и включают следующие сведения:

• имя файла длиной не более 16 символов, включая завершающий нулевой символ;
• размер файла;
• фактическое смещение файла, которое вычисляется вычитанием начального смещения файла, умноженного на 1024, из смещения начала файловой системы;
• время создания файловой системы.

Содержимое файловой системы шифруется отдельно по блокам с помощью собственного, по большей части импровизированного алгоритма. В разных версиях руткита применялись разные алгоритмы. Например, в некоторых модификациях использовался шифр RC4, ключом для которого служил адрес логического блока (logical block address, LBA) первого сектора, соответствующего каждому блоку. В другой модификации данные шифровались операцией XOR с фиксированным ключом `0x54`, который увеличивался при каждой операции XOR. Шифрование получалось настолько слабым, что характерный шаблон, соответствующий зашифрованному блоку из нулей, было легко заметить.

Из режима пользователя полезная нагрузка обращается к скрытому хранилищу, открывая дескриптор объекта устройства с именем `\Device\XXXXXXXX\YYYYYYYY`, где `XXXXXXXX` и `YYYYYYYY` являются случайно сгенерированными шестнадцатеричными числами. Обратите внимание, что путь выполнения кода, обеспечивающий доступ к этому хранилищу, опирается на множество стандартных компонентов Windows, которые, хочется надеяться, уже отлажены Microsoft и потому надёжны. Имя объекта устройства генерируется при каждой загрузке системы, а затем передаётся модулям полезной нагрузки в качестве параметра. Руткит отвечает за сопровождение этой файловой системы и обработку направленных ей запросов ввода-вывода.

Например, когда модуль полезной нагрузки выполняет операцию ввода-вывода с файлом, хранящимся в скрытой области, ОС передаёт запрос руткиту и запускает его функции точек входа для обработки этого запроса.

В этом шаблоне проектирования TDL3 иллюстрирует общую тенденцию, которой следуют руткиты. Вместо того чтобы предоставлять совершенно новый код для всех своих операций и вынуждать сторонних разработчиков вредоносного ПО изучать его особенности, руткит пользуется уже имеющимися и знакомыми функциями Windows, пока способы такого паразитирования и лежащие в их основе интерфейсы Windows не становятся общеизвестными. Конкретные методы заражения развиваются вслед за изменениями массово внедряемых защитных мер, но этот подход сохраняется, поскольку он следует общим принципам надёжности кода, разделяемым разработчиками как вредоносного, так и легитимного программного обеспечения.

Заключение: TDL3 встречает своего заклятого врага

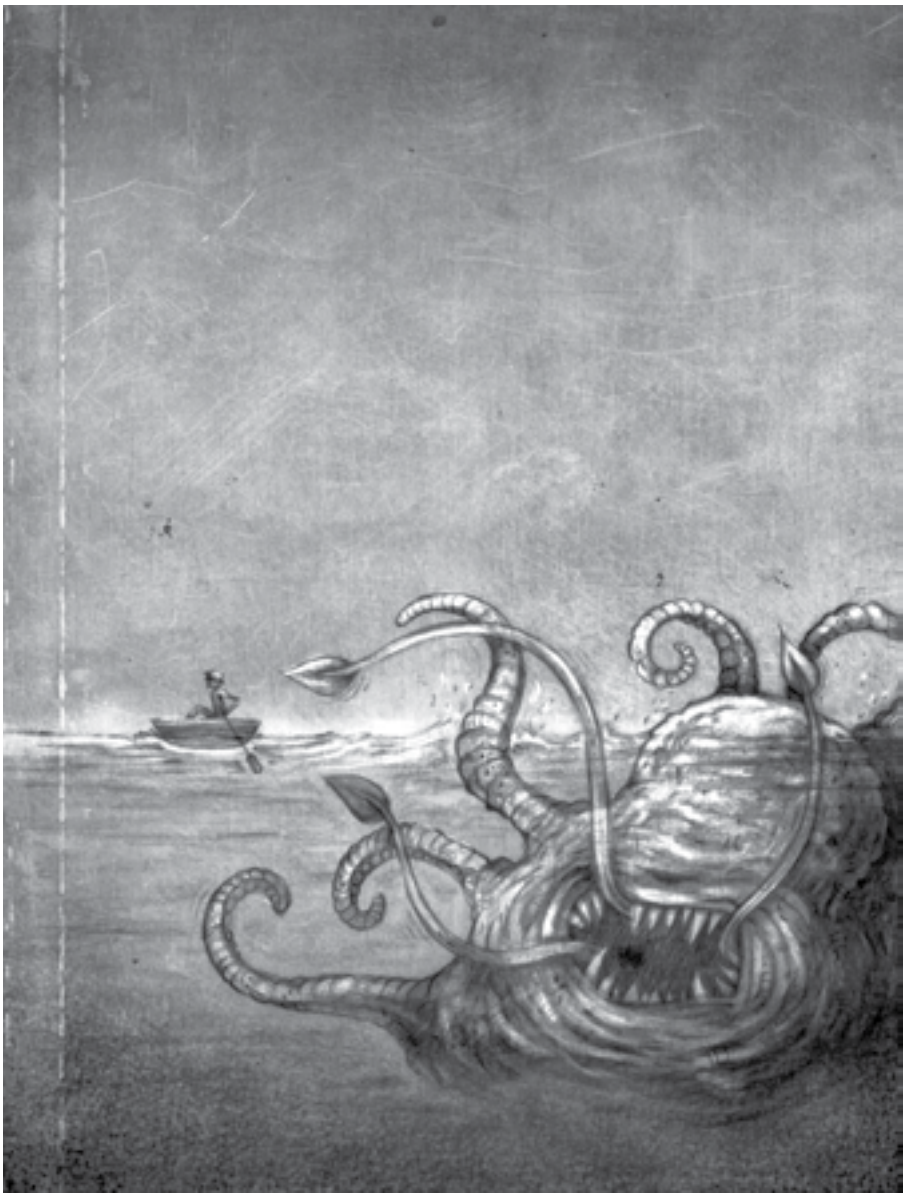
Как мы увидели, TDL3 представляет собой сложный руткит, который впервые применил несколько методов скрытой и устойчивой работы в заражённой системе. Его перехваты режима ядра и системы скрытого хранения не остались незамеченными другими разработчиками вредоносного ПО и впоследствии появились в иных сложных угрозах. Единственное ограничение процедуры заражения заключается в том, что её целями могут быть только 32-разрядные системы.

Когда TDL3 только начал распространяться, он справлялся с задачей, поставленной разработчиками, но по мере роста числа 64-разрядных систем усиливался спрос на возможность заражать системы x64. Для этого создателям вредоносного ПО предстояло выяснить, как преодолеть 64-разрядную политику подписывания кода режима ядра, чтобы загружать вредоносный код в адресное пространство режима ядра. Как мы обсудим в главе 7, авторы TDL3 решили обходить проверку подписей с помощью технологии буткитов.

Сноски

[1] [назад] static1.esetstatic.com
[2] [назад] Родриго Рубира Бранко, Габриэль Негрейра Барбоза и Педро Дримел Нето, «Научный, но не академический обзор технологий противодействия отладке, дизассемблированию и виртуальным машинам» (доклад на конференции Black Hat USA 2012, 21-26 июля, Лас-Вегас, Невада), media.blackhat.com.
[3] [назад] blogs.technet.microsoft.com

Глава 2. Руткит Festi: самый передовой бот для спама и DDoS-атак



Эта глава посвящена одной из самых передовых когда-либо обнаруженных бот-сетей для рассылки спама и распределённых атак типа «отказ в обслуживании» (distributed denial of service, DDoS), бот-сети Win32/Festi, которую далее мы будем называть просто Festi. Festi обладает мощными средствами рассылки спама и проведения DDoS-атак, а также интересными функциями руткита, позволяющими ему оставаться незамеченным благодаря перехвату операций файловой системы и системного реестра. Кроме того, Festi скрывает своё присутствие, активно противодействуя динамическому анализу с помощью методов уклонения от отладчиков и песочниц.

На высоком уровне Festi обладает хорошо спроектированной модульной архитектурой, целиком реализованной в драйвере режима ядра. Разумеется, программирование в режиме ядра сопряжено с опасностями: единственная ошибка в коде способна вызвать сбой системы и сделать её непригодной для использования, в результате чего пользователь может полностью переустановить систему и тем самым уничтожить вредоносное ПО.

По этой причине вредоносное ПО для рассылки спама редко в значительной степени полагается на программирование в режиме ядра. Тот факт, что Festi смог причинить столь большой ущерб, свидетельствует о серьёзных технических навыках его разработчика или разработчиков и глубоком понимании ими системы Windows. Они действительно приняли несколько интересных архитектурных решений, которые мы рассмотрим в этой главе.

История бот-сети Festi

Бот-сеть Festi впервые обнаружили осенью 2009 года, а к маю 2012 года она стала одной из самых мощных и активных бот-сетей для рассылки спама и проведения DDoS-атак. Поначалу взять бот-сеть в аренду мог любой желающий, однако после начала 2010 года доступ ограничили крупными партнёрами по рассылке спама, среди которых был Павел Врублевский, один из участников преступной деятельности с применением Festi, подробно описанной Брайаном Кребсом в книге *Spam Nation* (Sourcebooks, 2014).

Согласно показанной на рисунке 2-1 статистике M86 Security Labs, ныне Trustwave, за 2011 год, в отчётный период Festi входила в тройку самых активных бот-сетей мира, рассылавших спам.

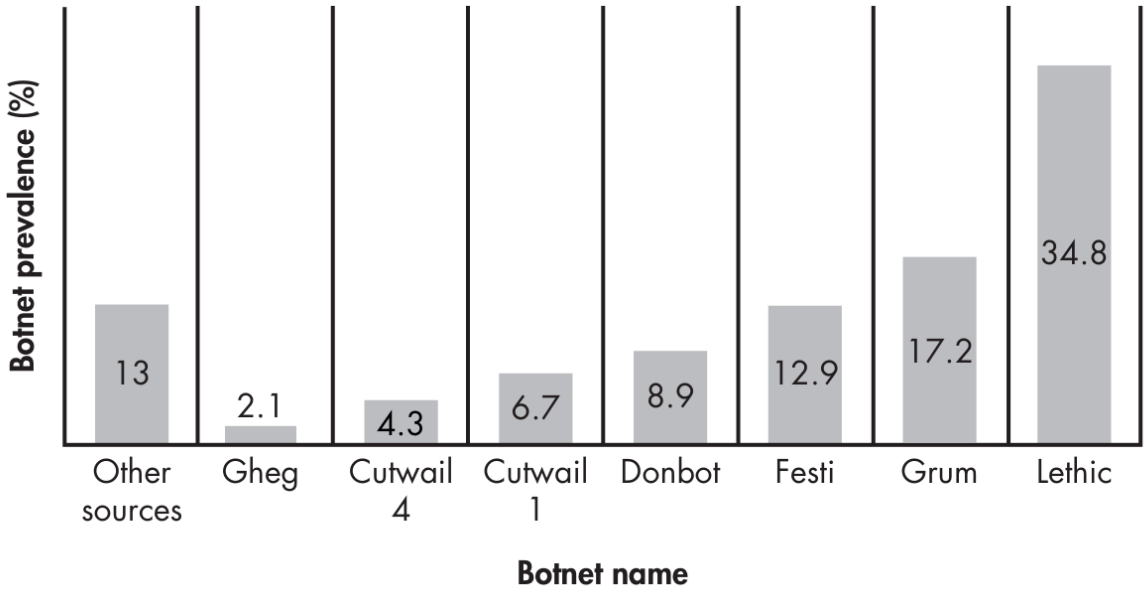


Рисунок 2-1. Самые распространённые бот-сети для рассылки спама по данным M86 Security Labs. Обозначения: Botnet prevalence (%) - распространённость бот-сети, %; Botnet name - название бот-сети; Other sources - другие источники. Значения: другие источники - 13; Gheg - 2,1; Cutwail 4 - 4,3; Cutwail 1 - 6,7; Donbot - 8,9; Festi - 12,9; Grum - 17,2; Lethic - 34,8

Рост известности Festi был связан с конкретной атакой на компанию Assist, обрабатывавшую платежи.[1] Assist была одной из компаний, претендовавших на контракт с крупнейшей российской авиакомпанией «Аэрофлот», однако за несколько недель до того, как «Аэрофлот» должен был принять решение, киберпреступники провели с помощью Festi масштабную DDoS-атаку на Assist. Атака на продолжительное время вывела систему обработки платежей из строя и в конечном счёте вынудила «Аэрофлот» отдать контракт другой компании. Это событие наглядно показывает, как руткиты могут использоваться в реальных преступлениях.

Разбор драйвера руткита

Руткит Festi распространяется главным образом по схеме PPI, сходной со схемой распространения руткита TDL3, рассмотренного в главе 1. Довольно простой дроппер устанавливает в систему драйвер режима ядра, в котором реализована основная логика вредоносного ПО. Компонент режима ядра регистрируется под случайно сгенерированным именем как драйвер режима ядра типа `system start`, то есть вредоносный драйвер загружается и выполняется при запуске системы во время инициализации.

> **Инфектор-дроппер.** Дроппер является особой разновидностью инфектора. Дроппер несёт полезную нагрузку внутри собственного файла в систему жертвы. Полезная нагрузка часто бывает сжата и зашифрована либо обфусцирована. После запуска дроппер извлекает полезную нагрузку из своего образа и устанавливает её в системе жертвы, то есть «сбрасывает» в систему, откуда и происходит название этого типа инфекторов. В отличие от дропперов, загрузчики, представляющие собой другой тип инфекторов, не содержат полезную нагрузку внутри себя, а загружают её с удалённого сервера.

Бот-сеть Festi нацелена только на платформу Microsoft Windows x86 и не имеет драйвера режима ядра для 64-разрядных платформ. Во время её распространения это не создавало проблем, поскольку в употреблении всё ещё находилось множество 32-разрядных операционных систем, однако теперь, когда число 64-разрядных систем превысило число 32-разрядных, руткит в значительной степени устарел.

У драйвера режима ядра две основные обязанности: запрашивать конфигурационные сведения у сервера управления (command and control, C&C), а также загружать и выполнять вредоносные модули в форме подключаемых модулей, как показано на рисунке 2-2. Каждый подключаемый модуль решает определённую задачу, например проводит DDoS-атаку на указанный сетевой ресурс или рассылает спам по списку адресов электронной почты, полученному от C&C-сервера.

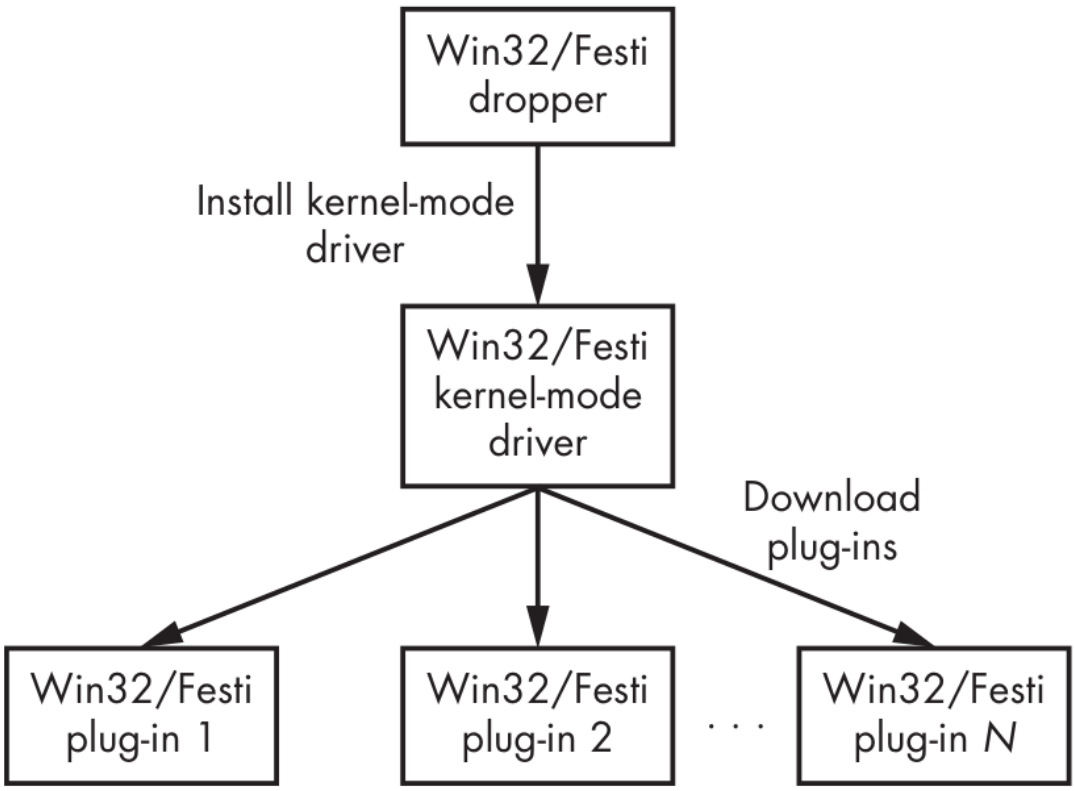


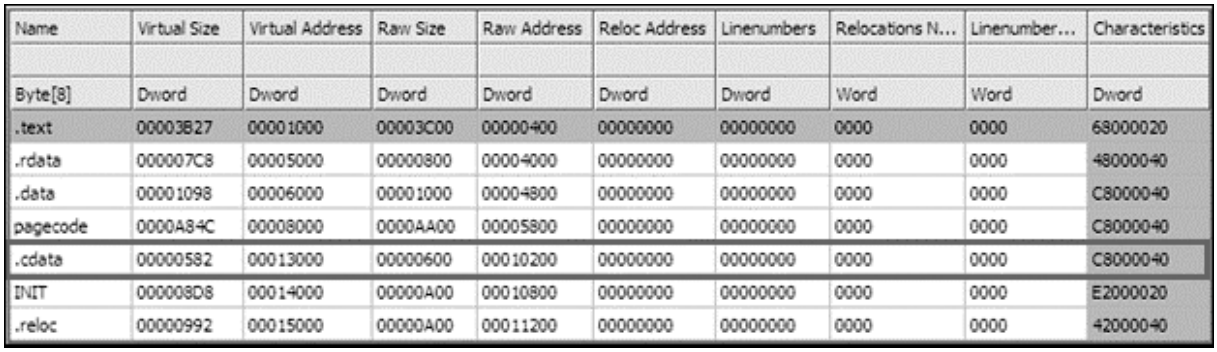
Рисунок 2-2. Работа руткита Festi. Обозначения: Win32/Festi dropper - дроппер Win32/Festi; Install kernel-mode driver - установка драйвера режима ядра; Win32/Festi kernel-mode driver - драйвер Win32/Festi режима ядра; Download plug-ins - загрузка подключаемых модулей; Win32/Festi plug-in 1, 2, N - подключаемые модули Win32/Festi 1, 2 и N

Интересно, что подключаемые модули хранятся не на жёстком диске системы, а в энергозависимой памяти, поэтому при выключении или перезагрузке заражённого компьютера они исчезают из системной памяти. Это значительно затрудняет криминалистический анализ вредоносного ПО, поскольку единственным файлом на жёстком диске остаётся основной драйвер режима ядра, который не содержит ни полезной нагрузки, ни каких-либо сведений о целях атак.

Конфигурационные сведения Festi для связи с C&C-сервером

Чтобы обеспечить связь с C&C-сервером, Festi распространяется с тремя заранее заданными элементами конфигурации: доменными именами C&C-серверов, ключом для шифрования данных, передаваемых между ботом и C&C-сервером, и сведениями о версии бота.

Эти конфигурационные сведения жёстко записаны в двоичном файле драйвера. На рисунке 2-3 показана таблица разделов драйвера режима ядра с доступным для записи разделом `.cdata`, в котором хранятся конфигурационные данные и строки, используемые для вредоносной деятельности.



Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumber...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00003B27	00001000	00003C00	00000400	00000000	00000000	0000	0000	68000020
.rdata	000007C8	00005000	00000800	00004000	00000000	00000000	0000	0000	48000040
.data	00001098	00006000	00001000	00004800	00000000	00000000	0000	0000	C8000040
pagecode	0000A84C	00008000	0000AA00	00005800	00000000	00000000	0000	0000	C8000040
.cdata	00000582	00013000	00000600	00010200	00000000	00000000	0000	0000	C8000040
INIT	000008D8	00014000	00000A00	00010800	00000000	00000000	0000	0000	E2000020
.reloc	00000992	00015000	00000A00	00011200	00000000	00000000	0000	0000	42000040

Рисунок 2-3. Таблица разделов драйвера режима ядра Festi. В строке `.cdata` *показан доступный для записи раздел конфигурационных данных; названия столбцов интерфейса IDA сохранены в исходном виде*

Вредоносное ПО обфусцирует содержимое простым алгоритмом, выполняющим операцию XOR над данными с 4-байтовым ключом. Раздел `.cdata` расшифровывается в самом начале инициализации драйвера.

Строки внутри раздела `.cdata`, перечисленные в таблице 2-1, могут привлечь внимание защитного ПО, поэтому их обфускация помогает боту избегать обнаружения.

Таблица 2-1. Зашифрованные строки в разделе конфигурационных данных Festi

Строка	**Назначение**
`\Device\Tcp` `\Device\Udp`	Имена объектов устройств, которые вредоносное ПО использует для отправки и получения данных по сети
`\REGISTRY\MACHINE\SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy\StandardProfile\GloballyOpenPorts\List`	Путь к разделу реестра с параметрами брандмауэра Windows, который вредоносное ПО использует для отключения локального брандмауэра
`ZwDeleteFile`, `ZwQueryInformationFile`, `ZwLoadDriver`, `KdDebuggerEnabled`, `ZwDeleteValueKey`, `ZwLoadDriver`	Имена системных служб, используемых вредоносным ПО

Объектно-ориентированный каркас Festi

В отличие от многих драйверов режима ядра, которые обычно пишут на обычном языке C в рамках процедурной парадигмы программирования, драйвер Festi обладает объектно-ориентированной архитектурой. Основными компонентами, или классами, реализованной вредоносным ПО архитектуры являются:

Компонент	**Назначение**
Диспетчер памяти	Выделяет и освобождает буферы памяти
Сетевые сокеты	Отправляют и получают данные по сети
Анализатор протокола C&C	Разбирает сообщения C&C и выполняет полученные команды
Диспетчер подключаемых модулей	Управляет загруженными подключаемыми модулями

Связи между этими компонентами показаны на рисунке 2-4.

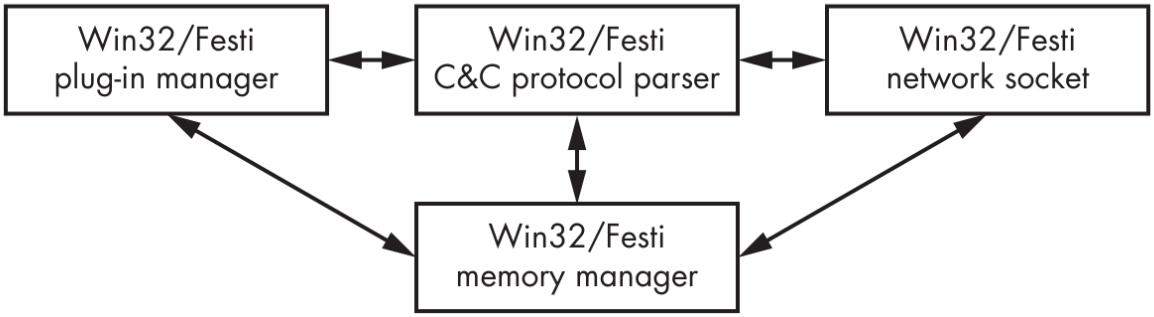


Рисунок 2-4. Архитектура драйвера Festi режима ядра. Обозначения: plug-in manager - диспетчер подключаемых модулей; C&C protocol parser - анализатор протокола C&C; network socket - сетевой сокет; memory manager - диспетчер памяти

Как видите, центральным компонентом, которым пользуются все остальные компоненты, является диспетчер памяти.

Такой объектно-ориентированный подход позволяет без труда переносить вредоносное ПО на другие платформы, например Linux. Для этого злоумышленнику потребовалось бы изменить только изолированный интерфейсом компонента системно-зависимый код, например код, вызывающий системные службы управления памятью и сетевого взаимодействия. Так, загружаемые подключаемые модули почти полностью полагаются на интерфейсы, предоставляемые основным модулем; они редко обращаются к предоставленным системой процедурам для выполнения системно-зависимых операций.

Управление подключаемыми модулями

Вредоносное ПО загружает и выполняет подключаемые модули, полученные с C&C-сервера. Для эффективного управления загруженными модулями Festi поддерживает массив указателей на специально определённые структуры `PLUGIN_INTERFACE`. Каждая структура соответствует определённому подключаемому модулю в памяти и предоставляет боту конкретные точки входа, то есть процедуры, отвечающие за обработку данных, полученных от C&C-сервера, как показано на рисунке 2-5. Так Festi ведёт учёт всех вредоносных подключаемых модулей, загруженных в память.

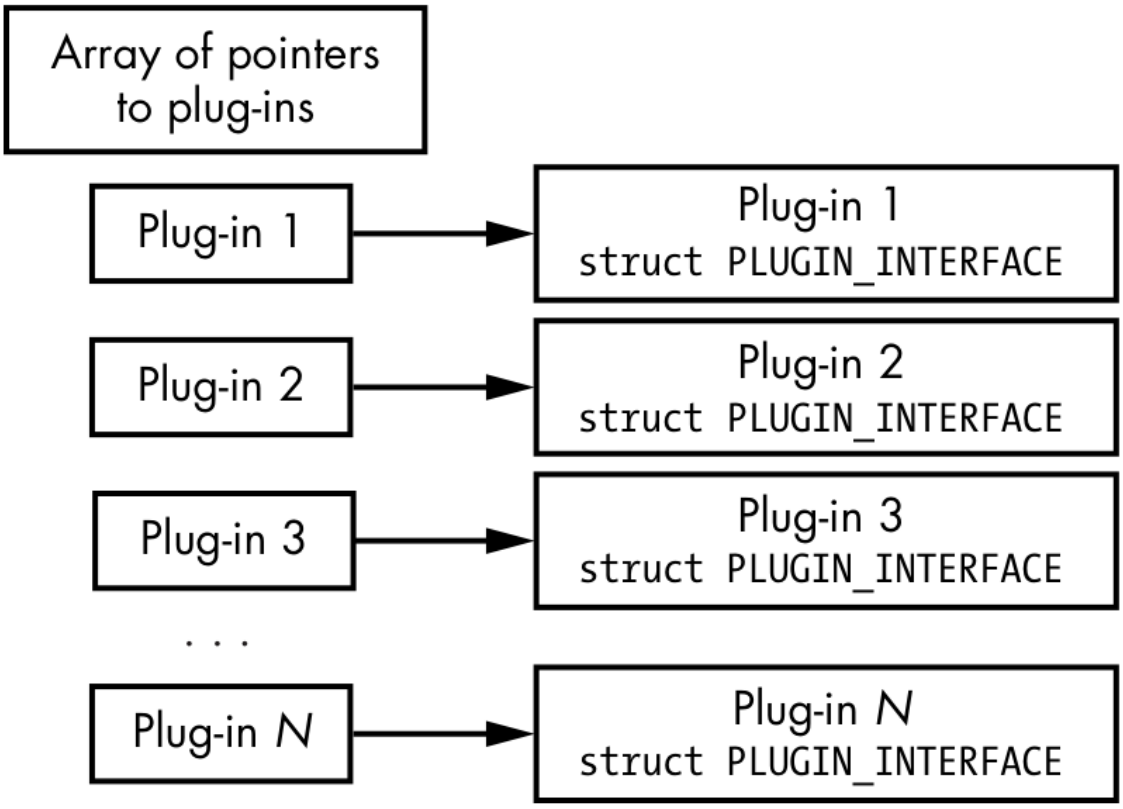


Рисунок 2-5. Структура массива указателей на структуры `PLUGIN_INTERFACE`. *Обозначения: Array of pointers to plug-ins - массив указателей на подключаемые модули; Plug-in 1, 2, 3, N - подключаемые модули 1, 2, 3 и N*

В листинге 2-1 показана структура `PLUGIN_INTERFACE`.

```
struct PLUGIN_INTERFACE
{
  // Инициализировать подключаемый модуль
  PVOID Initialize;
  // Освободить подключаемый модуль, выполнить операции очистки
  PVOID Release;
  // Получить сведения о версии подключаемого модуля
  PVOID GetVersionInfo_1;
  // Получить сведения о версии подключаемого модуля
  PVOID GetVersionInfo_2;
  // Записать относящиеся к модулю сведения в поток TCP
  PVOID WriteIntoTcpStream;
  // Прочитать относящиеся к модулю сведения из потока TCP и разобрать данные
  PVOID ReadFromTcpStream;
  // Зарезервированные поля
  PVOID Reserved_1;
  PVOID Reserved_2;
};
```

Листинг 2-1. Определение структуры `PLUGIN_INTERFACE`

Первые две процедуры, `Initialize` и `Release`, предназначены соответственно для инициализации и завершения работы подключаемого модуля. Следующие две процедуры, `GetVersionInfo_1` и `GetVersionInfo_2`, используются для получения сведений о версии соответствующего подключаемого модуля.

Процедуры `WriteIntoTcpStream` и `ReadFromTcpStream` используются для обмена данными между подключаемым модулем и C&C-сервером. Когда Festi передаёт данные C&C-серверу, он проходит

по массиву указателей на интерфейсы подключаемых модулей и выполняет процедуру `WriteIntoTcpStream` каждого зарегистрированного модуля, передавая ей в качестве параметра указатель на объект потока TCP. Объект потока TCP реализует функции интерфейса сетевого взаимодействия.

Получив данные от C&C-сервера, бот выполняет процедуру `ReadFromTcpStream` подключаемых модулей, чтобы зарегистрированные модули могли получить из сетевого потока параметры и относящиеся к ним конфигурационные сведения. В результате каждый загруженный подключаемый модуль способен обмениваться данными с C&C-сервером независимо от всех остальных модулей. Следовательно, модули можно разрабатывать независимо друг от друга, что повышает эффективность их разработки и устойчивость архитектуры.

Встроенные подключаемые модули

При установке основной вредоносный драйвер режима ядра реализует два встроенных подключаемых модуля: диспетчер конфигурационных сведений и диспетчер подключаемых модулей бота.

Диспетчер конфигурационных сведений

Подключаемый модуль диспетчера конфигурационных сведений отвечает за запрос конфигурационных сведений и загрузку подключаемых модулей с C&C-сервера. Этот простой модуль периодически соединяется с C&C-сервером для загрузки данных. Задержку между двумя последовательными запросами задаёт сам C&C-сервер, вероятно, чтобы избежать статических шаблонов, по которым защитное ПО могло бы обнаружить заражение. Протокол сетевого взаимодействия между ботом и C&C-сервером описан в разделе «Протокол сетевого взаимодействия Festi» на странице 26.

Диспетчер подключаемых модулей бота

Диспетчер подключаемых модулей бота отвечает за сопровождение массива загруженных модулей. Он получает удалённые команды от C&C-сервера и загружает в систему либо выгружает из неё определённые подключаемые модули, поступающие в сжатом виде. Каждый модуль имеет точку входа по умолчанию `DriverEntry` и экспортирует две процедуры, `CreateModule` и `DeleteModule`, как показано на рисунке 2-6.

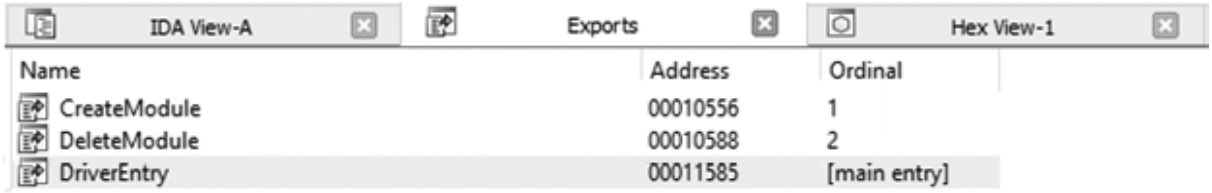


Рисунок 2-6. Таблица экспортируемых адресов подключаемого модуля Festi. Имена экспортируемых процедур `CreateModule`*,* `DeleteModule` *и* `DriverEntry` *сохранены без изменений*

Процедура `CreateModule` выполняется при инициализации подключаемого модуля и возвращает указатель на структуру `PLUGIN_INTERFACE`, описанную ранее в листинге 2-1. В качестве параметра она принимает указатель на несколько интерфейсов, предоставляемых основным модулем, в том числе на диспетчер памяти и сетевой интерфейс.

Процедура DeleteModule выполняется при выгрузке подключаемого модуля и освобождает все ранее выделенные ресурсы. На рисунке 2-7 показан алгоритм, по которому диспетчер подключаемых модулей загружает модуль.

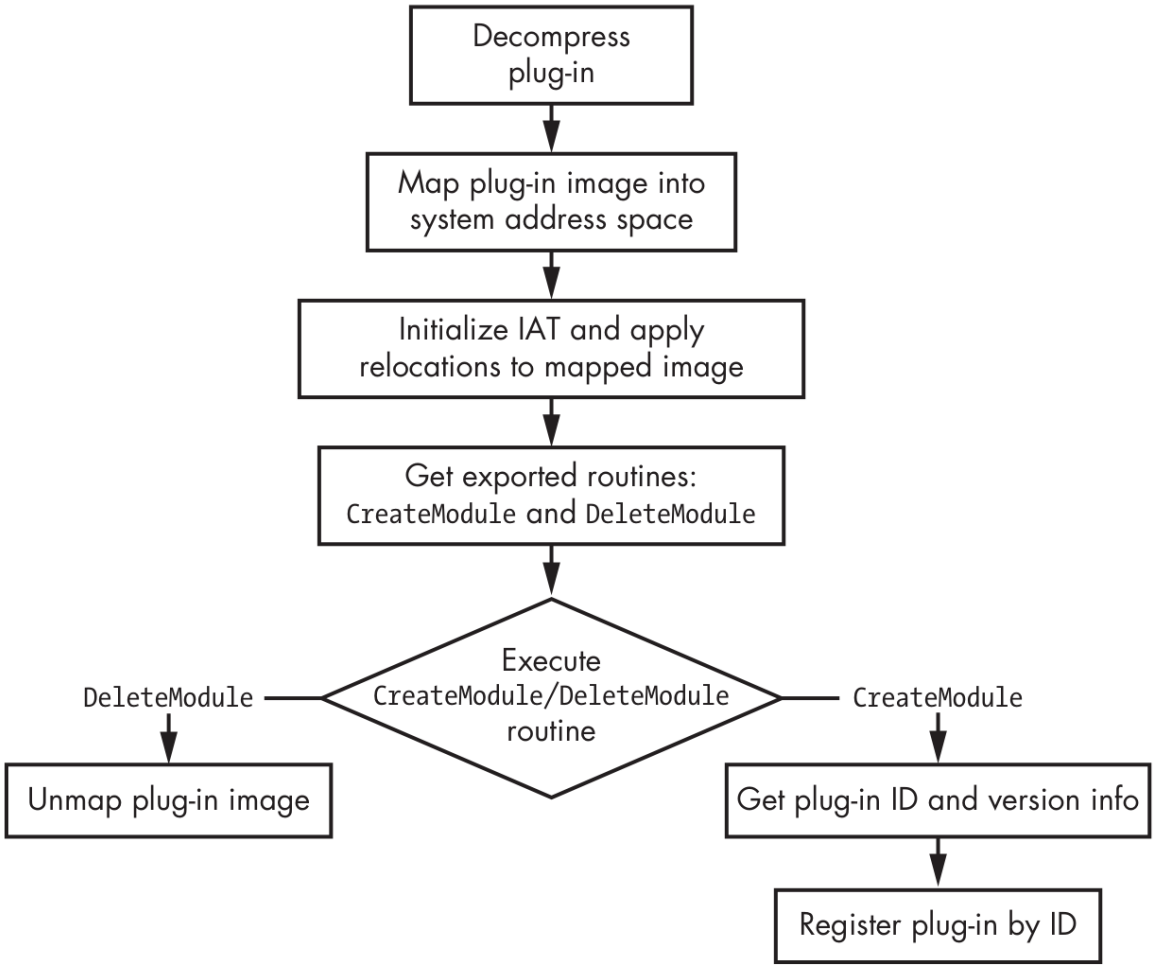


Рисунок 2-7. Алгоритм диспетчера подключаемых модулей. Обозначения: Decompress plug-in - распаковать подключаемый модуль; Map plug-in image into system address space - отобразить образ модуля в системное адресное пространство; Initialize IAT and apply relocations to mapped image - инициализировать IAT и применить перемещения к отображённому образу; Get exported routines - получить экспортируемые процедуры; Execute CreateModule/DeleteModule routine - выполнить процедуру `CreateModule` *или* `DeleteModule`*; Unmap plug-in image - отменить отображение образа модуля; Get plug-in ID and version info - получить идентификатор и сведения о версии модуля; Register plug-in by ID - зарегистрировать модуль по идентификатору*

Вредоносное ПО сначала распаковывает подключаемый модуль в буфер памяти, а затем отображает его в адресное пространство режима ядра как PE-образ. Диспетчер подключаемых модулей инициализирует таблицу адресов импорта (Import Address Table, IAT) и применяет перемещения к отображённому образу. В этом алгоритме Festi также имитирует обычные загрузчик времени выполнения операционной системы и динамический компоновщик модулей ОС.

В зависимости от того, загружается или выгружается подключаемый модуль, его диспетчер выполняет процедуру `CreateModule` либо `DeleteModule`. При загрузке диспетчер получает идентификатор и сведения о версии подключаемого модуля, а затем регистрирует его в структурах `PLUGIN_INTERFACE`.

При выгрузке подключаемого модуля вредоносное ПО освобождает всю память, ранее выделенную для его образа.

Методы противодействия виртуальным машинам

Чтобы уклоняться от песочниц и сред автоматизированного анализа вредоносного ПО, Festi умеет определять, выполняется ли он внутри виртуальной машины VMware. Он пытается получить версию имеющегося программного обеспечения VMware, выполняя код из листинга 2-2.

```
mov eax, 'VMXh'
mov ebx, 0
mov ecx, 0Ah
mov edx, 'VX'
in eax, dx
```

Листинг 2-2. Получение версии программного обеспечения VMware

Festi проверяет регистр `ebx`: если код выполняется в виртуальной среде VMware, этот регистр содержит значение `VMX`, а в противном случае - `0`.

Интересно, что при обнаружении виртуальной среды Festi не завершает работу немедленно, а продолжает её так, словно выполняется на физическом компьютере. Запрашивая подключаемые модули у C&C-сервера, вредоносное ПО отправляет определённые сведения, по которым можно понять, выполняется ли оно в виртуальной среде; если это так, C&C-сервер может не вернуть ни одного модуля.

Вероятно, это метод уклонения от динамического анализа: Festi не прекращает обмен данными с C&C-сервером, пытаясь заставить систему автоматического анализа поверить, что он её не заметил, тогда как C&C-сервер в действительности знает о наблюдении и потому не предоставляет никаких команд или подключаемых модулей. Обнаружив выполнение под отладчиком или в среде песочницы, вредоносное ПО часто завершает работу, чтобы не раскрывать конфигурационные сведения и модули полезной нагрузки.

Однако исследователи вредоносного ПО хорошо знают о таком поведении. Если вредоносная программа немедленно завершает работу, не совершив никаких вредоносных действий, это может привлечь внимание аналитика. Скорее всего, он проведёт более глубокий анализ, чтобы выяснить, почему она не сработала, и в конце концов обнаружит данные и код, которые программа пытается скрыть. Не завершая работу при обнаружении песочницы, Festi старается избежать таких последствий, но всё же предписывает своему C&C-серверу не передавать песочнице вредоносные модули и конфигурационные данные.

Кроме того, Festi проверяет наличие в системе программ наблюдения за сетевым трафиком, которое может означать, что вредоносное ПО запущено в среде анализа и наблюдения. Festi ищет драйвер режима ядра `npf.sys`, сетевой фильтр пакетов. Этот драйвер принадлежит библиотеке перехвата пакетов Windows WinPcap, которую программы сетевого наблюдения, например Wireshark, часто используют для доступа к канальному уровню сети. Наличие драйвера `npf.sys` указывает на то, что в системе установлены средства наблюдения за сетью, а значит, среда небезопасна для вредоносного ПО.

WinPcap. Библиотека перехвата пакетов Windows (WinPcap) позволяет приложениям перехватывать и передавать сетевые пакеты в обход стека протоколов. Она предоставляет функции фильтрации и наблюдения за сетевыми пакетами на уровне ядра. Эта библиотека широко используется как механизм фильтрации во многих открытых и коммерческих сетевых средствах: анализаторах протоколов, программах наблюдения за сетью, системах обнаружения сетевых вторжений и снифферах, включая такие общеизвестные средства, как Wireshark, Nmap, Snort и ntop.

Методы противодействия отладке

Festi также проверяет наличие в системе отладчика ядра, исследуя переменную `KdDebuggerEnabled`, экспортируемую из образа ядра операционной системы. Если к операционной системе подключён системный отладчик, эта переменная содержит значение `TRUE`, в противном случае - `FALSE`.

Festi активно противодействует системному отладчику, периодически обнуляя отладочные регистры с `dr0` по `dr3`. Эти регистры хранят адреса точек останова, поэтому удаление аппаратных точек останова затрудняет отладку. Код очистки отладочных регистров приведён в листинге 2-3.

```
char _thiscall ProtoHandler_1(STRUCT_4_4 *this, PKEVENT a1)
{
__writedr(0, 0); // mov dr0, 0
__writedr(1u, 0); // mov dr1, 0
__writedr(2u, 0); // mov dr2, 0
__writedr(3ut 0); // mov dr3, 0
  return _ProtoHandler(&this->struct43, a1);
}
```

Листинг 2-3. Очистка отладочных регистров в коде Festi

Выделенные инструкции `writedr` выполняют запись в отладочные регистры. Как видите, Festi записывает в эти регистры нули перед выполнением процедуры `_ProtoHandler`, которая отвечает за обработку протокола обмена данными между вредоносным ПО и C&C-серверами.

Метод сокрытия вредоносного драйвера на диске

Для защиты и сокрытия образа вредоносного драйвера режима ядра, хранящегося на жёстком диске, Festi перехватывает драйвер файловой системы. Благодаря этому он может перехватывать и изменять все направленные драйверу файловой системы запросы, исключая из них признаки своего присутствия.

Упрощённая версия процедуры установки перехвата приведена в листинге 2-4.

```
NTSTATUS __stdcall SetHookOnSystemRoot(PDRIVER_OBJECT DriverObject,
                                       int **HookParams)
{
  RtlInitUnicodeString(&DestinationString, L"\\SystemRoot");
  ObjectAttributes.Length = 24;
  ObjectAttributes.RootDirectory = 0;
  ObjectAttributes.Attributes = 64;
  ObjectAttributes.ObjectName = &DestinationString;
  ObjectAttributes.SecurityDescriptor = 0;
  ObjectAttributes.SecurityQualityOfService = 0;

  ❶ NTSTATUS Status = IoCreateFile(&hSystemRoot, 0x80000000, &ObjectAttributes,
                                   &IoStatusBlock, 0, 0, 3u, 1u, 1u, 0, 0, 0, 0,
                                   0x100u);
```

```
  if (Status < 0 )
    return Status;
  ❷ Status = ObReferenceObjectByHandle(hSystemRoot, 1u, 0, 0,
                                       &SystemRootFileObject, 0);
  if (Status < 0 )
    return Status;
  ❸ PDEVICE_OBJECT TargetDevice = IoGetRelatedDeviceObject(SystemRootFileObject);
  if ( !_ TargetDevice )
      return STATUS_UNSUCCESSFUL;

  ObfReferenceObject(TargetDevice);
  Status = IoCreateDevice(DriverObject, 0xCu, 0, TargetDev->DeviceType,
                          TargetDevice->Characteristics, 0, &SourceDevice);
  if (Status < 0 )
    return Status;
  ❹ PDEVICE_OBJECT DeviceAttachedTo = IoAttachDeviceToDeviceStack(SourceDevice,
                                                                  TargetDevice);
  if ( ! DeviceAttachedTo )
  {
    IoDeleteDevice(SourceDevice);
    return STATUS_UNSUCCESSFUL;
  }

  return STATUS_SUCCESS;
}
```

Листинг 2-4. Перехват стека драйверов устройств файловой системы

Сначала вредоносное ПО пытается получить дескриптор специального системного файла `SystemRoot`, соответствующего каталогу установки Windows ❶. Затем, выполнив системную процедуру `ObReferenceObjectByHandle` ❷, Festi получает указатель на `FILE_OBJECT`, соответствующий дескриптору `SystemRoot`. `FILE_OBJECT` представляет собой специальную структуру данных, которую операционная система использует для управления доступом к объектам устройств, и потому содержит указатель на связанный объект устройства. В нашем случае, поскольку мы открыли дескриптор `SystemRoot`, `DEVICE_OBJECT` связан с драйвером файловой системы операционной системы. Вредоносное ПО получает указатель на `DEVICE_OBJECT`, выполняя системную процедуру `IoGetRelatedDeviceObject` ❸, затем создаёт новый объект устройства и присоединяет его к полученному указателю на объект устройства вызовом `IoAttachDeviceToDeviceStack` ❹, как показано на схеме стека устройств файловой системы на рисунке 2-8. Вредоносный объект устройства Festi находится на вершине стека, поэтому запросы ввода-вывода, предназначенные файловой системе, перенаправляются вредоносному ПО. Это позволяет Festi скрывать себя, изменяя данные запросов к драйверу файловой системы и возвращаемые им данные.

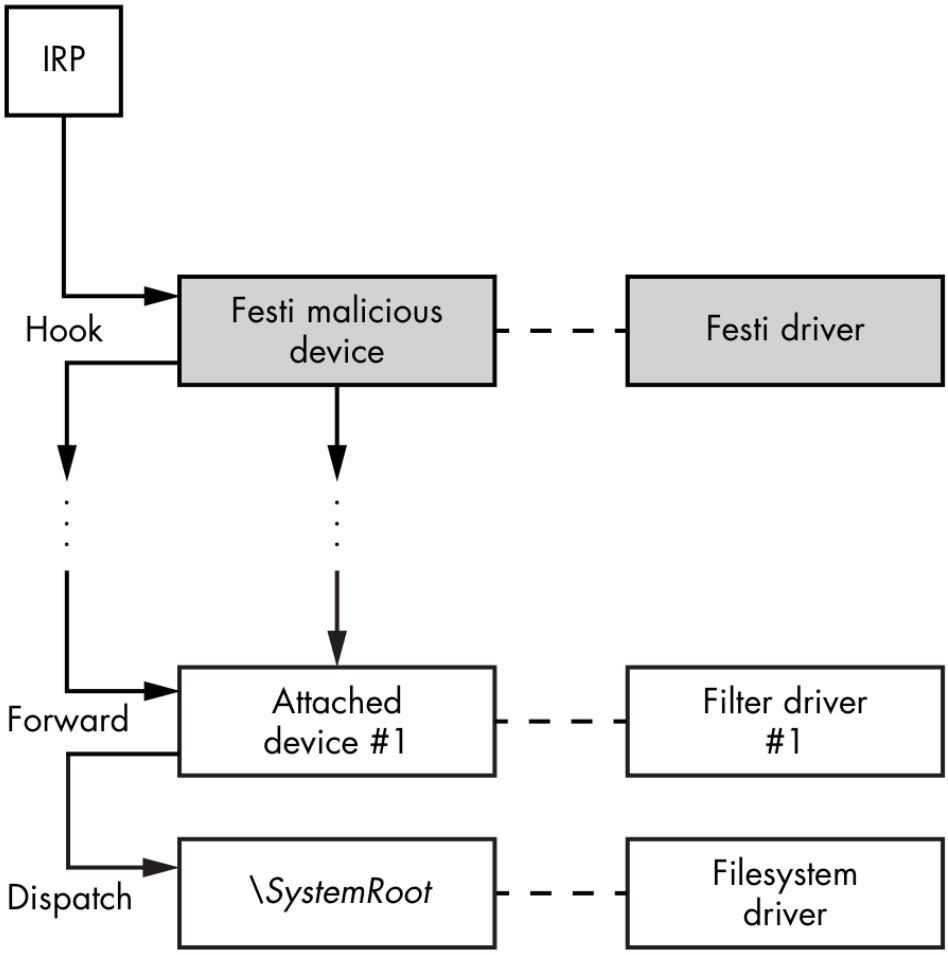


Рисунок 2-8. Структура стека устройств файловой системы, перехваченного Festi. Обозначения: Hook - перехват; Festi malicious device - вредоносное устройство Festi; Festi driver - драйвер Festi; Forward - передача дальше; Attached device #1 - присоединённое устройство 1; Filter driver #1 - драйвер-фильтр 1; Dispatch - диспетчеризация; Filesystem driver - драйвер файловой системы

В самой нижней части рисунка 2-8 показаны объект драйвера файловой системы и соответствующий объект устройства, обрабатывающий запросы ОС к файловой системе. Здесь могут быть присоединены и дополнительные фильтры файловой системы. Ближе к верхней части рисунка виден драйвер Festi, присоединённый к стеку устройств файловой системы.

Эта архитектура использует структуру многоуровневых драйверов ввода-вывода Windows и точно следует ей, воспроизводя шаблон проектирования собственной ОС. Вероятно, теперь вы уже видите тенденцию: руткит стремится аккуратно и надёжно слиться с ОС, воспроизводя в собственных модулях удачные шаблоны проектирования операционной системы. В действительности анализ таких особенностей руткитов, как обработка запросов ввода-вывода в Festi, позволяет многое узнать о внутреннем устройстве ОС.

В Windows запрос ввода-вывода к файловой системе представлен пакетом IRP, который проходит по стеку сверху вниз. Каждый драйвер в стеке может наблюдать и изменять запрос или возвращаемые данные. Следовательно, как показано на рисунке 2-8, Festi способен изменять IRP-запросы, адресованные драйверу файловой системы, и все соответствующие возвращаемые данные.

Festi наблюдает за пакетами IRP с кодом запроса `IRP_MJ_DIRECTORY_CONTROL`, который используется для запроса содержимого каталога, и отслеживает запросы, относящиеся к месту хранения драйвера вредоносного ПО режима ядра. Обнаружив такой запрос, Festi изменяет данные, возвращённые драйвером файловой системы, и исключает из них любую запись, соответствующую файлу вредоносного драйвера.

Метод защиты раздела реестра Festi

Сходным способом Festi скрывает раздел реестра, соответствующий зарегистрированному драйверу режима ядра. Этот раздел, расположенный по пути `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services`, содержит тип драйвера Festi и путь к образу драйвера в файловой системе. Из-за этого защитное ПО может обнаружить Festi, поэтому руткиту необходимо скрывать раздел.

Для этого Festi сначала перехватывает системную службу `ZwEnumerateKey`, которая запрашивает сведения об указанном разделе реестра и возвращает все его подразделы. Перехват выполняется посредством изменения таблицы описателей системных служб (System Service Descriptor Table, SSDT), специальной структуры данных в ядре операционной системы, содержащей адреса обработчиков системных служб. Festi заменяет адрес исходного обработчика `ZwEnumerateKey` адресом перехватчика.

> **Защита ядра Windows от исправлений.** Следует отметить, что такой подход к перехвату, то есть изменение SSDT, работает только в 32-разрядных операционных системах Microsoft Windows. Как упоминалось в главе 1, в 64-разрядных выпусках Windows реализована технология защиты ядра от исправлений (Kernel Patch Protection), также известная как PatchGuard, которая не позволяет программам изменять определённые системные структуры, включая SSDT. Если PatchGuard обнаруживает изменение любой из контролируемых структур данных, он вызывает аварийное завершение системы.

Перехватчик `ZwEnumerateKey` отслеживает запросы, адресованные разделу реестра `HKLM\System\CurrentControlSet\Service`, который содержит подразделы, относящиеся к установленным в системе драйверам режима ядра, включая драйвер Festi. В перехватчике Festi изменяет список подразделов, исключая из него запись, соответствующую своему драйверу. Любая

программа, которая получает список установленных драйверов режима ядра через `ZwEnumerateKey`, не заметит присутствия вредоносного драйвера Festi.

Если защитное ПО обнаружит раздел реестра и удалит его во время завершения работы системы, Festi способен восстановить этот раздел. Для этого Festi сначала выполняет системную процедуру `IoRegisterShutdownNotification`, чтобы получать уведомления о завершении работы при выключении системы. Обработчик такого уведомления проверяет, присутствуют ли в системе вредоносный драйвер и соответствующий раздел реестра. Если их нет, то есть если они были удалены, обработчик восстанавливает их, гарантируя сохранение Festi после перезагрузки.

Протокол сетевого взаимодействия Festi

Для связи с C&C-серверами и выполнения вредоносных действий Festi применяет собственный протокол сетевого взаимодействия, который ему необходимо защищать от прослушивания. Исследуя бот-сеть Festi,[2] мы получили список C&C-серверов, с которыми она связывалась, и выяснили, что некоторые из них занимались рассылкой спама, а другие проводили DDoS-атаки, но серверы обоих типов использовали единый протокол взаимодействия. Протокол Festi состоит из двух фаз: фазы инициализации, во время которой бот получает IP-адреса C&C-серверов, и рабочей фазы, когда он запрашивает у C&C-сервера описание задания.

Фаза инициализации

Во время фазы инициализации вредоносное ПО получает IP-адреса C&C-сервера, доменные имена которого хранятся в двоичном файле бота. Интересная особенность этого процесса состоит в том, что вредоносное ПО вручную определяет IP-адрес C&C-сервера по его доменным именам. В частности, оно формирует пакет DNS-запроса для разрешения доменного имени C&C-сервера и отправляет его одному из двух узлов, `8.8.8.8` или `8.8.4.4`, на порт `53`; оба узла являются DNS-серверами Google. В ответ Festi получает IP-адрес, который может использовать при дальнейшем обмене данными.

Ручное разрешение доменных имён повышает устойчивость бот-сети к попыткам её ликвидации. Если бы для разрешения доменных имён Festi приходилось полагаться на DNS-серверы местного поставщика интернет-услуг, поставщик мог бы заблокировать доступ к C&C-серверам, изменив хранящиеся на этих DNS-серверах сведения, например по предписанию правоохранительного органа заблокировать соответствующие доменные имена. Однако, самостоятельно формируя DNS-запросы и отправляя их серверам Google, вредоносное ПО обходит DNS-инфраструктуру поставщика и затрудняет ликвидацию бот-сети.

Рабочая фаза

Во время рабочей фазы Festi запрашивает у C&C-сервера сведения о задачах, которые ему предстоит выполнить. Обмен данными с C&C-серверами ведётся по протоколу TCP. Показанная на рисунке 2-9 структура пакета сетевого запроса, отправляемого C&C-серверу, состоит из заголовка сообщения и массива данных, относящихся к подключаемым модулям.

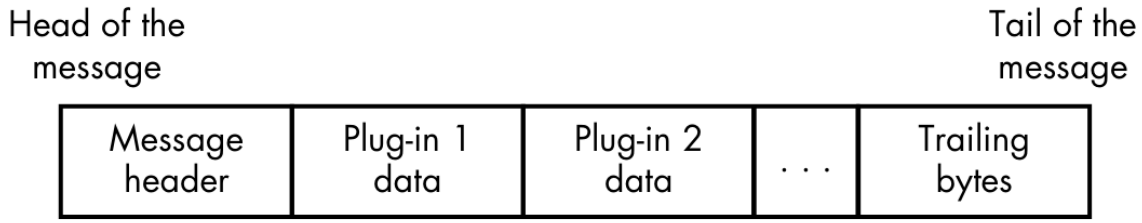


Рисунок 2-9. Структура сетевого пакета, отправляемого C&C-серверу. Обозначения: Head of the message - начало сообщения; Message header - заголовок сообщения; Plug-in 1/2 data - данные подключаемого модуля 1/2; Trailing bytes - завершающие байты; Tail of the message - конец сообщения

Заголовок сообщения создаётся подключаемым модулем диспетчера конфигурации и содержит следующие сведения:

• сведения о версии Festi;
• признак наличия системного отладчика;
• признак наличия программного обеспечения виртуализации VMware;
• признак наличия программного обеспечения для наблюдения за сетевым трафиком WinPcap;
• сведения о версии операционной системы.

Данные, относящиеся к подключаемому модулю, состоят из массива записей «тег - значение - терминатор»:

Поле	**Описание**
Тег	16-разрядное целое число, определяющее тип следующего за тегом значения
Значение	Конкретные данные в виде байта, слова, двойного слова, строки с завершающим нулевым символом или двоичного массива
Терминатор	Завершающее слово `0xABDC`, обозначающее конец записи

Схема «тег - значение - терминатор» предоставляет вредоносному ПО удобный способ сериализовать относящиеся к подключаемому модулю данные в сетевой запрос к C&C-серверу.

Перед отправкой по сети данные обфусцируются простым алгоритмом шифрования. Реализация этого алгоритма на Python приведена в листинге 2-5.

```
key = (0x17, 0xFB, 0x71,0x5C) ❶
def decr_data(data):
  for ix in xrange(len(data)):
    data[ix] ^= key[ix % 4]
```

Листинг 2-5. Реализация алгоритма сетевого шифрования на Python

Вредоносное ПО использует циклический алгоритм XOR с фиксированным 4-байтовым ключом ❶.

Обход защитного и криминалистического программного обеспечения

Чтобы связываться по сети с C&C-серверами, рассылать спам и проводить DDoS-атаки, одновременно уклоняясь от защитного ПО, Festi использует стек TCP/IP, реализованный в режиме ядра Windows.

Для отправки и получения пакетов вредоносное ПО открывает дескриптор устройства `\Device\Tcp` или `\Device\Udp` в зависимости от используемого протокола. При этом для получения дескриптора применяется довольно интересный метод, не привлекающий внимания защитного ПО. Разрабатывая этот метод, авторы Festi вновь продемонстрировали превосходное знание внутреннего устройства Windows.

Чтобы контролировать доступ к сети на узле, некоторые защитные программы наблюдают за обращениями к этим устройствам, перехватывая запросы `IRP_MJ_CREATE`, которые отправляются транспортному драйверу, когда кто-либо пытается открыть дескриптор для обмена данными с объектом устройства. Это позволяет защитному ПО определить, какой процесс пытается обмениваться данными по сети. В общем случае защитное ПО чаще всего отслеживает доступ к объектам устройств следующими способами:

• перехватывает обработчик системной службы `ZwCreateFile`, чтобы регистрировать все попытки открыть устройства;
• присоединяется к `\Device\Tcp` или `\Device\Udp`, чтобы перехватывать все отправляемые IRP-запросы.

Festi искусно обходит оба метода, устанавливая соединение с удалённым узлом по сети.

Во-первых, вместо системной реализации службы `ZwCreateFile` Festi реализует собственную системную службу, функции которой почти совпадают с исходной. Собственная реализация процедуры `ZwCreateFile` показана на рисунке 2-10.

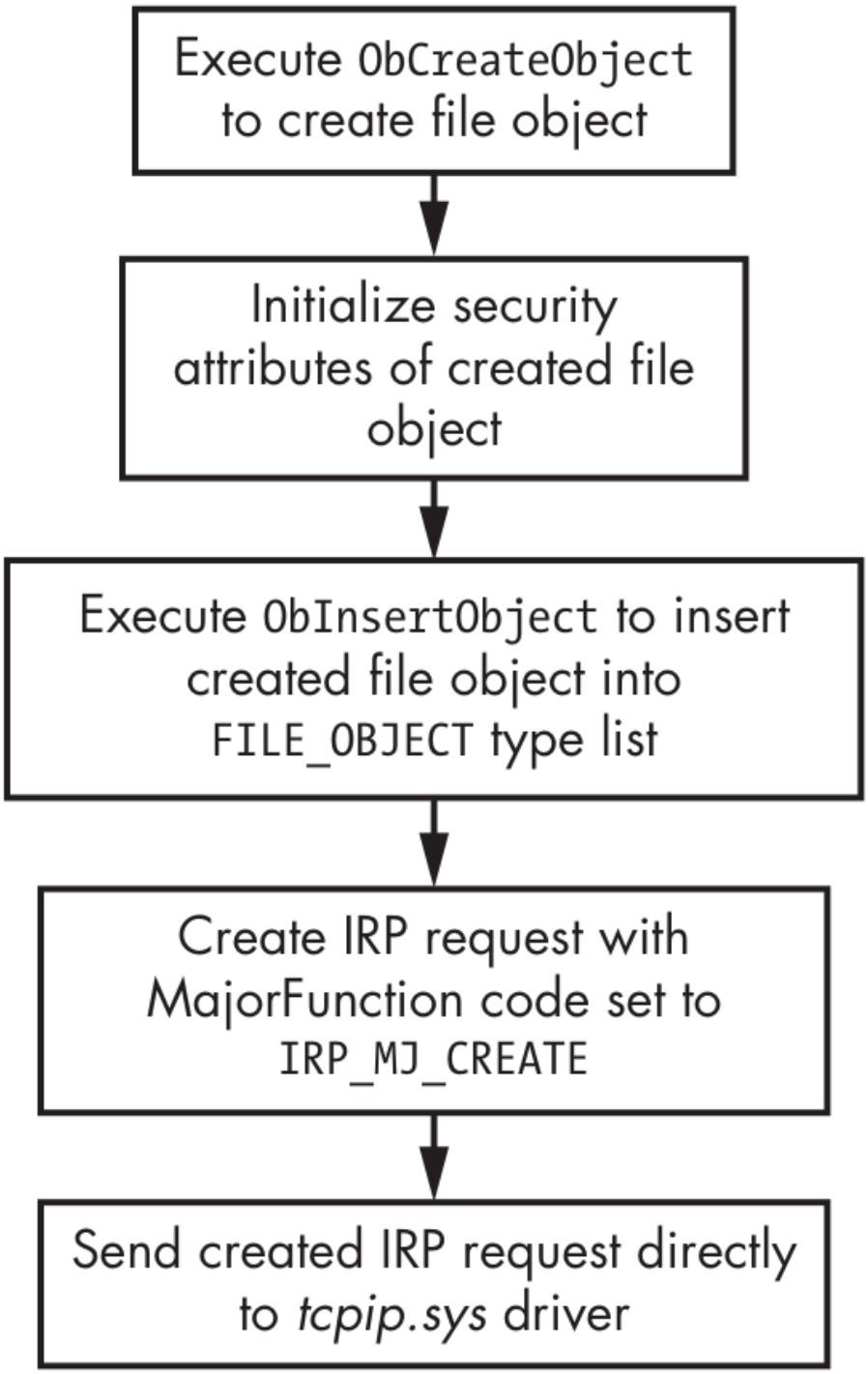


Рисунок 2-10. Собственная реализация процедуры `ZwCreateFile`*. Этапы: выполнить* `ObCreateObject`*, чтобы создать объект файла; инициализировать атрибуты безопасности созданного объекта; выполнить* `ObInsertObject`*, чтобы включить созданный объект в список типа* `FILE_OBJECT`*; создать IRP-запрос с кодом* `MajorFunction`*, равным* `IRP_MJ_CREATE`*; отправить созданный IRP-запрос непосредственно драйверу* `tcpip.sys`

Как видите, Festi вручную создаёт объект файла для связи с открываемым устройством и отправляет запрос `IRP_MJ_CREATE` непосредственно транспортному драйверу. Поэтому запрос минует все устройства, присоединённые к `\Device\Tcp` или `\Device\Udp`, и операция остаётся незамеченной защитным ПО, как показано на рисунке 2-11.

В левой части рисунка показана обычная обработка IRP. Пакет IRP проходит через весь стек драйверов, и все подключённые к нему драйверы, включая защитное ПО, получают пакет и исследуют его содержимое. В правой части показано, как Festi вместо этого отправляет пакет IRP непосредственно целевому драйверу, обходя все промежуточные драйверы.

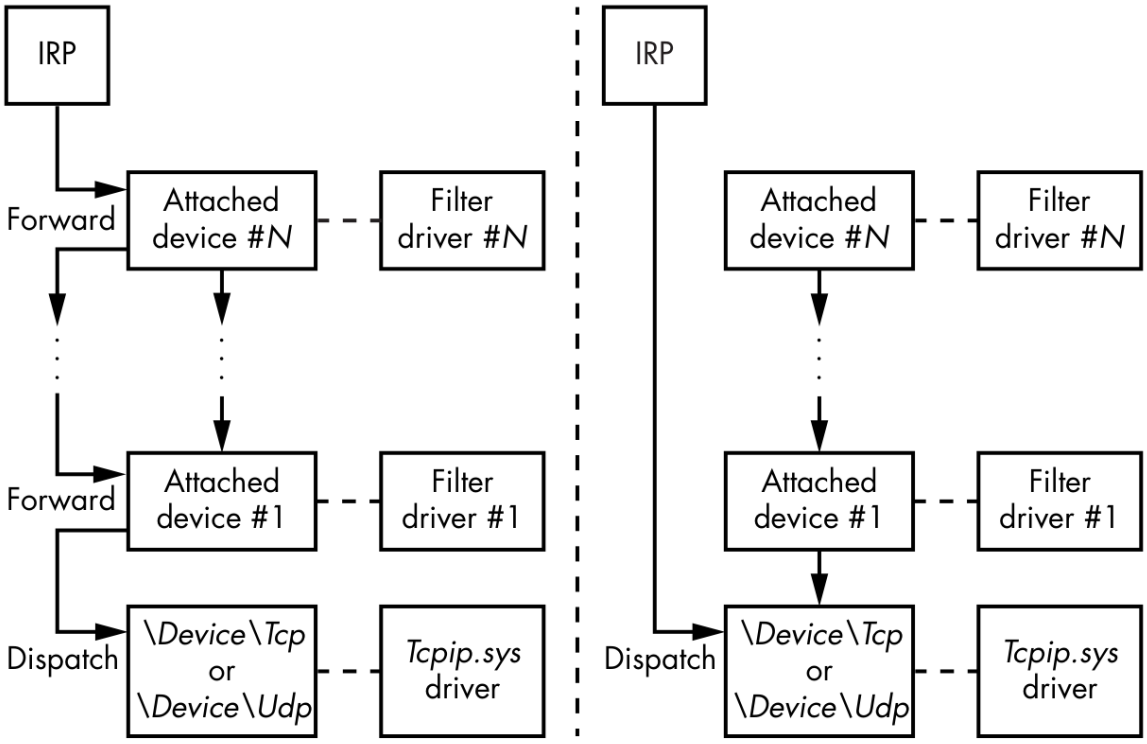


Рисунок 2-11. Обход защитного программного обеспечения для наблюдения за сетью. Левая схема показывает обычную передачу IRP через все присоединённые устройства и драйверы-фильтры; правая - прямую диспетчеризацию к `\Device\Tcp` *или* `\Device\Udp`. *Обозначения: Forward - передача дальше; Attached device - присоединённое устройство; Filter driver - драйвер-фильтр; Dispatch - диспетчеризация; Tcpip.sys driver - драйвер* `tcpip.sys`

Второй метод защитного ПО Festi обходит столь же ловко. Чтобы отправить запрос непосредственно устройству `\Device\Tcp` или `\Device\Udp`, вредоносному ПО нужны указатели на соответствующие объекты устройств. Фрагмент кода, выполняющий этот манёвр, приведён в листинге 2-6.

```
RtlInitUnicodeString(&DriverName, L"\\Driver\\Tcpip");
RtlInitUnicodeString(&tcp_name, L"\\Device\\Tcp");
RtlInitUnicodeString(&udp_name, L"\\Device\\Udp");
❶ if (!ObReferenceObjectByName(&DriverName,64,0,0x1F01FF,
                               IoDriverObjectType,0,0,&TcpipDriver))
{
  DevObj = TcpipDriver->DeviceObject;
  ❷ while ( DevObj )                          // перебрать связный список
  {                                           // структур DEVICE_OBJECT
    if ( !ObQueryNameString(DevObj, &Objname, 256, &v8) )
    {
      ❸ if ( RtlCompareUnicodeString(&tcp_name, &Objname, 1u) )
      {
       ❹ if ( !RtlCompareUnicodeString(&udp_name, &Objname, 1u) )
```

```
        {
          ObfReferenceObject(DevObj);
          this->DeviceUdp = DevObj;        // Сохранить указатель на \Device\Udp
        }
      } else
      {
        ObfReferenceObject(DevObj);
        this->DeviceTcp = DevObj;          // Сохранить указатель на \Device\Tcp
      }
    }
    DevObj = DevObj->NextDevice;       // Получить указатель на следующую
                                       // структуру DEVICE_OBJECT в списке
  }

  ObfDereferenceObject(TcpipDriver);
}
```

Листинг 2-6. Реализация метода обхода защитного программного обеспечения, наблюдающего за сетью

Festi получает указатель на объект драйвера `tcpip.sys`, выполняя недокументированную системную процедуру `ObReferenceObjectByName` ❶ и передавая ей в качестве параметра указатель на строку Unicode с именем целевого драйвера. Затем вредоносное ПО проходит по списку объектов устройств ❷, соответствующих объекту драйвера, и сравнивает их имена с `\Device\Tcp` ❸ и `\Device\Udp` ❹.

Получив таким способом дескриптор открытого устройства, вредоносное ПО использует его для отправки и получения данных по сети. Хотя Festi способен обходить защитное ПО, отправляемые им пакеты можно увидеть с помощью фильтров сетевого трафика, работающих на более низком уровне, чем Festi, например на уровне спецификации интерфейса сетевых драйверов (Network Driver Interface Specification, NDIS).

Алгоритм генерации доменов на случай отказа C&C-сервера

Ещё одна примечательная особенность Festi состоит в реализации алгоритма генерации доменных имён (domain generation algorithm, DGA), который применяется как резервный механизм, когда доменные имена C&C-серверов из конфигурационных данных бота недоступны. Такое может произойти, например, если правоохранительный орган ликвидирует доменные имена C&C-серверов Festi и вредоносное ПО лишится возможности загружать подключаемые модули и команды. Алгоритм получает на вход текущую дату и выдаёт доменное имя.

В таблице 2-2 перечислены доменные имена, созданные DGA для одного образца Festi. Как видите, все сгенерированные доменные имена являются псевдослучайными, что характерно для имён, создаваемых DGA.

Таблица 2-2. Доменные имена, сгенерированные Festi с помощью DGA

Дата	Доменное имя DGA
07/11/2012	`fzcbihskf.com`
08/11/2012	`pzcaihszf.com`
09/11/2012	`dzcxifsff.com`
10/11/2012	`azcgnfsmf.com`
11/11/2012	`bzcfnfsif.com`

Реализация DGA повышает устойчивость бот-сети к попыткам ликвидации. Даже если правоохранительным органам удастся отключить основные домены C&C-серверов, владелец бот-сети всё равно сможет вернуть контроль над ней, воспользовавшись DGA как резервным механизмом.

Вредоносные функции

Теперь, когда мы рассмотрели функции руткита, обратимся к вредоносным подключаемым модулям, загружаемым с C&C-серверов. В ходе исследования нам удалось получить образцы этих модулей и определить три их типа:

• `BotSpam.sys` для рассылки спама по электронной почте;
• `BotDos.sys` для проведения DDoS-атак;
• `BotSocks.sys` для предоставления прокси-служб.

Мы выяснили, что разные C&C-серверы обычно предоставляют разные типы подключаемых модулей: одни снабжают ботов только модулями рассылки спама, а другие работают исключительно с модулями DDoS-атак. Это показывает, что вредоносные функции программы зависят от C&C-серверов, перед которыми она отчитывается. Бот-сеть Festi не является монолитной, а состоит из подсетей ботов, предназначенных для разных целей.

Модуль рассылки спама

Подключаемый модуль `BotSpam.sys` отвечает за рассылку нежелательных электронных писем. C&C-сервер передаёт ему шаблон спам-сообщения и список адресов электронной почты получателей. Порядок работы подключаемого модуля рассылки спама показан на рисунке 2-12.

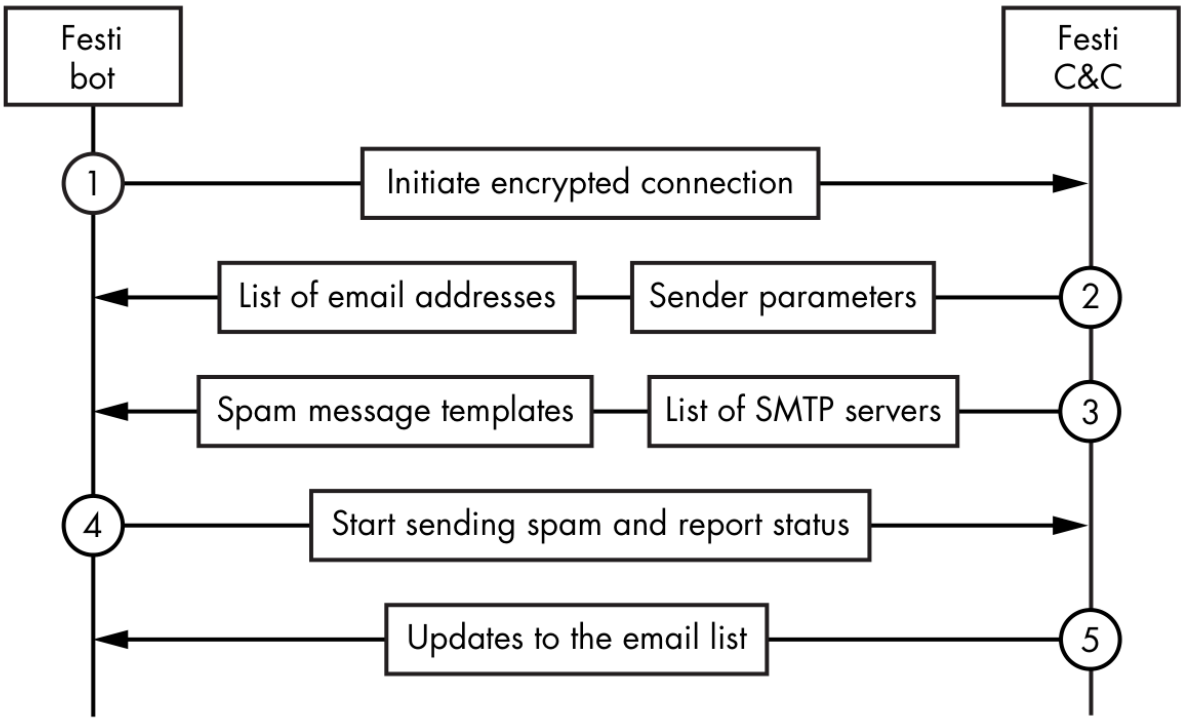


Рисунок 2-12. Схема работы подключаемого модуля рассылки спама Festi. Этапы: 1 - установить зашифрованное соединение; 2 - получить список адресов и параметры отправителя; 3 - получить шаблоны спам-сообщений и список SMTP-серверов; 4 - начать рассылку спама и сообщать состояние; 5 - получать обновления списка адресов

Сначала подключаемый модуль устанавливает зашифрованное соединение со своим C&C-сервером, чтобы загрузить список адресов электронной почты, параметры отправителя и сами шаблоны спам-сообщений. Затем он рассылает спам-письма получателям. Одновременно вредоносное ПО сообщает C&C-серверу состояние и запрашивает обновления списка адресов и шаблонов сообщений.

После этого подключаемый модуль проверяет состояние отправленных писем, ища в ответах SMTP-сервера определённые строки, обозначающие проблемы, например отсутствие получателя с указанным адресом, недоставленное письмо или классификацию письма как нежелательного. Если какая-либо из этих строк обнаружена в ответах SMTP-сервера, подключаемый модуль корректно завершает сеанс с сервером и переходит к следующему адресу в списке. Эта мера предосторожности помогает вредоносному ПО избежать ситуации, когда SMTP-сервер вносит IP-адрес заражённой машины в чёрный список отправителей спама и не позволяет программе продолжать рассылку.

Механизм DDoS-атак

Подключаемый модуль `BotDos.sys` позволяет боту проводить DDoS-атаки на указанные узлы. Модуль поддерживает несколько типов DDoS-атак на удалённые узлы с различной архитектурой и разным установленным программным обеспечением. Тип атаки зависит от конфигурационных данных, полученных от C&C-сервера; поддерживаются TCP-флуд, UDP-флуд, DNS-флуд и HTTP-флуд.

TCP-флуд

При TCP-флуде бот устанавливает большое число соединений с портом целевой машины. Каждый раз, когда Festi подключается к целевому порту сервера, сервер выделяет ресурсы для обработки входящего соединения. Вскоре его ресурсы исчерпываются, и он перестаёт отвечать клиентам.

По умолчанию целью является HTTP-порт `80`, однако его можно изменить соответствующими конфигурационными сведениями от C&C-сервера, благодаря чему вредоносное ПО способно атаковать HTTP-серверы, прослушивающие другие порты.

UDP-флуд

При UDP-флуде бот отправляет заполненные случайными данными UDP-пакеты случайно выбранной длины от 256 до 1024 байтов. Целевой порт также выбирается случайно и потому, скорее всего, будет закрыт. В результате атаки целевой узел формирует в ответ огромное число пакетов ICMP Destination Unreachable, после чего целевая машина становится недоступной.

DNS-флуд

Бот также способен проводить атаки типа DNS-флуд, отправляя на порт `53`, где работает служба DNS, целевого узла большое количество UDP-пакетов. Эти пакеты содержат запросы на разрешение случайно сгенерированного доменного имени в доменной зоне `.com`.

HTTP-флуд

Для атак типа HTTP-флуд на веб-серверы двоичный файл бота содержит множество разных строк агента пользователя. Они используются для создания большого числа HTTP-сеансов с веб-сервером, перегружая удалённый узел. Код формирования отправляемого HTTP-запроса приведён в листинге 2-7.

```
int __thiscall BuildHttpHeader(_BYTE *this, int a2)
{
  ❶ user_agent_idx = get_rnd() % 0x64u;
  str_cpy(http_header, "GET ");
  str_cat(http_header, &v4[204 * *(_DWORD *)(v2 + 4) + 2796]);
  str_cat(http_header, " HTTP/1.0\r\n");
  if ( v4[2724] & 2 )
  {
    str_cat(http_header, "Accept: */*\r\n");
    str_cat(http_header, "Accept-Language: en-US\r\n");
    str_cat(http_header, "User-Agent: ");
    ❷ str_cat(http_header, user_agent_strings[user_agent_idx]);
    str_cat(http_header, "\r\n");
  }
  str_cat(http_header, "Host: ");
  str_cat(http_header, &v4[204 * *(_DWORD *)(v2 + 4) + 2732]);
  str_cat(http_header, "\r\n");
  if ( v4[2724] & 2 )
    str_cat(http_header, "Connection: Keep-Alive\r\n");
  str_cat(http_header, "\r\n");
  result = str_len(http_header);
  *(_DWORD *)(v2 + 16) = result;
  return result;
}
```

Листинг 2-7. Фрагмент подключаемого модуля DDoS-атак Festi, формирующий HTTP-запрос

В точке ❶ код создаёт значение, которое затем в точке ❷ используется как индекс в массиве строк агентов пользователя.

Прокси-модуль Festi

Подключаемый модуль `BotSocks.sys` предоставляет злоумышленнику удалённую прокси-службу, реализуя сервер SOCKS поверх протоколов TCP и UDP. Сервер SOCKS устанавливает сетевое соединение с другим целевым сервером от имени клиента, а затем передаёт весь трафик в обоих направлениях между клиентом и целевым сервером.

В результате заражённая Festi машина становится прокси-сервером, позволяющим злоумышленникам подключаться к удалённым серверам через заражённую машину. Киберпреступники могут пользоваться такой службой для анонимизации, то есть сокрытия IP-адреса злоумышленника. Поскольку соединение устанавливается через заражённый узел, удалённый сервер видит IP-адрес жертвы, но не злоумышленника.

Подключаемый модуль Festi `BotSocks.sys` не использует никаких механизмов прокси с обратным соединением для обхода NAT (Network Address Translation, преобразования сетевых адресов), позволяющего нескольким компьютерам в сети совместно использовать один видимый извне IP-адрес. Загрузив модуль, вредоносное ПО открывает сетевой порт и начинает прослушивать входящие соединения. Номер порта выбирается случайно в диапазоне от 4000 до 65536. Подключаемый модуль отправляет номер прослушиваемого порта C&C-серверу, чтобы злоумышленник мог установить сетевое соединение с компьютером жертвы. В обычных условиях NAT препятствовал бы таким входящим соединениям, если только для целевого порта не настроено перенаправление.

Кроме того, подключаемый модуль `BotSocks.sys` пытается обойти брандмауэр Windows, который иначе может помешать открыть порт. Модуль изменяет раздел реестра `SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy\DomainProfile\GloballyOpenPorts\List`, содержащий список портов, которые разрешено открывать в профиле брандмауэра Windows. Вредоносное ПО добавляет в этот раздел два подраздела, разрешая соответственно входящие соединения TCP и UDP из любых источников.

> **SOCKS.** Socket Secure (SOCKS) - интернет-протокол, обеспечивающий обмен сетевыми пакетами между клиентом и сервером через прокси-сервер. Сервер SOCKS проксирует TCP-соединения клиента SOCKS с произвольным IP-адресом и предоставляет средство пересылки UDP-пакетов. Киберпреступники часто используют протокол SOCKS как средство обхода, которое позволяет трафику преодолевать интернет-фильтрацию и получать доступ к иначе заблокированному содержимому.

Заключение

Теперь у вас должно сложиться полное представление о том, что такое руткит Festi и на что он способен. Festi представляет собой интересный образец вредоносного ПО с хорошо спроектированной архитектурой и тщательно реализованными функциями. Каждый технический аспект программы соответствует принципам её проектирования: оставаться незаметной и быть устойчивой к автоматизированному анализу, системам наблюдения и криминалистическому анализу.

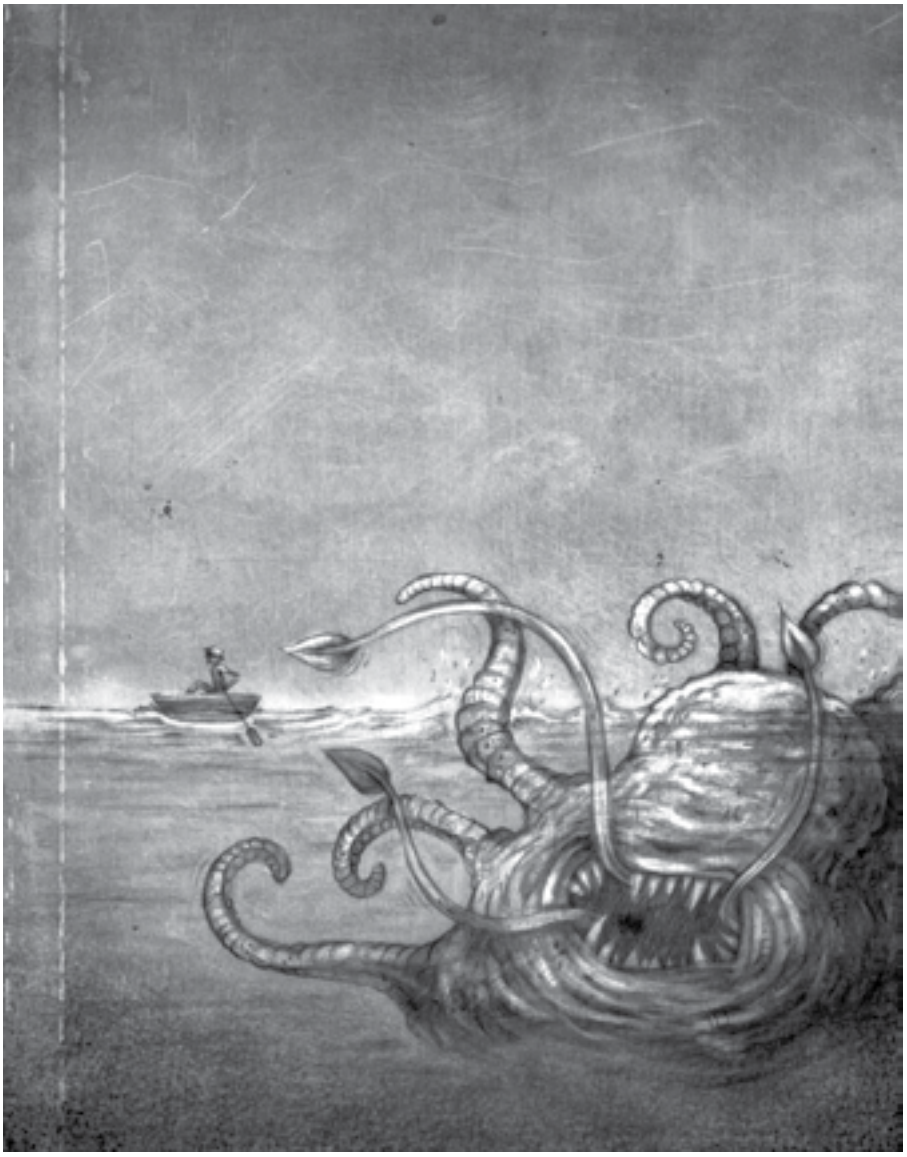
Энергозависимые вредоносные подключаемые модули, загружаемые с C&C-серверов, не оставляют следов на жёстком диске заражённой машины. Шифрование протокола сетевого взаимодействия с C&C-серверами затрудняет обнаружение Festi в сетевом трафике, а передовое применение сетевых сокетов режима ядра позволяет Festi обходить некоторые системы предотвращения вторжений на узел (Host Intrusion Prevention Systems, HIPS) и персональные брандмауэры.

Бот уклоняется от защитного ПО с помощью функций руткита, скрывающих в системе основной модуль и соответствующий раздел реестра. Эти методы были эффективны против защитного ПО на пике популярности Festi, но одновременно представляли собой один из его основных недостатков: руткит нацелен только на 32-разрядные системы. В 64-разрядных выпусках операционных систем Windows реализованы современные защитные средства, например PatchGuard, которые лишают эффективности арсенал методов вторжения Festi. Кроме того, 64-разрядные версии требуют, чтобы драйверы режима ядра имели действительную цифровую подпись, получить которую вредоносному ПО, очевидно, непросто. Как упоминалось в главе 1, для обхода этого ограничения разработчики вредоносного ПО создали технологию буткитов, которую мы подробно рассмотрим в части II.

Сноски

[1] [назад] Брайан Кребс, «Финансовый магнат связан с DDoS-атаками», блог Krebs on Security, 23 июня 2011 года, krebsonsecurity.com.
[2] [назад] Евгений Родионов и Александр Матросов, «Король спама: анализ бот-сети Festi», май 2012 года, welivesecurity.com.

Глава 3. Наблюдение за заражениями руткитов



Как проверить, не скрывается ли руткит в потенциально заражённой системе? Ведь само назначение руткита состоит в том, чтобы не позволить администраторам исследовать истинное состояние системы, поэтому поиск признаков заражения может превратиться в состязание умов или, точнее, в состязание на понимание внутренних структур системы. Поначалу аналитики не должны доверять никаким сведениям, полученным из заражённой системы; им следует стремиться найти более глубокие источники доказательств, которым можно доверять даже в скомпрометированном состоянии.

На примерах руткитов TDL3 и Festi мы знаем, что подходы к обнаружению руткитов, основанные на проверке целостности ядра в нескольких фиксированных местах, скорее всего, окажутся недостаточными. Руткиты постоянно развиваются, поэтому весьма вероятно, что новые образцы используют методы, неизвестные защитному ПО. В золотую эпоху руткитов в начале 2000-х годов их разработчики действительно постоянно внедряли новые приёмы, благодаря которым руткиты месяцами избегали обнаружения, пока специалисты по защите не успевали разработать и добавить в свои программы новые устойчивые методы выявления.

Такие задержки в разработке эффективной защиты создали нишу для нового типа программных средств, специализированных антируткитов. Ради более быстрого обнаружения руткитов они

позволяли себе вольности в алгоритмах выявления, а иногда и в отношении устойчивости системы. По мере созревания эти алгоритмы становились частью более традиционных систем предотвращения вторжений на узел (Host Intrusion Prevention System, HIPS), куда добавлялись новые эвристические методы «на переднем крае».

Столкнувшись с этими новшествами защитной стороны, разработчики руткитов ответили созданием способов активного нарушения работы антируткитов. Средства нападения и защиты на системном уровне совместно развивались в течение нескольких циклов. На протяжении этого совместного развития, и во многом благодаря ему, защитники существенно углубили понимание состава системы, поверхности атаки, целостности и профиля защиты. Здесь, как и в других областях компьютерной безопасности, справедливы слова старшего исследователя безопасности Microsoft Джона Ламберта: «Осуждая исследования атак, вы неверно оцениваете их вклад. Нападение и защита не равны. Защита - дитя нападения».

Таким образом, чтобы эффективно обнаруживать руткиты, специалист по защите должен научиться мыслить так же, как создатель руткита.

Методы перехвата

Чтобы помешать запуску или инициализации антируткитов, руткит должен перехватить управление в определённых точках операционной системы. Таких точек множество: они присутствуют и в стандартных, и в недокументированных механизмах ОС. Среди методов перехвата можно назвать изменение кода ключевых функций, замену указателей в различных структурах данных ядра и его драйверов, а также манипулирование данными с помощью таких методов, как непосредственное управление объектами ядра (Direct Kernel Object Manipulation, DKOM).

Чтобы внести порядок в этот, казалось бы, бесконечный перечень, мы рассмотрим три основных механизма ОС, которые руткиты могут перехватывать для получения контроля над запуском и инициализацией программ: системные события, системные вызовы и диспетчер объектов.

Перехват системных событий

Первый способ получить управление состоит в перехвате системных событий посредством обратных вызовов уведомления о событиях, то есть документированных интерфейсов ОС для обработки различных типов системных событий. Легитимным драйверам необходимо реагировать на создание новых процессов или потоков данных, загружая исполняемые двоичные файлы, а также создавая и изменяя разделы реестра. Чтобы разработчикам драйверов не приходилось создавать хрупкие недокументированные решения для перехвата, Microsoft предоставляет стандартизированные механизмы уведомления о событиях. Авторы вредоносного ПО используют те же механизмы, чтобы реагировать на системные события собственным кодом, оттесняя легитимную реакцию.

Например, процедура `CmRegisterCallbackEx` для драйверов режима ядра регистрирует функцию обратного вызова, которая выполняется всякий раз, когда кто-либо совершает операцию с системным реестром: создаёт, изменяет или удаляет раздел. Злоупотребляя этой возможностью, вредоносное ПО может перехватывать все запросы к системному реестру, исследовать их, а затем блокировать или разрешать.

Это позволяет руткиту защищать любой раздел реестра, соответствующий его драйверу режима ядра: скрывать раздел от защитного ПО и блокировать любые попытки удалить его.

Регистрация драйверов режима ядра в системном реестре. В Windows каждому драйверу режима ядра посвящена отдельная запись системного реестра, расположенная в разделе `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services`. В ней указываются имя и тип драйвера, расположение его образа на диске и момент загрузки: по запросу, при начальной загрузке, во время инициализации системы и так далее. Если удалить эту запись, ОС не сможет загрузить драйвер режима ядра. Поэтому, чтобы сохраняться в целевой системе, руткиты режима ядра часто защищают соответствующую запись реестра от удаления защитным ПО.

Другой вредоносный способ перехвата системных событий злоупотребляет процедурой драйвера режима ядра `PsSetLoadImageNotifyRoutine`. Эта процедура регистрирует функцию обратного вызова `ImageNotifyRoutine`, которая выполняется при каждом отображении исполняемого образа в память. Функция обратного вызова получает сведения о загружаемом образе: его имя и базовый адрес, а также идентификатор процесса, в адресное пространство которого загружается образ.

Руткиты часто злоупотребляют процедурой `PsSetLoadImageNotifyRoutine`, чтобы внедрить вредоносную полезную нагрузку в адресное пространство целевых процессов режима пользователя. Зарегистрировав процедуру обратного вызова, руткит получает уведомление при каждой загрузке образа и может исследовать переданные `ImageNotifyRoutine` сведения, чтобы определить, представляет ли интерес целевой процесс. Например, если руткит хочет внедрять полезную нагрузку режима пользователя только в веб-браузеры, он может проверить, соответствует ли загружаемый образ браузерному приложению, и действовать соответственно.

Ядро предоставляет и другие интерфейсы со сходными возможностями; мы обсудим их в следующих главах.

Перехват системных вызовов

Второй способ заражения предполагает перехват другого ключевого механизма ОС: системных вызовов, которые служат основным средством взаимодействия программ пользовательского пространства с ядром. Поскольку практически любой вызов API из пользовательского пространства порождает один или несколько соответствующих системных вызовов, руткит, способный диспетчеризировать системные вызовы, получает полный контроль над системой.

В качестве примера рассмотрим перехват вызовов файловой системы. Он особенно важен для руткитов, которым необходимо постоянно скрывать собственные файлы, чтобы предотвратить непредусмотренный доступ к ним. Когда защитное ПО или пользователь просматривает файловую систему в поисках подозрительных либо вредоносных файлов, система выполняет системный вызов, предписывающий драйверу файловой системы запросить файлы и каталоги. Перехватив такой системный вызов, руткит может изменить возвращаемые данные и исключить из результатов запроса сведения о своих вредоносных файлах, как мы видели в разделе «Метод сокрытия вредоносного драйвера на диске» на странице 22.

Чтобы понять, как противодействовать подобным злоупотреблениям и защищать вызовы файловой системы от руткитов, сначала нужно кратко рассмотреть структуру файловой подсистемы. Она служит прекрасным примером того, как внутренние механизмы ядра ОС разделяются на множество специализированных уровней и следуют многочисленным соглашениям о взаимодействии между ними. Эти понятия непрозрачны даже для большинства системных разработчиков, но не для авторов руткитов.

Файловая подсистема

Файловая подсистема Windows тесно интегрирована с подсистемой ввода-вывода. Эти подсистемы имеют модульную и иерархическую структуру, а за функции каждого уровня отвечают отдельные драйверы. Существует три основных типа драйверов.

Драйверы устройств хранения - низкоуровневые драйверы, взаимодействующие с контроллерами конкретных устройств, таких как порты, шины и накопители. Большинство этих драйверов поддерживают Plug and Play (PnP), загружаются диспетчером PnP и работают под его управлением.

Драйверы томов хранения - драйверы среднего уровня, управляющие абстракциями томов поверх разделов устройств хранения. Для взаимодействия с нижними уровнями дисковой подсистемы эти драйверы создают физический объект устройства (physical device object, PDO), представляющий каждый раздел. Когда на разделе монтируется файловая система, её драйвер создаёт объект устройства тома (volume device object, VDO), который представляет раздел драйверам файловой системы более высокого уровня, описанным далее.

Драйверы файловых систем реализуют конкретные файловые системы, такие как FAT32, NTFS, CDFS и другие, а также создают пару объектов: VDO и объект управляющего устройства (control device object, CDO), представляющий данную файловую систему, а не нижележащий раздел. Такие устройства CDO имеют имена наподобие `\Device\Ntfs`.

> **Примечание.** Подробнее о разных типах драйверов см. в документации Windows: docs.microsoft.com.

На рисунке 3-1 показана упрощённая версия этой иерархии объектов устройств на примере дискового устройства SCSI.

На уровне драйверов устройств хранения мы видим объекты устройства адаптера SCSI и дискового устройства. Эти объекты создаются и управляются тремя разными драйверами: драйвером шины PCI, который перечисляет, то есть обнаруживает, доступные на шине PCI адаптеры хранения; драйвером порта или минипорта SCSI, который инициализирует обнаруженный адаптер хранения SCSI и управляет им; а также драйвером класса диска, который управляет дисковым устройством, подключённым к адаптеру хранения SCSI.

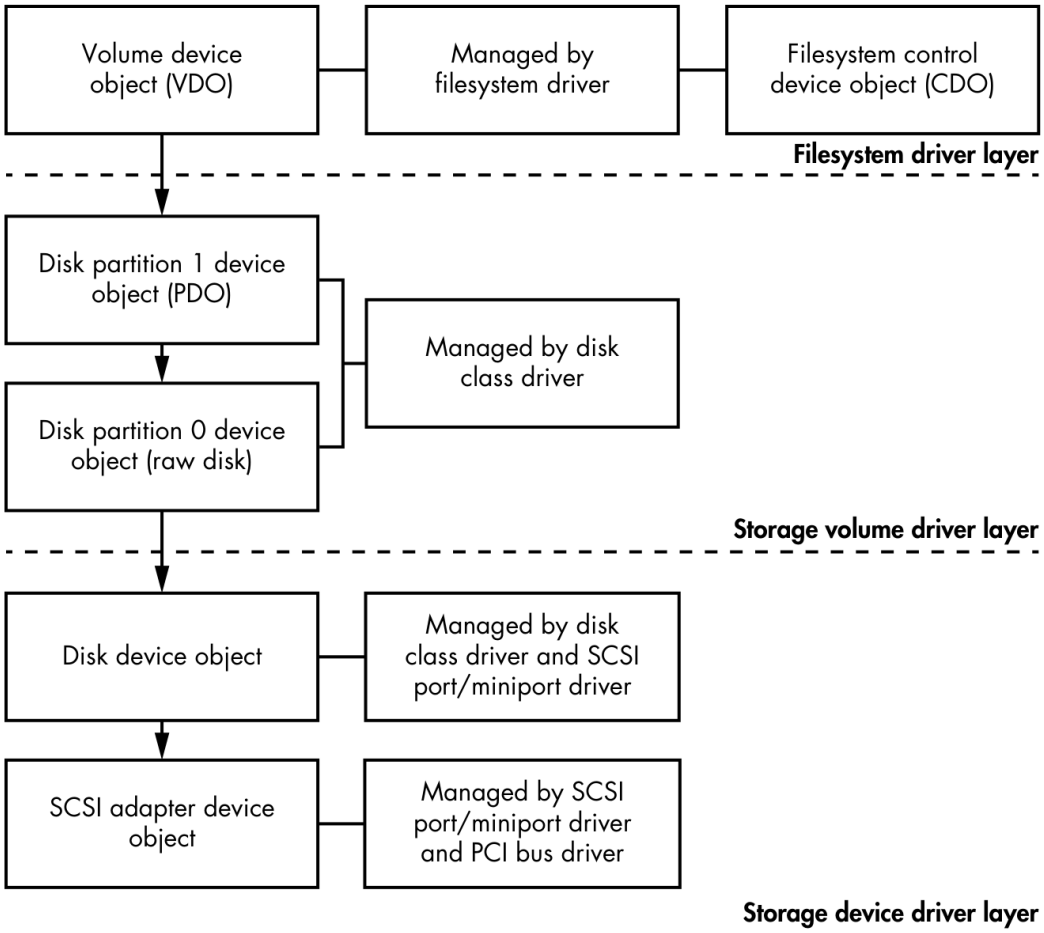


Рисунок 3-1. Пример стека драйверов устройств хранения. Уровни сверху вниз: драйвер файловой системы; драйвер тома хранения; драйвер устройства хранения. Объекты: объект устройства тома (VDO), объект управляющего устройства файловой системы (CDO), объекты разделов диска 1 (PDO) и 0 (необработанный диск), объект дискового устройства и объект устройства адаптера SCSI. Пояснения управления: драйвер файловой системы; драйвер класса диска; драйвер класса диска и драйвер порта или минипорта SCSI; драйвер порта или минипорта SCSI и драйвер шины PCI

На уровне драйверов томов хранения показаны разделы 0 и 1, которые также создаются драйвером класса диска. Раздел 0 представляет весь необработанный диск и существует всегда, независимо от того, разбит диск на разделы или нет. Раздел 1 представляет первый раздел дискового устройства. В нашем примере раздел всего один, поэтому показаны только разделы 0 и 1.

Раздел 1 необходимо предоставить пользователям, чтобы они могли сохранять файлы на дисковом устройстве и обращаться к ним. Для этого драйвер файловой системы создаёт VDO в верхней части стека хранения, на уровне драйвера файловой системы. Обратите внимание, что поверх VDO или между объектами устройств в стеке могут быть присоединены дополнительные фильтрующие объекты устройств хранения, которые для простоты на рисунке не показаны. Кроме

того, справа вверху виден CDO файловой системы, который ОС использует для управления её драйвером.

Рисунок показывает, как сложность стека драйверов хранения предоставляет руткитам возможности перехватывать операции файловой системы, изменять или скрывать данные.

Перехват файловых операций

Руткиту гораздо проще перехватывать файловые операции на верхнем уровне, то есть на уровне драйвера файловой системы, чем на нижних уровнях. Тогда руткит видит все такие операции на уровне прикладного программиста, и ему не приходится находить и разбирать невидимые программисту структуры файловой системы, соответствующие пакетам запросов ввода-вывода (IRP), которые передаются драйверам нижних уровней.

Если же руткит перехватывает операции на нижних уровнях, ему необходимо заново реализовать части файловых систем Windows, а это сложная и подверженная ошибкам задача. Однако перехваты на уровне низкоуровневых драйверов всё же существуют: посекторную карту диска получить сравнительно просто, а блокировать или перенаправлять операции с секторами можно даже на уровне минипорт-драйвера, как продемонстрировал TDL3.

Независимо от уровня, на котором руткит перехватывает операции ввода-вывода хранилища, существуют три основных метода перехвата:

1. Присоединение фильтрующего драйвера к стеку драйверов целевого устройства.
2. Замена указателей на функции обработки IRP или FastIO в структуре дескриптора драйвера.
3. Замена кода этих функций драйвера для обработки IRP или FastIO.

> **FastIO.** При выполнении операций ввода-вывода пакеты IRP проходят через весь стек устройств хранения, от самого верхнего объекта устройства до нижнего. FastIO - необязательный метод, предназначенный для быстрого выполнения синхронных операций ввода-вывода с кэшированными файлами. При операциях FastIO данные передаются непосредственно между буферами режима пользователя и системным кэшем в обход стека файловой системы и драйверов хранения. Благодаря этому операции ввода-вывода с кэшированными файлами выполняются гораздо быстрее.

В главе 2 мы рассмотрели руткит Festi, применявший первый метод перехвата: Festi присоединял вредоносный фильтрующий объект устройства к вершине стека драйверов хранения на уровне драйвера файловой системы.

Далее в книге мы обсудим буткиты TDL4 (глава 7), Olmasco (глава 10) и Rovnix (глава 11), каждый из которых применяет второй метод: они перехватывают дисковые операции ввода-вывода на самом нижнем возможном уровне, уровне драйвера устройства хранения. Буткит Gapz, рассматриваемый в главе 12, применяет третий метод, также на уровне драйвера устройства хранения. Подробности реализации каждого метода можно найти в соответствующих главах.

Этот краткий обзор файловой системы Windows показывает, что благодаря сложности системы руткит располагает богатым выбором целей в данном стеке драйверов. Руткит может перехватить управление на любом уровне этого стека, а то и сразу на нескольких уровнях. Антируткит должен учитывать все эти возможности, например устанавливая собственные перехватчики или проверяя, выглядят ли зарегистрированные обратные вызовы легитимными. Это, очевидно, трудная задача, но специалисты по защите как минимум должны понимать цепочку диспетчеризации соответствующих драйверов.

Перехват диспетчера объектов

Третья категория рассматриваемых в этой главе перехватов нацелена на методы диспетчера объектов Windows. Диспетчер объектов - подсистема, управляющая ресурсами ОС, которые в ветви архитектуры Windows NT, лежащей в основе всех современных выпусков Windows, представлены объектами ядра. Детали реализации диспетчера объектов и связанных структур данных могут различаться в разных версиях Windows. Этот раздел наиболее актуален для версий, предшествующих Windows 7, но общий подход применим и к другим версиям.

Один из способов получить контроль над диспетчером объектов состоит в перехвате образующих его функций ядра Windows с именами `Ob*`. Однако руткиты редко поступают так по той же причине, по которой они редко нацеливаются на записи таблицы системных вызовов верхнего уровня: такие перехваты слишком очевидны и легко обнаруживаются. На практике руткиты применяют более сложные приёмы, направленные на ядро, которые мы сейчас опишем.

По существу, каждый объект ядра представляет собой структуру в памяти режима ядра, которую можно приблизительно разделить на две части: заголовок с метаданными диспетчера и тело объекта, заполняемое по мере необходимости подсистемой, которая создаёт и использует объект. Заголовок имеет структуру `OBJECT_HEADER`, содержащую указатель на дескриптор типа объекта `OBJECT_TYPE`. Последний также является структурой и служит одним из основных атрибутов объекта. Как и положено современной системе типов, структура, представляющая тип, сама является объектом, в теле которого содержатся соответствующие сведения о типе. Такая архитектура реализует наследование объектов посредством метаданных, хранящихся в заголовке.

Однако для обычного программиста все эти тонкости системы типов не имеют большого значения. Работа с большинством объектов ведётся через системные службы, которые обращаются к каждому объекту по его дескриптору (`HANDLE`), скрывая внутреннюю логику диспетчеризации объектов и управления ими.

Тем не менее некоторые поля дескриптора типа объекта `OBJECT_TYPE` представляют интерес для руткита, например указатели на процедуры обработки определённых событий: открытия, закрытия и удаления объектов. Перехватывая эти процедуры, руткит может получить управление и манипулировать данными объекта либо изменять их.

При этом все присутствующие в системе типы можно перечислить в пространстве имён диспетчера как объекты каталога `ObjectTypes`. Чтобы выполнить перехват, руткит может атаковать эти сведения двумя способами: непосредственно заменить указатель на функции-обработчики, направив его на сам руткит, либо заменить указатель типа в заголовке объекта.

Поскольку отладчики Windows используют эти метаданные и доверяют им при исследовании объектов ядра, перехваты руткитов, эксплуатирующие системные метаданные того же типа, трудно обнаружить.

Ещё сложнее точно выявлять руткиты, которые захватывают метаданные типов существующих объектов. Получившийся перехват действует более избирательно, а потому менее заметен. Пример такого перехвата показан на рисунке 3-2.

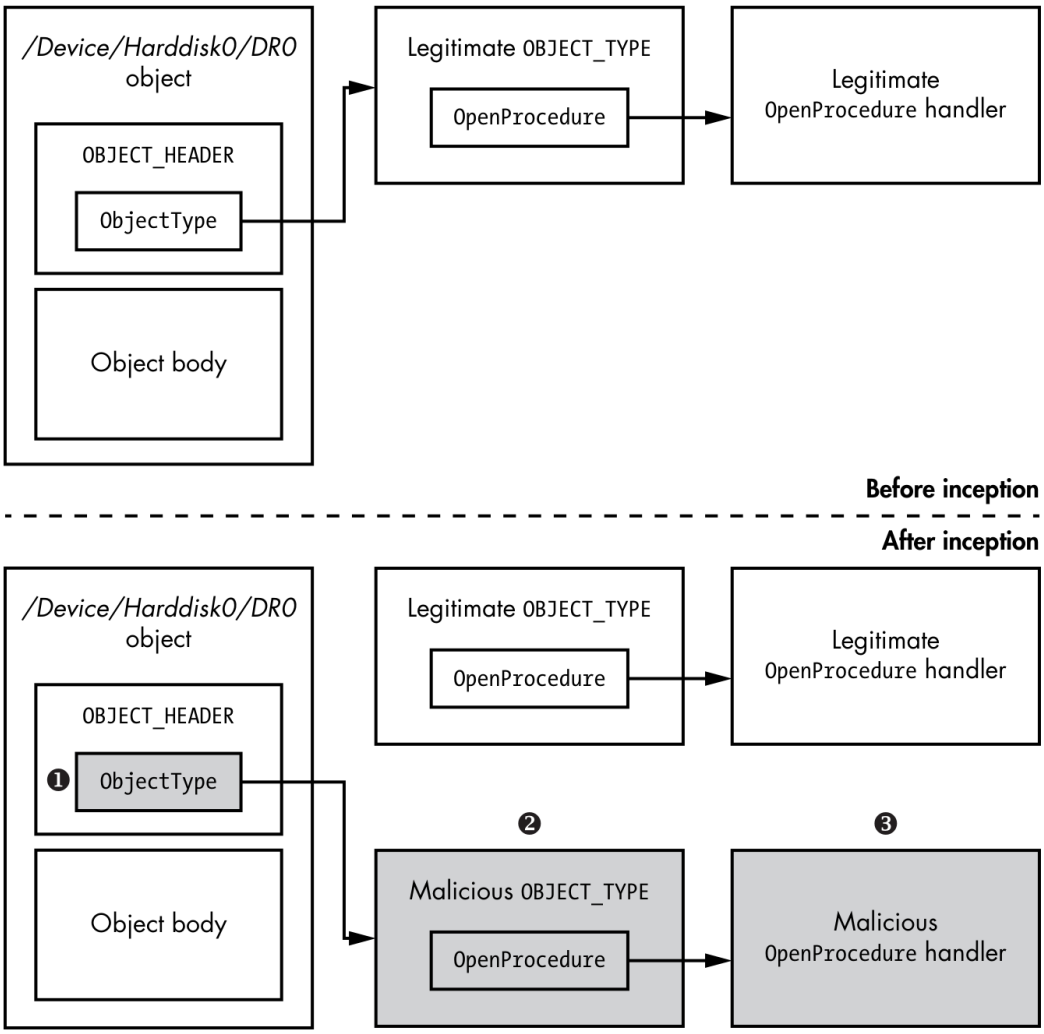


Рисунок 3-2. Перехват обработчика `OpenProcedure` *посредством изменения* `ObjectType`*. Верхняя схема показывает состояние до перехвата: легитимные* `OBJECT_TYPE` *и обработчик* `OpenProcedure`*. Нижняя схема показывает состояние после перехвата:* ❶ *заменённый указатель* `ObjectType`*,* ❷ *вредоносную копию* `OBJECT_TYPE` *и* ❸ *вредоносный обработчик* `OpenProcedure`*. Object body - тело объекта; OBJECT_HEADER - заголовок объекта*

В верхней части рисунка 3-2 показано состояние объекта до перехвата руткитом: заголовок объекта и дескриптор типа не затронуты и не изменены. В нижней части показано состояние после того, как руткит изменил дескриптор типа. Руткит получает указатель на объект, представляющий устройство хранения, например `\Device\Harddisk0\DR0`. Затем он создаёт собственную копию структуры `OBJECT_TYPE` для этого устройства ❷. Внутри копии указатель на интересующую его функцию, в нашем примере обработчик `OpenProcedure`, заменяется указателем на собственную функцию-обработчик руткита ❸. После этого указатель на структуру «злого двойника» заменяет указатель типа в дескрипторе исходного устройства ❶. Теперь поведение заражённого диска, описываемое его метаданными, почти не отличается от поведения нескомпрометированного объекта диска, за исключением обработчика, заменённого только для данного экземпляра объекта.

Обратите внимание, что легитимные структуры, описывающие все остальные объекты дисков того же типа, остаются нетронутыми. Изменённые метаданные присутствуют лишь в одной копии, на которую указывает только целевой объект. Чтобы найти и распознать это расхождение, алгоритм

обнаружения должен перечислить поля типов всех экземпляров объектов дисков. Систематический поиск подобных расхождений представляет собой чрезвычайно трудную задачу, требующую полного понимания реализации абстракций объектной подсистемы.

Восстановление ядра системы

У защитных механизмов может возникнуть соблазн попытаться нейтрализовать руткит глобально, то есть автоматически восстановить целостность скомпрометированной системы с помощью алгоритма, который проверяет содержимое различных внутренних таблиц диспетчеризации и структур метаданных, а также функции, на которые указывают эти структуры. При таком подходе сначала восстанавливается или проверяется таблица описателей системных служб (SSDT), код в начале нескольких стандартных функций системных вызовов ядра, а затем проверяются и восстанавливаются все структуры данных ядра, предположительно подвергшиеся изменению. Однако, как вы наверняка уже понимаете, эта стратегия восстановления сопряжена со множеством опасностей и вовсе не гарантирует успеха.

Найти или вычислить «чистые» значения указателей на функции системных вызовов и их обратные вызовы нижних уровней, необходимые для восстановления правильной диспетчеризации системных вызовов, совсем непросто. Не легче найти и чистые копии системных файлов, из которых можно было бы восстановить изменённые сегменты кода ядра.

Но даже если предположить, что эти задачи выполнимы, не каждое обнаруженное изменение ядра в действительности является вредоносным. Многие автономные легитимные программы, например рассмотренные ранее антируткиты, а также более традиционные брандмауэры, антивирусы и HIPS, устанавливают собственные безвредные перехватчики потока управления ядра. Перехваты антивируса бывает трудно отличить от перехватов руткита; фактически применяемые ими методы изменения потока управления могут быть неотличимы друг от друга. Поэтому легитимную антивредоносную программу можно принять за то, от чего она защищает, и отключить. То же относится к программным агентам управления цифровыми правами (digital rights management, DRM), которые настолько трудно отличить от руткитов, что DRM-агент Sony 2005 года стал известен как «руткит Sony».

Ещё одна трудность при обнаружении и нейтрализации руткитов заключается в обеспечении безопасности алгоритма восстановления. Структуры данных ядра используются постоянно, поэтому любая несинхронизированная запись в них, например чтение изменяемой структуры до того, как она будет правильно перезаписана, способна вызвать сбой ядра.

Кроме того, руткит может в любой момент попытаться восстановить свои перехватчики, создавая дополнительные угрозы устойчивости.

С учётом всех обстоятельств автоматическое восстановление целостности ядра лучше подходит как реактивная мера против известных угроз, чем как общее решение для получения достоверных сведений о ядре.

Однократно обнаружить и восстановить цепочки диспетчеризации функций ядра также недостаточно. Руткит может продолжать отслеживать любые изменения кода ядра и данных, на которые опираются его перехватчики, и постоянно пытаться восстанавливать их. Некоторые руткиты действительно наблюдают и за своими файлами и разделами реестра, восстанавливая их после удаления защитным ПО. Специалист по защите вынужден играть в современную версию классической программистской игры Core Wars 1984 года, в которой программы сражаются за контроль над памятью компьютера.

Если позаимствовать цитату из другой классики, фильма «Военные игры», то «единственный выигрышный ход - не играть». Осознав это, отрасль операционных систем разработала решения для защиты целостности ОС, которые начинали действовать во время загрузки и позволяли опередить атакующие руткиты. В результате защитникам больше не приходилось следить за бесчисленными таблицами указателей и заманчивыми фрагментами кода ОС, например прологами функций-обработчиков.

В полном соответствии с природой совместного развития нападения и защиты эти усилия побудили злоумышленников исследовать способы захвата процесса загрузки. Так появился буткит, которому посвящены последующие главы.

Если ваше путешествие по взлому Windows началось после Windows XP SP1, можете сразу перейти к следующей главе, пока мы предаёмся бесцельной ностальгии по отладке ОС. Но если рассказы седобородых ветеранов обладают для вас особой притягательностью, читайте дальше.

Великая гонка вооружений руткитов: ностальгическая заметка

Начало 2000-х годов было золотой эпохой руткитов: защитное ПО явно проигрывало гонку вооружений, поскольку могло реагировать на приёмы, обнаруженные в новых руткитах, но не предотвращать их. Причина заключалась в том, что в то время единственным доступным аналитикам руткитов инструментом был отладчик ядра, работавший с одним экземпляром ОС.

Несмотря на ограничения, этот отладчик ядра, NuMega SoftIce, мог остановить и надёжно исследовать состояние операционной системы, что остаётся сложной задачей даже для современных средств. До Windows XP Service Pack 2 SoftIce считался эталонным отладчиком ядра. Сочетание клавиш позволяло аналитикам полностью остановить ядро, перейти в локальную консоль отладчика, показанную на рисунке 3-3, и искать присутствие руткита во всей полностью замороженной памяти ОС, получая представление, которое руткиты ядра не могли исказить.

Осознав угрозу, которую представлял SoftIce, авторы руткитов быстро разработали методы обнаружения его присутствия в системе, однако эти приёмы ненадолго задержали аналитиков. Благодаря консоли SoftIce у защитников появился корень доверия, который злоумышленники не могли подменить, и положение изменилось в пользу защитников. Многие аналитики, начинавшие карьеру с отладочных возможностей SoftIce, сетуют на утрату способности мгновенно замораживать состояние всей ОС и переходить в консоль отладчика, показывавшую истинное состояние всей памяти.

Обнаружив руткит, аналитики могли сочетать статический и динамический анализ, чтобы найти нужные места в его коде, нейтрализовать проверки присутствия SoftIce, а затем пошагово выполнить код руткита и подробно изучить его работу.

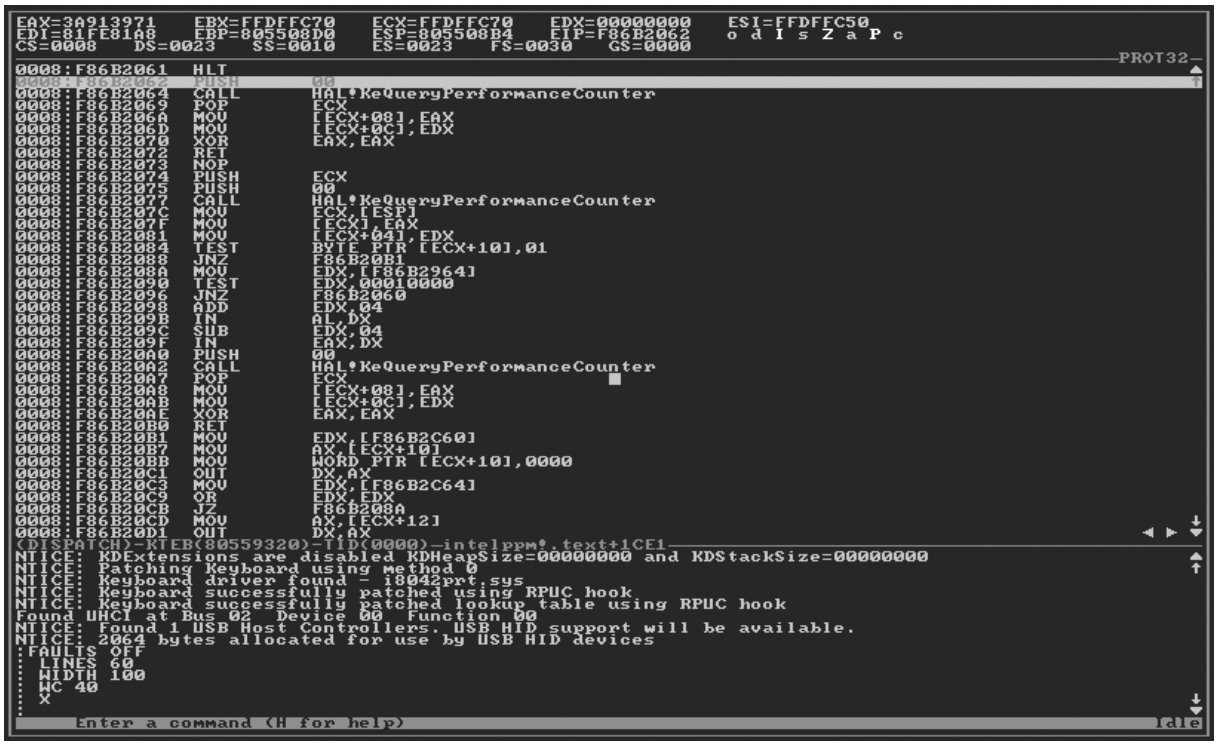


Рисунок 3-3. Локальная консоль отладчика SoftIce

Увы, SoftIce больше нет: Microsoft приобрела его производителя, в частности, чтобы усилить собственный отладчик ядра WinDbg. Сегодня WinDbg остаётся самым мощным средством анализа аномалий в работающем ядре Windows. Он может выполнять такой анализ даже удалённо, кроме случаев вредоносного вмешательства в работу самого отладчика. Однако независимая от ОС консоль наблюдения SoftIce исчезла.

Утрата консоли не обязательно сыграла на руку злоумышленникам. Хотя теоретически руткит может вмешиваться не только в работу защитного ПО, но и удалённого отладчика, такое вмешательство, вероятно, будет достаточно заметным, чтобы вызвать обнаружение. Для скрытных руткитов целевых атак столь заметное поведение означает провал задачи. В некоторых обнаруженных высококлассных вредоносных программах действительно имелись функции выявления удалённого отладчика, но эти проверки были слишком заметны, и аналитики легко обходили их.

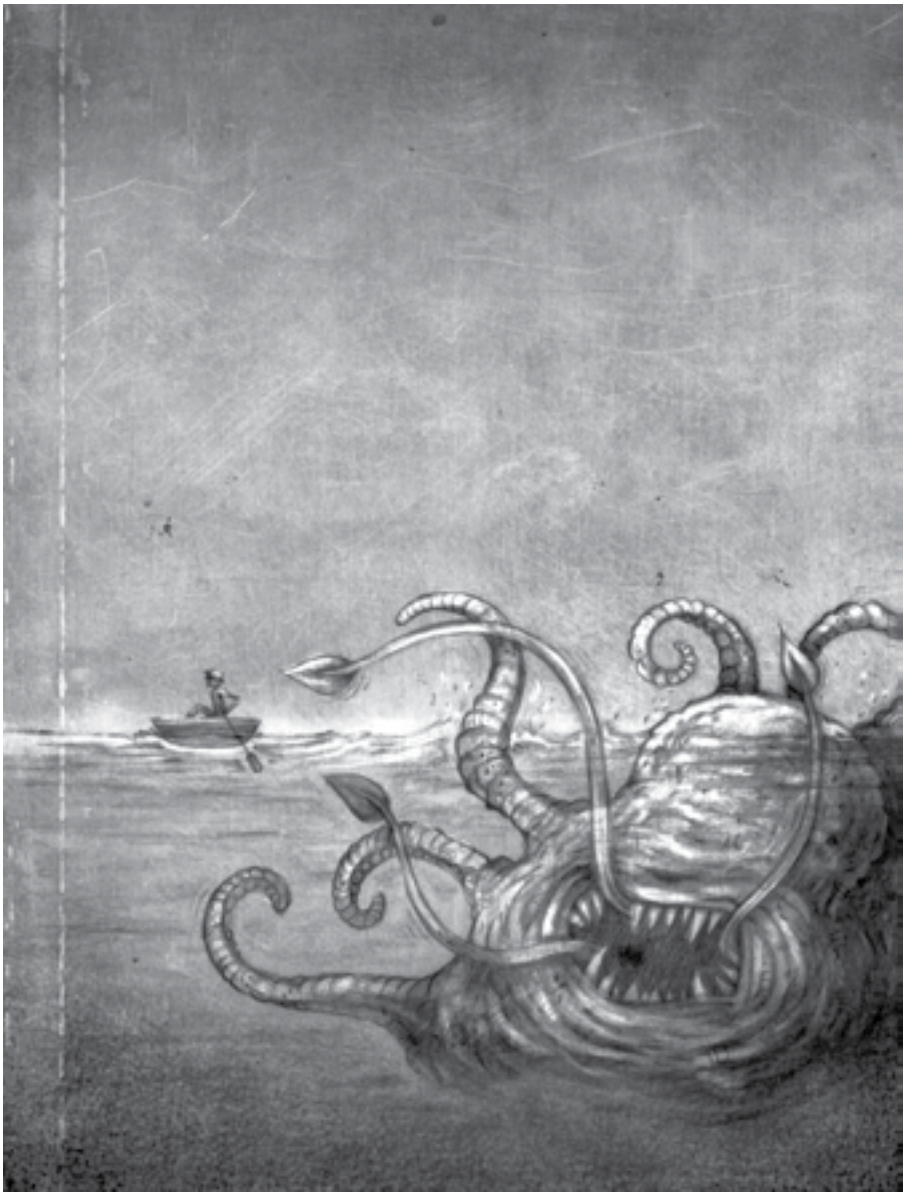
Преимущество атакующих по-настоящему начало ослабевать лишь тогда, когда Microsoft стала усложнять разработку руткитов с помощью особых защитных мер, которые мы обсудим далее в книге. В наши дни HIPS применяют подход обнаружения и реагирования на конечных точках (Endpoint Detection and Response, EDR): собирают как можно больше сведений о системе, загружают их на центральный сервер, а затем применяют алгоритмы обнаружения аномалий. В их число входят алгоритмы, предназначенные для выявления действий, которые вряд ли были инициированы известными пользователями системы и потому указывают на компрометацию. Очевидная необходимость собирать и использовать подобные сведения для обнаружения потенциального руткита показывает, насколько трудно отличить безвредное от вредоносного в одном образе ядра ОС.

Заключение

Гонка вооружений продолжается: обе стороны по-прежнему совместно развиваются, но теперь борьба переместилась в новую область, процесс загрузки. В следующих главах описываются новые технологии, призванные защитить целостность ядра ОС и лишить злоумышленников доступа к множеству целей в нём, а также ответ атакующих, которые скомпрометировали более ранние этапы нового защищённого процесса загрузки и выявили внутренние соглашения и слабости его архитектуры.

Часть II. Буткиты

Глава 4. Эволюция буткита



В этой главе вы познакомитесь с буткитом, вредоносной программой, которая заражает ранние этапы запуска системы до полной загрузки операционной системы. После периода упадка, вызванного изменениями процесса загрузки ПК, буткиты впечатляющим образом вернулись. Современные буткиты используют разновидности старых методов сокрытия и сохранения присутствия, применявшихся ранними буткитами, чтобы как можно дольше оставаться активными в целевой системе без ведома пользователя.

В этой главе мы рассмотрим самые ранние буткиты, проследим изменения их популярности, включая впечатляющее возвращение в последние годы, и обсудим современное вредоносное ПО, заражающее процесс загрузки.

Первые буткиты

История заражений буткитами началась ещё до появления IBM PC в продаже. Звание «первого буткита» обычно присваивают Creeper, самореплицирующейся программе, обнаруженной примерно в 1971 году. Creeper работала в сетевой операционной системе TENEX на компьютерах VAX PDP-10. Первым известным антивирусом стала программа Reaper, созданная для удаления заражений Creeper. В этом разделе мы рассмотрим ранние примеры буткитов, начиная с Creeper.

Инфекторы загрузочного сектора

Инфекторы загрузочного сектора (boot sector infectors, BSI) относились к числу самых ранних буткитов. Впервые их обнаружили во времена MS-DOS, неграфической операционной системы, предшествовавшей Windows, когда BIOS ПК по умолчанию пыталась загрузиться с любого диска, найденного в дисководе гибких дисков. Как следует из названия, эти вредоносные программы заражали загрузочные секторы дискет, расположенные в первом физическом секторе диска.

При запуске BIOS искала загрузочную дискету в дисководе A и выполняла любой код, найденный в загрузочном секторе. Если заражённая дискета оставалась в дисководе, она заражала систему с помощью BSI, даже если сама не была загрузочной.

Хотя некоторые BSI заражали и дискету, и файлы операционной системы, большинство являлись чистыми, то есть зависели от конкретного оборудования и не имели компонента ОС. Чистые BSI для связи с оборудованием и заражения дисков полагались исключительно на прерывания, предоставляемые BIOS. Поэтому заражённая дискета пыталась заразить любой IBM-совместимый ПК независимо от запущенной на нём ОС.

Elk Cloner и Load Runner

Вирусное ПО BSI впервые нацелилось на микрокомпьютер Apple II, операционная система которого обычно целиком находилась на дискетах. Первым вирусом, заразившим Apple II, считается Elk Cloner Рича Скренты. В 1982-1983 годах[1] он применял способ заражения, характерный для BSI, хотя появился за несколько лет до вирусов загрузочного сектора ПК.

По существу, Elk Cloner внедрял себя в загруженную Apple OS, чтобы изменить её. Затем вирус оставался в оперативной памяти и заражал другие дискеты, перехватывая обращения к диску и перезаписывая своим кодом их системные загрузочные секторы. При каждой пятидесятой загрузке он показывал следующее сообщение, которое иногда великодушно называют стихотворением:

```
Elk Cloner:
Программа с характером
    Она попадёт на все твои диски

      Она проникнет в твои микросхемы
        Да, это Cloner!
    Она приклеится к тебе намертво
      И даже изменит оперативную память
        Впустите Cloner!
```

Следующей известной вредоносной программой для Apple II стала Load Runner, впервые замеченная в 1989 году. Load Runner перехватывала команду сброса Apple, вызывавшуюся сочетанием клавиш Control-Command-Reset, и воспринимала её как сигнал записать себя на текущую дискету, что позволяло пережить сброс. Это был один из первых методов сохранения присутствия вредоносного ПО, предвосхитивший более сложные попытки незаметно оставаться в системе.

Вирус Brain

В 1986 году появился первый вирус для ПК, Brain. Исходная версия Brain поражала только дискеты ёмкостью 360 Кбайт. Будучи довольно крупным BSI, Brain заражал своим загрузчиком самый первый загрузочный сектор дискеты. Основное тело вируса и исходный загрузочный сектор сохранялись в свободных секторах дискеты. Brain помечал эти секторы, то есть секторы с исходным загрузочным кодом и основным телом, как «повреждённые», чтобы ОС не перезаписала их.

Некоторые методы Brain используются и в современных буткитах. Во-первых, Brain хранил свой код в скрытой области, как обычно делают современные буткиты. Во-вторых, он помечал заражённые секторы как повреждённые, защищая код от служебных операций ОС. В-третьих, он применял сокрытие: если вирус был активен при обращении к заражённому сектору, он перехватывал обработчик дискового прерывания, чтобы система вместо заражённого сектора показывала легитимный сектор загрузочного кода. Каждую из этих особенностей буткитов мы подробнее рассмотрим в нескольких следующих главах.

Эволюция буткитов

В этом разделе мы рассмотрим, как с развитием операционных систем сокращалось применение BSI. Затем мы разберём, как политика Microsoft по подписыванию кода режима ядра лишила эффективности прежние методы и побудила злоумышленников создавать новые способы заражения, а также как появление стандарта безопасности Secure Boot создало новые препятствия для современных буткитов.

Конец эпохи BSI

По мере усложнения операционных систем чистые BSI столкнулись с рядом трудностей. Новые версии ОС заменяли предоставляемые BIOS прерывания для связи с дисками драйверами, зависящими от ОС. В результате после загрузки ОС BSI больше не могли обращаться к прерываниям BIOS и потому не могли заражать другие диски в системе. Попытка выполнить прерывание BIOS в таких системах могла привести к непредсказуемому поведению.

По мере того как всё больше систем получали BIOS, способную загружаться с жёстких дисков, а не с дискет, заражённые дискеты становились менее эффективными и частота заражений BSI начала снижаться. Появление и рост популярности Microsoft Windows вместе с быстрым сокращением применения дискет нанесли окончательный удар BSI старой школы.

Политика подписывания кода режима ядра

Технология буткитов потребовала серьёзного пересмотра после введения в Windows Vista и последующих 64-разрядных версиях Windows политики Microsoft по подписыванию кода режима ядра. Новое требование к драйверам режима ядра изменило положение в пользу защитников. Начиная с Vista для выполнения каждого драйвера требовалась действительная цифровая подпись; неподписанные вредоносные драйверы режима ядра просто не загружались. Лишившись возможности внедрять код в ядро после полной загрузки ОС, злоумышленники были вынуждены искать способы обхода проверок целостности в современных компьютерных системах.

Все известные приёмы обхода проверки цифровой подписи Microsoft можно разделить на четыре группы, показанные на рисунке 4-1.

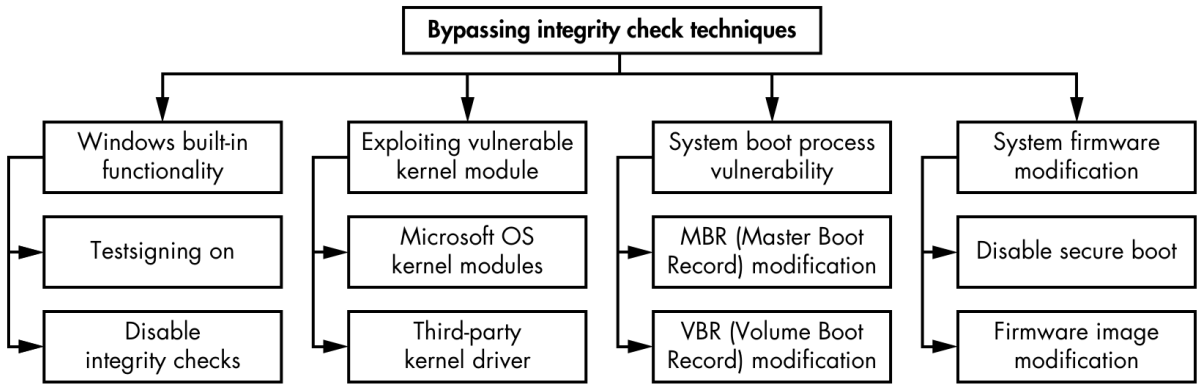


Рисунок 4-1. Методы обхода политики подписывания кода режима ядра. Четыре группы: встроенные возможности Windows, включая тестовое подписывание и отключение проверок целостности; эксплуатация уязвимого модуля ядра Microsoft или стороннего драйвера; уязвимость процесса загрузки с изменением MBR или VBR; изменение системной прошивки с отключением Secure Boot или изменением образа прошивки

Первая группа действует исключительно в режиме пользователя и полагается на встроенные средства Microsoft Windows, позволяющие легитимно отключать политику подписывания для отладки и испытания драйверов. ОС предоставляет интерфейс для временного отключения проверки подлинности образа драйвера либо включения тестового подписывания, при котором цифровая подпись драйверов проверяется с помощью собственного сертификата.

Вторая группа пытается использовать уязвимость в ядре системы или легитимном стороннем драйвере с действительной цифровой подписью, что позволяет вредоносному ПО проникнуть в режим ядра.

Третья группа нацелена на загрузчик ОС, чтобы изменить ядро ОС и отключить политику подписывания кода режима ядра. Именно такой подход применяют новые буткиты. Они выполняются до загрузки любого компонента ОС и потому могут вмешаться в ядро ОС, отключив проверки безопасности. Этот метод мы подробно обсудим в следующей главе.

Четвёртая группа стремится скомпрометировать системную прошивку. Как и у третьей группы, её цель состоит в выполнении кода в целевой системе раньше ядра ОС, чтобы отключить проверки безопасности. Единственное существенное различие заключается в том, что такие атаки нацелены на прошивку, а не на компоненты загрузчика.

На практике чаще всего применяется третий метод, компрометация процесса загрузки, поскольку он обеспечивает более устойчивую атаку. В результате злоумышленники вернулись к старым приёмам BSI, создавая современные буткиты. Необходимость обходить проверки целостности в современных компьютерных системах оказала огромное влияние на развитие буткитов.

Распространение Secure Boot

В наши дни компьютеры всё чаще поставляются с работающей защитой Secure Boot. Secure Boot - стандарт безопасности, предназначенный для обеспечения целостности компонентов процесса загрузки. Мы подробнее рассмотрим его в главе 17. Столкнувшись с Secure Boot, мир вредоносного ПО вновь был вынужден измениться: вместо процесса загрузки современные вредоносные программы всё чаще пытаются атаковать системную прошивку.

Подобно тому как политика Microsoft по подписыванию кода режима ядра искоренила руткиты режима ядра и открыла новую эпоху буткитов, Secure Boot теперь создаёт препятствия для современных буткитов. Мы всё чаще видим, как современное вредоносное ПО атакует BIOS. Этот тип угроз рассматривается в главе 15.

Современные буткиты

В области буткитов, как и в других областях компьютерной безопасности, доказательства концепции (proofs of concept, PoC) и реальные образцы вредоносного ПО обычно развиваются совместно. В данном случае PoC представляет собой вредоносную программу, созданную исследователями безопасности для доказательства реальности угрозы, в отличие от вредоносного ПО киберпреступников, преследующих злонамеренные цели.

Первым современным буткитом обычно считают PoC BootRoot компании eEye, представленный на конференции Black Hat 2005 года в Лас-Вегасе. Код BootRoot, написанный Дереком Сёдером и Райаном Перме, представлял собой потайной вход спецификации интерфейса сетевых драйверов (NDIS). Он впервые продемонстрировал, что исходную концепцию буткита можно использовать как модель атаки на современные операционные системы.

Однако, хотя презентация eEye стала важным шагом к созданию вредоносных буткитов, до обнаружения в реальных атаках нового вредоносного образца с функциями буткита прошло два года. Им стал Mebroot в 2007 году. Mebroot, одна из самых сложных угроз того времени, создал серьёзные трудности антивирусным компаниям, поскольку применял новые методы сокрытия, позволявшие пережить перезагрузку.

Обнаружение Mebroot совпало с выпуском в том же году на конференции Black Hat двух важных PoC-буткитов, Vbootkit и Stoned. Код Vbootkit показал, что ядро Microsoft Windows Vista можно атаковать посредством изменения загрузочного сектора. Авторы Vbootkit выпустили его код как проект с открытым исходным кодом. Буткит Stoned, также атаковавший ядро Vista, получил название в честь весьма успешного BSI Stoned, созданного несколькими десятилетиями ранее.

Выпуск обоих PoC сыграл важнейшую роль, показав отрасли безопасности, каких буткитов следует опасаться. Если бы исследователи не решились опубликовать результаты, авторы вредоносного ПО смогли бы опередить способность системы обнаруживать новые вредоносные буткиты. С другой стороны, как это часто бывает, авторы вредоносного ПО повторно использовали подходы из PoC, представленных исследователями безопасности, и вскоре после презентации PoC появилось новое вредоносное ПО в реальных атаках. Это совместное развитие показано на рисунке 4-2 и в таблице 4-1.

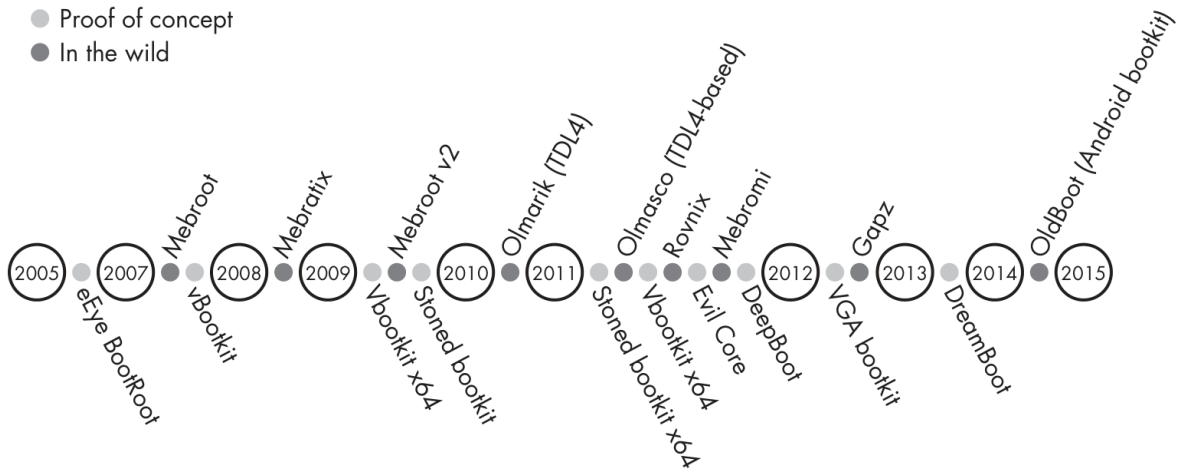


Рисунок 4-2. Хронология возрождения буткитов в 2005-2015 годах. Светлые метки обозначают доказательства концепции, тёмные - реальные угрозы. На временной шкале отмечены eEye BootRoot, Mebroot, Vbootkit, Mebratix, Vbootkit x64, Mebroot v2, Stoned, Olmarik (TDL4), Stoned x64, Olmasco, Rovnix, Mebromi, Evil Core, DeepBoot, VGA bootkit, Gapz, DreamBoot и OldBoot

Таблица 4-1. Развитие PoC-буткитов в сравнении с реальными угрозами-буткитами

Развитие PoC-буткитов	Развитие угроз-буткитов
eEye BootRoot (2005). Первый[2] буткит на основе MBR для операционных систем Microsoft Windows	**Mebroot (2007).** Первый широко известный современный буткит на основе MBR для операционных систем Microsoft Windows, встречавшийся в реальных атаках; буткиты на основе MBR подробно рассматриваются в главе 7
Vbootkit (2007). Первый буткит, атаковавший Microsoft Windows Vista	**Mebratix (2008).** Другое семейство вредоносного ПО, основанное на заражении MBR
Vbootkit x64 (2009).[3] Первый буткит, обошедший проверку цифровой подписи в Microsoft Windows 7	**Mebroot v2 (2009).** Развитая версия вредоносного ПО Mebroot
Stoned (2009). Ещё один пример заражения буткитом на основе MBR	**Olmarik (TDL4) (2010/11).** Первый 64-разрядный буткит в реальных атаках
Stoned x64 (2011). Буткит на основе MBR, поддерживающий заражение 64-разрядных операционных систем	**Olmasco (модификация TDL4) (2011).** Первое заражение буткитом на основе VBR
Evil Core (2011).[4] Концептуальный буткит, использовавший SMP (симметричную многопроцессорную обработку) для загрузки в защищённый режим	**Rovnix (2011).** Развитое заражение на основе VBR с полиморфным кодом
DeepBoot (2011).[5] Буткит, применявший интересные приёмы для перехода из реального режима в защищённый	**Mebromi (2011).** Первый замеченный в реальных атаках опыт практической реализации концепции BIOS-китов
VGA (2012).[6] Концепция буткита на основе VGA	**Gapz (2012).**[7] Следующий этап развития заражения VBR
DreamBoot (2013).[8] Первая общедоступная концепция UEFI-буткита	**OldBoot (2014).**[9] Первый буткит для ОС Android, замеченный в реальных атаках

Методы, применявшиеся этими буткитами, мы рассмотрим в следующих главах.

Заключение

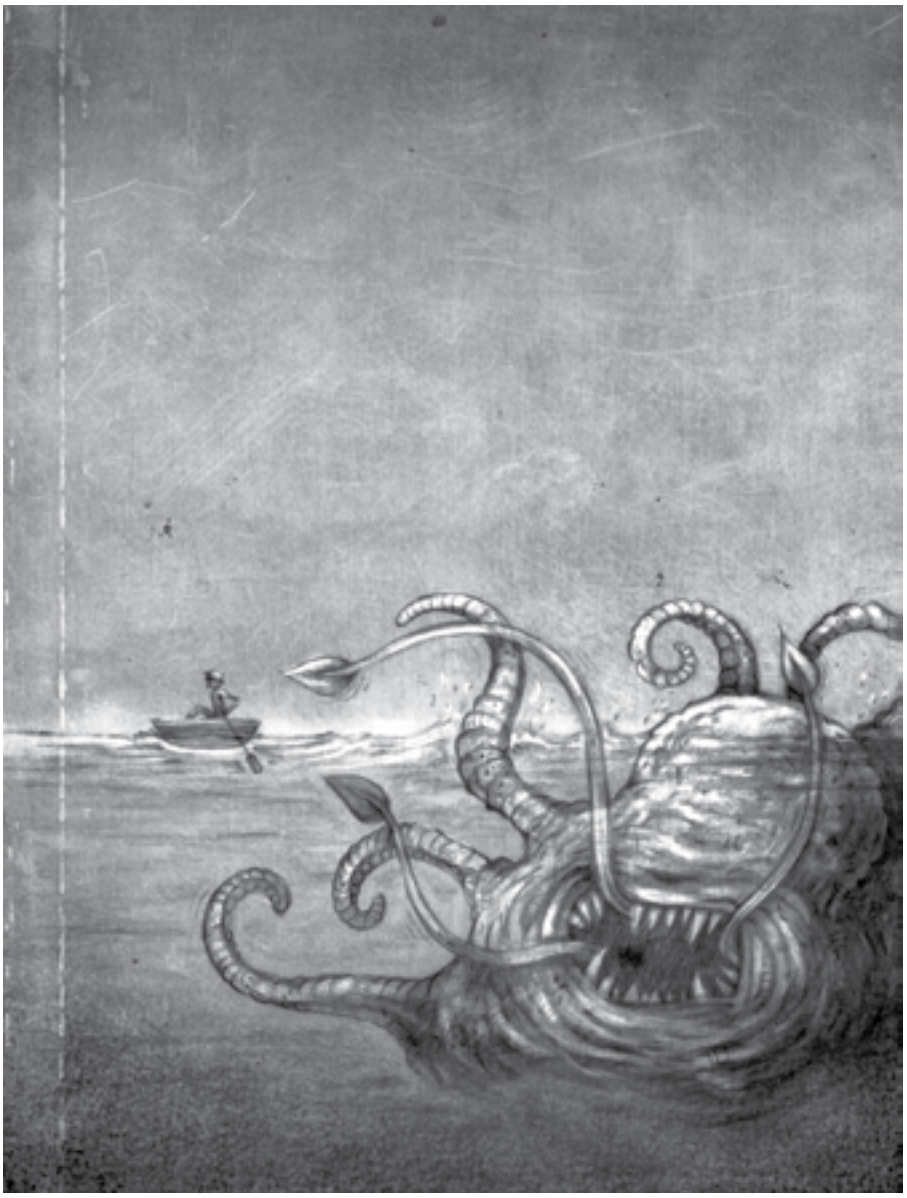
В этой главе обсуждались история и развитие компрометации процесса загрузки, что позволило составить общее представление о технологии буткитов. В главе 5 мы глубже рассмотрим политику подписывания кода режима ядра и изучим способы обхода этой технологии с помощью заражения буткитом, сосредоточившись на рутките TDSS. Развитие TDSS (также известного как TDL3) и буткита TDL4 наглядно иллюстрирует переход от руткитов режима ядра к буткитам как способу, позволяющему вредоносному ПО дольше сохраняться в скомпрометированной системе и оставаться необнаруженным.

На этой странице исходного издания нет основного текста.

Сноски

[1] [назад] Дэвид Харли, Роберт Слейд и Урс Э. Гаттикерд, *Viruses Revealed* (Нью-Йорк: McGraw-Hill/Osborne, 2001).
[2] [назад] Называя буткит «первым» в чём-либо, мы подразумеваем, что он является первым, насколько нам известно.
[3] [назад] Нитин Кумар и Витин Кумар, «VBootkit 2.0 - атака на Windows 7 через загрузочные секторы», HiTB 2009, conference.hitb.org.
[4] [назад] Вольфганг Эттлингер и Штефан Фибёк, «Буткит Evil Core», NinjaCon 2011, downloads.ninjacon.net.
[5] [назад] Николас А. Эконому и Андрес Лопес Луксенберг, «DeepBoot», Ekoparty 2011, ekoparty.org.
[6] [назад] Диего Хуарес и Николас А. Эконому, «Постоянный VGA-руткит», Ekoparty 2012, secureauth.com.
[7] [назад] Евгений Родионов и Александр Матросов, «Осторожно, Gapz: самый сложный из когда-либо проанализированных буткитов?», весна 2013 года, welivesecurity.com.
[8] [назад] Себастьян Качмарек, «UEFI и Dreamboot», HiTB 2013, conference.hitb.org.
[9] [назад] Цзыхан Сяо, Цин Дун, Хао Чжан и Сюсянь Цзян, «Oldboot: первый буткит для Android», blogs.360.cn.

Глава 5. Основы процесса загрузки операционной системы



В этой главе вы познакомитесь с наиболее важными для буткитов аспектами процесса загрузки Microsoft Windows. Поскольку цель буткита - скрыться в целевой системе на очень низком уровне, ему приходится вмешиваться в работу загрузочных компонентов ОС. Поэтому, прежде чем подробно разбирать устройство и поведение буткитов, необходимо понять, как работает процесс загрузки.

> **Примечание.** Сведения в этой главе относятся к Microsoft Windows Vista и более поздним версиям; процесс загрузки предыдущих версий Windows отличается, как объясняется в разделе «Модуль bootmgr и данные конфигурации загрузки» на странице 64 оригинального издания.

Процесс загрузки - один из самых важных и в то же время наименее понятных этапов работы операционной системы. Хотя его общая идея известна всем, лишь немногие программисты, в том числе системные, понимают его в подробностях, а у большинства нет и необходимых для этого

инструментов. В результате процесс загрузки становится благодатной почвой для злоумышленников, способных воспользоваться знаниями, полученными посредством обратной разработки и экспериментов, тогда как программистам зачастую приходится полагаться на неполную или устаревшую документацию.

С точки зрения безопасности процесс загрузки отвечает за запуск системы и приведение её в доверенное состояние. Логические средства, с помощью которых защитный код проверяет состояние системы, также создаются в ходе этого процесса, поэтому чем раньше злоумышленнику удастся скомпрометировать систему, тем проще ему будет скрыться от проверок защитного ПО.

В этой главе мы рассмотрим основы процесса загрузки Windows на компьютерах с устаревшей системной прошивкой. Процесс загрузки компьютеров с прошивкой UEFI, поддержка которой появилась в Windows 7 x64 SP1, существенно отличается от процесса на компьютерах с устаревшей прошивкой, поэтому он отдельно рассматривается в главе 14.

На протяжении всей главы мы будем смотреть на процесс загрузки с точки зрения злоумышленника. Ничто не мешает атакующему нацелиться на конкретный набор микросхем или периферийное устройство, и некоторые действительно так поступают, однако подобные атаки плохо масштабируются, а обеспечить надёжность их разработки трудно. Поэтому злоумышленнику выгоднее выбирать сравнительно универсальные интерфейсы, но не настолько универсальные, чтобы программистам защитного ПО было легко понять и проанализировать атаку.

Как всегда, наступательные исследования раздвигают границы и по мере того, как достижения становятся общедоступными и прозрачными, всё глубже проникают в систему. Это подчёркивает и структура главы: мы начнём с общего обзора, а затем перейдём к недокументированным на момент написания книги структурам данных и логическому потоку, которые можно установить лишь путём дизассемблирования системы. Именно таким путём идут как исследователи буткитов, так и создатели вредоносного ПО.

Общий обзор процесса загрузки Windows

На рисунке 5-1 показан общий ход современного процесса загрузки. Буткит способен атаковать почти любую его часть, но чаще всего мишенями становятся инициализация базовой системы ввода-вывода (BIOS), главная загрузочная запись (MBR) и загрузчик операционной системы.

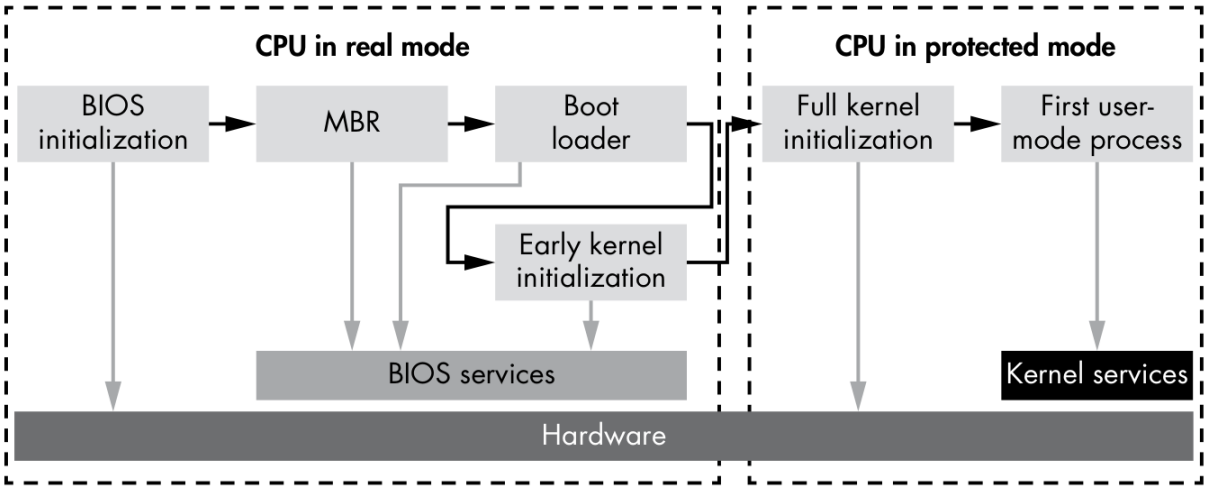


Рисунок 5-1. Ход процесса загрузки системы. Сначала процессор работает в реальном режиме: выполняются инициализация BIOS, MBR, загрузчик и ранняя инициализация ядра с использованием служб BIOS и оборудования. Затем процессор переходит в защищённый режим, где выполняются полная инициализация ядра и первый процесс пользовательского режима с использованием служб ядра

> **Примечание.** Технология Secure Boot, которую мы рассмотрим в главе 17, предназначена для защиты современного процесса загрузки, включая его сложные и многофункциональные компоненты UEFI.

По мере развития процесса загрузки среда выполнения усложняется, предлагая защитному ПО более богатые и привычные программные модели. Однако именно низкоуровневый код создаёт и поддерживает эти абстрактные модели, поэтому, атакуя его, злоумышленники могут воздействовать на модели, перехватывать ход загрузки и вмешиваться в состояние системы на более высоком уровне. Таким образом можно вывести из строя более абстрактные и мощные модели, а именно это и является назначением буткита.

Устаревший процесс загрузки

Чтобы понять технологию, полезно рассмотреть её предыдущие поколения. Ниже приведено краткое описание процесса загрузки в том виде, в каком он обычно выполнялся в эпоху расцвета вирусов загрузочного сектора (с 1980-х по 2000-е годы), таких как Brain, рассмотренный в главе 4:

1. Включение питания (холодная загрузка).
2. Самотестирование блока питания.

3. Выполнение BIOS из ПЗУ.
4. Проверка оборудования посредством BIOS из ПЗУ.
5. Проверка видеосистемы.
6. Проверка памяти.
7. Самотестирование при включении питания (Power-On Self-Test, POST), то есть полная проверка оборудования. Этот этап можно пропустить при горячей, или программной, загрузке, когда загрузка начинается из состояния, не являющегося полностью выключенным.
8. Поиск MBR в первом секторе загрузочного диска по умолчанию, указанного в настройках BIOS.
9. Выполнение MBR.
10. Инициализация файлов операционной системы.
11. Инициализация базовых драйверов устройств.
12. Проверка состояния устройств.
13. Чтение файлов конфигурации.
14. Загрузка командной оболочки.
15. Выполнение файла команд запуска оболочки.

Обратите внимание: ранняя стадия загрузки начинается с проверки и инициализации оборудования. Зачастую это верно и сегодня, хотя со времён Brain и его непосредственных преемников многие технологии оборудования и прошивок ушли далеко вперёд. Описанные далее в книге процессы загрузки отличаются от ранних вариантов терминологией и сложностью, но их общие принципы сходны.

Процесс загрузки Windows

На рисунке 5-2 в общих чертах показаны процесс загрузки Windows и участвующие в нём компоненты для Windows Vista и более поздних версий. Каждый блок на рисунке обозначает модуль, который выполняется и получает управление в ходе загрузки; порядок следования идёт сверху вниз. Как видите, эта схема довольно похожа на прежние варианты процесса загрузки. Однако с ростом сложности компонентов современных операционных систем Windows усложнились и модули, участвующие в загрузке.

В следующих разделах мы будем обращаться к этому рисунку, последовательно и более подробно разбирая процесс загрузки. Как показано на рисунке 5-2, при первом включении компьютера управление получает загрузочный код BIOS. С точки зрения программного обеспечения именно здесь начинается процесс загрузки; на уровне оборудования и прошивки выполняется и другая логика, например инициализация набора микросхем, но во время загрузки она программному обеспечению не видна.

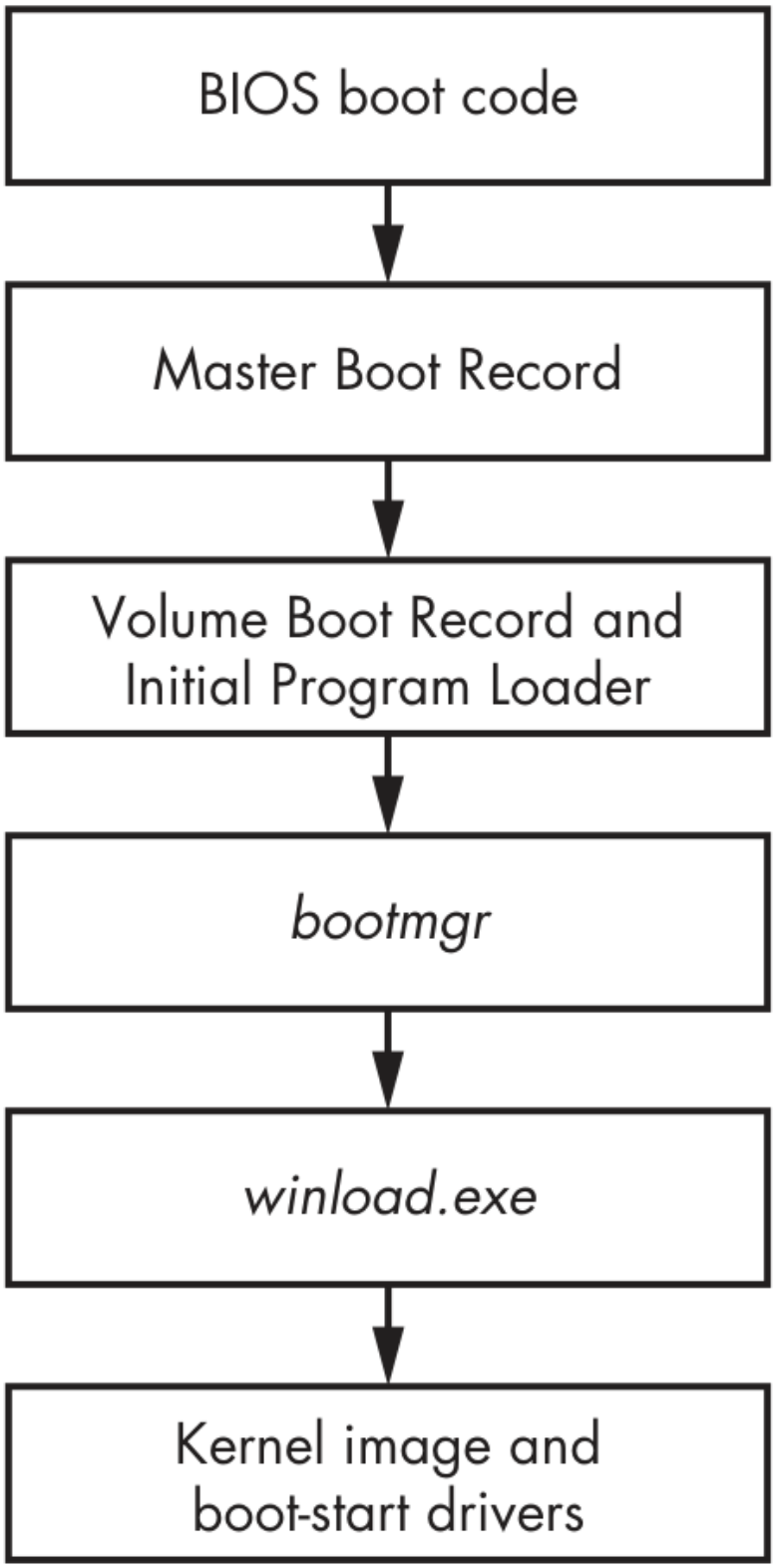


Рисунок 5-2. Общая схема процесса загрузки Windows: загрузочный код BIOS, главная загрузочная запись, загрузочная запись тома и начальный загрузчик программ, bootmgr, winload.exe, образ ядра и драйверы, запускаемые при загрузке

BIOS и предзагрузочная среда

BIOS выполняет базовую инициализацию системы и POST, чтобы убедиться в исправности критически важного системного оборудования. BIOS также предоставляет специализированную среду, включающую базовые службы, необходимые для взаимодействия с системными устройствами. Этот упрощённый интерфейс ввода-вывода впервые становится доступен в предзагрузочной среде, а позднее в большинстве сценариев Windows заменяется различными абстракциями операционной системы. С точки зрения анализа буткитов наиболее интересна дисковая служба, предоставляющая ряд точек входа для операций дискового ввода-вывода. Дисковая служба доступна через особый обработчик, известный как обработчик прерывания 13h, или просто INT 13h. Буткиты часто атакуют дисковую службу, вмешиваясь в работу INT 13h. Таким способом они пытаются отключить или обойти защиту ОС, изменяя компоненты операционной системы и загрузки, считываемые с жёсткого диска во время запуска системы.

Затем BIOS ищет загрузочный диск, на котором находится подлежащий загрузке экземпляр операционной системы. Это может быть жёсткий диск, USB-накопитель или компакт-диск. Определив загрузочное устройство, загрузочный код BIOS загружает MBR, как показано на рисунке 5-2.

Главная загрузочная запись

MBR - это структура данных, содержащая сведения о разделах жёсткого диска и загрузочный код. Её главная задача - определить активный раздел загрузочного жёсткого диска, содержащий подлежащий загрузке экземпляр ОС. Определив активный раздел, MBR считывает и выполняет его загрузочный код. В листинге 5-1 показана структура MBR.

```
typedef struct _MASTER_BOOT_RECORD{
  ❶ BYTE bootCode[0x1BE];  // место для фактического загрузочного кода
  ❷ MBR_PARTITION_TABLE_ENTRY partitionTable[4];
  USHORT mbrSignature;  // значение 0xAA55 обозначает формат MBR для ПК
} MASTER_BOOT_RECORD, *PMASTER_BOOT_RECORD;
```

Листинг 5-1. Структура MBR

Как видите, размер загрузочного кода MBR ❶ ограничен всего 446 байтами (0x1BE в шестнадцатеричном представлении - это значение хорошо знакомо специалистам по обратной разработке загрузочного кода), поэтому в нём можно реализовать только базовую функциональность. Затем MBR разбирает таблицу разделов, показанную под номером ❷, чтобы найти активный раздел, считывает загрузочную запись тома (Volume Boot Record, VBR) из его первого сектора и передаёт ей управление.

Таблица разделов

Таблица разделов в MBR представляет собой массив из четырёх элементов, каждый из которых описывается структурой `MBR_PARTITION_TABLE_ENTRY`, показанной в листинге 5-2.

```
typedef struct _MBR_PARTITION_TABLE_ENTRY {
   ❶ BYTE status;          // активен? 0 = нет, 128 = да
   BYTE chsFirst[3];       // номер начального сектора
   ❷ BYTE type;            // код типа ОС
   BYTE chsLast[3];        // номер конечного сектора
   ❸ DWORD lbaStart;       // первый сектор относительно начала диска
   DWORD size;             // количество секторов в разделе
} MBR_PARTITION_TABLE_ENTRY, *PMBR_PARTITION_TABLE_ENTRY;
```

Листинг 5-2. Структура элемента таблицы разделов

Первый байт ❶ структуры `MBR_PARTITION_TABLE_ENTRY`, поле `status`, указывает, является ли раздел активным. В каждый момент времени активным может быть помечен лишь один раздел; это состояние обозначается значением 128 (0x80 в шестнадцатеричном представлении).

В поле `type` ❷ указан тип раздела. Наиболее распространены следующие типы:

• Расширенный раздел MBR.
• Файловая система FAT12.
• Файловая система FAT16.
• Файловая система FAT32.
• IFS (Installable File System, устанавливаемая файловая система, используемая в процессе установки).
• LDM (Logical Disk Manager, диспетчер логических дисков Microsoft Windows NT).
• NTFS (основная файловая система Windows).

Тип 0 означает, что раздел не используется. Поля `lbaStart` и `size` ❸ определяют расположение раздела на диске, выраженное в секторах. Поле `lbaStart` содержит смещение раздела от начала жёсткого диска, а поле `size` - размер раздела.

Схема диска Microsoft Windows

На рисунке 5-3 показана типичная схема загрузочного жёсткого диска системы Microsoft Windows с двумя разделами.

Раздел Bootmgr содержит модуль `bootmgr` и некоторые другие загрузочные компоненты ОС, а раздел ОС - том с операционной системой и пользовательскими данными. Основное назначение модуля `bootmgr` - определить, какой именно экземпляр ОС следует загрузить. Если на компьютере установлено несколько операционных систем, `bootmgr` выводит диалоговое окно, предлагающее пользователю выбрать одну из них. Модуль `bootmgr` также предоставляет параметры, определяющие способ загрузки ОС: в безопасном режиме, с последней удачной конфигурацией, с отключённой обязательной проверкой подписи драйверов и так далее.

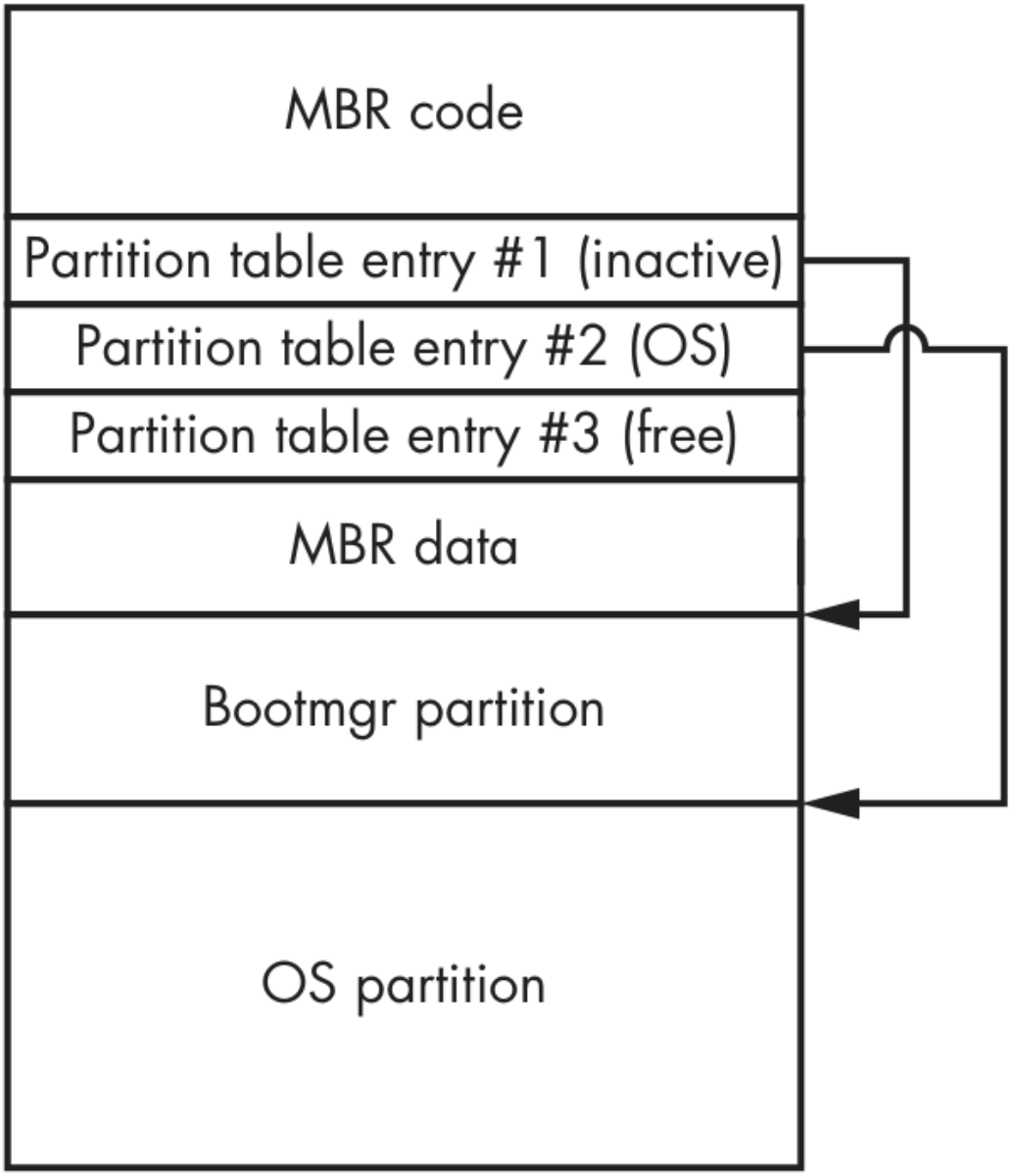


Рисунок 5-3. Типичная схема загрузочного жёсткого диска: код и данные MBR, три элемента таблицы разделов, раздел Bootmgr и раздел ОС

Загрузочная запись тома и начальный загрузчик программ

Жёсткий диск может содержать несколько разделов с несколькими экземплярами разных операционных систем, однако в нормальных условиях активным должен быть помечен только один раздел. В MBR нет кода для разбора конкретной файловой системы активного раздела, поэтому MBR считывает и выполняет первый сектор этого раздела - VBR, показанную на третьем уровне рисунка 5-2.

VBR содержит сведения о схеме раздела, в которых указаны тип используемой файловой системы и её параметры, а также код, считывающий из активного раздела модуль начального загрузчика программ (Initial Program Loader, IPL). Модуль IPL реализует функции разбора файловой системы, позволяющие читать файлы из файловой системы раздела.

В листинге 5-3 показана схема VBR, состоящая из структур `BIOS_PARAMETER_BLOCK_NTFS` и `BOOTSTRAP_CODE`. Схема структуры `BIOS_PARAMETER_BLOCK` (BPB) зависит от файловой системы тома. Структуры `BIOS_PARAMETER_BLOCK_NTFS` и `VOLUME_BOOT_RECORD` соответствуют тому NTFS.

```
typedef struct _BIOS_PARAMETER_BLOCK_NTFS {
   WORD SectorSize;
   BYTE SectorsPerCluster;

   WORD ReservedSectors;
   BYTE Reserved[5];
   BYTE MediaId;
   BYTE Reserved2[2];
   WORD SectorsPerTrack;
   WORD NumberOfHeads;
   ❶ DWORD HiddenSectors;
   BYTE Reserved3[8];
   QWORD NumberOfSectors;
   QWORD MFTStartingCluster;
   QWORD MFTMirrorStartingCluster;
   BYTE ClusterPerFileRecord;
   BYTE Reserved4[3];
   BYTE ClusterPerIndexBuffer;
   BYTE Reserved5[3];
   QWORD NTFSSerial;
   BYTE Reserved6[4];
} BIOS_PARAMETER_BLOCK_NTFS, *PBIOS_PARAMETER_BLOCK_NTFS;

typedef struct _BOOTSTRAP_CODE{
    BYTE    bootCode[420];               // машинный код загрузочного сектора
    WORD    bootSectorSignature;         // 0x55AA
} BOOTSTRAP_CODE, *PBOOTSTRAP_CODE;

typedef struct _VOLUME_BOOT_RECORD{
    ❷ WORD    jmp;
    BYTE    nop;
    DWORD   OEM_Name
    DWORD   OEM_ID; // NTFS
    BIOS_PARAMETER_BLOCK_NTFS BPB;
    BOOTSTRAP_CODE BootStrap;
} VOLUME_BOOT_RECORD, *PVOLUME_BOOT_RECORD;
```

Листинг 5-3. Схема VBR

Обратите внимание, что VBR начинается с инструкции `jmp` ❷, передающей управление системой коду VBR. В свою очередь, код VBR считывает из раздела IPL и выполняет его; местоположение IPL задаётся полем `HiddenSectors` ❶. IPL указывает своё смещение от начала жёсткого диска в секторах. Схема VBR представлена на рисунке 5-4.

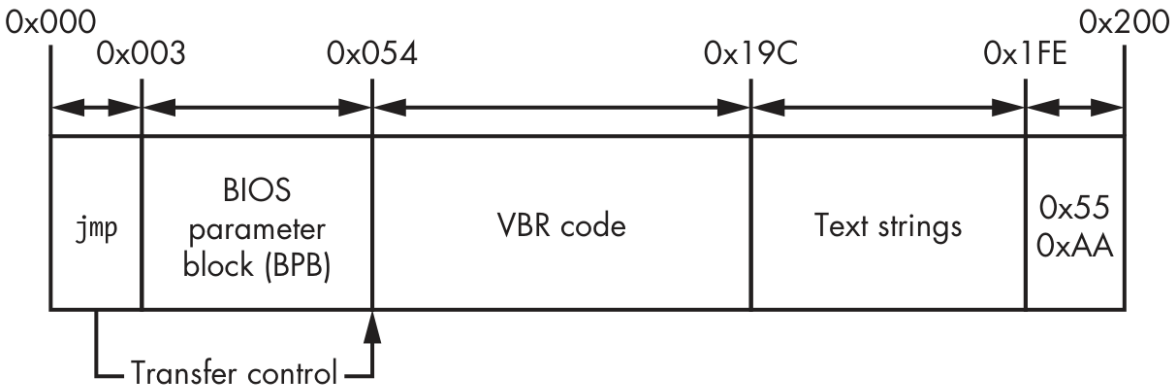


Рисунок 5-4. Структура VBR: инструкция jmp передаёт управление через блок параметров BIOS к коду VBR; далее следуют текстовые строки и двухбайтовая сигнатура 0x55AA

Как видите, VBR по существу состоит из следующих компонентов:

• Код VBR, отвечающий за загрузку IPL.
• Блок параметров BIOS - структура данных, в которой хранятся параметры тома.
• Текстовые строки, показываемые пользователю при возникновении ошибки.
• Значение 0xAA55 - двухбайтовая сигнатура VBR.

IPL обычно занимает 15 последовательных секторов по 512 байт каждый и расположен непосредственно после VBR. В нём реализован минимально необходимый код для разбора файловой системы раздела и продолжения загрузки модуля `bootmgr`. IPL и VBR используются совместно, поскольку VBR может занимать лишь один сектор, а столь малого объёма недостаточно для реализации функций разбора файловой системы тома.

Модуль bootmgr и данные конфигурации загрузки

IPL считывает из файловой системы и загружает модуль диспетчера загрузки ОС `bootmgr`, показанный на четвёртом уровне рисунка 5-2. После выполнения IPL управление процессом загрузки принимает `bootmgr`.

Модуль `bootmgr` считывает данные конфигурации загрузки (Boot Configuration Data, BCD), содержащие несколько важных параметров системы, в том числе влияющих на политики безопасности, такие как политика подписывания кода режима ядра, рассматриваемая в главе 6. Буткиты часто пытаются обойти реализованную в `bootmgr` проверку целостности кода.

Происхождение модуля bootmgr

Модуль `bootmgr` появился в Windows Vista, заменив загрузчик `ntldr`, применявшийся в предыдущих версиях Windows семейства NT. Замысел Microsoft состоял в добавлении ещё одного уровня абстракции в цепочку загрузки, чтобы изолировать предзагрузочную среду от уровня ядра ОС. Изоляция загрузочных модулей от ядра ОС улучшила управление загрузкой и безопасность Windows, упростив применение политик безопасности, установленных для модулей режима ядра, например политики подписывания кода режима ядра.

Устаревший модуль `ntldr` был разделён на два: `bootmgr` и `winload.exe` либо `winresume.exe`, если ОС загружается из состояния гибернации. Каждый модуль реализует собственную функциональность.

Модуль `bootmgr` управляет процессом загрузки, пока пользователь не выберет вариант загрузки, как показано на рисунке 5-5 для Windows 10. После выбора пользователя программа `winload.exe` либо `winresume.exe` загружает ядро, запускаемые при загрузке драйверы и некоторые данные системного реестра.

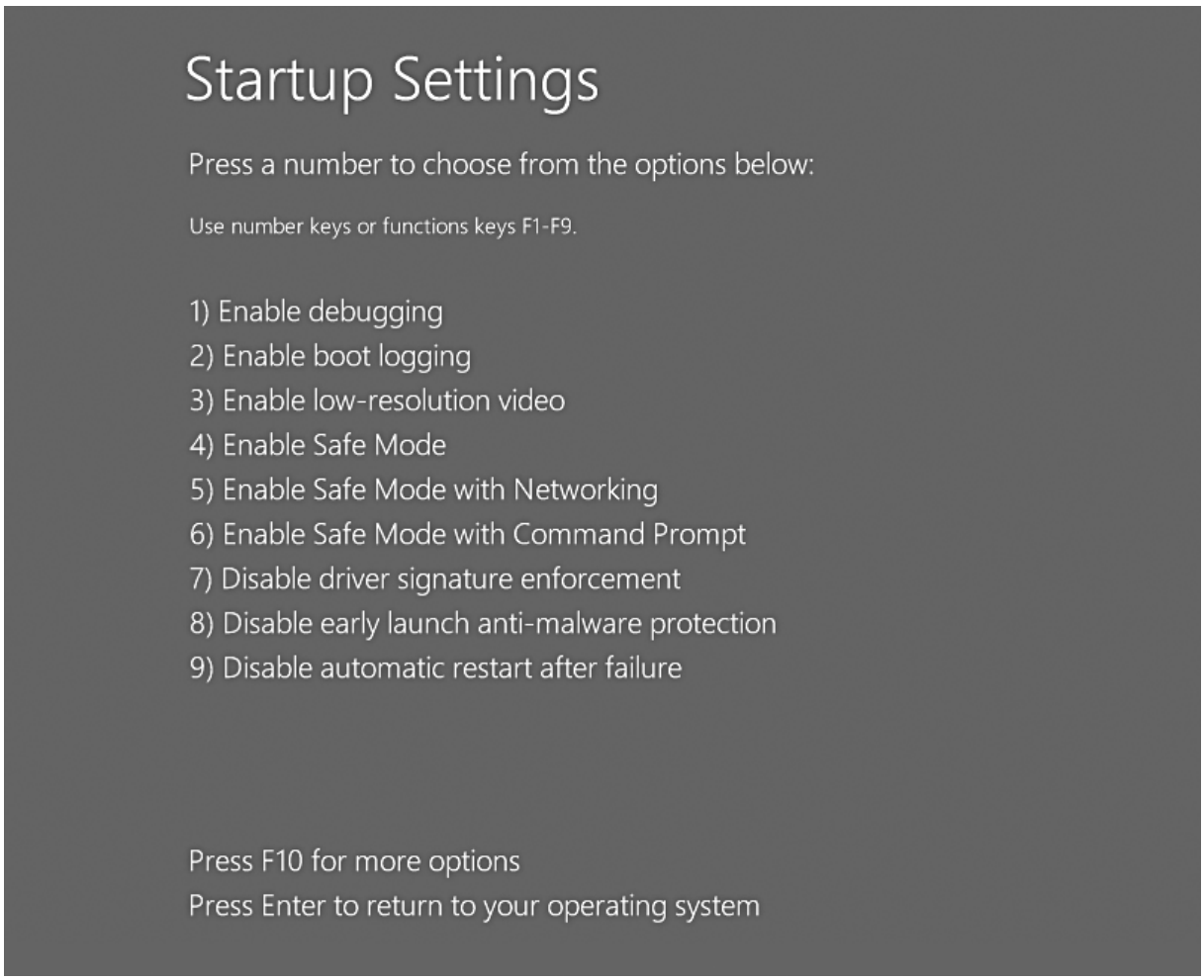


Рисунок 5-5. Меню загрузки bootmgr в Windows 10

Реальный режим и защищённый режим

Сразу после включения компьютера процессор работает в реальном режиме - унаследованном режиме выполнения с 16-разрядной моделью памяти, в которой каждый байт оперативной памяти адресуется указателем из двух слов по 2 байта: `segment_start:segment_offset`. Этот режим соответствует сегментной модели памяти, где адресное пространство разделено на сегменты. Адрес каждого целевого байта описывается адресом сегмента и смещением целевого байта внутри него. Здесь `segment_start` задаёт целевой сегмент, а `segment_offset` - смещение адресуемого байта в целевом сегменте.

Схема адресации реального режима позволяет использовать лишь небольшую часть доступной оперативной памяти системы. В частности, реальный, то есть физический, адрес в памяти вычисляется из наибольшего адреса, представленного как `ffff:ffff`, и составляет всего 1 114 095 байт (65 535 x 16 + 65 535). Следовательно, адресное пространство реального режима ограничено приблизительно 1 Мбайт, чего явно недостаточно современным операционным системам и приложениям. Чтобы обойти это ограничение и получить доступ ко всей имеющейся памяти, после перехода управления к `bootmgr` модули `bootmgr` и `winload.exe` переводят процессор в защищённый режим, который в 64-разрядных системах называется длинным режимом.

Модуль `bootmgr` состоит из 16-разрядного кода реального режима и сжатого PE-образа, который после распаковки выполняется в защищённом режиме. 16-разрядный код извлекает и распаковывает PE из образа `bootmgr`, переводит процессор в защищённый режим и передаёт управление распакованному модулю.

> **Примечание.** Чтобы сохранить управление выполнением загрузочного кода, буткиты должны правильно обрабатывать переключение режима выполнения процессора. После переключения вся схема памяти изменяется, и части кода, ранее располагавшиеся в одном непрерывном диапазоне адресов памяти, могут быть перемещены в другие сегменты памяти. Для обхода этого механизма и сохранения контроля над процессом загрузки буткитам приходится реализовывать весьма сложную функциональность.

Загрузочные переменные BCD

После инициализации защищённого режима модулем `bootmgr` управление получает распакованный образ, который загружает из BCD сведения о конфигурации загрузки. На жёстком диске BCD имеет ту же структуру, что и куст реестра. Чтобы просмотреть его содержимое, запустите `regedit` и перейдите к разделу `HKEY_LOCAL_MACHINE\BCD000000`.

> **Примечание.** Для чтения с жёсткого диска работающий в защищённом режиме `bootmgr` использует дисковую службу INT 13h, предназначенную для выполнения в реальном режиме. Для этого `bootmgr` сохраняет контекст выполнения процессора во временных переменных, временно переключается в реальный режим, выполняет обработчик INT 13h, затем возвращается в защищённый режим и восстанавливает сохранённый контекст.

Хранилище BCD содержит все сведения, необходимые `bootmgr` для загрузки ОС: путь к разделу с загружаемым экземпляром ОС, доступные загрузочные приложения, параметры целостности кода, а также параметры, предписывающие ОС загружаться в режиме предварительной установки, безопасном режиме и так далее.

В таблице 5-1 перечислены параметры BCD, представляющие наибольший интерес для авторов буткитов.

Таблица 5-1. Загрузочные переменные BCD

Имя переменной	Описание	Тип параметра	Идентификатор параметра
`BcdLibraryBoolean_DisableIntegrityCheck`	Отключает проверки целостности кода режима ядра	Boolean	`0x16000048`
`BcdOSLoaderBoolean_WinPEMode`	Предписывает ядру загружаться в режиме предварительной установки, побочным следствием чего становится отключение проверок целостности кода режима ядра	Boolean	`0x26000022`
`BcdLibraryBoolean_AllowPrereleaseSignatures`	Включает тестовое подписывание (`TESTSIGNING`)	Boolean	`0x1600004`

Переменная `BcdLibraryBoolean_DisableIntegrityCheck` используется для отключения проверок целостности и разрешения загрузки неподписанных драйверов режима ядра. В Windows 7 и более поздних версиях этот параметр игнорируется; его невозможно установить, если включена Secure Boot, рассматриваемая в главе 17.

Переменная `BcdOSLoaderBoolean_WinPEMode` указывает, что систему следует запустить в режиме среды предустановки Windows. По существу, это минимальная операционная система Win32 с ограниченным набором служб, используемая главным образом для подготовки компьютера к установке Windows. В этом режиме также отключаются проверки целостности ядра, включая обязательную для 64-разрядных систем политику подписывания кода режима ядра.

Переменная `BcdLibraryBoolean_AllowPrereleaseSignatures` позволяет использовать тестовые сертификаты подписи кода для загрузки драйверов режима ядра в целях тестирования. Такие сертификаты можно создавать с помощью инструментов, входящих в Windows Driver Kit. Руткит Necurs применяет этот процесс для установки в систему вредоносного драйвера режима ядра, подписанного специально созданным сертификатом.

Получив параметры загрузки, `bootmgr` проверяет собственную целостность. Если проверка завершается неудачно, `bootmgr` прекращает загрузку системы и выводит сообщение об ошибке. Однако `bootmgr` не выполняет самопроверку целостности, если в BCD переменная `BcdLibraryBoolean_DisableIntegrityCheck` или `BcdOSLoaderBoolean_WinPEMode` имеет значение `TRUE`. Следовательно, если хотя бы одна из этих переменных равна `TRUE`, `bootmgr` не заметит внесённых в него вредоносным кодом изменений.

После загрузки всех необходимых параметров BCD и успешной самопроверки целостности `bootmgr` выбирает загружаемое приложение. При обычной загрузке ОС с жёсткого диска `bootmgr` выбирает `winload.exe`, а при возобновлении работы после гибернации - `winresume.exe`. Эти PE-модули отвечают за загрузку и инициализацию модулей ядра ОС. `bootmgr` таким же образом проверяет целостность загрузочного приложения и опять пропускает проверку, если `BcdLibraryBoolean_DisableIntegrityCheck` или `BcdOSLoaderBoolean_WinPEMode` имеет значение `TRUE`.

На заключительном этапе процесса загрузки, после того как пользователь выбрал определённый экземпляр ОС, `bootmgr` загружает `winload.exe`. После правильной инициализации всех модулей `winload.exe`, соответствующий пятому уровню рисунка 5-2, передаёт управление ядру ОС, которое продолжает процесс загрузки на шестом уровне. Подобно `bootmgr`, `winload.exe` проверяет целостность всех модулей, за которые отвечает. Многие буткиты пытаются обойти эти проверки,

чтобы внедрить вредоносный модуль в адресное пространство режима ядра операционной системы.

Получив управление загрузкой операционной системы, `winload.exe` включает страничную организацию памяти в защищённом режиме, а затем загружает образ ядра ОС и его зависимости, включая следующие модули:

• `bootvid.dll` - библиотека поддержки видео VGA во время загрузки.
• `ci.dll` - библиотека целостности кода.
• `clfs.dll` - драйвер общей файловой системы журналирования.
• `hal.dll` - библиотека уровня абстрагирования оборудования.
• `kdcom.dll` - библиотека связи по протоколу отладчика ядра.
• `pshed.dll` - драйвер аппаратных ошибок, зависящий от платформы.

Помимо этих модулей, `winload.exe` загружает запускаемые при загрузке драйверы, в том числе драйверы запоминающих устройств, модули раннего запуска антивредоносного ПО (Early Launch Anti-Malware, ELAM), описанные в главе 6, и куст системного реестра.

> **Примечание.** Для чтения всех компонентов с жёсткого диска `winload.exe` использует интерфейс, предоставляемый `bootmgr`. Этот интерфейс опирается на дисковую службу BIOS INT 13h. Поэтому, если обработчик INT 13h перехвачен буткитом, вредоносная программа может подменять все данные, считываемые `winload.exe`.

При загрузке исполняемых файлов `winload.exe` проверяет их целостность в соответствии с системной политикой целостности кода. Загрузив все модули, `winload.exe` передаёт управление образу ядра ОС для их инициализации, как рассматривается в следующих главах.

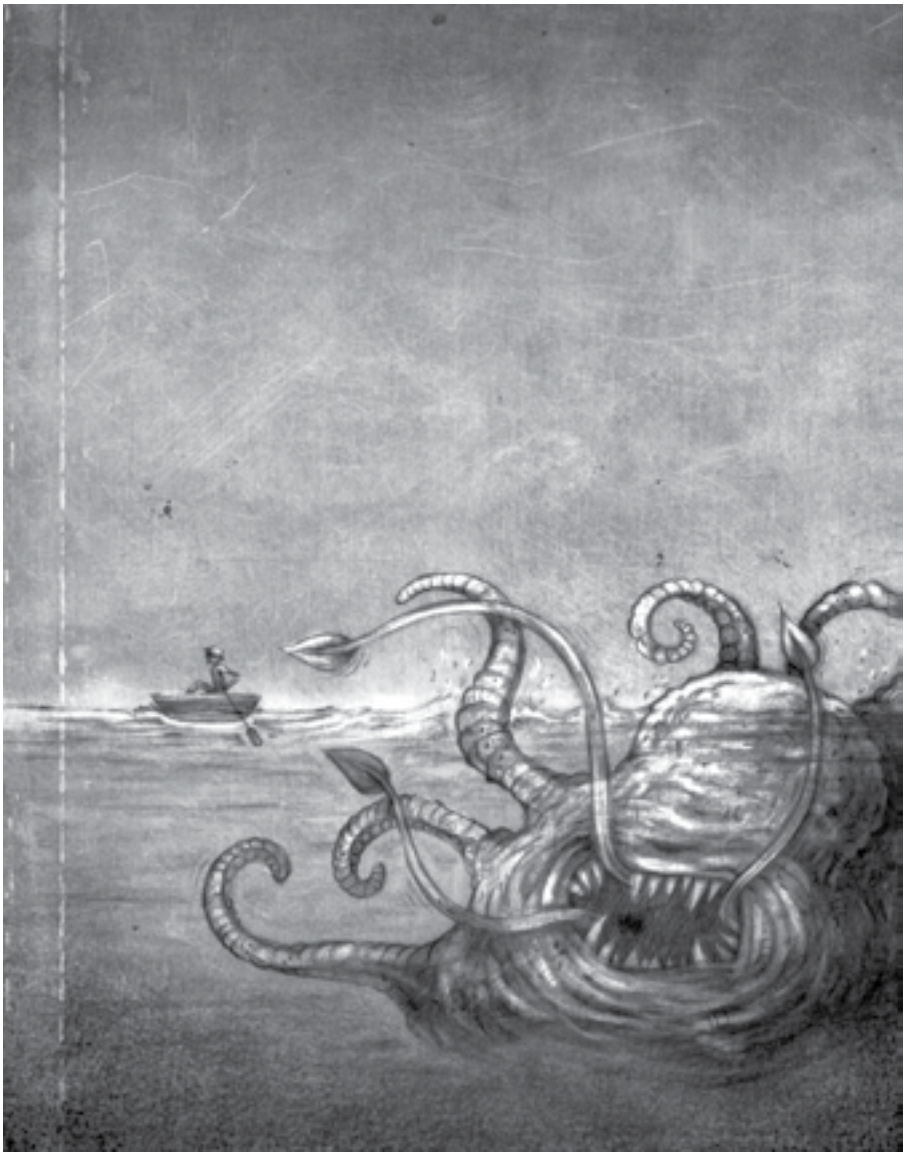
Заключение

В этой главе вы познакомились с MBR и VBR на ранних стадиях загрузки, а также с такими важными загрузочными компонентами, как `bootmgr` и `winload.exe`, с точки зрения создаваемых буткитами угроз.

Как вы увидели, передача управления между этапами загрузки не сводится к непосредственному переходу на следующий этап. Вместо этого ход выполнения в предзагрузочной среде определяется несколькими компонентами, связанными между собой различными структурами данных, такими как таблица разделов MBR, блок параметров BIOS в VBR и BCD. Эта нетривиальная взаимосвязь - одна из причин сложности буткитов и того, почему им приходится вносить так много изменений в загрузочные компоненты, чтобы передавать управление от исходного загрузочного кода собственному, а иногда и многократно переключаться между ними для выполнения необходимых задач.

В следующей главе мы рассмотрим безопасность процесса загрузки, сосредоточившись на ELAM и политике Microsoft по подписыванию кода режима ядра, которая сделала методы ранних руткитов неэффективными.

Глава 6. Безопасность процесса загрузки



В этой главе мы рассмотрим два важных механизма безопасности, реализованных в ядре Microsoft Windows: модуль раннего запуска антивредоносного ПО (Early Launch Anti-Malware, ELAM), появившийся в Windows 8, и политику подписывания кода режима ядра, введённую в Windows Vista. Оба механизма предназначены для предотвращения выполнения несанкционированного кода в адресном пространстве ядра, чтобы усложнить руткитам компрометацию системы. Мы разберём реализацию этих механизмов, обсудим их преимущества и слабые места, а также оценим их эффективность против руткитов и буткитов.

Модуль раннего запуска антивредоносного ПО

Модуль раннего запуска антивредоносного ПО (ELAM) - это механизм обнаружения в системах Windows, позволяющий стороннему защитному программному обеспечению, например антивирусу, зарегистрировать драйвер режима ядра, который гарантированно выполняется на очень раннем этапе загрузки, до загрузки любого другого стороннего драйвера. Поэтому, когда злоумышленник пытается загрузить вредоносный компонент в адресное пространство ядра Windows, защитное ПО может исследовать этот вредоносный драйвер и предотвратить его загрузку, поскольку драйвер ELAM уже активен.

Процедуры обратного вызова API

Драйвер ELAM регистрирует процедуры обратного вызова, которые ядро использует для оценки данных в кусте системного реестра и драйверов, запускаемых при загрузке. Эти процедуры обнаруживают вредоносные данные и модули и не позволяют Windows загрузить и инициализировать их.

Ядро Windows регистрирует и отменяет регистрацию таких процедур обратного вызова с помощью следующих функций API:

• `CmRegisterCallbackEx` и `CmUnRegisterCallback` - регистрируют и отменяют регистрацию процедур обратного вызова для наблюдения за данными реестра.
• `IoRegisterBootDriverCallback` и `IoUnRegisterBootDriverCallback` - регистрируют и отменяют регистрацию процедур обратного вызова для драйверов, запускаемых при загрузке.

Эти процедуры обратного вызова используют прототип `EX_CALLBACK_FUNCTION`, показанный в листинге 6-1.

```
NTSTATUS EX_CALLBACK_FUNCTION(
   ❶ IN PVOID CallbackContext,
   ❷ IN PVOID Argument1,         // тип обратного вызова
   ❸ IN PVOID Argument2          // предоставленная системой структура контекста
);
```

Листинг 6-1. Прототип процедур обратного вызова ELAM

Параметр `CallbackContext` ❶ получает контекст от драйвера ELAM после того, как драйвер выполнил одну из упомянутых выше процедур регистрации обратного вызова. Контекст представляет собой указатель на буфер памяти с параметрами, специфичными для драйвера ELAM, к которым может обращаться любая процедура обратного вызова. Этот же указатель используется для хранения текущего состояния драйвера ELAM. Аргумент ❷ задаёт тип обратного вызова, который для запускаемых при загрузке драйверов может принимать одно из следующих значений:

• `BdCbStatusUpdate` - передаёт драйверу ELAM обновления состояния при загрузке зависимостей драйверов или драйверов, запускаемых при загрузке.
• `BdCbInitializeImage` - используется драйвером ELAM для классификации запускаемых при загрузке драйверов и их зависимостей.

Классификация драйверов, запускаемых при загрузке

Аргумент ❸ предоставляет сведения, с помощью которых операционная система классифицирует запускаемый при загрузке драйвер как заведомо исправный (драйвер, о легитимности и чистоте которого известно), неизвестный (драйвер, который ELAM не может классифицировать) или заведомо вредоносный (драйвер, о вредоносности которого известно).

К сожалению, при классификации драйвера ELAM вынужден основывать решение на ограниченных сведениях об его образе:

• Имя образа.
• Раздел реестра, в котором образ зарегистрирован как запускаемый при загрузке драйвер.
• Издатель и эмитент сертификата образа.
• Хеш образа и название алгоритма хеширования.
• Отпечаток сертификата и название алгоритма вычисления отпечатка.

Драйвер ELAM не получает базовый адрес образа и не может обратиться к его двоичному файлу на жёстком диске, поскольку стек драйверов запоминающего устройства ещё не инициализирован: загрузка системы не завершена. ELAM должен решать, какие драйверы загружать, опираясь исключительно на хеш образа и его сертификат и не имея возможности исследовать сам образ. Поэтому на этой стадии защита драйверов не слишком эффективна.

Политика ELAM

Windows решает, следует ли загружать заведомо вредоносные или неизвестные драйверы, исходя из политики ELAM, указанной в разделе реестра `HKLM\System\CurrentControlSet\Control\EarlyLaunch\DriverLoadPolicy`.

В таблице 6-1 перечислены значения политики ELAM, определяющие, какие драйверы разрешено загружать.

Таблица 6-1. Значения политики ELAM

Имя политики	Значение политики	Описание
`PNP_INITIALIZE_DRIVERS_DEFAULT`	`0x00`	Загружать только заведомо исправные драйверы
`PNP_INITIALIZE_UNKNOWN_DRIVERS`	`0x01`	Загружать только заведомо исправные и неизвестные драйверы
`PNP_INITIALIZE_BAD_CRITICAL_DRIVERS`	`0x03`	Загружать заведомо исправные и неизвестные драйверы, а также заведомо вредоносные критические драйверы. Это настройка по умолчанию
`PNP_INITIALIZE_BAD_DRIVERS`	`0x07`	Загружать все драйверы

Как видите, политика ELAM по умолчанию, `PNP_INITIALIZE_BAD_CRITICAL_DRIVERS`, разрешает загрузку вредоносных критических драйверов. Это означает, что если критический драйвер классифицирован ELAM как заведомо вредоносный, система всё равно загрузит его несмотря на классификацию. Причина такой политики заключается в том, что критические системные драйверы

являются неотъемлемой частью операционной системы, поэтому сбой при их инициализации сделает ОС незагружаемой: система не запустится, если все критические драйверы не будут успешно загружены и инициализированы. Таким образом, ради доступности и возможности обслуживания эта политика ELAM жертвует частью безопасности.

Однако данная политика не загружает заведомо вредоносные некритические драйверы, то есть драйверы, без которых операционная система всё ещё способна успешно загрузиться. В этом состоит главное различие между политиками `PNP_INITIALIZE_BAD_CRITICAL_DRIVERS` и `PNP_INITIALIZE_BAD_DRIVERS`: последняя разрешает загрузку всех драйверов, включая заведомо вредоносные некритические.

Как буткиты обходят ELAM

ELAM даёт защитному программному обеспечению преимущество в борьбе с руткитами, но не с буткитами, да и не предназначен для этого. ELAM способен наблюдать только за драйверами, загружаемыми штатным способом, тогда как большинство буткитов загружает драйверы режима ядра с помощью недокументированных возможностей операционной системы. Поэтому буткит может обойти принудительные меры безопасности и внедрить свой код в адресное пространство ядра, несмотря на ELAM. Кроме того, как показано на рисунке 6-1, вредоносный код буткита выполняется до инициализации ядра операционной системы и до загрузки любого драйвера режима ядра, включая ELAM. Иными словами, буткит способен обойти защиту ELAM.

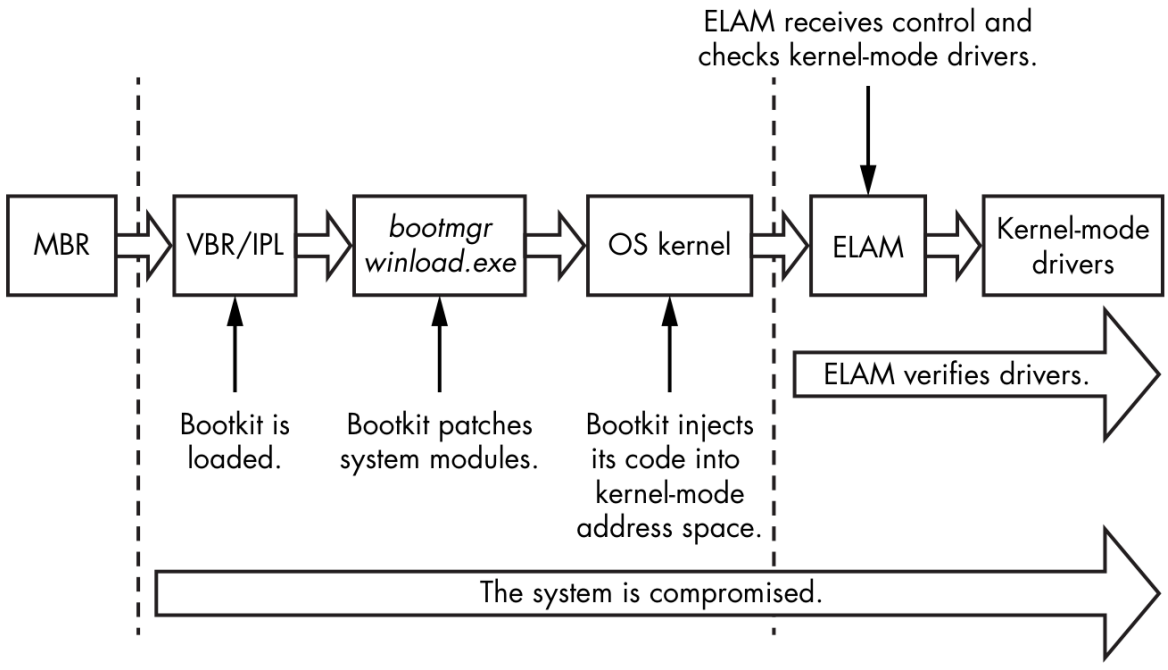


Рисунок 6-1. Ход процесса загрузки с ELAM. Буткит загружается после MBR, изменяет системные модули после VBR/IPL и bootmgr, а затем внедряет код в адресное пространство режима ядра. Лишь после этого ELAM получает управление и проверяет драйверы режима ядра; к этому моменту система уже скомпрометирована

Большинство буткитов загружает свой код режима ядра в середине инициализации ядра: после инициализации всех подсистем ОС, включая подсистему ввода-вывода, диспетчер объектов и диспетчер Plug and Play, но до выполнения ELAM. Разумеется, ELAM не может предотвратить выполнение вредоносного кода, загруженного раньше него, поэтому он бессилен против приёмов буткитов.

Политика Microsoft по подписыванию кода режима ядра

Политика подписывания кода режима ядра защищает операционную систему Windows, предъявляя требования к подписи кода модулей, предназначенных для загрузки в адресное пространство ядра. Эта политика значительно усложнила буткитам и руткитам компрометацию системы посредством выполнения драйверов режима ядра, тем самым подтолкнув разработчиков руткитов к переходу на методы буткитов. К сожалению, как объясняется далее в этой главе, злоумышленники могут отключить всю логику проверки подписи при загрузке, изменив несколько переменных, соответствующих параметрам конфигурации запуска.

Драйверы режима ядра, подлежащие проверкам целостности

Политика подписывания появилась в Windows Vista и применяется во всех последующих версиях Windows, хотя в 32- и 64-разрядных операционных системах она действует по-разному. Политика срабатывает при загрузке драйверов режима ядра и проверяет их целостность до отображения образов драйверов в адресное пространство ядра. В таблице 6-2 показано, какие драйверы режима ядра в 64- и 32-разрядных системах подлежат проверкам целостности.

Таблица 6-2. Требования политики подписывания кода режима ядра

Тип драйвера	Проверяется в 64-разрядной системе	Проверяется в 32-разрядной системе
Драйвер, запускаемый при загрузке	Да	Да
Драйвер PnP, не запускаемый при загрузке	Да	Нет
Драйвер не-PnP, не запускаемый при загрузке	Да	Нет, кроме драйверов, передающих защищённые мультимедийные данные

Как видно из таблицы, в 64-разрядных системах проверкам целостности подлежат все модули режима ядра независимо от их типа. В 32-разрядных системах политика подписывания применяется только к запускаемым при загрузке драйверам и драйверам мультимедиа; остальные драйверы не проверяются. Для установки устройств PnP действует требование проверки подписи во время установки.

Чтобы соответствовать требованиям целостности кода, драйвер должен иметь либо встроенную цифровую подпись сертификатом издателя программного обеспечения (Software Publisher Certificate, SPC), либо файл каталога с подписью SPC. Однако запускаемые при загрузке драйверы могут иметь только встроенную подпись, поскольку во время загрузки стек драйверов запоминающего устройства ещё не инициализирован и файлы каталогов драйверов недоступны.

Расположение подписей драйверов

Встроенная подпись драйвера в PE-файле, например запускаемом при загрузке драйвере, указывается в элементе `IMAGE_DIRECTORY_DATA_SECURITY` каталогов данных заголовка PE. Microsoft предоставляет функции API для перечисления всех содержащихся в образе сертификатов и получения сведений о них, как показано в листинге 6-2.

```
BOOL ImageEnumerateCertificates(
   _In_     HANDLE FileHandle,
   _In_     WORD TypeFilter,
   _Out_    PDWORD CertificateCount,
   _In_out_ PDWORD Indices,
   _In_opt_ DWORD IndexCount
);

BOOL ImageGetCertificateData(
   _In_    HANDLE FileHandle,
   _In_    DWORD CertificateIndex,
   _Out_   LPWIN_CERTIFICATE Certificate,
   _Inout_ PDWORD RequiredLength
);
```

Листинг 6-2. API Microsoft для перечисления и проверки сертификатов

Политика подписывания кода режима ядра повысила устойчивость безопасности системы, но у неё есть ограничения. В следующих разделах мы обсудим некоторые из этих недостатков и способы, которыми авторы вредоносного ПО пользовались ими для обхода защиты.

> **Политика подписывания при установке устройств Plug and Play**
>
> Помимо политики подписывания кода режима ядра, в Microsoft Windows действует ещё один вид политики подписывания: политика подписывания при установке устройств Plug and Play. Важно не путать эти две политики.
>
> Требования политики подписывания при установке устройств Plug and Play применяются только к драйверам устройств Plug and Play (PnP) и служат для проверки личности издателя и целостности установочного пакета драйвера PnP. Для прохождения проверки файл каталога пакета драйвера должен быть подписан либо сертификатом Windows Hardware Quality Labs (WHQL), либо сторонним SPC. Если пакет драйвера не соответствует требованиям политики PnP, диалоговое окно с предупреждением предлагает пользователю решить, следует ли разрешить установку пакета драйвера в системе.
>
> Системные администраторы могут отключить политику PnP, разрешив устанавливать в систему пакеты драйверов PnP без надлежащих подписей. Обратите также внимание, что эта политика применяется только при установке пакета драйвера, а не при загрузке драйверов. Хотя это может показаться слабостью типа TOCTOU (time of check to time of use, время проверки и время использования), ею не является: пакет драйвера PnP, успешно установленный в системе, не обязательно будет загружен, поскольку во время загрузки такие драйверы также проверяются политикой подписывания кода режима ядра.

Слабость прежнего механизма целостности кода

Логика политики подписывания кода режима ядра, обеспечивающая целостность кода, распределена между образом ядра Windows и библиотекой режима ядра `ci.dll`. Образ ядра использует эту библиотеку для проверки целостности всех модулей, загружаемых в адресное пространство ядра. Главная слабость процесса подписывания заключается в наличии единой точки отказа в этом коде.

В Microsoft Windows Vista и 7 в основе этого механизма лежит единственная переменная в образе ядра, определяющая, следует ли применять проверки целостности:

```
BOOL nt!g_CiEnabled
```

Эта переменная инициализируется во время загрузки в функции образа ядра `NTSTATUS SepInitializeCodeIntegrity()`. Операционная система проверяет, загружена ли она в режиме среды предустановки Windows (WinPE), и если да, инициализирует переменную `nt!g_CiEnabled` значением `FALSE` (`0x00`), отключающим проверки целостности.

Разумеется, злоумышленники обнаружили, что проверку целостности можно легко обойти, просто присвоив `nt!g_CiEnabled` значение `FALSE`. Именно это в 2011 году произошло с семейством вредоносных программ Uroburos, также известным как Snake и Turla. Uroburos обходил политику подписывания кода, сначала внедряя, а затем эксплуатируя уязвимость в стороннем драйвере. Легитимным подписанным сторонним драйвером был `VBoxDrv.sys`, драйвер VirtualBox. Получив возможность выполнять код в режиме ядра, эксплойт обнулял переменную `nt!g_CiEnabled`, после чего на атакованном компьютере можно было загрузить любой вредоносный неподписанный драйвер.

> **Уязвимость Linux**
>
> Такая слабость присуща не только Windows: злоумышленники сходным образом отключали принудительное управление доступом в SELinux. Если атакующему известен адрес переменной, содержащей состояние принудительного контроля SELinux, ему достаточно перезаписать её значение. Поскольку логика SELinux проверяет значение переменной до выполнения каких-либо проверок, после этого она сама становится неактивной. Подробный анализ уязвимости и код её эксплойта доступны по адресу grsecurity.net.

Если Windows загружена не в режиме WinPE, далее проверяются значения загрузочных параметров `DISABLE_INTEGRITY_CHECKS` и `TESTSIGNING`. Как следует из названия, `DISABLE_INTEGRITY_CHECKS` отключает проверки целостности. Пользователь любой версии Windows может вручную установить этот параметр во время загрузки с помощью пункта меню загрузки Disable Driver Signature Enforcement («Отключить обязательную проверку подписи драйверов»). Пользователи Windows Vista могут также с помощью `bcdedit.exe` присвоить параметру `nointegritychecks` значение `TRUE`; более поздние версии игнорируют этот параметр в данных конфигурации загрузки (BCD), когда включена Secure Boot. Подробнее о Secure Boot см. в главе 17.

Параметр `TESTSIGNING` изменяет способ проверки операционной системой целостности модулей режима ядра. Если ему присвоено значение `TRUE`, для проверки сертификата не требуется прослеживать всю цепочку до доверенного корневого центра сертификации (certificate authority, CA). Иными словами, в адресное пространство ядра можно загрузить любой драйвер с любой цифровой подписью. Руткит Necurs злоупотребляет параметром `TESTSIGNING`, присваивая ему значение `TRUE` и загружая свой драйвер режима ядра, подписанный специально созданным сертификатом.

В течение многих лет существовали ошибки браузеров, из-за которых при проверке цепочек доверия сертификатов X.509 не прослеживались промежуточные звенья до легитимного доверенного CA,[1] но и схемы подписывания модулей ОС по-прежнему не избегают упрощений в вопросах цепочек доверия.

Модуль ci.dll

Библиотека режима ядра `ci.dll`, отвечающая за применение политики целостности кода, содержит следующие функции:

• `CiCheckSignedFile` - проверяет дайджест и цифровую подпись.
• `CiFindPageHashesInCatalog` - проверяет, содержит ли проверенный системный каталог дайджест первой страницы памяти PE-образа.
• `CiFindPageHashesInSignedFile` - проверяет дайджест и цифровую подпись первой страницы памяти PE-образа.
• `CiFreePolicyInfo` - освобождает память, выделенную функциями `CiVerifyHashInCatalog`, `CiCheckSignedFile`, `CiFindPageHashesInCatalog` и `CiFindPageHashesInSignedFile`.
• `CiGetPEInformation` - создаёт зашифрованный канал связи между вызывающей стороной и модулем `ci.dll`.
• `CiInitialize` - инициализирует способность `ci.dll` проверять целостность файлов PE-образов.
• `CiVerifyHashInCatalog` - проверяет дайджест PE-образа, содержащийся в проверенном системном каталоге.

Для наших целей наиболее важна функция `CiInitialize`, поскольку она инициализирует библиотеку и создаёт её контекст данных. Соответствующий Windows 7 прототип показан в листинге 6-3.

```
NTSTATUS CiInitialize(
   ❶ IN ULONG CiOptions;
   PVOID Parameters;
   ❷ OUT PVOID g_CiCallbacks;
);
```

Листинг 6-3. Прототип функции CiInitialize

Функция `CiInitialize` получает в качестве параметров параметры целостности кода (`CiOptions`) ❶ и указатель на массив процедур обратного вызова (`OUT PVOID g_CiCallbacks`) ❷, который она при возврате заполняет адресами функций. Ядро использует эти процедуры обратного вызова для проверки целостности модулей режима ядра.

Функция `CiInitialize` также выполняет самопроверку, чтобы убедиться, что никто не внёс в неё изменения. Затем она проверяет целостность всех драйверов в списке загрузочных драйверов, который по существу содержит запускаемые при загрузке драйверы и их зависимости.

После завершения инициализации библиотеки `ci.dll` ядро использует процедуры обратного вызова из буфера `g_CiCallbacks` для проверки целостности модулей. В Windows Vista и 7, но не в Windows 8, функция `SeValidateImageHeader` решает, проходит ли конкретный образ проверку целостности. В листинге 6-4 показан лежащий в основе этой функции алгоритм.

```
NTSTATUS SeValidateImageHeader(Parameters) {
   NTSTATUS Status = STATUS_SUCCESS;
   VOID Buffer = NULL;
   ❶ if (g_CiEnabled == TRUE) {
         if (g_CiCallbacks[0] != NULL)
            ❷ Status = g_CiCallbacks[0](Parameters);
         else
            Status = 0xC0000428
   }
   else {
         ❸ Buffer = ExAllocatePoolWithTag(PagedPool, 1, 'hPeS');
         *Parameters = Buffer
         if (Buffer == NULL)
            Status = STATUS_NO_MEMORY;
   }
   return Status;
}
```

Листинг 6-4. Псевдокод функции SeValidateImageHeader

`SeValidateImageHeader` проверяет, имеет ли переменная `nt!g_CiEnabled` значение `TRUE` ❶. Если нет, функция пытается выделить буфер длиной один байт ❸ и в случае успеха возвращает значение `STATUS_SUCCESS`.

Если `nt!g_CiEnabled` равна `TRUE`, `SeValidateImageHeader` выполняет первую процедуру обратного вызова в буфере `g_CiCallbacks`, то есть `g_CiCallbacks[0]` ❷, которой назначена функция `CiValidateImageData`. Последняя проверяет целостность загружаемого образа.

Защитные изменения в Windows 8

В Windows 8 Microsoft внесла несколько изменений, призванных ограничить возможные в этом сценарии атаки. Во-первых, Microsoft объявила переменную ядра `nt!g_CiEnabled` устаревшей, устранив единую точку управления политикой целостности в образе ядра, которая существовала в предыдущих версиях Windows. В Windows 8 также изменилась структура буфера `g_CiCallbacks`.

Листинг 6-5 для Windows 7 и Vista и листинг 6-6 для Windows 8 показывают, как различается структура `g_CiCallbacks` в этих версиях ОС.

```
typedef struct _CI_CALLBACKS_WIN7_VISTA {
    PVOID CiValidateImageHeader;
    PVOID CiValidateImageData;

    PVOID CiQueryInformation;
} CI_CALLBACKS_WIN7_VISTA, *PCI_CALLBACKS_WIN7_VISTA;
```

Листинг 6-5. Структура буфера g_CiCallbacks в Windows Vista и Windows 7

Как видно из листинга 6-5, структура для Windows Vista и Windows 7 содержит лишь необходимый минимум. В структуре для Windows 8, показанной в листинге 6-6, напротив, больше полей для дополнительных процедур обратного вызова, проверяющих цифровую подпись PE-образа.

```
typedef struct _CI_CALLBACKS_WIN8 {
    ULONG ulSize;
    PVOID CiSetFileCache;
    PVOID CiGetFileCache;
    ❶ PVOID CiQueryInformation;
    ❷ PVOID CiValidateImageHeader;
    ❸ PVOID CiValidateImageData;
    PVOID CiHashMemory;
    PVOID KappxIsPackageFile;
} CI_CALLBACKS_WIN8, *PCI_CALLBACKS_WIN8;
```

Листинг 6-6. Структура буфера g_CiCallbacks в Windows 8.x

Помимо указателей на функции `CiQueryInformation` ❶, `CiValidateImageHeader` ❷ и `CiValidateImageData` ❸, присутствующих в обеих структурах, `CI_CALLBACKS_WIN7_VISTA` и `CI_CALLBACKS_WIN8`, структура `CI_CALLBACKS_WIN8` содержит поля, влияющие на обеспечение целостности кода в Windows 8.

Дополнительные сведения о ci.dll

Подробнее о реализации модуля `ci.dll` можно узнать по адресу github.com. В этой статье глубоко разбирается реализация зашифрованного хранилища памяти внутри `ci.dll`, которое другие компоненты ОС могут использовать для сохранения в тайне определённых подробностей и конфигурационных сведений. Хранилище защищено сильно обфусцированной виртуальной машиной (VM), что существенно затрудняет обратную разработку алгоритма его шифрования и расшифрования. Авторы статьи подробно анализируют метод обфускации VM и предоставляют свой подключаемый модуль WinDbg для расшифрования и шифрования хранилища на лету.

Технология Secure Boot

Технология Secure Boot появилась в Windows 8 для защиты процесса загрузки от заражения буткитом. Secure Boot использует унифицированный расширяемый интерфейс встроенного ПО (Unified Extensible Firmware Interface, UEFI), чтобы блокировать загрузку и выполнение любого загрузочного приложения или драйвера без действительной цифровой подписи и тем самым защищать целостность ядра операционной системы, системных файлов и критически важных для загрузки драйверов. На рисунке 6-2 показан процесс загрузки с включённой Secure Boot.

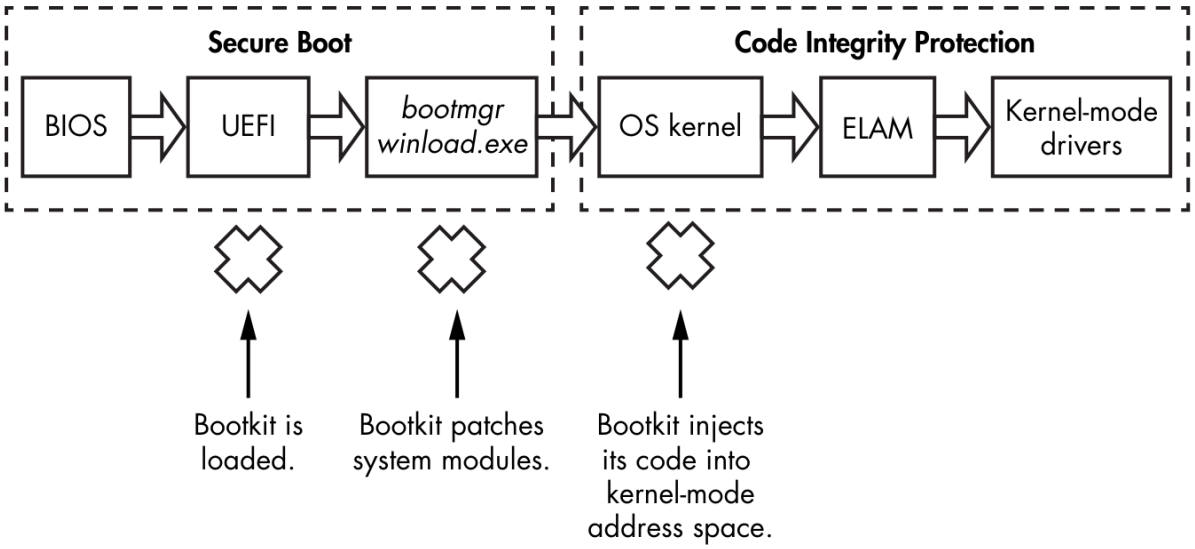


Рисунок 6-2. Ход процесса загрузки с Secure Boot. Secure Boot защищает этапы BIOS, UEFI, bootmgr и winload.exe, а защита целостности кода - ядро ОС, ELAM и драйверы режима ядра. Попытки загрузить буткит, изменить системные модули или внедрить код в адресное пространство ядра блокируются

Когда Secure Boot включена, BIOS проверяет целостность всех файлов UEFI и загрузочных файлов ОС, выполняемых при запуске, чтобы убедиться, что они получены из легитимного источника и имеют действительную цифровую подпись. В рамках проверки Secure Boot подписи всех критически важных для загрузки драйверов проверяются в `winload.exe` и драйвером ELAM. Secure Boot похожа на политику Microsoft по подписыванию кода режима ядра, но применяется к модулям, выполняемым до загрузки и инициализации ядра операционной системы. В результате недоверенные компоненты, то есть компоненты без действительных подписей, не загружаются и запускают процедуру исправления.

При первом запуске системы Secure Boot удостоверяется, что предзагрузочная среда и компоненты загрузчика не были изменены. В свою очередь, загрузчик проверяет целостность ядра и

запускаемых при загрузке драйверов. После успешной проверки целостности ядра Secure Boot проверяет остальные драйверы и модули. В основе Secure Boot лежит предположение о корне доверия: на раннем этапе выполнения система считается доверенной. Разумеется, если злоумышленники успеют провести атаку до этого момента, они, скорее всего, победят.

В последние несколько лет исследовательское сообщество в области безопасности уделяет значительное внимание уязвимостям BIOS, способным позволить злоумышленникам обойти Secure Boot. Мы подробно обсудим эти уязвимости в главе 16 и глубже рассмотрим Secure Boot в главе 17.

Безопасность на основе виртуализации в Windows 10

До появления Windows 10 механизмы целостности кода являлись частью самого ядра системы. По существу, это означает, что механизм целостности работает с тем же уровнем привилегий, который пытается защитить. Во многих случаях это эффективно, но одновременно позволяет злоумышленнику атаковать сам механизм целостности. Чтобы повысить эффективность механизма целостности кода, в Windows 10 появились две новые функции: виртуальный безопасный режим (Virtual Secure Mode) и Device Guard. Обе основаны на аппаратно поддерживаемой изоляции памяти. В общем виде эта технология называется трансляцией адресов второго уровня (Second Level Address Translation); она поддерживается процессорами как Intel, где известна под названием расширенных таблиц страниц (Extended Page Tables, EPT), так и AMD, где называется быстрой индексацией виртуализации (Rapid Virtualization Indexing, RVI).

Трансляция адресов второго уровня

Windows поддерживает трансляцию адресов второго уровня (Second Level Address Translation, SLAT) начиная с Windows 8 с Hyper-V, гипервизором Microsoft. Hyper-V применяет SLAT для управления памятью виртуальных машин, например для защиты доступа, и для снижения накладных расходов на трансляцию гостевых физических адресов, то есть памяти, изолированной технологиями виртуализации, в реальные физические адреса.

SLAT предоставляет гипервизорам промежуточный кеш трансляции виртуальных адресов в физические, что резко сокращает время, необходимое гипервизору для обработки запросов трансляции в физическую память узла. Эта технология также используется при реализации виртуального безопасного режима в Windows 10.

Виртуальный безопасный режим и Device Guard

Безопасность на основе виртуализации в виде виртуального безопасного режима (Virtual Secure Mode, VSM) впервые появилась в Windows 10 и основана на Hyper-V компании Microsoft. При работе VSM операционная система и критически важные системные модули выполняются в изолированных контейнерах, защищённых гипервизором. Поэтому даже при компрометации ядра критические компоненты, выполняемые в других виртуальных средах, остаются защищёнными: злоумышленник не может перейти из одного скомпрометированного виртуального контейнера в другой. VSM также изолирует компоненты целостности кода от самого ядра Windows в защищённом гипервизором контейнере.

Изоляция VSM делает невозможным применение уязвимых легитимных драйверов режима ядра для отключения целостности кода, если только не будет обнаружена уязвимость в самом механизме защиты. Поскольку потенциально уязвимый драйвер и библиотеки целостности кода расположены в разных виртуальных контейнерах, злоумышленники не должны иметь возможности отключить защиту целостности кода.

Технология Device Guard использует VSM, чтобы предотвращать выполнение недоверенного кода в системе. Для обеспечения этих гарантий Device Guard сочетает защищённую VSM проверку целостности кода с платформой и UEFI Secure Boot. Благодаря этому Device Guard применяет политику целостности кода с самого начала процесса загрузки и вплоть до загрузки драйверов режима ядра ОС и приложений пользовательского режима.

На рисунке 6-3 показано, как Device Guard влияет на способность Windows 10 защищаться от буткитов и руткитов. Secure Boot защищает от буткитов, проверяя все компоненты прошивки, выполняемые в предзагрузочной среде, включая загрузчик ОС. Чтобы предотвратить внедрение вредоносного кода в адресное пространство режима ядра, VSM изолирует от него критические компоненты ОС, отвечающие за обеспечение целостности кода. В данном контексте этот механизм называется обеспечиваемой гипервизором целостностью кода (Hypervisor-Enforced Code Integrity, HVCI).

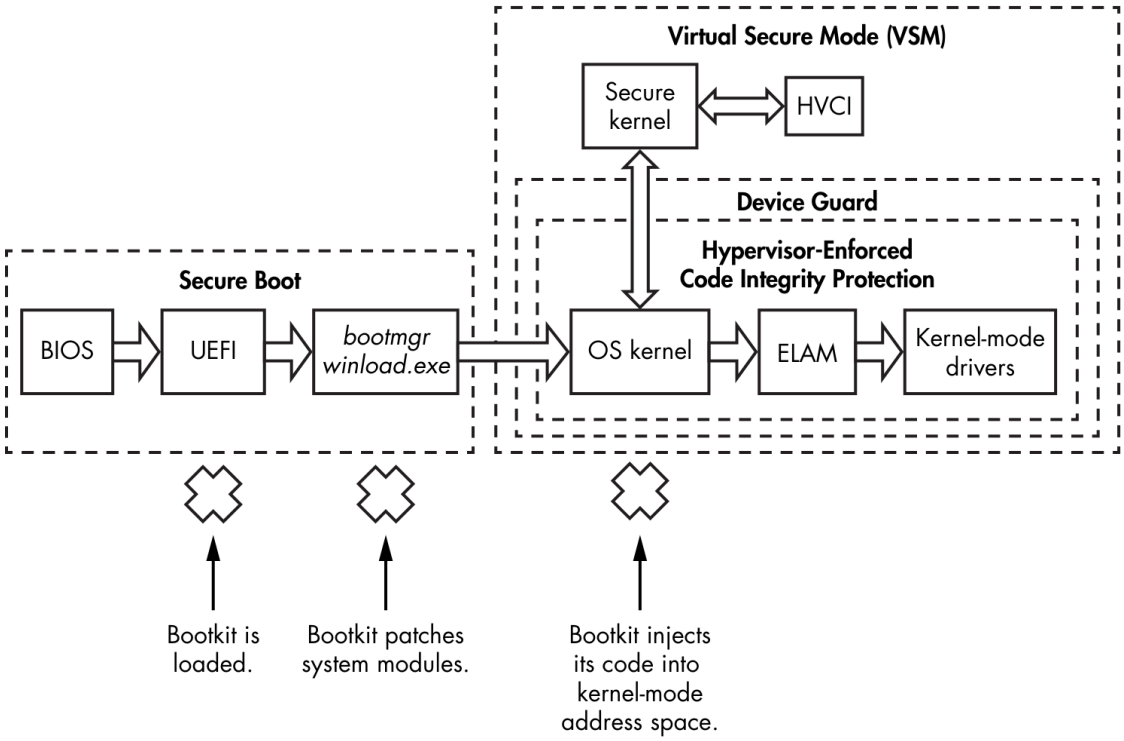


Рисунок 6-3. Процесс загрузки с включёнными Virtual Secure Mode и Device Guard. Secure Boot защищает BIOS, UEFI, bootmgr и winload.exe; Device Guard и HVCI защищают ядро ОС, ELAM и драйверы режима ядра в VSM, где защищённое ядро взаимодействует с HVCI

Ограничения Device Guard для разработки драйверов

Device Guard предъявляет особые требования и накладывает ограничения на процесс разработки драйверов, поэтому некоторые существующие драйверы при активной технологии работают неправильно. Все драйверы должны соблюдать следующие правила:

• Выделять всю невыгружаемую память из неисполняемого (no-execute, NX) невыгружаемого пула. PE-модуль драйвера не может содержать разделы, одновременно доступные для записи и исполнения.
• Не пытаться напрямую изменять исполняемую системную память.
• Не использовать динамический или самомодифицирующийся код в режиме ядра.
• Не загружать никакие данные как исполняемые.

Поскольку большинство современных руткитов и буткитов не соблюдает эти требования, они не могут выполняться при активной Device Guard, даже если драйвер имеет действительную подпись или способен обойти защиту целостности кода.

Заключение

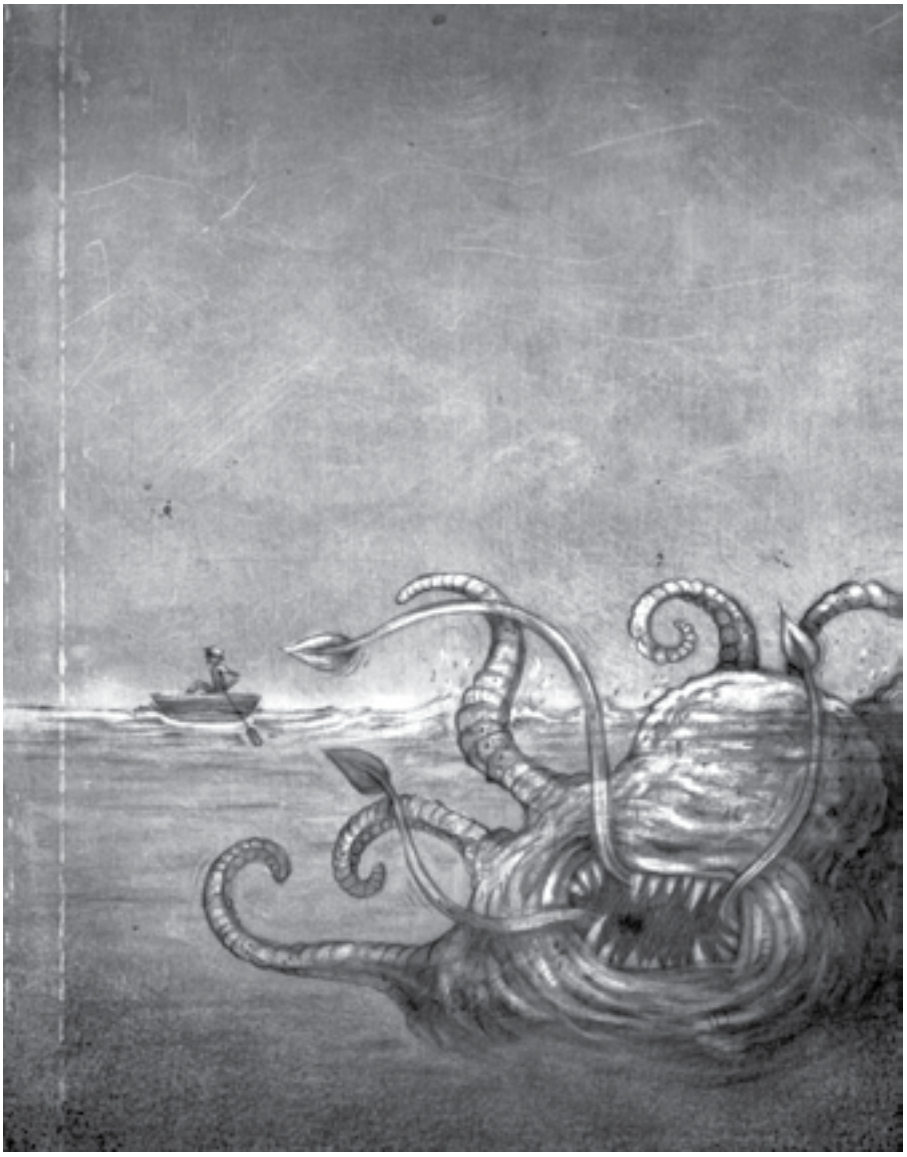
В этой главе был представлен обзор развития средств защиты целостности кода. Безопасность процесса загрузки - важнейший рубеж защиты операционных систем от атак вредоносного ПО. ELAM и механизмы защиты целостности кода представляют собой мощные средства безопасности, ограничивающие выполнение недоверенного кода на платформе.

Windows 10 подняла безопасность процесса загрузки на новый уровень, предотвращая обход целостности кода благодаря изоляции компонентов HVCI от ядра ОС с помощью VSM. Однако без активного механизма Secure Boot буткиты способны обойти эту защиту, атаковав систему до загрузки её компонентов. В следующих главах мы подробнее обсудим Secure Boot и атаки на BIOS, предназначенные для её обхода.

Сноски

[1] [назад] См. Moxie Marlinspike, «Уязвимость SSL в Internet Explorer», moxie.org.

Глава 7. Методы заражения буткитами



Изучив процесс загрузки Windows, теперь обсудим методы заражения буткитами, нацеленные на участвующие в запуске системы модули. Эти методы делятся на две группы в зависимости от атакуемых загрузочных компонентов: методы заражения MBR и методы заражения VBR/начального загрузчика программ (Initial Program Loader, IPL). На примере буткита TDL4 мы рассмотрим заражение MBR, а на примерах буткитов Rovnix и Gapz - два разных метода заражения VBR.

Методы заражения MBR

Подходы, основанные на изменении MBR, являются наиболее распространёнными методами, которые буткиты применяют для атаки на процесс загрузки Windows. Большинство методов заражения MBR непосредственно изменяет либо код MBR, либо данные MBR, например таблицу разделов, а иногда и то и другое.

При изменении кода MBR меняется только загрузочный код MBR, а таблица разделов остаётся нетронутой. Это самый прямолинейный метод заражения. Системный код MBR перезаписывается вредоносным кодом, а исходное содержимое MBR каким-либо образом сохраняется, например в скрытой области жёсткого диска.

Напротив, метод изменения данных MBR предполагает изменение таблицы разделов MBR без изменения загрузочного кода MBR. Это более сложный метод, поскольку содержимое таблицы разделов различается от системы к системе, из-за чего аналитикам трудно найти шаблон, однозначно выявляющий заражение.

Возможны и гибридные методы, объединяющие эти два приёма; они также встречались в реальных атаках.

Далее мы подробнее рассмотрим оба метода заражения MBR.

Изменение кода MBR: метод заражения TDL4

Чтобы продемонстрировать метод заражения с изменением кода MBR, подробно рассмотрим первый замеченный в реальных атаках буткит, нацеленный на 64-разрядную платформу Microsoft Windows, - TDL4. TDL4 повторно использует печально известные передовые методы уклонения от обнаружения и противодействия криминалистическому анализу своего предшественника, руткита TDL3, рассмотренного в главе 1, но дополнительно способен обходить политику подписывания кода режима ядра, обсуждавшуюся в главе 6, и заражать 64-разрядные системы Windows.

В 32-разрядных системах руткит TDL3 мог сохраняться после перезагрузки, изменяя запускаемый при загрузке драйвер режима ядра. Однако обязательная проверка подписей, появившаяся в 64-разрядных системах, не позволяла загрузить заражённый драйвер, из-за чего TDL3 становился неэффективным.

Стремясь обойти защиту 64-разрядной Microsoft Windows, разработчики TDL3 перенесли точку заражения на более раннюю стадию процесса загрузки и реализовали сохранение присутствия с помощью буткита. Так руткит TDL3 превратился в буткит TDL4.

Заражение системы

TDL4 заражает систему, перезаписывая MBR загрузочного жёсткого диска вредоносной MBR. Как уже обсуждалось, она выполняется раньше образа ядра Windows, поэтому способна вмешаться в него и отключить проверки целостности. Другие буткиты на основе MBR подробно описаны в главе 10.

Подобно TDL3, TDL4 создаёт скрытую область хранения в конце жёсткого диска и записывает туда исходную MBR и несколько собственных модулей, перечисленных в таблице 7-1. TDL4 сохраняет исходную MBR, чтобы загрузить её после заражения и создать видимость нормальной загрузки системы, а модули `mbr`, `ldr16`, `ldr32` и `ldr64` применяются буткитом во время загрузки для обхода проверок целостности Windows и последующей загрузки неподписанных вредоносных драйверов.

Таблица 7-1. Модули, записываемые в скрытое хранилище TDL4 при заражении системы

Имя модуля	Описание
`mbr`	Исходное содержимое загрузочного сектора заражённого жёсткого диска
`ldr16`	16-разрядный код загрузчика реального режима
`ldr32`	Поддельная библиотека `kdcom.dll` для систем x86
`ldr64`	Поддельная библиотека `kdcom.dll` для систем x64
`drv32`	Основной драйвер буткита для систем x86
`drv64`	Основной драйвер буткита для систем x64
`cmd.dll`	Полезная нагрузка для внедрения в 32-разрядные процессы
`cmd64.dll`	Полезная нагрузка для внедрения в 64-разрядные процессы
`cfg.ini`	Конфигурационные сведения
`bckfg.tmp`	Зашифрованный список URL-адресов командных серверов (C&C)

TDL4 записывает данные на жёсткий диск, отправляя запросы управляющего кода ввода-вывода `IOCTL_SCSI_PASS_THROUGH_DIRECT` непосредственно минипорт-драйверу диска - самому нижнему драйверу в стеке драйверов жёсткого диска. Это позволяет TDL4 обходить стандартные фильтрующие драйверы ядра и любые реализованные в них защитные меры. TDL4 отправляет запросы управляющего кода посредством API `DeviceIoControl`, передавая первым параметром дескриптор, открытый для символической ссылки `\??\PhysicalDriveXX`, где `XX` - номер заражаемого жёсткого диска.

Для открытия этого дескриптора с правом записи требуются административные привилегии, поэтому TDL4 эксплуатирует уязвимость MS10-092 в службе планировщика заданий Windows, впервые замеченную в Stuxnet, чтобы повысить свои привилегии. Вкратце, эта уязвимость позволяет злоумышленнику несанкционированно повысить привилегии конкретного задания. Для получения административных привилегий TDL4 регистрирует задание, которое планировщик заданий Windows должен выполнить с текущими привилегиями вредоносной программы. Затем программа изменяет XML-файл запланированного задания, предписывая запуск от имени учётной записи Local System с административными привилегиями, и обеспечивает совпадение контрольной суммы изменённого XML-файла с исходной. В результате планировщик обманом заставляют выполнить задание от имени Local System, а не обычного пользователя, что позволяет TDL4 успешно заразить систему.

Записывая данные таким способом, вредоносная программа обходит защитные средства, реализованные на уровне файловой системы, поскольку пакет запроса ввода-вывода (I/O Request Packet, IRP) - структура данных, описывающая операцию ввода-вывода, - поступает непосредственно обработчику драйвера класса дисков.

Установив все свои компоненты, TDL4 принудительно перезагружает систему, выполняя функцию собственного API `NtRaiseHardError`, показанную в листинге 7-1.

```
NTSYSAPI
NTSTATUS
NTAPI

NtRaiseHardError(
      IN NTSTATUS ErrorStatus,
      IN ULONG NumberOfParameters,
      IN PUNICODE_STRING UnicodeStringParameterMask OPTIONAL,
      IN PVOID *Parameters,
      ❶ IN HARDERROR_RESPONSE_OPTION ResponseOption,
      OUT PHARDERROR_RESPONSE Response
);
```

Листинг 7-1. Прототип функции NtRaiseHardError

В качестве пятого параметра код передаёт `OptionShutdownSystem` ❶, что вызывает синий экран смерти (Blue Screen of Death, BSoD). После BSoD система автоматически перезагружается, обеспечивая загрузку модулей руткита при следующем запуске и не предупреждая пользователя о заражении: со стороны кажется, что система просто аварийно завершила работу.

Обход защиты при загрузке заражённой TDL4 системы

На рисунке 7-1 показан процесс загрузки компьютера, заражённого TDL4. Схема на высоком уровне представляет действия вредоносной программы для обхода проверок целостности кода и загрузки её компонентов в систему.

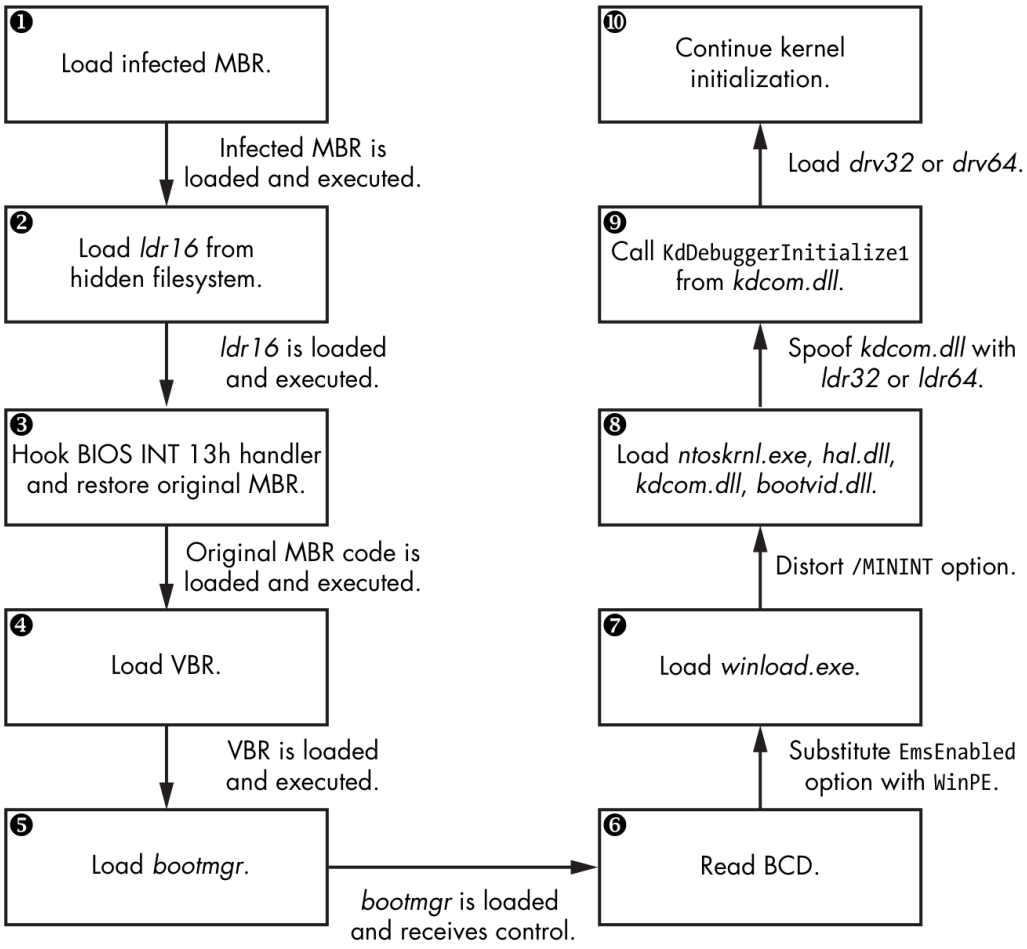


Рисунок 7-1. Процесс загрузки буткита TDL4: заражённая MBR загружает `ldr16`*; он перехватывает BIOS INT 13h, восстанавливает исходную MBR и продолжает штатную загрузку через VBR и* `bootmgr`*. Затем буткит изменяет BCD, загружает* `winload.exe`*, подменяет* `kdcom.dll`*, запускает* `drv32` *или* `drv64` *и продолжает инициализацию ядра*

После BSoD и последующего перезапуска системы BIOS считывает заражённую MBR в память и выполняет её, загружая первую часть буткита, отмеченную ❶ на рисунке 7-1. Затем заражённая MBR находит файловую систему буткита в конце загрузочного жёсткого диска, загружает и выполняет модуль `ldr16`. Этот модуль содержит код, который перехватывает обработчик прерывания 13h BIOS, повторно загружает исходную MBR, что отмечено ❷ и ❸ на рисунке 7-1, и передаёт ей управление. Благодаря этому загрузка может продолжаться обычным образом, но уже с перехваченным обработчиком прерывания 13h. Исходная MBR хранится в модуле `mbr` скрытой файловой системы, как указано в таблице 7-1.

Служба прерывания BIOS 13h предоставляет интерфейс для выполнения операций дискового ввода-вывода в предзагрузочной среде. Это крайне важно, поскольку в самом начале загрузки драйверы запоминающего устройства ещё не загружены в ОС, а стандартные загрузочные компоненты `bootmgr`, `winload.exe` и `winresume.exe` полагаются на службу 13h при чтении системных компонентов с жёсткого диска.

После передачи управления исходной MBR процесс загрузки продолжается как обычно: загружаются VBR и `bootmgr`, что отмечено ❹ и ❺ на рисунке 7-1. Однако находящийся в памяти буткит теперь контролирует все операции ввода-вывода с жёстким диском.

Наиболее интересная часть `ldr16` - функция, реализующая перехват обработчика прерывания дисковых служб 13h. Код, читающий данные с жёсткого диска во время загрузки, полагается на обработчик прерывания BIOS 13h, который теперь перехватывает буткит. Следовательно, буткит

может подделывать любые данные, читаемые с диска в процессе загрузки. Он использует эту возможность, чтобы заменить библиотеку `kdcom.dll` модулем `ldr32` или `ldr64`, в зависимости от операционной системы, из скрытой файловой системы, подменяя содержимое буфера памяти во время операции чтения. Как мы вскоре увидим, замена `kdcom.dll` вредоносной динамической библиотекой (DLL) позволяет буткиту одновременно загрузить собственный драйвер и отключить средства отладки режима ядра.

> **Гонка ко дну**
>
> Перехватывая обработчик дисковых прерываний BIOS, TDL4 следует стратегии руткитов, которые стремятся спускаться всё ниже по стеку служебных интерфейсов. Общее правило таково: побеждает тот, кто внедрился глубже. Поэтому иногда одно защитное ПО начинает бороться с другим за управление нижними уровнями стека. Эта гонка за перехват нижних уровней системы Windows с использованием методов, неотличимых от методов руткитов, привела к проблемам со стабильностью системы. Подробный анализ этих проблем был опубликован в двух статьях в Uninformed.[1]

Чтобы соответствовать требованиям интерфейса связи между ядром Windows и последовательным отладчиком, модули `ldr32` и `ldr64`, выбираемые в зависимости от операционной системы, экспортируют те же символы, что и исходная библиотека `kdcom.dll`, как показано в листинге 7-2.

```
Name                   Address           Ordinal
KdD0Transition         000007FF70451014  1
KdD3Transition         000007FF70451014  2
KdDebuggerInitialize0  000007FF70451020  3
KdDebuggerInitialize1  000007FF70451104  4
KdReceivePacket        000007FF70451228  5
KdReserved0            000007FF70451008  6
KdRestore              000007FF70451158  7
KdSave                 000007FF70451144  8
KdSendPacket           000007FF70451608  9
```

Листинг 7-2. Таблица адресов экспорта ldr32/ldr64

Большинство функций, экспортируемых вредоносной версией `kdcom.dll`, ничего не делает и лишь возвращает 0. Исключением является функция `KdDebuggerInitialize1`, вызываемая образом ядра Windows при инициализации ядра, что отмечено ❾ на рисунке 7-1. Она содержит код, загружающий в систему драйвер буткита. Функция вызывает `PsSetCreateThreadNotifyRoutine`, чтобы зарегистрировать процедуру обратного вызова `CreateThreadNotifyRoutine`, срабатывающую при создании или уничтожении потока. После срабатывания процедура создаёт вредоносный объект `DRIVER_OBJECT`, подключающийся к системным событиям, и ожидает, пока в ходе загрузки не будет построен стек драйверов устройства жёсткого диска.

После загрузки драйвера класса дисков буткит получает доступ к данным на жёстком диске. Поэтому он загружает свой драйвер режима ядра из модуля `drv32` или `drv64`, хранящегося в скрытой файловой системе вместе с модулем, которым была заменена `kdcom.dll`, и вызывает точку входа драйвера.

Отключение проверок целостности кода

Чтобы заменить исходную версию `kdcom.dll` вредоносной DLL в Windows Vista и более поздних версиях, вредоносной программе необходимо отключить проверки целостности кода режима ядра, как обсуждалось ранее. Чтобы избежать обнаружения, она отключает их лишь временно. Если проверки не отключить, `winload.exe` сообщит об ошибке и откажется продолжать загрузку.

Буткит отключает проверки целостности, предписывая `winload.exe` загрузить ядро в режиме предварительной установки, в котором эти проверки не включены; см. раздел «Слабость прежнего механизма целостности кода» на странице 74 оригинального издания. Для этого, когда `bootmgr` читает BCD с жёсткого диска, буткит тем же способом, которым TDL4 подменял `kdcom.dll`, заменяет элемент `BcdLibraryBoolean_EmsEnabled`, закодированный в данных конфигурации загрузки как `16000020`, на `BcdOSLoaderBoolean_WinPEMode`, закодированный в BCD как `26000022`. Этот шаг отмечен ❻ на рисунке 7-1. `BcdLibraryBoolean_EmsEnabled` является наследуемым объектом, который указывает, следует ли включить глобальное перенаправление служб аварийного управления, и по умолчанию имеет значение `TRUE`. В листинге 7-3 показан реализованный в `ldr16` ассемблерный код, подменяющий параметр `BcdLibraryBoolean_EmsEnabled`, что отмечено ❶, ❷ и ❸.

```
seg000:02E4   cmp     dword ptr es:[bx], '0061'     ; подмена BcdLibraryBoolean_EmsEnabled
seg000:02EC   jnz     short loc_30A                 ; подмена BcdLibraryBoolean_EmsEnabled
seg000:02EE   cmp     dword ptr es:[bx+4], '0200'   ; подмена BcdLibraryBoolean_EmsEnabled
seg000:02F7   jnz     short loc_30A                 ; подмена BcdLibraryBoolean_EmsEnabled
seg000:02F9 ❶ mov     dword ptr es:[bx], '0062'     ; подмена BcdLibraryBoolean_EmsEnabled
seg000:0301 ❷ mov     dword ptr es:[bx+4], '2200'   ; подмена BcdLibraryBoolean_EmsEnabled
seg000:030A   cmp     dword ptr es:[bx], 1666Ch     ; подмена BcdLibraryBoolean_EmsEnabled
seg000:0312   jnz     short loc_328                 ; подмена BcdLibraryBoolean_EmsEnabled
seg000:0314   cmp     dword ptr es:[bx+8], '0061'   ; подмена BcdLibraryBoolean_EmsEnabled
seg000:031D   jnz     short loc_328                 ; подмена BcdLibraryBoolean_EmsEnabled
seg000:031F ❸ mov     dword ptr es:[bx+8], '0062'   ; подмена BcdLibraryBoolean_EmsEnabled
seg000:0328   cmp     dword ptr es:[bx], 'NIM/'     ; подмена /MININT
seg000:0330   jnz     short loc_33A                 ; подмена /MININT
seg000:0332 ❹ mov     dword ptr es:[bx], 'M/NI'     ; подмена /MININT
```

Листинг 7-3. Часть кода ldr16, отвечающая за подмену параметров BcdLibraryBoolean_EmsEnabled и /MININT

Затем буткит включает режим предварительной установки на время, достаточное для загрузки вредоносной версии `kdcom.dll`. После её загрузки вредоносная программа отключает этот режим так, словно он никогда не был включён, чтобы удалить из системы все следы. Обратите внимание: злоумышленники могут отключить режим предварительной установки только пока он активен. Для этого во время чтения образа `winload.exe` с жёсткого диска портится строковый параметр `/MININT` в этом образе, что отмечено ❹ в листинге и ❼ на рисунке 7-1. При инициализации ядро получает от `winload.exe` список параметров, включающих определённые режимы и задающих характеристики среды загрузки, например число процессоров в системе, необходимость загрузки в режиме предварительной установки и необходимость показывать индикатор хода загрузки. Параметры, описываемые строковыми литералами, хранятся в `winload.exe`.

Образ `winload.exe` использует параметр `/MININT`, чтобы сообщить ядру о включённом режиме предварительной установки. В результате манипуляций вредоносной программы ядро получает недействительный параметр `/MININT` и продолжает инициализацию так, будто этот режим не был включён. Это заключительный этап процесса загрузки заражённой буткитом системы, отмеченный ❿ на рисунке 7-1. Вредоносный драйвер режима ядра успешно загружается в операционную систему в обход проверок целостности кода.

Шифрование вредоносного кода MBR

В листинге 7-4 показана часть вредоносного кода MBR буткита TDL4. Обратите внимание: чтобы избежать обнаружения статическим анализом, использующим статические сигнатуры, вредоносный код, начиная с отметки ❸, зашифрован.

```
seg000:0000       xor     ax, ax
seg000:0002       mov     ss, ax

seg000:0004       mov     sp, 7C00h
seg000:0007       mov     es, ax
seg000:0009       mov     ds, ax
seg000:000B       sti
seg000:000C       pusha
seg000:000D ❶     mov     cx, 0CFh        ; размер расшифрованных данных
seg000:0010       mov     bp, 7C19h       ; смещение зашифрованных данных
seg000:0013
seg000:0013 decrypt_routine:
seg000:0013 ❷     ror     byte ptr [bp+0], cl
seg000:0016       inc     bp
seg000:0017       loop    decrypt_routine
seg000:0017 ; ---------------------------------------------------------------
seg000:0019 ❸  db 44h                     ; начало зашифрованных данных
seg000:001A    db 85h
seg000:001C    db 0C7h
seg000:001D    db 1Ch
seg000:001E    db 0B8h
seg000:001F    db 26h
seg000:0020    db 04h
seg000:0021    --snip--
```

Листинг 7-4. Код TDL4 для расшифрования вредоносной MBR

Регистры `cx` и `bp` ❶ инициализируются соответственно размером и смещением зашифрованного кода. Значение регистра `cx` служит счётчиком цикла ❷, выполняющего побитовую логическую операцию `ror`, инструкцию циклического сдвига вправо, для расшифрования кода, отмеченного ❸ и адресуемого регистром `bp`. После расшифрования код перехватывает обработчик INT 13h, чтобы изменять другие модули ОС, отключать проверку целостности кода ОС и загружать вредоносные драйверы.

Изменение таблицы разделов MBR

Один из вариантов TDL4, известный как Olmasco, демонстрирует другой подход к заражению MBR: изменение таблицы разделов вместо кода MBR. Сначала Olmasco создаёт нераспределённый раздел в конце загрузочного жёсткого диска, а затем в том же месте создаёт скрытый раздел, изменяя свободный элемент 2 таблицы разделов MBR, как показано на рисунке 7-2.

Этот путь заражения возможен потому, что MBR содержит таблицу разделов, начинающуюся со смещения `0x1BE` и состоящую из четырёх 16-байтовых элементов, каждый из которых описывает соответствующий раздел жёсткого диска; массив `MBR_PARTITION_TABLE_ENTRY` показан ранее в листинге 5-2. Следовательно, на жёстком диске может быть не более четырёх основных разделов, и только один из них помечается как активный. Операционная система загружается с активного раздела. Olmasco записывает параметры собственного вредоносного раздела в пустой элемент таблицы, помечает этот раздел активным и инициализирует VBR вновь созданного раздела. Подробнее механизм заражения Olmasco рассматривается в главе 10.

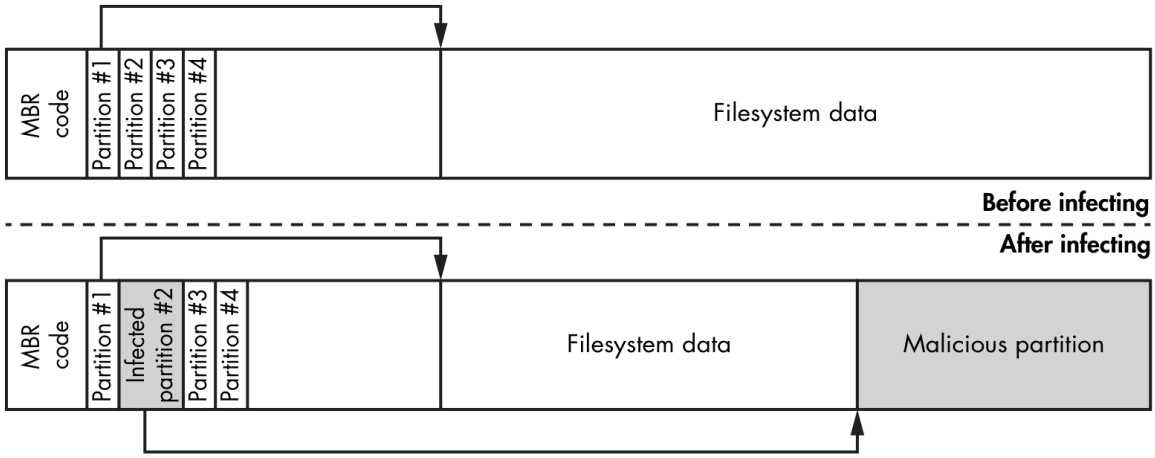


Рисунок 7-2. Изменение таблицы разделов MBR буткитом Olmasco. До заражения второй элемент таблицы свободен; после заражения он указывает на вредоносный раздел в конце диска

Методы заражения VBR/IPL

Иногда защитное программное обеспечение проверяет наличие несанкционированных изменений только в MBR, оставляя VBR и IPL без проверки. Инфекторы VBR/IPL, подобно первым VBR-буткитам, используют это для повышения своих шансов остаться необнаруженными.

Все известные методы заражения VBR относятся к одной из двух групп: изменение IPL, как у буткита Rovnix, и изменение блока параметров BIOS (BIOS parameter block, BPB), как у буткита Gapz.

Изменение IPL: Rovnix

Рассмотрим применяемый буткитом Rovnix метод заражения с изменением IPL. Вместо перезаписи сектора MBR Rovnix изменяет IPL в активном разделе загрузочного жёсткого диска и код начальной загрузки NTFS. Как показано на рисунке 7-3, Rovnix считывает 15 секторов после VBR, содержащих IPL, сжимает их, добавляет перед ними вредоносный код начальной загрузки и записывает изменённый код обратно в эти 15 секторов. Поэтому при следующем запуске системы управление получает вредоносный код начальной загрузки.

При выполнении вредоносный код начальной загрузки перехватывает обработчик INT 13h, чтобы изменить `bootmgr`, `winload.exe` и ядро и получить управление после загрузки компонентов загрузчика. Наконец, Rovnix распаковывает исходный код IPL и возвращает ему управление.

Загрузочный код Rovnix следует потоку выполнения операционной системы от начала загрузки через переключение режима выполнения процессора и до загрузки ядра. Более того, с помощью отладочных регистров от `DR0` до `DR7`, являющихся неотъемлемой частью архитектур x86 и x64, Rovnix сохраняет управление во время инициализации ядра и загружает собственный вредоносный драйвер в обход проверки целостности кода режима ядра. Эти отладочные регистры позволяют вредоносной программе устанавливать перехваты системного кода, фактически не изменяя его, и тем самым сохранять целостность перехватываемого кода.

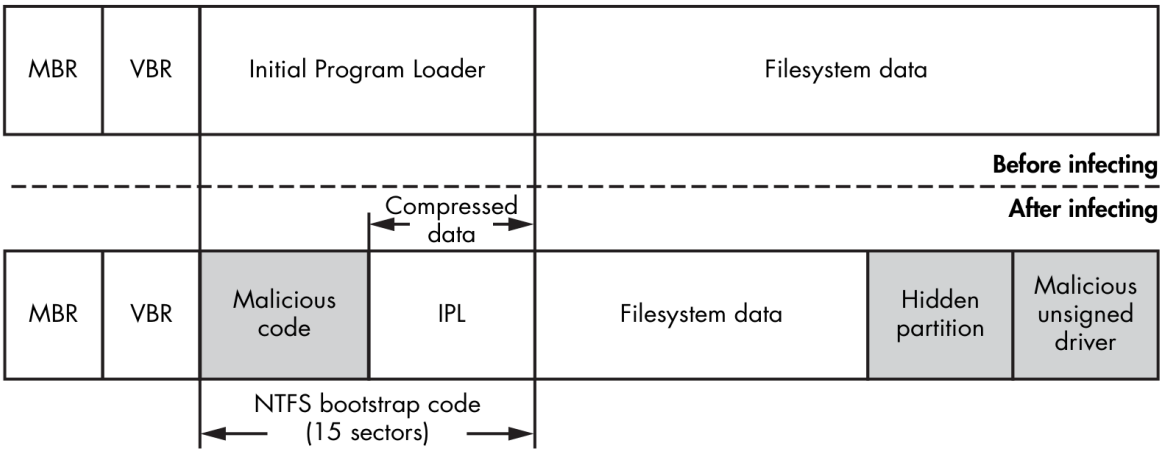


Рисунок 7-3. Изменение IPL буткитом Rovnix. Исходные 15 секторов кода начальной загрузки NTFS сжимаются; перед ними помещается вредоносный код, а в скрытом разделе хранится неподписанный вредоносный драйвер

Загрузочный код Rovnix тесно взаимодействует с компонентами загрузчика операционной системы и в значительной мере зависит от их средств отладки платформы и двоичного представления. Подробнее Rovnix рассматривается в главе 11.

Заражение VBR: Gapz

Буткит Gapz заражает VBR активного раздела, а не IPL. Gapz отличается высокой скрытностью, поскольку заражает лишь несколько байтов исходной VBR: изменяет поле `HiddenSectors`, показанное в листинге 5-3 на странице 63 оригинального издания, и оставляет все остальные данные и код в VBR и IPL нетронутыми.

В случае Gapz наиболее интересным для анализа блоком является BPB (`BIOS_PARAMETER_BLOCK`), в особенности его поле `HiddenSectors`. Значение этого поля задаёт количество секторов тома NTFS, предшествующих IPL, как показано на рисунке 7-4.

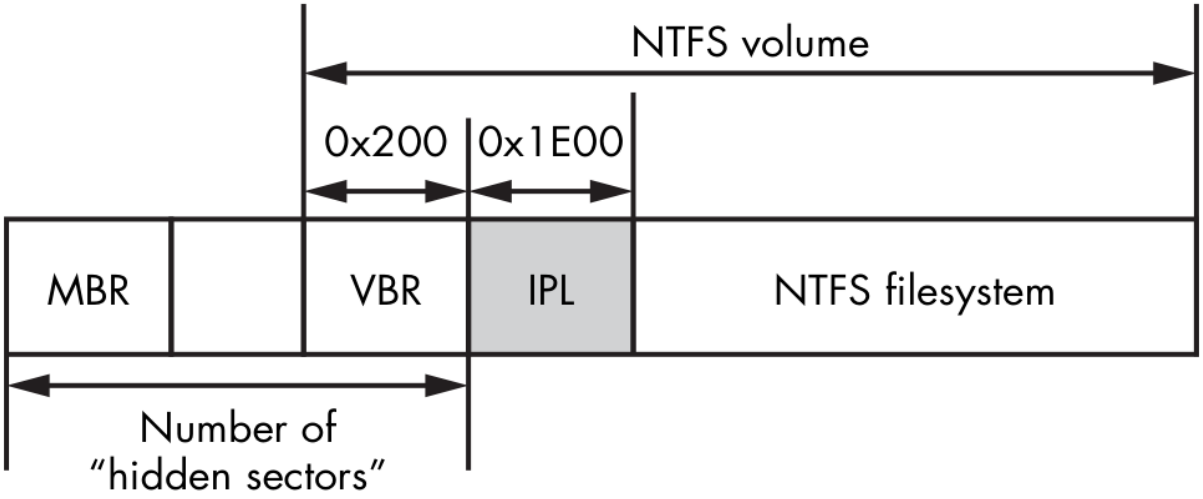


Рисунок 7-4. Расположение IPL. Поле количества «скрытых секторов» задаёт расстояние от начала диска до IPL внутри тома NTFS

Gapz перезаписывает поле `HiddenSectors` значением смещения в секторах вредоносного кода буткита, хранящегося на жёстком диске, как показано на рисунке 7-5. При следующем выполнении код VBR загружает и выполняет код буткита вместо легитимного IPL. Образ буткита Gapz записывается либо перед первым разделом, либо после последнего раздела жёсткого диска. Подробнее Gapz рассматривается в главе 12.

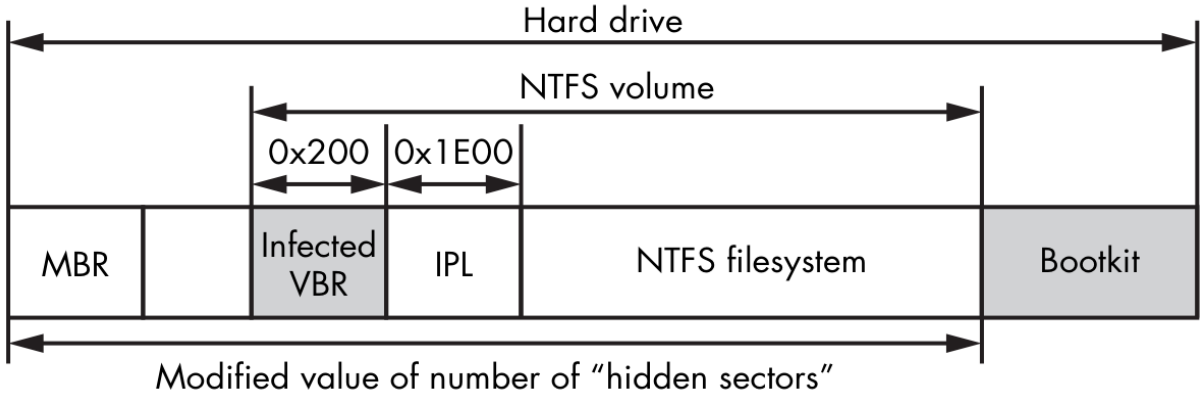


Рисунок 7-5. Заражение VBR буткитом Gapz. Изменённое поле числа «скрытых секторов» заражённой VBR указывает на буткит в конце жёсткого диска вместо штатного IPL

Заключение

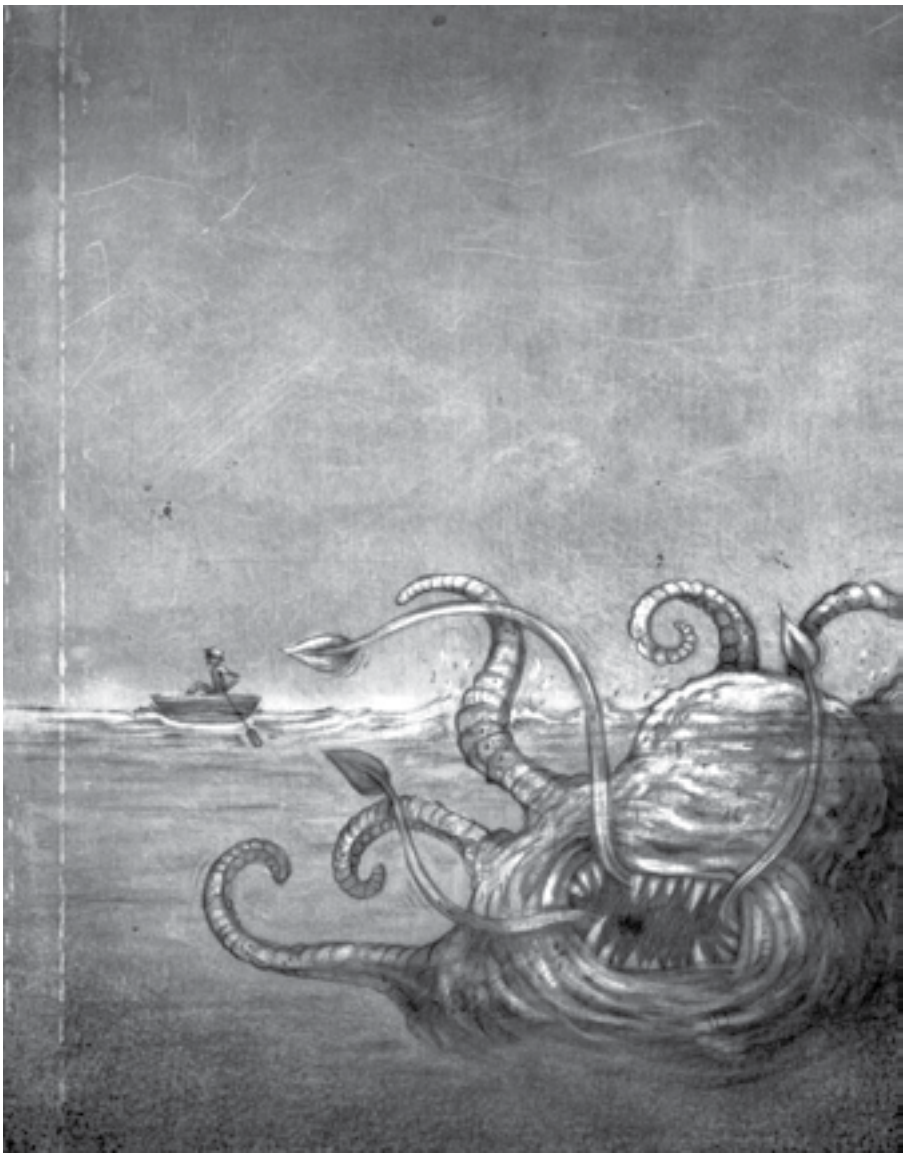
В этой главе вы познакомились с методами заражения MBR и VBR буткитами. Мы проследили превращение развитого руткита TDL3 в современный буткит TDL4 и увидели, как TDL4 получает управление загрузкой системы, заражая MBR посредством замены её вредоносным кодом. Как вы увидели, средства защиты целостности в 64-разрядных операционных системах Microsoft, прежде всего политика подписывания кода режима ядра, положили начало новой гонке в разработке буткитов, нацеленных на платформы x64. TDL4 стал первым замеченным в реальных атаках буткитом, который успешно преодолел это препятствие, применив ряд архитектурных решений, впоследствии перенятых другими буткитами. Мы также рассмотрели методы заражения VBR на примерах буткитов Rovnix и Gapz, которым посвящены соответственно главы 11 и 12.

На этой странице исходного издания нет основного текста.

Сноски

[1] [назад] skape, «О чём они думали? Неприятности, вызванные небезопасными предположениями», *Uninformed* 1 (май 2005 года), uninformed.org; Skywing, «О чём они думали? Когда антивирусное ПО работает неправильно», *Uninformed* 4 (июнь 2006 года), uninformed.org.

Глава 8. Статический анализ буткита с помощью IDA Pro



В этой главе представлены основные понятия статического анализа буткитов с помощью IDA Pro. К обратной разработке буткитов можно подходить по-разному, и для описания всех существующих подходов потребовалась бы отдельная книга. Мы сосредоточимся на дизассемблере IDA Pro, поскольку он предоставляет уникальные возможности для статического анализа буткитов.

Статический анализ буткитов радикально отличается от обратной разработки в большинстве традиционных прикладных сред, поскольку критически важные части буткита выполняются в предзагрузочной среде. Например, типичное приложение Windows опирается на стандартные библиотеки Windows и, как ожидается, вызывает стандартные библиотечные функции, известные таким средствам обратной разработки, как Hex-Rays IDA Pro. По вызываемым функциям можно многое заключить о приложении; то же верно для приложений Linux и системных вызовов POSIX. Но в предзагрузочной среде такие подсказки отсутствуют, поэтому средствам её анализа нужны дополнительные возможности, компенсирующие нехватку сведений. К счастью, они есть в IDA Pro, и в этой главе объясняется, как ими пользоваться.

Как обсуждалось в главе 7, буткит состоит из нескольких тесно связанных модулей: инфектора главной загрузочной записи (MBR) или загрузочной записи тома (VBR), вредоносного загрузчика, драйверов режима ядра и других компонентов. В этой главе мы ограничимся анализом MBR буткита и легитимной VBR операционной системы, которые можно использовать как образец при обратной разработке любого кода, выполняемого в предзагрузочной среде. Используемые здесь MBR и VBR можно загрузить из дополнительных материалов к книге. В конце главы мы обсудим работу с другими компонентами буткита, такими как вредоносный загрузчик и драйверы режима ядра. Если вы ещё не проработали главу 7, сделайте это сейчас.

Сначала мы покажем, как приступить к анализу буткита: вы узнаете, какие параметры IDA Pro использовать для загрузки кода в дизассемблер, какой API применяется в предзагрузочной среде, как управление передаётся между модулями и какие возможности IDA могут упростить обратную разработку. Затем вы научитесь создавать собственный загрузчик для IDA Pro, автоматизирующий задачи анализа. Наконец, мы предложим набор упражнений для дальнейшего изучения статического анализа буткитов. Материалы к этой главе можно загрузить по адресу nostarch.com.

Анализ MBR буткита

Сначала проанализируем MBR буткита в дизассемблере IDA Pro. Используемая в этой главе MBR похожа на ту, которую создаёт буткит TDL4, рассмотренный в главе 7. MBR TDL4 служит хорошим примером: она реализует традиционную функциональность буткита, но её код легко дизассемблировать и понять. Пример VBR в этой главе основан на легитимном коде реального тома Microsoft Windows.

Загрузка и расшифрование MBR

В следующих разделах вы загрузите MBR в IDA Pro и проанализируете код MBR в точке входа. Затем расшифруете код и изучите, как MBR управляет памятью.

Загрузка MBR в IDA Pro

Первый шаг статического анализа MBR буткита - загрузить её код в IDA. Поскольку MBR не является обычным исполняемым файлом и для неё нет специального загрузчика, её следует загрузить как двоичный модуль. IDA Pro просто поместит MBR в свою память как один непрерывный сегмент, подобно BIOS, без какой-либо дополнительной обработки. Требуется лишь указать начальный адрес памяти этого сегмента.

Откройте двоичный файл в IDA Pro. При первой загрузке MBR программа показывает сообщение с различными параметрами, представленное на рисунке 8-1.

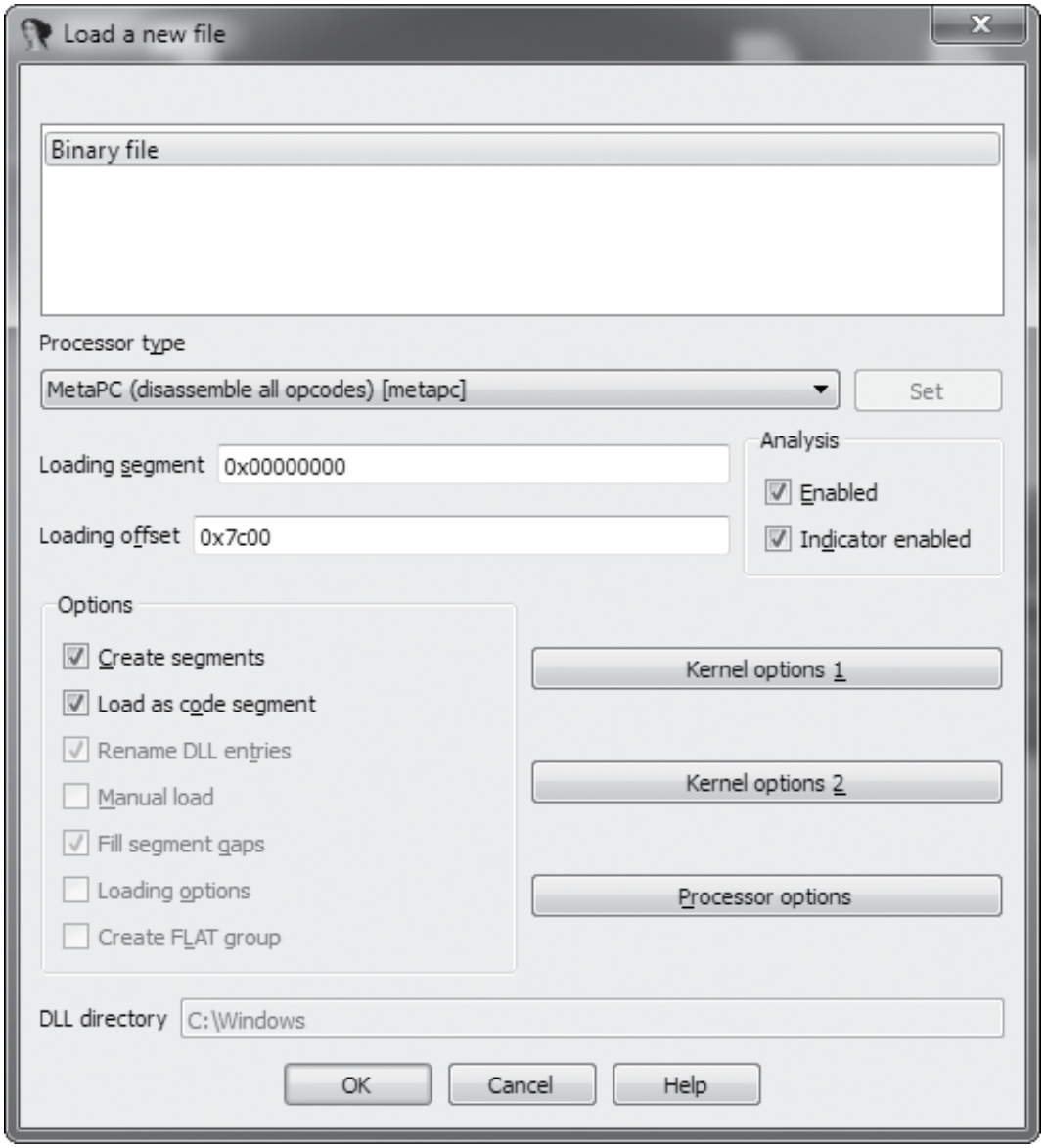


Рисунок 8-1. Диалог IDA Pro, отображаемый при загрузке MBR

Для большинства параметров можно оставить значения по умолчанию, но в поле `Loading offset` ❶, определяющее адрес загрузки модуля в памяти, необходимо ввести значение. Оно всегда должно быть равно `0x7C00` - фиксированному адресу, по которому загрузочный код BIOS помещает MBR. Введя смещение, нажмите `OK`. IDA Pro загрузит модуль и предложит выбрать 16- или 32-разрядный режим дизассемблирования, как показано на рисунке 8-2.

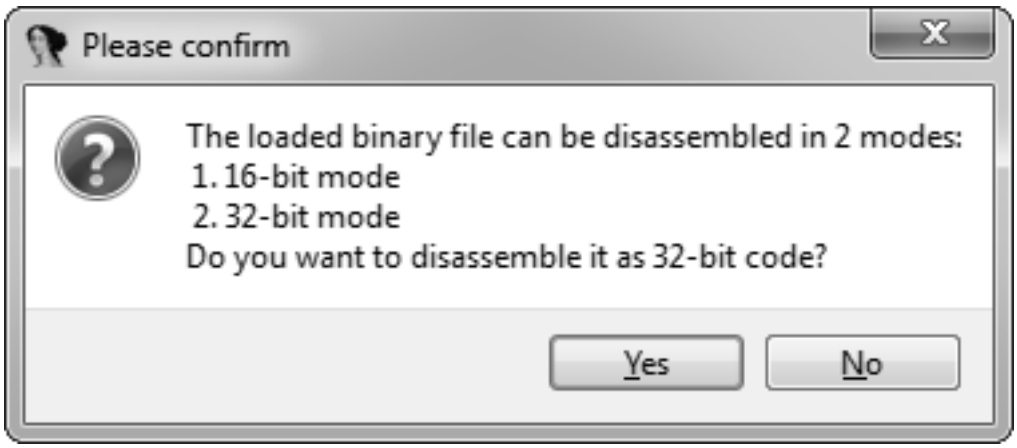


Рисунок 8-2. Диалог IDA Pro с запросом режима дизассемблирования

В этом примере выберите `No`. Тем самым вы укажете IDA дизассемблировать MBR как 16-разрядный код реального режима, то есть именно так, как его декодирует реальный процессор в самом начале загрузки.

Поскольку IDA Pro хранит результаты дизассемблирования в файле базы данных с расширением `.idb`, далее мы будем называть результат дизассемблирования базой данных. В ней IDA собирает все аннотации к коду, которые вы создаёте посредством действий в графическом интерфейсе и сценариев IDA. Базу данных можно считать неявным аргументом всех функций сценариев IDA: она представляет текущее состояние с трудом добытых знаний обратной разработки о двоичном файле, с которыми может работать IDA.

Если у вас нет опыта работы с базами данных, не беспокойтесь: интерфейсы IDA устроены так, что знать внутреннее устройство базы не требуется. Тем не менее понимание того, как IDA представляет полученные сведения о коде, очень помогает.

Анализ точки входа MBR

При загрузке BIOS заражённая буткитом MBR выполняется с первого байта. В качестве адреса её загрузки мы указали дизассемблеру IDA `0:7C00h` - именно туда её загружает BIOS. В листинге 8-1 показаны первые байты загруженного образа MBR.

```
seg000:7C00 ; Тип сегмента: чистый код
seg000:7C00 seg000          segment byte public 'CODE' use16
seg000:7C00                 assume cs:seg000
seg000:7C00                 ;org 7C00h
seg000:7C00                 assume es:nothing, ss:nothing, ds:nothing, fs:nothing, gs:nothing
seg000:7C00                 xor     ax, ax
seg000:7C02              ❶  mov     ss, ax
seg000:7C04                 mov     sp, 7C00h
seg000:7C07                 mov     es, ax
seg000:7C09                 mov     ds, ax
seg000:7C0B                 sti
seg000:7C0C                 pusha
seg000:7C0D                 mov     cx, 0CFh
seg000:7C10                 mov     bp, 7C19h
seg000:7C13
seg000:7C13 loc_7C13:                               ; CODE XREF: seg000:7C17
```

```
seg000:7C13              ❷  ror     byte ptr [bp+0], cl
seg000:7C16                 inc     bp
seg000:7C17                 loop    loc_7C13
seg000:7C17 ; ---------------------------------------------------------------------------
seg000:7C19 encrypted_code  db 44h, 85h, 1Dh, 0C7h, 1Ch, 0B8h, 26h, 4, 8, 68h, 62h
seg000:7C19              ❸  db 40h, 0Eh, 83h, 0Ch, 0A3h, 0B1h, 1Fh, 96h, 84h, 0F5h
```

Листинг 8-1. Точка входа MBR

В начале видна инициализационная заглушка ❶, настраивающая селектор сегмента стека `ss`, указатель стека `sp` и регистры селекторов сегментов `es` и `ds` для обращения к памяти и выполнения подпрограмм. После инициализационной заглушки находится процедура расшифрования ❷. Она расшифровывает остальную часть MBR ❸, побайтово циклически сдвигая биты инструкцией `ror`, а затем передаёт управление расшифрованному коду. Размер зашифрованного блока задаётся в регистре `cx`, а регистр `bp` указывает на этот блок. Такое специализированное шифрование предназначено для затруднения статического анализа и уклонения от обнаружения защитным ПО. Оно же создаёт первое препятствие для нас: чтобы продолжить анализ, теперь необходимо извлечь фактический код.

Расшифрование кода MBR

Чтобы продолжить анализ зашифрованной MBR, необходимо расшифровать код. Благодаря механизму сценариев IDA это легко сделать с помощью сценария Python из листинга 8-2.

```
❶ import idaapi

# начало зашифрованного кода и его размер в памяти
start_ea = 0x7C19
encr_size = 0xCF

❷ for ix in xrange(encr_size):
  ❸ byte_to_decr = idaapi.get_byte(start_ea + ix)
  to_rotate = (0xCF - ix) % 8
  byte_decr = (byte_to_decr >> to_rotate) | (byte_to_decr << (8 - to_rotate))
  ❹ idaapi.patch_byte(start_ea + ix, byte_decr)
```

Листинг 8-2. Сценарий Python для расшифрования кода MBR

Сначала импортируется пакет `idaapi` ❶, содержащий библиотеку API IDA. Затем выполняется цикл по зашифрованным байтам и их расшифрование ❷. Для получения байта из сегмента дизассемблирования применяется API `get_byte` ❸, принимающий единственным параметром адрес читаемого байта. После расшифрования байт записывается обратно в область дизассемблирования ❹ с помощью API `patch_byte`, который принимает адрес изменяемого байта и записываемое значение. Сценарий можно выполнить, выбрав в меню IDA `File > Script` или нажав `Alt-F7`.

> **Примечание.** Этот сценарий изменяет не фактический образ MBR, а её представление в IDA, то есть представление IDA о том, как будет выглядеть загруженный код в момент готовности к выполнению. Перед любыми изменениями дизассемблированного кода создайте резервную копию текущей версии базы данных IDA. Тогда, если сценарий изменения кода MBR содержит ошибки и исказит код, вы сможете легко восстановить последнюю версию.

Анализ управления памятью в реальном режиме

Расшифровав код, продолжим его анализ. В расшифрованном коде вы обнаружите инструкции, показанные в листинге 8-3. Они инициализируют вредоносный код, сохраняя входные параметры MBR и выделяя память.

```
seg000:7C19             ❶ mov     ds:drive_no, dl
seg000:7C1D             ❷ sub     word ptr ds:413h, 10h
seg000:7C22               mov     ax, ds:413h
seg000:7C25               shl     ax, 6
seg000:7C28             ❸ mov     ds:buffer_segm, ax
```

Листинг 8-3. Выделение памяти в предзагрузочной среде

Ассемблерная инструкция ❶ сохраняет содержимое регистра `dl` в памяти по смещению относительно сегмента `ds`. Исходя из опыта анализа такого кода, можно предположить, что `dl` содержит номер жёсткого диска, с которого выполняется MBR; пометьте это смещение как переменную `drive_no`. IDA Pro записывает аннотацию в базу данных и показывает её в листинге. При операциях ввода-вывода этот целочисленный индекс позволяет различать доступные системе диски. В следующем разделе эта переменная будет использована с дисковой службой BIOS.

Аналогичным образом в листинге 8-3 показана аннотация `buffer_segm` ❸ для смещения, по которому код выделяет буфер. IDA Pro автоматически распространяет такие аннотации на другой код, использующий те же переменные.

Под номером ❷ видно выделение памяти. В предзагрузочной среде нет диспетчера памяти в современном смысле, например логики ОС, обеспечивающей вызовы `malloc()`. Вместо этого BIOS хранит количество килобайт доступной памяти в слове - 16-разрядном значении архитектуры x86 - по адресу `0:413h`. Чтобы выделить `X` Кбайт памяти, значение `X` вычитают из общего размера доступной памяти, хранящегося в слове по адресу `0:413h`, как показано на рисунке 8-3.

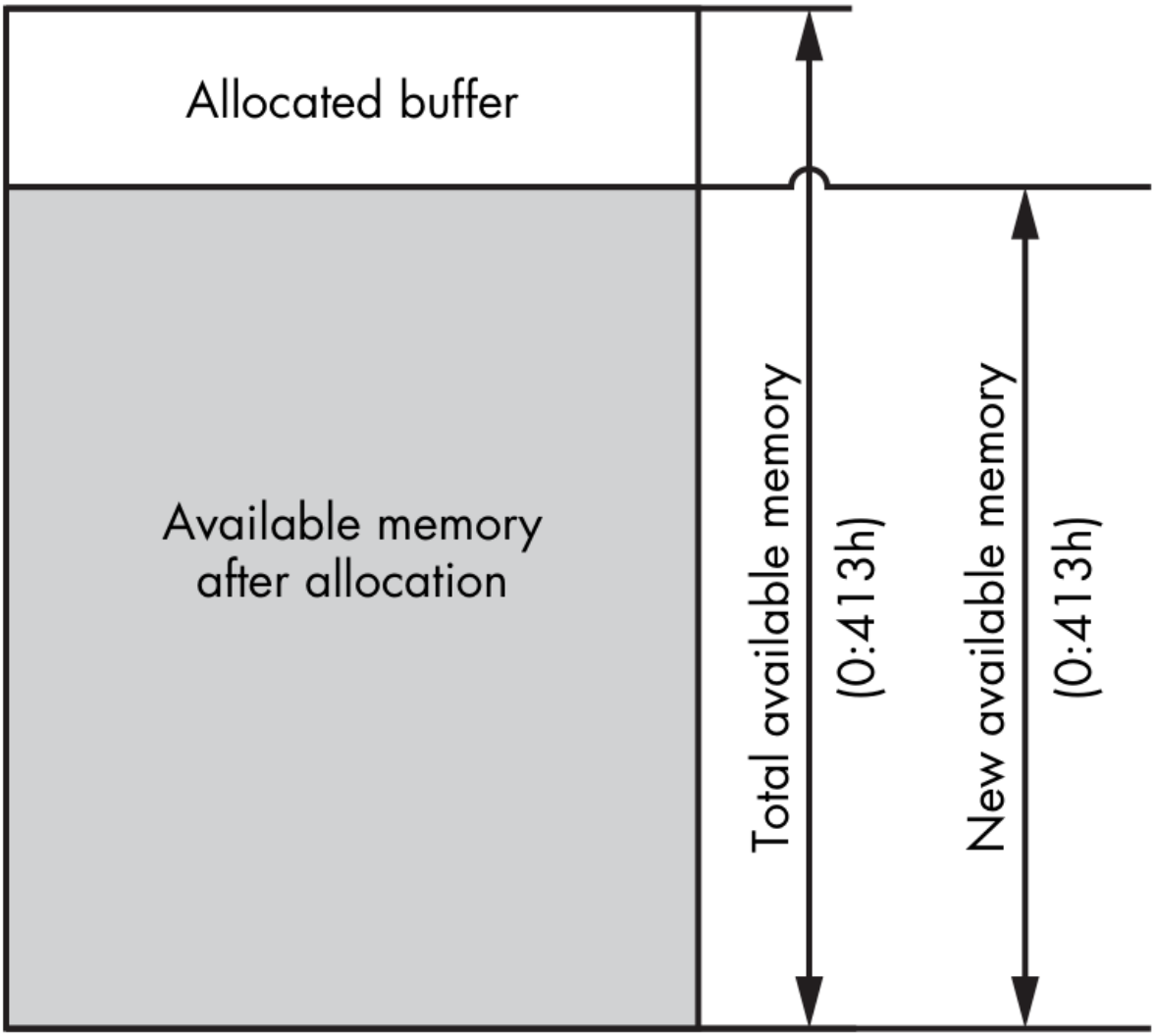


Рисунок 8-3. Управление памятью в предзагрузочной среде. После уменьшения значения общего объёма доступной памяти по адресу `0:413h` верхняя часть прежнего диапазона становится выделенным буфером

В листинге 8-3 код выделяет буфер размером 10 Кбайт, вычитая `10h` из общего доступного объёма. Фактический адрес сохраняется в переменной `buffer_segm` ❸. Затем MBR использует выделенный буфер для хранения данных, прочитанных с жёсткого диска.

Анализ дисковой службы BIOS

Ещё одна уникальная особенность предзагрузочной среды - дисковая служба BIOS, то есть API для взаимодействия с жёстким диском. В контексте анализа буткитов этот API особенно интересен по двум причинам. Во-первых, буткиты используют его для чтения данных с жёсткого диска, поэтому для понимания кода буткита важно знать наиболее часто применяемые команды API. Во-вторых, сам этот API часто становится целью буткитов. В наиболее распространённом сценарии буткит перехватывает API, чтобы изменять легитимные модули, которые другой код читает с жёсткого диска во время загрузки.

Дисковая служба BIOS доступна посредством инструкции `INT 13h`. Для выполнения операций ввода-вывода программа передаёт параметры через регистры процессора и выполняет `INT 13h`, передающую управление соответствующему обработчику. Код, или идентификатор, операции ввода-вывода передаётся в регистре `ah`, старшей части регистра `ax`. Регистр `dl` используется для передачи индекса нужного диска. Флаг переноса процессора (carry flag, CF) показывает, возникла ли ошибка при выполнении службы: если CF равен 1, произошла ошибка, а подробный код ошибки возвращается в регистре `ah`. Это соглашение BIOS о передаче аргументов функции появилось раньше современных соглашений о системных вызовах ОС. Если оно кажется запутанным, помните: именно отсюда возникла идея единообразных интерфейсов системных вызовов.

Прерывание INT 13h служит точкой входа в дисковую службу BIOS и позволяет программам предзагрузочной среды выполнять базовые операции ввода-вывода с дисковыми устройствами: жёсткими дисками, дисководами и приводами CD-ROM. Команды перечислены в таблице 8-1.

Таблица 8-1. Команды INT 13h

Код операции	Описание операции
`2h`	Чтение секторов в память
`3h`	Запись секторов диска
`8h`	Получение параметров диска
`41h`	Проверка установки расширений
`42h`	Расширенное чтение
`43h`	Расширенная запись
`48h`	Расширенное получение параметров диска

Операции в таблице 8-1 делятся на две группы: к первой относятся расширенные операции с кодами `41h`, `42h`, `43h` и `48h`, а ко второй - устаревшие операции с кодами `2h`, `3h` и `8h`.

Единственное различие между группами состоит в том, что расширенные операции могут использовать схему адресации на основе логической адресации блоков (Logical Block Addressing, LBA), тогда как устаревшие операции полагаются исключительно на прежнюю схему адресации «цилиндр, головка, сектор» (Cylinder Head Sector, CHS). В схеме LBA секторы нумеруются на диске линейно, начиная с индекса 0, а в схеме CHS каждый сектор адресуется кортежем `(c,h,s)`, где `c` - номер цилиндра, `h` - номер головки, а `s` - номер сектора. Хотя буткиты могут использовать любую группу, почти всё современное оборудование поддерживает схему адресации LBA.

Получение параметров диска для поиска скрытого хранилища

Продолжая просматривать код MBR после выделения 10 Кбайт памяти, вы увидите выполнение инструкции `INT 13h`, показанной в листинге 8-4.

```
seg000:7C2B              ❶ mov     ah, 48h
seg000:7C2D              ❷ mov     si, 7CF9h
seg000:7C30                mov     ds:drive_param.bResultSize, 1Eh
seg000:7C36                int     13h         ; DISK - IBM/MS Extension
                                               ; ❸ GET DRIVE PARAMETERS
                                               ; (DL - drive, DS:SI - buffer)
```

Листинг 8-4. Получение параметров диска посредством дисковой службы BIOS

Небольшой размер MBR, 512 байт, ограничивает функциональность реализуемого в ней кода. Поэтому буткит загружает для выполнения дополнительный код, называемый вредоносным загрузчиком, который размещается в скрытом хранилище в конце жёсткого диска. Чтобы получить координаты этого хранилища на диске, код MBR использует расширенную операцию получения параметров диска с кодом `48h` из таблицы 8-1. Она возвращает сведения о размере и геометрии жёсткого диска, позволяющие буткиту вычислить смещение дополнительного кода.

В листинге 8-4 показан автоматически созданный IDA Pro комментарий к инструкции `INT 13h` ❸. При анализе кода IDA Pro распознаёт параметры, передаваемые обработчику дисковой службы BIOS, и создаёт комментарий с названием запрошенной операции дискового ввода-вывода и именами регистров, через которые передаются параметры. Этот код MBR выполняет `INT 13h` с параметром `48h` ❶. При выполнении функция заполняет специальную структуру `EXTENDED_GET_PARAMS`, содержащую параметры диска. Адрес структуры хранится в регистре `si` ❷.

Исследование EXTENDED_GET_PARAMS

Структура `EXTENDED_GET_PARAMS` приведена в листинге 8-5.

```
typedef struct _EXTENDED_GET_PARAMS {
   WORD bResultSize;              // размер результата
   WORD InfoFlags;                // информационные флаги
   DWORD CylNumber;               // число физических цилиндров диска
   DWORD HeadNumber;              // число физических головок диска
   DWORD SectorsPerTrack;         // число секторов на дорожке
   ❶ QWORD TotalSectors;          // общее число секторов диска

   ❷ WORD BytesPerSector;         // байт в секторе
} EXTENDED_GET_PARAMS, *PEXTENDED_GET_PARAMS;
```

Листинг 8-5. Структура EXTENDED_GET_PARAMS

Из возвращённой структуры буткит фактически использует только число секторов жёсткого диска ❶ и размер сектора диска в байтах ❷. Перемножив эти два значения, буткит вычисляет полный размер жёсткого диска в байтах, а затем использует результат для поиска скрытого хранилища в конце диска.

Чтение секторов вредоносного загрузчика

Получив параметры жёсткого диска и вычислив смещение скрытого хранилища, код MBR буткита читает скрытые данные с диска посредством операции расширенного чтения дисковой службы BIOS. Эти данные представляют собой вредоносный загрузчик следующего этапа, предназначенный для обхода проверок безопасности ОС и загрузки вредоносного драйвера режима ядра. В листинге 8-6 показан код, считывающий его в оперативную память.

```
seg000:7C4C read_loop:                          ; CODE XREF: seg000:7C5D j
seg000:7C4C             ❶ call    read_sector
seg000:7C4F                mov     si, 7D1Dh
seg000:7C52                mov     cx, ds:word_7D1B
seg000:7C56                rep movsb
seg000:7C58                mov     ax, ds:word_7D19
seg000:7C5B                test    ax, ax
seg000:7C5D                jnz     short read_loop
seg000:7C5F                popa
seg000:7C60             ❷ jmp     far boot_loader
```

Листинг 8-6. Код загрузки дополнительного вредоносного загрузчика с диска

В цикле `read_loop` код многократно читает секторы жёсткого диска с помощью функции `read_sector` ❶ и сохраняет их в ранее выделенном буфере памяти. Затем инструкция `jmp far` ❷ передаёт управление вредоносному загрузчику.

В коде функции `read_sector`, приведённом в листинге 8-7, видно применение `INT 13h` с параметром `42h`, соответствующим операции расширенного чтения.

```
seg000:7C65 read_sector     proc near
seg000:7C65                 pusha
seg000:7C66             ❶ mov     ds:disk_address_packet.PacketSize, 10h
seg000:7C6B             ❷ mov     byte ptr ds:disk_address_packet.SectorsToTransfer, 1
seg000:7C70                 push    cs
seg000:7C71                 pop     word ptr ds:disk_address_packet.TargetBuffer+2
seg000:7C75             ❸ mov     word ptr ds:disk_address_packet.TargetBuffer, 7D17h
seg000:7C7B                 push    large [dword ptr ds:drive_param.TotalSectors_l]
seg000:7C80             ❹ pop     large [ds:disk_address_packet.StartLBA_l]
seg000:7C85                 push    large [dword ptr ds:drive_param.TotalSectors_h]

seg000:7C8A             ❺ pop     large [ds:disk_address_packet.StartLBA_h]
seg000:7C8F                 inc     eax
seg000:7C91                 sub     ds:disk_address_packet.StartLBA_l, eax
seg000:7C96                 sbb     ds:disk_address_packet.StartLBA_h, 0
seg000:7C9C                 mov     ah, 42h
seg000:7C9E             ❻ mov     si, 7CE9h
seg000:7CA1                 mov     dl, ds:drive_no
seg000:7CA5             ❼ int     13h             ; DISK - IBM/MS Extension
                                                    ; EXTENDED READ
                                                    ; (DL - drive, DS:SI - disk address packet)
seg000:7CA7                 popa
seg000:7CA8                 retn
seg000:7CA8 read_sector     endp
```

Листинг 8-7. Чтение секторов с диска

Перед выполнением `INT 13h` ❼ код буткита инициализирует структуру `DISK_ADDRESS_PACKET` необходимыми параметрами: размером структуры ❶, числом передаваемых секторов ❷, адресом буфера для результата ❸ и адресами читаемых секторов ❹ и ❺. Адрес этой структуры передаётся обработчику INT 13h через регистры `ds` и `si` ❻. Обратите внимание на созданные вручную аннотации смещений структуры: IDA распознаёт и распространяет их. Дисковая служба BIOS использует `DISK_ADDRESS_PACKET`, чтобы однозначно определить, какие секторы жёсткого диска следует прочитать. Полная структура `DISK_ADDRESS_PACKET` с комментариями приведена в листинге 8-8.

```
typedef struct _DISK_ADDRESS_PACKET {
   BYTE PacketSize;                // размер структуры
   BYTE Reserved;
   WORD SectorsToTransfer;         // число секторов для чтения/записи
   DWORD TargetBuffer;             // segment:offset буфера данных
   QWORD StartLBA;                 // LBA-адрес начального сектора
} DISK_ADDRESS_PACKET, *PDISK_ADDRESS_PACKET;
```

Листинг 8-8. Структура DISK_ADDRESS_PACKET

После чтения загрузчика в буфер памяти буткит выполняет его.

На этом анализ кода MBR завершён, и теперь мы перейдём к разбору другой важнейшей части MBR - таблицы разделов. Полную версию дизассемблированной и прокомментированной вредоносной MBR можно загрузить по адресу nostarch.com.

Анализ таблицы разделов заражённой MBR

Таблица разделов MBR часто становится целью буткитов, поскольку содержащиеся в ней данные, хотя и немногочисленны, играют важнейшую роль в логике загрузки. Как рассказывалось в главе 5, таблица разделов расположена по смещению `0x1BE` в MBR и состоит из четырёх элементов размером `0x10` байт каждый. Она перечисляет доступные на жёстком диске разделы, описывает их тип и расположение и указывает, куда код MBR должен передать управление после завершения. Обычно единственное назначение легитимного кода MBR - найти в этой таблице активный раздел, то есть раздел, отмеченный соответствующим битовым флагом и содержащий VBR, и загрузить его. Перехватить этот поток выполнения на очень раннем этапе загрузки можно, просто изменив сведения в таблице и не затрагивая сам код MBR; такой метод реализует буткит Olmasco, рассматриваемый в главе 10.

Это иллюстрирует важный принцип разработки буткитов и руткитов: если можно достаточно скрытно изменить данные и тем самым направить поток управления в нужную сторону, такой подход предпочтительнее изменения кода. Разработчик вредоносной программы избавляется от необходимости тестировать новый изменённый код - хороший пример того, как повторное использование кода повышает надёжность.

Сложные структуры данных вроде MBR и VBR, как известно, дают атакующим множество возможностей трактовать их как своего рода байт-код, а потребляющий эти данные машинный код - как виртуальную машину, программируемую входными данными. Подход к безопасности с точки зрения теории языков (language-theoretic security, LangSec; langsec.org) объясняет причины этого явления.

Умение читать и понимать таблицу разделов MBR необходимо для обнаружения такого раннего перехвата буткитом. Рассмотрите таблицу разделов на рисунке 8-4, где каждая строка размером 16, или 10h, байт представляет один элемент таблицы.

```
7DBE   80 20 21 00 07 DF 13 0C   00 08 00 00 00 20 03 00
7DCE   00 DF 14 0C 07 FE FF FF   00 28 03 00 00 D0 FC 04
7DDE   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00
7DEE   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00
```

Рисунок 8-4. Таблица разделов MBR

Как видите, в таблице два элемента - две верхние строки, а значит, на диске всего два раздела. Первый элемент начинается по адресу `0x7DBE`; его самый первый байт ❶ показывает, что раздел активен, поэтому код MBR должен загрузить и выполнить его VBR, то есть первый сектор этого раздела. Байт по смещению `0x7DC2` ❷ описывает тип раздела - конкретный тип файловой системы, который ОС, загрузчик или другой низкоуровневый код доступа к диску ожидает там обнаружить. В данном случае `0x07` соответствует NTFS компании Microsoft. Подробнее о типах разделов см. в разделе «Процесс загрузки Windows» на странице 60 оригинального издания.

Следующее двойное слово по адресу `0x7DC5` ❸ в элементе таблицы показывает, что раздел начинается со смещения `0x800` секторов от начала жёсткого диска. Последнее двойное слово ❹ задаёт размер раздела в секторах: `0x32000`. Конкретный пример с рисунка 8-4 подробно представлен в таблице 8-2. В столбцах начального смещения и размера раздела фактические значения указаны в секторах, а в скобках - в байтах.

Таблица 8-2. Содержимое таблицы разделов MBR

Индекс раздела	Активен	Тип	Начальное смещение, секторы (байты)	Размер раздела, секторы (байты)
0	True	NTFS (`0x07`)	`0x800` (`0x100000`)	`0x32000` (`0x6400000`)
1	False	NTFS (`0x07`)	`0x32800` (`0x6500000`)	`0x4FCD000` (`0x9F9A00000`)
2	Н/Д	Н/Д	Н/Д	Н/Д
3	Н/Д	Н/Д	Н/Д	Н/Д

Восстановленная таблица разделов указывает, где продолжить анализ последовательности загрузки, а именно - где находится VBR. Координаты VBR хранятся в столбце начального смещения элемента основного раздела. В данном случае VBR расположена со смещением `0x100000` байт от начала жёсткого диска, и именно там следует продолжить анализ.

Методы анализа VBR

В этом разделе рассмотрим подходы к статическому анализу VBR с помощью IDA и сосредоточимся на важном понятии VBR - блоке параметров BIOS (BIOS parameter block, BPB), который играет значительную роль в процессе загрузки и заражении буткитами. VBR также часто становится целью буткитов, как кратко объяснялось в главе 7. В главе 12 подробнее рассматривается буткит Gapz, заражающий VBR для сохранения присутствия в заражённой системе. Буткит Rovnix из главы 11 также использует VBR для заражения системы.

Загрузите VBR в дизассемблер практически так же, как MBR, поскольку она тоже выполняется в реальном режиме. Загрузите файл VBR `vbr_sample_ch8.bin` из каталога примеров главы 8 как двоичный модуль по адресу `0:7C00h` в 16-разрядном режиме дизассемблирования.

Анализ IPL

Основное назначение VBR - найти начальный загрузчик программ (IPL) и прочитать его в оперативную память. Расположение IPL на жёстком диске задаётся структурой `BIOS_PARAMETER_BLOCK_NTFS`, рассмотренной в главе 5. Она хранится непосредственно в VBR и содержит ряд полей, определяющих геометрию тома NTFS: число байтов в секторе, число секторов в кластере, расположение главной файловой таблицы и другие параметры.

Поле `HiddenSectors`, содержащее количество секторов от начала жёсткого диска до начала тома NTFS, определяет фактическое расположение IPL. VBR предполагает, что том NTFS начинается с VBR, непосредственно за которой следует IPL. Поэтому код VBR загружает IPL, получая значение поля `HiddenSectors`, увеличивая его на 1 и читая `0x2000` байт, что соответствует 16 секторам, с вычисленного смещения. Загрузив IPL с диска, код VBR передаёт ему управление.

В листинге 8-9 показана часть структуры блока параметров BIOS в нашем примере.

```
seg000:000B bpb     dw 200h      ; SectorSize
seg000:000D         db 8         ; SectorsPerCluster
seg000:001E         db 3 dup(0)  ; reserved
seg000:0011         dw 0         ; RootDirectoryIndex
seg000:0013         dw 0         ; NumberOfSectorsFAT
seg000:0015         db 0F8h      ; MediaId
seg000:0016         db 2 dup(0)  ; Reserved2
seg000:0018         dw 3Fh       ; SectorsPerTrack
seg000:001A         dw 0FFh      ; NumberOfHeads
seg000:001C         dd 800h      ; HiddenSectors ❶
```

Листинг 8-9. Блок параметров BIOS в VBR

Значение `HiddenSectors` ❶ равно `0x800`, что соответствует начальному смещению активного раздела на диске в таблице 8-2. Следовательно, IPL расположен со смещением `0x801` от начала диска. Буткиты используют эти сведения для перехвата управления во время загрузки. Например, буткит Gapz изменяет содержимое поля `HiddenSectors`, чтобы код VBR читал и выполнял вредоносный IPL вместо легитимного. Rovnix применяет другую стратегию: изменяет код легитимного IPL. Обе манипуляции перехватывают управление на раннем этапе загрузки системы.

Оценка других компонентов буткита

Получив управление, IPL загружает `bootmgr`, хранящийся в файловой системе тома. После этого могут вступить в действие другие компоненты буткита, например вредоносные загрузчики и драйверы режима ядра. Полный анализ этих модулей выходит за рамки главы, но мы кратко обозначим некоторые подходы.

Вредоносные загрузчики

Вредоносные загрузчики являются важной частью буткитов. Их основные задачи - пережить переключение режимов выполнения процессора, обойти проверки безопасности ОС, например обязательную проверку подписи драйверов, и загрузить вредоносные драйверы режима ядра. Они реализуют функциональность, которая из-за ограничений размера не помещается в MBR и VBR, и хранятся на жёстком диске отдельно. Буткиты размещают свои загрузчики в скрытых хранилищах либо в конце жёсткого диска, где обычно остаётся неиспользуемое пространство, либо в свободном дисковом пространстве между разделами, если оно имеется.

Вредоносный загрузчик может содержать разный код для выполнения в разных режимах процессора:

• **16-разрядный реальный режим:** функциональность перехвата прерывания 13h.
• **32-разрядный защищённый режим:** обход проверок безопасности 32-разрядной версии ОС.
• **64-разрядный защищённый режим (длинный режим):** обход проверок безопасности 64-разрядной версии ОС.

Однако дизассемблер IDA Pro не может хранить в одной базе данных IDA код, дизассемблированный в разных режимах, поэтому для разных режимов выполнения придётся поддерживать отдельные версии базы данных IDA Pro.

Драйверы режима ядра

В большинстве случаев загружаемые буткитами драйверы режима ядра представляют собой действительные PE-образы. Они реализуют функциональность руткита, позволяющую вредоносной программе избегать обнаружения защитным ПО, предоставляют скрытые каналы связи и выполняют другие задачи. Современные буткиты обычно содержат две версии драйвера режима ядра, скомпилированные для платформ x86 и x64. Такие модули можно анализировать традиционными методами статического анализа исполняемых образов. IDA Pro хорошо справляется с загрузкой подобных файлов и предоставляет множество дополнительных инструментов и сведений для их анализа. Однако далее мы рассмотрим, как автоматизировать анализ буткитов с помощью возможностей IDA Pro, предварительно обрабатывая их во время загрузки.

Расширенное использование IDA Pro: создание собственного загрузчика MBR

Одна из наиболее впечатляющих особенностей дизассемблера IDA Pro - широта поддержки форматов файлов и архитектур процессоров. Загрузка определённых типов исполняемых файлов реализуется в специальных модулях, называемых загрузчиками. По умолчанию IDA Pro содержит ряд загрузчиков для наиболее распространённых типов исполняемых файлов, включая PE в Windows, ELF в Linux, Mach-O в macOS и форматы образов прошивок. Список доступных загрузчиков можно получить, просмотрев содержимое каталога `$IDADIR\loaders`, где `$IDADIR` - каталог установки дизассемблера. Находящиеся там файлы являются загрузчиками, а их имена соответствуют платформам и двоичным форматам. Расширения файлов имеют следующие значения:

• `ldw` - двоичная реализация загрузчика для 32-разрядной версии IDA Pro.
• `l64` - двоичная реализация загрузчика для 64-разрядной версии IDA Pro.
• `py` - реализация загрузчика на Python для обеих версий IDA Pro.

На момент написания главы загрузчика MBR или VBR по умолчанию не было, поэтому приходится указывать IDA загружать MBR или VBR как двоичный модуль. В этом разделе показано, как создать для IDA Pro собственный загрузчик MBR на Python, который загружает MBR в 16-разрядном режиме дизассемблера по адресу `0x7C00` и разбирает таблицу разделов.

Знакомство с loader.hpp

Начать следует с файла `loader.hpp`, входящего в SDK IDA Pro и содержащего множество полезных сведений о загрузке исполняемых файлов в дизассемблер. В нём определены используемые структуры и типы, перечислены прототипы процедур обратного вызова и описаны их параметры. Согласно `loader.hpp`, в загрузчике следует реализовать следующие процедуры:

• `accept_file` проверяет, относится ли загружаемый файл к поддерживаемому формату.
• `load_file` выполняет фактическую загрузку файла в дизассемблер, то есть разбирает формат и отображает содержимое файла в новую базу данных.
• `save_file` - необязательная процедура. Если она реализована, то создаёт исполняемый файл из дизассемблированного представления при выборе команды меню `File > Produce File > Create EXE File`.
• `move_segm` - необязательная процедура, выполняемая при перемещении пользователем сегмента внутри базы данных. В основном она применяется, когда образ содержит сведения о перемещениях, которые пользователь должен учитывать. Поскольку в MBR перемещений нет, эту процедуру можно пропустить, но при создании загрузчика двоичных файлов PE или ELF это было бы невозможно.
• `init_loader_options` - необязательная процедура, которая после выбора загрузчика запрашивает у пользователя дополнительные параметры загрузки определённого типа файла. Её тоже можно пропустить, поскольку специальных параметров добавлять не требуется.

Теперь рассмотрим фактическую реализацию этих процедур в нашем загрузчике MBR.

Реализация accept_file

В процедуре `accept_file`, показанной в листинге 8-10, мы проверяем, является ли рассматриваемый файл главной загрузочной записью.

```
def accept_file(li, n):
    # проверка размера файла
    file_size = li.size()
    if file_size < 512:
        ❶ return 0

    # проверка сигнатуры MBR
    li.seek(510, os.SEEK_SET)

    mbr_sign = li.read(2)
    if mbr_sign[0] != '\x55' or mbr_sign[1] != '\xAA':
        ❷ return 0

    # все проверки пройдены
    ❸ return 'MBR'
```

Листинг 8-10. Реализация accept_file

Формат MBR довольно прост, поэтому для проверки достаточно следующих признаков:

• **Размер файла:** файл должен занимать не менее 512 байт, что соответствует минимальному размеру сектора жёсткого диска.
• **Сигнатура MBR:** действительная MBR должна заканчиваться байтами `0xAA55`.

Если условия выполнены и файл распознан как MBR, код возвращает строку с именем загрузчика ❸. Если файл не является MBR, возвращается 0 ❶❷.

Реализация load_file

После того как `accept_file` возвращает ненулевое значение, IDA Pro пытается загрузить файл, выполняя реализованную в загрузчике процедуру `load_file`. Она должна выполнить следующие действия:

1. Прочитать весь файл в буфер.
2. Создать и инициализировать новый сегмент памяти, в который сценарий загрузит содержимое MBR.
3. Назначить самое начало MBR точкой входа для дизассемблирования.
4. Разобрать содержащуюся в MBR таблицу разделов.

Реализация `load_file` показана в листинге 8-11.

```
def load_file(li):
    # выбор модуля процессора ПК
    ❶ idaapi.set_processor_type("metapc", SETPROC_ALL|SETPROC_FATAL)

    # чтение MBR в буфер
    ❷ li.seek(0, os.SEEK_SET); buf = li.read(li.size())
    mbr_start = 0x7C00       # начало сегмента
    mbr_size = len(buf)      # размер сегмента
    mbr_end  = mbr_start + mbr_size

    # создание сегмента
    ❸ seg = idaapi.segment_t()
    seg.startEA = mbr_start
    seg.endEA   = mbr_end
    seg.bitness = 0 # 16-разрядный
    ❹ idaapi.add_segm_ex(seg, "seg0", "CODE", 0)

    # копирование байтов
    ❺ idaapi.mem2base(buf, mbr_start, mbr_end)

    # добавление точки входа
    idaapi.add_entry(mbr_start, mbr_start, "start", 1)

    # разбор таблицы разделов
    ❻ struct_id = add_struct_def()
    struct_size = idaapi.get_struc_size(struct_id)
    ❼ idaapi.doStruct(start + 0x1BE, struct_size, struct_id)
```

Листинг 8-11. Реализация load_file

Сначала задаётся тип процессора `metapc` ❶, соответствующий универсальному семейству ПК и предписывающий IDA дизассемблировать двоичный файл как код команд IBM PC. Затем MBR читается в буфер ❷, а посредством API `segment_t` ❸ создаётся сегмент памяти. Этот вызов выделяет пустую структуру `seg`, описывающую создаваемый сегмент, после чего она заполняется фактическими значениями.

Начальный адрес сегмента задаётся равным `0x7C00`, как в разделе «Загрузка MBR в IDA Pro» на странице 96 оригинального издания, а размер сегмента - соответствующим размеру MBR. Флаг `bitness` структуры устанавливается равным 0, сообщая IDA, что новый сегмент является

16-разрядным; значение 1 соответствует 32-разрядным сегментам, а 2 - 64-разрядным. Затем вызовом API `add_segm_ex` ❹ в базу данных дизассемблирования добавляется новый сегмент. API `add_segm_ex` принимает структуру, описывающую создаваемый сегмент, имя сегмента `seg0`, класс сегмента `CODE` и флаги, оставленные равными 0.

После этого вызовом ❺ содержимое MBR копируется во вновь созданный сегмент и добавляется указатель точки входа.

Далее добавляется автоматический разбор таблицы разделов MBR посредством вызова API `doStruct` ❼ со следующими параметрами: адрес начала таблицы разделов, размер таблицы в байтах и идентификатор структуры, к которой следует привести таблицу. Реализованная в загрузчике функция `add_struct_def` ❻ создаёт эту структуру. Она импортирует в базу данных структуры `PARTITION_TABLE_ENTRY`, описывающие таблицу разделов.

Создание структуры таблицы разделов

В листинге 8-12 определена функция `add_struct_def`, создающая структуру `PARTITION_TABLE_ENTRY`.

```
def add_struct_def(li, neflags, format):
    # добавление структуры PARTITION_TABLE_ENTRY к типам IDA
    sid_partition_entry = AddStrucEx(-1, "PARTITION_TABLE_ENTRY", 0)

    # добавление полей структуры
    AddStrucMember(sid_partition_entry, "status", 0, FF_BYTE, -1, 1)
    AddStrucMember(sid_partition_entry, "chsFirst", 1, FF_BYTE, -1, 3)
    AddStrucMember(sid_partition_entry, "type", 4, FF_BYTE, -1, 1)
    AddStrucMember(sid_partition_entry, "chsLast", 5, FF_BYTE, -1, 3)

    AddStrucMember(sid_partition_entry, "lbaStart", 8, FF_DWRD, -1, 4)
    AddStrucMember(sid_partition_entry, "size", 12, FF_DWRD, -1, 4)

    # добавление структуры PARTITION_TABLE к типам IDA
    sid_table = AddStrucEx(-1, "PARTITION_TABLE", 0)
    AddStrucMember(sid_table, "partitions", 0, FF_STRU, sid, 64)
    return sid_table
```

Листинг 8-12. Импорт структур данных в базу данных дизассемблирования

Завершив модуль загрузчика, скопируйте его в каталог `$IDADIR\loaders` как файл `mbr.py`. Когда пользователь попытается загрузить MBR в дизассемблер, появится диалог, показанный на рисунке 8-5, подтверждающий, что загрузчик успешно распознал образ MBR. Нажатие `OK` выполняет реализованную в загрузчике процедуру `load_file`, которая применяет к загружаемому файлу описанные выше настройки.

> **Примечание.** При разработке собственных загрузчиков для IDA Pro ошибки реализации сценария могут привести к аварийному завершению IDA Pro. В таком случае просто удалите сценарий загрузчика из каталога `loaders` и перезапустите дизассемблер.

В этом разделе был показан небольшой пример возможностей разработки расширений дизассемблера. Более полное справочное руководство по созданию расширений IDA Pro см. в книге Криса Игла *The IDA Pro Book* (No Starch Press, 2011).

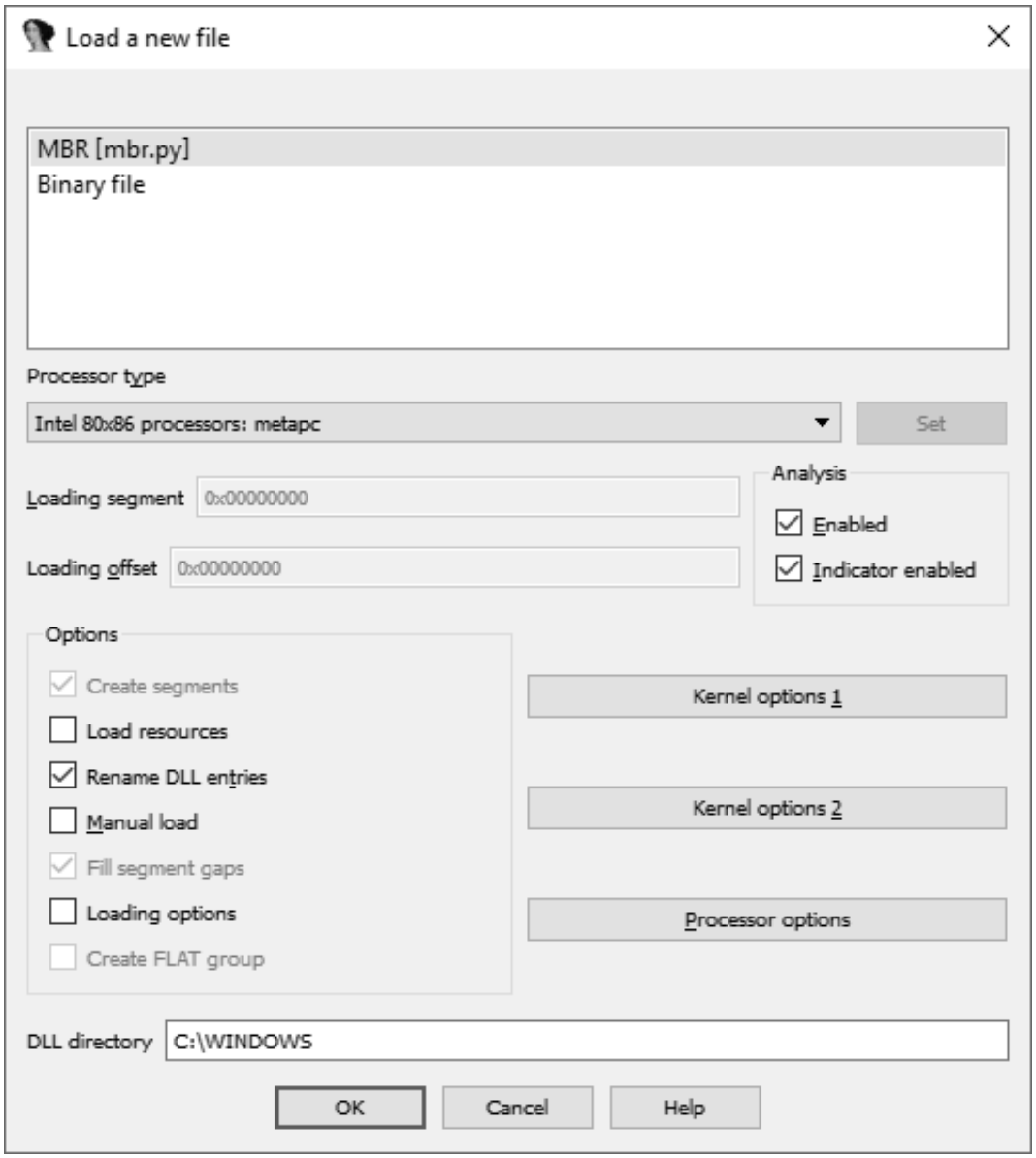


Рисунок 8-5. Выбор собственного загрузчика MBR

Заключение

В этой главе мы описали несколько простых шагов статического анализа MBR и VBR. Приведённые примеры легко распространить на любой код, выполняемый в предзагрузочной среде. Вы также увидели, что дизассемблер IDA Pro предоставляет ряд уникальных возможностей, делающих его удобным инструментом статического анализа.

С другой стороны, статический анализ имеет ограничения, прежде всего невозможность увидеть код в работе и проследить, как он обрабатывает данные. Во многих случаях статический анализ не способен ответить на все вопросы специалиста по обратной разработке. Тогда важно исследовать фактическое выполнение кода, чтобы лучше понять его функциональность или получить сведения,

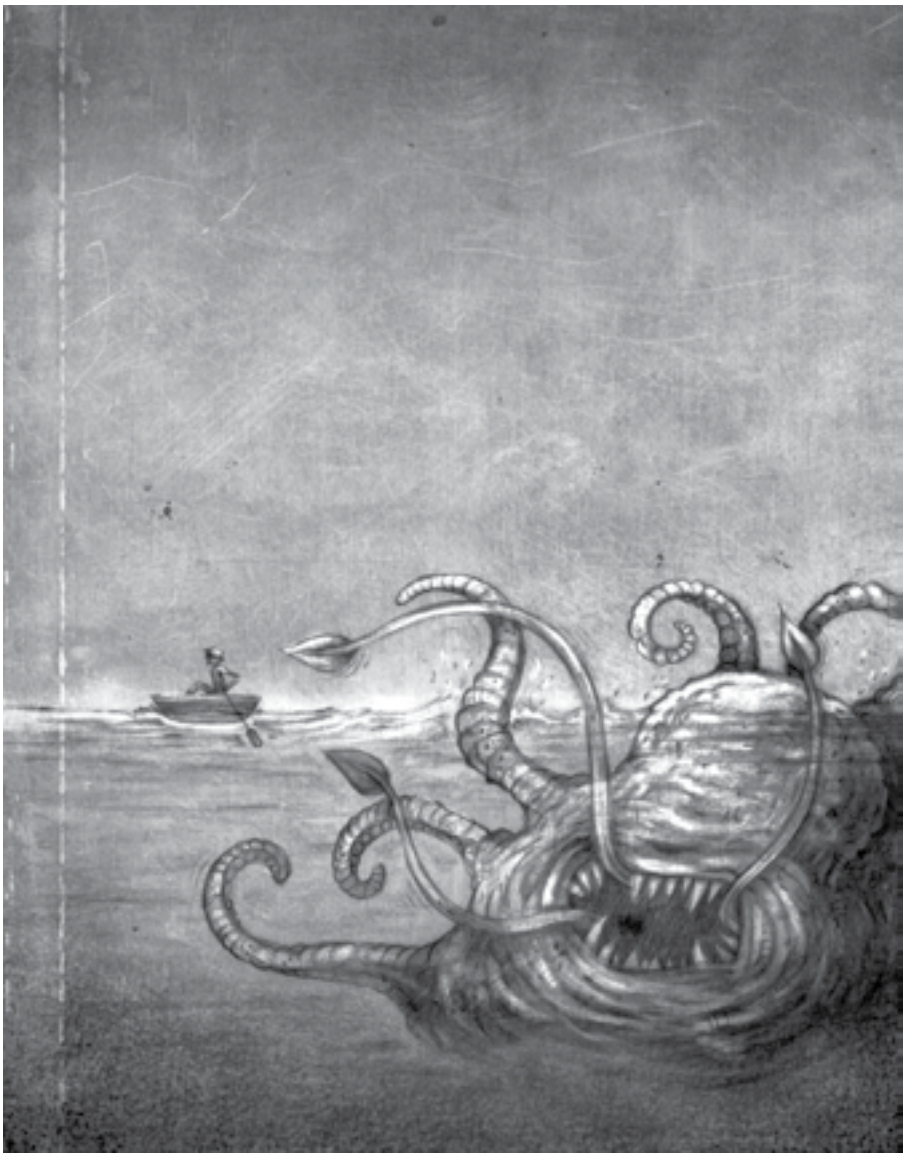
отсутствовавшие в статическом контексте, например ключи шифрования. Это подводит нас к динамическому анализу, методы и инструменты которого будут рассмотрены в следующей главе.

Упражнения

Выполните следующие упражнения, чтобы лучше усвоить материал главы. Вам потребуется загрузить образ диска по адресу nostarch.com. Для упражнений нужны дизассемблер IDA Pro и интерпретатор Python.

1. Извлеките MBR из образа, прочитав первые 512 байт и сохранив их в файл `mbr.mbr`. Загрузите извлечённую MBR в дизассемблер IDA Pro. Исследуйте и опишите код в точке входа.
2. Найдите код, расшифровывающий MBR. Какой вид шифрования используется? Найдите ключ расшифрования MBR.
3. Напишите сценарий Python, который расшифровывает остальной код MBR, и выполните его. Используйте в качестве образца код листинга 8-2.
4. Чтобы загружать дополнительный код с диска, код MBR выделяет буфер памяти. Где расположен код, выделяющий этот буфер? Сколько байтов памяти он выделяет? Где хранится указатель на выделенный буфер?
5. После выделения буфера памяти код MBR пытается загрузить с диска дополнительный код. С какого смещения и в каких секторах код MBR начинает читать эти секторы? Сколько секторов он читает?
6. Похоже, загруженные с диска данные зашифрованы. Найдите код MBR, расшифровывающий прочитанные секторы. По какому адресу будет загружен этот код MBR?
7. Извлеките зашифрованные секторы из образа диска, прочитав с найденного смещения количество байтов, определённое в упражнении 4, и сохраните их в файл `stage2.mbr`.
8. Реализуйте сценарий Python для расшифрования извлечённых секторов и выполните его. Загрузите расшифрованные данные в дизассемблер тем же способом, что и MBR, и исследуйте результат.
9. Найдите таблицу разделов в MBR. Сколько в ней разделов? Какой из них активен? Где в образе расположены эти разделы?
10. Извлеките VBR активного раздела из образа, прочитав первые 512 байт и сохранив их в файл `vbr.vbr`. Загрузите извлечённую VBR в IDA Pro. Исследуйте и опишите код в точке входа.
11. Какое значение хранится в поле `HiddenSectors` блока параметров BIOS в VBR? По какому смещению расположен код IPL? Исследуйте код VBR и определите размер IPL, то есть сколько байтов IPL считывается.
12. Извлеките код IPL из образа диска и сохраните его в файл `ipl.vbr`. Загрузите извлечённый IPL в IDA Pro. Найдите расположение точки входа в IPL. Исследуйте и опишите код в точке входа.
13. Создайте собственный загрузчик VBR для IDA Pro, который автоматически разбирает блок параметров BIOS. Используйте структуру `BIOS_PARAMETER_BLOCK_NTFS`, определённую в главе 5.

Глава 9. Динамический анализ буткита: эмуляция и виртуализация



В главе 8 вы увидели, что статический анализ является мощным средством обратной разработки буткитов. Однако в некоторых ситуациях он не позволяет получить нужные сведения, и тогда приходится применять методы динамического анализа. Это часто верно для буткитов с зашифрованными компонентами, расшифрование которых затруднено, или для таких буткитов, как Rovnix из главы 11, которые во время выполнения используют несколько перехватов для отключения защитных механизмов ОС. Средства статического анализа не всегда способны определить, в какие модули вмешивается буткит, поэтому в таких случаях динамический анализ эффективнее.

Динамический анализ обычно опирается на средства отладки исследуемой платформы, но предзагрузочная среда не предоставляет традиционных средств отладки. Для отладки в ней, как правило, требуются специальное оборудование, программное обеспечение и знания, что превращает её в сложную задачу.

Чтобы преодолеть это препятствие, необходим дополнительный программный уровень - эмулятор или виртуальная машина (VM). Средства эмуляции и виртуализации позволяют выполнять загрузочный код в управляемой предзагрузочной среде с традиционными интерфейсами отладки.

В этой главе мы изучим оба подхода к динамическому анализу буткитов: эмуляцию с помощью Bochs и виртуализацию с помощью VMware Workstation. Оба подхода сходны: позволяют исследователю наблюдать поведение загрузочного кода непосредственно во время выполнения, дают одинаковое представление об отлаживаемом коде и одинаковый доступ к регистрам процессора и памяти.

Различие заключается в реализации. Эмулятор Bochs интерпретирует эмулируемый код целиком на виртуальном процессоре, тогда как VMware Workstation для выполнения большинства инструкций гостевой ОС использует реальный физический процессор.

Компоненты буткита, используемые для анализа в этой главе, доступны в материалах книги по адресу nostarch.com. Вам потребуются MBR в файле `mbr.mbr`, а также VBR и IPL в файле `partition0.data`.

Эмуляция с помощью Bochs

Bochs (bochs.sourceforge.net; произносится «бокс») - эмулятор платформы Intel x86-64 с открытым исходным кодом, способный эмулировать целый компьютер. Для нас особенно важно, что он предоставляет отладочный интерфейс, позволяющий трассировать эмулируемый код. Благодаря этому Bochs можно использовать для отладки модулей, выполняющихся в предзагрузочной среде, например MBR и VBR/IPL. Кроме того, Bochs работает как единый процесс пользовательского режима, поэтому для поддержки эмулируемой среды не требуется устанавливать драйверы режима ядра или специальные системные службы.

Другие средства, например эмулятор QEMU с открытым исходным кодом (wiki.qemu.org), предоставляют ту же функциональность и тоже пригодны для анализа буткитов. Мы выбрали Bochs, поскольку, согласно нашему обширному опыту, на платформах Microsoft Windows он лучше интегрируется с Hex-Rays IDA Pro. Архитектура Bochs компактнее и сосредоточена только на эмуляции платформ x86/x64. Кроме того, в Bochs встроен интерфейс отладки, позволяющий отлаживать загрузочный код без IDA Pro, хотя совместное применение с IDA Pro повышает эффективность, как будет показано далее в разделе «Совместное использование Bochs и IDA» на странице 123 оригинального издания.

Стоит отметить, что QEMU эффективнее и поддерживает больше архитектур, включая Advanced RISC Machine (ARM). Внутренний интерфейс GNU Debugger (GDB) в QEMU также позволяет начинать отладку на раннем этапе загрузки VM. Поэтому, если после этой главы вы захотите глубже изучить отладку, стоит попробовать QEMU.

Установка Bochs

Последнюю версию Bochs можно загрузить по адресу sourceforge.net. Доступны два варианта: установщик Bochs и ZIP-архив с компонентами. Установщик содержит больше компонентов и инструментов, включая рассматриваемую далее программу `bximage`, поэтому рекомендуем загрузить именно его. Установка проста: последовательно проходите этапы, оставляя значения параметров по умолчанию. Далее каталог установки Bochs называется рабочим каталогом Bochs.

Создание среды Bochs

Чтобы использовать эмулятор Bochs, сначала необходимо создать для него среду из конфигурационного файла Bochs и образа диска. Конфигурационный файл представляет собой текстовый файл со всеми основными сведениями, которые нужны эмулятору для выполнения кода: используемый образ диска, параметры процессора и так далее. Образ диска содержит гостевую ОС и эмулируемые загрузочные модули.

Создание конфигурационного файла

В листинге 9-1 представлены наиболее часто используемые параметры для отладки буткитов; эту конфигурацию Bochs мы будем применять на протяжении главы. Создайте текстовый файл и введите содержимое листинга. Либо используйте файл `bochsrc.bxrc` из материалов книги. Сохраните его под именем `bochsrc.bxrc` в рабочем каталоге Bochs. Расширение `.bxrc` означает, что файл содержит параметры конфигурации Bochs.

```
megs: 512
romimage: file="../BIOS-bochs-latest"                          ❶
vgaromimage: file="../VGABIOS-lgpl-latest"                     ❷
boot: cdrom, disk                                              ❸
ata0-master: type=disk, path="win_os.img", mode=flat, cylinders=6192, heads=16, spt=63 ❹
mouse: enabled=0                                               ❺
cpu: ips=90000000                                              ❻
```

Листинг 9-1. Пример конфигурационного файла Bochs

Первый параметр, `megs`, задаёт ограничение оперативной памяти эмулируемой среды в мегабайтах. Для отладки загрузочного кода 512 Мбайт более чем достаточно. Параметры `romimage` ❶ и `vgaromimage` ❷ задают пути к модулям BIOS и VGA-BIOS для эмулируемой среды. Bochs поставляется со стандартными модулями BIOS, но при необходимости, например при разработке прошивки, можно использовать собственные. Поскольку наша цель - отладка кода MBR и VBR, мы воспользуемся стандартным модулем BIOS. Параметр `boot` задаёт последовательность загрузочных устройств ❸. При показанных настройках Bochs сначала пытается загрузиться с компакт-диска, а при неудаче переходит к жёсткому диску. Следующий параметр, `ata0-master`, задаёт тип и характеристики жёсткого диска, эмулируемого Bochs ❹. Он содержит несколько параметров:

• `type` - тип устройства: `disk` или `cdrom`.
• `path` - путь к файлу образа диска в файловой системе узла.
• `mode` - тип образа. Параметр действителен только для дисковых устройств; подробнее он рассматривается в разделе «Совместное использование Bochs и IDA» на странице 123 оригинального издания.
• `cylinders` - количество цилиндров диска, определяющее его размер.

• `heads` - количество головок диска, определяющее его размер.
• `spt` - число секторов на дорожке, определяющее размер диска.

> **Примечание.** В следующем разделе показано, как создать образ диска с помощью входящего в Bochs инструмента `bximage`. После создания нового образа `bximage` выводит параметры, которые следует указать в параметре `ata0-master`.

Параметр `mouse` включает мышь в гостевой ОС ❺. Параметр `cpu` задаёт характеристики виртуального процессора внутри эмулятора Bochs ❻. В нашем примере `ips` указывает число эмулируемых инструкций в секунду. Его можно настраивать для изменения производительности. Например, для Bochs 2.6.8 и процессора Intel Core i7 типичное значение `ips` составляет от 85 до 95 MIPS, то есть миллионов инструкций в секунду, чему соответствует использованное здесь значение.

Создание образа диска

Для создания образа диска Bochs можно использовать либо утилиту `dd` в Unix, либо инструмент `bximage` из состава эмулятора Bochs. Мы выберем `bximage`, поскольку он работает как в Linux, так и в Windows.

Запустите инструмент создания образов дисков `bximage`. После запуска он покажет список параметров, представленный на рисунке 9-1. Введите `1`, чтобы создать новый образ ❶.

Затем инструмент спросит, следует ли создать образ дискеты или жёсткого диска. В нашем случае укажите `hd` ❷ для создания образа жёсткого диска. Далее будет запрошен тип образа. Обычно он определяет расположение данных диска в файле образа. Инструмент поддерживает несколько типов; полный список см. в документации Bochs. Выберите `flat` ❸, чтобы получить один файл образа с плоской структурой. Тогда смещение внутри файла образа соответствует смещению на диске, что позволяет легко редактировать и изменять образ.

```
c:\Program Files (x86)\Bochs>bximage.exe
========================================================================
                                bximage
  Disk Image Creation / Conversion / Resize and Commit Tool for Bochs
         $Id: bximage.cc 12690 2015-03-20 18:01:52Z vruppert $
========================================================================

1. Create new floppy or hard disk image
2. Convert hard disk image to other format (mode)
3. Resize hard disk image
4. Commit 'undoable' redolog to base image
5. Disk image info

0. Quit

Please choose one [0] 1

Create image

Do you want to create a floppy disk image or a hard disk image?
Please type hd or fd. [hd] hd

What kind of image should I create?
Please type flat, sparse, growing, vpc or vmware4. [flat] flat

Enter the hard disk size in megabytes, between 10 and 8257535
[10] 10

What should be the name of the image?
[c.img] disk_image.img

Creating hard disk image 'disk_image.img' with CHS=20/16/63

The following line should appear in your bochsrc:
  ata0-master: type=disk, path="disk_image.img", mode=flat
(The line is stored in your windows clipboard, use CTRL-V to paste)

Press any key to continue
```

Рисунок 9-1. Создание образа диска Bochs с помощью инструмента bximage

Далее необходимо указать размер диска в мегабайтах. Значение зависит от назначения Bochs. Если вы хотите установить в образ операционную систему, размер должен быть достаточным для всех файлов ОС. Если же образ нужен только для отладки загрузочного кода, достаточно 10 Мбайт ❹.

Наконец, `bximage` запрашивает имя образа - путь к файлу в файловой системе узла, где будет храниться образ ❺. Если указать только имя без полного пути, файл сохранится в каталоге Bochs. После ввода имени Bochs создаёт образ диска и выводит строку конфигурации ❻, которую следует вставить в строку `ata0-master` конфигурационного файла Bochs из листинга 9-1. Чтобы избежать путаницы, либо укажите полный путь к файлу образа в `bximage`, либо скопируйте созданный файл образа в каталог с конфигурационным файлом. Тогда Bochs сможет найти и загрузить его.

Заражение образа диска

Создав образ, можно заразить диск буткитом одним из двух способов. Первый вариант - установить гостевую ОС в образ диска Bochs, а затем выполнить инфектор буткита в гостевой среде. Вредоносная программа заразит образ буткитом. Этот подход позволяет глубже анализировать вредоносное ПО, поскольку оно устанавливает в гостевую систему все компоненты, включая буткит и драйверы режима ядра. Но у него есть недостатки:

• Созданный ранее образ диска должен быть достаточно большим для размещения ОС.
• Эмуляция инструкций во время установки ОС и выполнения вредоносной программы существенно увеличивает время работы.
• Некоторые современные вредоносные программы реализуют противодействие эмуляции: обнаруживают выполнение в эмуляторе и завершаются, не заражая систему.

По этим причинам мы выберем второй вариант: заразим образ диска, извлекая компоненты буткита, то есть MBR, VBR и IPL, из вредоносной программы и записывая их непосредственно в образ. Этот подход требует значительно меньшего диска и обычно работает гораздо быстрее. Но он не позволяет наблюдать и анализировать другие компоненты вредоносной программы, например драйверы режима ядра, и требует предварительного понимания программы и её архитектуры. Мы выбираем его ещё и потому, что он даёт более глубокое представление об использовании Bochs для динамического анализа.

Запись MBR в образ диска

Убедитесь, что загрузили и сохранили код `mbr.mbr` из материалов по адресу nostarch.com. Код Python из листинга 9-2 записывает вредоносную MBR в образ диска. Скопируйте его в текстовый редактор и сохраните как отдельный файл Python.

```
# чтение MBR из файла
mbr_file = open("path_to_mbr_file", "rb") ❶
mbr = mbr_file.read()
mbr_file.close()

# запись MBR в самое начало образа диска
disk_image_file = open("path_to_disk_image", "r+b") ❷
disk_image_file.seek(0)
disk_image_file.write(mbr) ❸
disk_image_file.close()
```

Листинг 9-2. Запись кода MBR в образ диска

В этом примере вместо `path_to_mbr_file` ❶ укажите расположение файла MBR, а вместо `path_to_disk_image` ❷ - расположение образа диска, после чего сохраните код в файле с расширением `.py`. Выполните команду `python path_to_the_script_file.py`, и интерпретатор Python выполнит код в Bochs. Записанная в образ MBR ❸ содержит в таблице разделов только один активный раздел с номером 0, как показано в таблице 9-1.

Таблица 9-1. Таблица разделов MBR

Номер раздела	Тип	Начальный сектор	Размер раздела в секторах
0	`0x80` (загрузочный)	`0x10` ❶	`0x200`
1	0 (раздел отсутствует)	0	0
2	0 (раздел отсутствует)	0	0
3	0 (раздел отсутствует)	0	0

Далее необходимо записать VBR и IPL в образ диска. Загрузите и сохраните код `partition0.data` из материалов по адресу nostarch.com. Эти модули следует записать по смещению ❶ из таблицы 9-1, соответствующему начальному смещению активного раздела.

Запись VBR и IPL в образ диска

Чтобы записать VBR и IPL в образ, введите код листинга 9-3 в текстовом редакторе и сохраните его как сценарий Python.

```
# чтение VBR и IPL из файла
vbr_file = open("path_to_vbr_file", "rb") ❶
vbr = vbr_file.read()
vbr_file.close()

# запись VBR и IPL по смещению 0x2000
disk_image_file = open("path_to_disk_image", "r+b") ❷
disk_image_file.seek(0x10 * 0x200)
disk_image_file.write(vbr)
disk_image_file.close()
```

Листинг 9-3. Запись VBR и IPL в образ диска

Как и в листинге 9-2, перед запуском сценария замените `path_to_vbr_file` ❶ путём к файлу VBR, а `path_to_disk_image` ❷ - расположением образа.

После выполнения сценария образ диска готов к отладке в Bochs. Мы успешно записали в образ вредоносную MBR и VBR/IPL и можем анализировать их в отладчике Bochs.

Использование встроенного отладчика Bochs

Отладчик Bochs - отдельное приложение `bochsdbg.exe` с интерфейсом командной строки. Поддерживаемые им функции, включая точки останова, работу с памятью, трассировку и дизассемблирование, позволяют исследовать загрузочный код на предмет вредоносной активности или расшифровывать полиморфный код MBR. Чтобы начать сеанс отладки, вызовите `bochsdbg.exe` из командной строки, указав путь к конфигурационному файлу Bochs `bochsrc.bxrc`:

```
bochsdbg.exe -q -f bochsrc.bxrc
```

Эта команда запускает виртуальную машину и открывает консоль отладки. Сначала установим точку останова в начале загрузочного кода, чтобы отладчик остановил выполнение MBR на первой инструкции и дал возможность проанализировать код. Первая инструкция MBR находится по адресу `0x7c00`, поэтому введите `lb 0x7c00`. Для продолжения выполнения используется команда `c`, как показано на рисунке 9-2. Для вывода дизассемблированных инструкций с текущего адреса применяется команда `u`; например, на рисунке показаны первые 10 инструкций, выведенные командой `u /10`.

```
C:\Program Files (x86)\Bochs\Win_Infected>..\bochsdbg -q -f bochsrc.bxrc
========================================================================
                    Bochs x86 Emulator 2.6.8
            Built from SVN snapshot on May 3, 2015
              Compiled on May  3 2015 at 10:18:44
========================================================================
00000000000i[      ] reading configuration from bochsrc.bxrc
00000000000i[      ] installing win32 module as the Bochs GUI
00000000000i[      ] using log file bochsout.txt
Next at t=0
(0) [0x0000fffffff0] f000:fff0 (unk. ctxt): jmpf 0xf000:e05b          ; ea5be000f0
<bochs:1> lb 0x7c00
<bochs:2> c
(0) Breakpoint 1, 0x0000000000007c00 in ?? ()
Next at t=277379862
(0) [0x000000007c00] 0000:7c00 (unk. ctxt): xor ax, ax                ; 33c0
<bochs:3> u /10
00007c00: (                   ): xor ax, ax                 ; 33c0
00007c02: (                   ): mov ss, ax                 ; 8ed0
00007c04: (                   ): mov sp, 0x7c00             ; bc007c
00007c07: (                   ): sti                        ; fb
00007c08: (                   ): push ax                    ; 50
00007c09: (                   ): pop es                     ; 07
00007c0a: (                   ): push ax                    ; 50
00007c0b: (                   ): pop ds                     ; 1f
00007c0c: (                   ): cld                        ; fc
00007c0d: (                   ): mov si, 0x7c1b             ; be1b7c
<bochs:4>
```

Рисунок 9-2. Интерфейс командной строки отладчика Bochs

Полный список команд отладчика можно получить командой `help` или найти в документации по адресу bochs.sourceforge.net. Ниже перечислены некоторые наиболее полезные команды:

- `c` - продолжить выполнение.
- `s [count]` - выполнить `count` инструкций пошагово; значение по умолчанию равно 1.
- `q` - завершить работу отладчика и выполнение.
- `Ctrl-C` - остановить выполнение и вернуться к приглашению командной строки.
- `lb addr` - установить точку останова по линейному адресу инструкции.
- `info break` - показать состояние всех текущих точек останова.
- `bpe n` - включить точку останова.
- `bpd n` - отключить точку останова.
- `del n` - удалить точку останова.

Хотя встроенного отладчика Bochs достаточно для базового динамического анализа, связывание с IDA расширяет возможности, главным образом потому, что навигация по коду в IDA значительно мощнее пакетной отладки. В сеансе IDA можно также продолжить статический анализ созданной базы данных IDA Pro и использовать такие возможности, как декомпилятор.

Совместное использование Bochs и IDA

Теперь, когда заражённый образ диска подготовлен, запустим Bochs и начнём эмуляцию. Начиная с версии 5.4, IDA Pro предоставляет интерфейс к отладчику DBG, который можно использовать совместно с Bochs для отладки гостевых операционных систем. Чтобы запустить отладчик Bochs в IDA Pro, откройте IDA Pro и выберите `Debugger > Run > Local Bochs debugger`.

Откроется диалог параметров, показанный на рисунке 9-3. В поле `Application` укажите путь к созданному ранее конфигурационному файлу Bochs.

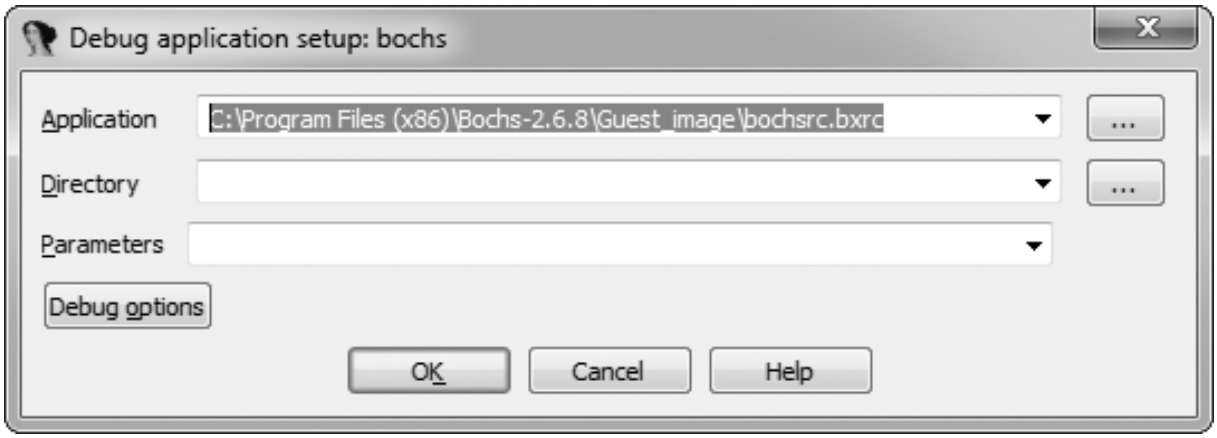


Рисунок 9-3. Указание пути к конфигурационному файлу Bochs

Теперь зададим дополнительные параметры. Нажмите `Debug options`, затем перейдите к `Set specific options`. Появится диалог, похожий на показанный на рисунке 9-4, с тремя режимами работы Bochs:

- `Disk image` - запустить Bochs и выполнить образ диска.
- `IDB` - эмулировать выбранную часть кода внутри Bochs.
- `PE` - загрузить и эмулировать PE-образ внутри Bochs.

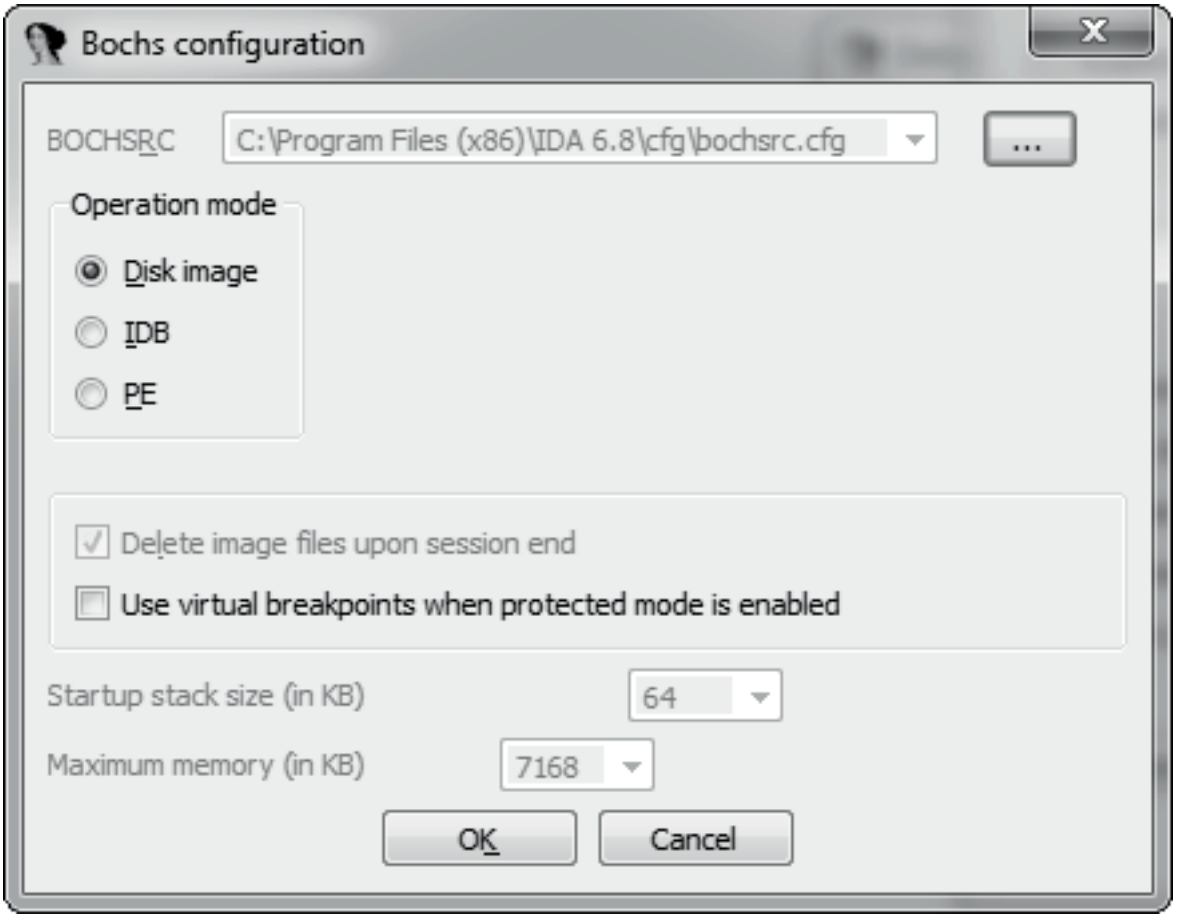


Рисунок 9-4. Выбор режима работы Bochs

В нашем случае выберите `Disk image` ❶, чтобы Bochs загрузил и выполнил созданный и заражённый ранее образ диска.

Затем IDA Pro запускает Bochs с указанными параметрами. Благодаря установленной ранее точке останова выполнение остановится на первой инструкции MBR по адресу `0000:7c00h`. После этого загрузочные компоненты можно отлаживать с помощью стандартного интерфейса отладчика IDA Pro, показанного на рисунке 9-5.

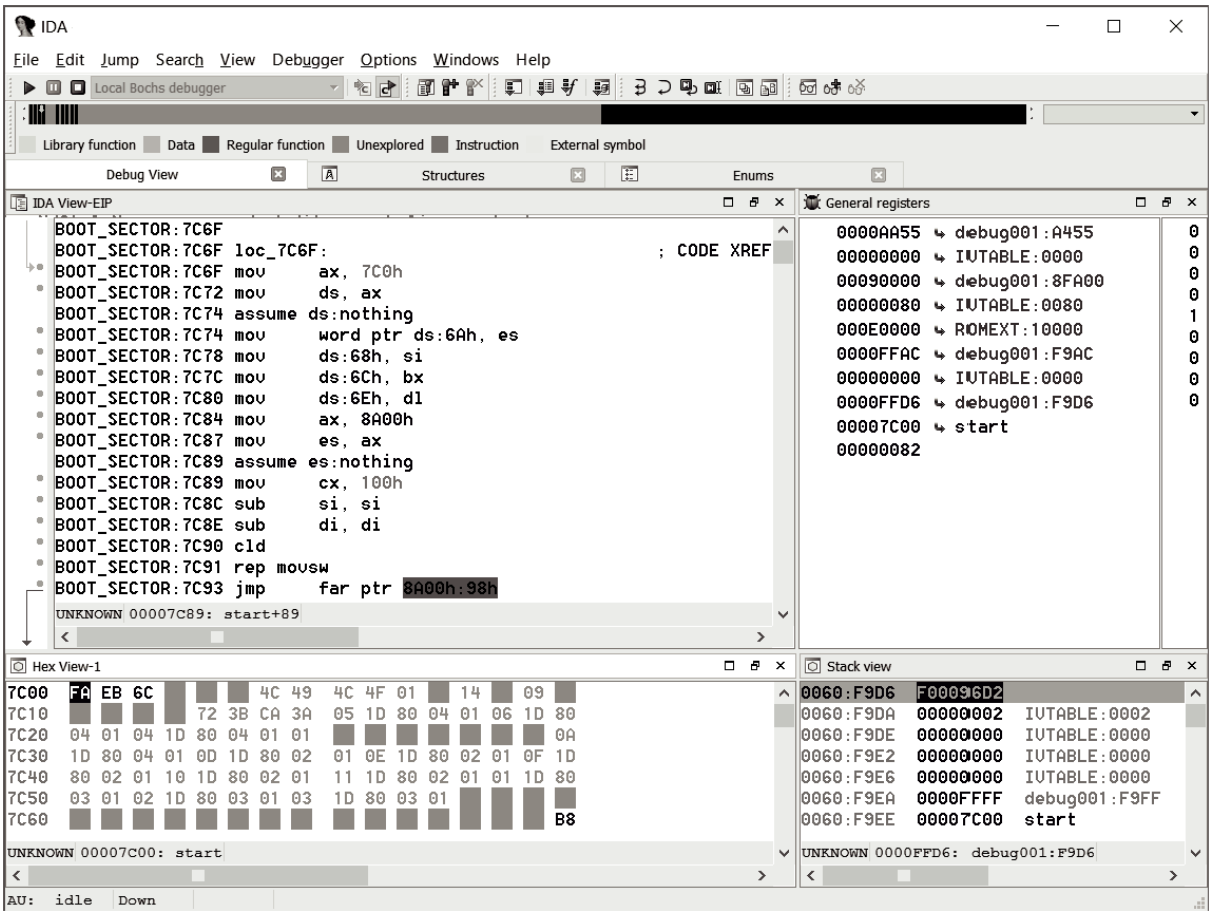


Рисунок 9-5. Отладка MBR из интерфейса IDA в виртуальной машине Bochs

Интерфейс на рисунке 9-5 значительно удобнее интерфейса командной строки отладчика Bochs, показанного ранее на рисунке 9-2. В одном окне видны дизассемблированный загрузочный код ❶, содержимое регистров процессора ❷, дамп памяти ❸ и стек процессора ❹. Это существенно упрощает отладку загрузочного кода.

Виртуализация с помощью VMware Workstation

IDA Pro и Bochs образуют мощное сочетание для анализа загрузочного кода. Однако отладка процессов загрузки ОС в Bochs иногда нестабильна, а метод эмуляции имеет ограничения производительности. Например, для глубокого анализа вредоносной программы требуется создать образ диска с предварительно установленной ОС. Из-за природы эмуляции этот этап может занять много времени. Кроме того, в Bochs нет удобной системы управления снимками состояния эмулируемой среды - незаменимой возможности при анализе вредоносного ПО.

Для более стабильной и эффективной работы можно использовать внутренний интерфейс отладки GDB в VMware совместно с IDA. В этом разделе мы познакомимся с отладчиком VMware GDB и покажем, как настроить сеанс отладки.

Особенности отладки загрузчиков Microsoft Windows будут рассмотрены в следующих нескольких главах, посвящённых буткитам MBR и VBR. Мы также взглянем на переключение из реального режима в защищённый с точки зрения отладки.

VMware Workstation - мощный инструмент воспроизведения операционных систем и сред. Он позволяет создавать виртуальные машины с гостевыми ОС и выполнять их на том же компьютере, где работает ОС узла. Гостевая и основная системы работают, не мешая друг другу, как если бы находились на двух разных физических компьютерах. Это очень удобно для отладки: две программы, отладчик и отлаживаемое приложение, легко выполнять на одном узле. В этом отношении VMware Workstation похожа на Bochs, но Bochs эмулирует инструкции процессора, а VMware Workstation выполняет их на физическом процессоре. Поэтому код в VM выполняется быстрее, чем в Bochs.

Последние версии VMware Workstation, начиная с 6.5, содержат заглушку GDB для отладки работающих внутри VMware виртуальных машин. Она позволяет отлаживать VM с самого начала выполнения, ещё до того, как BIOS запустит код MBR. Начиная с версии 5.4, IDA Pro содержит модуль отладчика с поддержкой протокола GDB, который можно использовать совместно с VMware.

На момент написания главы VMware Workstation выпускалась в двух вариантах: Professional, коммерческая версия, и Workstation Player, бесплатная версия. Professional предоставляет расширенную функциональность, включая создание и редактирование VM, тогда как Workstation Player позволяет только запускать виртуальные машины и изменять их конфигурацию. Но обе версии содержат отладчик GDB и пригодны для анализа буткитов. В этой главе используется Professional, чтобы можно было создать VM.

> **Примечание.** Прежде чем использовать отладчик GDB VMware, необходимо создать экземпляр виртуальной машины в VMware Workstation и предварительно установить в него операционную систему. Создание VM выходит за рамки главы, но все необходимые сведения приведены в документации по адресу vmware.com.

Настройка VMware Workstation

После создания виртуальной машины VMware Workstation помещает образ VM и конфигурационный файл в указанный пользователем каталог, который далее называется каталогом виртуальной машины.

Чтобы включить работу VMware с GDB, укажите определённые параметры в конфигурационном файле виртуальной машины, показанном в листинге 9-4. Это текстовый файл с расширением `.vmx`, расположенный в каталоге виртуальной машины. Откройте его в текстовом редакторе и скопируйте параметры из листинга.

```
❶ debugStub.listen.guest32 = "TRUE"
❷ debugStub.hideBreakpoints= "TRUE"
❸ monitor.debugOnStartGuest32 = "TRUE"
```

Листинг 9-4. Включение заглушки GDB в виртуальной машине

Первый параметр ❶ разрешает отладку гостевой системы с локального узла. Он включает заглушку GDB VMware, благодаря которой к отлаживаемой VM можно подключить отладчик с поддержкой протокола GDB. Если бы отладчик и VM работали на разных компьютерах, вместо этого потребовалось бы включить удалённую отладку командой `debugStub.listen.guest32.remote`.

Второй параметр ❷ включает аппаратные точки останова вместо программных. Аппаратные точки используют средства отладки процессора, а именно регистры `dr0` - `dr7`, тогда как программные точки обычно реализуются выполнением инструкции `int 3`. При отладке вредоносных программ это означает, что аппаратные точки устойчивее и их труднее обнаружить.

Последний параметр ❸ предписывает GDB остановить отладчик на самой первой инструкции процессора, то есть сразу после запуска VM. Если пропустить этот параметр, VMware Workstation начнёт выполнять загрузочный код без остановки, и отлаживать его будет невозможно.

Отладка 32- или 64-разрядного кода

Суффикс `32` в параметрах `debugStub.listen.guest32` и `debugStub.debugOnStartGuest32` указывает, что отлаживается 32-разрядный код. Для 64-разрядной ОС вместо них можно использовать `debugStub.listen.guest64` и `debugStub.debugOnStartGuest64`. Однако для предзагрузочного кода MBR/VBR, выполняемого в 16-разрядном реальном режиме, подойдёт как 32-, так и 64-разрядный вариант.

Совместное использование VMware GDB и IDA

Настроив VM, можно запускать сеанс отладки. Сначала включите виртуальную машину в VMware Workstation, выбрав `VM > Power > Power On`.

Затем запустите отладчик IDA Pro для подключения к VM. Выберите `Debugger`, затем `Attach > Remote GDB debugger`.

Теперь настройте параметры отладки. Сначала укажите имя узла и порт цели. VM работает на том же узле, поэтому в качестве имени задайте `localhost`, как показано на рисунке 9-6, а в качестве порта - `8832`. Заглушка GDB слушает этот порт при использовании `debugStub.listen.guest32` в конфигурационном файле VM. При использовании `debugStub.listen.guest64` номер порта равен `8864`. Остальные параметры отладки можно оставить по умолчанию.

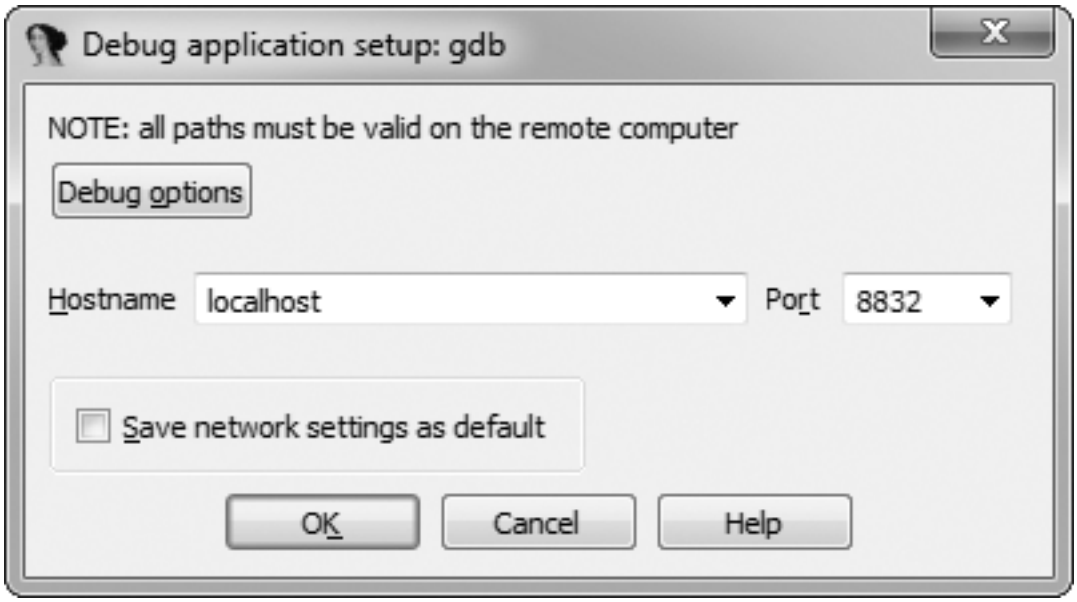


Рисунок 9-6. Указание параметров GDB

После установки всех параметров IDA Pro пытается подключиться к цели и предлагает список процессов. Поскольку отладка предзагрузочных компонентов уже началась, выберите `<attach to the process started on target>`, как показано на рисунке 9-7.

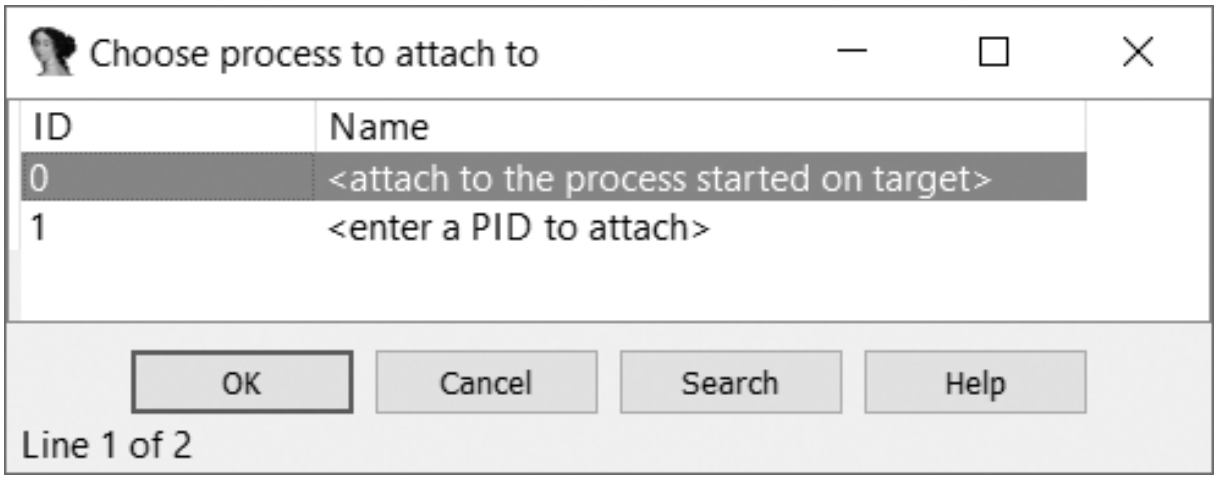
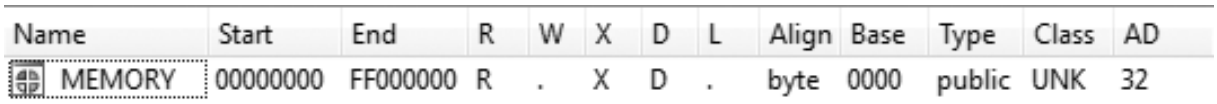


Рисунок 9-7. Выбор целевого процесса

Теперь IDA Pro подключается к VM и останавливается на первой выполняемой инструкции.

Настройка сегмента памяти

Прежде чем продолжить, необходимо изменить тип сегмента памяти, созданного отладчиком. При запуске сеанса IDA Pro создала 32-разрядный сегмент памяти, похожий на показанный на рисунке 9-8.



Name	Start	End	R	W	X	D	L	Align	Base	Type	Class	AD
MEMORY	00000000	FF000000	R	.	X	D	.	byte	0000	public	UNK	32

Рисунок 9-8. Параметры сегмента памяти в IDA Pro

В предзагрузочной среде процессор работает в реальном режиме, поэтому для правильного дизассемблирования кода необходимо изменить сегмент с 32- на 16-разрядный. Щёлкните целевой сегмент правой кнопкой мыши и выберите `Change segment attributes`. В появившемся диалоге выберите `16-bit` ❶ в области `Segment bitness`, как показано на рисунке 9-9.

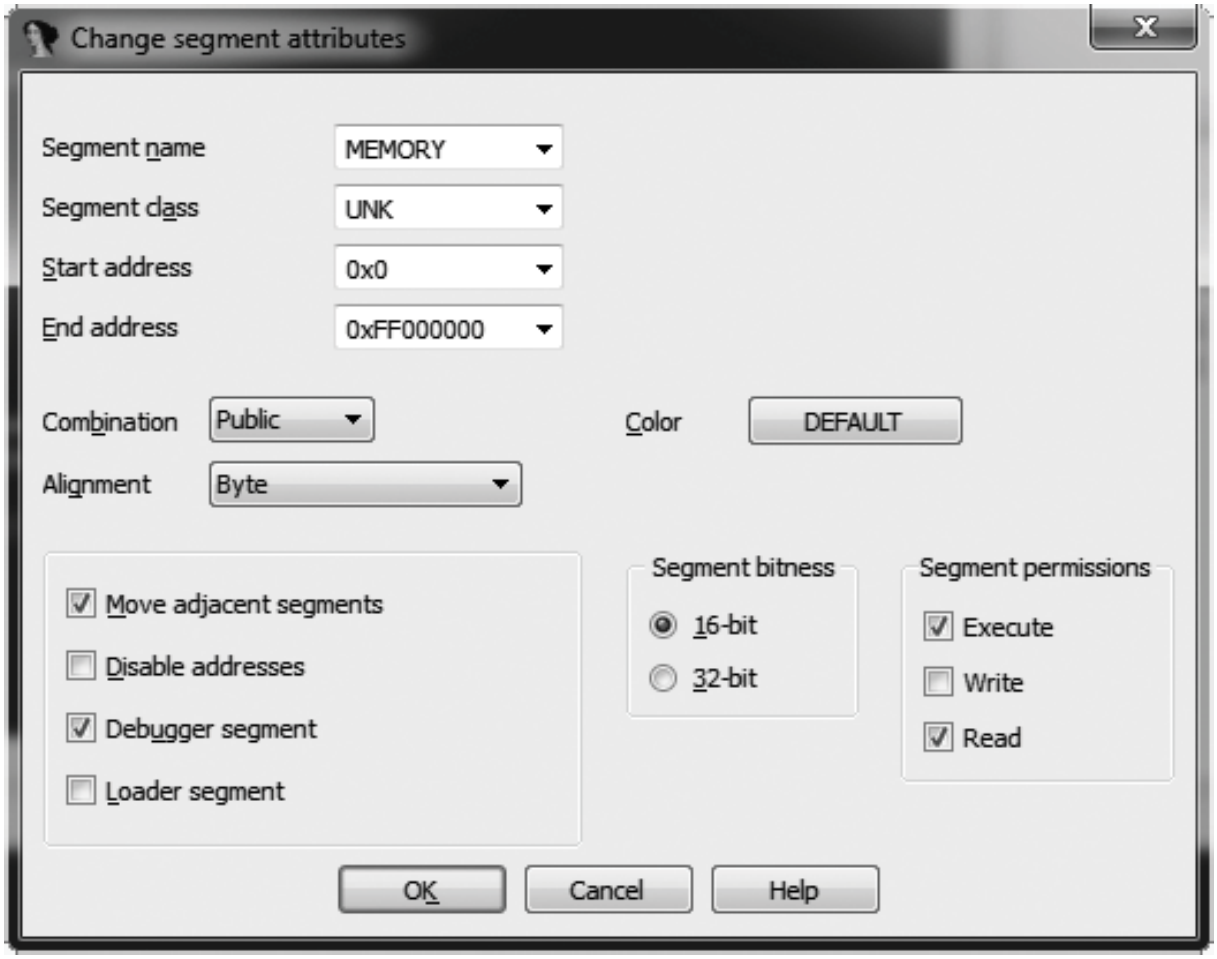


Рисунок 9-9. Изменение разрядности сегмента памяти

После этого сегмент станет 16-разрядным, и все инструкции загрузочных компонентов будут дизассемблированы правильно.

Запуск отладчика

Задав все параметры, можно перейти к загрузке MBR. Поскольку отладчик подключился к VM в самом начале выполнения, код MBR ещё не загружен. Чтобы загрузить его, установите точку останова в самом начале кода по адресу `0000:7c00h`, а затем продолжите выполнение. Перейдите по этому адресу в окне дизассемблирования и нажмите `F2`. Появится диалог параметров точки останова, показанный на рисунке 9-10.

Поле `Location` ❶ задаёт адрес точки останова: `0x7c00`, соответствующий виртуальному адресу `0000:7c00h`. В области `Settings` ❷ установите флажки `Enabled` и `Hardware`. Первый включает точку останова: когда поток выполнения достигнет указанного адреса, она сработает. Второй предписывает отладчику использовать регистры отладки процессора и активирует параметры режима аппаратной точки ❸. В нашем случае выберите `Execute`, чтобы остановиться при выполнении инструкции по адресу `0000:7c00h`. Другие типы предназначены для чтения и записи памяти и здесь не нужны. Меню `Size` ❹ задаёт размер контролируемой памяти. Оставьте значение по умолчанию 1: точка будет контролировать один байт по адресу `0000:7c00h`. Затем нажмите `OK` и возобновите выполнение клавишей `F9`.

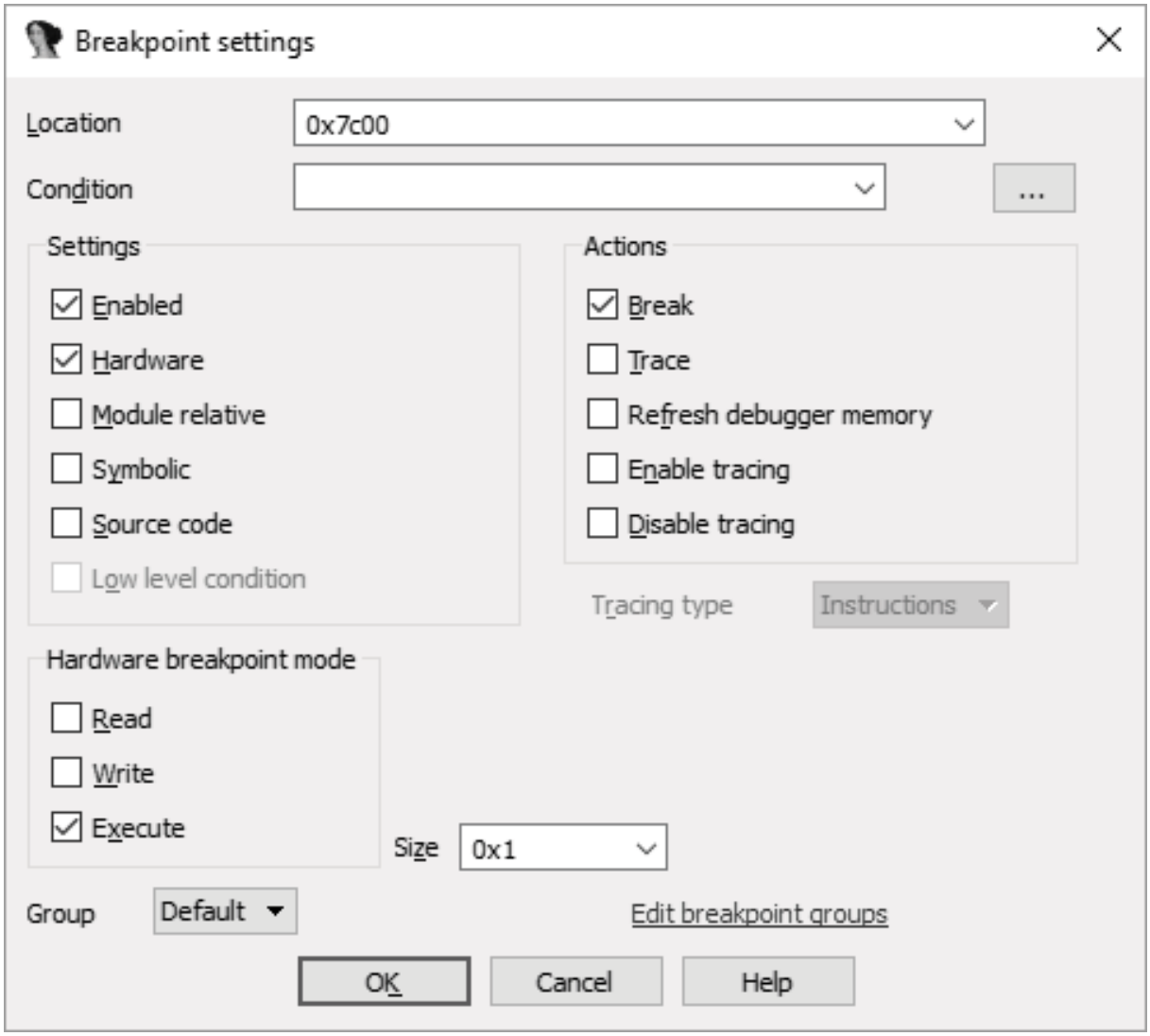


Рисунок 9-10. Диалог параметров точки останова

После загрузки и выполнения MBR отладчик остановится. Его окно показано на рисунке 9-11.

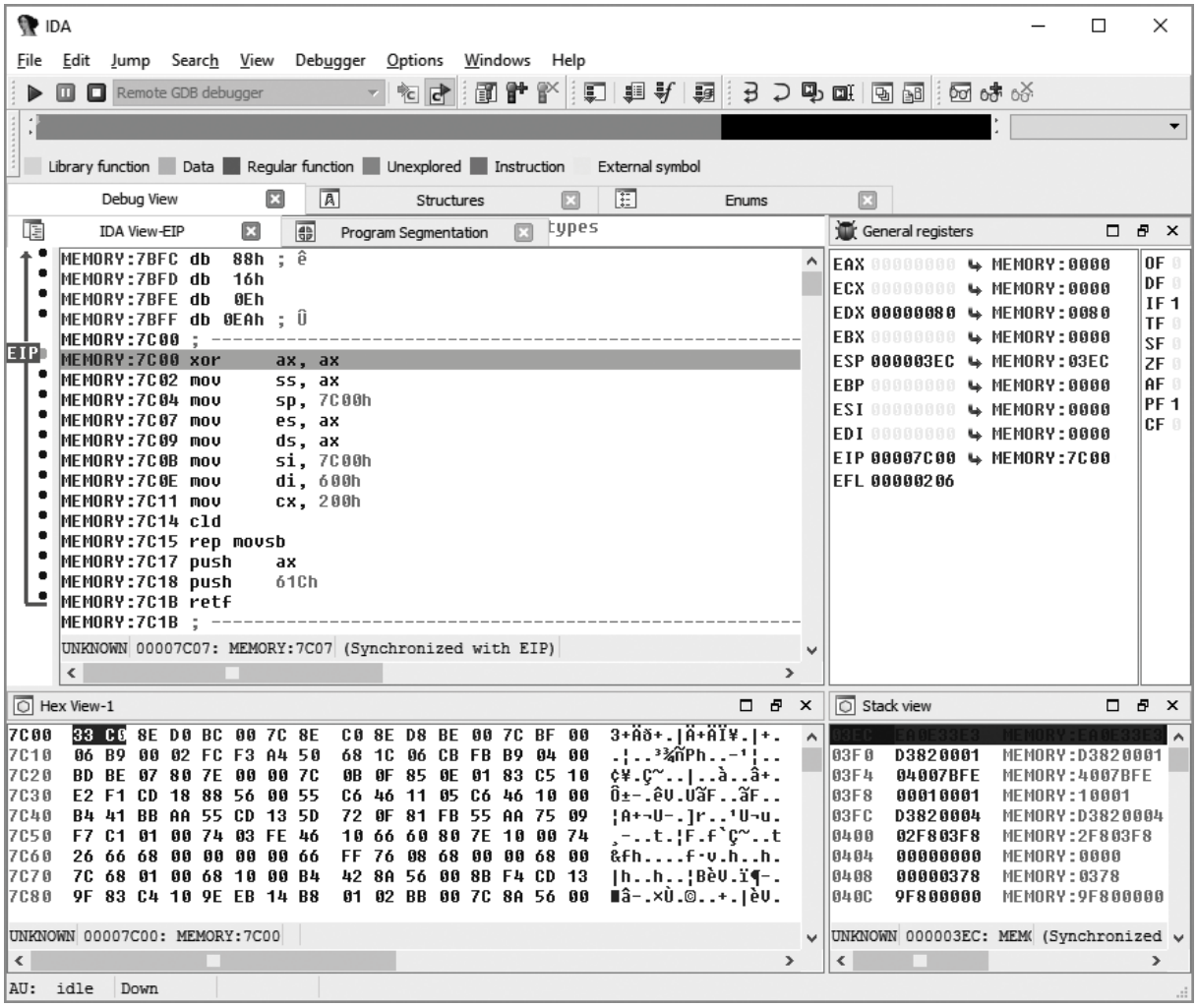


Рисунок 9-11. Интерфейс отладчика IDA Pro

Теперь мы находимся на самой первой инструкции кода MBR: регистр указателя инструкций ❶ указывает на `0000:7c00h`. В окне дампа памяти и дизассемблированном представлении видно, что MBR успешно загружена. Отсюда можно продолжать отладку кода MBR, пошагово выполняя каждую инструкцию.

> **Примечание.** Цель раздела состояла лишь в том, чтобы познакомить вас с возможностью совместного использования отладчика GDB VMware Workstation и IDA Pro, поэтому глубже рассматривать GDB в этой главе мы не будем. Дополнительные сведения появятся в следующих главах при анализе буткита Rovnix.

Microsoft Hyper-V и Oracle VirtualBox

В этой главе не рассматриваются диспетчер виртуальных машин Hyper-V, входящий в клиентские операционные системы Microsoft начиная с Windows 8, и диспетчер виртуальных машин VirtualBox с открытым исходным кодом. На момент написания ни у одной программы не было документированного интерфейса, позволяющего начинать отладку VM достаточно рано для анализа вредоносного загрузочного кода.

На момент публикации Microsoft Hyper-V являлась единственной системой виртуализации с поддержкой VM, в которых включена Secure Boot; возможно, это одна из причин отсутствия интерфейса отладки ранних стадий загрузки. Технология Secure Boot и её уязвимости глубже рассматриваются в главе 17. Эти две программы упомянуты потому, что широко применяются при анализе вредоносного ПО, но отсутствие интерфейсов отладки ранней загрузки - главная причина, по которой для вредоносного загрузочного кода мы предпочитаем VMware Workstation.

Заключение

В этой главе мы показали, как отлаживать код MBR и VBR буткита с помощью эмулятора Bochs и VMware Workstation. Эти методы динамического анализа полезны, когда требуется глубже исследовать вредоносный загрузочный код. Они дополняют статический анализ и помогают отвечать на вопросы, на которые тот ответить не может.

Мы вновь воспользуемся этими средствами и методами в главе 11 для анализа буткита Rovnix, архитектура и функциональность которого слишком сложны для эффективного применения одних лишь методов статического анализа.

Упражнения

Ниже приведены упражнения для проверки навыков, полученных в главе. Вы соберёте образ ПК для Bochs из MBR, VBR/IPL и раздела файловой системы NTFS, а затем выполните динамический анализ посредством интерфейса IDA Pro для Bochs. Сначала загрузите по адресу nostarch.com следующие материалы:

• `mbr.mbr` - двоичный файл с MBR.
• `partition0.data` - образ раздела NTFS, содержащий VBR и IPL.
• `bochs.bochsrc` - конфигурационный файл Bochs.

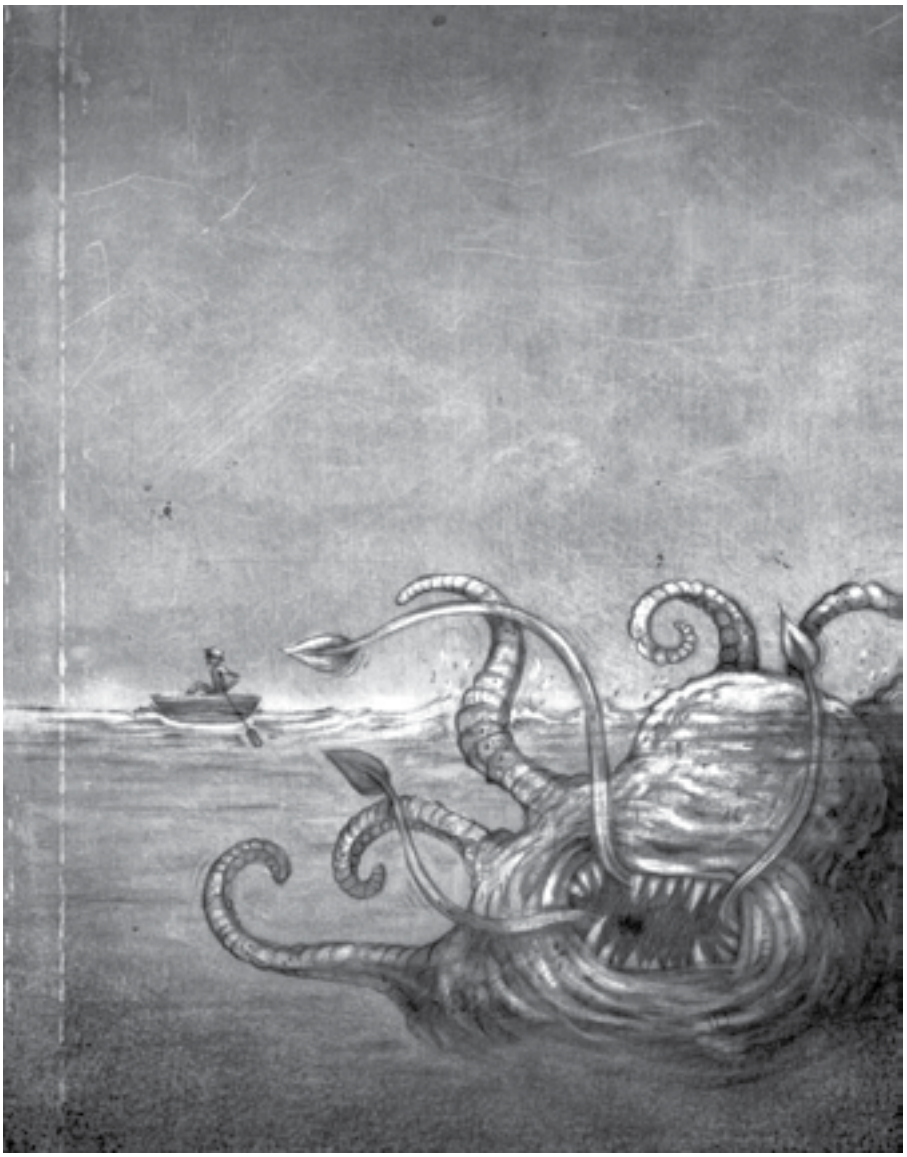
Также потребуются дизассемблер IDA Pro, интерпретатор Python и эмулятор Bochs. Используя эти средства и материал главы, выполните следующие упражнения:

1. Создайте образ Bochs и измените значения в шаблоне конфигурации `bochs.bochsrc`, чтобы он соответствовал листингу 9-1. С помощью `bximage`, как описано в разделе «Создание образа диска» на странице 118 оригинального издания, создайте плоский образ размером 10 Мбайт и сохраните его в файле.
2. Измените параметр `ata0-master` в шаблоне конфигурации, чтобы он использовал образ из упражнения 1. Примените параметры листинга 9-1.

3. Подготовив образ Bochs, запишите в него компоненты буткита MBR и VBR. Сначала откройте `mbr.mbr` в IDA Pro и проанализируйте его. Убедитесь, что код MBR зашифрован. Найдите процедуру расшифрования и опишите её алгоритм.
4. Проанализируйте таблицу разделов MBR и ответьте: сколько разделов существует; какой из них активен; где активный раздел расположен на жёстком диске; каковы его смещение от начала диска и размер в секторах?
5. Найдя активный раздел, запишите файл `mbr.mbr` в образ Bochs сценарием Python из листинга 9-2. Запишите `partition0.data` в образ по смещению, найденному в предыдущем упражнении, с помощью сценария из листинга 9-3. После этого заражённый образ Bochs будет готов к эмуляции.
6. Запустите эмулятор Bochs с изменённой конфигурацией `bochs.bochsrc` через интерфейс IDA Pro, описанный в разделе «Совместное использование Bochs и IDA» на странице 123 оригинального издания. Отладчик IDA Pro должен остановить выполнение. Установите точку останова по адресу `0000:7c00h`, куда будет загружен код MBR.
7. После срабатывания точки по адресу `0000:7c00h` убедитесь, что код MBR всё ещё зашифрован. Установите точку останова на найденной ранее процедуре расшифрования и продолжите выполнение. После её срабатывания трассируйте процедуру до полного расшифрования кода MBR. Сохраните дамп расшифрованной MBR в файл для дальнейшего статического анализа. Методы статического анализа MBR описаны в главе 8.

На этой странице исходного издания нет основного текста.

Глава 10. Эволюция методов заражения MBR и VBR: Olmasco



В ответ на первую волну буткитов разработчики средств безопасности начали создавать антивирусные продукты, специально проверявшие изменения кода MBR, и вынудили злоумышленников искать другие методы заражения. В начале 2011 года семейство TDL4 превратилось в новое вредоносное ПО с приёмами заражения, прежде не встречавшимися в реальных атаках.

Один из примеров - буткит Olmasco, в значительной мере основанный на TDL4, но имеющий ключевое отличие: Olmasco заражает таблицу разделов MBR, а не код MBR. Благодаря этому он заражает систему и обходит политику подписывания кода режима ядра, избегая обнаружения всё более опытным антивредоносным ПО.

Olmasco также стал первым известным буткитом, сочетавшим методы заражения MBR и VBR, хотя по-прежнему в первую очередь атаковал MBR. Это отличает его от заражающих VBR буткитов Rovnix и Carberp, рассматриваемых в главе 11.

Подобно предшественникам из семейства TDL, Olmasco распространяется по бизнес-модели оплаты за установку (PPI), знакомой по обсуждению руткита TDL3 в главе 1. Модель PPI похожа на

схемы распространения панелей инструментов браузеров, например Google Toolbar, и использует встроенные уникальные идентификаторы (UID), позволяющие распространителям отслеживать число установок и, следовательно, доход. Сведения о распространителе встраиваются в исполняемый файл, а специальные серверы подсчитывают установки. За определённое число установок распространителю выплачивается фиксированная сумма.[1]

В этой главе мы рассмотрим три основных аспекта Olmasco: дроппер, заражающий систему; компонент буткита, заражающий таблицу разделов MBR; и руткит, который перехватывает драйвер жёсткого диска, доставляет полезную нагрузку, использует скрытую файловую систему и перенаправляет сетевое взаимодействие.

Дроппер

Дроппер - специальное вредоносное приложение, служащее носителем другого вредоносного ПО, сохранённого как зашифрованная полезная нагрузка. Дроппер попадает на компьютер жертвы, распаковывает и выполняет нагрузку - в нашем случае инфектор Olmasco, - которая устанавливает и запускает компоненты буткита. Обычно до распаковки нагрузки дроппер также выполняет проверки против отладки и эмуляции, чтобы уклониться от автоматизированных систем анализа вредоносного ПО.

> **Дроппер и загрузчик**
>
> Другой распространённый вид вредоносного приложения для доставки вредоносного ПО - загрузчик. Как следует из названия, загрузчик получает полезную нагрузку с удалённого сервера, а не несёт её внутри себя подобно дропперу. Однако на практике термин «дроппер» употребляется чаще и нередко служит синонимом загрузчика.

Ресурсы дроппера

Дроппер имеет модульную структуру и хранит большинство вредоносных компонентов буткита в разделе ресурсов. Каждый компонент, например значение идентификатора, компонент загрузчика или полезная нагрузка, помещён в отдельный элемент ресурсов и зашифрован RC4; подробнее см. блок «Потоковый шифр RC4» на странице 136 оригинального издания. Размер элемента ресурсов используется как ключ расшифрования.

В таблице 10-1 перечислены компоненты буткита в разделе ресурсов дроппера.

Таблица 10-1. Компоненты буткита в дроппере Olmasco

Имя ресурса	Описание
`affid`	Уникальный идентификатор партнёра
`subid`	Субидентификатор партнёра; связан с идентификатором партнёра, у которого может быть несколько субидентификаторов
`boot`	Первая часть вредоносного загрузчика; выполняется в начале процесса загрузки
`cmd32`	Полезная нагрузка пользовательского режима для 32-разрядных процессов
`cmd64`	Полезная нагрузка пользовательского режима для 64-разрядных процессов

Имя ресурса	Описание
dbg32	Третья часть вредоносного загрузчика, поддельная библиотека kdcom.dll, для 32-разрядных систем
dbg64	Третья часть вредоносного загрузчика, поддельная библиотека kdcom.dll, для 64-разрядных систем
drv32	Вредоносный драйвер режима ядра для 32-разрядных систем
drv64	Вредоносный драйвер режима ядра для 64-разрядных систем
ldr32	Вторая часть вредоносного загрузчика; выполняется компонентом boot в 32-разрядных системах
ldr64	Вторая часть вредоносного загрузчика; выполняется компонентом boot в 64-разрядных системах
main	Назначение неизвестно
build	Номер сборки дроппера
name	Имя дроппера
vbr	VBR вредоносного раздела Olmasco на жёстком диске

Идентификаторы affid и subid используются в схеме PPI для подсчёта установок. affid - уникальный идентификатор партнёра, то есть распространителя. subid различает установки из разных источников. Например, если партнёр PPI распространяет вредоносную программу через два разных файлообменных сервиса, у полученных из них образцов будет одинаковый affid, но разные subid. Так партнёр может сравнить число установок для каждого subid и определить более прибыльный источник.

Компоненты boot, vbr, dbg32, dbg64, drv32, drv64, ldr32 и ldr64 вскоре будут рассмотрены подробнее, а main, build и name описаны только в таблице.

Потоковый шифр RC4

RC4 - потоковый шифр, разработанный в 1987 году Роном Ривестом из RSA Security. RC4 принимает ключ переменной длины и создаёт поток псевдослучайных байтов, используемых для шифрования открытого текста. Благодаря компактной и простой реализации он становится всё популярнее среди разработчиков вредоносного ПО. Поэтому многие руткиты и буткиты используют RC4 для защиты полезной нагрузки, взаимодействия с командными серверами (C&C) и конфигурационных сведений.

Функциональность трассировки для дальнейшей разработки

В дроппере Olmasco появилась отчётность об ошибках, помогающая разработчикам совершенствовать программу. После успешного выполнения каждого этапа заражения, то есть шага алгоритма установки буткита, программа сообщает командным серверам о «контрольной точке». Если установка завершается неудачно, разработчики могут точно определить этап сбоя. При ошибках буткит дополнительно отправляет подробное сообщение, предоставляющее достаточно сведений для выявления причины.

Трассировочные сведения отправляются методом HTTP GET командному серверу, доменное имя которого жёстко задано в дроппере. В листинге 10-1 показана декомпилированная Hex-Rays функция инфектора Olmasco, создающая строку запроса для сообщения о состоянии заражения.

```
HINTERNET __cdecl ReportCheckPoint(int check_point_code){
  char query_string[0x104];
  memset(&query_string, 0, 0x104u);
  ❶ _snprintf(
    &query_string,
    0x104u,
    "/testadd.php?aid=%s&sid=%s&bid=%s&mode=%s%u%s%s",
    *FILE_affid,
    *FILE_subid,
    &bid,
    "check_point",
    check_point_code,
    &bid,
    &bid);
  ❷ return SendDataToServer(0, &query_string, "GET", 0, 0);
}
```

Листинг 10-1. Отправка трассировочных сведений командному серверу

Под номером ❶ вредоносная программа выполняет функцию `_snprintf`, создавая строку запроса с параметрами дроппера. Под номером ❷ она отправляет запрос. Значение `check_point_code` соответствует порядковому номеру этапа алгоритма установки, отправившего сообщение. Например, 1 соответствует самому первому этапу алгоритма, 2 - второму и так далее. После успешной установки командный сервер получает последовательность вроде `1, 2, 3, 4, ... N`, где `N` - последний этап. При неудаче полной установки сервер получает `1, 2, 3, ... P`, где `P` - этап сбоя алгоритма. Это позволяет разработчикам вредоносной программы найти и исправить неисправный шаг алгоритма заражения.

Приёмы противодействия отладке и эмуляции

Olmasco также представил новые способы обхода анализа в песочнице и защиты от дампов памяти. Дроппер сжат специальным упаковщиком, который после выполнения распаковывает исходный дроппер и стирает в памяти некоторые поля его PE-заголовка, например адрес исходной точки входа и таблицу разделов. На рисунке 10-1 показан PE-заголовок до и после удаления данных: слева он частично разрушен, справа не изменён.

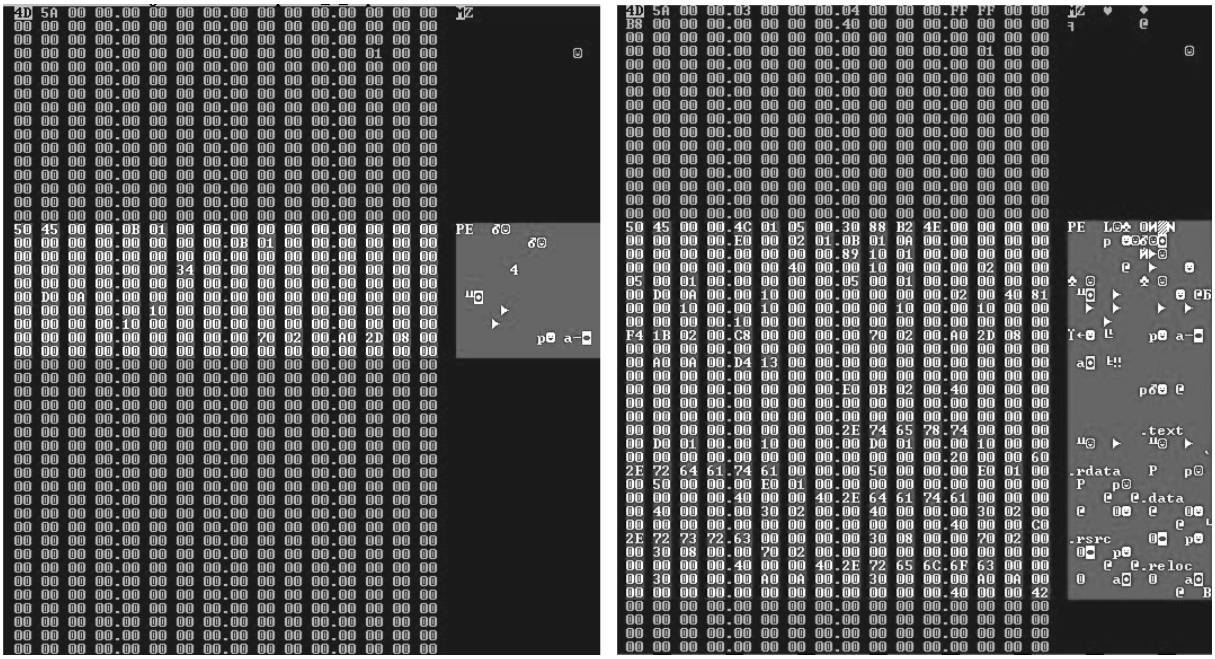


Рисунок 10-1. Стирание данных PE-заголовка

Этот приём хорошо защищает от создания дампа памяти во время отладки и автоматической распаковки. Удаление действительного PE-заголовка затрудняет определение геометрии PE-файла и правильное создание дампа, поскольку программа не может установить точное расположение разделов кода и данных. Без этих сведений она не способна правильно восстановить PE-образ и завершается неудачей.

Olmasco также содержит меры против средств слежения за ботами, основанных на виртуальных машинах. Во время установки программа посредством интерфейса `IWbemServices` инструментария управления Windows (Windows Management Instrumentation, WMI) определяет, выполняется ли дроппер в виртуальной среде, и отправляет эти сведения командному серверу. Обнаружив виртуальную среду, дроппер прекращает работу и удаляет себя из файловой системы, вместо того чтобы распаковать вредоносный двоичный файл и открыть его средствам анализа.

> **Примечание.** Microsoft WMI - набор интерфейсов для управления данными и операциями на платформах Windows. Одно из его основных назначений - автоматизация административных задач на удалённых компьютерах. С точки зрения вредоносного ПО WMI предоставляет богатый набор объектов Component Object Model (COM), позволяющих собирать подробные сведения о системе: платформе, работающих процессах, защитном ПО и многом другом.

Вредоносная программа также использует WMI для сбора следующих сведений о целевой системе:

• **Компьютерная система:** имя системы, имя пользователя, доменное имя, рабочая группа, число процессоров и так далее.
• **Процессор:** число ядер, название, разрядность данных и число логических процессоров.
• **Контроллер SCSI:** название и изготовитель.
• **Контроллер IDE:** название и изготовитель.
• **Дисковый накопитель:** название, модель и тип интерфейса.
• **BIOS:** название и изготовитель.
• **ОС:** основной и дополнительный номера версии, номер пакета обновления и другие сведения.

Операторы вредоносной программы могут проверить аппаратную конфигурацию заражённой системы и определить её полезность. Например, по названию и изготовителю BIOS можно выявить виртуальные среды VMware, VirtualBox, Bochs или QEMU, часто используемые для автоматизированного анализа вредоносного ПО и потому неинтересные операторам.

С другой стороны, имя системы и домена позволяют определить компанию - владельца заражённого компьютера и доставить специальную полезную нагрузку, нацеленную именно на неё.

Функциональность буткита

После проверок песочницы дроппер устанавливает компонент буткита. Он был изменён по сравнению с TDL4, который, как обсуждалось в главе 7, перезаписывает MBR и резервирует место в конце загрузочного жёсткого диска для вредоносных компонентов. Olmasco применяет существенно иной подход.

Метод заражения буткита

Сначала Olmasco создаёт раздел в конце загрузочного жёсткого диска. В таблицах разделов дисков Windows всегда имеется некоторое неразмеченное, или нераспределённое, пространство в конце, обычно достаточное для компонентов буткита, а иногда и большее. Вредоносная программа занимает его и создаёт раздел, изменяя свободный элемент таблицы разделов исходной легитимной MBR, чтобы тот указывал на новый раздел. Как ни странно, размер вредоносного раздела ограничен 50 Гбайт независимо от объёма доступного неразмеченного пространства. Возможно, это сделано, чтобы не привлекать внимание пользователя занятием всего свободного пространства.

Как обсуждалось в главе 5, таблица разделов MBR расположена по смещению `0x1BE` от начала MBR и состоит из четырёх 16-байтовых элементов, описывающих соответствующие разделы диска. На диске может быть не более четырёх основных разделов, а активным помечается только один, поэтому буткит может загружаться лишь из одного раздела. Вредоносная программа записывает параметры своего раздела в первый пустой элемент таблицы, помечает его активным и инициализирует VBR созданного раздела, как показано в листинге 10-2.

```
Первый раздел                      00212000    0C13DF07    00000800    00032000
Второй раздел (ОС)                 0C14DF00    FFFFFE07    00032800    00FCC800
Третий раздел (Olmasco), активный FFFFFE80    FFFFFE1B  ❶00FFF000  ❷00000FB0
Четвёртый раздел (пустой)          00000000    00000000    00000000    00000000
```

Листинг 10-2. Таблица разделов после заражения Olmasco

Здесь видны начальный адрес вредоносного раздела ❶ и его размер в секторах ❷. Если Olmasco не находит свободного элемента таблицы, он сообщает об этом командному серверу и завершает работу. На рисунке 10-2 показано, как меняется таблица разделов после заражения.

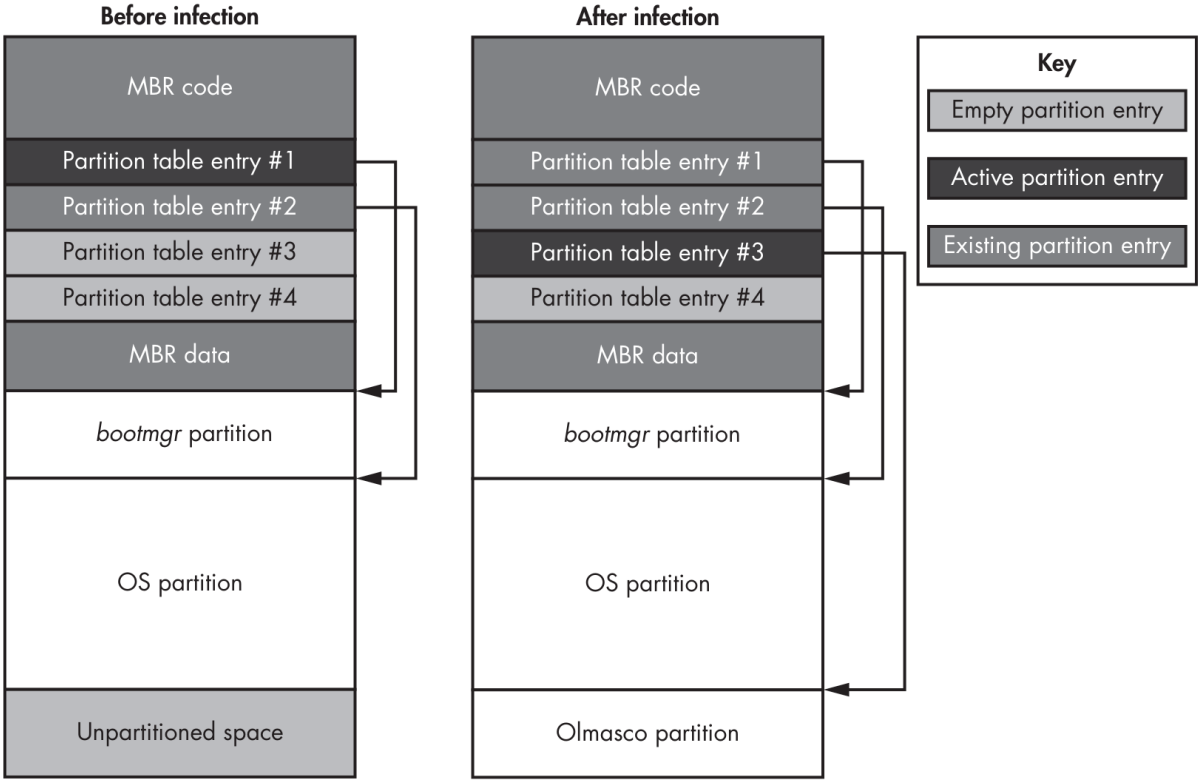


Рисунок 10-2. Схема жёсткого диска до и после заражения Olmasco. Пустой элемент таблицы становится активным и указывает на раздел Olmasco в конце диска

После заражения ранее пустой элемент таблицы подключается к разделу Olmasco и становится активным. Сам код MBR остаётся нетронутым; изменяется только таблица разделов MBR. Для дополнительной скрытности первый сектор раздела Olmasco выглядит очень похожим на легитимную VBR, поэтому защитное ПО может принять раздел за законный раздел жёсткого диска.

Процесс загрузки заражённой системы

После заражения Olmasco система загружается по схеме, представленной на рисунке 10-3.

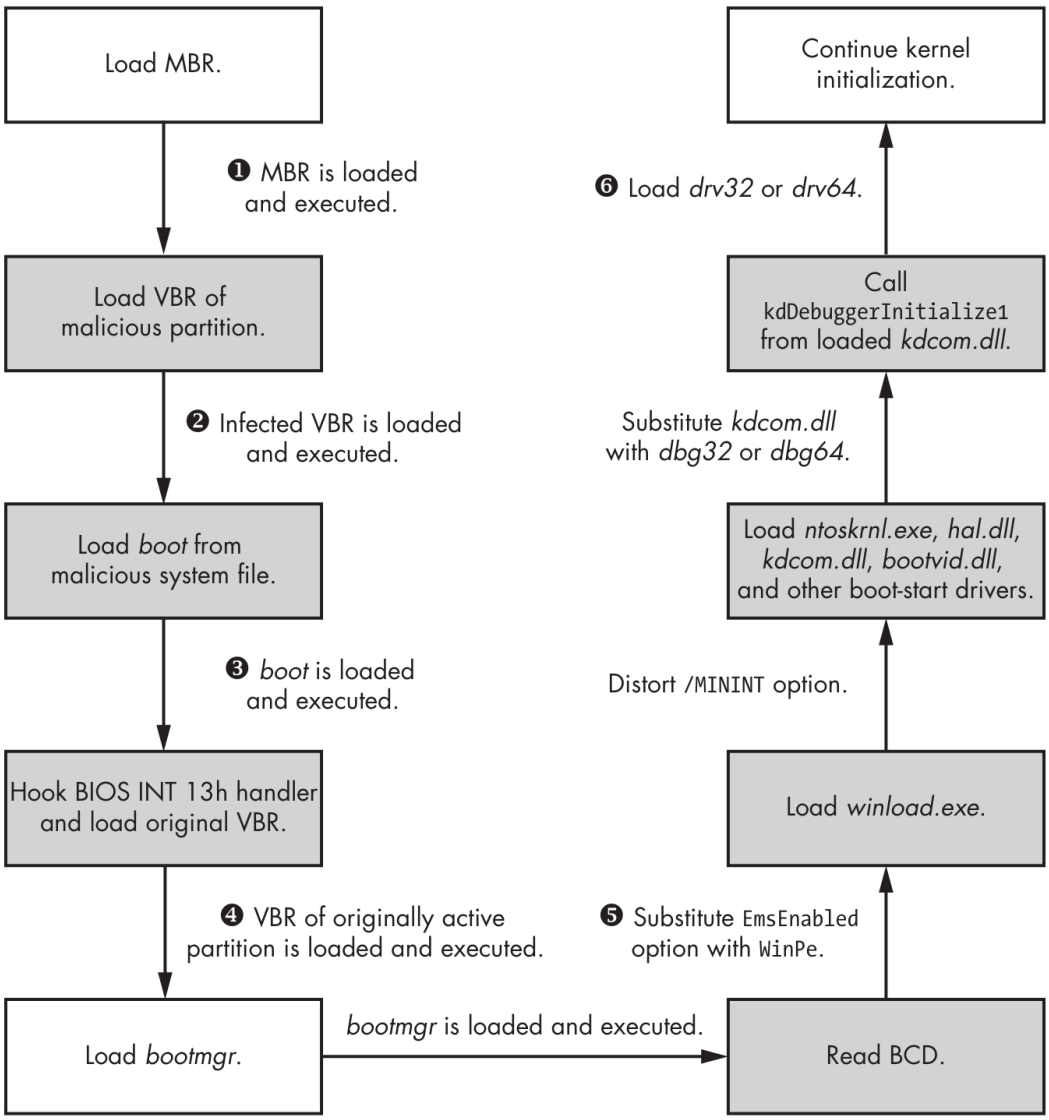


Рисунок 10-3. Процесс загрузки системы, заражённой Olmasco: после MBR выполняется VBR вредоносного раздела, загружается `boot`*, перехватывается INT 13h и запускается исходная VBR. Затем* `bootmgr` *читает изменённую BCD,* `winload.exe` *загружает подменённую* `kdcom.dll`*, после чего выполняется* `drv32` *или* `drv64`

При следующей загрузке заражённой машины вредоносная VBR раздела Olmasco ❷ получает управление сразу после выполнения кода MBR ❶ и до загрузки компонентов загрузчика ОС. Это позволяет вредоносной программе опередить ОС. Получив управление, вредоносная VBR читает файл `boot` из корневого каталога скрытой файловой системы Olmasco ❸ и передаёт ему управление. Компонент `boot` выполняет ту же роль, что и `ldr16` в предыдущих версиях TDL4: перехватывает обработчик прерывания BIOS 13h ❹ для изменения данных конфигурации загрузки (BCD) ❺ и загружает VBR исходно активного раздела.

Концептуально процессы загрузки Olmasco и TDL4 очень похожи, а компоненты совпадают, за исключением других имён элементов скрытой файловой системы Olmasco, перечисленных в таблице 10-2. Процесс загрузки TDL4 подробно рассматривался в главе 7.

Таблица 10-2. Загрузочные компоненты Olmasco и TDL4

Olmasco	TDL4
`boot`	`ldr16`
`dbg32`, `dbg64`	`ldr32`, `ldr64`

Функциональность руткита

Работа буткита завершается после загрузки вредоносного драйвера режима ядра, отмеченной ❻ на рисунке 10-3. Этот драйвер реализует функциональность руткита Olmasco, отвечающую за следующее:

• Перехват объекта устройства жёсткого диска.
• Внедрение полезной нагрузки из скрытой файловой системы в процессы.
• Поддержку скрытой файловой системы.
• Реализацию интерфейса транспортных драйверов (Transport Driver Interface, TDI) для перенаправления сетевого взаимодействия.

Перехват объекта устройства жёсткого диска и внедрение нагрузки

Первые два пункта по существу совпадают с TDL4: Olmasco применяет те же методы перехвата объекта устройства жёсткого диска и внедрения полезной нагрузки из скрытой файловой системы в процессы. Перехват не позволяет защитному ПО восстановить содержимое исходной MBR и помогает Olmasco сохраняться после перезагрузки. Olmasco перехватывает все запросы чтения и записи жёсткого диска и блокирует попытки изменить MBR или прочитать скрытую файловую систему.

Поддержка скрытой файловой системы

Скрытая файловая система важна для таких сложных угроз, как руткиты и буткиты, поскольку предоставляет тайный канал хранения сведений на компьютере жертвы. Традиционное вредоносное ПО использует файловую систему ОС, например NTFS, FAT32 или extX, для хранения компонентов, но это делает его уязвимым для криминалистического анализа и обнаружения защитным ПО. Поэтому некоторые развитые вредоносные программы реализуют собственную файловую систему в нераспределённой области жёсткого диска. В подавляющем большинстве современных конфигураций в конце диска остаётся не менее нескольких сотен мегабайт свободного пространства, достаточного для вредоносных компонентов и конфигурации. Файлы скрытой системы недоступны через обычные API вроде Win32 `CreateFileX` и `ReadFileX`, но вредоносная программа взаимодействует с хранилищем и обращается к данным через специальный интерфейс. Обычно содержимое дополнительно шифруется, затрудняя криминалистический анализ.

На рисунке 10-4 показан пример скрытой файловой системы. Она расположена сразу после файловой системы ОС и не мешает нормальной работе.

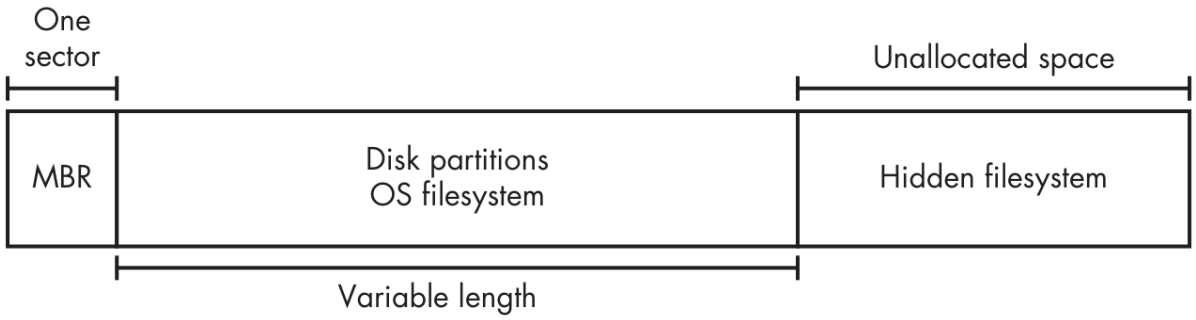


Рисунок 10-4. Скрытая файловая система на жёстком диске: после MBR и разделов с файловой системой ОС в нераспределённом пространстве размещается скрытая файловая система переменной длины

Методы хранения модулей полезной нагрузки Olmasco почти полностью унаследованы от TDL4: в конце диска резервируется место под файловую систему, содержимое которой защищено низкоуровневыми перехватами и потоковым шифром RC4. Однако разработчики Olmasco расширили архитектуру и реализацию: добавили иерархию файлов и каталогов, проверку целостности файлов на повреждения и более эффективное управление внутренними структурами.

Поддержка иерархии каталогов

Скрытая файловая система TDL4 могла хранить только файлы, а система Olmasco поддерживает файлы и каталоги. Корневой каталог обозначается обычной обратной косой чертой (\). В листинге 10-3 показан фрагмент VBR скрытого раздела Olmasco, загружающий файл `boot` из корневого каталога по пути `\boot` ❶.

```
seg000:01F4                 hlt
seg000:01F4 sub_195         endp
seg000:01F5                 jmp     short loc_1F4
seg000:01F7 aBoot         ❶ db '\boot',0
seg000:01FD                 db    0
```

Листинг 10-3. Фрагмент VBR раздела Olmasco

Проверка целостности

При чтении файла Olmasco проверяет, не повреждено ли содержимое. В TDL4 такая возможность не наблюдалась. Olmasco добавляет в структуру данных каждого файла поле для контрольной суммы CRC32. Обнаружив повреждение, программа удаляет соответствующий элемент из файловой системы и освобождает занятые секторы, как показано в листинге 10-4.

```
unsigned int stdcall RkFsLoadFile(FS_DATA_STRUCT *a1, PDEVICE_OBJECT
  DeviceObject, const char *FileName, FS_LIST_ENTRY_STRUCT *FileEntry)
{
  unsigned int result;
  // поиск файла в корневом каталоге
  ❶ result = RkFsLocateFileInDir(&a1->root_dir, FileName, FileEntry);
  if ( (result & 0xC0000000) != 0xC0000000 ) {
    // чтение файла с жёсткого диска
    ❷ result = RkFsReadFile(a1, DeviceObject, FileEntry);
    if ( (result & 0xC0000000) != 0xC0000000 ) {
      // проверка целостности файла
      ❸ result = RkFsCheckFileCRC32(FileEntry);
```

```
        if ( result == 0xC000003F ) {
          // освобождение занятых секторов
          ❹ MarkBadSectorsAsFree(a1, FileEntry->pFileEntry);
          // удаление соответствующего элемента
          RkFsRemoveFile(a1, &a1->root_dir, FileEntry->pFileEntry->FileName);
          RkFsFreeFileBuffer(FileEntry);
          // обновление каталога
          RkFsStoreFile(a1, DeviceObject, &a1->root_dir);
          RkFsStoreFile(a1, DeviceObject, &a1->bad_file);
          // обновление карты занятых секторов
          RkFsStoreFile(a1, DeviceObject, &a1->bitmap_file);
          // обновление корневого каталога
          RkFsStoreFile(a1, DeviceObject, &a1->root);
          result = 0xC000003F;
        }
      }
    }
    return result;
  }
```

Листинг 10-4. Чтение файла из скрытой файловой системы Olmasco

Функция `RkFsLocateFileInDir` ❶ находит файл в каталоге, читает его содержимое ❷, вычисляет контрольную сумму CRC32 и сравнивает её ❸ со значением в файловой системе. При несовпадении функция удаляет файл и освобождает занятые им секторы ❹. Это повышает надёжность скрытой файловой системы и стабильность руткита, снижая вероятность загрузки и выполнения повреждённого файла.

Управление файловой системой

Файловая система Olmasco совершеннее системы TDL4, поэтому требует более эффективного управления свободным пространством и структурами данных. Для этого добавлены два специальных файла: `$bad` и `$bitmap`.

Файл `$bitmap` содержит карту свободных секторов скрытой файловой системы - массив битов, каждый из которых соответствует сектору. Значение 1 означает, что сектор занят. `$bitmap` помогает найти место для хранения нового файла.

Файл `$bad` - битовая маска секторов, содержащих повреждённые файлы. Поскольку Olmasco захватывает под скрытую систему неразмеченное пространство в конце диска, другое ПО может записать туда данные и повредить файлы. Вредоносная программа отмечает такие секторы в `$bad`, чтобы больше их не использовать.

Оба системных файла находятся на одном уровне с корневым каталогом, недоступны полезной нагрузке и предназначены только для системных нужд. Интересно, что файлы с теми же именами существуют в NTFS. Возможно, Olmasco использует это, чтобы убедить пользователя в легитимности вредоносного раздела.

Реализация TDI для перенаправления сетевого взаимодействия

В скрытой файловой системе Olmasco находятся модули `tdi32` и `tdi64`, работающие с интерфейсом транспортных драйверов (TDI). Это сетевой интерфейс режима ядра, предоставляющий уровень абстракции между транспортными протоколами вроде TCP/IP и клиентами TDI вроде сокетов. Он открыт на верхней границе всех стеков транспортных протоколов. Фильтр TDI позволяет вредоносной программе перехватывать сетевое взаимодействие до его поступления транспортным протоколам.

Драйверы tdi32/tdi64 загружаются основным драйвером руткита drv32/drv64 посредством недокументированного API IoCreateDriver(L"\\Driver\\usbprt", tdi32EntryPoint), где tdi32EntryPoint - точка входа вредоносного драйвера TDI. В листинге 10-5 показана функция подключения TDI к объектам устройств.

```
NTSTATUS ___stdcall_ AttachToNetworkDevices(PDRIVER_OBJECT DriverObject,
                                    PUNICODE_STRING a2)
{
  NTSTATUS result;
  PDEVICE_OBJECT AttachedToTcp;
  PDEVICE_OBJECT AttachedToUdp;
  PDEVICE_OBJECT AttachedToIp;
  PDEVICE_OBJECT AttachedToRawIp;
  result = AttachToDevice(DriverObject, L"\\Device\\CFPTcpFlt",
                          ❶ L"\\Device\\Tcp", 0xF8267A6F, &AttachedToTcp);

  if ( result >= 0 ) {
    result = AttachToDevice(DriverObject, L"\\Device\\CFPUdpFlt",
                          ❷ L"\\Device\\Udp", 0xF8267AF0, &AttachedToUdp);
    if ( result >= 0 ) {
      AttachToDevice(DriverObject, L"\\Device\\CFPIpFlt",
                          ❸ L"\\Device\\Ip", 0xF8267A16, &AttachedToIp);
      AttachToDevice(DriverObject, L"\\Device\\CFPRawFlt",
                          ❹ L"\\Device\\RawIp", 0xF8267A7E, &AttachedToRawIp);
      result = 0;
    }
  }
  return result;
}
```

Листинг 10-5. Подключение драйвера TDI к сетевым устройствам

После этого вредоносный драйвер TDI подключается к следующим объектам сетевых устройств:

• \Device\Tcp предоставляет доступ к протоколу TCP ❶.
• \Device\Udp предоставляет доступ к протоколу UDP ❷.
• \Device\IP предоставляет доступ к протоколу IP ❸.
• \Device\RawIp предоставляет доступ к необработанному протоколу IP, то есть необработанным сокетам, ❹.

Основная функция вредоносного драйвера TDI - наблюдение за запросами TDI_CONNECT. При попытке подключиться по одному из перехваченных протоколов к IP-адресу 1.1.1.1 программа заменяет его адресом 69.175.67.172 и устанавливает номер порта 0x5000. Одна из причин - обход сетевого защитного ПО, работающего выше уровня TDI. Вредоносные компоненты могут пытаться подключиться к невредоносному адресу 1.1.1.1, который не должен привлечь внимание защиты и обрабатывается выше уровня TDI. Затем вредоносный компонент tdi заменяет исходное назначение адресом 69.175.67.172, и соединение перенаправляется на другой узел.

Заключение

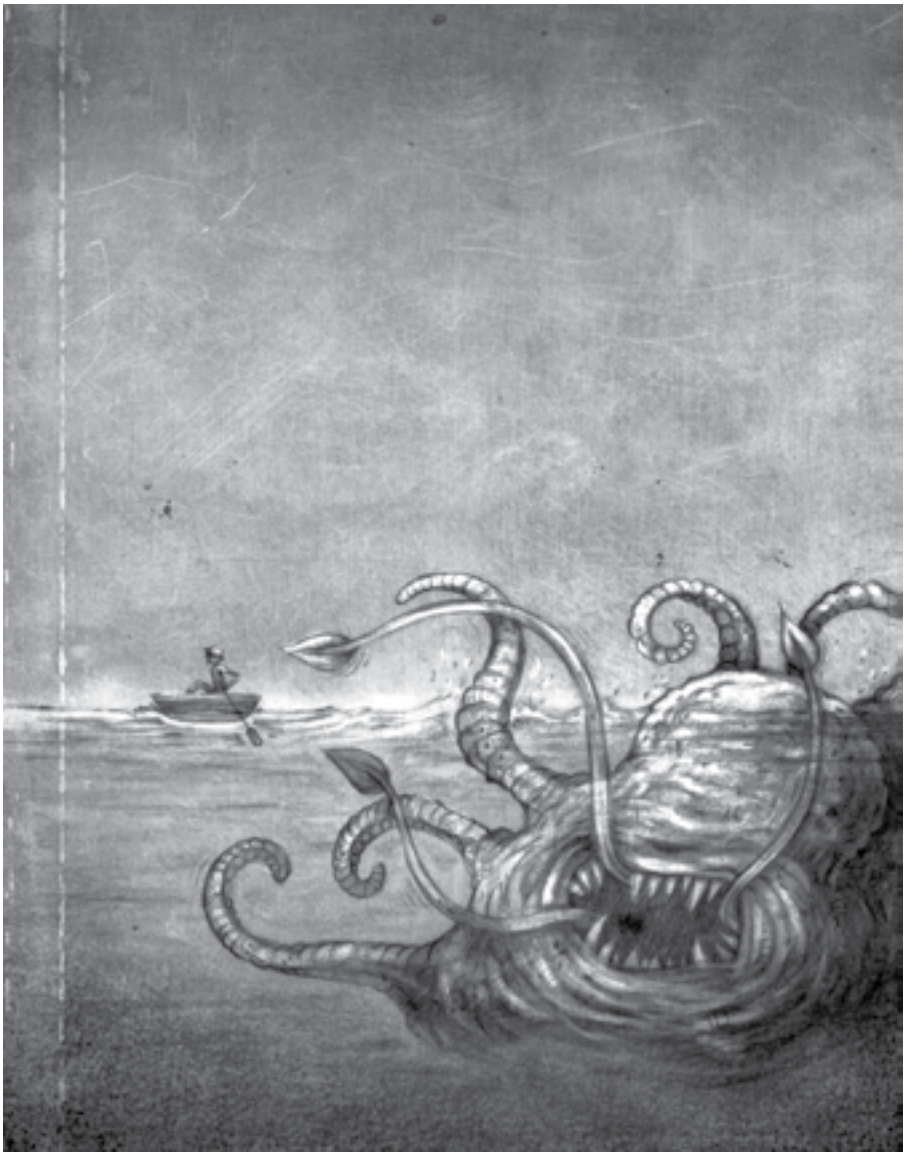
В этой главе мы рассмотрели, как буткит Olmasco использует таблицу разделов MBR как ещё один вектор заражения. Olmasco является потомком печально известного TDL4 и наследует значительную часть его функциональности, добавляя собственные приёмы. Сочетание изменения таблицы разделов MBR и поддельной VBR делает его скрытнее предшественника. В следующих главах будут рассмотрены ещё два буткита, атакующих VBR сложными методами заражения: Rovnix и Gapz.

На этой странице исходного издания нет основного текста.

Сноски

[1] [назад] Подробнее о схеме PPI для буткитов этого типа см. Андрей Рассохин и Дмитрий Олексюк, «Ботнет TDSS: полное раскрытие», web.archive.org.

Глава 11. IPL-буткиты: Rovnix и Carberp



Распространение Rovnix, первого известного буткита, заражавшего код IPL активного раздела загрузочного жёсткого диска, началось в конце 2011 года. К тому времени защитные продукты уже научились отслеживать MBR, как обсуждалось в главе 10, чтобы защищаться от таких буткитов, как TDL4 и Olmasco. Поэтому появление Rovnix в реальных атаках стало вызовом для защитного ПО. Поскольку Rovnix заходил дальше в процессе загрузки и заражал код IPL, выполнявшийся после кода VBR (см. главу 5), он несколько месяцев оставался незамеченным, пока индустрия безопасности не сумела наверстать упущенное.

В этой главе мы сосредоточимся на технических деталях платформы буткита Rovnix: изучим, как он заражает целевые системы и обходит политику подписывания кода режима ядра, чтобы загрузить вредоносный драйвер режима ядра. Особое внимание мы уделим вредоносному коду IPL и будем отлаживать его с помощью VMware и GDB для IDA Pro, как обсуждалось в главе 9. Наконец, мы рассмотрим реализацию Rovnix, встречавшуюся в реальных атаках: банковский троян Carberp, который использовал модификацию Rovnix для закрепления на компьютерах жертв.

Эволюция Rovnix

Впервые Rovnix рекламировался на закрытом подпольном форуме, показанном на рисунке 11-1, как новый пакет Ring0 с обширной функциональностью.

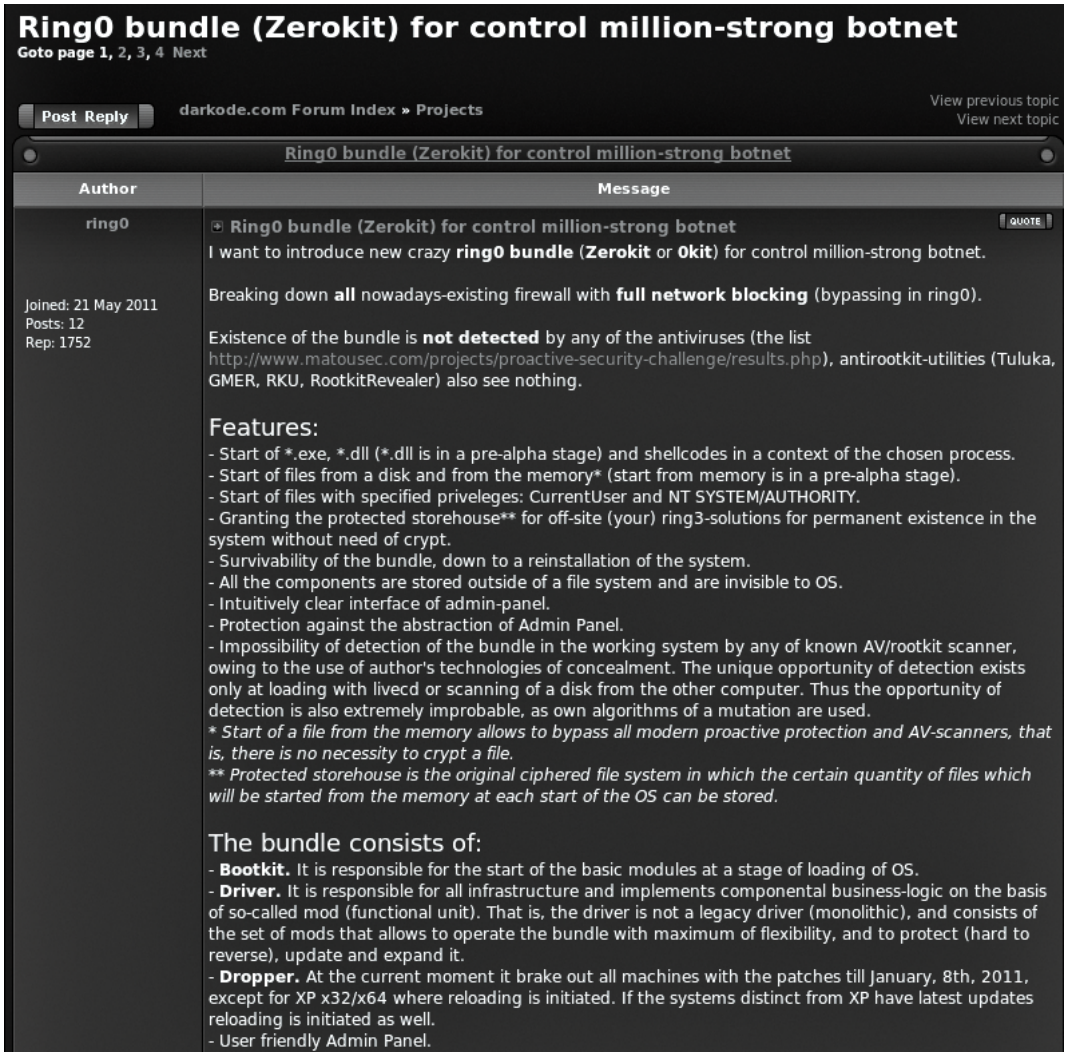


Рисунок 11-1. Реклама Rovnix на закрытом подпольном форуме

Он имел модульную архитектуру, делавшую его очень привлекательным для разработчиков и распространителей вредоносного ПО. Вероятно, его разработчики уделяли больше внимания продаже платформы, чем распространению и использованию вредоносной программы.

С момента первого появления в реальных атаках Rovnix прошёл через несколько итераций. В этой главе основное внимание будет уделено новейшему на момент написания поколению, но мы затронем и более ранние версии, чтобы дать представление о его развитии.

Первые итерации Rovnix использовали простой инфектор IPL, внедрявший полезную нагрузку в адресное пространство пользовательского режима загрузочных процессов. Во всех ранних итерациях применялся один и тот же вредоносный код IPL, поэтому индустрия безопасности смогла быстро разработать методы обнаружения на основе простых статических сигнатур.

Следующие версии Rovnix сделали эти методы обнаружения неэффективными, реализовав полиморфный вредоносный код IPL. В Rovnix появилась и другая новая возможность: скрытая файловая система для тайного хранения данных конфигурации, модулей полезной нагрузки и прочего. Вдохновившись буткитами, подобными TDL4, разработчики Rovnix также начали

реализовывать функциональность, отслеживавшую запросы чтения и записи к заражённому жёсткому диску, что затрудняло удаление вредоносной программы из системы.

В более поздней итерации появился скрытый канал связи, позволявший Rovnix обмениваться данными с удалёнными командными серверами и обходить контроль трафика, выполняемый персональными брандмауэрами и системами предотвращения вторжений на узел.

Теперь мы обратимся к новейшим известным на момент написания модификациям Rovnix, также называемым Win32/Rovnix.D, и подробно рассмотрим их возможности.

Архитектура буткита

Сначала рассмотрим архитектуру Rovnix на высоком уровне. На рисунке 11-2 показаны основные компоненты Rovnix и связи между ними.

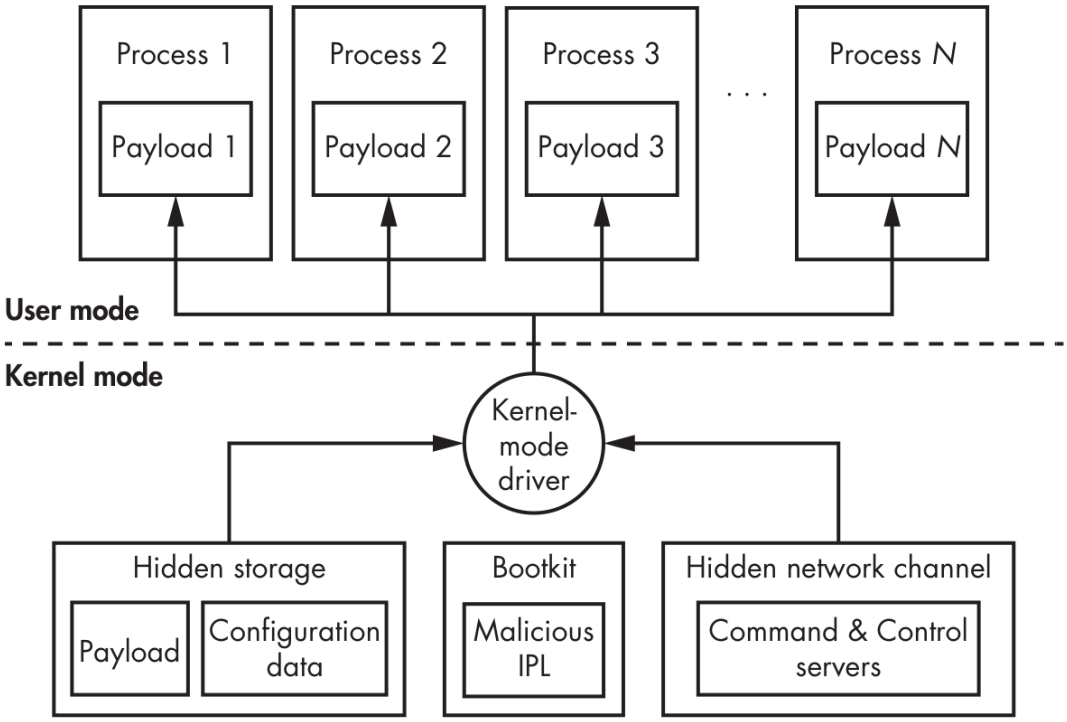


Рисунок 11-2. Архитектура Rovnix

В основе Rovnix находится вредоносный драйвер режима ядра, основное назначение которого - внедрять модули полезной нагрузки в процессы системы. Rovnix может хранить несколько полезных нагрузок для внедрения в разные процессы.

Пример такой полезной нагрузки - банковский троян, создающий поддельные транзакции, такой как рассматриваемый далее в этой главе троян Carberp. Вредоносный драйвер режима ядра Rovnix содержит жёстко заданный модуль полезной нагрузки по умолчанию, но способен загружать дополнительные модули с удалённых командных серверов через скрытый сетевой канал, рассматриваемый в разделе «Скрытый канал связи» на странице 169. Драйвер режима ядра также реализует скрытое хранилище для загруженных полезных нагрузок и конфигурационных сведений, подробно описанное в разделе «Скрытая файловая система» на странице 167.

Заражение системы

Продолжим анализ Rovnix разбором его алгоритма заражения, изображённого на рисунке 11-3.

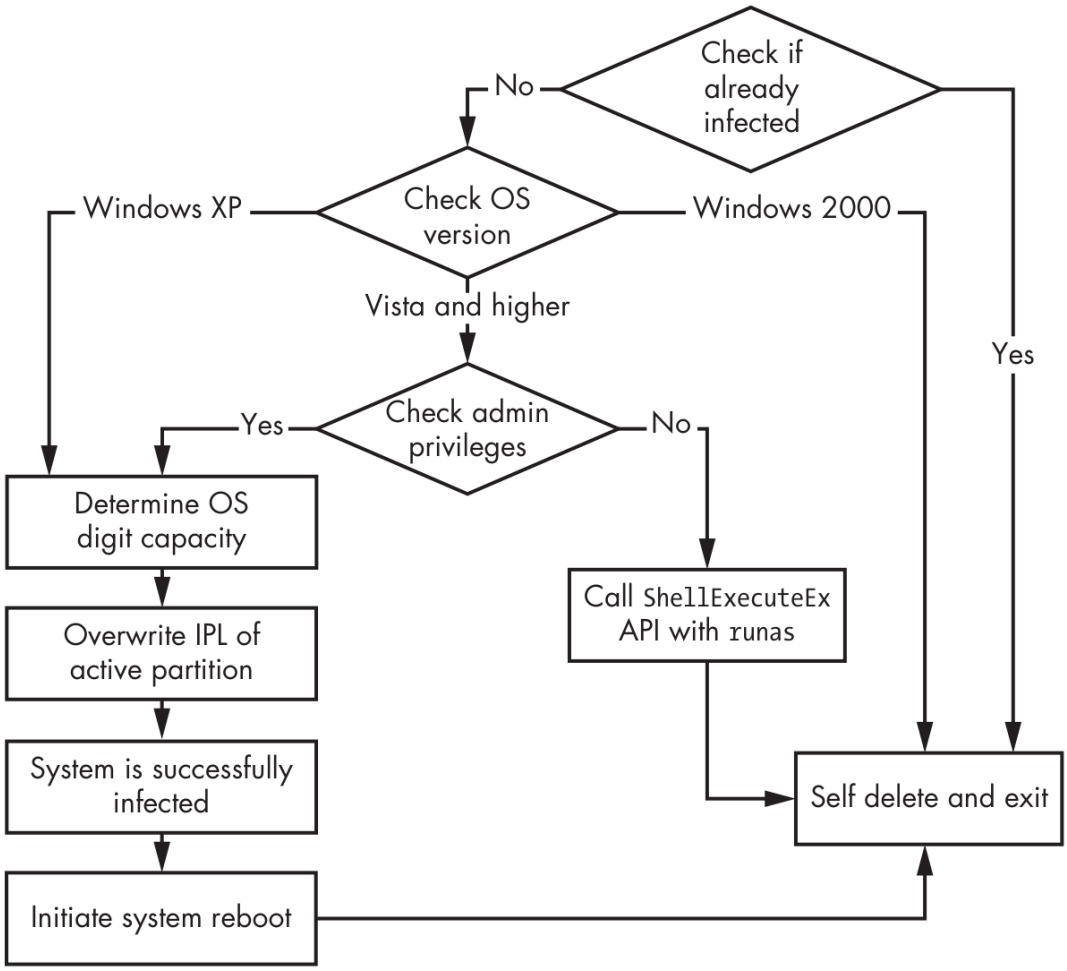


Рисунок 11-3. Алгоритм заражения дроппера Rovnix

Сначала Rovnix проверяет, не заражена ли система, обращаясь к разделу системного реестра `HKLM\Software\Classes\CLSID\<XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX>`, где символы `X` формируются из серийного номера тома файловой системы. Если этот раздел реестра существует, система уже заражена Rovnix, поэтому вредоносная программа завершает работу и удаляет себя из системы.

Если система ещё не заражена, Rovnix запрашивает версию операционной системы. Для низкоуровневого доступа к жёсткому диску вредоносной программе требуются права администратора. В Windows XP обычному пользователю по умолчанию предоставлены права администратора, поэтому в этой ОС Rovnix может продолжить работу от имени обычного пользователя без проверки прав.

Однако в Windows Vista корпорация Microsoft представила новую защитную функцию - контроль учётных записей (User Account Control, UAC), понижающую права приложений, работающих под учётной записью администратора. Поэтому в Vista и более новых версиях Rovnix должен проверять наличие административных прав. Если дроппер работает без них, Rovnix пытается повысить права, повторно запуская себя через API `ShellExecuteEx` с командой `runas`. Манифест дроппера содержит свойство `requireAdministrator`, поэтому команда `runas` пытается выполнить дроппер с повышенными правами. В системах с включённым UAC появляется диалоговое окно, предлагающее пользователю разрешить программе работу с правами администратора. Если

пользователь выбирает «Да», вредоносная программа запускается с повышенными правами и заражает систему. Если пользователь выбирает «Нет», вредоносная программа не выполняется. Если в системе нет UAC или он отключён, вредоносная программа просто работает с правами текущей учётной записи.

Получив необходимые права, Rovnix организует низкоуровневый доступ к жёсткому диску посредством функций нативного API `ZwOpenFile`, `ZwReadFile` и `ZwWriteFile`.

Сначала вредоносная программа вызывает `ZwOpenFile`, указывая в качестве имени файла `\??\PhysicalDrive0`, и получает дескриптор жёсткого диска. Затем Rovnix передаёт полученный дескриптор процедурам `ZwReadFile` и `ZwWriteFile`, чтобы читать данные с жёсткого диска и записывать их на него.

Для заражения системы вредоносная программа просматривает таблицу разделов в MBR жёсткого диска, затем читает IPL активного раздела и уменьшает его размер при помощи библиотеки сжатия aPlib. После этого Rovnix создаёт новый вредоносный IPL, помещая вредоносный код загрузчика перед сжатым легитимным IPL, как показано на рисунке 11-4.

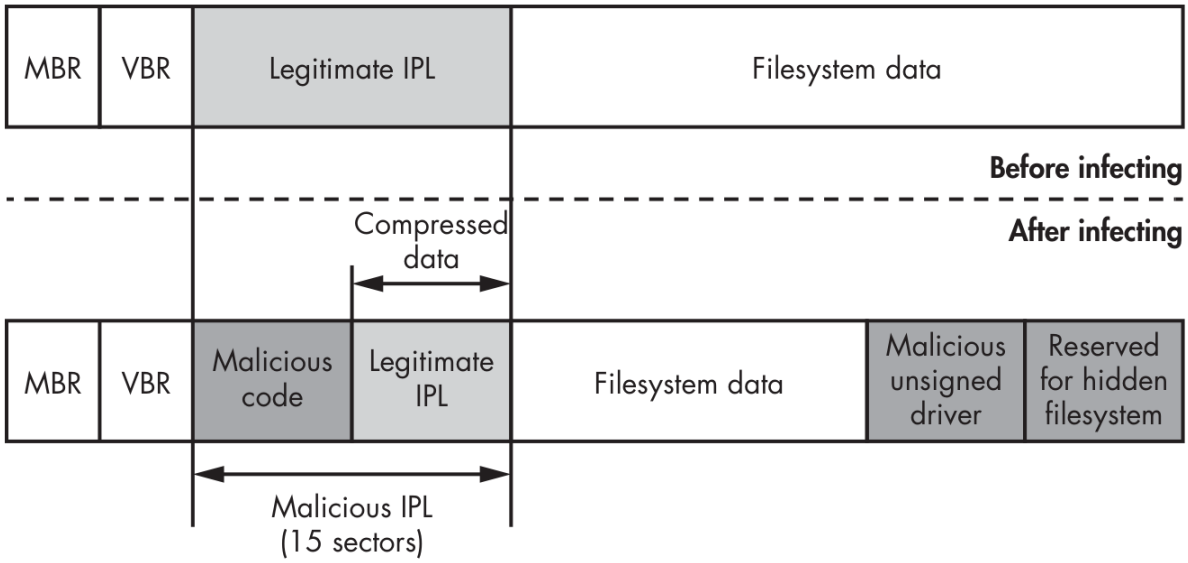


Рисунок 11-4. Разметка жёсткого диска до и после заражения Rovnix

Изменив IPL, Rovnix записывает в конец жёсткого диска вредоносный драйвер режима ядра, который вредоносный код IPL загрузит во время запуска системы. Вредоносная программа резервирует в конце жёсткого диска пространство для скрытой файловой системы, которую мы опишем далее в этой главе.

> **aPLIB**
>
> aPlib - небольшая библиотека сжатия, применяемая главным образом для сжатия исполняемого кода. Она основана на алгоритме сжатия, используемом программой aPack для упаковки исполняемых файлов. Одна из отличительных особенностей библиотеки - хорошее соотношение степени сжатия и скорости при крошечном размере распаковщика, что особенно важно в предзагрузочной среде с очень малым объёмом памяти. Библиотека сжатия aPlib также часто используется во вредоносном ПО для упаковки и запутывания полезной нагрузки.

Наконец, Rovnix создаёт раздел системного реестра, помечающий систему как заражённую, и инициирует перезапуск, вызывая Win32 API `ExitWindowsEx` с параметрами `EWX_REBOOT | EWX_FORCE`.

Процесс загрузки после заражения и IPL

После того как Rovnix заражает компьютер и принудительно перезагружает его, загрузочный код BIOS работает как обычно, загружая и выполняя неизменённую MBR загрузочного жёсткого диска. MBR находит активный раздел на жёстком диске и выполняет легитимную неизменённую VBR. Затем VBR загружает и выполняет заражённый код IPL.

Реализация полиморфного дешифратора

Заражённый IPL начинается с небольшого дешифратора, задача которого - расшифровать остальную часть вредоносного кода IPL и выполнить её (рисунок 11-5). Полиморфность дешифратора означает, что каждый экземпляр Rovnix содержит уникальный код дешифратора.

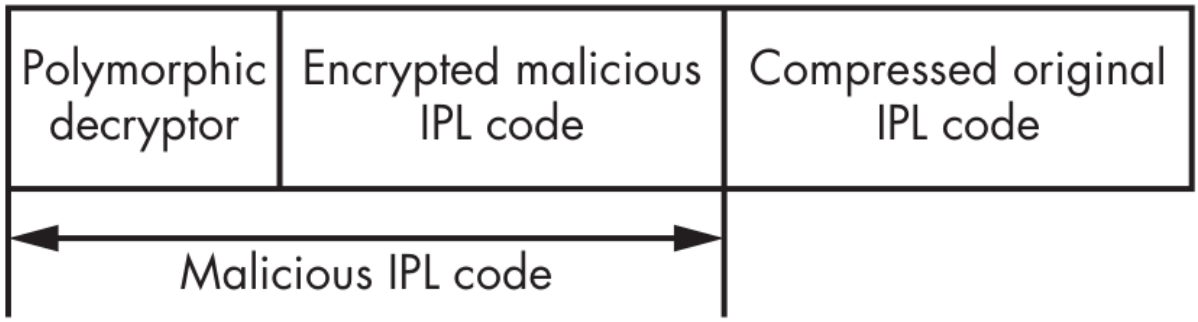


Рисунок 11-5. Разметка заражённого IPL

Рассмотрим реализацию дешифратора. Прежде чем анализировать сам полиморфный код, дадим общее описание алгоритма расшифрования. Для расшифрования содержимого вредоносного IPL дешифратор выполняет следующие действия:

1. Выделяет буфер памяти для хранения расшифрованного кода.
2. Инициализирует ключ расшифрования и счётчики расшифрования - соответственно смещение и размер зашифрованных данных.
3. Расшифровывает код IPL в выделенный буфер.
4. Инициализирует регистры перед выполнением расшифрованного кода.
5. Передаёт управление расшифрованному коду.

Чтобы сделать процедуру расшифрования уникальной, Rovnix случайным образом разбивает её на базовые блоки - наборы последовательных инструкций без ветвлений, каждый из которых содержит небольшое число ассемблерных инструкций процедуры. Затем Rovnix перемешивает базовые блоки и располагает их в случайном порядке, связывая инструкциями `jmp`, как показано на рисунке 11-6. В результате каждый экземпляр Rovnix получает уникальный код расшифрования.

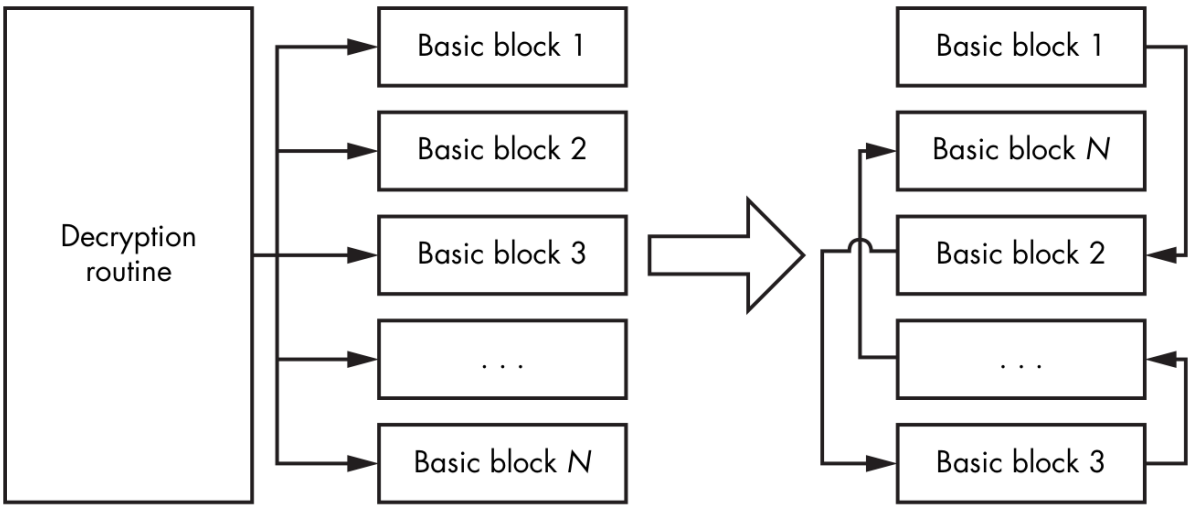


Рисунок 11-6. Формирование полиморфного дешифратора

Этот механизм полиморфизма на самом деле довольно прост по сравнению с некоторыми другими методами запутывания кода, применяемыми в современном вредоносном ПО. Но поскольку последовательность байтов процедуры меняется в каждом экземпляре Rovnix, этого достаточно, чтобы избегать обнаружения защитным ПО, использующим статические сигнатуры.

Однако полиморфизм не неуязвим, и один из самых распространённых способов его преодоления - программная эмуляция. При эмуляции защитное ПО применяет поведенческие шаблоны для обнаружения вредоносных программ.

Расшифрование загрузчика Rovnix при помощи VMware и IDA Pro

Рассмотрим фактическую реализацию процедуры расшифрования при помощи виртуальной машины VMware и IDA Pro. Все необходимые сведения о настройке VMware с IDA Pro приведены в главе 9.

В этой демонстрации мы воспользуемся образом VMware, заранее заражённым буткитом Win32/Rovnix.D. Его можно скачать с nostarch.com в составе файла `bootkit_files.zip`.

Наша цель - получить расшифрованный вредоносный код IPL посредством динамического анализа. Мы проведём вас через процесс отладки, быстро миновав этапы MBR и VBR, чтобы сосредоточиться на анализе полиморфного дешифратора IPL.

Наблюдение за кодом MBR и VBR

Вернитесь к разделу «Совместное использование GDB VMware и IDA» на странице 126 и выполните приведённые там действия, чтобы расшифровать MBR из `bootkit_files.zip`. Код MBR находится по адресу `0000:7c00h`. На рисунке 11-7 адрес `0000:7c00h` обозначен как `MEMORY:7c00h`, поскольку IDA Pro выводит имя сегмента, в нашем случае `MEMORY`, вместо базового адреса сегмента `0000h`. Rovnix заражает код IPL, а не MBR, поэтому показанный в отладчике код MBR легитимен, и мы не будем подробно его разбирать.

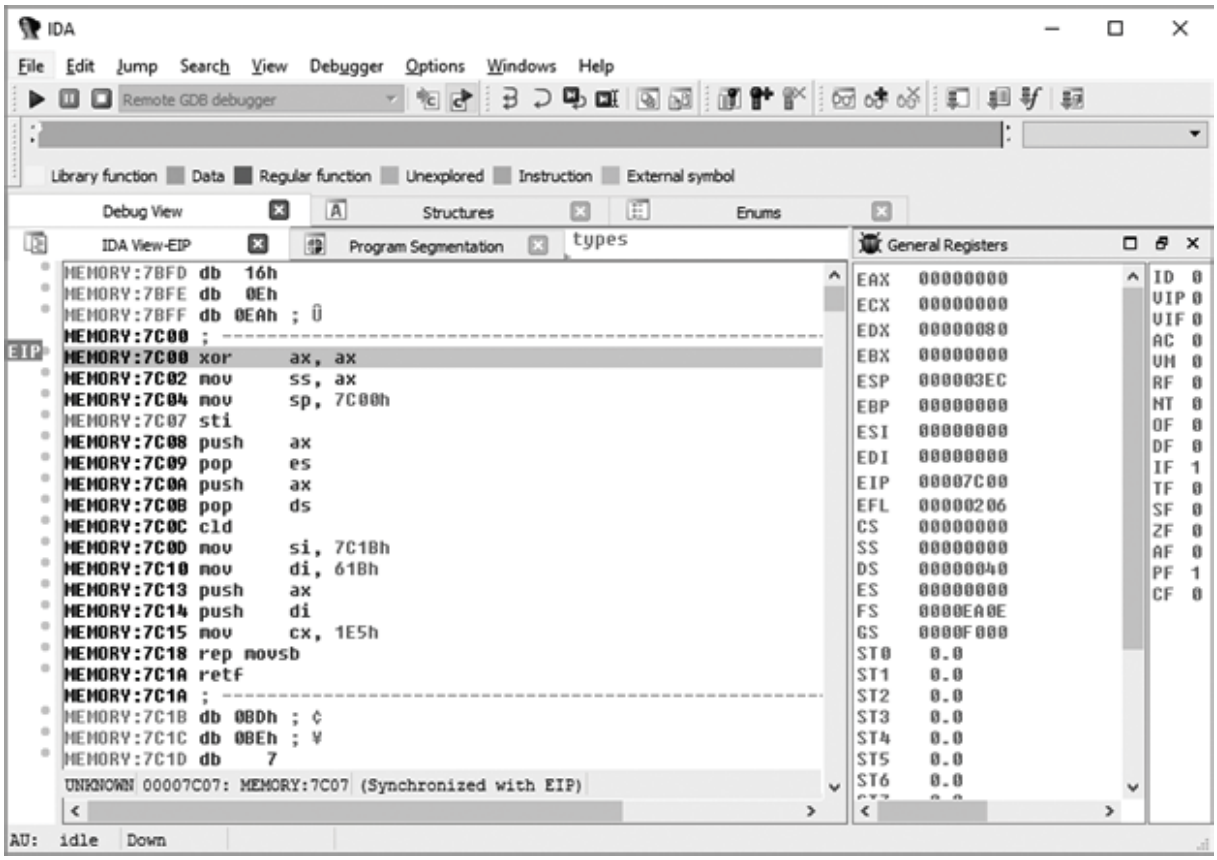


Рисунок 11-7. Начало кода MBR

Этот код процедуры перемещает MBR по другому адресу памяти, чтобы повторно использовать память по адресу `0000:7c00h` для чтения и хранения VBR активного раздела. Регистр `si` ❷ инициализируется значением `7C1h`, соответствующим исходному адресу, а регистр `di` ❸ - значением `61Bh`, адресом назначения. Регистр `cx` ❹ инициализируется значением `1E5h`, числом копируемых байтов, а инструкция `rep movsb` ❺ копирует байты. Инструкция `retf` ❻ передаёт управление скопированному коду.

В этот момент регистр указателя инструкций `ip` указывает на адрес `0000:7c00h` ❶. Выполняйте каждую инструкцию листинга, нажимая F8, пока не дойдёте до последней инструкции `retf` ❻. После выполнения `retf` управление передаётся коду, только что скопированному по адресу `0000:061Bh`, то есть основной процедуре MBR, задача которой - найти активный раздел в таблице разделов MBR и загрузить его самый первый сектор, VBR.

VBR также остаётся неизменённой, поэтому перейдём к следующему этапу, установив точку останова в самом конце процедуры. Инструкция `retf` по адресу `0000:069Ah` передаёт управление непосредственно коду VBR активного раздела, поэтому установим точку останова на инструкции `retf`, выделенной на рисунке 11-8. Переместите курсор на этот адрес и нажмите F2, чтобы включить точку останова. Если после нажатия F2 появится диалоговое окно, просто нажмите «OK», чтобы использовать значения по умолчанию.

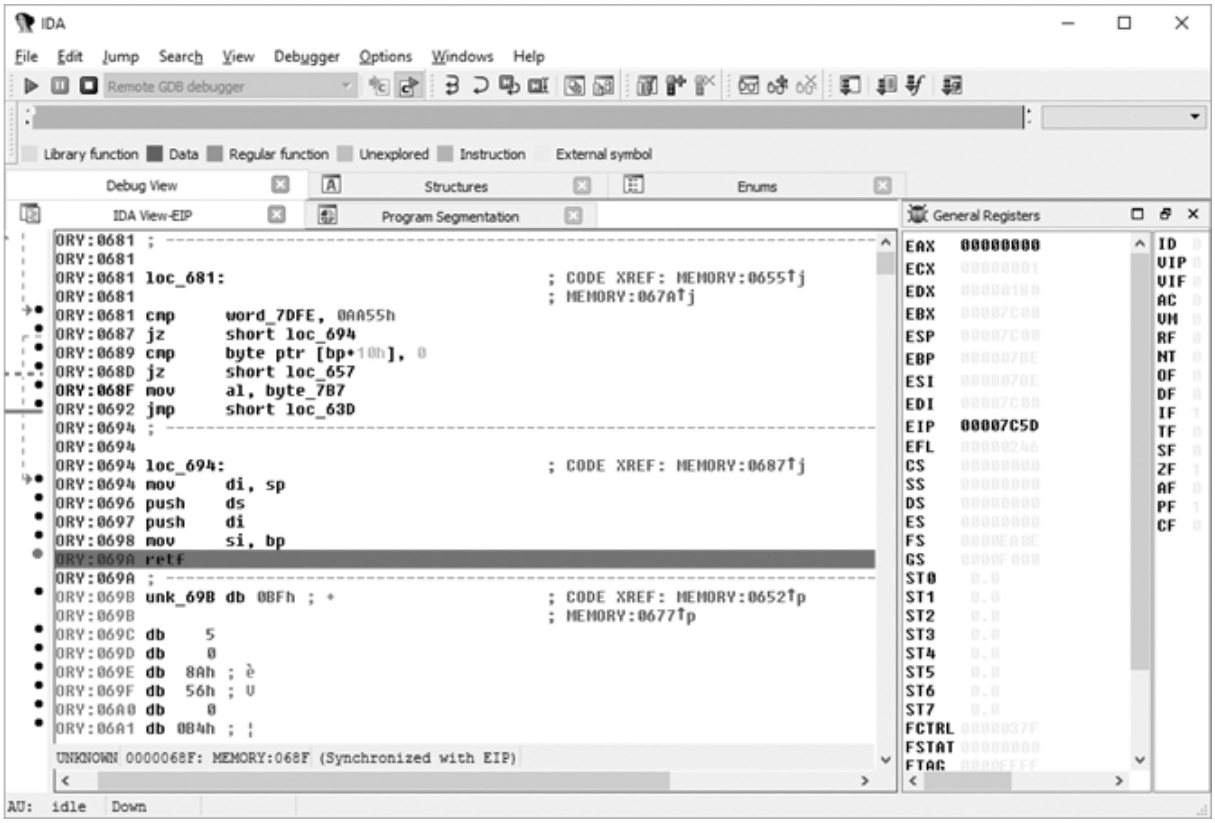


Рисунок 11-8. Установка точки останова в конце кода MBR

Установив точку останова, нажмите F9, чтобы продолжить анализ до неё. При этом выполнится основная процедура MBR. Когда выполнение достигнет точки останова, VBR уже будет прочитана в память, и мы сможем перейти к ней, выполнив инструкцию `retf` клавишей F8.

Код VBR начинается с инструкции `jmp`, передающей управление процедуре, которая читает IPL в память и выполняет его. Дизассемблированный код процедуры показан на рисунке 11-9. Чтобы перейти непосредственно к вредоносному коду IPL, установите точку останова на последней инструкции процедуры VBR по адресу `0000:7C7Ah` ❶ и снова нажмите F9, чтобы продолжить выполнение. Когда выполнение достигнет точки останова, отладчик остановится на инструкции `retf`. Выполните эту инструкцию клавишей F8, чтобы перейти к вредоносному коду IPL.

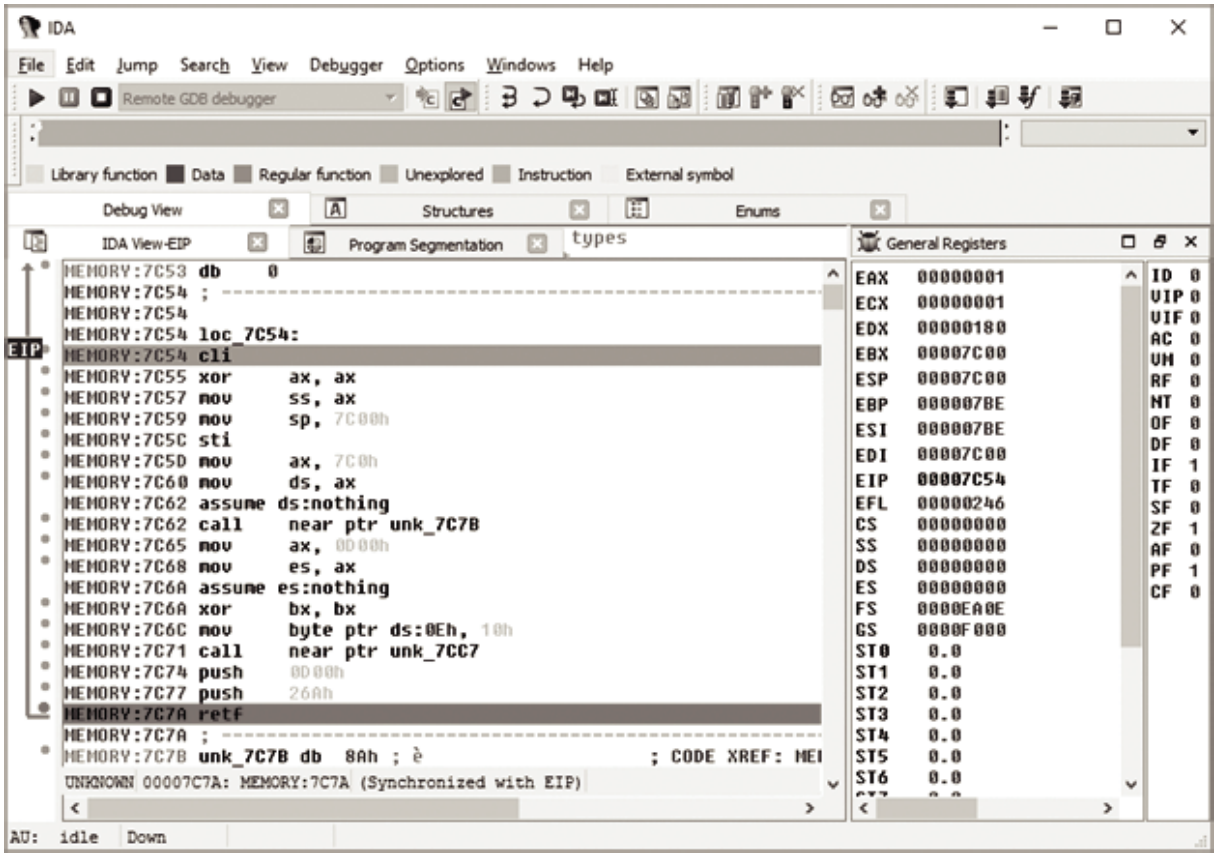


Рисунок 11-9. Код VBR

Разбор полиморфного дешифратора IPL

Вредоносный код IPL начинается с последовательности инструкций, объединённых в базовые блоки, которые инициализируют регистры перед выполнением дешифратора. За ними следует инструкция `call`, передающая управление дешифратору IPL.

Код в первом базовом блоке дешифратора (листинг 11-1) получает базовый адрес вредоносного IPL в памяти ❶ и сохраняет его в стеке ❷. Инструкция `jmp` ❸ передаёт управление второму базовому блоку (вспомните рисунок 11-6).

```
MEMORY:D984 pop     ax
MEMORY:D985 sub     ax, 0Eh ❶
MEMORY:D988 push    cs
MEMORY:D989 push    ax ❷
MEMORY:D98A push    ds
MEMORY:D98B jmp     short loc_D9A0 ❸
```

Листинг 11-1. Базовый блок 1 полиморфного дешифратора

Второй и третий базовые блоки вместе реализуют один этап алгоритма расшифрования - выделение памяти, поэтому они показаны вместе в листинге 11-2.

```
; Базовый блок № 2
MEMORY:D9A0 push    es
MEMORY:D9A1 pusha
MEMORY:D9A2 mov     di, 13h
MEMORY:D9A5 push    40h ; '@'

MEMORY:D9A7 pop     ds
MEMORY:D9A8 jmp     short loc_D95D
--snip--
; Базовый блок № 3
MEMORY:D95D mov     cx, [di]
MEMORY:D95F sub     ecx, 3 ❶
MEMORY:D963 mov     [di], cx
MEMORY:D965 shl     cx, 6
MEMORY:D968 push    cs
MEMORY:D98B jmp     short loc_D98F ❷
```

Листинг 11-2. Базовые блоки 2 и 3 полиморфного дешифратора

Код выделяет 3 Кбайт памяти (о выделении памяти в реальном режиме см. главу 5) и сохраняет адрес памяти в регистре `cx`. Выделенная память будет использоваться для хранения расшифрованного вредоносного кода IPL. Затем код читает общий объём доступной в реальном режиме выполнения памяти по адресу `0040:0013h` и уменьшает это значение на 3 Кбайт ❶. Инструкция `jmp` ❷ передаёт управление следующему базовому блоку.

Базовые блоки с 4-го по 8-й, показанные в листинге 11-3, реализуют инициализацию ключа и счётчика расшифрования, а также цикл расшифрования.

```
; Базовый блок № 4
MEMORY:D98F pop     ds
MEMORY:D990 mov     bx, sp
MEMORY:D992 mov     bp, 4D4h
MEMORY:D995 jmp     short loc_D954
--snip--
; Базовый блок № 5
MEMORY:D954 push    ax
MEMORY:D955 push    cx
MEMORY:D956 add     ax, 0Eh
❶ MEMORY:D959 mov     si, ax
MEMORY:D95B jmp     short loc_D96B
--snip--
; Базовый блок № 6
MEMORY:D96B add     bp, ax
MEMORY:D96D xor     di, di
❷ MEMORY:D96F pop     es
MEMORY:D970 jmp     short loc_D93E
--snip--
; Базовый блок № 7
❸ MEMORY:D93E mov     dx, 0FCE8h
MEMORY:D941 cld
❹ MEMORY:D942 mov     cx, 4C3h
MEMORY:D945 loc_D945:
❺ MEMORY:D945 mov     ax, [si]
❻ MEMORY:D947 xor     ax, dx
MEMORY:D949 jmp     short loc_D972
--snip--

; Базовый блок № 8
❼ MEMORY:D972 mov     es:[di], ax
MEMORY:D975 add     si, 2
MEMORY:D978 add     di, 2
MEMORY:D97B loop    loc_D945
MEMORY:D97D pop     di
MEMORY:D97E mov     ax, 25Eh
MEMORY:D981 push    es
❽ MEMORY:D982 jmp     short loc_D94B
```

Листинг 11-3. Базовые блоки с 4-го по 8-й полиморфного дешифратора

По адресу `0000:D959h` регистр `si` инициализируется адресом зашифрованных данных ❶. Инструкции в точке ❷ инициализируют регистры `es` и `di` адресом буфера, выделенного для хранения расшифрованных данных. Регистр `dx` по адресу `0000:D93Eh` ❸ инициализируется ключом расшифрования `0FCE8h`, а регистр `cx` - числом операций XOR, выполняемых в цикле расшифрования ❹. При каждой операции XOR два байта зашифрованных данных объединяются по XOR с ключом расшифрования, поэтому значение регистра `cx` равно `число_расшифровываемых_байтов`, делённому на 2.

Инструкции в цикле расшифрования читают два байта из источника ❺, объединяют их по XOR с ключом ❻ и записывают результат в буфер назначения ❼. После завершения расшифрования инструкция `jmp` ❽ передаёт управление следующему базовому блоку.

Базовые блоки с 9-го по 11-й инициализируют регистры и передают управление расшифрованному коду (листинг 11-4).

```
; Базовый блок № 9
MEMORY:D94B push    ds
MEMORY:D94C pop     es
MEMORY:D94D mov     cx, 4D4h
MEMORY:D950 add     ax, cx
MEMORY:D952 jmp     short loc_D997
--snip--
; Базовый блок № 10
MEMORY:D997 mov     si, 4B2h
❶ MEMORY:D99A push    ax
MEMORY:D99B push    cx
MEMORY:D99C add     si, bp
MEMORY:D99E jmp     short loc_D98D
--snip--
; Базовый блок № 11
MEMORY:D98D pop     bp
❷ MEMORY:D98E retf
```

Листинг 11-4. Базовые блоки с 9-го по 11-й полиморфного дешифратора

Инструкции в точке ❶ сохраняют в стеке адрес расшифрованного кода IPL, который будет выполнен после расшифрования, а инструкция `retf` ❷ извлекает этот адрес из стека и передаёт ему управление.

Чтобы получить расшифрованный код IPL, нужно определить адрес буфера с расшифрованными данными. Для этого установим точку останова по адресу `0000:D970h` сразу после инструкции ❷ в листинге 11-3 и продолжим выполнение, как показано на рисунке 11-10.

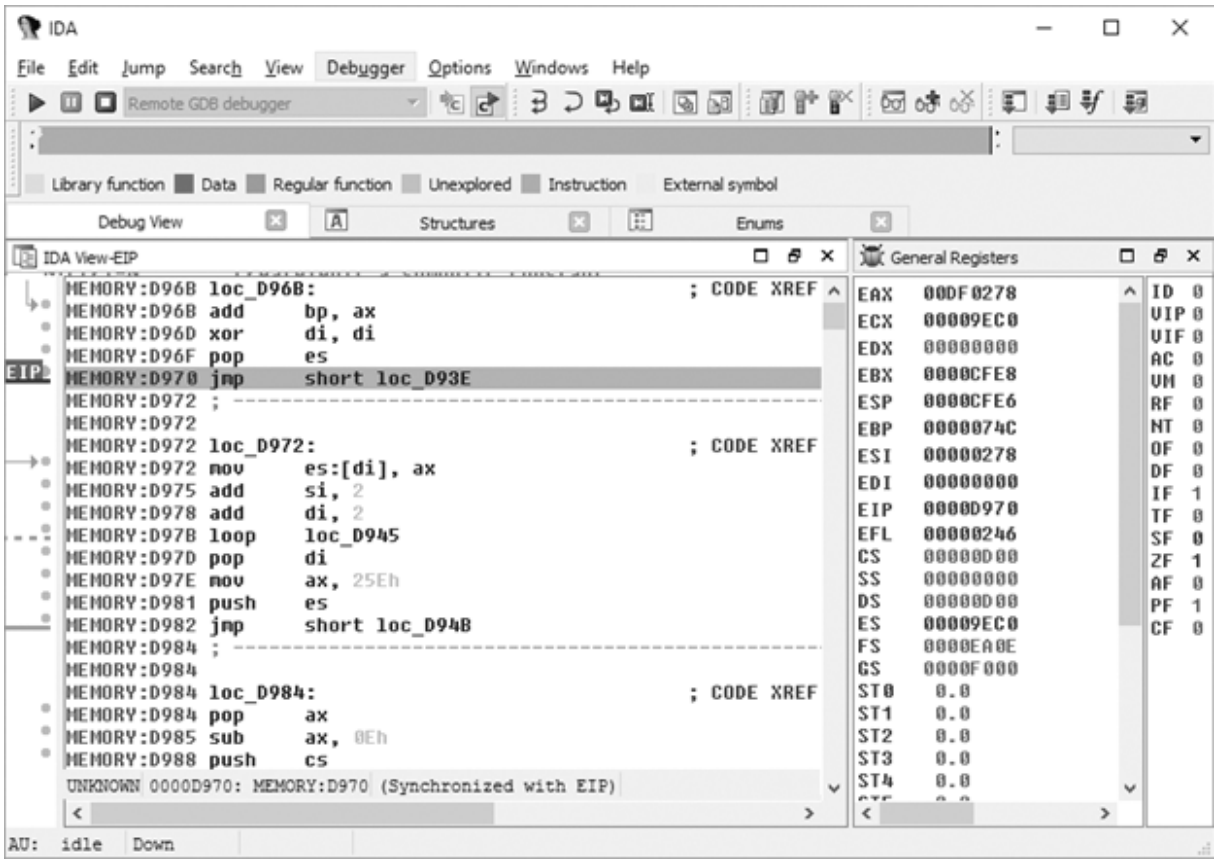


Рисунок 11-10. Установка точки останова в IDA Pro

Затем установим точку останова по адресу `0000:D98Eh` (❷ в листинге 11-4), на последней инструкции полиморфного дешифратора, и позволим выполниться остальному коду дешифратора. Когда отладчик остановится по этому адресу, выполним последнюю инструкцию `retf`, которая перенесёт нас непосредственно к расшифрованному коду по адресу `9EC0:0732h`.

Теперь вредоносный код IPL расшифрован в памяти и доступен для дальнейшего анализа. Обратите внимание: после расшифрования первая процедура вредоносного IPL расположена не в самом начале расшифрованного буфера по адресу `9EC0:0000h`, а со смещением `732h`, что обусловлено разметкой вредоносного IPL. Если вы хотите выгрузить содержимое буфера из памяти в файл на диске для статического анализа, начинайте выгрузку с адреса `9EC0:0000h`, где начинается буфер.

Получение управления посредством исправления загрузчика Windows

Основное назначение кода IPL Rovnix - загрузить вредоносный драйвер режима ядра. Вредоносный загрузочный код тесно взаимодействует с компонентами загрузчика ОС и следует за ходом выполнения с самого начала процесса загрузки, через переключение режима выполнения процессора и вплоть до загрузки ядра ОС. Загрузчик в значительной степени полагается на средства отладки платформы и двоичное представление компонентов загрузчика ОС.

После выполнения расшифрованного вредоносного кода IPL он перехватывает обработчик INT 13h, чтобы отслеживать все данные, читаемые с жёсткого диска, и устанавливать дальнейшие перехваты в компонентах загрузчика ОС. Затем вредоносный IPL распаковывает исходный код IPL и возвращает ему управление, возобновляя обычный процесс загрузки.

На рисунке 11-11 показаны действия Rovnix по вмешательству в процесс загрузки и компрометации ядра ОС. Мы уже рассмотрели этапы до четвёртого блока, поэтому продолжим описание функциональности буткита с этапа «Загрузить bootmgr» ❶.

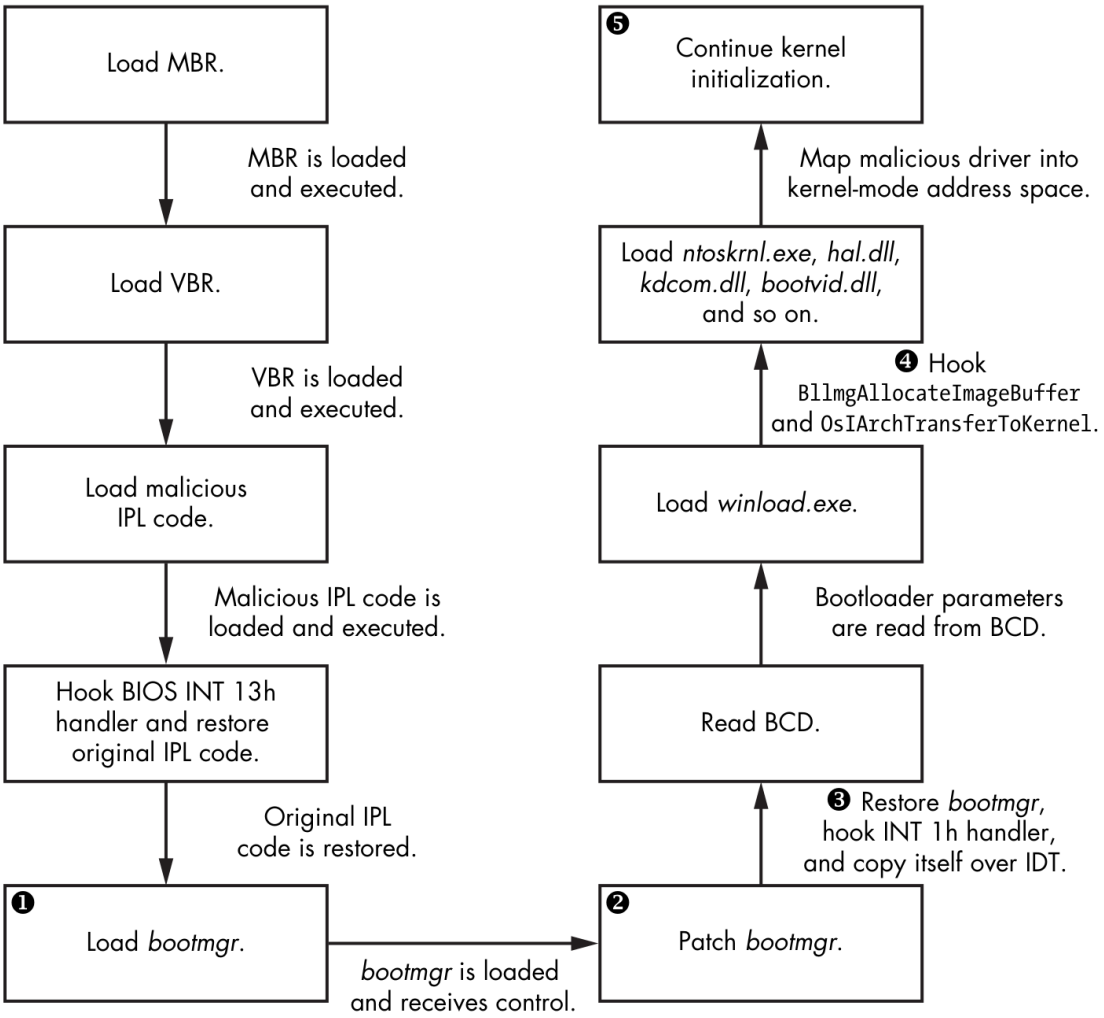


Рисунок 11-11. Процесс загрузки кода IPL Rovnix

Перехватив обработчик INT 13h, Rovnix отслеживает все данные, читаемые с жёсткого диска, и ищет определённую последовательность байтов, соответствующую `bootmgr` операционной системы. Обнаружив совпадающую последовательность, он изменяет `bootmgr` ❷, чтобы тот мог обнаружить переключение процессора из реального режима в защищённый - стандартный этап процесса загрузки. Это переключение режима выполнения изменяет преобразование виртуальных адресов в физические и, как следствие, разметку виртуальной памяти, что вытеснило бы Rovnix. Поэтому, чтобы перенести себя через переключение и сохранить управление процессом загрузки, Rovnix перехватывает `bootmgr`, исправляя его инструкцией `jmp`. Благодаря этому Rovnix получает управление непосредственно перед переключением режима выполнения ОС.

Прежде чем продолжить, рассмотрим, как Rovnix скрывает свои перехваты, а затем выясним, каким именно образом он сохраняется при переключении режимов.

Злоупотребление интерфейсом отладки для сокрытия перехватов

Одна из особенностей, делающих Rovnix ещё интереснее других буткитов, - скрытность его перехватов управления. Он перехватывает обработчик INT 1h ❸, чтобы получать управление в определённые моменты инициализации ядра ОС, и злоупотребляет отладочными регистрами с `dr0` по `dr7`, устанавливая перехваты, которые не изменяют перехватываемый код и тем самым избегают обнаружения. Обработчик INT 1h отвечает за обработку событий отладки, таких как трассировка и аппаратные точки останова, при помощи регистров с `dr0` по `dr7`.

Восемь отладочных регистров с `dr0` по `dr7` обеспечивают аппаратную поддержку отладки на платформах Intel x86 и x64. Первые четыре регистра, с `dr0` по `dr3`, служат для указания линейных адресов точек останова. Регистр `dr7` позволяет выборочно задавать и включать условия срабатывания точек останова: например, можно установить точку, срабатывающую при выполнении кода либо при обращении к памяти (чтении или записи) по определённому адресу. Регистр `dr6` - регистр состояния, позволяющий определить, какое условие отладки возникло, то есть какая точка останова сработала. Регистры `dr4`[1] и `dr5` зарезервированы и не используются. После срабатывания аппаратной точки останова выполняется INT 1h, чтобы определить возникшее условие отладки и соответствующим образом передать его на обработку.

Именно эта функциональность позволяет буткиту Rovnix устанавливать скрытные перехваты без исправления кода. Rovnix задаёт в регистрах с `dr0` по `dr4` адреса предполагаемых мест перехвата и включает аппаратные точки останова для каждого регистра, устанавливая соответствующую битовую маску в регистре `dr7`.

Злоупотребление таблицей дескрипторов прерываний для сохранения при загрузке

Помимо злоупотребления средствами отладки платформы первые итерации Rovnix применяли интересный приём, позволявший пережить переключение процессора из реального режима в защищённый. Перед переключением выполнения в защищённый

режим `bootmgr` инициализирует важные системные структуры, такие как глобальная таблица дескрипторов и таблица дескрипторов прерываний (Interrupt Descriptor Table, IDT). Последняя заполняется дескрипторами обработчиков прерываний.

Таблица дескрипторов прерываний

IDT - специальная системная структура, которую центральный процессор в защищённом режиме использует для задания обработчиков прерываний процессора. В реальном режиме IDT, также называемая таблицей векторов прерываний (Interrupt Vector Table, IVT), устроена просто: это всего лишь массив четырёхбайтовых адресов обработчиков, начинающийся по адресу `0000:0000h`. Иными словами, адрес обработчика INT 0h - `0000:0000h`, адрес обработчика INT 1h - `0000:0004h`, адрес обработчика INT 2h - `0000:0008h` и так далее. В защищённом режиме IDT имеет более сложную разметку: это массив восьмибайтовых дескрипторов обработчиков прерываний. Базовый адрес IDT можно получить инструкцией процессора `sidt`. Дополнительные сведения об IDT приведены в документации Intel по адресу intel.com.

Rovnix копирует вредоносный код IPL поверх второй половины IDT, которая в этот момент не используется системой. Поскольку каждый дескриптор занимает 8 байт, а таблица содержит 256 дескрипторов, Rovnix получает 1 Кбайт памяти IDT, чего достаточно для размещения его вредоносного кода. IDT находится в защищённом режиме, поэтому хранение кода в IDT гарантирует, что Rovnix сохранится при переключении режимов, а адрес IDT легко получить инструкцией `sidt`. Общая разметка IDT после изменений Rovnix показана на рисунке 11-12.

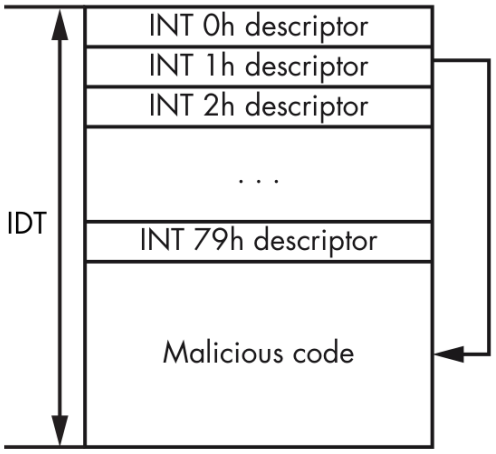


Рисунок 11-12. Как Rovnix злоупотребляет IDT для переноса через переключение режима выполнения

Загрузка вредоносного драйвера режима ядра

Перехватив обработчик INT 1h, Rovnix продолжает перехватывать другие компоненты загрузчика ОС, такие как `winload.exe` и образ ядра ОС, например `ntoskrnl.exe`. Rovnix ожидает, пока код `bootmgr` загрузит `winload.exe`, а затем перехватывает процедуру `BlImgAllocateImageBuffer` (см. ❹ на рисунке 11-11), выделяющую буфер для исполняемого образа, устанавливая аппаратную точку останова по её начальному адресу. Таким способом выделяется память для вредоносного драйвера режима ядра.

Вредоносная программа также перехватывает процедуру `OslArchTransferToKernel` в `winload.exe`. Эта процедура передаёт управление из `winload.exe` точке входа ядра `KiSystemStartup`, которая начинает инициализацию ядра. Перехватив `OslArchTransferToKernel`, Rovnix получает управление непосредственно перед вызовом `KiSystemStartup` и использует эту возможность для внедрения вредоносного драйвера режима ядра.

Процедура `KiSystemStartup` принимает единственный параметр `KeLoaderBlock`, представляющий собой указатель на `LOADER_PARAMETER_BLOCK` - недокументированную структуру, которую инициализирует `winload.exe` и которая содержит важные системные сведения, такие как параметры загрузки и загруженные модули. Эта структура показана в листинге 11-5.

```
typedef struct _LOADER_PARAMETER_BLOCK
{
     LIST_ENTRY LoadOrderListHead;
     LIST_ENTRY MemoryDescriptorListHead;
     ❶ LIST_ENTRY BootDriverListHead;
     ULONG KernelStack;
     ULONG Prcb;
     ULONG Process;
     ULONG Thread;
     ULONG RegistryLength;
     PVOID RegistryBase;
```

```
    PCONFIGURATION_COMPONENT_DATA ConfigurationRoot;
    CHAR * ArcBootDeviceName;
    CHAR * ArcHalDeviceName;
    CHAR * NtBootPathName;
    CHAR * NtHalPathName;
    CHAR * LoadOptions;
    PNLS_DATA_BLOCK NlsData;
    PARC_DISK_INFORMATION ArcDiskInformation;
    PVOID OemFontFile;
    _SETUP_LOADER_BLOCK * SetupLoaderBlock;
    PLOADER_PARAMETER_EXTENSION Extension;
    BYTE u[12];
    FIRMWARE_INFORMATION_LOADER_BLOCK FirmwareInformation;
} LOADER_PARAMETER_BLOCK, *PLOADER_PARAMETER_BLOCK;
```

Листинг 11-5. Описание `LOADER_PARAMETER_BLOCK`

Rovnix интересует поле `BootDriverListHead` ❶, содержащее начало списка специальных структур данных, соответствующих драйверам загрузочного режима. Эти драйверы загружаются `winload.exe` одновременно с образом ядра. Однако процедура `DriverEntry`, инициализирующая драйверы, вызывается лишь после того, как образ ядра ОС получит управление. Код инициализации ядра ОС обходит записи в `BootDriverListHead` и вызывает процедуру `DriverEntry` соответствующего драйвера.

Когда срабатывает перехват `OslArchTransferToKernel`, Rovnix получает из стека адрес структуры `KeLoaderBlock` и посредством поля `BootDriverListHead` вставляет в список загрузочных драйверов запись, соответствующую вредоносному драйверу. Теперь вредоносный драйвер загружен в память так, словно это драйвер режима ядра с легитимной цифровой подписью. Затем Rovnix передаёт управление процедуре `KiSystemStartup`, которая возобновляет процесс загрузки и начинает инициализацию ядра (❺ на рисунке 11-11).

На некотором этапе инициализации ядро обходит список загрузочных драйверов в `KeLoaderBlock` и вызывает их процедуры инициализации, в том числе процедуру вредоносного драйвера (рисунок 11-13). Именно так выполняется процедура `DriverEntry` вредоносного драйвера режима ядра.

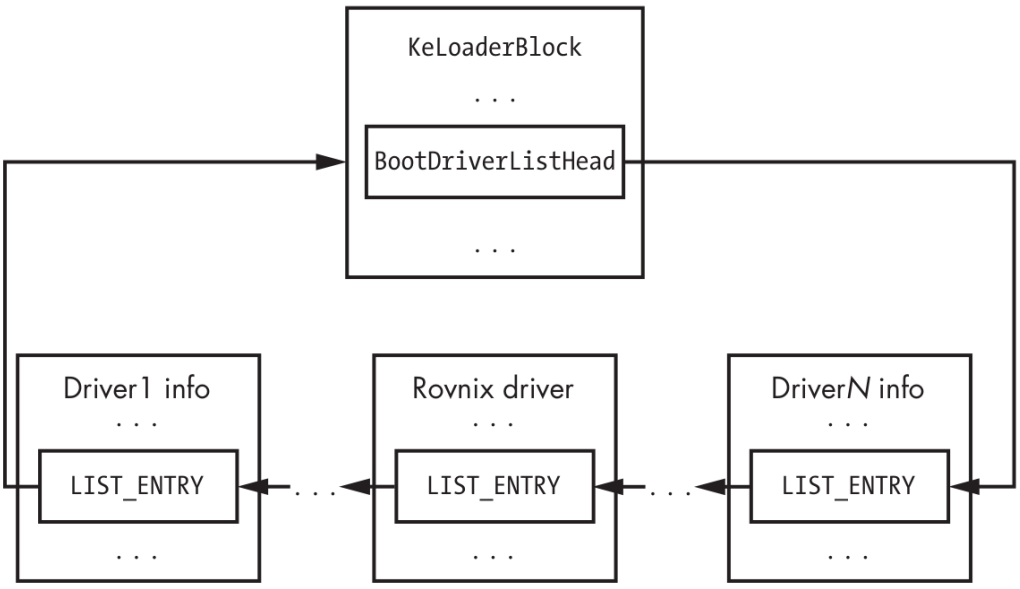


Рисунок 11-13. Вредоносный драйвер Rovnix, вставленный в `BootDriverList`

Функциональность драйвера режима ядра

Основная функция вредоносного драйвера - внедрять полезную нагрузку, которая хранится в двоичном файле драйвера и, как обсуждалось выше, сжата при помощи aPlib, в целевые процессы системы, главным образом в `explorer.exe` и браузеры.

Внедрение модуля полезной нагрузки

Модуль полезной нагрузки содержит в своей сигнатуре код `JFA`, поэтому для его извлечения Rovnix ищет сигнатуру `JFA` в свободном пространстве между таблицей разделов драйвера и его первым разделом. Эта сигнатура обозначает начало блока конфигурационных данных, пример которого приведён в листинге 11-6.

```
typedef struct _PAYLOAD_CONFIGURATION_BLOCK
{

   DWORD Signature;              // "JFA\0"
   DWORD PayloadRva;             // RVA начала полезной нагрузки
   DWORD PayloadSize;            // Размер полезной нагрузки
   DWORD NumberOfProcessNames;   // Число строк с завершающим NULL в ProcessNames
   char ProcessNames[0];         // Массив завершающихся NULL имён процессов для внедрения
    нагрузки
} PAYLOAD_CONFIGURATION_BLOCK, *PPAYLOAD_CONFIGURATION_BLOCK;
```

Листинг 11-6. Структура `PAYLOAD_CONFIGURATION_BLOCK`*, описывающая конфигурацию полезной нагрузки*

Поля `PayloadRva` и `PayloadSize` задают координаты сжатого образа полезной нагрузки в драйвере режима ядра. Массив `ProcessNames` содержит имена процессов, в которые следует внедрить полезную нагрузку. Число элементов массива задаётся полем `NumberOfProcessNames`. На рисунке 11-14 показан пример такого блока данных из вредоносного драйвера режима ядра, обнаруженного в реальных атаках. Как видно, полезная нагрузка должна внедряться в `explorer.exe` и браузеры `iexplore.exe`, `firefox.exe` и `chrome.exe`.

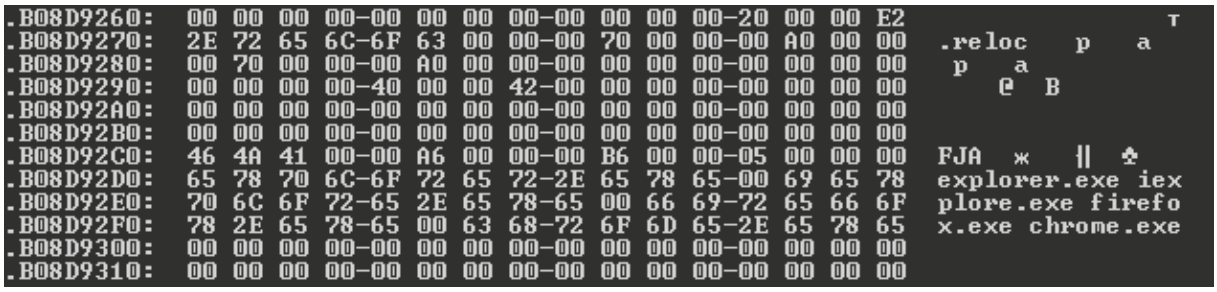


Рисунок 11-14. Блок конфигурации полезной нагрузки

Сначала Rovnix распаковывает полезную нагрузку в буфер памяти. Затем он применяет традиционный приём, часто используемый руткитами для внедрения полезной нагрузки и состоящий из следующих этапов:

1. Регистрирует `CreateProcessNotifyRoutine` и `LoadImageNotifyRoutine` посредством стандартного документированного API режима ядра. Это позволяет Rovnix получать управление при каждом создании нового процесса или загрузке нового образа в адресное пространство целевого процесса.
2. Отслеживает новые процессы в системе и ищет целевой процесс, определяемый по имени образа.
3. Сразу после загрузки целевого процесса отображает полезную нагрузку в его адресное пространство и ставит в очередь асинхронный вызов процедуры (APC), который передаёт управление полезной нагрузке.

Рассмотрим этот приём подробнее. Процедура `CreateProcessNotify` позволяет Rovnix установить специальный обработчик, срабатывающий при каждом создании нового процесса в системе. Так вредоносная программа может обнаружить запуск целевого процесса. Однако вредоносный обработчик создания процесса срабатывает в самом начале создания процесса, когда все необходимые системные структуры уже инициализированы, но исполняемый файл целевого процесса ещё не загружен в его адресное пространство. Поэтому на данном этапе вредоносная программа не может внедрить полезную нагрузку.

Вторая процедура, `LoadImageNotifyRoutine`, позволяет Rovnix установить обработчик, срабатывающий при каждой загрузке или выгрузке исполняемого модуля - файла `.exe`, библиотеки DLL и так далее - в системе. Этот обработчик отслеживает основной исполняемый образ и уведомляет Rovnix, когда образ загружен в адресное пространство целевого процесса. В этот момент Rovnix внедряет полезную нагрузку и выполняет её, создавая APC.

Скрытные механизмы самозащиты

Драйвер режима ядра реализует те же защитные механизмы, что и буткит TDL4: он перехватывает обработчик `IRP_MJ_INTERNAL_CONTROL` минипортового `DRIVER_OBJECT` жёсткого диска. Этот обработчик является самым низкоуровневым аппаратно-независимым интерфейсом с доступом к данным на жёстком диске и предоставляет вредоносной программе надёжный способ управления данными, читаемыми с жёсткого диска и записываемыми на него.

Так Rovnix может перехватывать все запросы чтения и записи и защищать критически важные области от чтения или перезаписи. В частности, он защищает:

• заражённый код IPL;
• сохранённый драйвер режима ядра;
• раздел скрытой файловой системы.

В листинге 11-7 приведён псевдокод процедуры-перехватчика IRP_MJ_INTERNAL_CONTROL, которая решает, заблокировать или разрешить операцию ввода-вывода в зависимости от того, какая часть жёсткого диска читается или записывается.

```
int __stdcall NewIrpMjInternalHandler(PDEVICE_OBJECT DeviceObject, PIRP Irp)
{
  UCHAR ScsiCommand;
  NTSTATUS Status;
  unsigned __int64 Lba;
  PVOID pTransferBuffer;
  ❶ if ( DeviceObject != g_DiskDevObj )
    return OriginalIrpMjInternalHandler(DeviceObject, Irp);
  ❷ ScsiCommand = GetSrbParameters(_Irp, &Lba, &DeviceObject, &pTransferBuffer,
                                                              Irp);
  if ( ScsiCommand == 0x2A || ScsiCommand == 0x3B )
  {
    // Команды записи SCSI
    ❸ if ( CheckSrbParams(Lba, DeviceObject)
    {
       Status = STATUS_ACCESS_DENIED;
       ❹ Irp->IoStatus.Status = STATUS_ACCESS_DENIED;
       IofCompleteRequest(Irp, 0);
    } else
    {
       return OriginalIrpMjInternalHandler(DeviceObject, Irp);
    }
  } else if ( ScsiCommand == 0x28 || ScsiCommand == 0x3C)
  {

      // Команды чтения SCSI
      if ( CheckSrbParams(Lba, DeviceObject)
      {
        ❺ Status = SetCompletionRoutine(DeviceObject, Irp, Lba,
                                        DeviceObject, pTransferBuffer, Irp);
      } else
      {
        return OriginalIrpMjInternalHandler(DeviceObject, Irp);
      }
  }

  if ( Status == STATUS_REQUEST_NOT_ACCEPTED )
    return OriginalIrpMjInternalHandler(DeviceObject, Irp);
  return Status;
}
```

Листинг 11-7. Псевдокод вредоносного обработчика IRP_MJ_INTERNAL_CONTROL

Сначала код проверяет, адресован ли запрос ввода-вывода объекту устройства жёсткого диска ❶. Если да, вредоносная программа проверяет, является ли операция чтением или записью и к какой области жёсткого диска выполняется обращение ❷. Процедура CheckSrbParams ❸ возвращает TRUE при обращении к областям, защищённым буткитом. Если кто-то пытается записать данные в защищённую буткитом область, код отклоняет операцию ввода-вывода и возвращает STATUS_ACCESS_DENIED ❹. Если кто-то пытается прочитать защищённую буткитом область, вредоносная программа устанавливает вредоносную процедуру завершения ❺ и передаёт запрос ввода-вывода объекту устройства жёсткого диска для выполнения операции чтения. После завершения чтения срабатывает вредоносная процедура завершения и стирает буфер с прочитанными данными, записывая в него нули. Так вредоносная программа защищает свои данные на жёстком диске.

Скрытая файловая система

Ещё одна важная возможность Rovnix - скрытый, то есть невидимый операционной системе, раздел файловой системы (ФС), используемый для тайного хранения конфигурационных данных и дополнительных модулей полезной нагрузки. Реализация скрытого хранилища - не новый приём буткитов: его применяли и другие руткиты, например TDL4 и Olmasco. Однако реализация Rovnix несколько отличается.

Для физического размещения скрытого раздела Rovnix занимает пространство в начале или в конце жёсткого диска, в зависимости от того, где достаточно свободного места. Если перед первым разделом имеется не менее `0x7D0` свободных секторов - 2000 в десятичной системе, почти 1 Мбайт, - Rovnix размещает скрытый раздел сразу после сектора MBR и растягивает его на все `0x7D0` свободных секторов. Если в начале жёсткого диска места недостаточно, Rovnix пытается разместить скрытый раздел в его конце. Для доступа к данным в скрытом разделе Rovnix использует исходный обработчик `IRP_MJ_INTERNAL_CONTROL`, перехваченный, как описано в предыдущем разделе.

Форматирование раздела как виртуальной системы FAT

Выделив пространство для скрытого раздела, Rovnix форматирует его как файловую систему Virtual File Allocation Table (VFAT) - модификацию файловой системы FAT, способную хранить файлы с длинными именами в Юникоде, до 256 байт. Исходная файловая система FAT ограничивает длину имени схемой 8 + 3: до восьми символов для имени файла и трёх символов для расширения.

Шифрование скрытой файловой системы

Для защиты данных в скрытой файловой системе Rovnix реализует прозрачное для раздела шифрование алгоритмом RC6 в режиме электронной кодовой книги (Electronic Code Book, ECB) с длиной ключа 128 бит. В режиме ECB шифруемые данные делятся на блоки равной длины, каждый из которых независимо от остальных шифруется одним и тем же ключом. Ключ хранится в последних 16 байтах самого первого сектора скрытого раздела, как показано на рисунке 11-15, и используется для шифрования и расшифрования всего раздела.

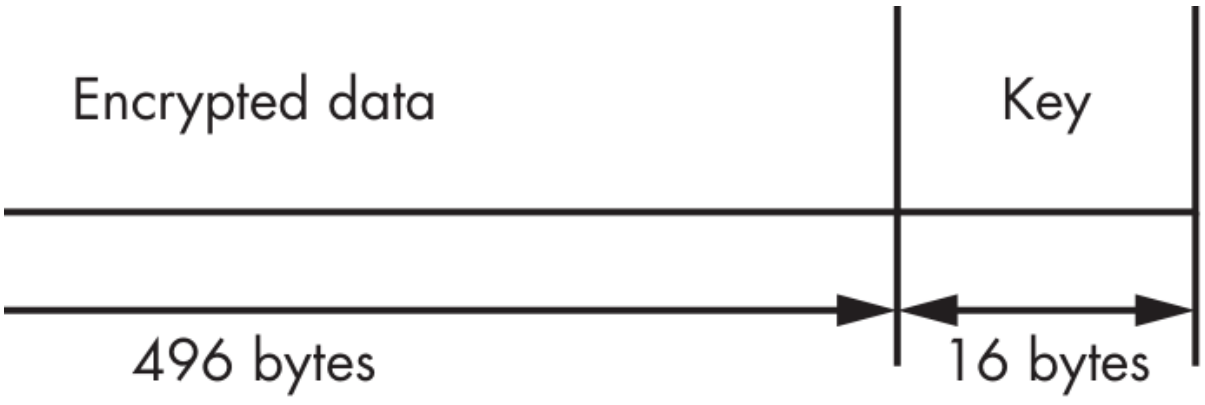


Рисунок 11-15. Расположение ключа шифрования в первом секторе скрытого раздела

RC6

Rivest cipher 6, или RC6, - блочный шифр с симметричным ключом, созданный Роном Ривестом, Мэттом Робшоу, Рэем Сидни и Ицюнь Лизой Инь для соответствия требованиям конкурса Advanced Encryption Standard (AES). RC6 имеет размер блока 128 бит и поддерживает ключи длиной 128, 192 и 256 бит.

Доступ к скрытой файловой системе

Чтобы сделать скрытую файловую систему доступной модулям полезной нагрузки, Rovnix создаёт специальный объект, называемый символической ссылкой. Грубо говоря, символическая ссылка - альтернативное имя объекта устройства скрытого хранилища, которым могут пользоваться модули в процессах пользовательского режима. Rovnix формирует строку `\DosDevices\<XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX>`, где каждый символ `X` - случайно сформированное шестнадцатеричное число от 0 до F, и использует её как имя символической ссылки на скрытое хранилище.

Одно из преимуществ скрытой файловой системы состоит в том, что к ней можно обращаться как к обычной файловой системе посредством стандартных функций Win32 API, предоставляемых операционной системой, таких как `CreateFile`, `CloseFile`, `ReadFile` и `WriteFile`. Например, чтобы создать файл `file_to_create` в корневом каталоге скрытой файловой системы, вредоносная полезная нагрузка вызывает `CreateFile`, передавая строку символической ссылки `\DosDevices\<%XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX>\file_to_create` как параметр имени файла. После такого вызова модулем полезной нагрузки операционная система перенаправляет запрос вредоносному драйверу режима ядра, отвечающему за обработку запросов к скрытой файловой системе.

На рисунке 11-16 показано, как вредоносный драйвер реализует функциональность драйвера файловой системы. Получив запрос ввода-вывода от полезной нагрузки, Rovnix направляет его через перехваченный обработчик жёсткого диска, чтобы выполнить операции чтения и записи для скрытой файловой системы на жёстком диске.

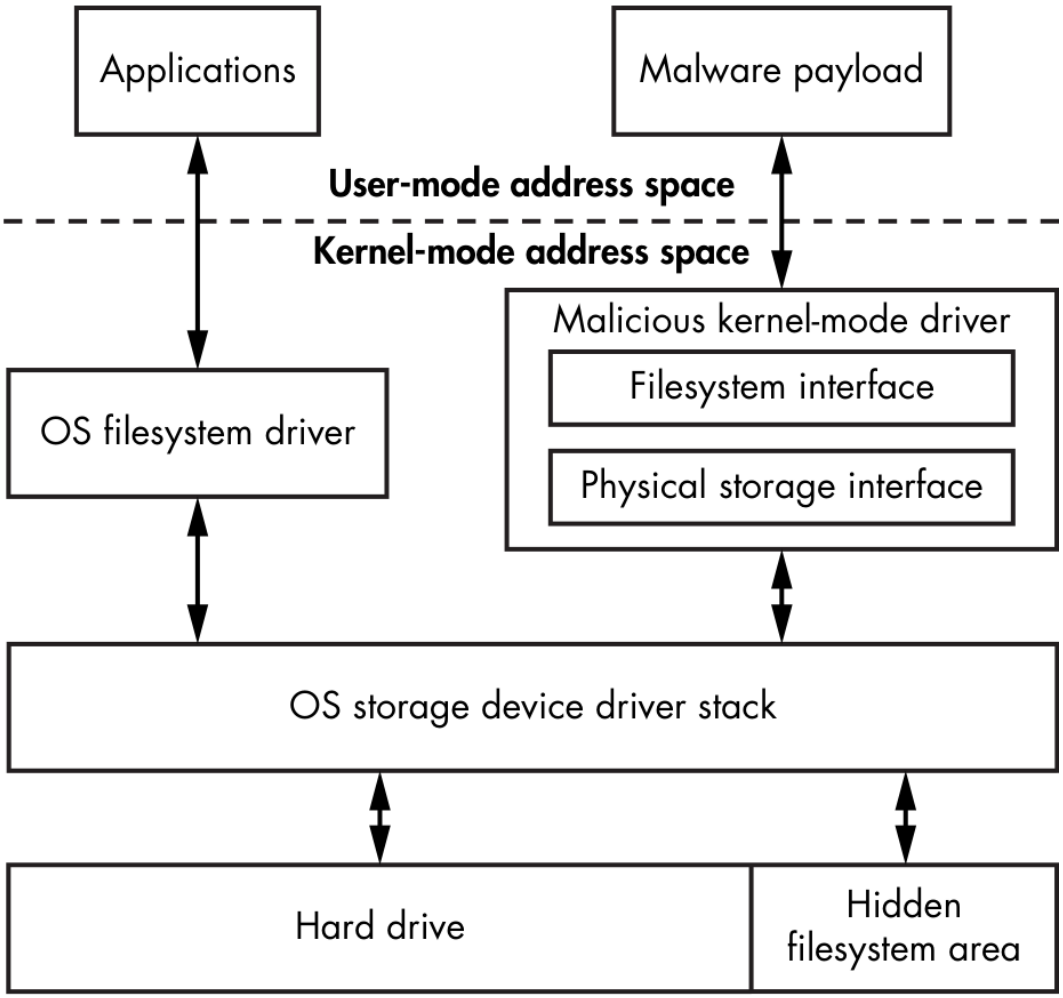


Рисунок 11-16. Архитектура файловой системы скрытого хранилища Rovnix

В этом сценарии операционная система и вредоносная скрытая файловая система сосуществуют на одном жёстком диске, но операционная система не знает об области диска, используемой для хранения скрытых данных.

Вредоносная скрытая файловая система потенциально может изменить легитимные данные, хранящиеся в файловой системе операционной системы, но вероятность этого невелика благодаря размещению скрытой файловой системы в начале или конце жёсткого диска.

Скрытый канал связи

В распоряжении Rovnix есть и другие приёмы сокрытия. Драйвер режима ядра Rovnix реализует стек протоколов TCP/IP для тайного взаимодействия с удалёнными командными серверами. Предоставляемые ОС сетевые интерфейсы часто перехватываются защитным ПО для наблюдения и управления проходящим через сеть трафиком. Вместо использования этих сетевых интерфейсов с риском обнаружения защитным ПО Rovnix загружает модули полезной нагрузки с командных серверов посредством собственной реализации сетевых протоколов, не зависящей от операционной системы.

Чтобы отправлять и принимать данные по этой сети, драйвер режима ядра Rovnix реализует полный сетевой стек, включающий следующие интерфейсы:

• интерфейс минипорта Microsoft Network Driver Interface Specification (NDIS) для отправки пакетов данных через физический сетевой интерфейс Ethernet;
• интерфейс транспортных драйверов для сетевых протоколов TCP/IP;
• интерфейс сокетов;
• протокол HTTP для взаимодействия с удалёнными командными серверами.

Как показано на рисунке 11-17, уровень минипорта NDIS отвечает за взаимодействие с сетевой платой при отправке и получении сетевых пакетов. Интерфейс транспортных драйверов предоставляет интерфейс TCP/IP вышележащему интерфейсу сокетов, который, в свою очередь, используется протоколом HTTP Rovnix для передачи данных.

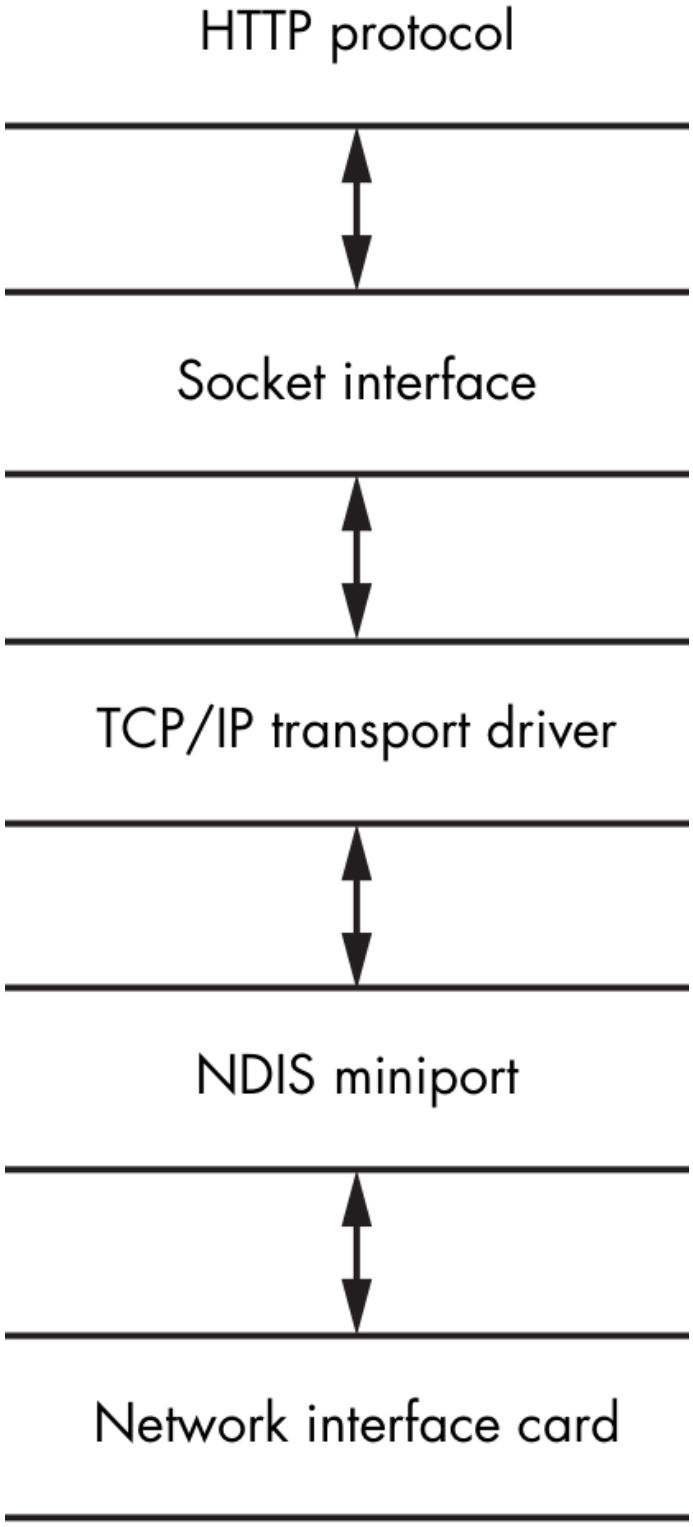


Рисунок 11-17. Архитектура пользовательской реализации сетевого стека Rovnix

Создатели Rovnix не разрабатывали эту систему скрытого сетевого взаимодействия с нуля: такая реализация требует тысяч строк кода и потому подвержена ошибкам. Вместо этого они основали её на лёгкой библиотеке TCP/IP с открытым исходным кодом под названием lwIP.

Библиотека lwIP - небольшая независимая реализация набора протоколов TCP/IP, ориентированная на сокращение потребления ресурсов при сохранении полноценного стека TCP/IP. Согласно её сайту, lwIP занимает десятки килобайт оперативной памяти и около 40 Кбайт кода, что идеально подходит буткиту.

Такие возможности, как скрытый канал связи, позволяют Rovnix обходить локальное защитное ПО, контролирующее сеть. Поскольку Rovnix содержит собственный стек сетевых протоколов, сетевое защитное ПО не знает о его обмене данными по сети и потому не способно его отслеживать. От самого верхнего уровня протоколов до самого нижнего уровня драйвера минипорта NDIS Rovnix использует только собственные сетевые компоненты, что делает его очень скрытным буткитом.

Практический пример: связь с Carberp

Один из реальных примеров использования Rovnix в атаках - вредоносная программа, троян Carberp, разработанный самой заметной киберпреступной группой в России. Carberp применялся для закрепления банковского трояна в системе жертвы.[2] Рассмотрим несколько особенностей Carberp и то, как он был создан на основе буткита Rovnix.

> **Вредоносные программы, связанные с Carberp**
>
> По оценкам, группа, разработавшая Carberp, получала в среднем несколько миллионов долларов США в неделю и вкладывала значительные средства в другие вредоносные технологии, например дроппер Hodprot,[3] причастный к установке Carberp, RDPdoor и Sheldor.[4] RDPdoor был особенно опасен: он устанавливал Carberp, чтобы открыть лазейку в заражённой системе и вручную выполнять мошеннические банковские транзакции.

Разработка Carberp

В ноябре 2011 года мы заметили, что один из командных серверов, созданных стоявшей за Carberp киберпреступной группой, начал распространять дроппер с буткитом на основе платформы Rovnix. Мы стали отслеживать троян Carberp и обнаружили, что в тот период его распространение было очень ограниченным.

Два обстоятельства в нашем анализе указывали, что бот работал в тестовом режиме и потому активно разрабатывался. Первым признаком было

изобилие отладочных и трассировочных сведений, относившихся к установке бота и поведению двоичного файла. Вторым признаком, обнаруженным после получения доступа к журналам командного сервера бота, было то, что командному серверу отправлялись огромные объёмы сведений о сбоях установки. На рисунке 11-18 показан пример сведений, которые передавал Carberp.

Total bots: 2831

Sort
Status
Step
Alias

Other
Del

ID	step	info	status	data
TEST_BK_KIT_EXPLORER0D9493DFECAE8C4B0	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_KIT_EXPLORER08D7BD1230A905D00	6	BkInstall	FALSE	0000-00-00 00:00:00
123213oob	1	infa	false	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV0F1B889AC4F21B5CA	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV0049C4497DE79EC77	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV082A52B2218EEED1A	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV06F0743BC19E94740	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV0DA631E2FA5B562AF	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV079943F8A64F9587B	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV09A01A1B010A8035A	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_MY_DRV07AA547C0940C1901	3	BkInstall0 GetLastError = 0	FALSE	0000-00-00 00:00:00
TEST_BK_EX_ORIG_DRV0B61FDB428F96A87B	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_ORIG_DRV0AE10F7A3602E42CB	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_ORIG_DRV06627C6A2AB3A2480	1	IsUserAdmin	FALSE	0000-00-00 00:00:00
TEST_BK_EX_ORIG_DRV0623F20AD27008003	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_ORIG_DRV03E797730D59441E7	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_ORIG_DRV0F7988F6217265D14	1	probapera	false	0000-00-00 00:00:00
TEST_BK_EX_ORIG_DRV0F7988F6317265D14	1	probapera	false	0000-00-00 00:00:00
TEST_TEST_TEST0123324234243	1	infa	false	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV01E6A389EE0D306DA	2	SetSystemPrivileges	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV08C893A82AB121144	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV074B2240F14F7F098	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0018A1BBAC95DCF46	2	SetSystemPrivileges	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0143930074B642759	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0598877EB08A14360	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0D8781E848009A04A	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV05910FAB2AB121144	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV09FC9B32DCEBACF5A	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV039034BD2E81688D0	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0AC2F4C7B405B2000	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0E75B71B1CF9C074E	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0804FAAA06CB8B686	6	BkInstall	FALSE	0000-00-00 00:00:00
TEST_BK_EX_CHANGE_DRV0AC37DCBF566138A1	6	BkInstall	FALSE	0000-00-00 00:00:00
NEW_BK_TEST012B7B297A8FC6244	2	SetSystemPrivileges	FALSE	0000-00-00 00:00:00
NEW_BK_TEST0B6424B774E7188FC	6	BkInstall	FALSE	0000-00-00 00:00:00
NEW_BK_TEST0A29E1011ACCF989B	6	BkInstall	FALSE	0000-00-00 00:00:00
NEW_BK_TEST099E961A9D26824C0	6	BkInstall	FALSE	0000-00-00 00:00:00
NEW_BK_TEST0084B77CA30C0481F	3	BkInstall0 GetLastError = 0	FALSE	0000-00-00 00:00:00
NEW_BK_TEST_CHECKED0809EB7F457A58CC6	6	BkInstall	FALSE	0000-00-00 00:00:00
NEW_BK_TEST_CHECKED089583D04428F269B	6	BkInstall	FALSE	0000-00-00 00:00:00
NEW_BK_TEST_CHECKED0DC3B31D927AC1529	6	BkInstall	FALSE	0000-00-00 00:00:00

Рисунок 11-18. Пример журналов дроппера Rovnix

Столбец `ID` задаёт уникальный идентификатор экземпляра Rovnix, а столбец `status` содержит сведения о том, удалось ли скомпрометировать систему жертвы. Алгоритм заражения был разделён на несколько этапов, и сведения отправлялись командному серверу непосредственно после каждого из них. Столбец `step` указывает выполняемый этап, а столбец `info` содержит описание любой ошибки, возникшей во время установки. По столбцам `step` и `info` операторы ботнета могли определить, на каком этапе и по какой причине не удалось выполнить заражение.

Использованная Carberp версия Rovnix содержала множество отладочных строк и отправляла командному серверу большое количество подробных сообщений. На рисунке 11-19 показаны примеры строк, которые она могла отправлять. Эти сведения оказались чрезвычайно полезными при анализе угрозы и изучении её функциональности. Оставленные в двоичном файле отладочные сведения раскрывали имена реализованных в нём процедур и их назначение. Они документировали логику кода. Благодаря этим данным мы смогли легче восстановить контекст вредоносного кода.

Рисунок 11-19. Отладочные строки, оставленные разработчиками в дроппере Rovnix

Усовершенствования дроппера

Платформа Rovnix, использованная в Carberp, была практически такой же, как буткит, описанный в начале главы; единственное существенное изменение появилось в дроппере. В разделе «Заражение системы» на странице 150 мы упоминали, что Rovnix пытается повысить свои права посредством Win32 API `ShellExecuteEx`, чтобы получить права администратора на компьютере жертвы. В версии Rovnix из Carberp дроппер эксплуатировал для повышения прав следующие уязвимости системы:

• **MS10-073 в модуле `win32k.sys`.** Эта уязвимость первоначально использовалась червём Stuxnet и эксплуатирует неправильную обработку специально сформированного файла раскладки клавиатуры.
• **MS10-092 в планировщике заданий Windows.** Эта уязвимость также впервые была обнаружена в Stuxnet и эксплуатирует механизм проверки целостности в планировщике Windows.
• **MS11-011 в модуле `win32k.sys`.** Эта уязвимость приводит к переполнению стекового буфера в процедуре `win32k.sys!RtlQueryRegistryValues`.
• **Уязвимость .NET Runtime Optimization.** Это уязвимость службы Microsoft .NET Runtime Optimization Service, приводящая к выполнению вредоносного кода с правами `SYSTEM`.

Ещё одна интересная особенность установщика Carberp состояла в том, что непосредственно перед установкой трояна или буткита в систему он удалял различные перехваты из списка системных процедур, показанного в листинге 11-8. Эти процедуры часто становятся целями перехвата защитного ПО, такого как песочницы и системы предотвращения и защиты от вторжений на узел. Снимая перехваты с этих функций, вредоносная программа повышала способность избегать обнаружения.

```
ntdll!ZwSetContextThread
ntdll!ZwGetContextThread
ntdll!ZwUnmapViewOfSection
ntdll!ZwMapViewOfSection
ntdll!ZwAllocateVirtualMemory
ntdll!ZwWriteVirtualMemory
ntdll!ZwProtectVirtualMemory
ntdll!ZwCreateThread
ntdll!ZwOpenProcess
ntdll!ZwQueueApcThread
ntdll!ZwTerminateProcess
ntdll!ZwTerminateThread
ntdll!ZwResumeThread
ntdll!ZwQueryDirectoryFile
ntdll!ZwCreateProcess
ntdll!ZwCreateProcessEx
ntdll!ZwCreateFile
ntdll!ZwDeviceIoControlFile
ntdll!ZwClose
ntdll!ZwSetInformationProcess
kernel32!CreateRemoteThread
kernel32!WriteProcessMemory
kernel32!VirtualProtectEx
kernel32!VirtualAllocEx
kernel32!SetThreadContext
kernel32!CreateProcessInternalA
kernel32!CreateProcessInternalW
kernel32!CreateFileA
kernel32!CreateFileW
kernel32!CopyFileA
kernel32!CopyFileW
kernel32!CopyFileExW
ws2_32!connect
ws2_32!send
ws2_32!recv
ws2_32!gethostbyname
```

Листинг 11-8. Список процедур, с которых дроппер Rovnix снимает перехваты

Разделы буткита и драйвера режима ядра в модификации Rovnix для Carberp остались такими же, как в исходной версии буткита. После успешной установки в систему вредоносный код IPL загружал драйвер режима ядра, а драйвер внедрял полезную нагрузку трояна Carberp в системные процессы.

Утечка исходного кода

В июне 2013 года исходный код Carberp и Rovnix стал общедоступным. Полный архив можно было скачать; он содержал весь исходный код, необходимый злоумышленникам для сборки собственного буткита Rovnix. Несмотря на это, в реальных атаках мы наблюдали не так много пользовательских модификаций Rovnix и Carberp, как можно было бы ожидать. Мы предполагаем, что причина заключается в сложности этой технологии буткитов.

Заключение

В этой главе был представлен подробный технический анализ Rovnix в контексте непрерывной гонки вооружений между буткитами и индустрией безопасности. Когда защитное ПО догнало современные буткиты, заражавшие MBR, Rovnix предложил другой вектор заражения - IPL, запустив очередной виток развития антивирусных технологий. Благодаря заражению IPL, реализации скрытого хранилища и скрытых каналов сетевого взаимодействия Rovnix является одним из самых сложных буткитов, встречавшихся в реальных атаках. Эти возможности делают его опасным оружием в руках киберпреступников, что подтверждает пример Carberp.

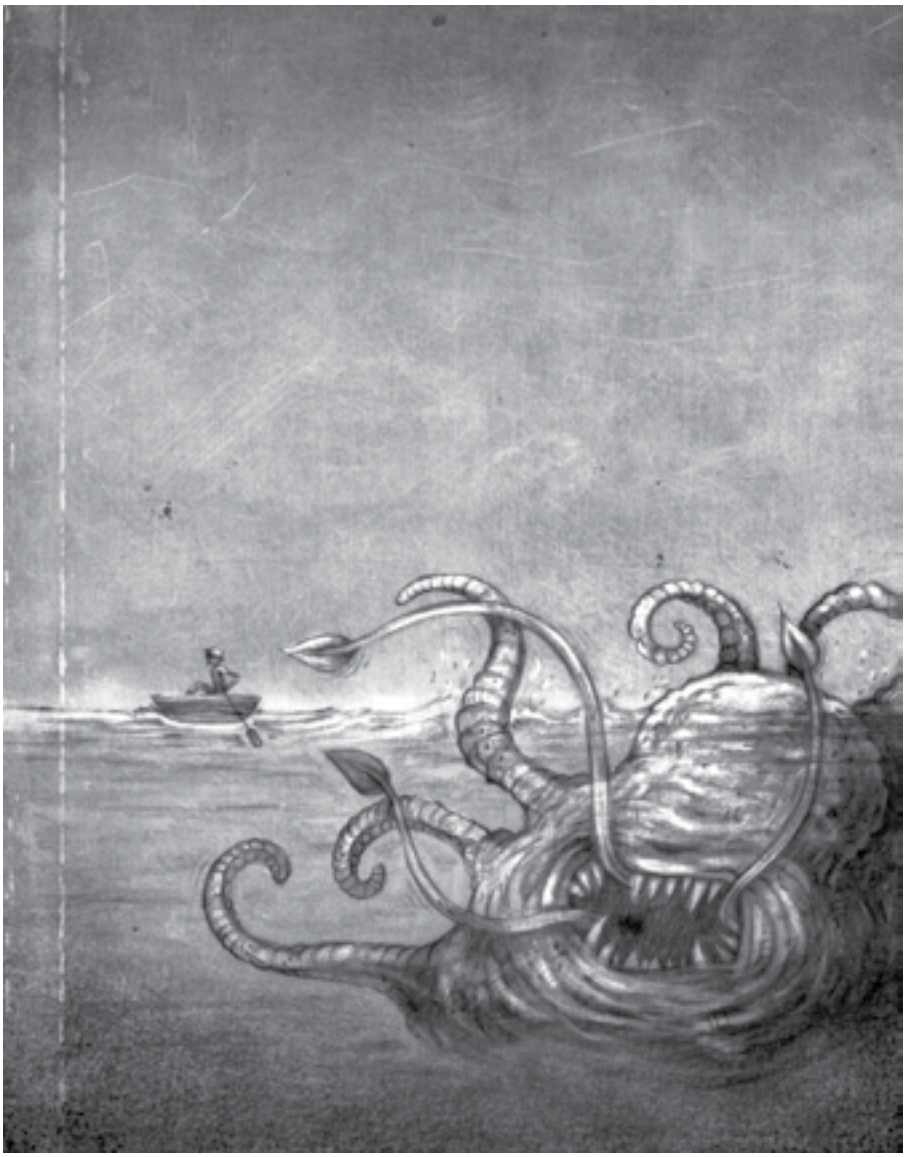
В этой главе мы уделили особое внимание разбору кода IPL Rovnix при помощи VMware и IDA Pro, продемонстрировав практическое применение этих инструментов в контексте анализа буткитов. Все необходимые данные для повторения описанных действий или собственного углублённого исследования кода IPL Rovnix можно скачать по адресу nostarch.com.

На этой странице исходного издания нет основного текста.

Сноски

[1] [назад] Отладочные регистры `dr4` и `dr5` зарезервированы, когда включены расширения отладки, то есть установлен флаг DE в управляющем регистре `cr4`; попытки обратиться к `dr4` и `dr5` вызывают исключения недопустимого кода операции (#UD). Если расширения отладки не включены, то есть флаг DE сброшен, эти регистры являются псевдонимами отладочных регистров `dr6` и `dr7`.
[2] [назад] welivesecurity.com; welivesecurity.com.
[3] [назад] welivesecurity.com.
[4] [назад] welivesecurity.com.

Глава 12. Gapz: усовершенствованное заражение VBR



В этой главе рассматривается один из самых скрытных буткитов, когда-либо встречавшихся в реальных атаках, - буткит Win32/Gapz. Мы изучим его технические характеристики и функциональность, начав с компонентов дроппера и буткита и затем перейдя к полезной нагрузке пользовательского режима.

По нашему опыту Gapz - самый сложный буткит из всех, которые когда-либо подвергались анализу. Каждая особенность его проекта и реализации - тщательно разработанный дроппер, усовершенствованное заражение буткитом и расширенная функциональность руткита - помогает Gapz заражать компьютеры жертв, закрепляться на них и долгое время оставаться незамеченным.

Gapz устанавливается в систему жертвы дроппером, который эксплуатирует несколько уязвимостей локального повышения прав и реализует необычный приём обхода систем предотвращения вторжений на узел (Host Intrusion Prevention Systems, HIPS).

Успешно проникнув в систему жертвы, дроппер устанавливает буткит, занимающий очень мало места и трудно обнаружимый в заражённой системе. Буткит загружает в режим ядра вредоносный код, реализующий функциональность руткита Gapz.

Функциональность руткита очень богата и включает пользовательский сетевой стек TCP/IP, усовершенствованный механизм перехвата, криптографическую библиотеку и механизм внедрения полезной нагрузки.

В этой главе мы подробно разберём каждую из этих мощных возможностей.

> **Почему он называется Gapz?**
>
> Этот буткит получил имя от строки `'GAPZ'`, используемой во всех двоичных файлах и шелл-коде как тег при выделении памяти. Например, приведённый ниже фрагмент кода режима ядра выделяет память, выполняя процедуру `ExAllocatePoolWithTag` с третьим параметром `'ZPAG'` ❶, то есть `'GAPZ'` в обратном порядке:

```
int _stdcall alloc_mem(STRUCT_IPL_THREAD_2 *a1, int pBuffer, unsigned int
Size, int Pool)
{
   v7 = -1;
    for ( i = -30000000; ; (a1->KeDelagExecutionThread)(0, 0, &i) )
    {
        v4 = (a1->ExAllocatePoolWithTag)(Pool, Size,
❶'ZPAG');
        if ( v4 )
            break;
    }
    memset(v4, 0, Size);
    result = pBuffer;
    *pBuffer = v4;
    return result;
}
```

Дроппер Gapz

Gapz устанавливается в целевую систему сложным дроппером. Существует несколько вариантов дроппера Gapz, и все они содержат схожую полезную нагрузку, которая будет рассмотрена далее в разделе «Функциональность руткита Gapz» на странице 191. Различия между дропперами заключаются в приёме буткита и числе эксплуатируемых каждым из них уязвимостей локального повышения прав (Local Privilege Escalation, LPE).

Первым экземпляром Gapz, обнаруженным в реальных атаках в апреле 2012 года, был Win32/Gapz.C.[1] Этот вариант дроппера для закрепления на компьютере жертвы применял буткит на основе MBR - тот же приём, который рассматривался в главе 7 на примере буткита TDL4. Win32/Gapz.C был примечателен множеством подробных строк для отладки и тестирования, а также

очень ограниченным распространением на раннем этапе. Это позволяет предположить, что первые версии Gapz не предназначались для массового распространения, а были тестовыми версиями для отладки функциональности вредоносной программы.

Второй вариант, Win32/Gapz.B, вообще не устанавливал буткит в целевую систему. Для закрепления в системе жертвы Gapz просто устанавливал вредоносный драйвер режима ядра. Однако такой подход не работал бы на 64-разрядных платформах Microsoft Windows из-за отсутствия у драйвера режима ядра действительной цифровой подписи, поэтому эта модификация ограничивалась только 32-разрядными операционными системами Microsoft Windows.

Последняя известная и наиболее интересная итерация дроппера, Win32/Gapz.A, является версией, на которой мы сосредоточимся в этой главе. Эта версия содержала VBR-буткит. В остальной части главы под именем «Gapz» будет подразумеваться Win32/Gapz.A.

В таблице 12-1 перечислены различные версии дроппера.

Таблица 12-1. Версии дроппера Win32/Gapz

Имя обнаружения	Дата компиляции	Эксплойты LPE	Приём буткита
Win32/Gapz.A	09/11/2012; 10/30/2012	CVE-2011-3402; CVE-2010-4398; повышение прав COM	VBR
Win32/Gapz.B	11/06/2012	CVE-2011-3402; повышение прав COM	Без буткита
Win32/Gapz.C	04/19/2012	CVE-2010-4398; CVE-2011-2005; повышение прав COM	MBR

В столбце «Имя обнаружения» указаны обозначения вариантов Gapz, принятые антивирусной индустрией. Значения в столбце «Дата компиляции» взяты из заголовка PE дропперов Gapz и считаются достоверными временными метками. Столбец «Приём буткита» показывает, какой вид буткита применяет дроппер.

Наконец, в столбце «Эксплойты LPE» перечислены уязвимости локального повышения прав, которые эксплуатируют дропперы Gapz, чтобы получить права администратора в системах жертв. Уязвимость повышения прав COM используется для обхода защитной функции контроля учётных записей (User Account Control, UAC), чтобы внедрить код в системный процесс из белого списка UAC. Уязвимость CVE-2011-3402 относится к функциональности разбора шрифтов TrueType, реализованной в модуле `win32k.sys`. Уязвимость CVE-2010-4398 обусловлена переполнением стекового буфера в процедуре `RtlQueryRegistryValues`, также расположенной в модуле `win32k.sys`. Уязвимость CVE-2011-2005, находящаяся в модуле `afd.sys`, вспомогательном функциональном драйвере, позволяет злоумышленникам перезаписывать данные в адресном пространстве режима ядра.

Все перечисленные в таблице 12-1 варианты дроппера Gapz содержат одну и ту же полезную нагрузку.

Алгоритм дроппера

Прежде чем подробно изучать дроппер Gapz, вспомним, что ему требуется для скрытной и успешной установки Gapz в систему.

Во-первых, дропперу нужны административные права, чтобы получить доступ к жёсткому диску и изменить данные MBR, VBR и IPL. Если у учётной записи пользователя дроппера нет прав администратора, он должен повысить свои права, эксплуатируя уязвимости LPE в системе.

Во-вторых, ему необходимо обойти защитное ПО, например антивирусные программы, персональные брандмауэры и системы предотвращения вторжений на узел. Чтобы оставаться незамеченным, Gapz использует усовершенствованные средства и методы, включая запутывание, противодействие отладке и эмуляции. Помимо этих методов дроппер Gapz применяет уникальный и довольно интересный приём обхода HIPS, рассматриваемый далее в этой главе.

Системы предотвращения вторжений на узел

Как следует из названия, система предотвращения вторжений на узел, или HIPS, - пакет защитного ПО, предназначенный для предотвращения доступа злоумышленника к целевой системе. Он сочетает различные методы, включая, помимо прочего, применение сигнатур и эвристик и наблюдение за подозрительной активностью отдельного узла, например созданием новых процессов в системе, выделением в другом процессе буфера памяти с исполняемыми страницами и новыми сетевыми соединениями. В отличие от компьютерного антивирусного ПО, анализирующего только исполняемые файлы, HIPS анализирует события и ищет отклонения от нормального состояния системы. Если вредоносной программе удастся обойти компьютерное антивирусное ПО и запуститься на компьютере, HIPS всё ещё может обнаружить и заблокировать нарушителя, выявив изменения во взаимодействии различных событий.

С учётом этих препятствий дроппер Gapz выполняет для успешного заражения системы следующие действия:

1. Внедряет себя в `explorer.exe`, чтобы обойти HIPS, как рассматривается в разделе «Обход HIPS» на странице 181.
2. Эксплуатирует уязвимость LPE в целевой системе, чтобы повысить права пользователя.
3. Устанавливает буткит в систему.

Анализ дроппера

Если загрузить распакованный дроппер в дизассемблер IDA Pro, его таблица адресов экспорта будет выглядеть примерно как на рисунке 12-1. Таблица адресов экспорта показывает все символы, экспортируемые из двоичного файла, и наглядно обобщает этапы алгоритма выполнения дроппера.

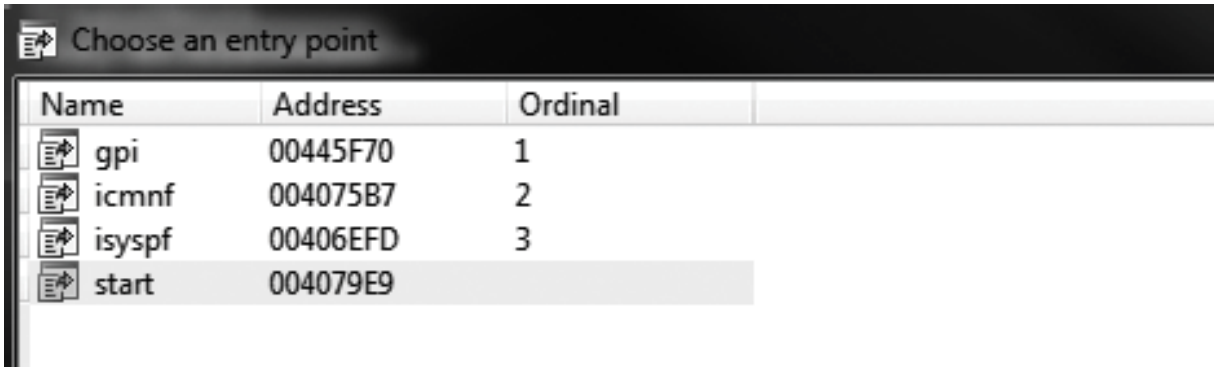


Рисунок 12-1. Таблица адресов экспорта дроппера Gapz

Двоичный файл экспортирует три процедуры: одну основную точку входа и две процедуры со случайно сформированными именами. У каждой процедуры своё назначение:

• `start` внедряет дроппер в адресное пространство `explorer.exe`;
• `icmnf` эксплуатирует уязвимости LPE в системе, чтобы повысить права;
• `isyspf` заражает компьютер жертвы.

На рисунке 12-1 также показан экспортируемый символ `gpi`. Он указывает на общую память в образе дроппера, которую перечисленные процедуры используют для внедрения дроппера в процесс `explorer.exe`.

На рисунке 12-2 изображены эти этапы. Основная точка входа не заражает систему буткитом Gapz. Вместо этого она выполняет процедуру `start`, внедряющую дроппер в `explorer.exe` для обхода обнаружения защитным ПО. После внедрения дроппер пытается получить права администратора, эксплуатируя уязвимости LPE в системе посредством процедуры `icmnf`. Получив необходимые права, дроппер выполняет процедуру `isyspf`, чтобы заразить жёсткий диск буткитом.

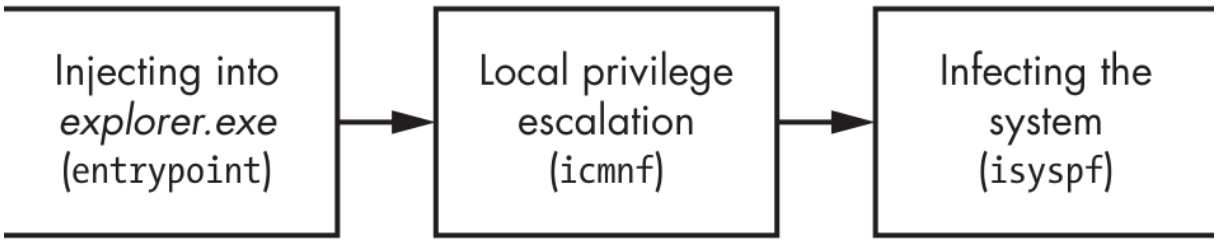


Рисунок 12-2. Рабочий процесс дроппера Gapz

Рассмотрим подробнее процесс внедрения дроппера и обхода HIPS.

Обход HIPS

У компьютерных вирусов существует множество методов маскировки под безопасное ПО, позволяющих не привлекать внимание защитных программ. Руткит TDL3, рассмотренный в главе 1, применяет другой интересный приём обхода HIPS, злоупотребляя системными API `AddPrintProvidor`/`AddPrintProvider`, чтобы оставаться незамеченным. Эти функции API используются для загрузки пользовательских модулей в доверенный системный процесс `spoolsvc.exe`, отвечающий за поддержку печати в системах Windows. Процедура `AddPrintProvidor` - именно с таким ошибочным написанием - представляет собой исполняемый модуль для установки локального поставщика печати в систему и часто исключается из списка объектов, отслеживаемых защитным ПО. TDL3 просто создаёт исполняемый файл с вредоносным кодом и загружает его в `spoolsvc.exe`, выполняя `AddPrintProvidor`. После выполнения процедуры вредоносный код работает внутри доверенного системного процесса, позволяя TDL3 атаковать, не опасаясь обнаружения.

Gapz также внедряет свой код в доверенный системный процесс для обхода HIPS, но использует сложный нестандартный метод, основная цель которого - внедрить в процесс `explorer` шелл-код, загружающий и выполняющий вредоносный образ. Дроппер выполняет следующие действия:

1. Открывает один из общих разделов из `\BaseNamedObjects`, отображённых в адресное пространство `explorer.exe` (см. листинг 12-1), и записывает в него шелл-код. Каталог `\BaseNamedObjects` в пространстве имён диспетчера объектов Windows содержит имена объектов мьютексов, событий, семафоров и разделов.
2. После записи шелл-кода ищет окно `Shell_TrayWnd`. Это окно соответствует панели задач Windows. Gapz нацеливается именно на него, потому что оно создаётся и управляется `explorer.exe` и почти наверняка присутствует в системе.
3. Вызывает функцию Win32 API `GetWindowLong`, чтобы получить адрес процедуры, связанной с обработчиком окна `Shell_TrayWnd`.
4. Вызывает функцию Win32 API `SetWindowLong`, чтобы изменить адрес процедуры, связанной с обработчиком окна `Shell_TrayWnd`.
5. Вызывает `SendNotifyMessage`, чтобы запустить выполнение шелл-кода в адресном пространстве `explorer.exe`.

Объекты разделов используются, чтобы сделать часть памяти определённого процесса общей с другими процессами; иными словами, они представляют область памяти, которая может совместно использоваться системными процессами. В листинге 12-1 показаны объекты разделов в `\BaseNamedObjects`, которые вредоносная программа ищет на этапе 1. Эти объекты соответствуют системным разделам, то есть создаются операционной системой и содержат системные данные. Gapz перебирает список объектов разделов и открывает их, проверяя наличие в системе. Если объект раздела существует, дроппер прекращает перебор и возвращает дескриптор соответствующего раздела.

```
char _stdcall OpenSection_(HANDLE *hSection, int pBase, int *pRegSize)
{
    sect_name = L"\\BaseNamedObjects\\ShimSharedMemory";
    v7 = L"\\BaseNamedObjects\\windows_shell_global_counters";
    v8 = L"\\BaseNamedObjects\\MSCTF.Shared.SFM.MIH";
    v9 = L"\\BaseNamedObjects\\MSCTF.Shared.SFM.AMF";
    v10 = L"\\BaseNamedObjectsUrlZonesSM_Administrator";
    i = 0;
    while ( OpenSection(hSection, (&sect_name)[i], pBase, pRegSize) < 0 )
    {
        if ( ++i >= 5 )
            return 0;
    }

    if ( VirtualQuery(*pBase, &Buffer, 0x1Cu) )
        *pRegSize = v7;
    return 1;
}
```

Листинг 12-1. Имена объектов, используемые в дроппере Gapz

Открыв существующий раздел, вредоносная программа приступает к внедрению своего кода в процесс `explorer.exe`, как показано в листинге 12-2.

```
char __cdecl InjectIntoExplorer()
{
  returnValue = 0;
  if ( OpenSectionObject(&hSection, &SectionBase, &SectSize) )  // открыть раздел SHIM
  {
❶ TargetBuffer = (SectionBase + SectSize - 0x150);            // найти свободное место
                                                               // в конце раздела
    memset(TargetBuffer, 0, 0x150u);
    qmemcpy(TargetBuffer->code, sub_408468, sizeof(TargetBuffer->code));

    hKernel32 = GetModuleHandleA("kernel32.dll");
❷ TargetBuffer->CloseHandle = GetExport(hKernel32, "CloseHandle", 0);
```

```
    TargetBuffer->MapViewOfFile = GetExport(hKernel32, "MapViewOfFile", 0);
    TargetBuffer->OpenFileMappingA = GetExport(hKernel32, "OpenFileMappingA", 0);
    TargetBuffer->CreateThread = GetExport(hKernel32, "CreateThread", 0);
    hUser32 = GetModuleHandleA("user32.dll");
    TargetBuffer->SetWindowLongA = GetExport(hUser32, "SetWindowLongA", 0);

❸ TargetBuffer_ = ConstructTargetBuffer(TargetBuffer);
    if ( TargetBuffer_ )
    {
      hWnd = FindWindowA("Shell_TrayWnd", 0);
    ❹ originalWinProc = GetWindowLongA(hWnd, 0);
      if ( hWnd && originalWinProc )
      {
        TargetBuffer->MappingName[10] = 0;
        TargetBuffer->Shell_TrayWnd = hWnd;
        TargetBuffer->Shell_TrayWnd_Long_0 = originalWinProc;

        TargetBuffer->icmnf = GetExport(CurrentImageAllocBase, "icmnf", 1);
        qmemcpy(&TargetBuffer->field07, &MappingSize, 0xCu);
        TargetBuffer->gpi = GetExport(CurrentImageAllocBase, "gpi", 1);
        BotId = InitBid();
        lstrcpynA(TargetBuffer->MappingName, BotId, 10);
        if ( CopyToFileMappingAndReloc(TargetBuffer->MappingName, CurrentImageAllocBase,
                                       CurrentImageSizeOfImage, &hObject) )
        {
          BotEvent = CreateBotEvent();
          if ( BotEvent )
          {
           ❺ SetWindowLongA(hWnd, 0, &TargetBuffer_->pKiUserApcDispatcher);
           ❻ SendNotifyMessageA(hWnd, 0xFu, 0, 0);

            if ( !WaitForSingleObject(BotEvent, 0xBB80u) )
              returnValue = 1;
            CloseHandle(BotEvent);
          }
          CloseHandle(hObject);
        }
      }
    }
    NtUnmapViewOfSection(-1, SectionBase);
    NtClose(hSection);
  }
  return returnValue;
}
```

Листинг 12-2. Внедрение дроппера Gapz в `explorer.exe`

Вредоносная программа использует 336 (`0x150`) байт ❶ пространства в конце раздела для записи шелл-кода. Чтобы шелл-код выполнялся правильно, вредоносная программа также предоставляет адреса нескольких процедур API, используемых при внедрении: `CloseHandle`, `MapViewOfFile`, `OpenFileMappingA`, `CreateThread` и `SetWindowLongA` ❷. Шелл-код воспользуется этими процедурами, чтобы загрузить дроппер Gapz в пространство памяти `explorer.exe`.

Gapz выполняет шелл-код посредством программирования, ориентированного на возврат (Return-Oriented Programming, ROP). ROP использует тот факт, что в архитектурах x86 и x64 инструкция `ret` может возвращать управление родительской процедуре после выполнения дочерней подпрограммы. Инструкция `ret` предполагает, что адрес возврата управления находится на вершине стека, поэтому извлекает адрес возврата из стека и передаёт ему управление. Получив управление стеком посредством инструкции `ret`, злоумышленник может выполнить произвольный код.

Gapz использует ROP для выполнения шелл-кода потому, что память, соответствующая объекту общего раздела, может не быть исполняемой, и попытка выполнить из неё инструкции приведёт к исключению. Чтобы преодолеть это ограничение, вредоносная программа применяет небольшую программу ROP, выполняемую перед шелл-кодом. Программа ROP выделяет исполняемую память внутри целевого процесса, копирует шелл-код в этот буфер и выполняет его оттуда.

Gapz находит гаджет для запуска шелл-кода в процедуре `ConstructTargetBuffer` ❸. В 32-разрядных системах Gapz использует системную процедуру `ntdll!KiUserApcDispatcher`, чтобы передать управление программе ROP.

Изменение процедуры Shell_TrayWnd

Записав шелл-код в объект раздела и найдя все необходимые гаджеты ROP, вредоносная программа переходит к следующему этапу - изменению оконной процедуры `Shell_TrayWnd`. Эта процедура отвечает за обработку всех событий и сообщений, возникающих и отправляемых окну. При каждом изменении размера или перемещении окна, нажатии кнопки и так далее система вызывает процедуру `Shell_TrayWnd`, чтобы уведомить и обновить окно. Система задаёт адрес оконной процедуры при создании окна.

Дроппер Gapz получает адрес исходной оконной процедуры, чтобы вернуться к ней после внедрения, выполняя процедуру `GetWindowLongA` ❹. Эта процедура используется для получения параметров окна и принимает два аргумента: дескриптор окна и индекс извлекаемого параметра. Как видно, Gapz вызывает процедуру со значением индекса 0, указывающим адрес исходной оконной процедуры `Shell_TrayWnd`. Вредоносная программа сохраняет это значение в буфере памяти, чтобы после внедрения восстановить исходный адрес.

Затем вредоносная программа выполняет процедуру `SetWindowLongA` ❺, изменяя адрес оконной процедуры `Shell_TrayWnd` на адрес системной процедуры `ntdll!KiUserApcDispatcher`. Перенаправляя выполнение на адрес внутри системного модуля, а не непосредственно на шелл-код, Gapz дополнительно защищается от обнаружения защитным ПО. Теперь шелл-код готов к выполнению.

Выполнение шелл-кода

Gapz запускает шелл-код посредством API `SendNotifyMessageA` ❻, отправляя сообщение окну `Shell_TrayWnd` и передавая управление оконной процедуре. Как объяснялось в предыдущем разделе, после изменения адреса оконной процедуры новый адрес указывает на процедуру `KiUserApcDispatcher`. В конечном итоге это приводит к передаче управления шелл-коду, отображённому в адресное пространство процесса `explorer.exe`, как показано в листинге 12-3.

```
int __stdcall ShellCode(int a1, STRUCT_86_INJECT *a2, int a3, int a4)
{
  if ( !BYTE2(a2->injected) )
  {
    BYTE2(a2->injected) = 1;
 ❶ hFileMapping = (a2->call_OpenFileMapping)(38, 0, &a2->field4);
    if ( hFileMapping )
    {
   ❷ ImageBase = (a2->call_MapViewOfFile)(hFileMapping, 38, 0, 0, 0);
      if ( ImageBase )
      {
        qmemcpy((ImageBase + a2->bytes_5), &a2->field0, 0xCu);
     ❸ (a2->call_CreateThread)(0, 0, ImageBase + a2->routineOffs, ImageBase, 0, 0);
      }
      (a2->call_CloseHandle)( hFileMapping );
    }
  }

❹ (a2->call_SetWindowLongA)(a2->hWnd, 0, a2->OriginalWindowProc);
  return 0;
}
```

Листинг 12-3. Отображение образа дроппера Gapz в адресное пространство `explorer.exe`

Здесь видно использование процедур API `OpenFileMapping`, `MapViewOfFile`, `CreateThread` и `CloseHandle`, адреса которых были заполнены ранее, в точке ❷ листинга 12-2. С помощью этих процедур шелл-код отображает представление файла, соответствующего дропперу, в адресное пространство `explorer.exe` (❶ и ❷). Затем он создаёт поток ❸ в процессе `explorer.exe`, чтобы выполнить отображённый образ, и восстанавливает исходное значение индекса, изменённое функцией WinAPI `SetWindowLongA` ❹. Вновь созданный поток выполняет следующую часть дроппера, повышающую его права. Получив достаточные права, дроппер пытается заразить систему, и на этом этапе в дело вступает функциональность буткита.

Влияние Power Loader

Описанный здесь приём внедрения не был изобретён разработчиками Gapz: ранее он появился в программном обеспечении для создания вредоносных программ Power Loader. Power Loader - специальный конструктор ботов для создания загрузчиков других семейств вредоносного ПО и ещё один пример специализации и модульности производства вредоносных программ. Впервые Power Loader был обнаружен в реальных атаках в сентябре 2012 года. С ноября 2012 года вредоносная программа Win32/Redyms использовала компоненты Power Loader в собственном дроппере. На момент написания пакет Power Loader, включавший один комплект конструктора с командной панелью, стоил на российском киберпреступном рынке около 500 долларов.

Заражение системы буткитом Gapz

Gapz использует два разных варианта метода заражения: один нацелен на MBR загрузочного жёсткого диска, другой - на VBR активного раздела. Однако функциональность буткита в обеих версиях почти одинакова. Версия MBR стремится закрепиться на компьютере жертвы, изменяя код MBR подобно буткиту TDL4. Версия VBR применяет для заражения системы жертвы более тонкие и скрытные методы, и, как уже упоминалось, именно на ней мы сосредоточимся.

Мы вкратце затрагивали приём буткита Gapz в главе 7, а теперь подробно рассмотрим реализацию. Используемый Gapz метод заражения - один из самых скрытных, когда-либо встречавшихся в реальных атаках: он изменяет лишь несколько байтов VBR, поэтому защитному ПО очень трудно его обнаружить.

Повторное рассмотрение блока параметров BIOS

Главная цель вредоносной программы - структура данных блока параметров BIOS (BIOS Parameter Block, BPB), расположенная в VBR; подробнее см. главу 5. Эта структура содержит сведения о томе файловой системы в разделе и играет важнейшую роль в процессе загрузки. Разметка BPB различается в разных файловых системах, таких как FAT и NTFS, но мы сосредоточимся на NTFS. Содержимое структуры BPB для NTFS показано в листинге 12-4, для удобства взятом из листинга 5-3.

```
typedef struct _BIOS_PARAMETER_BLOCK_NTFS {
   WORD SectorSize;
   BYTE SectorsPerCluster;
   WORD ReservedSectors;
   BYTE Reserved[5];
   BYTE MediaId;
   BYTE Reserved2[2];
   WORD SectorsPerTrack;
```

```
    WORD NumberOfHeads;
    ❶ DWORD HiddenSectors;
    BYTE Reserved3[8];
    QWORD NumberOfSectors;
    QWORD MFTStartingCluster;
    QWORD MFTMirrorStartingCluster;
    BYTE ClusterPerFileRecord;
    BYTE Reserved4[3];
    BYTE ClusterPerIndexBuffer;
    BYTE Reserved5[3];
    QWORD NTFSSerial;
    BYTE Reserved6[4];
} BIOS_PARAMETER_BLOCK_NTFS, *PBIOS_PARAMETER_BLOCK_NTFS;
```

Листинг 12-4. Разметка `BIOS_PARAMETER_BLOCK` *для NTFS*

Как вы, возможно, помните из главы 5, поле `HiddenSectors` ❶, расположенное со смещением 14 от начала структуры, определяет положение IPL на жёстком диске (см. рисунок 12-3). Код VBR использует `HiddenSectors`, чтобы найти IPL на диске и выполнить его.

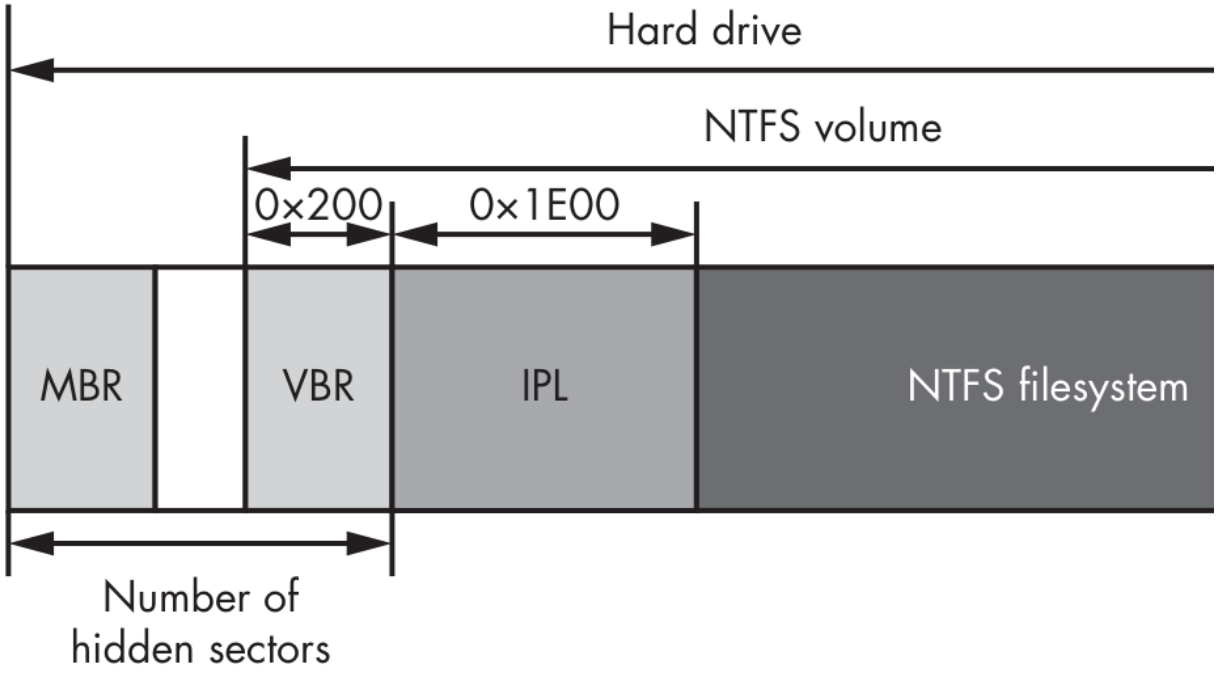


Рисунок 12-3. Расположение IPL на жёстком диске

Заражение VBR

Gapz перехватывает поток управления при загрузке системы, манипулируя значением поля `HiddenSectors` внутри BPB. При заражении компьютера Gapz записывает тело буткита перед самым первым разделом, если там достаточно места, либо после последнего раздела в противном случае. Затем он изменяет поле `HiddenSectors`, чтобы оно указывало не на легитимный код IPL, а на начало тела руткита на жёстком диске (см. рисунок 12-4). В результате при следующей загрузке код VBR загружает и выполняет код буткита Gapz из конца жёсткого диска.

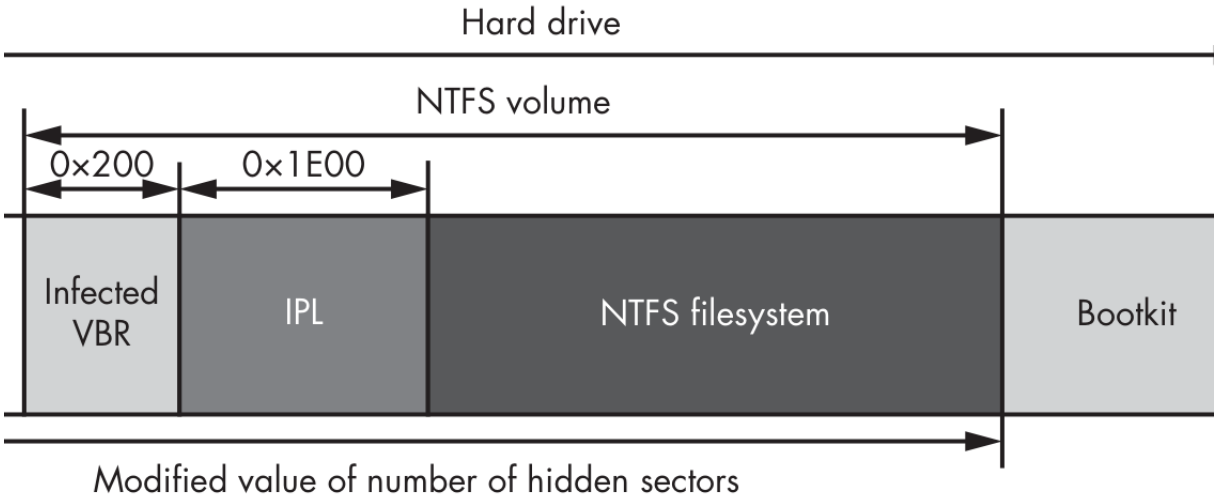


Рисунок 12-4. Разметка заражения буткитом Gapz

Особенно хитрым этот приём делает то, что он изменяет лишь 4 байта данных VBR, значительно меньше других буткитов. Например, TDL4 изменяет код MBR размером 446 байт; Olmasco меняет элемент таблицы разделов MBR размером 16 байт; а Rovnix изменяет код IPL, занимающий 15 секторов, или 7680 байт.

Gapz появился в 2012 году, когда индустрия безопасности уже догнала современные буткиты, а наблюдение за кодом MBR, VBR и IPL стало обычной практикой. Однако, изменяя поле `HiddenSectors` BPB, Gapz вывел методы заражения буткитов на новый уровень и снова оставил индустрию безопасности позади. До Gapz защитное ПО обычно не проверяло поля BPB на аномалии. Индустрии безопасности потребовалось некоторое время, чтобы разобраться в новом методе заражения и разработать решения.

Ещё одно отличие Gapz состоит в том, что содержимое поля `HiddenSectors` не фиксировано для структур BPB: оно может различаться от системы к системе. Значение `HiddenSectors` в значительной степени зависит от схемы разделов жёсткого диска. В общем случае защитное ПО не может определить заражение системы только по значению `HiddenSectors`; ему необходимо глубже анализировать фактический код, расположенный по этому смещению.

На рисунке 12-5 показано содержимое VBR из реальной системы, заражённой Gapz. BPB расположена со смещением 11, а поле `HiddenSectors`, содержащее значение `0x00000800`, выделено.

Рисунок 12-5. Значение `HiddenSectors` *в заражённой системе*

Чтобы обнаружить Gapz, защитное ПО должно проанализировать данные, расположенные со смещением `0x00000800` от начала жёсткого диска. Именно там находится вредоносный загрузчик.

Загрузка вредоносного кода режима ядра

Как и у многих современных буткитов, основное назначение кода буткита Gapz - скомпрометировать операционную систему, загрузив вредоносный код в адресное пространство режима ядра. Получив управление, код буткита Gapz приступает к обычной процедуре исправления загрузочных компонентов ОС, описанной в предыдущих главах.

После выполнения код буткита перехватывает обработчик INT 13h, чтобы отслеживать данные, читаемые с жёсткого диска. Затем он загружает с жёсткого диска исходный код IPL и выполняет его, возобновляя процесс загрузки. На рисунке 12-6 показан процесс загрузки в системе, заражённой Gapz.

Перехватив INT 13h ❶, вредоносная программа отслеживает данные, читаемые с жёсткого диска, и ищет модуль `bootmgr`, который затем исправляет в памяти, чтобы перехватить процедуру `Archx86TransferTo32BitApplicationAsm` (`Archx86TransferTo64BitApplicationAsm` для платформ Windows x64) ❷. Эта процедура передаёт управление из `bootmgr` точке входа `winload.exe`. Перехват используется для исправления модуля `winload.exe`. Когда срабатывает перехват в `bootmgr`, модуль `winload.exe` уже находится в памяти, и вредоносная программа может исправить его. Буткит перехватывает процедуру `OslArchTransferToKernel` ❸ в модуле `winload.exe`.

Как обсуждалось в предыдущей главе, Rovnix также начинал с перехвата обработчика INT 13h, исправления `bootmgr` и перехвата `OslArchTransferToKernel`. Но, в отличие от Gapz, на следующем этапе Rovnix компрометировал ядро, исправляя процедуру ядра `KiSystemStartup`.

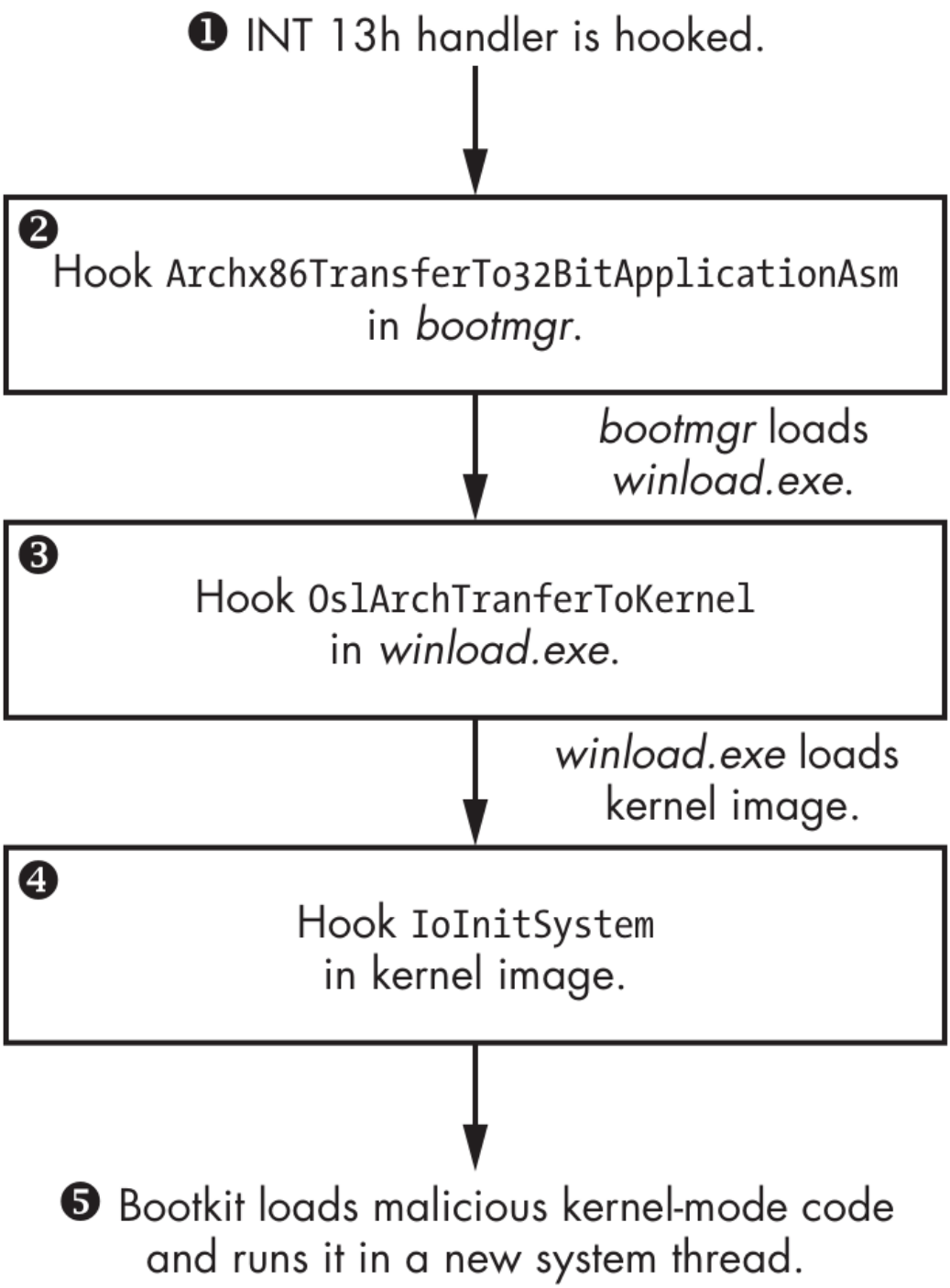


Рисунок 12-6. Рабочий процесс буткита

Gapz, напротив, перехватывает другую процедуру образа ядра - `IoInitSystem` ❹. Назначение этой процедуры - завершить инициализацию ядра, инициализировав различные подсистемы ОС и вызвав точки входа загрузочных драйверов. При выполнении `IoInitSystem` срабатывает вредоносный перехват, который восстанавливает исправленные байты процедуры `IoInitSystem` и перезаписывает адрес возврата `IoInitSystem` в стеке адресом вредоносного кода. Затем буткит Gapz возвращает управление процедуре `IoInitSystem`.

После завершения процедуры управление возвращается вредоносному коду. Когда `IoInitSystem` выполнена, ядро правильно инициализировано, и буткит может пользоваться предоставляемыми им службами для доступа к жёсткому диску, выделения памяти, создания потоков и других операций. Затем вредоносная программа читает с жёсткого диска оставшийся код буткита, создаёт системный поток и, наконец, возвращает управление ядру. После выполнения вредоносного кода режима ядра в адресном пространстве режима ядра работа буткита завершена ❺.

> **Как избежать обнаружения защитным ПО**
>
> В самом начале процесса загрузки Gapz удаляет заражение буткита из заражённой VBR и восстанавливает его позднее, во время выполнения своего модуля режима ядра. Одно из возможных объяснений состоит в том, что некоторые защитные продукты проверяют систему при запуске, поэтому, удаляя в этот момент свидетельства заражения из VBR, Gapz может остаться незамеченным.

Функциональность руткита Gapz

В этом разделе мы сосредоточимся на функциональности руткита вредоносной программы - самом интересном аспекте Gapz после функциональности буткита. Функциональность руткита Gapz мы будем называть модулем режима ядра, поскольку он не является допустимым драйвером режима ядра: фактически это вообще не образ PE. Он организован как позиционно-независимый код, состоящий из нескольких блоков, каждый из которых реализует определённую функциональность вредоносной программы для выполнения конкретной задачи. Назначение модуля режима ядра - тайно и незаметно внедрять полезную нагрузку в системные процессы.

Один из наиболее интересных аспектов модуля режима ядра Gapz состоит в реализации пользовательского сетевого стека TCP/IP для взаимодействия с командными серверами. Для защиты конфигурационных данных и канала связи с командными серверами он использует криптографическую библиотеку с собственными реализациями таких криптографических примитивов, как RC4, MD5, SHA1, AES и BASE64. Как и любая другая сложная угроза, Gapz реализует скрытое хранилище для тайного хранения полезной нагрузки пользовательского режима и конфигурационных сведений. Gapz также содержит мощный механизм перехвата со встроенным дизассемблером для установки устойчивых и скрытных перехватов. В оставшейся части раздела мы подробно рассмотрим эти и другие особенности модуля режима ядра Gapz.

Модуль режима ядра Gapz - не обычный образ PE, а набор блоков с позиционно-независимым кодом (Position-Independent Code, PIC), который не использует абсолютные адреса для обращения к данным. Поэтому его буфер памяти может находиться по любому допустимому виртуальному адресу в адресном пространстве процесса. Каждый блок служит определённой цели. Перед блоком находится заголовок, описывающий его размер и положение в модуле, а также некоторые константы, используемые для вычисления адресов процедур, реализованных в блоке. Разметка заголовка показана в листинге 12-5.

```
struct GAPZ_BASIC_BLOCK_HEADER
{
    // Константа, используемая для получения адресов
    // процедур, реализованных в блоке
    ❶ unsigned int ProcBase;
    unsigned int Reserved[2];

    // Смещение следующего блока
    ❷ unsigned int NextBlockOffset;
    // Смещение процедуры, выполняющей инициализацию блока
    ❸ unsigned int BlockInitialization;
    // Смещение конфигурационных сведений
    // от конца модуля режима ядра
    // действительно только для первого блока
    unsigned int CfgOffset;

    // Заполнено нулями
    unsigned int Reserved1[2];
}
```

Листинг 12-5. Заголовок блока модуля режима ядра Gapz

Заголовок начинается с целочисленной константы `ProcBase` ❶, используемой для вычисления смещений процедур, реализованных в базовом блоке. `NextBlockOffset` ❷ задаёт смещение следующего блока внутри модуля, позволяя Gapz перечислить все блоки модуля режима ядра. `BlockInitialization` ❸ содержит смещение от начала блока до процедуры инициализации блока, выполняемой при инициализации модуля режима ядра. Эта процедура инициализирует все необходимые структуры данных соответствующего блока и должна выполняться до любой другой функции, реализованной в блоке.

Gapz использует глобальную структуру, в которой хранятся все данные, относящиеся к его коду режима ядра: адреса реализованных процедур, указатели на выделенные буферы и так далее. Эта структура позволяет Gapz определять адреса всех процедур, реализованных в блоках позиционно-независимого кода, а затем выполнять их.

Позиционно-независимый код обращается к глобальной структуре посредством шестнадцатеричной константы `0xBBBBBBBB` для модуля x86. В самом начале выполнения вредоносного кода режима ядра Gapz выделяет буфер памяти для глобальной структуры. Затем он использует процедуру `BlockInitialization`, чтобы пройти по коду каждого блока и заменить каждое вхождение `0xBBBBBBBB` указателем на глобальную структуру.

Дизассемблированный код процедуры `OpenRegKey`, реализованной в модуле режима ядра, выглядит примерно как в листинге 12-6. Константа `0xBBBBBBBB` снова используется для обращения к адресу глобального контекста, но во время выполнения она заменяется фактическим адресом глобальной структуры в памяти, чтобы код выполнялся правильно.

```
int __stdcall OpenRegKey(PHANDLE hKey, PUNICODE_STRING Name)
{
    OBJECT_ATTRIBUTES obj_attr; // [esp+0h] (ebp-1Ch)@1
    int _global_ptr; // [esp+18h] (ebp-4h)@1
    _global_ptr = 0xBBBBBBBB;
    obj_attr.ObjectName = Name;
    obj_attr.RootDirectory = 0;
    obj_attr.SecurityDescriptor = 0;
    obj_attr.SecurityQualityOfService = 0;
    obj_attr.Length = 24;
    obj_attr.Attributes = 576;
    return (MEMORY[0xBBBBBBBB]->ZwOpenKey)(hKey, 0x20019, &obj_attr);
}
```

Листинг 12-6. Использование глобального контекста в коде режима ядра Gapz

Всего Gapz реализует в модуле режима ядра 12 блоков кода, перечисленных в таблице 12-2. Последний блок реализует основную процедуру модуля режима ядра, которая запускает выполнение модуля, инициализирует остальные блоки, устанавливает перехваты и начинает взаимодействие с командными серверами.

Таблица 12-2. Блоки кода режима ядра Gapz

Номер блока	Реализованная функциональность
1	Общий API, сбор сведений о жёстких дисках, строковые процедуры CRT и так далее
2	Криптографическая библиотека: RC4, MD5, SHA1, AES, BASE64 и прочее
3	Механизм перехвата, механизм дизассемблера
4	Реализация скрытого хранилища
5	Перехваты драйвера жёсткого диска, самозащита
6	Диспетчер полезной нагрузки
7	Внедрение полезной нагрузки в адресное пространство пользовательского режима процессов
8	Сетевое взаимодействие: канальный уровень
9	Сетевое взаимодействие: транспортный уровень
10	Сетевое взаимодействие: протокольный уровень
11	Интерфейс взаимодействия полезной нагрузки
12	Основная процедура

Скрытое хранилище

Как и большинство буткитов, Gapz реализует скрытое хранилище для безопасного размещения полезной нагрузки и конфигурационных сведений. Образ скрытой файловой системы находится в файле на жёстком диске по пути `\??\C:\System Volume Information\<XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX>`, где `X` обозначает шестнадцатеричные числа, сформированные на основе конфигурационных сведений. Скрытое хранилище имеет разметку файловой системы FAT32. На рисунке 12-7 показан пример содержимого каталога скрытого хранилища `\usr\overlord`. В каталоге находятся три файла: `overlord32.dll`, `overlord64.dll` и `conf.z`. Первые два соответствуют полезной нагрузке пользовательского режима, внедряемой в системные процессы. Третий файл, `conf.z`, содержит конфигурационные данные.

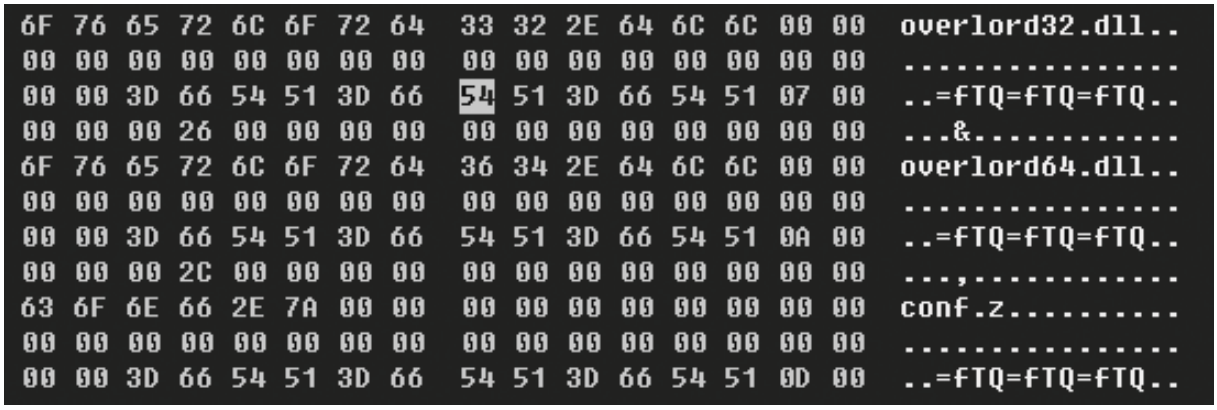


Рисунок 12-7. Содержимое каталога скрытого хранилища `\usr\overlord`

Чтобы сохранить сведения в скрытой файловой системе в тайне, её содержимое шифруется, как показано в листинге 12-7.

```
int __stdcall aes_crypt_sectors_cbc(int IV, int c_text, int p_text, int num_of_sect,
                                    int bEncrypt, STRUCT_AES_KEY *Key)
{
    int result; // eax@1
    int _iv; // edi@2
    int cbc_iv[4]; // [esp+0h] [ebp-14h]@3
    STRUCT_IPL_THREAD_1 *gl_struct; // [esp+10h] [ebp-4h]@1
    gl_struct = 0xBBBBBBBB;
    result = num_of_sect;
    if ( num_of_sect )
    {
        ❶ _iv = IV;
        do
        {
            cbc_iv[3] = 0;
            cbc_iv[2] = 0;
            cbc_iv[1] = 0;
            cbc_iv[0] = _iv; // Начальное значение CBC
            result = (gl_struct->crypto->aes_crypt_cbc)(Key, bEncrypt, 512, cbc_iv,
                                                        p_text, c_text);
            p_text += 512; // Открытый текст
            c_text += 512; // Шифротекст
         ❷ ++_iv;
            --num_of_sect;
        }
        while( num_of_sect );
    }
    return result;
}
```

Листинг 12-7. Шифрование секторов скрытого хранилища

Для шифрования и расшифрования каждого сектора скрытого хранилища Gapz использует собственную реализацию алгоритма Advanced Encryption Standard с длиной ключа 256 бит в режиме сцепления блоков шифротекста (Cipher Block Chaining, CBC). В качестве начального значения (Initialization Vector, IV) режима CBC Gapz использует номер первого шифруемого или расшифровываемого сектора ❶, как показано в листинге 12-7. Затем IV каждого следующего сектора увеличивается на 1 ❷. Хотя для шифрования каждого сектора жёсткого диска используется один и тот же ключ, разные IV для разных секторов каждый раз дают разные шифротексты.

Самозащита от антивредоносного ПО

Чтобы защититься от удаления из системы, Gapz перехватывает две процедуры минипортового драйвера жёсткого диска: `IRP_MJ_INTERNAL_DEVICE_CONTROL` и `IRP_MJ_DEVICE_CONTROL`. В этих перехватах вредоносную программу интересуют только следующие запросы.

• `IOCTL_SCSI_PASS_THROUGH`;
• `IOCTL_SCSI_PASS_THROUGH_DIRECT`;
• `IOCTL_ATA_PASS_THROUGH`;
• `IOCTL_ATA_PASS_THROUGH_DIRECT`.

Эти перехваты защищают заражённую VBR или MBR и образ Gapz на жёстком диске от чтения и перезаписи.

В отличие от TDL4, Olmasco и Rovnix, которые перезаписывают указатель на обработчики в структуре `DRIVER_OBJECT`, Gapz использует склейку, то есть исправляет непосредственно код обработчиков. В листинге 12-8 показана перехваченная процедура образа драйвера `scsiport.sys` в памяти. В этом примере `scsiport.sys` - минипортовый драйвер диска, реализующий обработчики запросов `IOCTL_SCSI_XXX` и `IOCTL_ATA_XXX`, и главная цель перехватов Gapz.

```
SCSIPORT!ScsiPortGlobalDispatch:
f84ce44c 8bff                    mov     edi,edi
❶ f84ce44e e902180307            jmp     ff4ffc55
f84ce453 088b42288b40            or      byte ptr [ebx+408B2842h],c1
f84ce459 1456                    adc     al,56h
f84ce45b 8b750c                  mov     esi,dword ptr [ebp+0Ch]
f84ce45e 8b4e60                  mov     ecx,dword ptr [esi+60h]
f84ce461 0fb609                  movzx   ecx,byte ptr [ecx]
f84ce464 56                      push    esi
f84ce465 52                      push    edx
f84ce466 ff1488                  call    dword ptr [eax+ecx*4]
f84ce469 5e                      pop     esi
f84ce46a 5d                      pop     ebp
f84ce46b c20800                  ret     8
```

Листинг 12-8. Перехват процедуры `scsiport!ScsiPortGlobalDispatch`

Обратите внимание: Gapz исправляет процедуру не в самом начале, по адресу `0xf84ce44c`, ❶, как часто делают другие вредоносные программы. В листинге 12-9 видно, что он пропускает некоторые инструкции в начале перехватываемой процедуры, например `nop` и `mov edi, edi`.

Одна из возможных причин - повышение стабильности и скрытности модуля режима ядра. Некоторые защитные программы при поиске исправленных или перехваченных процедур проверяют на изменения лишь первые несколько байтов, поэтому пропуск первых нескольких инструкций перед перехватом даёт Gapz возможность обойти защитные проверки.

Пропуск первых нескольких инструкций перехватываемой процедуры также не позволяет Gapz нарушить легитимные перехваты, уже установленные в процедурах. Например, в допускающих оперативное исправление исполняемых образах Windows компилятор помещает инструкции `mov edi, edi` в самое начало функций, что видно в листинге 12-8. Эта инструкция служит заполнителем для легитимного перехвата, который может установить ОС. Пропуская её, Gapz не нарушает возможности ОС по исправлению кода.

В листинге 12-9 приведён фрагмент процедуры перехвата, анализирующей инструкции обработчика в поисках наилучшего места для установки перехвата. Она проверяет коды операций инструкций `0x90`, соответствующий `nop`, и `0x8B`/`0x89`, соответствующие `mov edi, edi`. Эти инструкции могут означать, что целевая процедура принадлежит допускающему оперативное исправление образу и потому потенциально может быть исправлена ОС. Так вредоносная программа узнаёт, что при размещении перехвата их следует пропустить.

```
for ( patch_offset = code_to_patch; ; patch_offset += instr.len )
{
    (v42->proc_buff_3->disasm)(patch_offset, &instr);
    if ( (instr.len != 1 || instr.opcode != 0x90u)
        && (instr.len != 2 || instr.opcode != 0x89u &&
            instr.opcode != 0x8Bu || instr.modrm_rm != instr.modrm_reg) ) )
    {
        break;
    }
}
```

Листинг 12-9. Gapz использует дизассемблер, чтобы пропускать первые байты перехватываемых процедур

Для этого анализа Gapz реализует механизм Hacker Disassembler Engine, доступный для платформ x86 и x64. Он позволяет вредоносной программе получать не только длину инструкций, но и другие характеристики, например код операции инструкции и её операнды.

> **Hacker Disassembler Engine**
>
> Hacker Disassembler Engine (HDE) - небольшой, простой и удобный механизм дизассемблирования, предназначенный для анализа кода x86 и x64. Он предоставляет длину команды, код операции и другие параметры инструкции, такие как префиксы ModR/M и SIB. HDE часто используется вредоносными программами для дизассемблирования пролога процедур при установке вредоносных перехватов, как в только что описанном случае, или для обнаружения и удаления перехватов, установленных защитным ПО.

Внедрение полезной нагрузки

Модуль режима ядра Gapz внедряет полезную нагрузку в адресное пространство пользовательского режима следующим образом:

1. Читает конфигурационные сведения, чтобы определить, какие модули полезной нагрузки следует внедрить в конкретные процессы, а затем читает эти модули из скрытого хранилища.
2. Выделяет в адресном пространстве целевого процесса буфер памяти для хранения образа полезной нагрузки.
3. Создаёт и запускает в целевом процессе поток для выполнения кода загрузчика; поток отображает образ полезной нагрузки, инициализирует IAT и исправляет перемещения.

Каталог `\sys` скрытой файловой системы содержит конфигурационный файл, указывающий, какие модули полезной нагрузки следует внедрить в конкретные процессы. Имя конфигурационного файла получается из ключа шифрования AES скрытой файловой системы посредством алгоритма хеширования SHA1. Конфигурационный файл состоит из заголовка и нескольких элементов, каждый из которых описывает целевой процесс, как показано на рисунке 12-8.

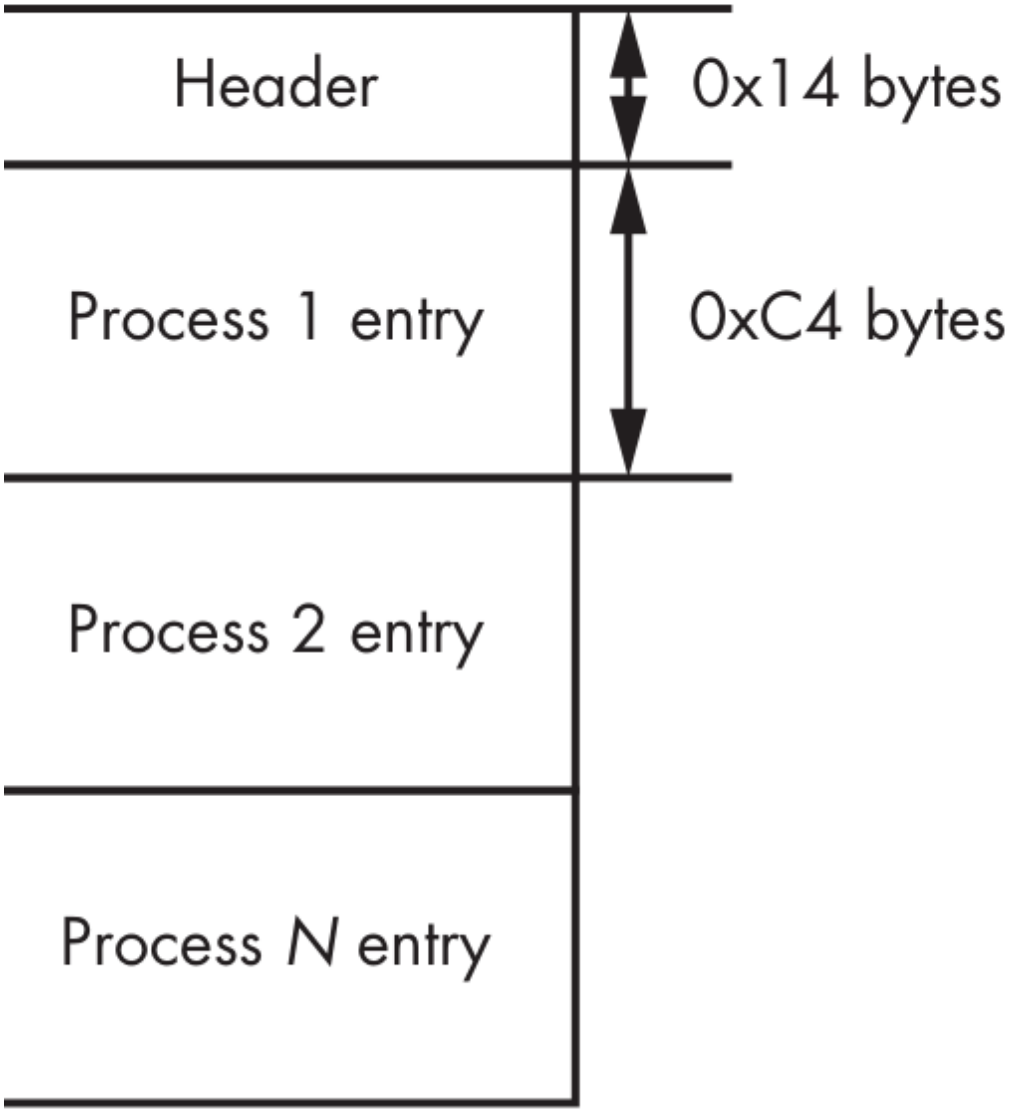


Рисунок 12-8. Разметка конфигурационного файла для внедрения полезной нагрузки

Каждый элемент процесса имеет разметку, показанную в листинге 12-10.

```
struct GAPZ_PAYLOAD_CFG
{
  // Полный путь к модулю полезной нагрузки в скрытом хранилище
  char PayloadPath[128];
  // Имя образа процесса
  ❶ char TargetProcess[64];
  // Задаёт параметры загрузки: x86 или x64 и так далее
  ❷ unsigned char LoadOptions;
  // Зарезервировано
  unsigned char Reserved[2];
  // Тип полезной нагрузки: overlord или другой
  ❸ unsigned char PayloadType;
}
```

Листинг 12-10. Разметка элемента конфигурации полезной нагрузки в конфигурационном файле

Поле `TargetProcess` ❶ содержит имя процесса, в который следует внедрить полезную нагрузку. Поле `LoadOptions` ❷ указывает, является ли модуль полезной нагрузки 32- или 64-разрядным образом, в зависимости от заражённой системы. Поле `PayloadType` ❸ показывает, является ли внедряемый модуль модулем `overlord` или другой полезной нагрузкой.

Модуль `overlord32.dll` (`overlord64.dll` для 64-разрядного процесса) внедряется в процессы `svchost.exe` системы. Назначение модуля `overlord32.dll` - выполнять команды Gapz, выдаваемые вредоносным кодом режима ядра. Эти команды могут выполнять следующие задачи:

• собирать сведения обо всех установленных в системе сетевых адаптерах и их свойствах;
• собирать сведения о наличии определённого программного обеспечения в системе;
• проверять подключение к интернету, пытаясь обратиться к update.microsoft.com;
• отправлять данные удалённому узлу и получать их от него посредством сокетов Windows;
• получать системное время с time.windows.com;
• получать IP-адрес узла по его доменному имени посредством Win32 API `gethostbyname`;
• получать оболочку Windows, запрашивая значение `shell` раздела реестра `Software\Microsoft\Windows NT\CurrentVersion\Winlogon`.

Затем результаты этих команд передаются обратно в режим ядра. На рисунке 12-9 показан пример конфигурационных сведений, извлечённых из скрытого хранилища заражённой системы.

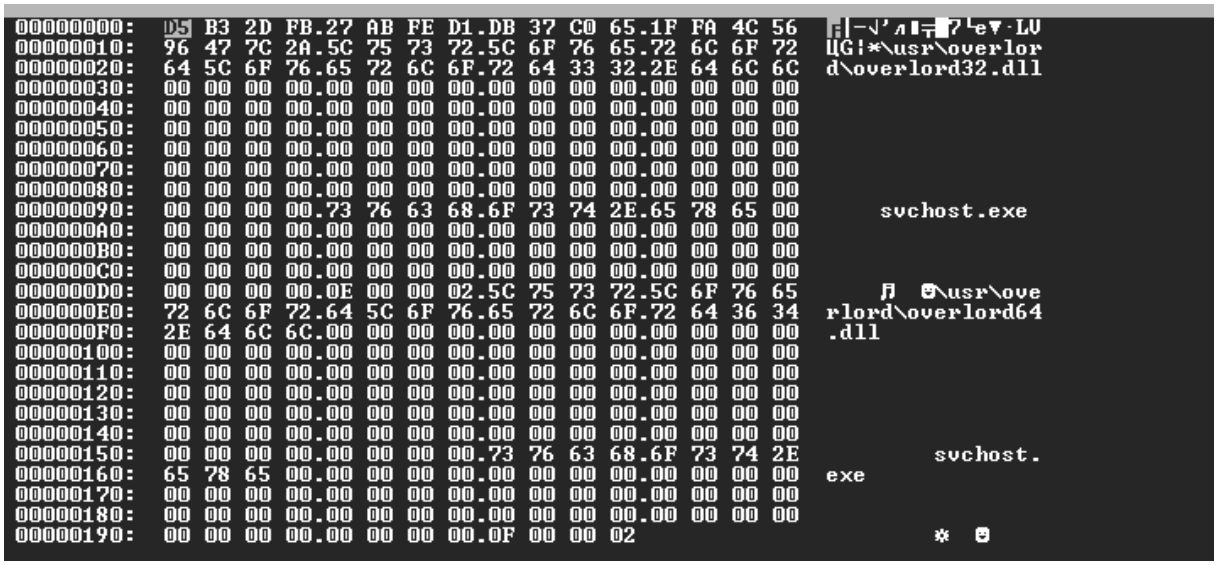


Рисунок 12-9. Пример конфигурационного файла полезной нагрузки

Видны два модуля, `overlord32.dll` и `overlord64.dll`, предназначенные для внедрения в процессы `svchost.exe` соответственно в системах x86 и x64.

Определив модуль полезной нагрузки и целевой процесс, Gapz выделяет буфер памяти в адресном пространстве целевого процесса и копирует в него модуль полезной нагрузки. Затем вредоносная программа создаёт в целевом процессе поток для выполнения кода загрузчика. Если операционная система - Windows Vista или более новая, Gapz может создать новый поток простым выполнением системной процедуры `NtCreateThreadEx`.

В операционных системах до Vista, таких как Windows XP или Server 2003, всё несколько сложнее, поскольку процедура `NtCreateThreadEx` не экспортируется ядром ОС. В таких случаях Gapz повторно реализует часть функциональности `NtCreateThreadEx` в модуле режима ядра и выполняет следующие действия:

1. Вручную выделяет стек для нового потока.
2. Инициализирует контекст потока и блок окружения потока (Thread Environment Block, TEB).
3. Создаёт структуру потока посредством недокументированной процедуры `NtCreateThread`.
4. При необходимости регистрирует вновь созданный поток в подсистеме времени выполнения «клиент-сервер» (Client/Server Runtime Subsystem, CSRSS).
5. Выполняет новый поток.

Код загрузчика отвечает за отображение полезной нагрузки в адресное пространство процесса и выполняется в пользовательском режиме. В зависимости от типа полезной нагрузки существуют разные реализации кода загрузчика, как показано на рисунке 12-10. Для модулей полезной нагрузки, реализованных как библиотеки DLL, есть два загрузчика: загрузчик DLL и исполнитель команд. Для модулей полезной нагрузки, реализованных как модули EXE, также имеются два загрузчика.

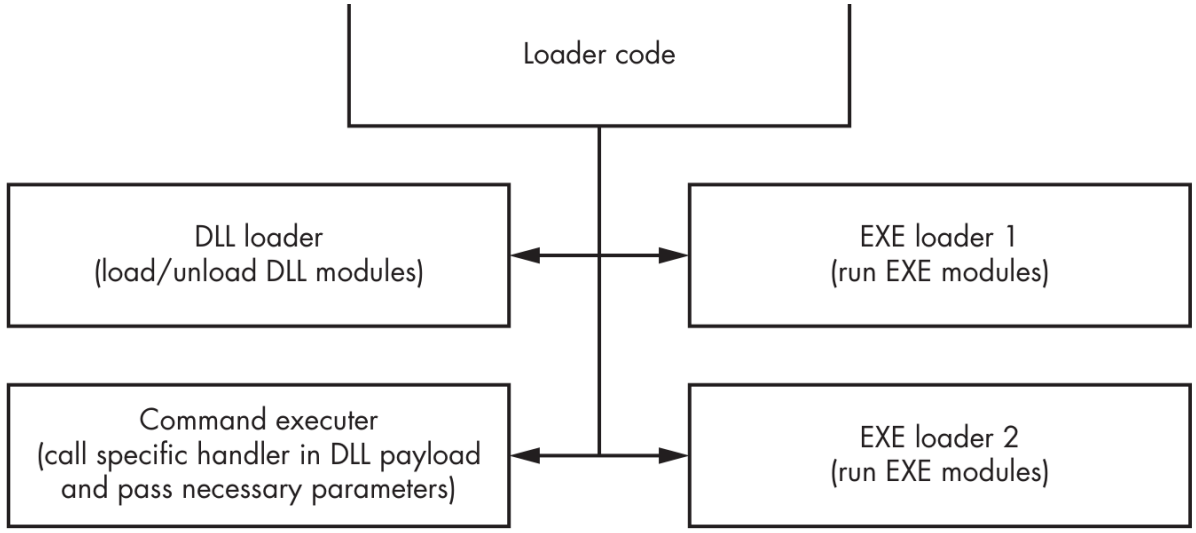


Рисунок 12-10. Возможности внедрения Gapz

Теперь рассмотрим каждый загрузчик.

Код загрузчика DLL

Процедура загрузчика DLL Gapz отвечает за загрузку и выгрузку DLL. Она отображает исполняемый образ в адресное пространство пользовательского режима целевого процесса, инициализирует его IAT, исправляет перемещения и выполняет следующие экспортируемые процедуры в зависимости от того, загружается или выгружается полезная нагрузка:

• **Экспортируемая процедура № 1 (загрузка полезной нагрузки)** инициализирует загруженную полезную нагрузку.
• **Экспортируемая процедура № 2 (выгрузка полезной нагрузки)** деинициализирует загруженную полезную нагрузку.

На рисунке 12-11 показан модуль полезной нагрузки `overlord32.dll`.

Name	Address	Ordinal
overlord32_1	10001505	1
overlord32_2	10001707	2
overlord32_3	10001765	3

Рисунок 12-11. Таблица адресов экспорта полезной нагрузки Gapz

На рисунке 12-12 изображена процедура. При выгрузке полезной нагрузки Gapz выполняет экспортируемую процедуру № 2 и освобождает память, использованную для хранения образа полезной нагрузки. При загрузке Gapz выполняет все необходимые действия для отображения образа в адресное пространство процесса, а затем выполняет экспортируемую процедуру № 1.

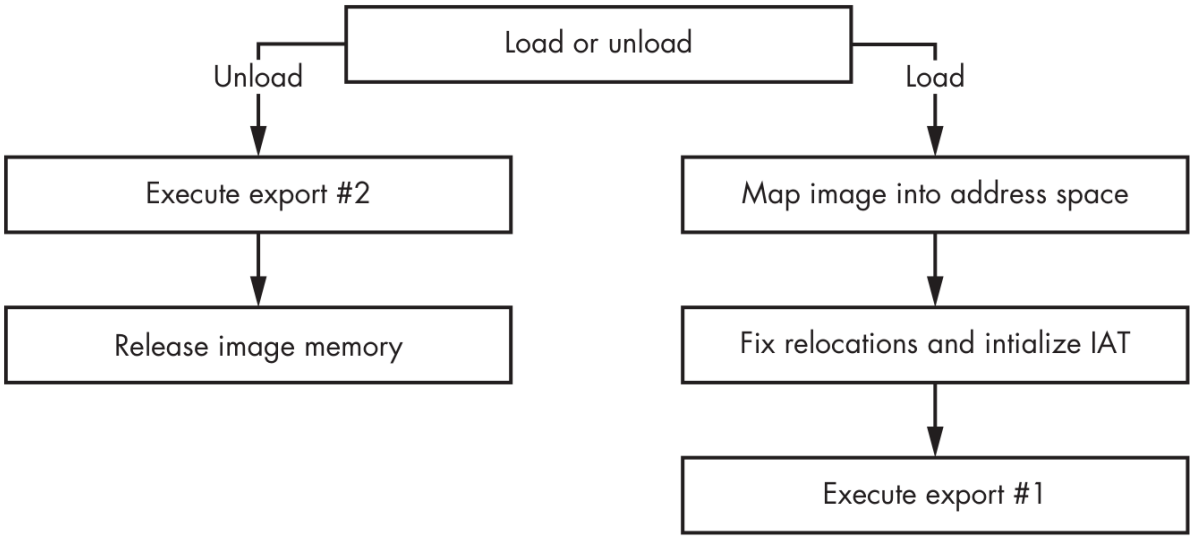


Рисунок 12-12. Алгоритм загрузки полезной нагрузки DLL Gapz

Код исполнителя команд

Процедура исполнителя команд отвечает за выполнение команд по указанию загруженного модуля DLL полезной нагрузки. Эта процедура всего лишь вызывает экспортируемую процедуру № 3 полезной нагрузки (рисунок 12-11) и передаёт её обработчику все необходимые параметры.

Код загрузчика EXE

Две оставшиеся процедуры загрузчика используются для запуска загруженных исполняемых файлов в заражённой системе. Первая реализация запускает исполняемую полезную нагрузку из каталога `TEMP`: образ сохраняется в каталоге `TEMP`, после чего выполняется API `CreateProcess`, как показано на рисунке 12-13.

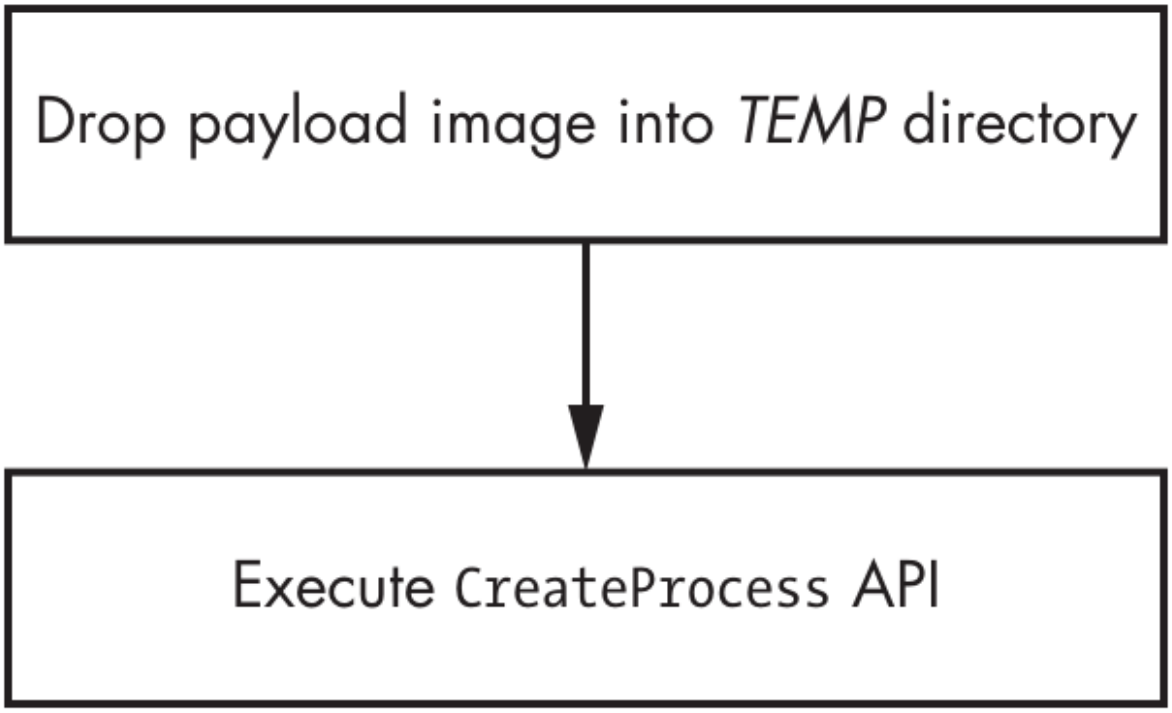


Рисунок 12-13. Алгоритм запуска полезной нагрузки EXE Gapz посредством `CreateProcess`

Вторая реализация запускает полезную нагрузку, создавая приостановленный легитимный процесс, затем перезаписывает образ легитимного процесса вредоносным образом и после этого возобновляет процесс, как показано на рисунке 12-14.

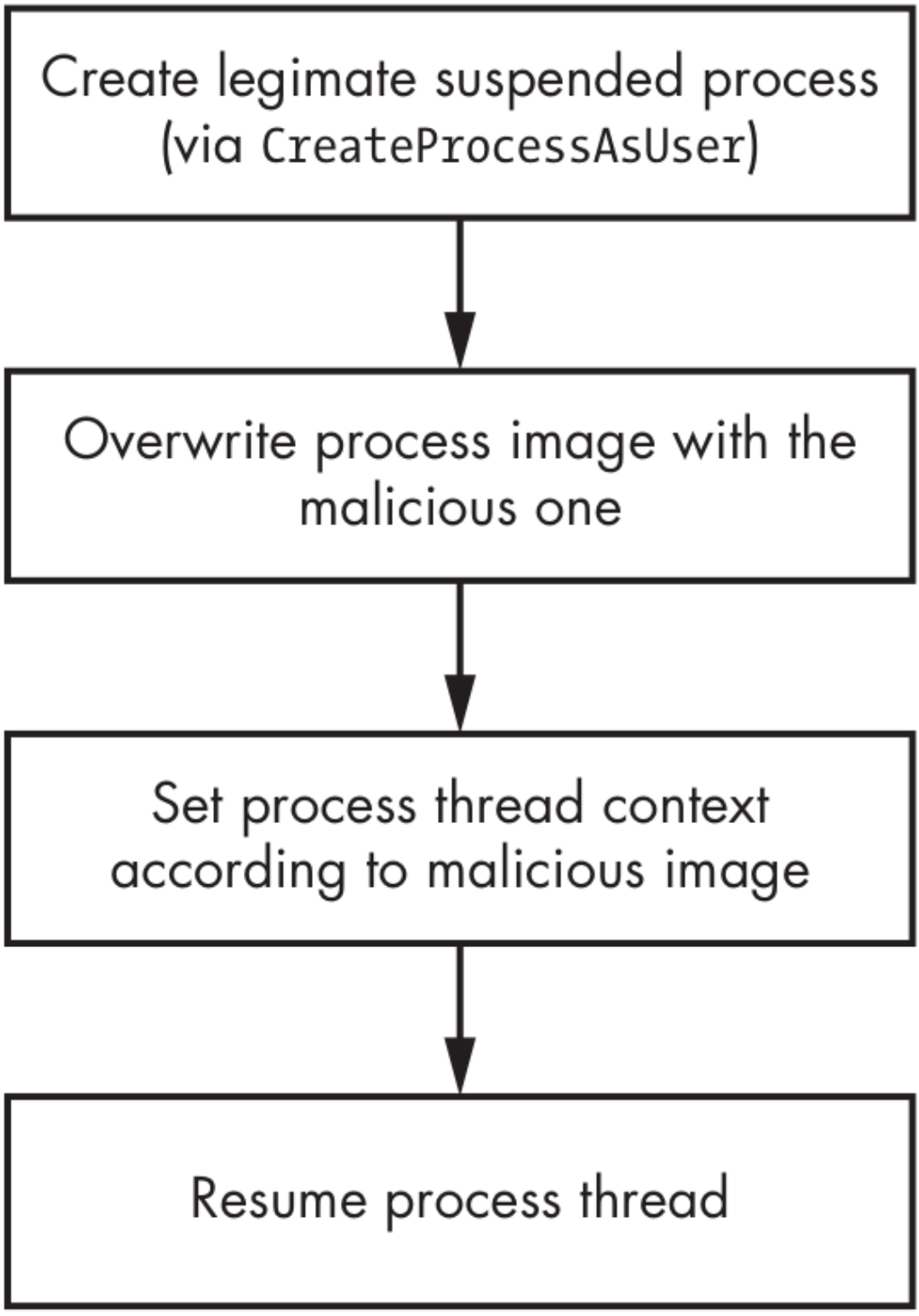


Рисунок 12-14. Алгоритм запуска полезной нагрузки EXE Gapz посредством `CreateProcessAsUser`

Второй способ загрузки исполняемой полезной нагрузки скрытнее первого и с меньшей вероятностью будет обнаружен. Первый способ просто запускает полезную нагрузку без каких-либо предосторожностей, тогда как второй сначала создаёт процесс с легитимным исполняемым файлом и только затем заменяет исходный образ вредоносной полезной нагрузкой. Это может заставить защитное ПО разрешить выполнение полезной нагрузки.

Интерфейс взаимодействия с полезной нагрузкой

Для взаимодействия с внедрённой полезной нагрузкой Gapz реализует специальный интерфейс весьма необычным способом: выдавая себя за обработчик запросов полезной нагрузки в драйвере `null.sys`. Этот приём показан на рисунке 12-15.

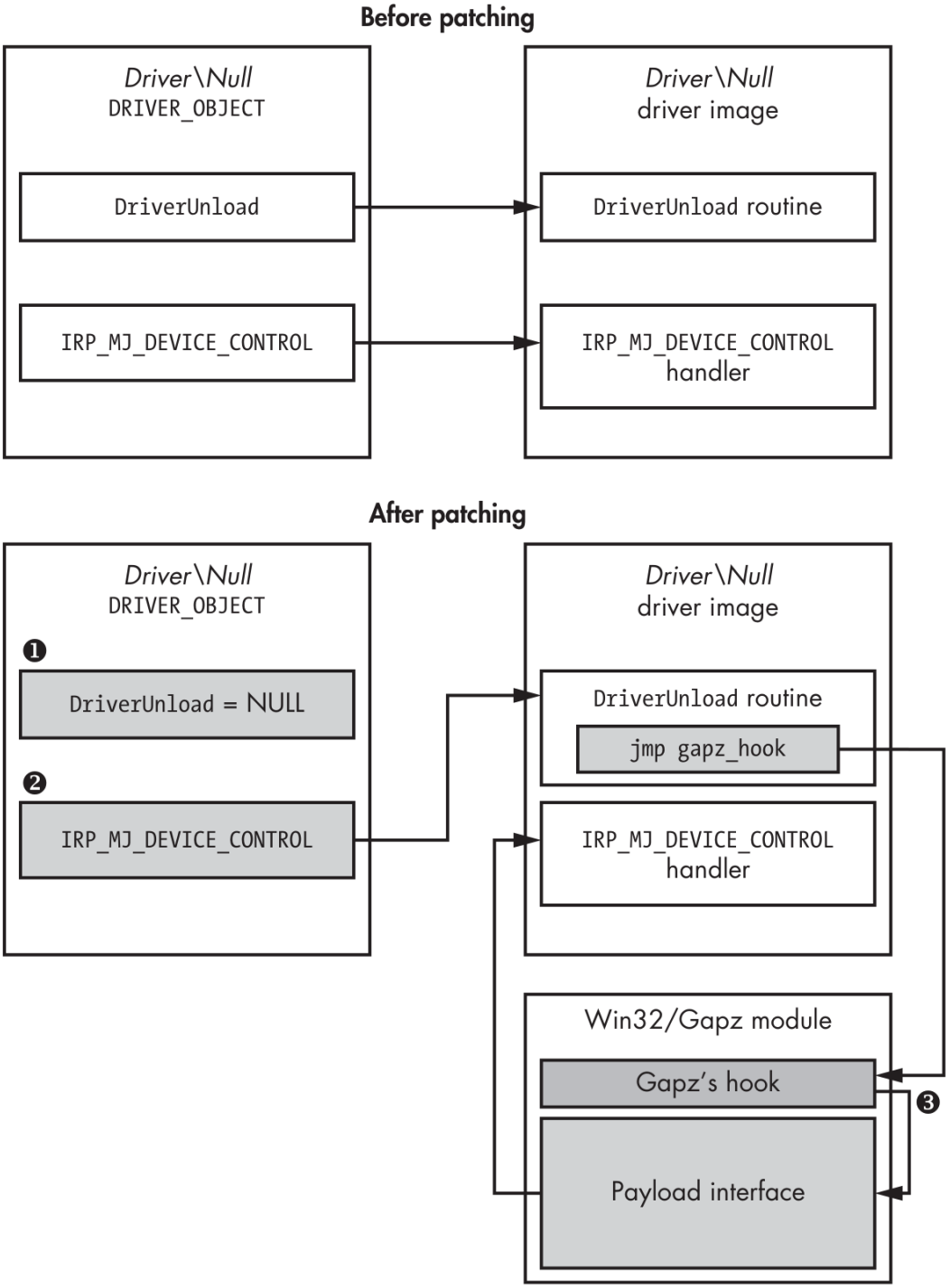


Рисунок 12-15. Архитектура интерфейса полезной нагрузки Gapz

Сначала вредоносная программа устанавливает в 0 поле `DriverUnload` ❶ структуры `DRIVER_OBJECT`, соответствующей объекту устройства `\Device\Null`; в этом поле хранится указатель на обработчик, который выполняется при выгрузке драйвера операционной системой. Затем она перехватывает исходную процедуру `DriverUnload`. После этого она перезаписывает адрес обработчика `IRP_MJ_DEVICE_CONTROL` в `DRIVER_OBJECT` адресом перехваченной процедуры `DriverUnload` ❷.

Перехват проверяет параметры запроса `IRP_MJ_DEVICE_CONTROL`, чтобы определить, был ли запрос инициирован полезной нагрузкой. Если да, вместо исходного обработчика `IRP_MJ_DEVICE_CONTROL` вызывается обработчик интерфейса полезной нагрузки ❸.

Фрагмент перехвата DriverUnload показан в листинге 12-11.

```
hooked_ioctl = MEMORY[0xBBBBBBE3]->IoControlCode_HookArray;
❶ while ( *hooked_ioctl != IoStack->Parameters.DeviceIoControl.IoControlCode )
{
    ++i; // Проверить, поступил ли запрос от полезной нагрузки
    ++hooked_ioctl;
    if ( i >= IRP_MJ_SYSTEM_CONTROL )
        goto LABEL_11;
}
UserBuff = Irp->UserBuffer;
OutputBufferLength = IoStack->Parameters.DeviceIoControl.OutputBufferLength;
if ( UserBuff )
{
    // Расшифровать запрос полезной нагрузки
 ❷ (MEMORY[0xBBBBBBBF]->rc4)(UserBuff, OutputBufferLength,
                             MEMORY[0xBBBBBBBB]->rc4_key, 48);
    v4 = 0xBBBBBBBB;
    // Проверить сигнатуру
    if ( *UserBuff == 0x34798977 )
    {
        hooked_ioctl = MEMORY[0xBBBBBBE3];
        IoStack = i;
        // Определить обработчик
        if ( UserBuff[1] == MEMORY[0xBBBBBBE3]->IoControlCodeSubCmd_Hook[i] )
        {
           ❸ (MEMORY[0xBBBBBBE3]->IoControlCode_HookDpc[i])(UserBuff);
           ❹ (MEMORY[0xBBBBBBBF]->rc4)( // Зашифровать ответ
                UserBuff,
                OutputBufferLength,
                MEMORY[0xBBBBBBBB]->rc4_key,
                48);
             v4 = 0xBBBBBBBB;
        }
        _Irp = Irp;
    }
}
```

Листинг 12-11. Перехват DriverUnload *драйвера* null.sys

> **Процедура DriverUnload**
>
> Перед выгрузкой драйвера режима ядра ядро операционной системы выполняет специальную процедуру DriverUnload. Эта необязательная процедура, реализуемая выгружаемым драйвером режима ядра, используется для выполнения всех операций, необходимых перед выгрузкой драйвера системой. Указатель на процедуру хранится в поле DriverUnload соответствующей структуры DRIVER_OBJECT. Если эта процедура не реализована, поле DriverUnload содержит NULL, и драйвер невозможно выгрузить.

В точке ❶ Gapz проверяет, поступил ли запрос от полезной нагрузки. Если да, он расшифровывает запрос шифром RC4 ❷ и выполняет соответствующий обработчик ❸. Обработав запрос, Gapz шифрует результат ❹ и отправляет его полезной нагрузке.

Полезная нагрузка может отправлять запросы модулю режима ядра Gapz посредством кода из листинга 12-12.

```
// Открыть дескриптор \Device\NULL
❶ HANDLE hNull = CreateFile(_T("\\??\\NUL"), ...);
if(hNull != INVALID_HANDLE_VALUE) {
  // Отправить запрос модулю режима ядра
❷ DWORD dwResult = DeviceIoControl(hNull, WIN32_GAPZ_IOCTL, InBuffer, InBufferSize,
                                   OutBuffer, OutBufferSize, &BytesRead);
  CloseHandle(hNull);
}
```

Листинг 12-12. Отправка запроса из полезной нагрузки пользовательского режима модулю режима ядра

Полезная нагрузка открывает дескриптор устройства `NULL` ❶. Это системное устройство, поэтому операция не должна привлечь внимание защитного ПО. Получив дескриптор, полезная нагрузка взаимодействует с модулем режима ядра посредством системного API `DeviceIoControl` ❷.

Пользовательский стек сетевых протоколов

Буткит взаимодействует с командными серверами по протоколу HTTP, основное назначение которого - запрашивать и загружать полезную нагрузку и сообщать о состоянии бота. Вредоносная программа обязательно применяет шифрование, чтобы защищать конфиденциальность передаваемых сообщений и проверять подлинность их источника, предотвращая подмену командами от фальшивых командных серверов.

Самая примечательная особенность сетевого взаимодействия - способ его реализации. Вредоносная программа может отправить сообщение командному серверу двумя способами: посредством модуля полезной нагрузки пользовательского режима (`overlord32.dll` или `overlord64.dll`) либо посредством собственной реализации стека протоколов TCP/IP в режиме ядра. Эта схема сетевого взаимодействия показана на рисунке 12-16.

Полезная нагрузка пользовательского режима, `overlord32.dll` или `overlord64.dll`, отправляет сообщение командному серверу посредством реализации сокетов Windows. Собственная реализация стека протоколов TCP/IP опирается на минипортовый драйвер адаптера. Обычно запросы сетевого взаимодействия проходят через стек сетевых драйверов и на разных его уровнях могут проверяться драйверами защитного ПО. Согласно спецификации сетевого интерфейса драйверов Microsoft (Network Driver Interface Specification, NDIS), минипортовый драйвер находится ниже всех остальных драйверов сетевого стека. Поэтому, отправляя пакеты сетевого ввода-вывода непосредственно объекту устройства минипорта, Gapz может обойти все промежуточные драйверы и избежать проверки (см. рисунок 12-17).

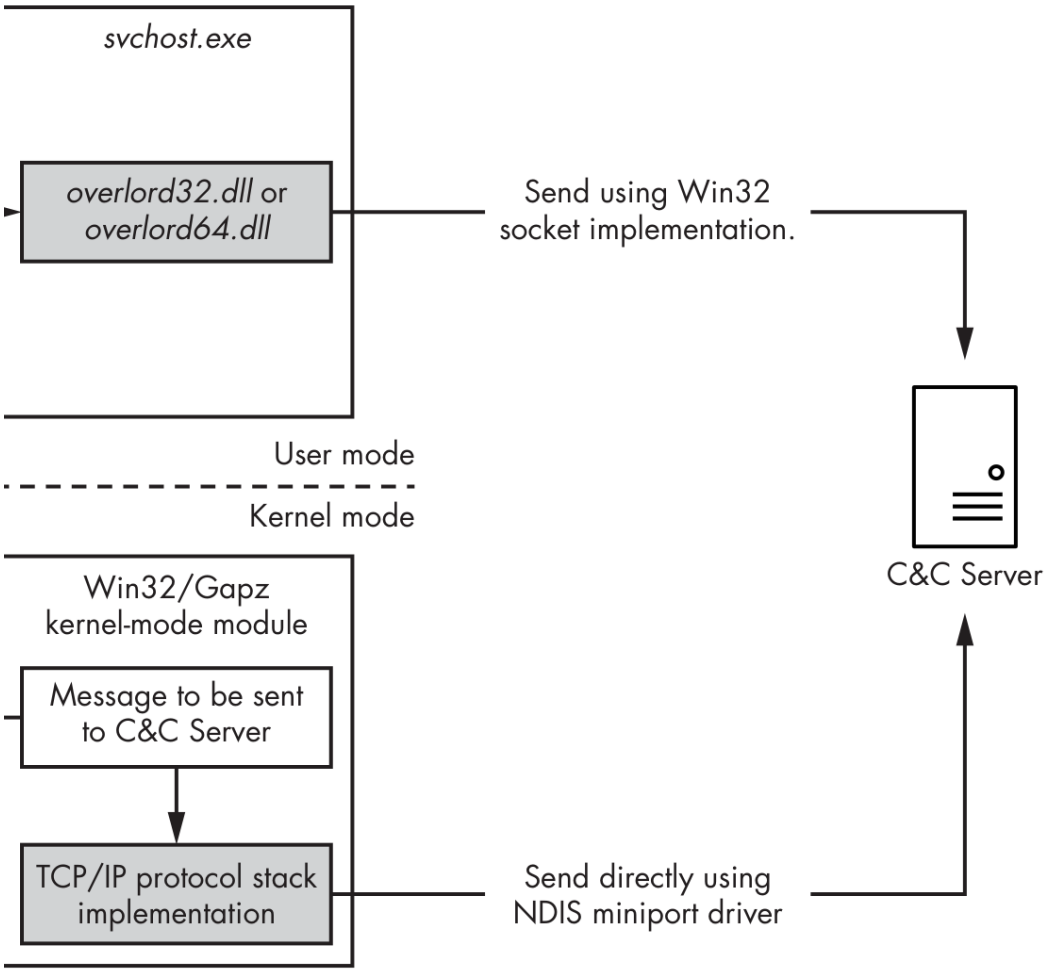


Рисунок 12-16. Схема сетевого взаимодействия Gapz

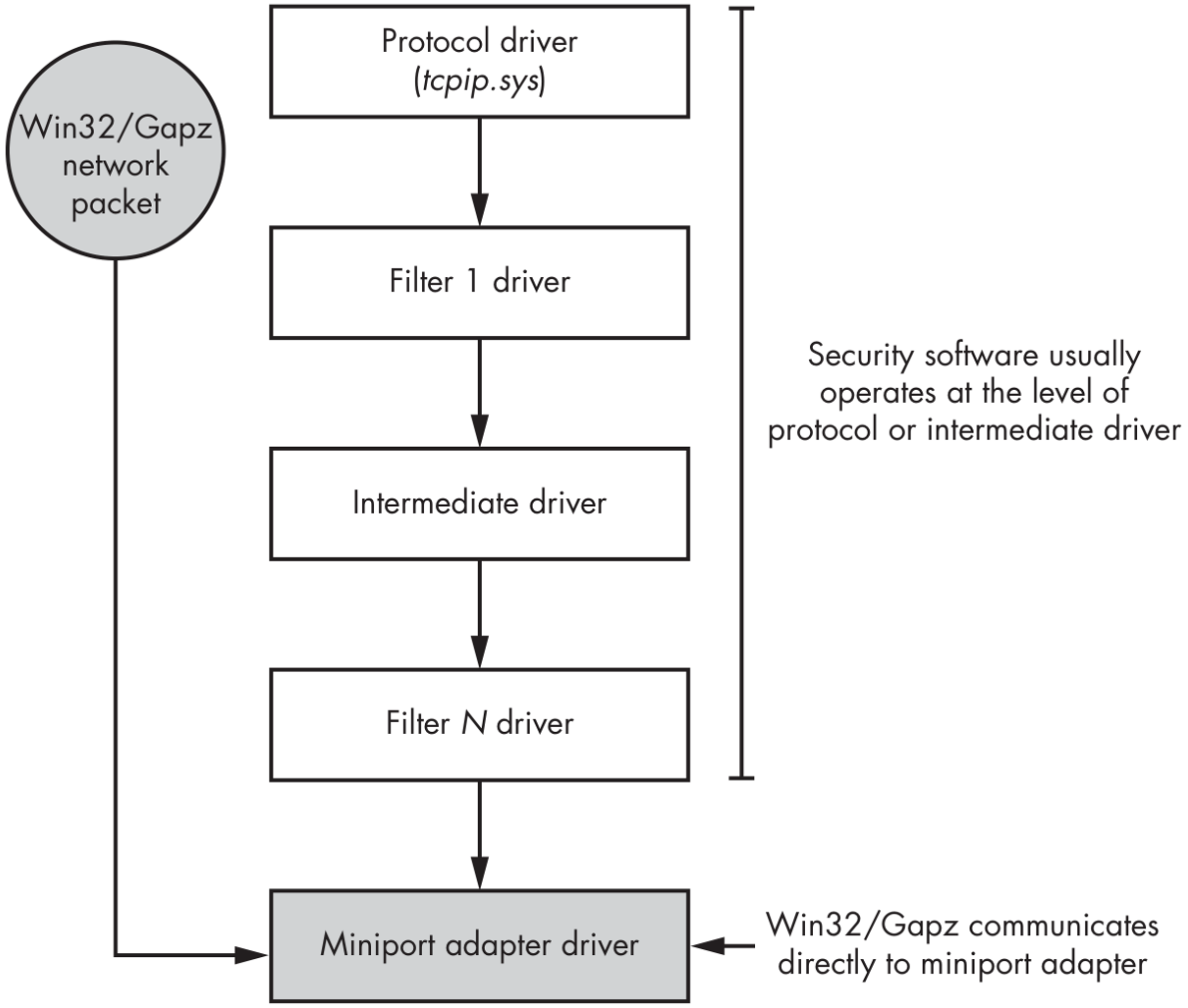


Рисунок 12-17. Пользовательская сетевая реализация Gapz

Gapz получает указатель на структуру, описывающую минипортовый адаптер, вручную исследуя код библиотеки NDIS (`ndis.sys`). Процедура, отвечающая за работу с минипортовыми адаптерами NDIS, реализована в блоке № 8 модуля режима ядра.

Такой подход позволяет Gapz использовать интерфейс сокетов для незаметного взаимодействия с командным сервером. Архитектура сетевой подсистемы Gapz обобщена на рисунке 12-18.

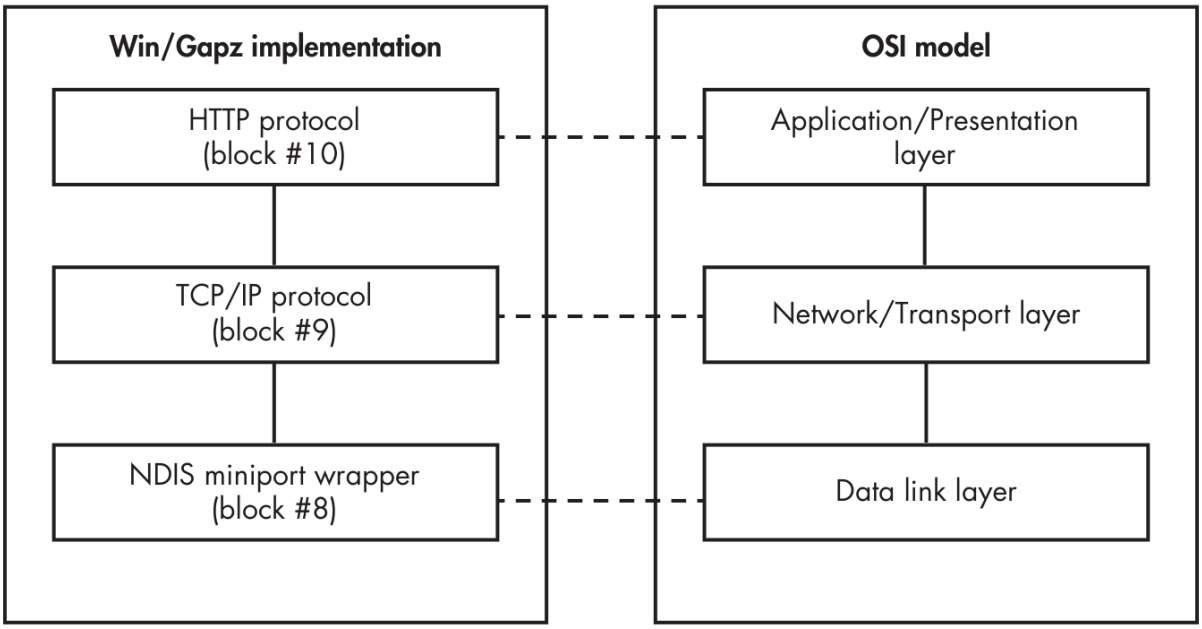


Рисунок 12-18. Сетевая архитектура Gapz

Как видно, сетевая архитектура Gapz реализует большинство уровней модели взаимодействия открытых систем (Open Systems Interconnection, OSI): канальный, транспортный и прикладной. Для отправки сетевых пакетов физическому объекту устройства, представляющему сетевую плату, и получения пакетов от него Gapz использует соответствующий интерфейс, доступный в системе и предоставляемый драйвером сетевой платы. Однако вся работа по созданию и разбору сетевых кадров полностью реализована в собственном сетевом стеке вредоносной программы.

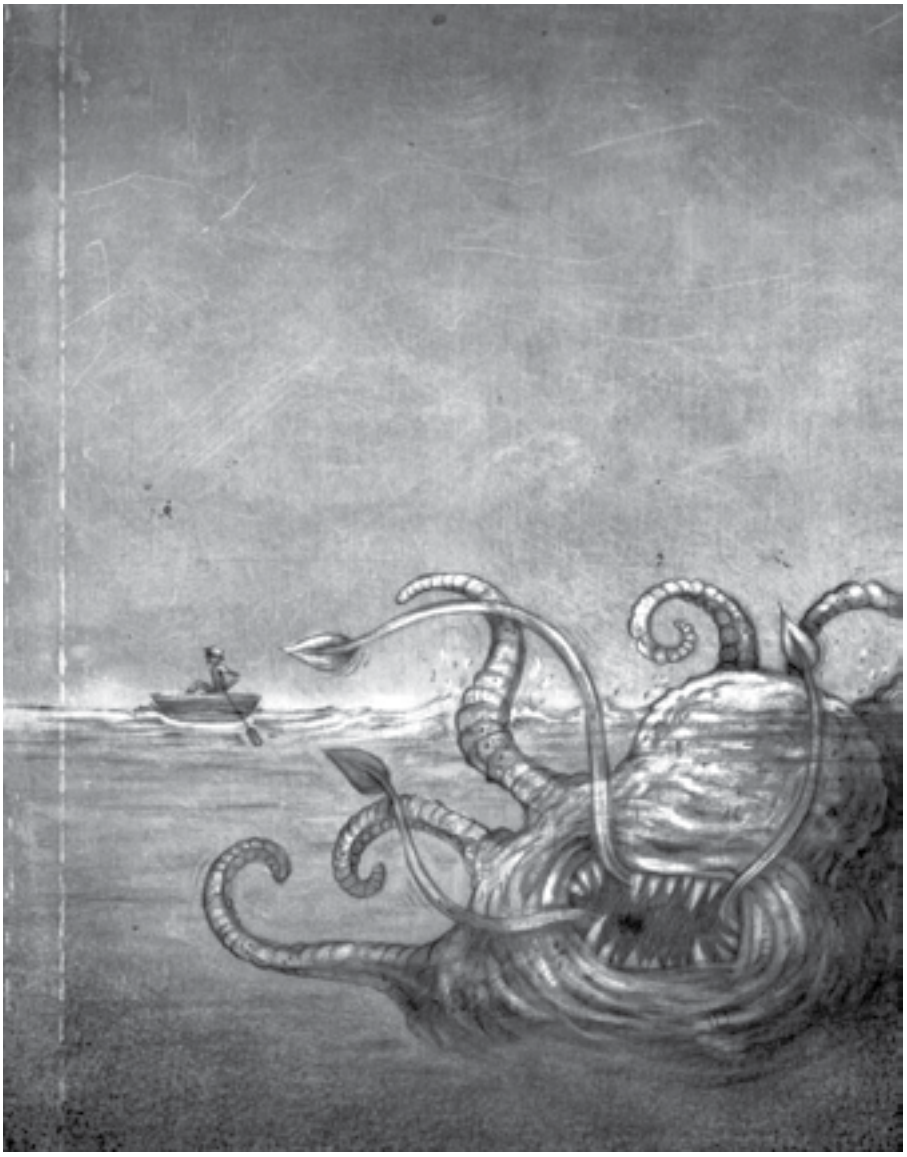
Заключение

Как вы увидели, Gapz - сложная вредоносная программа с очень тщательно разработанной реализацией и один из наиболее выдающихся скрытных буткитов благодаря своему методу заражения VBR. Ни один ранее известный буткит не мог похвастаться настолько элегантным и одновременно тонким подходом к заражению. Его обнаружение заставило индустрию безопасности усовершенствовать методы выявления буткитов и углубить сканирование MBR и VBR, обращая внимание не только на изменения кода MBR и VBR, но и на параметры и структуры данных, ранее считавшиеся не относящимися к проверке.

Сноски

[1] [назад] Eugene Rodionov and Aleksandr Matrosov, “Mind the Gapz,” Spring 2013, welivesecurity.com.

Глава 13. Расцвет программ-вымогателей MBR



До сих пор все описанные в этой книге примеры вредоносных программ относились к определённому классу: компьютерным троянам с функциональностью руткита или буткита, задача которых - достаточно долго сохраняться в системах жертв, чтобы выполнять различные вредоносные действия, например мошенничать с переходами в браузере, рассылать спам, открывать лазейку или создавать HTTP-прокси. Эти трояны используют методы закрепления буткитов, чтобы сохраняться на заражённых компьютерах, и функциональность руткитов, чтобы оставаться незамеченными.

В этой главе мы рассмотрим программы-вымогатели - семейство вредоносных программ с совершенно иным образом действий. Как следует из названия, основное назначение программы-вымогателя - полностью лишить пользователей доступа к их данным или компьютерной системе и потребовать выкуп за восстановление доступа.

В большинстве известных случаев программы-вымогатели используют шифрование, чтобы лишить пользователей их данных. После запуска вредоносная программа пытается зашифровать всё ценное для пользователя - документы, фотографии, электронные письма и так далее, - а затем требует заплатить выкуп за получение ключа шифрования, которым можно расшифровать данные.

Большинство программ-вымогателей нацелено на пользовательские файлы в файловой системе компьютера, но такие методы не реализуют развитой функциональности руткита или буткита и потому не относятся к теме этой книги. Однако некоторые семейства программ-вымогателей вместо этого шифруют секторы жёсткого диска, чтобы заблокировать пользователю доступ к системе, и применяют для этого функциональность буткита.

В этой главе мы сосредоточимся на последней категории: программах-вымогателях, которые атакуют жёсткие диски компьютеров и лишают жертв не только файлов, но и доступа ко всей компьютерной системе. Такая программа-вымогатель шифрует определённые области жёсткого диска и устанавливает вредоносный загрузчик в MBR. Вместо загрузки операционной системы загрузчик выполняет низкоуровневое шифрование содержимого жёсткого диска и выводит жертве сообщение с требованием выкупа. Особое внимание мы уделим двум семействам, получившим широкое освещение в СМИ: Petya и Satana.

Краткая история современных программ-вымогателей

Первые признаки вредоносной программы, похожей на программу-вымогатель, проявились в компьютерном вирусе AIDS, впервые обнаруженном в реальных атаках в 1989 году. AIDS применял методы, похожие на методы современных программ-вымогателей, для заражения старых исполняемых файлов COM в MS-DOS, перезаписывая начало файлов вредоносным кодом так, что восстановить их было невозможно. Однако AIDS не требовал от жертв выкуп за восстановление доступа к заражённым программам: он просто уничтожал сведения без возможности их вернуть.

Первой известной вредоносной программой, требовавшей выкуп, был троян GpCode, впервые появившийся в 2004 году. Он прославился применением 660-битного алгоритма шифрования RSA для блокировки пользовательских файлов. Достижения в разложении целых чисел на множители сделали в 2004 году почти осуществимым разложение 600-битных целых чисел; в 2005 году была присуждена денежная премия за успешное разложение RSA-640, 640-битного числа. Последующие модификации перешли на 1024-битное шифрование RSA, повысив устойчивость вредоносной программы к атакам полным перебором. GpCode распространялся во вложении электронной почты, выдававшем себя за заявление о приёме на работу. После запуска в системе жертвы он шифровал пользовательские файлы и выводил сообщение с требованием выкупа.

Несмотря на эти ранние появления, до 2012 года программы-вымогатели не были широко распространённой угрозой, но с тех пор сохраняют распространённость. Одним из факторов, вероятно сыгравших важную роль в их росте, стало повышение популярности анонимизированных сетевых служб, таких как платёжные системы Bitcoin и Tor. Разработчики программ-вымогателей могли использовать такие системы для получения выкупов без отслеживания правоохранительными органами. Этот киберпреступный бизнес оказался чрезвычайно прибыльным, что привело к разнообразному развитию и широкому распространению программ-вымогателей.

Программой-вымогателем, положившей начало всплеску 2012 года, была Reveton. Она маскировалась под сообщение правоохранительной организации, подобранное под местонахождение пользователя. Например, жертвам в Соединённых Штатах показывалось сообщение якобы от ФБР. Жертв обвиняли в незаконных действиях, таких как использование защищённых авторским правом материалов без разрешения или просмотр и распространение порнографии, и предписывали уплатить штраф через такие службы, как Ukash, Paysafe или MoneyPak.

Вскоре в реальных атаках появились новые угрозы со схожей функциональностью. Обнаруженная в 2013 году CryptoLocker была ведущей угрозой среди программ-вымогателей того времени. Она

применяла 2048-битное шифрование RSA и распространялась главным образом через скомпрометированные сайты и вложения электронной почты. Одна из интересных особенностей CryptoLocker состояла в том, что жертвы должны были платить выкуп в Bitcoin или предоплаченными денежными ваучерами. Применение Bitcoin добавляло угрозе ещё один уровень анонимности и чрезвычайно затрудняло отслеживание злоумышленников.

Ещё одна примечательная программа-вымогатель - CTB-Locker, появившаяся в 2014 году. Сокращение CTB означает Curve/TOR/Bitcoin и указывает на основные технологии, применяемые угрозой. CTB-Locker использовала алгоритм шифрования на эллиптических кривых (Elliptic Curve Cryptography, ECC) и стала первой известной программой-вымогателем, использовавшей протокол TOR для сокрытия командных серверов.

Киберпреступный бизнес остаётся чрезвычайно прибыльным и по сей день, а программы-вымогатели продолжают развиваться, причём регулярно появляются многочисленные модификации. Рассмотренные здесь семейства программ-вымогателей составляют лишь малую долю всех известных угроз этого класса.

Программы-вымогатели с функциональностью буткита

В 2016 году были обнаружены два новых семейства программ-вымогателей: Petya и Satana. Вместо шифрования пользовательских файлов в файловой системе Petya и Satana шифровали части жёсткого диска, делая ОС незагружаемой, и выводили жертвам сообщение с требованием оплаты за восстановление зашифрованных секторов. Простейший способ реализовать интерфейс вывода сообщения с требованием выкупа - воспользоваться методами заражения буткита на основе MBR.

Petya лишала пользователей доступа к системам, шифруя содержимое главной файловой таблицы (Master File Table, MFT) на жёстком диске. MFT - важнейшая специальная структура данных тома NTFS, содержащая сведения обо всех хранящихся в нём файлах, например их местонахождении на томе, именах и других атрибутах. Она в основном используется как индекс для поиска расположения файлов на жёстком диске. Зашифровав MFT, Petya делала невозможным поиск файлов, из-за чего жертвы не могли получить доступ к файлам тома и даже загрузить систему.

Petya распространялась главным образом как ссылка в электронном письме, якобы открывавшая заявление о приёме на работу. В действительности заражённая ссылка указывала на вредоносный ZIP-архив с дроппером Petya. Для размещения ZIP-архивов вредоносная программа использовала даже легитимную службу Dropbox.

Обнаруженная вскоре после Petya программа Satana также лишала жертв доступа к системам, шифруя MBR жёсткого диска. Хотя её возможности заражения MBR были не такими сложными, как у Petya, и даже содержали несколько ошибок, они представляли достаточный интерес, чтобы кратко обсудить Satana.

Shamoon: потерянный троян

Shamoon - троян, появившийся примерно одновременно с Satana и Petya и имевший схожую функциональность. Он получил печальную известность за уничтожение данных в целевых системах и лишение их возможности загружаться. Его основным назначением было нарушение работы целевых организаций, преимущественно энергетического и нефтяного сектора, но, поскольку он не требовал выкуп от жертв, подробно здесь не рассматривается. Shamoon содержал компонент легитимного средства работы с файловой системой, который использовался для низкоуровневого доступа к жёсткому диску и перезаписи пользовательских файлов, включая сектор MBR, фрагментами собственных данных. Эта атака вызвала серьёзные сбои во многих целевых организациях. Одной из жертв, Saudi Aramco, потребовалась неделя для восстановления служб.

Образ действий программ-вымогателей

Прежде чем переходить к техническому анализу компонентов загрузчика Petya и Satana, рассмотрим на высоком уровне работу современных программ-вымогателей. У каждого семейства есть собственные особенности, немного отличающиеся от представленной здесь картины, но рисунок 13-1 отражает наиболее распространённую схему работы программы-вымогателя.

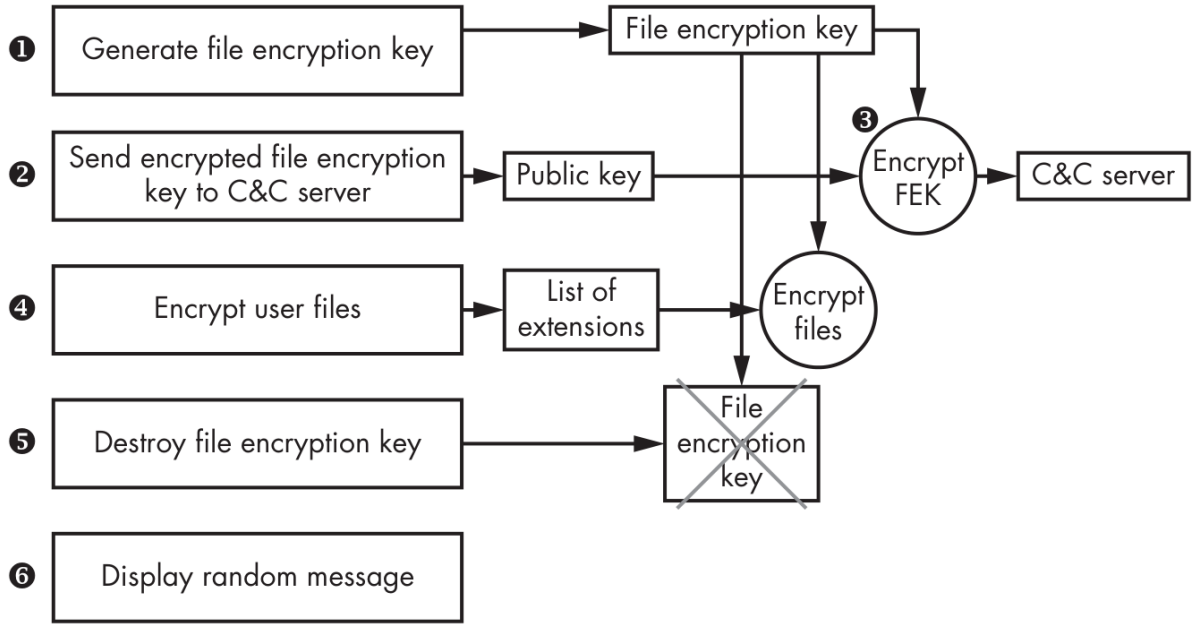


Рисунок 13-1. Образ действий современных программ-вымогателей

Вскоре после запуска в системе жертвы программа-вымогатель формирует уникальный ключ шифрования ❶ для симметричного шифра, то есть любого блочного или потокового шифра, например AES, RC4 или RC5. Этот ключ, который мы будем называть ключом шифрования файлов (File Encryption Key, FEK), используется для шифрования пользовательских файлов. Вредоносная программа применяет генератор (псевдо)случайных чисел, чтобы сформировать уникальный ключ, который невозможно угадать или предсказать.

После формирования ключа шифрования файлов он передаётся командному серверу ❷ для хранения. Чтобы предотвратить перехват программами наблюдения за сетевым трафиком, вредоносная программа шифрует ключ шифрования файлов встроенным в неё открытым ключом

❸, часто применяя алгоритмы RSA или ECC, как CTB-Locker и Petya. Закрытый ключ в теле вредоносной программы отсутствует и известен только злоумышленникам, поэтому никто другой не может получить доступ к ключу шифрования файлов.

После подтверждения командным сервером получения ключа шифрования файлов вредоносная программа приступает к шифрованию пользовательских файлов на жёстком диске ❹. Чтобы уменьшить объём подлежащих шифрованию файлов, программа-вымогатель использует встроенный список расширений, отфильтровывая несущественные файлы, например исполняемые и системные, и шифрует лишь определённые пользовательские файлы, которые, вероятнее всего, наиболее ценны для жертвы: документы, изображения и фотографии.

После шифрования вредоносная программа уничтожает ключ шифрования файлов в системе жертвы ❺, делая восстановление содержимого файлов без уплаты выкупа практически невозможным. Теперь ключ шифрования файлов обычно существует только на командном сервере злоумышленника, хотя в некоторых случаях его зашифрованная версия хранится в системе жертвы. Даже тогда, без знания закрытого ключа шифрования, пользователь практически не может восстановить ключ шифрования файлов и доступ к файлам.

Затем вредоносная программа выводит пользователю сообщение с требованием выкупа ❻ и инструкциями по его уплате. В одних случаях сообщение встроено в тело вредоносной программы, в других она получает страницу с требованием выкупа с командного сервера.

TorrentLocker: роковая ошибка

Не все ранние программы-вымогатели были настолько непроницаемыми из-за недостатков реализации процесса шифрования. Например, ранние версии TorrentLocker использовали для шифрования файлов шифр Advanced Encryption Standard (AES) в режиме счётчика. В режиме счётчика шифр AES формирует последовательность ключевых символов, которая затем объединяется по XOR с содержимым файла для его шифрования. Слабость этого подхода состоит в том, что при одинаковых ключе и начальном значении он выдаёт одинаковую ключевую последовательность независимо от содержимого файла. Чтобы восстановить ключевую последовательность, жертва может объединить по XOR зашифрованный файл с соответствующей исходной версией, а затем использовать полученную последовательность для расшифрования других файлов. После обнаружения этой особенности TorrentLocker была обновлена и стала применять шифр AES в режиме сцепления блоков шифротекста (Cipher Block Chaining, CBC), устранив слабость. В режиме CBC перед шифрованием блок открытого текста объединяется по XOR с блоком шифротекста из предыдущей итерации, поэтому даже небольшое различие входных данных приводит к значительному различию зашифрованного результата. Это делает описанный способ восстановления данных против TorrentLocker неэффективным.

Анализ программы-вымогателя Petya

В этом разделе мы сосредоточимся на техническом анализе функциональности Petya для шифрования жёсткого диска. Petya попадает на компьютер жертвы в виде вредоносного дроппера, который после запуска распаковывает полезную нагрузку с основной функциональностью программы-вымогателя, реализованной как файл DLL.

Получение прав администратора

Большинству программ-вымогателей права администратора не нужны, но Petya они требуются для прямой записи данных на жёсткий диск системы жертвы. Без этих прав Petya не смогла бы изменить содержимое MBR и установить вредоносный загрузчик. Исполняемый файл дроппера содержит манифест, указывающий, что его можно запустить только с правами администратора. В листинге 13-1 приведён фрагмент манифеста дроппера.

```
<trustInfo xmlns="urn:schemas-microsoft-com:asm.v2">
 <security>
  <requestedPrivileges>
❶ <requestedExecutionLevel level="requireAdministrator" uiAccess="false"/>
  </requestedPrivileges>
 </security>
</trustInfo>
```

Листинг 13-1. Фрагмент манифеста дроппера Petya

Раздел `security` содержит параметр `requestedExecutionLevel`, установленный в `requireAdministrator` ❶. Когда пользователь пытается выполнить дроппер, загрузчик ОС проверяет текущий уровень выполнения пользователя. Если он ниже `Administrator`, ОС выводит диалоговое окно с вопросом, хочет ли пользователь запустить программу с повышенными правами, если у его учётной записи есть административные права, либо запрашивает учётные данные администратора, если у учётной записи пользователя административных прав нет. Если пользователь отказывается предоставить приложению права администратора, дроппер не запускается и система не получает повреждений. Если пользователя удаётся заманить в запуск дроппера с правами администратора, вредоносная программа приступает к заражению системы.

Petya заражает систему в два этапа. На этапе 1 она собирает сведения о целевой системе, определяет вид разбиения жёсткого диска, формирует конфигурационные сведения - ключи шифрования и сообщение с требованием выкупа, - создаёт вредоносный загрузчик для этапа 2, затем заражает MBR компьютера вредоносным загрузчиком и инициирует перезагрузку системы.

После перезагрузки выполняется вредоносный загрузчик, запускающий второй этап заражения. Вредоносный загрузчик MBR шифрует секторы жёсткого диска, содержащие MFT, а затем ещё раз перезагружает компьютер. После второй перезагрузки вредоносный загрузчик выводит сообщение с требованием выкупа, сформированное на этапе 1.

В следующих разделах мы подробнее рассмотрим эти этапы.

Заражение жёсткого диска (этап 1)

Petya начинает заражение MBR с получения имени файла, представляющего физический жёсткий диск. В операционных системах Windows к жёсткому диску можно обратиться напрямую, выполнив API `CreateFile` и передав ему строку `'\\.\PhysicalDriveX'` как параметр имени файла, где `X` соответствует индексу жёсткого диска в системе. В системе с одним жёстким диском имя файла физического диска будет `'\\.\PhysicalDrive0'`. Однако при наличии нескольких жёстких дисков вредоносная программа использует индекс диска, с которого загружена система.

Petya получает его, отправляя тому NTFS с текущим экземпляром Windows специальный запрос `IOCTL_VOLUME_GET_VOLUME_DISK_EXTENTS`, который выполняется посредством API `DeviceIoControl`. Этот запрос возвращает массив структур, описывающих все жёсткие диски, используемые для размещения тома NTFS. Точнее, он возвращает массив экстентов тома NTFS. Экстент тома - непрерывная последовательность секторов на одном диске. Например, один том NTFS может размещаться на двух жёстких дисках; тогда запрос вернёт массив из двух экстентов. Разметка возвращаемых структур показана в листинге 13-2.

```
typedef struct _DISK_EXTENT {
  ❶ DWORD         DiskNumber;
  ❷ LARGE_INTEGER StartingOffset;
  ❸ LARGE_INTEGER ExtentLength;
} DISK_EXTENT, *PDISK_EXTENT;
```

Листинг 13-2. Разметка `DISK_EXTENT`

Поле `StartingOffset` ❷ описывает положение экстента тома на жёстком диске как смещение от начала диска в секторах, а `ExtentLength` ❸ задаёт его длину. Параметр `DiskNumber` ❶ содержит индекс соответствующего жёсткого диска в системе, также совпадающий с индексом в имени файла диска. Вредоносная программа использует поле `DiskNumber` самой первой структуры возвращённого массива экстентов тома, чтобы построить имя файла и обратиться к жёсткому диску.

Построив имя файла физического жёсткого диска, вредоносная программа определяет схему его разделов запросом `IOCTL_DISK_GET_PARTITION_INFO_EX`, отправленным жёсткому диску.

Petya способна заражать жёсткие диски как с разделами на основе MBR, так и с разделами таблицы разделов GUID (GUID Partition Table, GPT); разметка раздела GPT описана в главе 14. Сначала рассмотрим заражение Petya дисков на основе MBR, а затем особенности заражения дисков GPT.

Заражение жёсткого диска MBR

Чтобы заразить схему разделов MBR, Petya сначала читает MBR и вычисляет объём свободного дискового пространства между началом жёсткого диска и началом самого первого раздела. Это пространство используется для хранения вредоносного загрузчика и его конфигурационных сведений. Petya получает номер начального сектора самого первого раздела. Если он начинается с сектора, индекс которого меньше 60 (`0x3C`), места на жёстком диске недостаточно, поэтому Petya прекращает процесс заражения и завершает работу.

Если индекс равен 60 или больше, пространства достаточно, и вредоносная программа приступает к созданию вредоносного загрузчика из двух компонентов: вредоносного кода MBR и загрузчика второго этапа. На рисунке 13-2 показана разметка первых 57 секторов жёсткого диска после заражения.

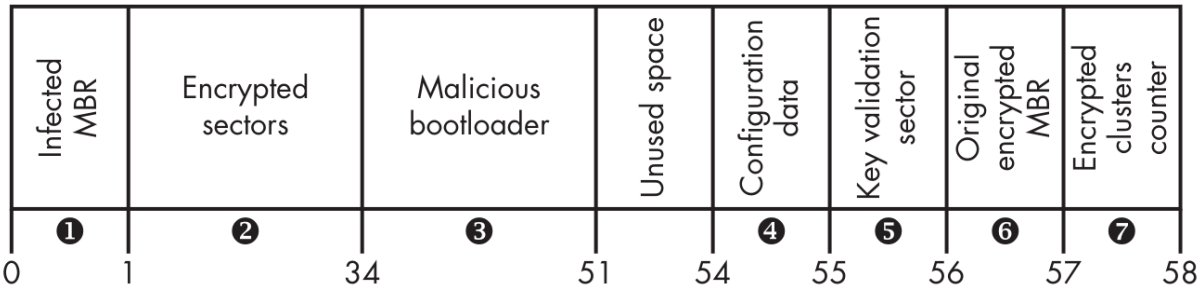


Рисунок 13-2. Разметка секторов жёсткого диска при заражении Petya для дисков MBR

Для создания вредоносной MBR Petya объединяет таблицу разделов исходной MBR с вредоносным кодом MBR и записывает результат в самый первый сектор жёсткого диска ❶ вместо исходной MBR. Исходная MBR объединяется по XOR с фиксированным байтовым значением `0x37`, и результат записывается в сектор 56 ❻.

Вредоносный загрузчик второго этапа занимает 17 последовательных секторов (`0x2E00` байт) дискового пространства и записывается на жёсткий диск в секторы с 34-го по 50-й ❸. Вредоносная программа также запутывает секторы с 1-го по 33-й ❷, объединяя их содержимое по XOR с фиксированным байтовым значением `0x37`.

Конфигурационные данные вредоносного загрузчика хранятся в секторе 54 ❹ и используются загрузчиком на этапе 2 процесса заражения. Мы подробно разберём структуру конфигурационных данных в разделе «Шифрование посредством конфигурационных данных вредоносного загрузчика» на странице 215.

Petya также использует сектор 55 ❺ для хранения 512-байтового буфера, заполненного байтовыми значениями `0x37`. Он будет использоваться для проверки предоставленного жертвой пароля и разблокировки жёсткого диска, как рассматривается в разделе «Вывод сообщения с требованием выкупа» на странице 224.

На этом заражение MBR завершено. Хотя на рисунке 13-2 сектор 57 ❼ помечен как «Счётчик зашифрованных кластеров», на данном этапе заражения он не используется. Код вредоносного загрузчика применит его на этапе 2 для хранения числа зашифрованных кластеров MFT.

Заражение жёсткого диска GPT

Процесс заражения жёсткого диска GPT похож на заражение диска MBR, но включает несколько дополнительных действий. Первое дополнительное действие шифрует резервную копию заголовка GPT, затрудняя восстановление системы. Заголовок GPT содержит сведения о разметке жёсткого диска GPT, а резервная копия позволяет системе восстановить его в случае повреждения или недействительности.

Чтобы найти резервный заголовок GPT, Petya читает сектор со смещением 1 от начала жёсткого диска, содержащий заголовок GPT, а затем обращается к полю со смещением резервной копии.

Определив местонахождение, Petya запутывает резервный заголовок GPT и 32 предшествующих ему сектора, объединяя их по XOR с фиксированной константой `0x37`, как показано на рисунке 13-3 ❶. Эти секторы содержат резервную GPT.

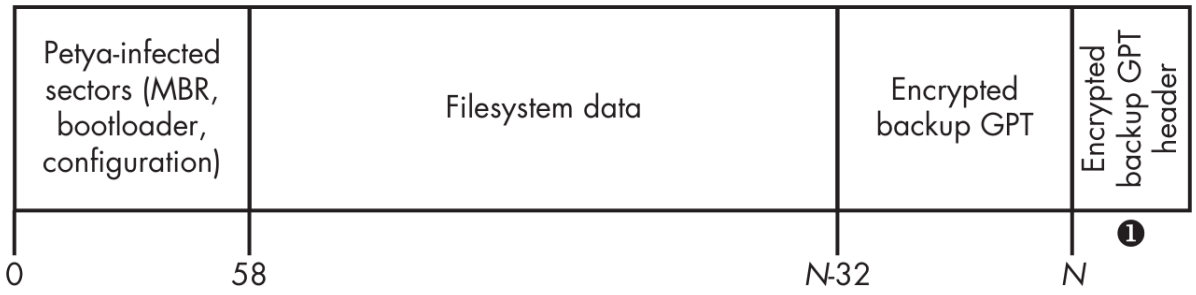


Рисунок 13-3. Разметка секторов жёсткого диска при заражении Petya для дисков GPT

Поскольку схема разделов GPT имеет иную разметку жёсткого диска, чем схема MBR, Petya не может просто повторно использовать таблицу разделов GPT без изменений для создания вредоносной MBR, как в случае диска MBR. Вместо этого она вручную создаёт в таблице разделов заражённой MBR элемент, представляющий весь жёсткий диск.

За исключением этих моментов заражение жёсткого диска GPT полностью совпадает с заражением дисков MBR. Однако важно отметить, что этот подход не работает в системах с включённой загрузкой UEFI. Как вы узнаете из главы 14, в процессе загрузки UEFI за загрузку системы отвечает код UEFI, а не код MBR. Если Petya выполняется в системе UEFI, она просто сделает систему незагружаемой, поскольку загрузчик UEFI не сможет прочитать зашифрованную GPT или её резервную копию, чтобы определить местонахождение загрузчика ОС.

Заражение Petya работает в гибридных системах, использующих устаревший загрузочный код BIOS и схему разделов GPT, например при включённом режиме поддержки совместимости BIOS, поскольку в таких системах сектор MBR по-прежнему хранит код системного загрузчика первого этапа, но изменённый для распознавания разделов GPT.

Шифрование посредством конфигурационных данных вредоносного загрузчика

Мы упоминали, что на этапе 1 процесса заражения Petya записывает конфигурационные данные загрузчика в сектор 54 жёсткого диска. Загрузчик использует их для завершения шифрования секторов жёсткого диска. Рассмотрим, как формируются эти данные.

Структура конфигурационных данных показана в листинге 13-3.

```
typedef struct _PETYA_CONFIGURATION_DATA {
  ❶ BYTE EncryptionStatus;
  ❷ BYTE SalsaKey[32];
  ❸ BYTE SalsaNonce[8];
  CHAR RansomURLs[128];

  BYTE RansomCode[343];
} PETYA_CONFIGURATION_DATA, *PPETYA_CONFIGURATION_DATA;
```

Листинг 13-3. Разметка конфигурационных данных Petya

Структура начинается с флага ❶, указывающего, зашифрована ли MFT жёсткого диска. На этапе 1 процесса заражения вредоносная программа сбрасывает этот флаг, поскольку MFT на данном этапе не шифруется. Вредоносный загрузчик устанавливает флаг на этапе 2, когда начинает шифрование MFT. За флагом следуют ключ шифрования ❷ и начальное значение (IV) ❸, используемые для шифрования MFT и рассматриваемые далее.

Формирование криптографических ключей

Для реализации криптографической функциональности Petya использует общедоступную библиотеку mbedtls («embedded TLS»), предназначенную для встраиваемых решений. Эта крошечная библиотека реализует широкий набор современных криптографических алгоритмов для симметричного и асимметричного шифрования данных, хеш-функции и многое другое. Её небольшой объём памяти идеально соответствует ограниченным ресурсам, доступным на этапе работы вредоносного загрузчика, когда выполняется шифрование MFT.

Одна из наиболее интересных особенностей Petya состоит в применении редкого шифра Salsa20 для шифрования MFT. Этот шифр формирует поток ключевых символов, объединяемый по XOR с открытым текстом для получения шифротекста, и принимает на вход 256-битный ключ и 64-битное начальное значение. Для алгоритма шифрования с открытым ключом Petya использует ECC. На рисунке 13-4 показан высокоуровневый процесс формирования криптографических ключей.

Чтобы сформировать ключ шифрования Salsa20, вредоносная программа сначала создаёт пароль - случайную 16-байтовую строку алфавитно-цифровых символов ❶. Затем Petya расширяет эту строку в 32-байтовый ключ Salsa20 ❷ посредством алгоритма из листинга 13-4; этот ключ шифрует содержимое секторов MFT на жёстком диске. Вредоносная программа также формирует 64-битное одноразовое значение, то есть начальное значение Salsa20, посредством генератора псевдослучайных чисел.

```
do
{
  config_data->salsa20_key[2 * i] = password[i] + 0x7A;
  config_data->salsa20_key[2 * i + 1] = 2 * password[i];
  ++i;
} while ( i < 0x10 );
```

Листинг 13-4. Расширение пароля в ключ шифрования Salsa20

Затем Petya формирует ключ выкупа как строку, которая будет показана на странице с требованием выкупа. Жертва должна передать этот ключ выкупа командному серверу, чтобы получить пароль для расшифрования MFT.

Формирование ключа выкупа

Только злоумышленник должен иметь возможность извлечь пароль из ключа выкупа, поэтому для его защиты Petya применяет схему шифрования ECC с открытым ключом, встроенным во вредоносную программу. Этот открытый ключ командного сервера мы будем называть `ecc_cc_public_key`.

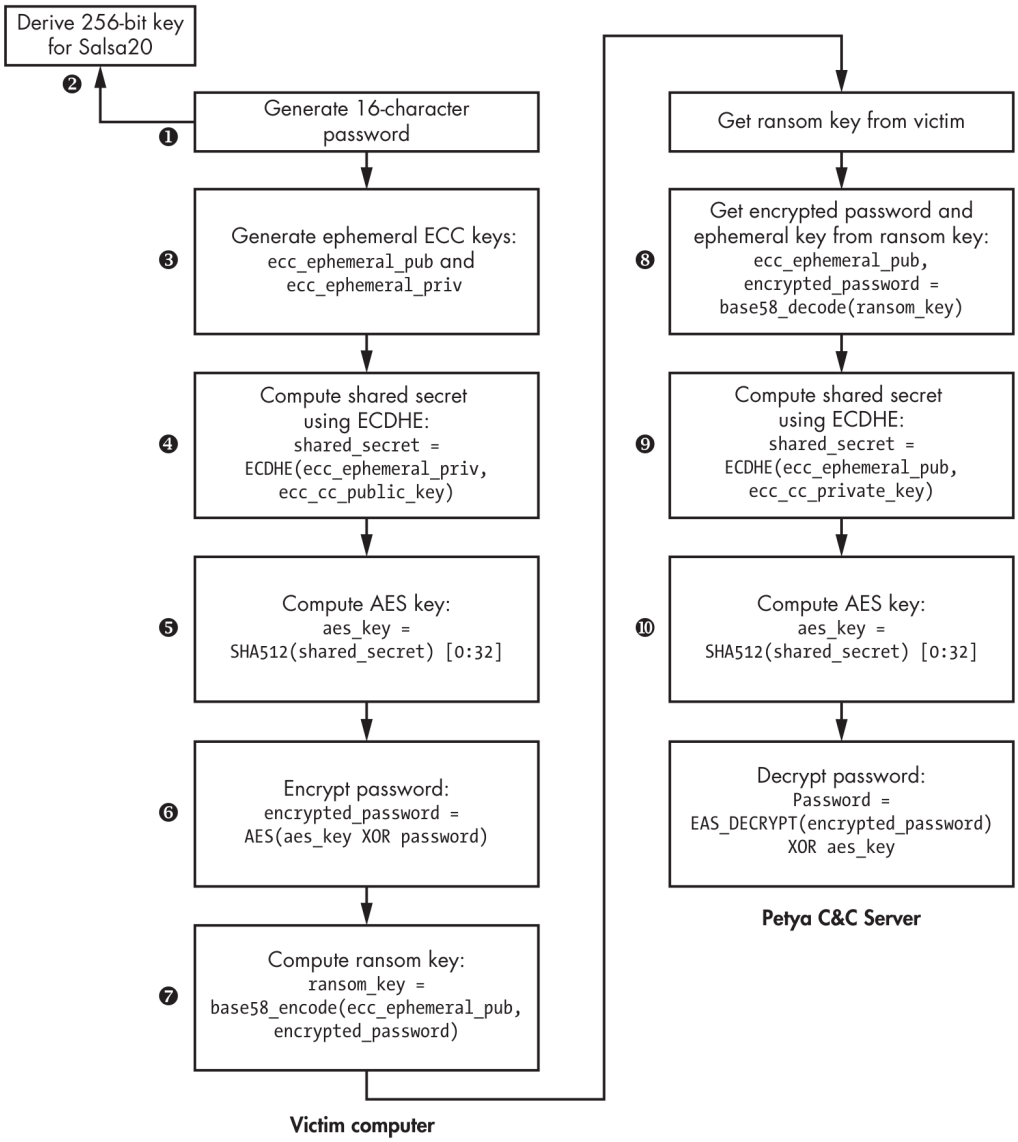


Рисунок 13-4. Формирование ключа шифрования

Сначала Petya формирует в системе жертвы временную пару ключей ECC ❸, называемую эфемерной, для установления защищённого взаимодействия с командным сервером: `ecc_ephemeral_pub` и `ecc_ephemeral_priv`.

Затем она формирует общий секрет, то есть общий ключ, посредством алгоритма согласования ключей Диффи-Хеллмана на эллиптических кривых ❹. Этот алгоритм позволяет двум сторонам разделить секрет, известный только им; прослушивающий обмен противник не сможет его определить. На компьютере жертвы общий секрет вычисляется как `shared_secret = ECDHE(ecc_ephemeral_priv, ecc_cc_public_key)`, где ECDHE - процедура согласования ключей Диффи-Хеллмана. Она принимает два параметра: закрытый эфемерный ключ жертвы и открытый ключ командного сервера, встроенный во вредоносную программу. Такой же секрет вычисляется злоумышленником как `shared_secret = ECDHE(ecc_ephemeral_pub, ecc_cc_private_key)`, где используются его собственный закрытый ключ командного сервера и открытый эфемерный ключ жертвы.

Получив `shared_secret`, вредоносная программа вычисляет его хеш алгоритмом SHA512 и использует первые 32 байта хеша как ключ AES ❺: `aes_key = SHA512(shared_secret)[0:32]`.

Затем она шифрует пароль ❻ только что полученным `aes_key` следующим образом: `encrypted_password = AES(aes_key XOR password)`. Как видно, перед шифрованием пароля вредоносная программа объединяет его по XOR с ключом AES.

Наконец, Petya кодирует эфемерный открытый ключ и зашифрованный пароль алгоритмом base58, получая строку ASCII, используемую как ключ выкупа ❼: `ransom_key = base58_encode(ecc_ephemeral_pub, encrypted_password)`.

Проверка ключа выкупа

Если пользователь платит выкуп, злоумышленник предоставляет пароль для расшифрования данных. Рассмотрим, как злоумышленник проверяет ключ выкупа, чтобы восстановить пароль жертвы.

Получив ключ выкупа от жертвы, Petya декодирует его алгоритмом base58 и получает открытый эфемерный ключ жертвы и зашифрованный пароль: `ecc_ephemeral_pub, encrypted_password = base58_decode(ransom_key)` ❽.

Затем злоумышленник вычисляет общий секрет по протоколу согласования ключей ECDHE, описанному в предыдущем разделе: `shared_secret = ECDHE(ecc_ephemeral_pub, ecc_cc_private_key)` ❾.

Имея общий секрет, злоумышленник может получить ключ шифрования AES, вычислив хеш SHA512 общего секрета так же, как раньше: `aes_key = SHA512(shared_secret)[0:32]` ❿.

После вычисления ключа AES злоумышленник может расшифровать пароль и получить пароль жертвы как `password = AES_DECRYPT(encrypted_password) XOR aes_key`.

Теперь злоумышленник получил пароль жертвы из ключа выкупа; никто другой не может сделать этого без закрытого ключа злоумышленника.

Формирование URL для уплаты выкупа

Последним элементом конфигурационных сведений для второго этапа загрузчика Petya формирует URL для уплаты выкупа. Они выводятся в сообщении с требованием выкупа, объясняющем жертве, как заплатить выкуп и восстановить данные системы. Вредоносная программа случайным образом формирует алфавитно-цифровой идентификатор жертвы, а затем объединяет его с вредоносным доменным именем, получая URL вида `http://<malicious_domain>/<victim_id>`. На рисунке 13-5 показана пара примеров URL.

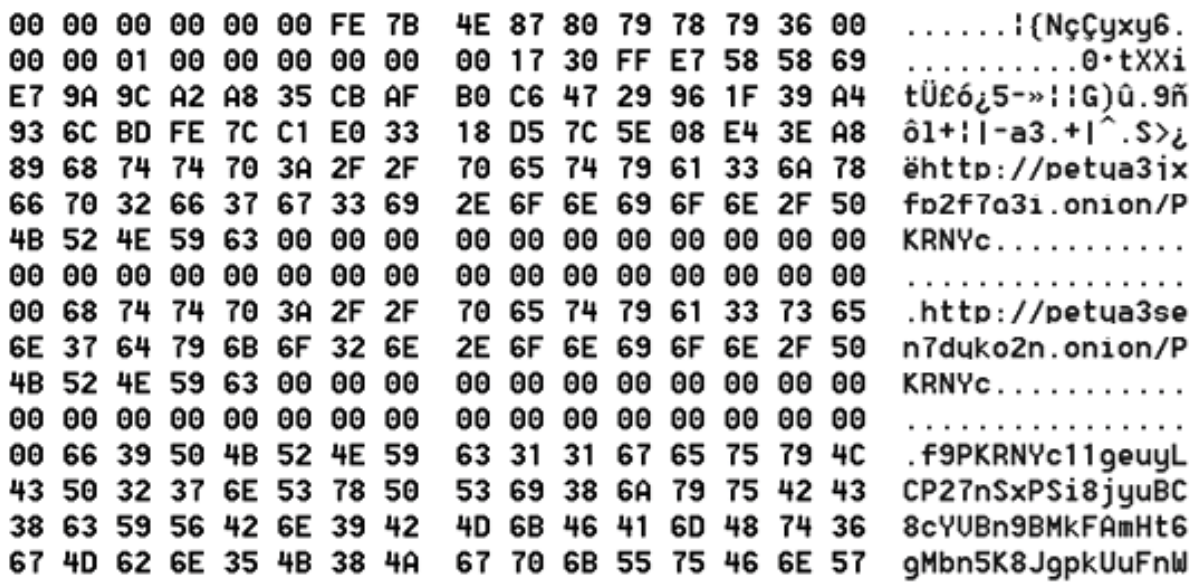


Рисунок 13-5. Конфигурационные данные Petya с URL для уплаты выкупа

Видно, что имя домена верхнего уровня - `.onion`, а значит, вредоносная программа использует TOR для формирования URL.

Аварийное завершение работы системы

После записи вредоносного загрузчика и его конфигурационных данных на жёсткий диск Petya вызывает аварийное завершение работы системы и принудительную перезагрузку, чтобы выполнить вредоносный загрузчик и завершить заражение системы. В листинге 13-5 показано, как это делается.

```
void __cdecl RebootSystem()
{
  hProcess = GetCurrentProcess();
  if ( OpenProcessToken(hProcess, 0x28u, &TokenHandle) )
  {
    LookupPrivilegeValueA(0, "SeShutdownPrivilege", NewState.Privileges);
    NewState.PrivilegeCount = 1;
    NewState.Privileges[0].Attributes = 2;
    ❶ AdjustTokenPrivileges(TokenHandle, 0, &NewState, 0, 0, 0);
    if ( !GetLastError() )
    {
      v1 = GetModuleHandleA("NTDLL.DLL");
      NtRaiseHardError = GetProcAddress(v1, "NtRaiseHardError");
      ❷ (NtRaiseHardError)(0xC0000350, 0, 0, 0, 6, &v4);
    }
  }
}
```

Листинг 13-5. Процедура Petya для принудительной перезагрузки системы

Petya выполняет системную процедуру API `NtRaiseHardError` ❷, чтобы аварийно завершить работу системы; эта процедура уведомляет систему о серьёзной ошибке, препятствующей нормальной работе и требующей перезагрузки во избежание потери или повреждения данных.

Для выполнения этой процедуры вызывающему процессу нужны права `SeShutdownPrivilege`, которые легко получить, поскольку Petya запускается с правами учётной записи администратора. Как показано в листинге 13-5, перед выполнением `NtRaiseHardError` Petya изменяет текущие права вызовом `AdjustTokenPrivileges` ❶.

Шифрование MFT (этап 2)

Теперь сосредоточимся на втором этапе заражения. Загрузчик состоит из двух компонентов: вредоносной MBR и загрузчика второго этапа, который в этом разделе мы будем называть вредоносным загрузчиком. Единственное назначение вредоносного кода MBR - загрузить загрузчик второго этапа в память и выполнить его, поэтому анализ вредоносной MBR мы пропустим. Именно загрузчик второго этапа реализует наиболее интересную функциональность программы-вымогателя.

Поиск доступных дисков

Получив управление, загрузчик должен собрать сведения о доступных дисках системы. Для этого он использует хорошо известную службу INT 13h, как показано в листинге 13-6.

```
❶ mov     dl, [bp+disk_no]
❷ mov     ah, 8
int     13h
```

Листинг 13-6. Использование INT 13h для проверки наличия дисков в системе

Чтобы проверить наличие и размер жёстких дисков, вредоносная программа сохраняет номера индексов в регистре `dl` ❶, а затем выполняет INT 13h. Номера индексов назначаются дискам последовательно, поэтому Petya ищет жёсткие диски системы, проверяя индексы от 0 до 15. Затем она помещает значение 8 в регистр `ah` ❷, что обозначает функцию INT 13h «получить текущие параметры диска». После этого вредоносная программа выполняет INT 13h. Если после выполнения `ah` равен 0, указанный диск присутствует в системе, а регистры `dx` и `cx` содержат сведения о его размере. Если регистр `ah` не равен 0, диска с данным индексом в системе нет.

Затем вредоносный загрузчик читает конфигурационные данные из сектора 54 и проверяет шифрование MFT жёстких дисков по самому первому байту прочитанного буфера, соответствующему полю `EncryptionStatus` конфигурационных данных. Если флаг сброшен, то есть содержимое MFT не зашифровано, вредоносная программа приступает к шифрованию MFT доступных в системе жёстких дисков, завершая процесс заражения. Если MFT уже зашифрована, вредоносный загрузчик выводит жертве сообщение с требованием выкупа. Мы вскоре обсудим это сообщение, но сначала рассмотрим, как вредоносный загрузчик выполняет шифрование.

Шифрование MFT

Если флаг `EncryptionStatus` конфигурационных данных сброшен, то есть равен 0, вредоносная программа читает ключ шифрования Salsa20 и IV соответственно из параметров `SalsaKey` и `SalsaNonce` и использует их для шифрования данных жёсткого диска. Затем загрузчик устанавливает флаг `EncryptionStatus` и уничтожает `SalsaKey` в конфигурационных данных сектора 54, чтобы предотвратить расшифрование данных.

После этого загрузчик читает сектор 55 заражённого жёсткого диска, который позднее будет использоваться для проверки введённого жертвой пароля. В этот момент сектор заполнен байтами `0x37`. Petya шифрует его алгоритмом Salsa20 с ключом и IV из конфигурационных данных, а затем записывает результат обратно в сектор 55.

Теперь вредоносный загрузчик готов зашифровать MFT жёстких дисков системы. Шифрование значительно увеличивает продолжительность загрузки, поэтому, чтобы не вызвать подозрений, Petya выводит поддельное сообщение `chkdsk`, показанное на рисунке 13-6. Системная утилита `chkdsk` используется для исправления файловых систем на жёстком диске, и после аварийного завершения работы системы такое сообщение выглядит обычно. Пока на экране отображается поддельное сообщение, вредоносная программа выполняет следующий алгоритм для каждого доступного жёсткого диска системы.

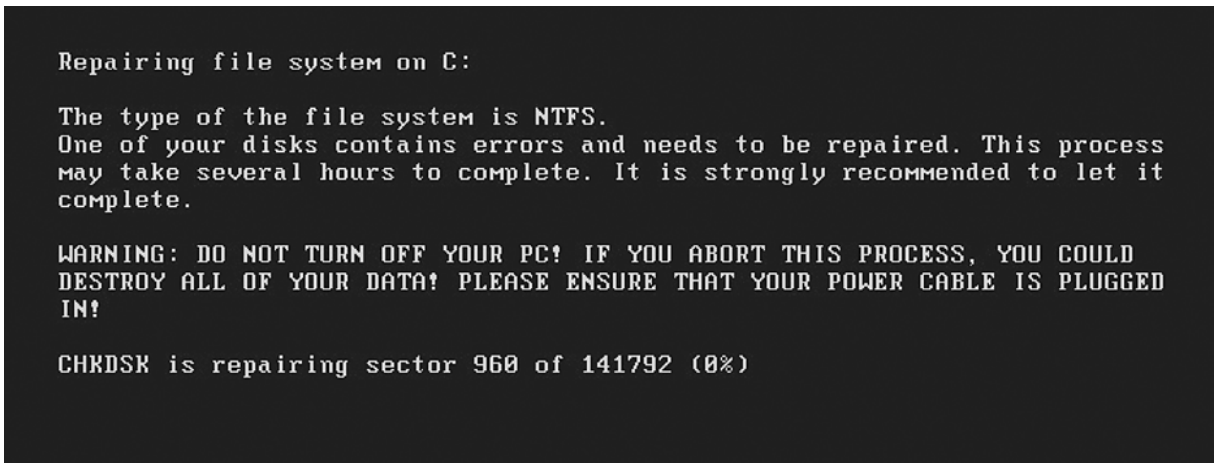


Рисунок 13-6. Поддельное сообщение `chkdsk`

Сначала вредоносная программа читает MBR жёсткого диска и перебирает таблицу разделов MBR в поисках доступных разделов. Она проверяет параметр, описывающий тип файловой системы раздела, и пропускает все разделы, значение типа которых отличается от `0x07`, означающего раздел с томом NTFS, `0xEE` и `0xEF`, означающих разметку GPT жёсткого диска. Если жёсткий диск действительно имеет разметку GPT, вредоносный загрузочный код получает местонахождение раздела из таблицы разделов GPT.

Разбор таблицы разделов GPT

В случае таблиц разделов GPT вредоносная программа выполняет дополнительное действие для поиска разделов на жёстком диске: читает таблицу разделов GPT с диска, начиная с третьего сектора. Каждый элемент таблицы разделов GPT имеет длину 128 байт и структуру, показанную в листинге 13-7.

```
typedef struct _GPT_PARTITION_TABLE_ENTRY {
  BYTE PartitionTypeGuid[16];
  BYTE PartitionUniqueGuid[16];
  QWORD PartitionStartLba;
  QWORD PartitionLastLba;
  QWORD PartitionAttributes;
  BYTE PartitionName[72];
} GPT_PARTITION_TABLE_ENTRY, *PGPT_PARTITION_TABLE_ENTRY;
```

Листинг 13-7. Разметка элемента таблицы разделов GPT

Самое первое поле, `PartitionTypeGuid`, - массив из 16 байт с идентификатором типа раздела, определяющим вид данных, для хранения которых предназначен раздел. Вредоносный загрузочный код проверяет это поле и отфильтровывает все элементы разделов, кроме тех, где `PartitionTypeGuid` равен `{EBD0A0A2-B9E5-4433-87C0-68B6B72699C7}`. Этот тип известен как основной раздел данных операционной системы Windows и используется для хранения томов NTFS. Именно они интересуют вредоносную программу.

Если вредоносный загрузочный код обнаруживает основной раздел данных, он читает поля `PartitionStartLba` и `PartitionLastLba`, содержащие соответственно адреса самого первого и последнего секторов раздела, чтобы определить положение целевого раздела на жёстком диске. Получив координаты раздела, загрузочный код Petya переходит к следующему этапу.

Поиск MFT

Чтобы найти MFT, вредоносная программа читает VBR выбранных разделов с жёсткого диска; разметка VBR подробно описана в главе 5. Параметры файловой системы описываются блоком параметров BIOS (BPB), структура которого показана в листинге 13-8.

```
typedef struct _BIOS_PARAMETER_BLOCK_NTFS {
  WORD SectorSize;
  ❶ BYTE SectorsPerCluster;
  WORD ReservedSectors;
  BYTE Reserved[5];
  BYTE MediaId;
  BYTE Reserved2[2];
  WORD SectorsPerTrack;
  WORD NumberOfHeads;
  DWORD HiddenSectors;
  BYTE Reserved3[8];
  QWORD NumberOfSectors;
  ❷ QWORD MFTStartingCluster;
  QWORD MFTMirrorStartingCluster;
  BYTE ClusterPerFileRecord;
  BYTE Reserved4[3];
  BYTE ClusterPerIndexBuffer;
  BYTE Reserved5[3];
  QWORD NTFSSerial;

  BYTE Reserved6[4];
} BIOS_PARAMETER_BLOCK_NTFS, *PBIOS_PARAMETER_BLOCK_NTFS;
```

Листинг 13-8. Разметка блока параметров BIOS в VBR

Вредоносный загрузочный код проверяет `MFTStartingCluster` ❷, задающий положение MFT как смещение от начала раздела в кластерах. Кластер - минимальная адресуемая единица хранения в файловой системе. Размер кластера может различаться от системы к системе и задаётся полем `SectorsPerCluster` ❶, которое также проверяет вредоносная программа. Например, наиболее типичное значение этого поля для NTFS - 8, что соответствует 4096 байтам при размере сектора 512 байт. Посредством этих двух полей Petya вычисляет смещение MFT от начала раздела.

Разбор MFT

MFT организована как массив элементов, каждый из которых описывает определённый файл или каталог. Мы не будем углубляться в формат MFT, поскольку он настолько сложен, что заслуживает по меньшей мере отдельной главы. Вместо этого приведём лишь сведения, необходимые для понимания вредоносного загрузчика Petya.

На данном этапе у вредоносной программы есть начальный адрес MFT из `MFTStartingCluster`, но для получения точных положений Petya также должна знать размер MFT. Более того, MFT может храниться на жёстком диске не как непрерывная последовательность секторов, а быть разделена на небольшие последовательности секторов, разбросанные по диску. Чтобы получить сведения о точном расположении MFT, вредоносный код читает и разбирает специальный файл метаданных `$MFT`, находящийся среди файлов метаданных NTFS, соответствующих первым 16 записям MFT.

Каждый из этих файлов содержит важнейшие сведения, обеспечивающие правильную работу файловой системы:

• `$MFT` - ссылка MFT на саму себя, содержащая сведения о размере и расположении MFT на жёстком диске;
• `$MFTMirr` - зеркало MFT с копиями первых 16 записей;
• `$LogFile` - файл журнала тома с данными транзакций;
• `$BadClus` - список всех повреждённых кластеров тома, помеченных как «плохие».

Как видно, самый первый файл метаданных, `$MFT`, содержит все сведения, необходимые для определения точного расположения MFT на жёстком диске. Вредоносный код разбирает этот файл, получает местонахождение непрерывных последовательностей секторов, а затем шифрует их шифром Salsa20.

После шифрования всех MFT на присутствующих в системе жёстких дисках процесс заражения завершён, и вредоносная программа выполняет INT 19h, чтобы начать процесс загрузки сначала. Этот обработчик прерывания заставляет загрузочный код BIOS загрузить MBR загрузочного жёсткого диска в память и выполнить её код. На этот раз, когда вредоносный загрузочный код читает конфигурационные сведения из сектора 54, флаг `EncryptionStatus` установлен в 1, показывая, что шифрование MFT завершено, и вредоносная программа переходит к выводу сообщения с требованием выкупа.

Вывод сообщения с требованием выкупа

Сообщение с требованием выкупа, выводимое загрузочным кодом, показано на рисунке 13-7.

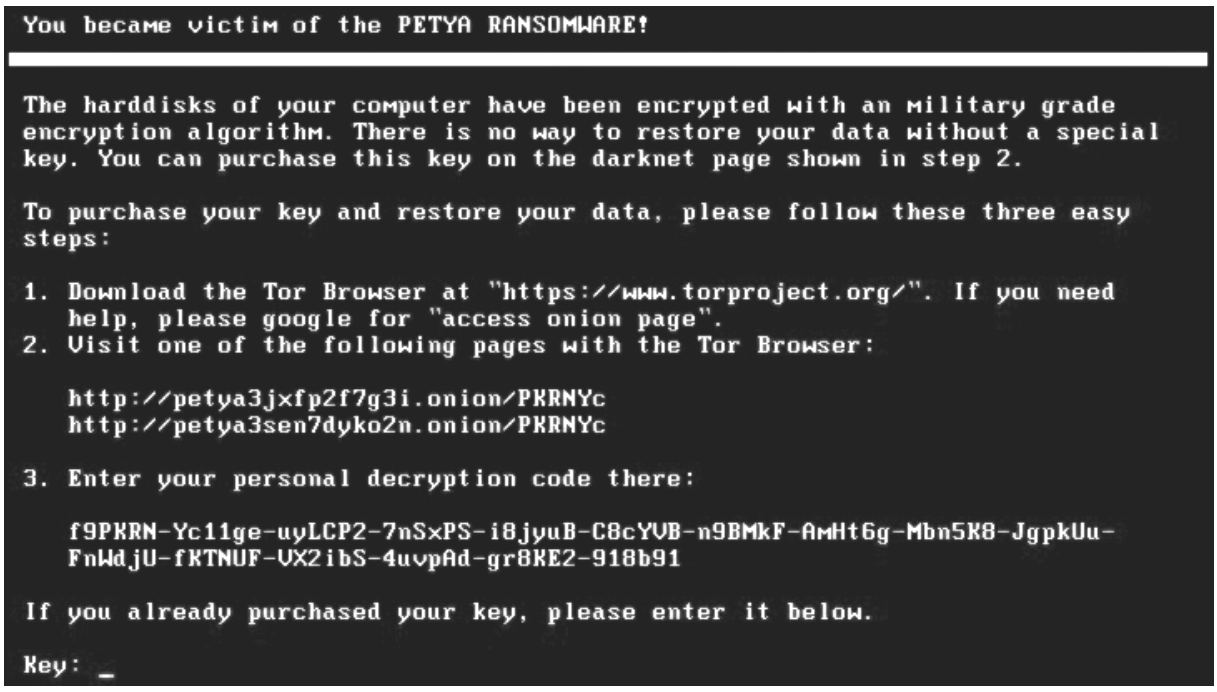


Рисунок 13-7. Сообщение Petya с требованием выкупа

Сообщение уведомляет жертву, что её система скомпрометирована программой-вымогателем Petya, а жёсткий диск зашифрован алгоритмом шифрования военного уровня. Затем оно содержит инструкции по разблокировке данных. Видно список URL, сформированных Petya на первом этапе заражения. Страницы по этим URL содержат дальнейшие инструкции для жертвы. Вредоносная программа также выводит код выкупа, который пользователь должен ввести, чтобы получить пароль для расшифрования.

Вредоносная программа формирует ключ Salsa20 из пароля, введённого на странице уплаты выкупа, и пытается расшифровать сектор 55, используемый для проверки ключа. Если пароль верен, результатом расшифрования сектора 55 становится буфер, заполненный байтами `0x37`. В этом случае программа-вымогатель принимает пароль, расшифровывает MFT и восстанавливает исходную MBR. Если пароль неверен, вредоносная программа выводит сообщение `"Incorrect key! Please try again."`

Подведение итогов: заключительные мысли о Petya

На этом обсуждение процесса заражения Petya завершено, но остаётся несколько заключительных замечаний об интересных особенностях её подхода.

Во-первых, в отличие от других программ-вымогателей, шифрующих пользовательские файлы, Petya работает с жёстким диском в низкоуровневом режиме, читая и записывая необработанные данные, и потому требует прав администратора. Однако она не эксплуатирует уязвимости локального повышения прав (LPE), а полагается на встроенные во вредоносную программу сведения манифеста, как обсуждалось ранее в этой главе. Таким образом, если пользователь отказывается предоставить приложению права администратора, вредоносная программа не запускается из-за требований манифеста. Даже если бы она выполнилась без административных прав, Petya не смогла бы открыть дескриптор устройства жёсткого диска и потому не причинила бы вреда. В этом случае процедура `CreateFile`, посредством которой Petya получает дескриптор жёсткого диска, вернула бы значение `INVALID_HANDLE`, что привело бы к ошибке.

Чтобы обойти это ограничение, Petya часто распространялась вместе с другой программой-вымогателем - Mischa. Mischa - обычная программа-вымогатель, шифрующая пользовательские файлы, а не жёсткий диск, и не требующая административных прав доступа к системе. Если Petya не удавалось получить права администратора, вредоносный дроппер вместо неё выполнял Mischa. Обсуждение Mischa выходит за рамки этой главы.

Во-вторых, как уже обсуждалось, вместо шифрования содержимого файлов на жёстком диске Petya шифрует метаданные MFT, чтобы файловая система не могла получить сведения о расположении и атрибутах файлов. Поэтому, хотя содержимое файлов не зашифровано, жертвы всё равно не могут получить к ним доступ. Это означает, что содержимое файлов потенциально можно восстановить средствами и методами восстановления данных. Такие средства часто используются при криминалистическом анализе для восстановления сведений из повреждённых образов.

Наконец, как вы, возможно, уже поняли, Petya - довольно сложная вредоносная программа, написанная опытными разработчиками. Реализованная функциональность говорит о глубоком понимании файловых систем и загрузчиков. Эта вредоносная программа отмечает очередной этап развития программ-вымогателей.

Анализ программы-вымогателя Satana

Теперь рассмотрим другой пример программы-вымогателя, атакующей процесс загрузки, - Satana. В отличие от Petya, заражающей только MBR жёсткого диска, Satana также шифрует файлы жертвы.

Более того, MBR не является основным вектором заражения Satana. Мы покажем, что вредоносный код загрузчика, записываемый вместо исходной MBR, содержит недостатки и, вероятно, находился в разработке во время распространения Satana.

В этом разделе мы сосредоточимся только на функциональности заражения MBR, поскольку шифрование файлов в пользовательском режиме выходит за рамки главы.

Дроппер Satana

Начнём с дроппера Satana. После распаковки в памяти вредоносная программа копирует себя в файл со случайным именем в каталоге `TEMP` и выполняет его. Для заражения MBR Satana требуются права администратора, и, как Petya, она не эксплуатирует уязвимости LPE для повышения прав. Вместо этого она проверяет уровень прав своего процесса процедурой API `setupapi!IsUserAdmin`, которая, в свою очередь, проверяет, является ли маркер безопасности текущего процесса членом группы администраторов. Если у дроппера нет прав для заражения системы, он выполняет копию в каталоге `TEMP` и пытается запустить вредоносную программу под учётной записью администратора посредством процедуры API `ShellExecute` с параметром `runas`. При этом выводится сообщение с просьбой предоставить приложению права администратора. Если пользователь выбирает «Нет», вредоносная программа снова и снова вызывает `ShellExecute` с теми же параметрами, пока пользователь не выберет «Да» или не завершит вредоносный процесс.

Заражение MBR

Получив права администратора, Satana приступает к заражению жёсткого диска. В ходе заражения вредоносная программа извлекает несколько компонентов из образа дроппера и записывает их на жёсткий диск. На рисунке 13-8 показана разметка первых секторов жёсткого диска, заражённого Satana. В этом разделе мы подробно опишем каждый элемент заражения MBR. Для упрощения объяснения предполагается, что нумерация секторов начинается с 0.

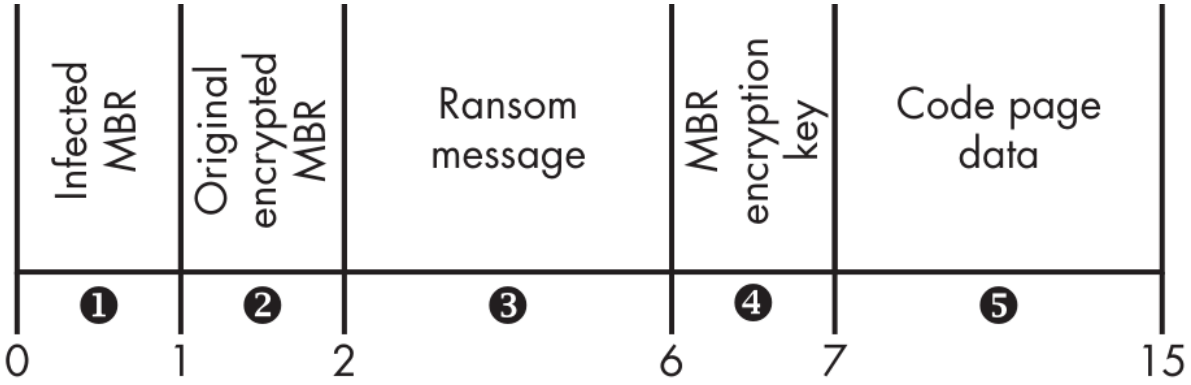


Рисунок 13-8. Разметка жёсткого диска при заражении Satana

Для низкоуровневого доступа к жёсткому диску вредоносная программа использует те же API, что и Petya: `CreateFile`, `DeviceIoControl`, `WriteFile` и `SetFilePointer`. Чтобы открыть дескриптор файла, представляющего жёсткий диск, Satana вызывает процедуру `CreateFile` со строкой `'\\.\PhysicalDrive0'` в качестве аргумента `FileName`. Затем дроппер выполняет процедуру `DeviceIoControl` с параметром `IOCTL_DISK_GET_DRIVE_GEOMETRY`, чтобы получить параметры жёсткого диска, например общее число секторов и размер сектора в байтах.

> **Примечание.** Метод получения дескриптора жёсткого диска посредством `'\\.\PhysicalDrive0'` надёжен не на 100 процентов, поскольку предполагает, что загрузочный жёсткий диск всегда имеет индекс 0. Хотя это верно для большинства систем, гарантии нет. В этом отношении Petya осторожнее: она динамически определяет индекс текущего жёсткого диска во время заражения, тогда как Satana использует жёстко заданное значение.

Прежде чем заражать MBR, Satana убеждается, что между MBR и первым разделом жёсткого диска достаточно свободного места для хранения компонентов вредоносного загрузчика. Для этого она перечисляет разделы, находит первый раздел и его начальный сектор. Если между MBR и первым разделом меньше 15 секторов, Satana прекращает заражение и продолжает шифровать пользовательские файлы. В противном случае она пытается заразить MBR.

Сначала Satana должна записать буфер со сведениями о пользовательском шрифте в секторы, начинающиеся с сектора 7 ❺. Буфер может занимать до восьми секторов жёсткого диска. Записанные в эти секторы сведения предназначены для вывода вредоносным загрузчиком сообщения с требованием выкупа на языке, отличном от используемого по умолчанию английского. Однако в проанализированных нами образцах Satana эта возможность не применялась. Вредоносная программа ничего не записывала в сектор 7 и потому выводила сообщение с требованием выкупа на английском языке по умолчанию.

Сообщение с требованием выкупа, которое будет показано пользователю при загрузке, Satana записывает в секторы со 2-го по 5-й ❸ открытым текстом без шифрования.

Затем вредоносная программа читает исходную MBR из самого первого сектора и шифрует её посредством XOR с 512-байтовым ключом, сформированным на этапе заражения генератором псевдослучайных чисел. Satana заполняет буфер из 512 байт случайными данными и объединяет каждый байт MBR по XOR с соответствующим байтом буфера ключа. Зашифровав MBR, вредоносная программа сохраняет ключ шифрования в секторе 6 ❹, а зашифрованную исходную MBR - в секторе 1 ❷ жёсткого диска.

Наконец, вредоносная программа записывает вредоносную MBR в самый первый сектор жёсткого диска ❶. Перед перезаписью MBR Satana шифрует заражённую MBR посредством XOR со случайно сформированным байтовым значением и записывает ключ в конец заражённой MBR, чтобы вредоносный код MBR мог расшифровать себя при загрузке системы.

На этом заражение MBR завершено, и Satana продолжает шифровать пользовательские файлы. Чтобы запустить выполнение вредоносной MBR, вскоре после шифрования пользовательских файлов Satana перезагружает компьютер.

Отладочные сведения дроппера

Прежде чем продолжить анализ вредоносного кода MBR, отметим особенно интересную особенность дроппера. Проанализированные нами образцы Satana содержали множество подробных отладочных сведений, документировавших реализованный в дроппере код, подобно тому, что мы обнаружили в трояне Carberp, рассмотренном в главе 11.

Наличие в дроппере отладочных сведений укрепляет предположение, что во время нашего анализа Satana находилась в разработке. Satana использует API `OutputDebugString` для вывода отладочных сообщений, которые можно увидеть в отладчике или других средствах, перехватывающих отладочный вывод. В листинге 13-9 приведён фрагмент отладочной трассы вредоносной программы, перехваченной средством DebugMonitor.

```
00000042 ❶ 27.19946671 [2760] Engine: Try to open drive \\.\PHYSICALDRIVE0
00000043    27.19972229 [2760] Engine: \\.\PHYSICALDRIVE0 opened
00000044 ❷ 27.21799088 [2760] Total sectors:83875365
00000045    27.21813583 [2760] SectorSize: 512
00000046    27.21813583 [2760] ZeroSecNum:15
00000047    27.21813583 [2760] FirstZero:2
00000048    27.21813583 [2760] LastZero:15
00000049 ❸ 27.21823502 [2760] XOR key=0x91
00000050    27.21839333 [2760] Message len: 1719
00000051 ❹ 27.21941948 [2760] Message written to Disk
00000052    27.22294235 [2760] Try write MBR to Disk: 0

00000053 ❺ 27.22335243 [2760] Random sector written
00000054    27.22373199 [2760] DAY: 2
00000055 ❻ 27.22402954 [2760] MBR written to Disk# 0
```

Листинг 13-9. Отладочный вывод дроппера Satana

В этом выводе видно, что вредоносная программа пытается обратиться к `'\\.\PhysicalDrive0'` ❶, чтобы читать секторы жёсткого диска и записывать их. В точке ❷ Satana получает параметры жёсткого диска: размер и общее число секторов. В точке ❹ она записывает на жёсткий диск сообщение с требованием выкупа, а перед этим формирует ключ для шифрования заражённой MBR ❸. Она сохраняет ключ шифрования ❺, а затем перезаписывает MBR заражённым кодом ❻. Эти сообщения раскрывают функциональность вредоносной программы без многочасового реверс-инжиниринга.

Вредоносная MBR Satana

Вредоносный загрузчик Satana относительно мал и прост по сравнению с загрузчиком Petya. Вредоносный код содержится в одном секторе и реализует функциональность вывода сообщения с требованием выкупа.

После загрузки системы вредоносный код MBR расшифровывает себя, читая ключ расшифрования из конца секторов MBR и объединяя зашифрованный код MBR по XOR с ключом. В листинге 13-10 показан код дешифратора вредоносной MBR.

```
seg000:0000    pushad
seg000:0002    cld
seg000:0003 ❶ mov     si, 7C00h
seg000:0006    mov     di, 600h
seg000:0009    mov     cx, 200h
seg000:000C ❷ rep movsb
seg000:000E    mov     bx, 7C2Ch
seg000:0011    sub     bx, 7C00h
seg000:0015    add     bx, 600h
seg000:0019    mov     cx, bx
seg000:001B decr_loop:
seg000:001B    mov     al, [bx]
seg000:001D ❸ xor     al, byte ptr ds:xor_key
seg000:0021    mov     [bx], al
seg000:0023    inc     bx
seg000:0024    cmp     bx, 7FBh
seg000:0028    jnz     short loc_1B
seg000:002A ❹ jmp     cx
```

Листинг 13-10. Дешифратор вредоносной MBR Satana

Сначала дешифратор инициализирует регистры `si`, `di` и `cx` ❶, чтобы скопировать зашифрованный код MBR в другое место памяти, а затем расшифровывает скопированный код посредством XOR с байтовым значением ❸. После расшифрования инструкция в точке ❹ передаёт поток выполнения расшифрованному коду по адресу в `cx`.

Если внимательно посмотреть на строку, копирующую зашифрованный код MBR в другое место памяти, можно заметить ошибку. Копирование выполняется инструкцией `rep movsb` ❷, которая копирует число байтов, заданное регистром `cx`, из исходного буфера с адресом в `ds:si` в буфер назначения с адресом в регистрах `es:di`. Однако сегментные регистры `ds` и `es` в коде MBR не инициализируются. Вместо этого вредоносная программа предполагает, что регистр `ds`, сегмент данных, имеет точно такое же значение, как регистр `cs`, сегмент кода, то есть что `ds:si` должен преобразовываться в `cs:7c00h`, соответствующий адресу MBR в памяти.

Однако это верно не всегда: регистр `ds` может содержать другое значение. Тогда вредоносная программа попытается скопировать неверные байты из памяти по адресу `ds:si`, совершенно отличающемуся от положения MBR. Чтобы исправить ошибку, регистры `ds` и `es` необходимо инициализировать значением регистра `cs`, то есть `0x0000`, поскольку MBR загружена по адресу `0000:7c00h` и регистр `cs` содержит `0x0000`.

> **Среда выполнения до MBR**
>
> Самый первый код, выполняемый после выхода центрального процессора из сброса, - не код MBR, а код BIOS, осуществляющий базовую инициализацию системы. Содержимое сегментных регистров `cs`, `ds`, `es`, `ss` и других инициализируется BIOS до выполнения MBR. Поскольку на разных платформах BIOS реализован по-разному, содержимое некоторых сегментных регистров на разных платформах может различаться. Поэтому именно код MBR должен обеспечить наличие в сегментных регистрах ожидаемых значений.

Функциональность расшифрованного кода проста: вредоносная программа читает сообщение с требованием выкупа из секторов со 2-го по 5-й в буфер памяти. Если в секторах с 7-го по 15-й записан шрифт, Satana загружает его службой INT 10h. Затем вредоносная программа выводит сообщение с требованием выкупа той же службой INT 10h и читает ввод с клавиатуры. Сообщение Satana с требованием выкупа показано на рисунке 13-9.

В нижней части сообщения пользователю предлагается ввести пароль для разблокировки MBR. Однако здесь есть подвох: после ввода пароля вредоносная программа в действительности не разблокирует MBR. Как видно из процедуры проверки пароля в листинге 13-11, вредоносная программа не восстанавливает исходную MBR.

```
seg000:01C2 ❶ mov     si, 2800h
seg000:01C5    mov     cx, 8
seg000:01C8 ❷ call    compute_checksum
seg000:01CB    add     al, ah
seg000:01CD ❸ cmp     al, ds:2900h

seg000:01D1 infinit_loop:
seg000:01D1 ❹ jmp     short infinit_loop
```

Листинг 13-11. Процедура проверки пароля Satana

```
You had bad luck.There was crypting of all your files in a FS bootkit virus
                              <!SATANA!>
To decrypt you need send on this E-mail: ryanqw31@gmail.com
your private code: A3D90235E1136671AB1195C6078184FF and pay on
a Bitcoin Wallet: XpVh1a3MqRPea2e1GJEvAYeVkpvF98sqhS total 0,5 btc
After that during 1 - 2 days the software will be sent to you - decryptor -
and the necessary instructions. All changes in hardware configurations of
your computer can make the decryption of your files absolutely impossible!
Decryption of your files is possible only on your PC!
Recovery is possible during 7 days, after which the program - decryptor -
can not ask for the necessary signature from a public certificate server.
Please contact via e-mail, which you can find as yet in the form of a text
document in a folder with encrypted files, as well as in the name of all
encrypted files.If you do not appreciate your files we recommend you format
all your disks and reinstall the system. Read carefully this warning as it is
no longer able to see at startup of the computer. We remind once again- it is
all serious! Do not touch the configuration of your computer!
E-mail: ryanqw31@gmail.com          - this is our mail
CODE: A3D90235E1136671AB1195C6078184FF this is code; you must send
BTC: XpVh1a3MqRPea2e1GJEvAYeVkpvF98sqhS here need to pay 0,5 bitcoins
How to pay on the Bitcoin wallet you can easily find on the Internet.
Enter your unlock code, obtained by E-mail here and press "ENTER" to
continue the normal download on your computer. Good luck! May God help you!
                              <!SATANA!>

_
```

Рисунок 13-9. Сообщение Satana с требованием выкупа

Процедура `compute_checksum` ❷ вычисляет контрольную сумму восьмибайтовой строки по адресу `ds:2800h` ❶ и сохраняет результат в регистре `ax`. Затем код сравнивает контрольную сумму со значением по адресу `ds:2900h` ❸. Однако независимо от результата сравнения код бесконечно зацикливается в точке ❹. Это означает, что поток выполнения отсюда дальше не идёт, хотя вредоносная MBR содержит код для расшифрования исходной MBR и её восстановления в самом первом секторе. Жертва, заплатившая выкуп за разблокировку системы, фактически не сможет сделать этого без программного обеспечения для восстановления системы. Это яркое напоминание о том, что жертвам программ-вымогателей не следует платить выкуп, поскольку никто не может гарантировать возвращение данных.

Подведение итогов: заключительные мысли о Satana

Satana - пример программы-вымогателя, которая всё ещё догоняла современные тенденции. Недостатки реализации и обилие отладочных сведений говорят о том, что вредоносная программа находилась в разработке, когда мы впервые увидели её в реальных атаках.

По сравнению с Petya Satana не отличается сложностью. Несмотря на то что она никогда не восстанавливает исходную MBR, её подход к заражению MBR не так разрушителен, как у Petya. Единственный загрузочный компонент, затрагиваемый Satana, - MBR, поэтому жертва может восстановить доступ к системе, исправив MBR посредством установочного DVD Windows, который способен восстановить сведения о системных разделах и заново создать MBR с действительной таблицей разделов.

Жертвы также могут восстановить доступ к системе, прочитав зашифрованную MBR из сектора 1 и объединив её по XOR с ключом шифрования из сектора 6. Так получается исходная MBR, которую следует записать в самый первый сектор для восстановления доступа к системе. Однако, даже если жертве удастся восстановить доступ к системе посредством восстановления MBR, содержимое файлов, зашифрованных Satana, всё равно останется недоступным.

Заключение

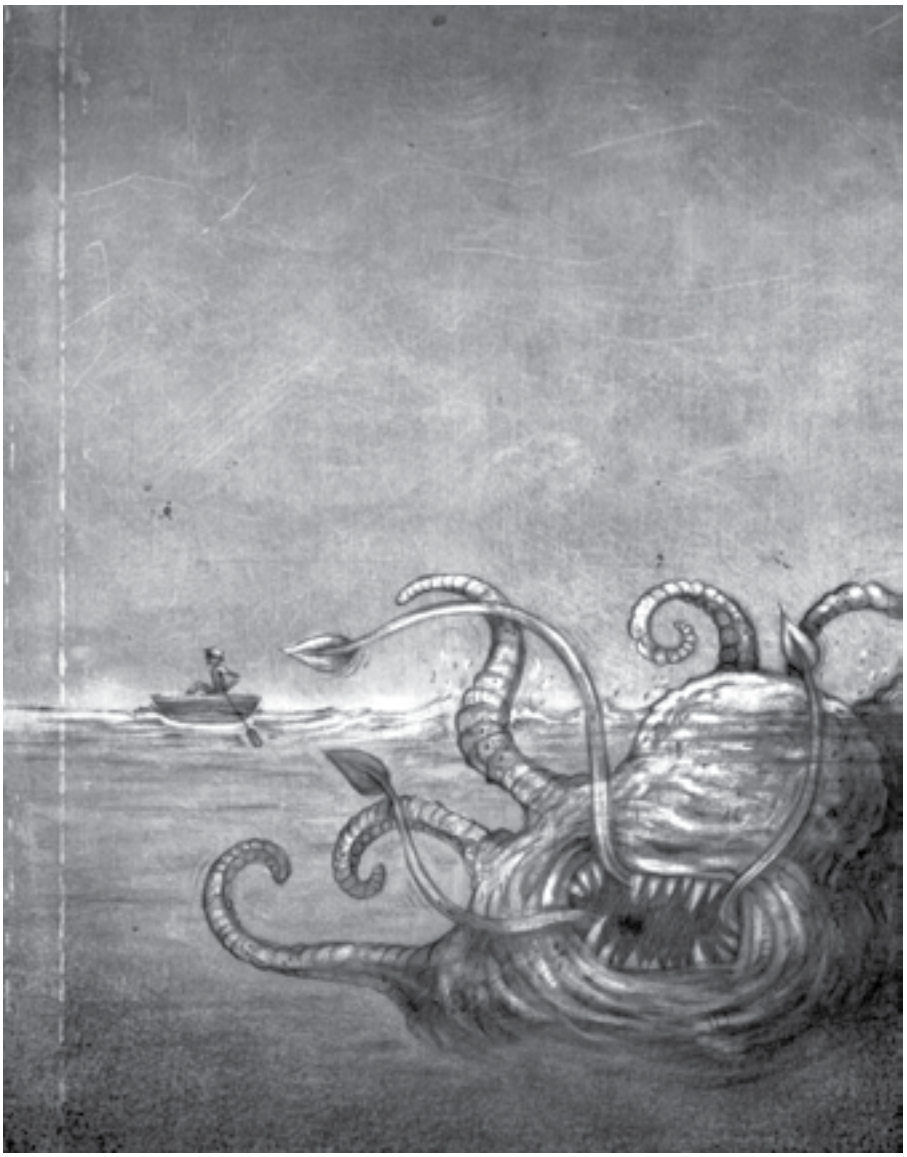
В этой главе рассмотрены некоторые основные этапы развития современных программ-вымогателей. Атаки как на домашних пользователей, так и на организации составляют современную тенденцию развития вредоносных программ, за которой антивирусная индустрия пыталась угнаться после всплеска троянов, шифрующих содержимое пользовательских файлов, в 2012 году.

Хотя эта новая тенденция среди программ-вымогателей набирает популярность, разработка компонентов буткита требует иных навыков и знаний, чем разработка троянов для шифрования пользовательских файлов. Недостатки компонента загрузчика Satana ясно показывают этот разрыв в квалификации.

Как мы видели на примере других вредоносных программ, гонка вооружений между разработчиками вредоносного и защитного ПО заставила программы-вымогатели развиваться и осваивать методы заражения буткитов, чтобы оставаться незамеченными. По мере появления всё большего числа программ-вымогателей многие защитные практики стали обычными, например резервное копирование данных - один из лучших способов защиты от широкого спектра угроз, особенно программ-вымогателей.

На этой странице исходного издания нет основного текста.

Глава 14. Загрузка UEFI против процесса загрузки MBR/VBR



Как мы видели, развитие буткитов следует за развитием процесса загрузки. Вместе с появлением в Windows 7 политики подписывания кода режима ядра, затруднившей загрузку произвольного кода в ядро, возродились буткиты, нацеленные на логику процесса загрузки до применения каких-либо проверок подписи, например на VBR, которую в то время невозможно было защитить. Аналогично, поскольку поддерживаемый в Windows 8 стандарт UEFI заменяет устаревшие процессы загрузки, такие как поток MBR/VBR, он также становится следующей целью заражения загрузки.

Современный UEFI сильно отличается от устаревших подходов. Устаревшая BIOS развивалась вместе с микропрограммой первых PC-совместимых компьютеров и поначалу представляла собой простой фрагмент кода, который во время начальной настройки конфигурировал оборудование ПК для загрузки всего остального программного обеспечения. Но по мере усложнения оборудования ПК для его настройки потребовался более сложный код микропрограммы, поэтому для единообразного управления разросшейся сложностью был разработан стандарт UEFI. Сегодня ожидается, что почти все современные компьютерные системы будут использовать для настройки

микропрограмму UEFI; устаревший процесс BIOS всё чаще ограничивается более простыми встроенными системами.

До появления стандарта UEFI реализации BIOS разных производителей не имели общей структуры. Такая несогласованность создавала препятствия злоумышленникам, вынужденным атаковать каждую реализацию BIOS отдельно, но также была проблемой для защитников, у которых не было единого механизма защиты целостности процесса загрузки и потока управления. Стандарт UEFI позволил создать такой механизм, получивший название UEFI Secure Boot.

Частичная поддержка UEFI появилась в Windows 7, но UEFI Secure Boot стала поддерживаться только в Windows 8. Наряду с Secure Boot Microsoft продолжает поддерживать устаревший процесс загрузки на основе MBR посредством модуля поддержки совместимости UEFI (Compatibility Support Module, CSM), который несовместим с Secure Boot и не предоставляет её гарантий целостности, как вскоре будет рассмотрено. Независимо от того, будет ли в будущем отключена устаревшая поддержка через CSM, UEFI явно является следующим этапом развития процесса загрузки, а значит, ареной совместного развития кода буткитов и защиты загрузки.

В этой главе мы сосредоточимся на особенностях процесса загрузки UEFI, особенно на его отличиях от устаревших подходов заражения загрузки MBR/VBR.

Унифицированный расширяемый интерфейс микропрограмм

UEFI - спецификация (uefi.org), определяющая программный интерфейс между операционной системой и микропрограммой. Первоначально она была разработана Intel для замены сильно различавшегося устаревшего загрузочного ПО BIOS, которое также было ограничено 16-разрядным режимом и потому не подходило новому оборудованию. Сегодня микропрограмма UEFI доминирует на рынке ПК с центральными процессорами Intel, и производители ARM также переходят на неё. Как уже упоминалось, из соображений совместимости некоторые микропрограммы на основе UEFI содержат модуль поддержки совместимости для устаревшего процесса загрузки BIOS предыдущих поколений операционных систем; однако Secure Boot в CSM не поддерживается.

Микропрограмма UEFI напоминает миниатюрную операционную систему, у которой есть даже собственный сетевой стек. Она содержит несколько миллионов строк кода, преимущественно на C, с добавлением ассемблера для зависящих от платформы частей. Поэтому микропрограмма UEFI значительно сложнее и функциональнее своих устаревших предшественников BIOS. В отличие от устаревшей BIOS её основные части имеют открытый исходный код. Эта особенность вместе с утечками кода, например исходного кода AMI в 2013 году, открыла возможности для внешних исследователей уязвимостей. За прошедшие годы действительно было опубликовано множество сведений об уязвимостях и векторах атак UEFI, некоторые из которых будут рассмотрены в главе 16.

> **Примечание.** Внутренняя сложность микропрограммы UEFI - одна из главных причин множества уязвимостей и векторов атак UEFI, о которых сообщалось в разные годы. Доступность исходного кода и большая открытость подробностей реализации микропрограммы UEFI такими причинами не являются. Доступность исходного кода не должна отрицательно влиять на безопасность и фактически оказывает противоположное действие.

Различия между процессами загрузки устаревшей BIOS и UEFI

С точки зрения безопасности основные отличия процесса загрузки UEFI обусловлены целью поддержки Secure Boot: логика потока MBR/VBR устранена и полностью заменена компонентами UEFI. Мы уже несколько раз упоминали Secure Boot, а теперь рассмотрим её подробнее при изучении процесса UEFI.

Сначала вспомним уже рассмотренные примеры вредоносных изменений загрузки ОС и выполняющие их буткиты:

• изменение загрузочного кода MBR (TDL4);
• изменение таблицы разделов MBR (Olmasco);
• изменение блока параметров BIOS VBR (Gapz);
• изменение загрузочного кода IPL (Rovnix).

Из этого списка видно, что все методы заражения процесса загрузки зависят от нарушения целостности загружаемого следующего этапа. UEFI Secure Boot призвана изменить эту схему, устанавливая цепочку доверия, в которой целостность каждого этапа потока проверяется до его загрузки и получения им управления.

Поток процесса загрузки

Задача устаревшей BIOS на основе MBR состояла лишь в применении необходимых настроек оборудования и передаче управления каждому следующему этапу загрузочного кода: от загрузочного кода к MBR, затем к VBR и, наконец, к загрузчику ОС, например `bootmgr` и `winload.exe` в Windows. Остальная логика потока не входила в её обязанности.

Процесс загрузки UEFI существенно отличается. MBR и VBR больше нет; вместо них за загрузку `bootmgr` отвечает единый загрузочный код самого UEFI.

Разделы диска: MBR против GPT

UEFI отличается от устаревшей BIOS и типом используемой таблицы разделов. В отличие от устаревшей BIOS с таблицей разделов в стиле MBR, UEFI поддерживает таблицу разделов GUID (GUID Partition Table, GPT). GPT существенно отличается от MBR. Таблицы MBR поддерживают только четыре слота основных или расширенных разделов, при необходимости с несколькими логическими разделами внутри расширенного, тогда как GPT поддерживает значительно больше разделов, каждый из которых обозначен уникальным 16-байтовым глобально уникальным идентификатором (Globally Unique Identifier, GUID). В целом правила разделов MBR сложнее правил GPT; стиль GPT допускает большие размеры разделов и имеет плоскую структуру таблицы ценой использования меток GUID вместо небольших целых чисел для идентификации разделов. Плоская структура упрощает некоторые аспекты управления разделами в UEFI.

Для поддержки процесса загрузки UEFI новая схема разделов GPT задаёт отдельный раздел, из которого загружается загрузчик ОС UEFI; в устаревшей таблице MBR эту роль играл флаг «активный», установленный для основного раздела. Этот специальный раздел называется системным разделом EFI и форматируется файловой системой FAT32, хотя также возможны FAT12 и FAT16. Путь к загрузчику в файловой системе раздела задаётся отдельной переменной энергонезависимой оперативной памяти (Nonvolatile Random Access Memory, NVRAM), также называемой переменной UEFI. NVRAM - небольшой модуль памяти на материнской плате ПК, используемый для хранения параметров конфигурации BIOS и операционной системы.

В Microsoft Windows путь к загрузчику в системе UEFI выглядит как `\EFI\Microsoft\Boot\bootmgfw.efi`. Назначение этого модуля - найти загрузчик ядра операционной системы, `winload.efi` в современных версиях Windows с поддержкой UEFI, и передать ему управление. Функциональность `winload.efi` в сущности совпадает с функциональностью `winload.exe`: загрузить и инициализировать образ ядра операционной системы.

На рисунке 14-1 показан поток процесса загрузки для устаревшей BIOS и UEFI, где в последнем отсутствуют этапы MBR и VBR.

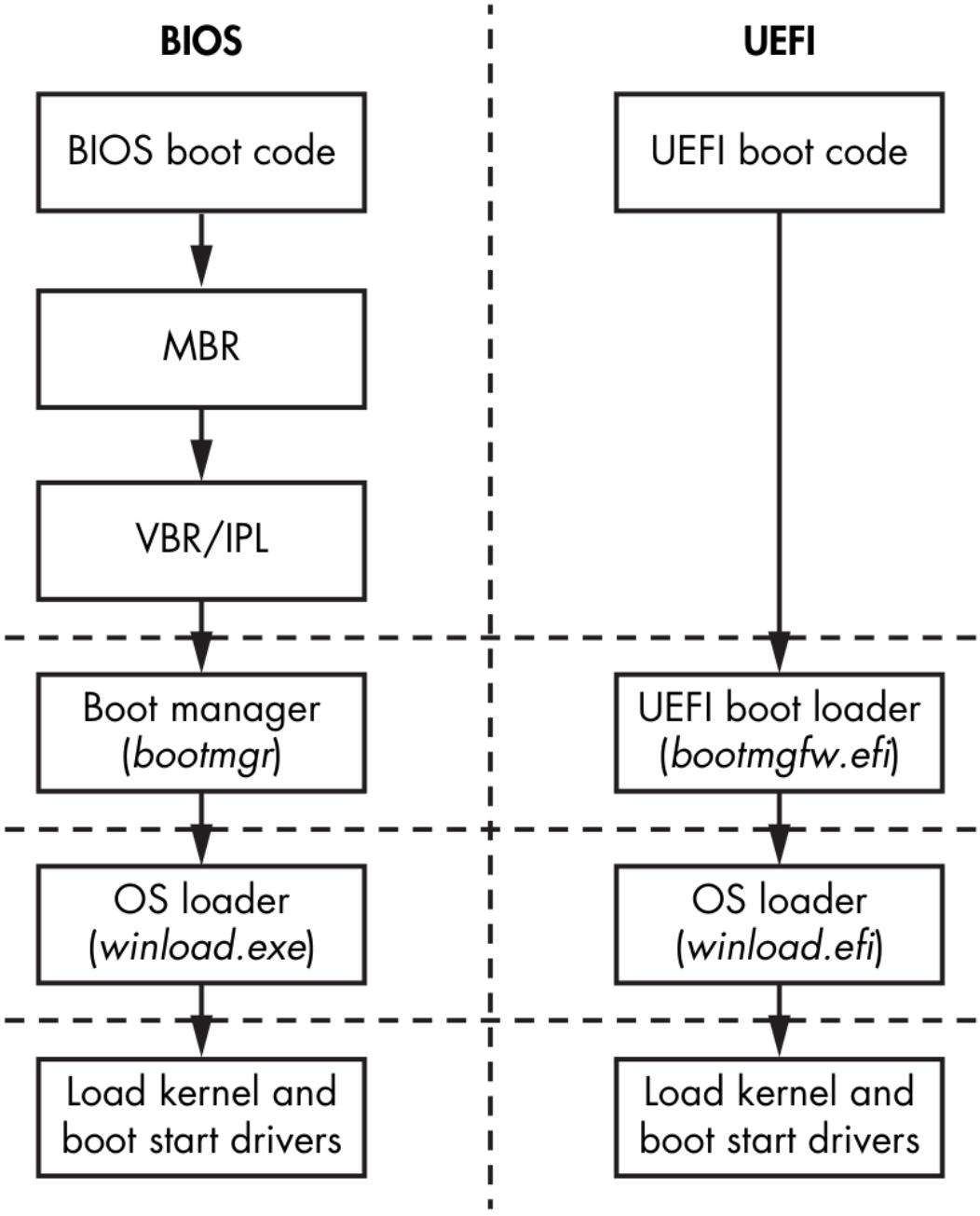


Рисунок 14-1. Различие потоков загрузки в устаревших системах BIOS и системах UEFI

Как видно, до передачи управления загрузчику операционной системы системы на основе UEFI делают в микропрограмме намного больше, чем устаревшая BIOS. Здесь нет промежуточных этапов, таких как загрузочный код MBR/VBR: процесс загрузки полностью управляется только микропрограммой UEFI. Микропрограмма BIOS занималась лишь инициализацией платформы, оставляя остальное загрузчикам операционной системы, `bootmgr` и `winload.exe`.

Другие различия

Ещё одно огромное изменение UEFI состоит в том, что почти весь его код работает в защищённом режиме, за исключением небольшой начальной заглушки, получающей управление от центрального процессора при включении питания или сбросе. Защищённый режим поддерживает выполнение 32- и 64-разрядного кода, хотя также позволяет эмулировать другие устаревшие режимы, не используемые современной логикой загрузки. В отличие от него устаревшая логика загрузки выполняла большую часть кода в 16-разрядном режиме до передачи управления загрузчикам ОС.

Другое отличие микропрограммы UEFI от устаревшей BIOS состоит в том, что большая часть UEFI написана на C, а некоторые производители даже компилируют её компилятором C++, и только небольшая часть - на ассемблере. Это повышает качество кода по сравнению с полностью ассемблерными реализациями устаревшей микропрограммы BIOS.

Другие различия устаревшей BIOS и микропрограммы UEFI приведены в таблице 14-1.

Таблица 14-1. Сравнение устаревшей BIOS и микропрограммы UEFI

Характеристика	Устаревшая BIOS	Микропрограмма UEFI
Архитектура	Неопределённый процесс разработки микропрограммы; все производители BIOS независимо поддерживают собственную кодовую базу	Единая спецификация разработки микропрограммы и эталонный код Intel (EDKI/EDKII)
Реализация	Преимущественно ассемблер	C/C++
Модель памяти	16-разрядный реальный режим	32-/64-разрядный защищённый режим
Загрузочный код	MBR и VBR	Отсутствует, процессом загрузки управляет микропрограмма
Схема разделов	Таблица разделов MBR	Таблица разделов GUID (GPT)
Ввод-вывод диска	Системные прерывания	Службы UEFI
Загрузчики	`bootmgr` и `winload.exe`	`bootmgfw.efi` и `winload.efi`
Взаимодействие с ОС	Прерывания BIOS	Службы UEFI
Сведения конфигурации загрузки	Память CMOS, без понятия переменных NVRAM	Хранилище переменных UEFI NVRAM

Прежде чем переходить к подробностям процесса загрузки UEFI и загрузчика операционной системы, внимательно рассмотрим особенности GPT. Понимание различий схем разделов MBR и GPT необходимо для изучения процесса загрузки UEFI.

Особенности таблицы разделов GUID

Если открыть основной жёсткий диск Windows, отформатированный с GPT, в шестнадцатеричном редакторе, в первых двух секторах (1 сектор = 512 байт) не будет загрузочного кода MBR или VBR. Пространство, которое в устаревшей BIOS содержало бы код MBR, почти полностью заполнено нулями. Вместо этого в начале второго сектора со смещением `0x200` сразу после знакомой конечной метки MBR `55 AA` видна сигнатура `EFI PART` (рисунок 14-2). Это сигнатура таблицы разделов EFI в заголовке GPT, идентифицирующая его.

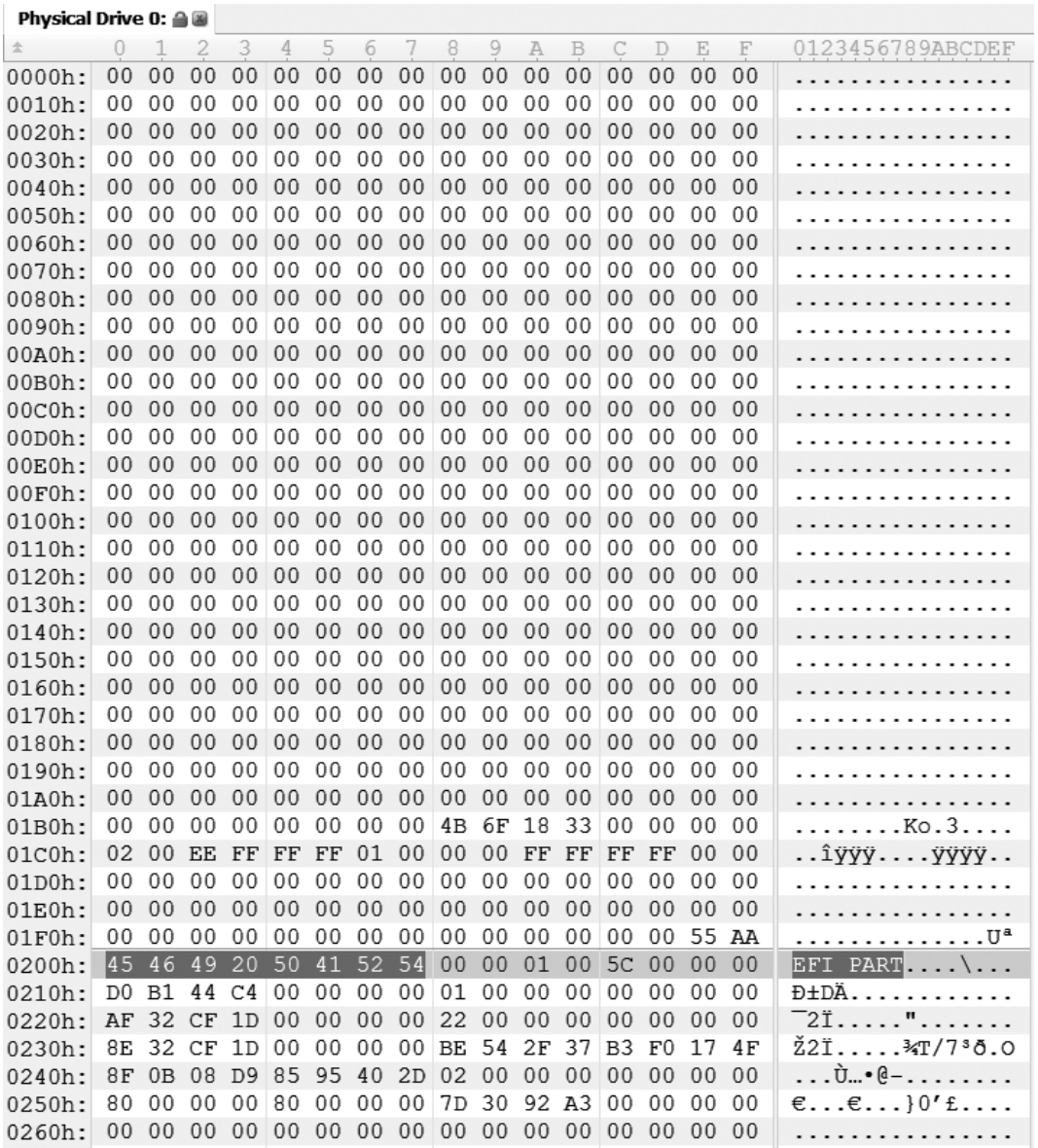
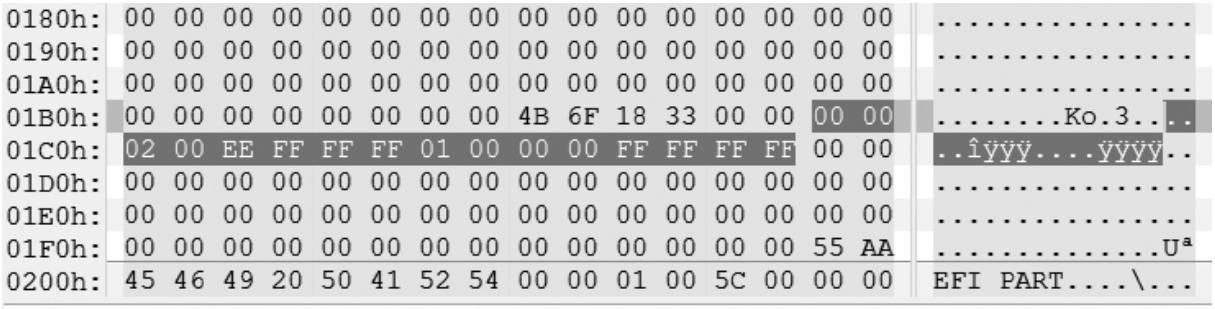


Рисунок 14-2. Дамп сигнатуры таблицы разделов GUID из `\\.\PhysicalDrive0`

Однако структура таблицы разделов MBR исчезла не полностью. Для совместимости с устаревшими процессами загрузки и такими средствами, как низкоуровневые редакторы дисков эпохи до GPT, в начале GPT эмулируется старая таблица MBR. Теперь эта эмулируемая таблица разделов MBR содержит только один элемент для всего диска GPT, как показано на рисунке 14-3. Такая форма схемы MBR называется защитной MBR.



Template Results - Drive.bt

Name	Value	Start	Size
struct MASTER_BOOT_RECORD boot_mbr		0h	200h
UBYTE BootCode[446]		0h	1BEh
struct PARTITION_ENTRY partitions[4]		1BEh	40h
struct PARTITION_ENTRY partitions[0]	LEGACY_MBR_EFI_HEADER	1BEh	10h
enum BOOTINDICATOR BootIndicator	NOBOOT (0)	1BEh	1h
UBYTE StartingHead	0	1BFh	1h
WORD StartingSectCylinder	2	1C0h	2h
enum SYSTEMID SystemID	LEGACY_MBR_EFI_HEADER (238)	1C2h	1h
UBYTE EndingHead	255	1C3h	1h
WORD EndingSectCylinder	65535	1C4h	2h
DWORD RelativeSector	1	1C6h	4h
DWORD TotalSectors	4294967295	1CAh	4h
struct PARTITION_ENTRY partitions[1]	EMPTY	1CEh	10h
struct PARTITION_ENTRY partitions[2]	EMPTY	1DEh	10h
struct PARTITION_ENTRY partitions[3]	EMPTY	1EEh	10h
WORD EndOfSectorMarker	AA55h	1FEh	2h

Рисунок 14-3. Заголовок устаревшей MBR, разобранный в 010 Editor шаблоном Drive.bt

Защитная MBR не позволяет устаревшему программному обеспечению, например дисковым утилитам, случайно уничтожить разделы GUID, помечая всё дисковое пространство занятым одним разделом. Устаревшие средства, не знающие о GPT, не принимают разделённые GPT части за свободное пространство. Защитная MBR имеет тот же формат, что и обычная MBR, хотя является лишь заглушкой. Микропрограмма UEFI распознаёт защитную MBR и не пытается выполнить из неё какой-либо код.

Главное отличие от устаревшего процесса загрузки BIOS состоит в том, что теперь весь код, отвечающий за ранние этапы загрузки системы, заключён в самой микропрограмме UEFI и находится в микросхеме флеш-памяти, а не на диске. Это означает, что методы заражения MBR, изменявшие MBR или VBR на диске, например использованные TDL4 и Olmasco и рассмотренные соответственно в главах 7 и 10, не влияют на поток загрузки систем на основе GPT даже без включённой Secure Boot.

Проверка поддержки GPT

Проверить поддержку GPT в системе Windows можно командами Microsoft PowerShell. В частности, команда `Get-Disk` (листинг 14-1) возвращает таблицу, последний столбец которой, `Partition Style`, показывает поддерживаемый тип таблицы разделов. При совместимости с GPT в этом столбце будет указано `GPT`, иначе - `MBR`.

```
PS C:\> Get-Disk
Number Friendly Name  Operational Status  Total Size  Partition Style
------ -------------  ------------------  ----------  ---------------
0      Microsoft      Online                   127GB  GPT
       Virtual Disk
```

Листинг 14-1. Вывод `Get-Disk`

В таблице 14-2 описаны значения заголовка GPT.

Таблица 14-2. Заголовок GPT

Имя	Смещение	Длина
Сигнатура `EFI PART`	`0x00`	8 байт
Ревизия версии GPT	`0x08`	4 байта
Размер заголовка	`0x0C`	4 байта
CRC32 заголовка	`0x10`	4 байта
Зарезервировано	`0x14`	4 байта
Текущий LBA (логическая блочная адресация)	`0x18`	8 байт
Резервный LBA	`0x20`	8 байт
Первый доступный для разделов LBA	`0x28`	8 байт
Последний доступный LBA	`0x30`	8 байт
GUID диска	`0x38`	16 байт
Начальный LBA массива элементов разделов	`0x48`	8 байт
Число элементов разделов в массиве	`0x50`	4 байта
Размер одного элемента раздела	`0x54`	4 байта
CRC32 массива разделов	`0x58`	4 байта
Зарезервировано	`0x5C`	*

Как видно, заголовок GPT содержит только постоянные поля, а не код. С точки зрения криминалистического анализа важнее всего поля «Начальный LBA массива элементов разделов» и «Число элементов разделов в массиве». Они задают соответственно положение и размер таблицы разделов на жёстком диске.

Ещё одно интересное поле заголовка GPT - `Backup LBA`, задающее положение резервной копии заголовка GPT. Это позволяет восстановить основной заголовок GPT в случае повреждения. Мы уже затрагивали резервный заголовок GPT в главе 13 при обсуждении программы-вымогателя Petya, которая шифровала и основной, и резервный заголовки GPT, затрудняя восстановление системы.

Как показано на рисунке 14-4, каждый элемент таблицы разделов содержит сведения о свойствах и положении раздела на жёстком диске.

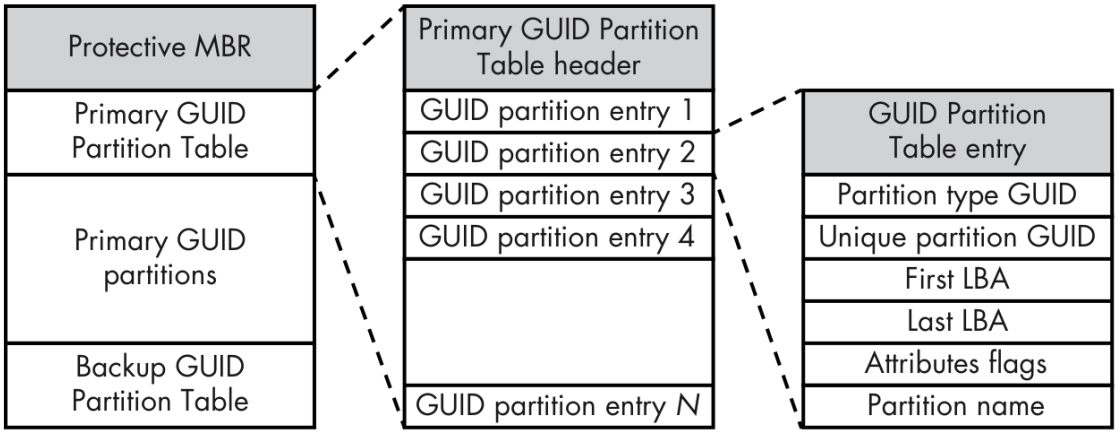


Рисунок 14-4. Таблица разделов GUID

Два 64-битных поля `First LBA` и `Last LBA` задают соответственно адреса самого первого и последнего секторов раздела. Поле `Partition type GUID` содержит значение GUID, идентифицирующее тип раздела. Например, для системного раздела EFI, упомянутого ранее в разделе «Разделы диска: MBR против GPT» на странице 235, тип равен `C12A7328-F81F-11D2-BA4B-00A0C93EC93B`.

Отсутствие исполняемого кода в схеме GPT создаёт проблему для заражения буткитом: как разработчикам вредоносных программ передать управление процесса загрузки своему вредоносному коду в схеме GPT? Одна из идей - изменить загрузчики EFI до передачи ими управления ядру ОС. Но прежде чем это исследовать, рассмотрим основы архитектуры микропрограммы UEFI и процесса загрузки.

Разбор диска GPT посредством SweetScape

Для разбора полей диска GPT на работающем компьютере или в дампе раздела можно использовать условно-бесплатный SweetScape 010 Editor (sweetscape.com) с шаблоном Drive.bt Бенджамина Верну, находящимся на сайте SweetScape в репозитории Templates раздела Downloads. 010 Editor обладает очень мощным механизмом разбора на основе шаблонов со структурами, похожими на C (см. рисунок 14-3).

Как работает микропрограмма UEFI

Изучив схему разделов GPT, мы теперь понимаем, где находится загрузчик ОС и как микропрограмма UEFI находит его на жёстком диске. Теперь рассмотрим, как микропрограмма UEFI загружает и выполняет загрузчик ОС. Мы приведём исходные сведения об этапах, через которые проходит процесс загрузки UEFI при подготовке среды выполнения загрузчика.

Микропрограмма UEFI, интерпретирующая упомянутые структуры данных таблицы GPT для поиска загрузчика ОС, хранится в микросхеме флеш-памяти материнской платы, также называемой SPI flash, где SPI обозначает интерфейс шины, соединяющей микросхему с остальным набором микросхем. При запуске системы логика набора микросхем отображает содержимое памяти микросхемы флеш-памяти в определённую область оперативной памяти, начальный и конечный адреса которой настроены в аппаратном наборе микросхем и зависят от конфигурации центрального процессора. Получив управление при включении питания, код отображённой микросхемы SPI flash инициализирует оборудование и загружает различные драйверы, диспетчер загрузки ОС, загрузчик ОС и, наконец, само ядро ОС. Эту последовательность можно обобщить следующим образом:

1. Микропрограмма UEFI выполняет инициализацию платформы UEFI, центрального процессора и набора микросхем и загружает модули платформы UEFI, также называемые драйверами UEFI; они отличаются от зависящего от устройства кода, загружаемого на следующем этапе.
2. Диспетчер загрузки UEFI перечисляет устройства на внешних шинах, например PCI, загружает драйверы устройств UEFI, а затем загружает загрузочное приложение.
3. Диспетчер загрузки Windows (`bootmgfw.efi`) загружает загрузчик Windows.
4. Загрузчик Windows (`winload.efi`) загружает ОС Windows.

Код, отвечающий за этапы 1 и 2, находится в SPI flash. Код этапов 3 и 4 извлекается из файловой системы специального раздела UEFI на жёстком диске после того, как этапы 1 и 2 сделали возможным чтение диска. Спецификация UEFI дополнительно делит микропрограмму на компоненты, отвечающие за разные части инициализации оборудования или работы процесса загрузки, как показано на рисунке 14-5.

Загрузчик ОС в основном полагается на загрузочные службы EFI и службы времени выполнения EFI, предоставляемые микропрограммой UEFI, чтобы загружать систему и управлять ею. Как будет объяснено в разделе «Внутри загрузчика операционной системы» на странице 245, загрузчик ОС использует эти службы для создания среды, в которой можно загрузить ядро ОС. После того как загрузчик ОС получает от микропрограммы UEFI управление потоком загрузки, загрузочные службы удаляются и становятся недоступными операционной системе. Однако службы времени выполнения остаются доступны ОС во время работы и предоставляют интерфейс чтения и записи переменных UEFI NVRAM, обновления микропрограммы посредством Capsule Update и перезагрузки или выключения системы.

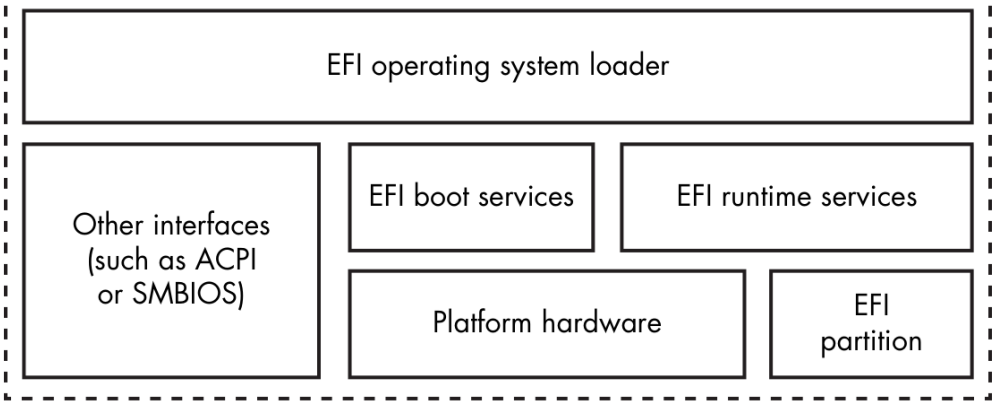


Рисунок 14-5. Обзор структуры UEFI

Обновление капсулы микропрограммы

Capsule Update - технология безопасного обновления микропрограммы UEFI. Операционная система загружает образ капсулы обновления микропрограммы в память и посредством службы времени выполнения сообщает микропрограмме UEFI о наличии капсулы. В результате микропрограмма UEFI перезагружает систему и обрабатывает капсулу обновления при следующей загрузке. Capsule Update пытается стандартизировать и повысить безопасность процесса обновления микропрограммы UEFI. Подробнее она будет рассмотрена в главе 15.

Спецификация UEFI

В отличие от устаревшей загрузки BIOS спецификация UEFI охватывает каждый этап, начиная с инициализации оборудования. До появления этой спецификации производители оборудования имели больше свободы в разработке микропрограмм, но эта свобода также допускала неразбериху и, следовательно, уязвимости. Спецификация описывает четыре основных последовательных этапа процесса загрузки, каждый со своими обязанностями:

• **Безопасность (Security, SEC).** Инициализирует временную память посредством кешей центрального процессора и находит загрузчик фазы PEI. Код этапа SEC выполняется из памяти SPI flash.
• **Инициализация до EFI (Pre-EFI Initialization, PEI).** Настраивает контроллер памяти, инициализирует набор микросхем и обрабатывает возобновление S3. Код этого этапа выполняется во временной памяти до инициализации контроллера памяти. После неё код PEI выполняется из постоянной памяти.
• **Среда выполнения драйверов (Driver Execution Environment, DXE).** Инициализирует режим управления системой (System Management Mode, SMM) и службы DXE - ядро, диспетчер, драйверы и прочее, - а также загрузочные службы и службы времени выполнения.
• **Выбор загрузочного устройства (Boot Device Selection, BDS).** Обнаруживает аппаратное устройство, с которого можно загрузить ОС, например перечисляя периферийные устройства на шине PCI, которые могут содержать совместимый с UEFI загрузчик, такой как загрузчик ОС.

Все компоненты, используемые в процессе загрузки, находятся в SPI flash, кроме загрузчика ОС, расположенного в файловой системе диска. Код фаз DXE/BDS на основе SPI flash находит его по пути файловой системы, сохранённому в переменной UEFI NVRAM, как обсуждалось ранее.

Этапы инициализации SMM и DXE - одни из наиболее интересных областей для внедрения руткитов. SMM на кольце -2 является самым привилегированным режимом системы, более привилегированным, чем гипервизоры на кольце -1. Подробнее о SMM и уровнях привилегий колец см. во врезке «Режим управления системой». Из этого режима вредоносный код может полностью управлять системой.

Аналогично драйверы DXE предоставляют ещё одну мощную точку для реализации функциональности буткита. Хороший пример вредоносной программы на основе DXE - реализация руткита микропрограммы Hacking Team, рассматриваемая в главе 15.

Теперь исследуем последний этап и процесс, посредством которого ядро операционной системы получает управление. Подробнее DXE и SMM будут рассмотрены в следующей главе.

Режим управления системой

System Management Mode - специальный режим центральных процессоров x86, выполняемый с особыми повышенными правами «кольца -2». «Минус два» ниже и мощнее «кольца -1», которое, в свою очередь, мощнее «кольца 0», исторически самого доверенного уровня прав; как удачно, что у нас есть бесконечный запас целых чисел меньше нуля. SMM появился в процессорах Intel 386 главным образом как средство управления электропитанием, но в современных центральных процессорах значительно выросли его сложность и важность. Теперь SMM - неотъемлемая часть микропрограммы, отвечающая за всю инициализацию и настройку разделения памяти в процессе загрузки. Код SMM выполняется в отдельном адресном пространстве, которое должно быть изолировано от обычной разметки адресного пространства операционной системы, включая пространство ядра ОС. В главах 15 и 16 мы подробнее рассмотрим, как руткиты UEFI используют SMM.

Внутри загрузчика операционной системы

Теперь, когда хранящийся в SPI код микропрограммы UEFI выполнил свою работу, он передаёт управление загрузчику ОС, хранящемуся на диске. Код загрузчика также 64- или 32-разрядный, в зависимости от версии операционной системы; 16-разрядному коду загрузчика MBR или VBR в процессе загрузки места нет.

Загрузчик ОС состоит из нескольких файлов в системном разделе EFI, включая модули `bootmgfw.efi` и `winload.efi`. Первый называется диспетчером загрузки Windows, второй - загрузчиком Windows. Положение этих модулей также задаётся переменными NVRAM. В частности, путь UEFI к диску, определяемый способом перечисления портов и шин материнской платы стандартом UEFI, с ESP хранится в переменной NVRAM порядка загрузки `BOOT_ORDER`, которую пользователь обычно может изменить в конфигурации BIOS. Путь внутри файловой системы ESP хранится в другой переменной, `BOOT`, обычно в `\EFI\Microsoft\Boot\`.

Доступ к диспетчеру загрузки Windows

Диспетчер загрузки микропрограммы UEFI обращается к переменным UEFI NVRAM, чтобы найти ESP, а затем, в случае Windows, специальный диспетчер загрузки ОС `bootmgfw.efi` внутри него. После этого диспетчер загрузки создаёт образ времени выполнения этого файла в памяти. Для этого он полагается на микропрограмму UEFI при чтении загрузочного жёсткого диска и разборе его файловой системы. В другой ОС переменная NVRAM содержала бы путь к её загрузчику; например, для Linux она указывает на загрузчик GRUB (`grub.efi`).

После загрузки `bootmgfw.efi` диспетчер загрузки микропрограммы UEFI переходит к точке входа `bootmgfw.efi`, `EfiEntry`. Это начало процесса загрузки ОС, где хранящаяся в SPI flash микропрограмма передаёт управление коду на жёстком диске.

Создание среды выполнения

Точка входа `EfiEntry`, прототип которой показан в листинге 14-2, вызывает диспетчер загрузки Windows `bootmgfw.efi` и используется для настройки обратных вызовов микропрограммы UEFI для загрузчика Windows `winload.efi`, вызываемого сразу после него. Эти обратные вызовы соединяют код `winload.efi` со службами времени выполнения микропрограммы UEFI, необходимыми для операций с периферией, например чтения жёсткого диска. Эти службы продолжат использоваться Windows даже после полной загрузки посредством обёрток уровня аппаратных абстракций (Hardware Abstraction Layer, HAL), настройку которых мы вскоре увидим.

```
EFI_STATUS EfiEntry (
   ❶ EFI_HANDLE ImageHandle,       // Дескриптор образа UEFI загруженного приложения
   ❷ EFI_SYSTEM_TABLE *SystemTable // Указатель на системную таблицу UEFI
);
```

Листинг 14-2. Прототип процедуры `EfiEntry` *(*`EFI_IMAGE_ENTRY_POINT`*)*

Первый параметр `EfiEntry` ❶ указывает на модуль `bootmgfw.efi`, отвечающий за продолжение процесса загрузки и вызов `winload.efi`. Второй параметр ❷ содержит указатель на таблицу конфигурации UEFI (`EFI_SYSTEM_TABLE`), являющуюся ключом к доступу к большей части конфигурационных данных служб среды EFI (рисунок 14-6).

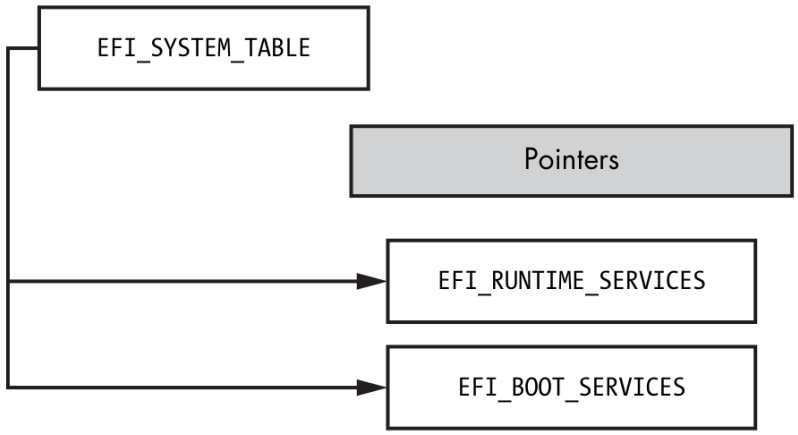


Рисунок 14-6. Высокоуровневая структура `EFI_SYSTEM_TABLE`

Загрузчик `winload.efi` использует службы UEFI для загрузки ядра операционной системы со стеком драйверов загрузочного устройства и инициализации `EFI_RUNTIME_TABLE` в пространстве ядра для будущего доступа ядра через код библиотеки HAL (`hal.dll`). HAL потребляет `EFI_SYSTEM_TABLE` и экспортирует остальному ядру функции-обёртки функций времени выполнения UEFI. Ядро вызывает их для выполнения таких задач, как чтение переменных NVRAM и обработка обновлений BIOS посредством так называемого Capsule Update, передаваемого микропрограмме UEFI.

Обратите внимание на схему множества обёрток, которые каждый следующий уровень создаёт поверх зависящего от оборудования кода UEFI, настроенного на самых ранних этапах загрузки. Никогда не знаешь, насколько глубоко в кроличью нору UEFI может уйти системный вызов ОС!

Структура EFI_RUNTIME_SERVICES, используемая модулем HAL hal.dll, показана на рисунке 14-7.

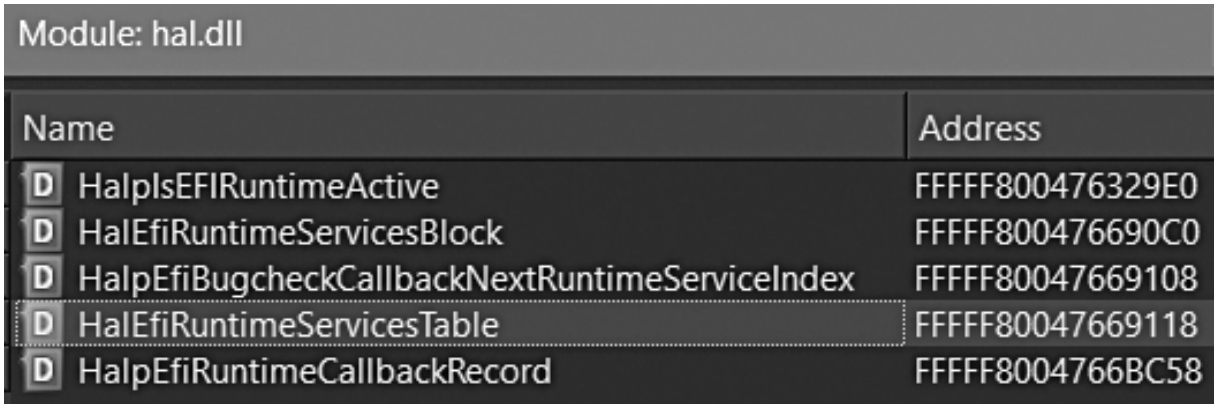


Рисунок 14-7. EFI_RUNTIME_SERVICES *в представлении* hal.dll

HalEfiRuntimeServiceTable хранит указатель на EFI_RUNTIME_SERVICES, которая, в свою очередь, содержит адреса точек входа служебных процедур для получения или установки переменной NVRAM, выполнения Capsule Update и других операций.

В следующих главах мы проанализируем эти структуры в контексте уязвимостей микропрограмм, эксплуатации и руткитов. Пока же подчеркнём, что EFI_SYSTEM_TABLE и особенно находящаяся в ней EFI_RUNTIME_SERVICES - ключи к поиску структур, отвечающих за доступ к конфигурационным сведениям UEFI, и что часть этих сведений доступна из режима ядра операционной системы.

На рисунке 14-8 показана дизассемблированная процедура `EfiEntry`. Одна из её первых инструкций запускает вызов функции `EfiInitCreateInputParametersEx()`, которая преобразует параметры `EfiEntry` в формат, ожидаемый `bootmgfw.efi`. Внутри `EfiInitCreateInputParametersEx()` процедура `EfiInitpCreateApplicationEntry()` создаёт элемент для `bootmgfw.efi` в данных конфигурации загрузки (Boot Configuration Data, BCD) - двоичном хранилище параметров конфигурации загрузчика Windows. После возврата `EfiInitCreateInputParametersEx()` управление получает процедура `BmMain`, выделенная на рисунке 14-8. Обратите внимание: чтобы правильно обращаться к операциям аппаратных устройств, включая любой ввод-вывод жёсткого диска, и инициализировать память, диспетчер загрузки Windows в этот момент должен использовать только службы EFI, поскольку основные стеки драйверов Windows ещё не загружены и недоступны.

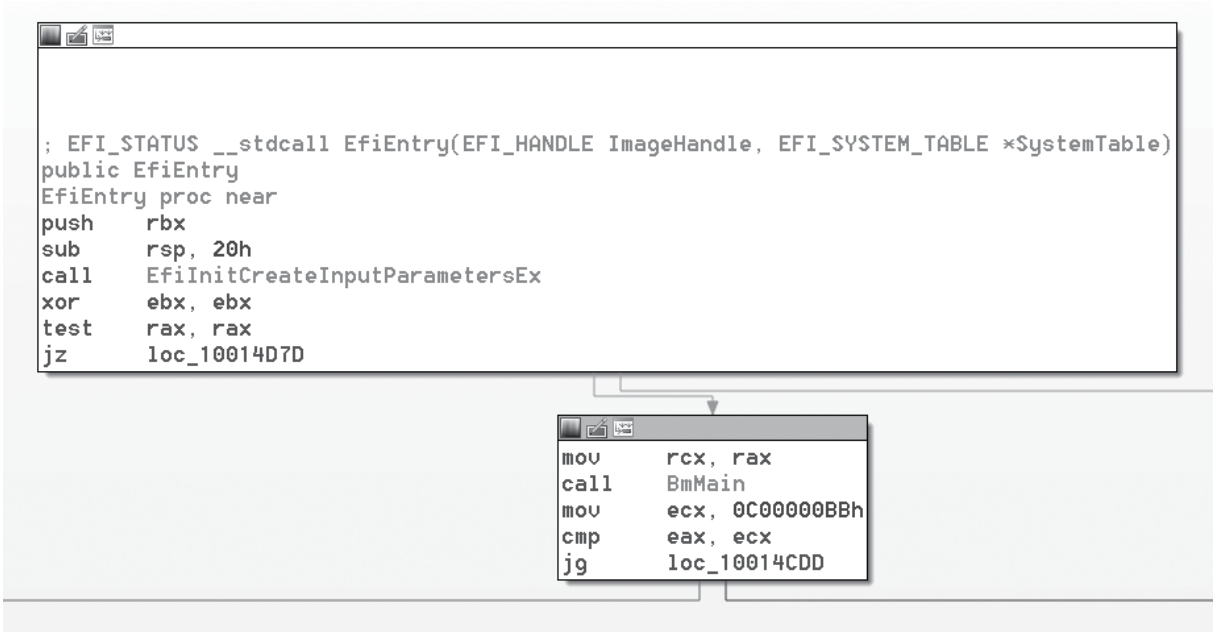


Рисунок 14-8. Дизассемблированная процедура `EfiEntry`

Чтение данных конфигурации загрузки

На следующем этапе `BmMain` вызывает следующие процедуры:

• `BmFwInitializeBootDirectoryPath` - процедура для инициализации пути загрузочного приложения (`\EFI\Microsoft\Boot`);
• `BmOpenDataStore` - процедура для подключения и чтения файла базы данных BCD (`\EFI\Microsoft\Boot\BCD`) посредством служб UEFI, то есть дискового ввода-вывода;
• `BmpLaunchBootEntry` и `ImgArchEfiStartBootApplication` - процедуры для выполнения загрузочного приложения (`winload.efi`).

В листинге 14-3 показаны данные конфигурации загрузки, выведенные стандартным средством командной строки `bcdedit.exe`, которое входит во все современные версии Microsoft Windows. Пути к модулям диспетчера загрузки Windows и загрузчика Windows отмечены соответственно ❶ и ❷.

```
PS C:\WINDOWS\system32> bcdedit
Windows Boot Manager
--------------------
identifier              {bootmgr}
device                  partition=\Device\HarddiskVolume2
❶ path                    \EFI\Microsoft\Boot\bootmgfw.efi
description             Windows Boot Manager
locale                  en-US
inherit                 {globalsettings}
default                 {current}
resumeobject            {c68c4e64-6159-11e8-8512-a4c49440f67c}
displayorder            {current}
toolsdisplayorder       {memdiag}
timeout                 30
Windows Boot Loader
-------------------
identifier              {current}
device                  partition=C:
❷ path                    \WINDOWS\system32\winload.efi
description             Windows 10
locale                  en-US
inherit                 {bootloadersettings}
recoverysequence        {f5b4c688-6159-11e8-81bd-8aecff577cb6}
displaymessageoverride  Recovery
recoveryenabled         Yes
isolatedcontext         Yes
allowedinmemorysettings 0x15000075
osdevice                partition=C:
systemroot              \WINDOWS
resumeobject            {c68c4e64-6159-11e8-8512-a4c49440f67c}
nx                      OptIn
bootmenupolicy          Standard
```

Листинг 14-3. Вывод консольной команды `bcdedit`

Диспетчер загрузки Windows (`bootmgfw.efi`) также отвечает за проверку политики загрузки и инициализацию компонентов целостности кода и Secure Boot, рассматриваемых в следующих главах.

На следующем этапе процесса загрузки `bootmgfw.efi` загружает и проверяет загрузчик Windows (`winload.efi`). Перед началом загрузки `winload.efi` диспетчер загрузки Windows инициализирует карту памяти для перехода в защищённый режим памяти, предоставляющий виртуальную память и страничную организацию. Важно, что эта настройка выполняется посредством служб времени выполнения UEFI, а не напрямую. Это создаёт надёжный уровень абстракции для структур данных виртуальной памяти ОС, таких как GDT, которые ранее обрабатывались устаревшей BIOS в 16-разрядном ассемблерном коде.

Передача управления Winload

На заключительном этапе диспетчера загрузки Windows процедура `BmpLaunchBootEntry()` загружает и выполняет `winload.efi`, загрузчик Windows. На рисунке 14-9 представлен полный граф вызовов от `EfiEntry()` до `BmpLaunchBootEntry()`, сформированный дизассемблером Hex-Rays IDA Pro со сценарием IDAPathFinder (devttys0.com).

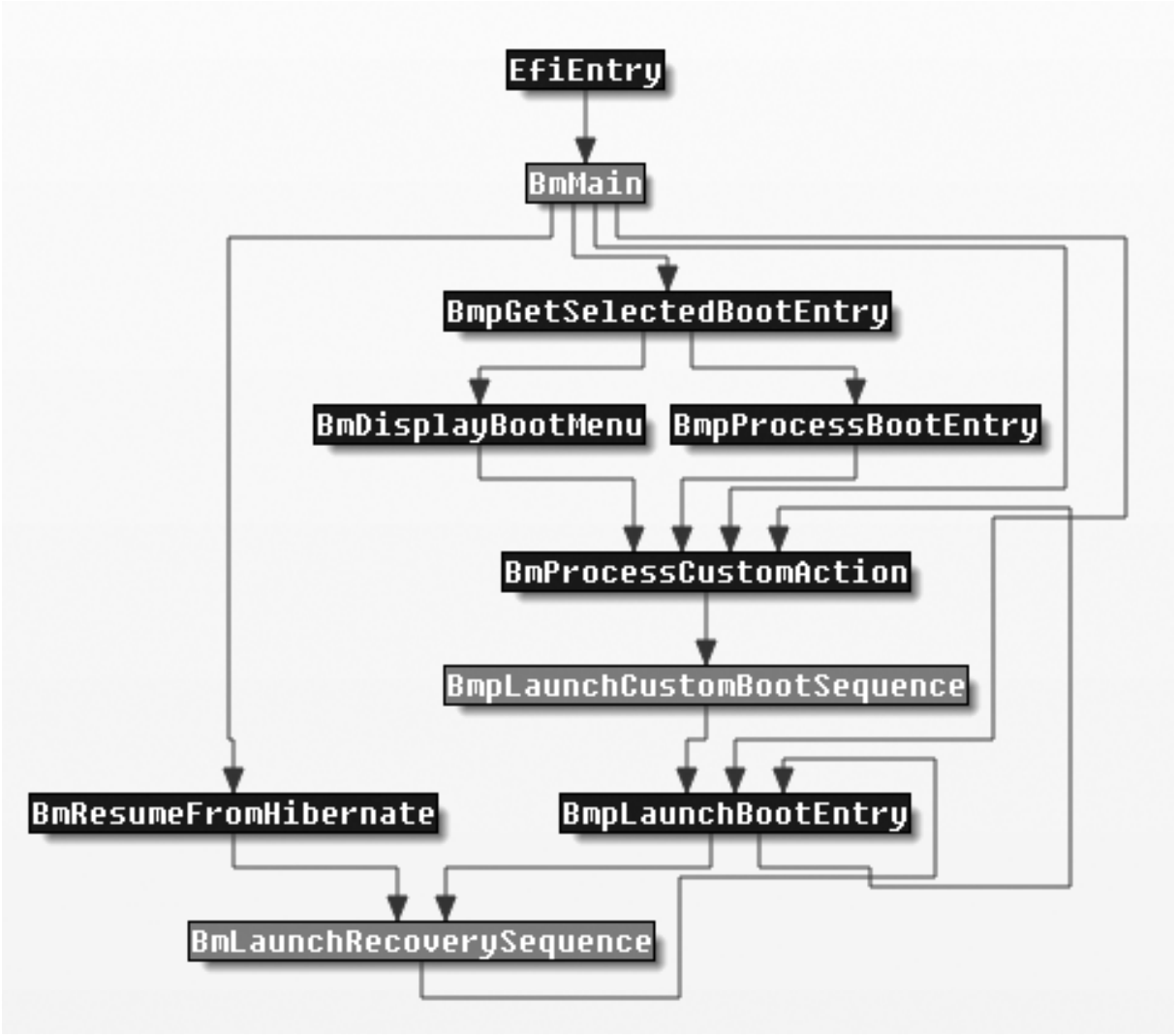


Рисунок 14-9. Поток графа вызовов от `EfiEntry()` *до* `BmpLaunchBootEntry()`

Поток управления перед функцией `BmpLaunchBootEntry()` выбирает правильный загрузочный элемент на основе значений хранилища BCD. Если включено полное шифрование тома BitLocker, диспетчер загрузки расшифровывает системный раздел до передачи управления загрузчику. Функция `BmpLaunchBootEntry()`, за которой следует `BmpTransferExecution()`, проверяет параметры загрузки и передаёт выполнение `BlImgLoadBootApplication()`, которая затем вызывает `ImgArchEfiStartBootApplication()`. Процедура `ImgArchEfiStartBootApplication()` отвечает за инициализацию защищённого режима памяти для `winload.efi`. После этого управление передаётся функции `Archpx64TransferTo64BitApplicationAsm()`, завершающей подготовку к запуску `winload.efi` (рисунок 14-10).

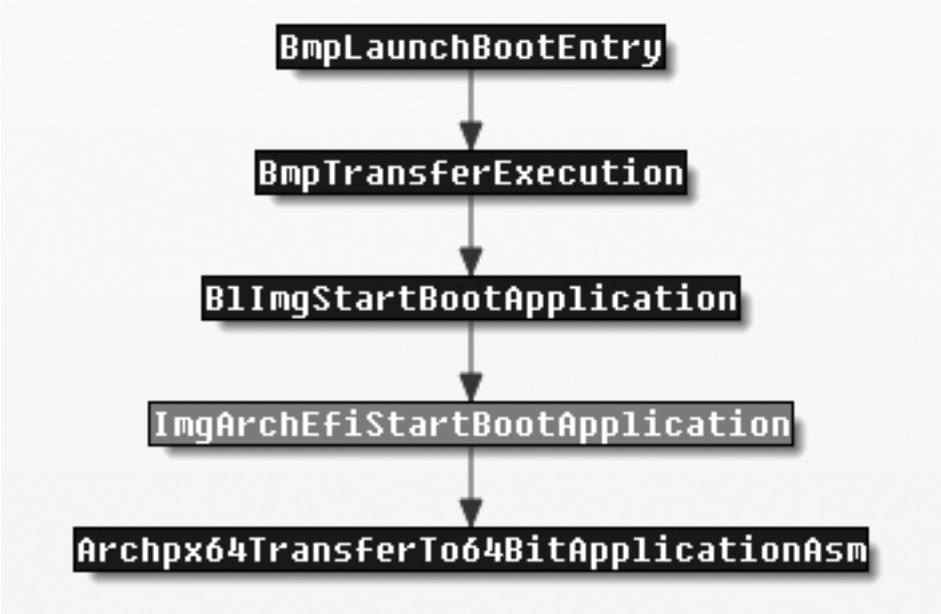


Рисунок 14-10. Поток графа вызовов от `BmpLaunchBootEntry()` *до* `Archpx64TransferTo64BitApplicationAsm()`

После этой критической точки весь поток выполнения передаётся `winload.efi`, который отвечает за загрузку и инициализацию ядра Windows. До этого момента выполнение происходит в среде UEFI поверх загрузочных служб и в плоской модели физической памяти.

> **Примечание.** Если Secure Boot отключена, вредоносный код может изменять память на этом этапе процесса загрузки произвольным образом, поскольку модули режима ядра ещё не защищены технологией Windows Kernel Patch Protection (KPP), также известной как PatchGuard. PatchGuard инициализируется лишь на более поздних этапах процесса загрузки. После активации она значительно затрудняет вредоносные изменения модулей ядра.

Загрузчик Windows

Загрузчик Windows выполняет следующие действия по настройке:

• инициализирует отладчик ядра, если ОС загружается в режиме отладки, включая режим отладки гипервизора;
• заключает загрузочные службы UEFI в абстракции HAL для последующего использования кодом режима ядра Windows и вызывает Exit Boot Services;
• проверяет центральный процессор на наличие возможностей поддержки гипервизора Hyper-V и настраивает их, если они поддерживаются;
• проверяет политики Virtual Secure Mode (VSM) и DeviceGuard, только в Windows 10;
• проверяет целостность самого ядра и компонентов Windows, затем передаёт управление ядру.

Загрузчик Windows начинает выполнение с процедуры `OslMain()`, показанной в листинге 14-4, которая выполняет все описанные действия.

```
__int64 __fastcall OslpMain(__int64 a1)
{
  __int64 v1; // rbx@1
  unsigned int v2; // eax@3
  __int64 v3; // rdx@3
  __int64 v4; // rcx@3
  __int64 v5; // r8@3
  __int64 v6; // rbx@5
  unsigned int v7; // eax@7
  __int64 v8; // rdx@7
  __int64 v9; // rcx@7
  __int64 v10; // rdx@9
  __int64 v11; // rcx@9
  unsigned int v12; // eax@10
  char v14; // [rsp+20h] [rbp-18h]@1
  int v15; // [rsp+2Ch] [rbp-Ch]@1
  char v16; // [rsp+48h] [rbp+10h]@3
  v1 = a1;
  BlArchCpuId(0x80000001, 0i64, &v14);
  if ( !(v15 & 0x100000) )
    BlArchGetCpuVendor();
  v2 = OslPrepareTarget(v1, &v16);
  LODWORD(v5) = v2;
  if ( (v2 & 0x80000000) == 0 && v16 )
  {
    v6 = OslLoaderBlock;
    if ( !BdDebugAfterExitBootServices )
      BlBdStop(v4, v3, v2);
  ❶ v7 = OslFwpKernelSetupPhase1(v6);
    LODWORD(v5) = v7;
    if ( (v7 & 0x80000000) == 0 )
    {
      ArchRestoreProcessorFeatures(v9, v8, v7);
      OslArchHypervisorSetup(1i64, v6);
    ❷ LODWORD(v5) = BlVsmCheckSystemPolicy(1i64);
      if ( (signed int)v5 >= 0 )
      {
        if ( (signed int)OslVsmSetup(1i64, 0xFFFFFFFFi64, v6) >= 0
          ❸ || (v12 = BlVsmCheckSystemPolicy(2i64), v5 = v12,
                (v12 & 0x80000000) == 0 ) )
        {
          BlBdStop(v11, v10, v5);
        ❹ OslArchTransferToKernel(v6, OslEntryPoint);
          while ( 1 )
            ;
        }
      }
    }
  }
}
```

Листинг 14-4. Декомпилированная функция `OslMain()` *(Windows 10)*

Загрузчик Windows начинает с настройки адресного пространства памяти ядра вызовом функции `OslBuildKernelMemoryMap()` (рисунок 14-11). Затем он готовится к загрузке ядра вызовом функции `OslFwpKernelSetupPhase1()` ❶. Функция `OslFwpKernelSetupPhase1()` вызывает `EfiGetMemoryMap()`, чтобы получить указатель на настроенную ранее структуру `EFI_BOOT_SERVICE`, а затем сохраняет его в глобальной переменной для будущих операций из режима ядра через службы HAL.

После этого процедура `OslFwpKernelSetupPhase1()` вызывает функцию EFI `ExitBootServices()`. Эта функция уведомляет операционную систему о том, что она скоро получит полное управление; такой обратный вызов позволяет выполнить последние настройки перед переходом в ядро.

Проверки политики загрузки VSM реализованы в процедуре `BlVsmCheckSystemPolicy` ❷❸, которая проверяет среду на соответствие политике Secure Boot и читает переменную UEFI `VbsPolicy` в память, заполняя структуру `BlVsmpSystemPolicy`.

Наконец, поток выполнения достигает ядра операционной системы, в нашем случае образа `ntoskrnl.exe`, ❹ посредством `OslArchTransferToKernel()` (листинг 14-5).

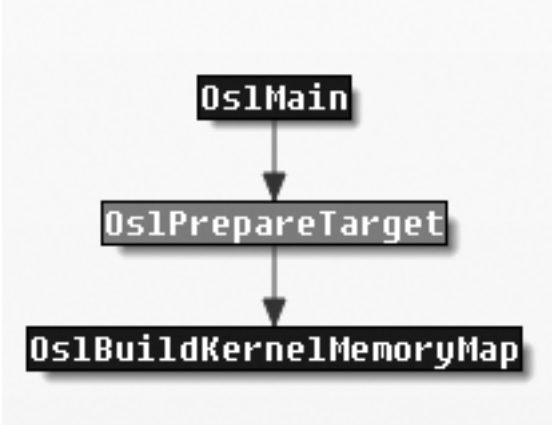


Рисунок 14-11. Поток графа вызовов от `OslMain()` *до* `OslBuildKernelMemoryMap()`

```
.text:0000000180123C90 OslArchTransferToKernel proc near
.text:0000000180123C90                 xor     esi, esi
.text:0000000180123C92                 mov     r12, rcx
.text:0000000180123C95                 mov     r13, rdx
.text:0000000180123C98                 wbinvd
.text:0000000180123C9A                 sub     rax, rax
.text:0000000180123C9D                 mov     ss, ax
.text:0000000180123CA0                 mov     rsp, cs:OslArchKernelStack
.text:0000000180123CA7                 lea     rax, OslArchKernelGdt
.text:0000000180123CAE                 lea     rcx, OslArchKernelIdt
.text:0000000180123CB5                 lgdt    fword ptr [rax]
.text:0000000180123CB8                 lidt    fword ptr [rcx]
.text:0000000180123CBB                 mov     rax, cr4
.text:0000000180123CBE                 or      rax, 680h
.text:0000000180123CC4                 mov     cr4, rax
.text:0000000180123CC7                 mov     rax, cr0
.text:0000000180123CCA                 or      rax, 50020h
.text:0000000180123CD0                 mov     cr0, rax
.text:0000000180123CD3                 xor     ecx, ecx
.text:0000000180123CD5                 mov     cr8, rcx
.text:0000000180123CD9                 mov     ecx, 0C0000080h
.text:0000000180123CDE                 rdmsr
.text:0000000180123CE0                 or      rax, cs:OslArchEferFlags
.text:0000000180123CE7                 wrmsr
.text:0000000180123CE9                 mov     eax, 40h
.text:0000000180123CEE                 ltr     ax
```

```
.text:0000000180123CF1                  mov     ecx, 2Bh
.text:0000000180123CF6                  mov     gs, ecx
.text:0000000180123CF8                  assume gs:nothing
.text:0000000180123CF8                  mov     rcx, r12
.text:0000000180123CFB                  push    rsi
.text:0000000180123CFC                  push    10h
.text:0000000180123CFE                  push    r13
.text:0000000180123D00                  retfq
.text:0000000180123D00 OslArchTransferToKernel endp
```

Листинг 14-5. Дизассемблированная функция `OslArchTransferToKernel()`

Эта функция упоминалась в предыдущих главах, поскольку некоторые буткиты, например Gapz, перехватывают её, чтобы внедрить собственные перехваты в образ ядра.

Преимущества безопасности микропрограммы UEFI

Как мы видели, устаревшие буткиты на основе MBR и VBR не способны получить управление в схеме загрузки UEFI, поскольку заражаемый ими загрузочный код больше не выполняется в потоке процесса загрузки UEFI. Однако наибольшее влияние UEFI на безопасность обусловлено поддержкой технологии Secure Boot. Secure Boot меняет правила заражения руткитами и буткитами, поскольку не позволяет злоумышленникам изменять какие-либо загрузочные компоненты до ОС, если только они не найдут способ обойти Secure Boot.

Более того, недавно выпущенная Intel технология Boot Guard отмечает очередной этап развития Secure Boot. Boot Guard - аппаратная технология защиты целостности, пытающаяся защитить систему ещё до запуска Secure Boot. Вкратце, Boot Guard позволяет производителю платформы установить криптографические ключи, поддерживающие целостность Secure Boot.

Другая современная технология, поставляемая начиная с поколения центральных процессоров Intel Skylake, - BIOS Guard, защищающая платформы от изменений флеш-хранилища микропрограммы. Даже если злоумышленник получает доступ к флеш-памяти, BIOS Guard может защитить её от установки вредоносного имплантата, тем самым также предотвращая выполнение вредоносного кода во время загрузки.

Эти защитные технологии непосредственно повлияли на направление развития современных буткитов, заставив разработчиков вредоносных программ совершенствовать свои подходы для противодействия защите.

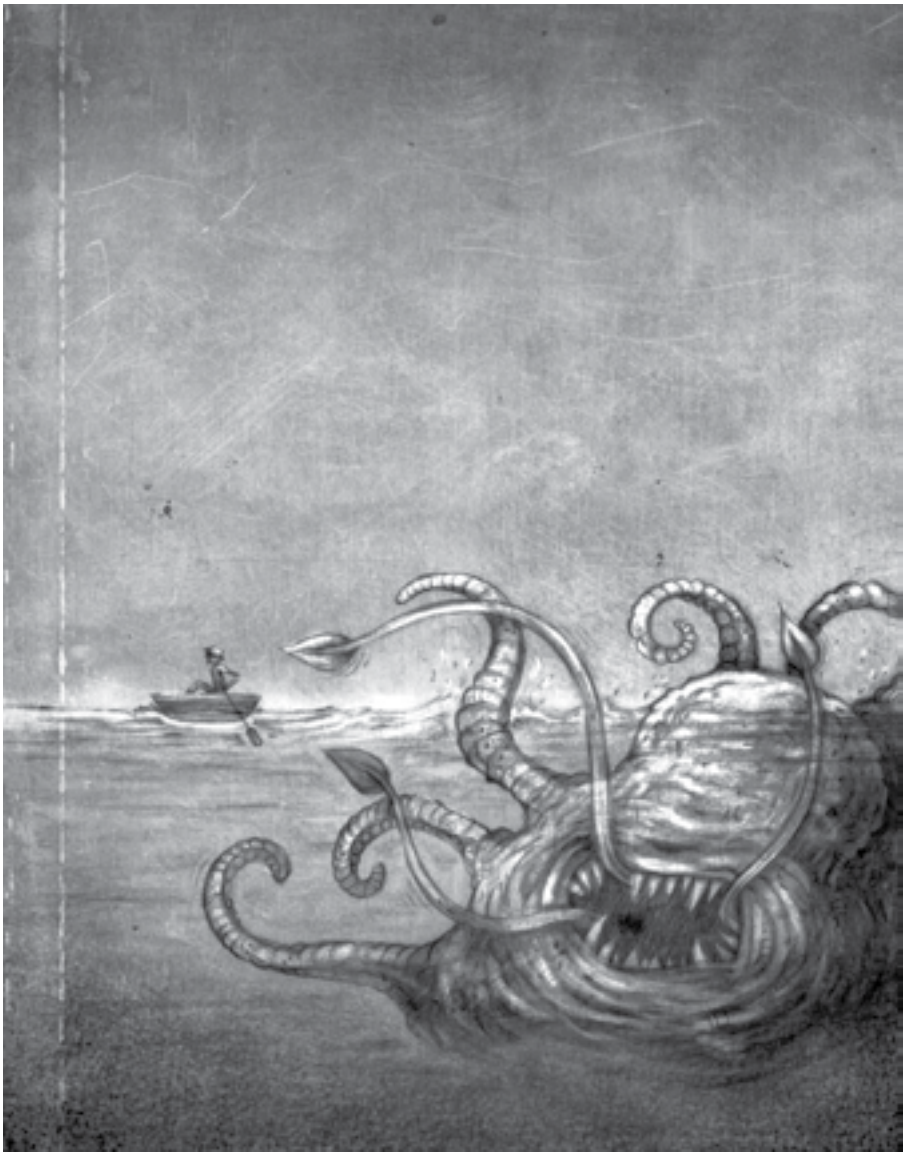
Заключение

Переход современных ПК на микропрограмму UEFI, начавшийся с Microsoft Windows 7, стал первым шагом к изменению потока процесса загрузки и перестройке экологии буткитов. Методы, полагавшиеся на прерывания устаревшей BIOS для передачи управления вредоносному коду, устарели, поскольку такие структуры исчезли из систем, загружающихся через UEFI.

Технология Secure Boot полностью изменила правила игры, поскольку непосредственно изменять такие компоненты загрузчика, как `bootmgfw.efi` и `winload.efi`, стало невозможно.

Теперь весь поток процесса загрузки пользуется доверием и проверяется микропрограммой с аппаратной поддержкой. Злоумышленникам приходится погружаться глубже в микропрограмму, искать и эксплуатировать уязвимости BIOS для обхода этих защитных возможностей UEFI. В главе 16 будет представлен обзор современного ландшафта уязвимостей BIOS, но сначала глава 15 затронет развитие угроз руткитов и буткитов в свете атак на микропрограммы.

Глава 15. Современные буткиты UEFI



В наши дни редко удаётся встретить в реальных условиях новый и новаторский руткит или буткит. Большинство вредоносных угроз переместилось в пользовательский режим, поскольку современные технологии безопасности сделали старые методы руткитов и буткитов устаревшими. Такие средства безопасности, как политика Microsoft подписывания кода режима ядра, PatchGuard, Virtual Secure Mode (VSM) и Device Guard, накладывают ограничения на модификации кода режима ядра и повышают порог сложности разработки руткитов режима ядра.

Переход к системам на основе UEFI и распространение схемы Secure Boot изменили облик разработки буткитов, увеличив стоимость разработки руткитов и буткитов режима ядра. Подобно тому как введение политики подписывания кода режима ядра заставило разработчиков вредоносного ПО искать новые функциональные возможности буткитов вместо поиска способов развития руткитов для обхода защиты подписывания кода, последние изменения побудили исследователей безопасности обратить внимание на прошивку BIOS.

С точки зрения злоумышленника следующий логичный шаг при заражении системы состоит в том, чтобы переместить точку заражения ниже по программному стеку, за код, инициализируемый при загрузке, и проникнуть в BIOS (это показано на рисунке 15-1). BIOS запускает начальные этапы настройки оборудования в процессе загрузки, а значит, уровень прошивки BIOS является последней границей перед аппаратным обеспечением.

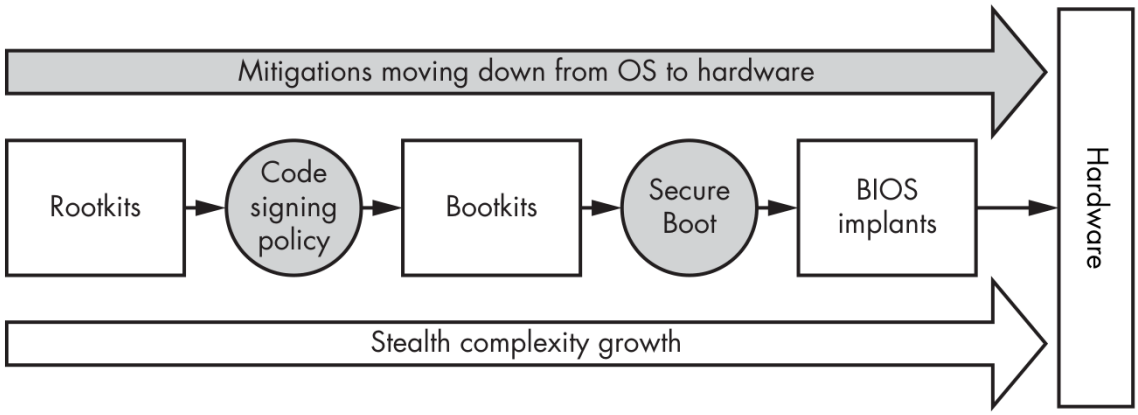


Рисунок 15-1. Развитие руткитов и буткитов в ответ на развитие средств безопасности

Уровень постоянства, достижимый в BIOS, сильно отличается от всего, что мы до сих пор рассматривали в этой книге. Имплантаты прошивки способны пережить переустановку операционной системы и даже замену жёсткого диска, а значит, заражение руткитом потенциально остаётся активным на протяжении всего срока службы заражённого оборудования.

Эта глава посвящена заражению прошивки UEFI буткитами, поскольку на момент написания книги большая часть системных прошивок для платформ x86 основана на спецификациях UEFI. Однако прежде чем перейти к современным методам заражения прошивки UEFI, мы ради исторической перспективы рассмотрим несколько буткитов для устаревшей BIOS.

Обзор исторических угроз для BIOS

Вредоносное ПО для BIOS всегда считалось сложным, а с учётом всех современных функций BIOS, с которыми вредоносной программе приходится работать или которые ей приходится обходить, сегодня это верно как никогда. Ещё до того, как поставщики начали относиться к нему серьёзно, у вредоносного ПО для BIOS была богатая история. Мы подробно рассмотрим пару ранних примеров вредоносного ПО для BIOS, а затем кратко перечислим основные особенности всех угроз, обнаруженных со времени появления первой вредоносной программы, заражавшей BIOS, - WinCIH.

WinCIH - первая вредоносная программа, нацеленная на BIOS

Вирус WinCIH, также известный как Chernobyl, стал первой публично известной вредоносной программой, атаковавшей BIOS. Он был разработан тайваньским студентом Чэнь Инхао, обнаружен в реальных условиях в 1998 году и очень быстро распространялся через пиратское программное обеспечение. WinCIH заражал исполняемые файлы Microsoft Windows 95 и 98; после запуска заражённого файла вирус оставался в памяти и устанавливал перехватчики файловой системы, чтобы заражать другие программы по мере обращения к ним. Этот метод делал WinCIH чрезвычайно эффективным в распространении, но наиболее разрушительной частью вируса была его попытка перезаписать память микросхемы flash BIOS на заражённой машине.

Срабатывание разрушительной полезной нагрузки WinCIH было приурочено к дате чернобыльской ядерной катастрофы, 26 апреля. Если перезапись flash BIOS проходила успешно, машина больше не могла загрузиться, пока не была восстановлена исходная BIOS. В ресурсах к этой главе (https://nostarch.com/rootkits/) можно загрузить исходный ассемблерный код WinCIH в том виде, в каком его распространял автор.

> **Примечание.** Если вам интересно подробнее узнать об обратном проектировании и архитектуре устаревшей BIOS, рекомендуем книгу Дармавана Маппатуту Салихуна (Darmawan Mappatutu Salihun), также известного как pinczakko, *BIOS Disassembly Ninjutsu Uncovered*. Электронную копию книги можно бесплатно загрузить из учётной записи автора на GitHub: https://github.com/pinczakko/BIOS-Disassembly-Ninjutsu-Uncovered.

Mebromi

После WinCIH следующая обнаруженная в реальных условиях вредоносная программа, атаковавшая BIOS, появилась лишь в 2011 году. Она была известна под именами Mebromi и BIOSkit и нацеливалась на машины с устаревшей BIOS. К тому времени исследователи безопасности разработали и представили на конференциях и в электронных журналах идеи заражения BIOS и доказательства концепции (proof of concept, PoC) таких атак. Большинство этих идей было трудно воплотить в реальном самораспространяющемся вредоносном ПО, однако заражение BIOS считалось интересным теоретическим направлением для целевых атак, которым требовалось длительное постоянство заражения.

Вместо реализации этих теоретических приёмов Mebromi использовал заражение BIOS как простой способ постоянно поддерживать заражение MBR при загрузке системы. Mebromi мог восстанавливать заражение даже после возврата MBR в исходное состояние или переустановки ОС и даже после замены жёсткого диска: часть заражения в BIOS сохранялась и повторно заражала остальную систему.

На начальном этапе Mebromi использовал исходное программное обеспечение для обновления BIOS, чтобы доставлять вредоносные обновления прошивки, в частности в системах Award BIOS, одного из самых популярных на тот момент поставщиков BIOS (в 1998 году его приобрела Phoenix BIOS). Во время существования Mebromi средств защиты от вредоносных обновлений устаревшей BIOS было мало. Как и WinCIH, Mebromi модифицировал обработчик системного прерывания управления (System Management Interrupt, SMI) процедуры обновления BIOS, чтобы доставить изменённое вредоносное обновление BIOS. Поскольку таких мер, как подписывание прошивки, в то время не существовало, заражение было сравнительно простым; вы можете самостоятельно изучить этот классический образец вредоносного ПО по ссылкам на ресурсы на странице https://nostarch.com/rootkits/.

> **Примечание.** Если вам интересно подробнее узнать о Mebromi, обстоятельный анализ приведён в статье Чжитао Чжоу (Zhitao Zhou) «A New BIOS Rootkit Spreads in China»: https://www.virusbulletin.com/virusbulletin/2011/10/new-bios-rootkit-spreads-china/.

Обзор других угроз и средств противодействия

Теперь рассмотрим временную шкалу угроз BIOS, обнаруженных в реальных условиях, и связанную с ними деятельность исследователей безопасности. Как видно на рисунке 15-2, наиболее активный период обнаружения руткитов и имплантатов BIOS начался в 2013 году и продолжается по настоящее время.

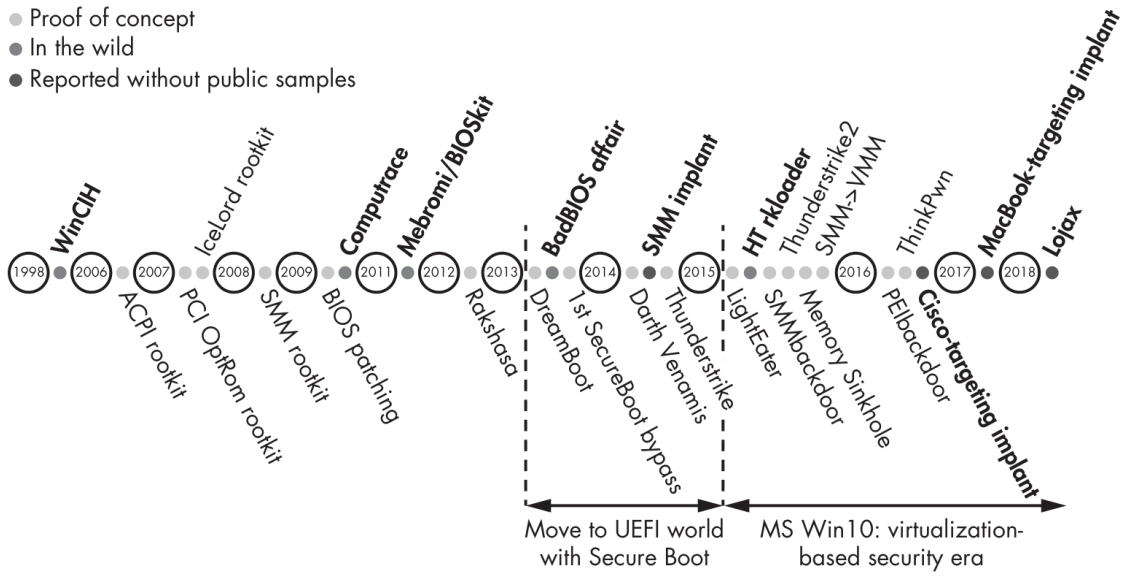


Рисунок 15-2. Временная шкала угроз BIOS

Чтобы дать краткое представление об эволюции буткитов BIOS, в таблице 15-1 мы перечислили основные события, связанные с каждой угрозой, в хронологическом порядке. В левом столбце показана эволюция PoC, разработанных исследователями для демонстрации проблем безопасности, а в среднем столбце перечислены реальные образцы угроз BIOS, найденные в естественной среде. В третьем столбце приведены ресурсы для дальнейшего чтения.

Многие из этих угроз эксплуатируют обработчики SMI, отвечающие за взаимодействие аппаратного обеспечения с ОС и выполняемые в режиме системного управления (System Management Mode, SMM). Для целей этой главы мы приводим краткое описание наиболее часто эксплуатируемых уязвимостей обработчиков SMI, применяемых для заражения BIOS. Более подробное обсуждение различных уязвимостей прошивки UEFI приведено в главе 16.

Таблица 15-1. Историческая временная шкала руткитов BIOS

Эволюция PoC-буткитов BIOS	Эволюция угроз буткитов BIOS	Дополнительные ресурсы
	WinCIH, 1998 Первая известная вредоносная программа, атаковавшая BIOS из ОС	
Руткит ACPI, 2006 Первый руткит на основе ACPI (Advanced Configuration and Power Interface), представленный Джоном Хисманом (John Heasman) на Black Hat		«Implementing and Detecting an ACPI BIOS Rootkit», Black Hat 2006, https://www.blackhat.com/presentations/bh-europe-06/bh-eu-06-Heasman.pdf

Таблица 15-1. Историческая временная шкала руткитов BIOS (продолжение)

Эволюция PoC-буткитов BIOS	Эволюция угроз буткитов BIOS	Дополнительные ресурсы
Руткит PCI Option ROM, 2007 Первый руткит Option ROM для PCI, представленный Джоном Хисманом на Black Hat		«Implementing and Detecting a PCI Rootkit», Black Hat 2007, https://www.blackhat.com/presentations/bh-dc-07/Heasman/Paper/bh-dc-07-Heasman-WP.pdf
Руткит IceLord, 2007 Китайский PoC-буткит BIOS; двоичные файлы были публично опубликованы на форуме исследователя		
Руткит SMM, 2007 Первый известный PoC руткита SMM от Родриго Бранко (Rodrigo Branco), показанный на конференции H2HC в Бразилии		«System Management Mode Hack Using SMM for 'Other Purposes'», http://phrack.org/issues/65/7.html
Руткит SMM, 2008 Второй известный PoC руткита SMM, показанный на Black Hat		«SMM Rootkits: A New Breed of OS Independent Malware», Black Hat 2008, http://dl.acm.org/citation.cfm?id=1460892; см. также http://phrack.org/issues/65/7.html
Исправление BIOS, 2009 Несколько исследователей опубликовали статьи о модификации образов BIOS		
	Computrace, 2009 Первое известное исследование обратного проектирования, опубликованное Анибалом Сакко (Anibal Sacco) и Альфредо Ортегой (Alfredo Ortega)	«Deactivate the Rootkit», Black Hat 2009, https://www.coresecurity.com/corelabs-research/publications/deactivate-rootkit/
	Mebromi, 2011 Первый обнаруженный в реальных условиях буткит BIOS; Mebromi использует идеи, сходные с IceLord	«Mebromi: The First BIOS Rootkit in the Wild», https://www.webroot.com/blog/2011/09/13/mebromi-the-first-bios-rootkit-in-the-wild/
Rakshasa, 2012 PoC постоянного руткита BIOS, представленный Джонатаном Броссаром (Jonathan Brossard) на Black Hat		
DreamBoot, 2013 Первый публичный PoC буткита UEFI		«UEFI and Dreamboot», HiTB 2013, https://conference.hitb.org/hitbsecconf2013ams/materials/D2T1%20-%20Sebastien%20Kaczmarek%20-%20Dreamboot%20UEFI%20Bootkit.pdf
	BadBIOS, 2013 Предполагаемый постоянный руткит BIOS, о котором сообщил Драгос Руиу (Dragos Ruiu)	«Meet 'badBIOS,' the Mysterious Mac and PC Malware That Jumps Airgaps», https://arstechnica.com/information-technology/2013/10/meet-badbios-the-mysterious-mac-and-pc-malware-that-jumps-air-gaps/
Буткит x86 в памяти, 2013 PoC основанного на UEFI буткита, работающего в памяти		«x86 Memory Bootkit», https://github.com/AaLI86/retroware/tree/master/MemoryBootkit

(продолжение следует)

Таблица 15-1. Историческая временная шкала руткитов BIOS (продолжение)

Эволюция PoC-буткитов BIOS	Эволюция угроз буткитов BIOS	Дополнительные ресурсы
Обход Secure Boot из BIOS, 2013 Первый опубликованный обход Secure Boot для Microsoft Windows 8		«A Tale of One Software Bypass of Windows 8 Secure Boot», Black Hat 2013, http://c7zero.info/stuff/Windows8SecureBoot_Bulygin-Furtak-Bazhniuk_BHUSA2013.pdf
Реализация и последствия скрытого бэкдора жёсткого диска, 2013 Йонас Заддах (Jonas Zaddach) и другие демонстрируют PoC бэкдора прошивки жёсткого диска		«Implementation and implications of a stealth hard drive backdoor», Annual Computer Security Applications Conference (ACSAC) 2013, http://www.syssec-project.eu/m/page-media/3/acsac13_zaddach.pdf
Darth Venamis, 2014 Рафал Войтчук (Rafal Wojtczuk) и Кори Калленберг (Corey Kallenberg) обнаружили уязвимость S3BootScript (VU#976132)	**Первые сообщения о предположительно поддерживаемом государством имплантате на основе SMM опубликованы**	«VU#976132», https://www.kb.cert.org/vuls/id/976132/
Thunderstrike, 2014 Атака на устройства Apple с помощью вредоносной Option ROM через порт Thunderbolt, представленная Траммеллом Хадсоном (Trammell Hudson) на конференции 31C3		«Thunderstrike: EFI Bootkits for Apple MacBooks», https://events.ccc.de/congress/2014/Fahrplan/events/6128.html
LightEater, 2015 Основанный на UEFI руткит, демонстрирующий раскрытие конфиденциальных сведений из памяти в прошивке, представленный Кори Калленбергом и Ксено Ковой (Xeno Kovah)		
	Hacking Team rkloader, 2015 Первая известная утечка буткита прошивки UEFI коммерческого уровня, раскрытого благодаря rkloader компании Hacking Team	
SmmBackdoor, 2015 Первый публичный PoC буткита прошивки UEFI, выпущенный с исходным кодом на GitHub		«Building Reliable SMM Backdoor for UEFI-Based Platforms», http://blog.cr4.sh/2015/07/building-reliable-smm-backdoor-for-uefi.html
Thunderstrike2, 2015 Демонстрация смешанного подхода к атаке с применением эксплойтов Darth Venamis и Thunderstrike		«Thunderstrike 2: Sith Strike - A MacBook Firmware Worm», Black Hat 2015, http://legbacore.com/Research_files/ts2-blackhat.pdf
Memory Sinkhole, 2015 Обнаруженная Кристофером Домасом (Christopher Domas) уязвимость в Advanced Programmable Interrupt Controller (APIC), которая могла позволить злоумышленнику атаковать используемую ОС область памяти SMM; злоумышленник мог воспользоваться этой уязвимостью для установки руткита		«The Memory Sinkhole», Black Hat 2015, https://github.com/xoreaxeaxeax/sinkhole/

(продолжение следует)

Таблица 15-1. Историческая временная шкала руткитов BIOS (продолжение)

Эволюция PoC-буткитов BIOS	Эволюция угроз буткитов BIOS	Дополнительные ресурсы
Повышение привилегий из SMM в VMM, 2015 Группа исследователей Intel представила PoC повышения привилегий из SMM до гипервизора и продемонстрировала PoC раскрытия областей памяти, защищённых VMM в MS Hyper-V и Xen		«Attacking Hypervisors via Firmware and Hardware», Black Hat 2015, http://2015.zeronights.org/assets/files/10-Matrosov.pdf
PeiBackdoor, 2016 Первый публично выпущенный PoC руткита UEFI, работавшего на этапе PEI (Pre-EFI Initialization) загрузки; выпущен с исходным кодом на GitHub	**Имплантат, нацеленный на маршрутизаторы Cisco, 2016** Сообщения о предположительно поддерживаемом государством имплантате для BIOS маршрутизаторов Cisco	«PeiBackdoor», https://github.com/Cr4sh/PeiBackdoor/
ThinkPwn, 2016 Уязвимость повышения привилегий до SMM, первоначально обнаруженная Дмитрием Олексюком, также известным как Cr4sh, в серии ноутбуков ThinkPad		«Exploring and Exploiting Lenovo Firmware Secrets», http://blog.cr4.sh/2016/06/exploring-and-exploiting-lenovo.html
	Имплантат, нацеленный на MacBook, 2017 Сообщения о предположительно поддерживаемом государством имплантате UEFI, нацеленном на ноутбуки Apple	
	Имплантат LoJax, 2018 Руткит UEFI, обнаруженный в реальных условиях исследователями ESET	«LOJAX», https://www.welivesecurity.com/wp-content/uploads/2018/09/ESET-LoJax.pdf

Прошивка BIOS всегда была сложной целью для исследователей как из-за недостатка информации, так и из-за трудности модификации или инструментирования BIOS путём добавления нового кода, выполняемого в процессе загрузки. Однако с 2013 года мы наблюдаем более масштабные усилия исследовательского сообщества в области безопасности, направленные на поиск новых эксплойтов и демонстрацию слабостей и атак на недавно появившиеся средства безопасности, такие как Secure Boot.

Рассматривая эволюцию реального вредоносного ПО для BIOS, можно заметить, что лишь очень немногие PoC угроз BIOS действительно превратились в распространённое направление имплантатов на основе прошивки, а большинство использовалось для целевых атак. Здесь мы сосредоточимся на подходах к заражению BIOS постоянным руткитом, способным пережить не только перезагрузки операционной системы, но и любые изменения оборудования, кроме материнской платы, при заражённой flash-памяти прошивки BIOS. Многочисленные сообщения СМИ о доступности имплантатов UEFI субъектам, поддерживаемым государством, позволяют предположить, что такие имплантаты являются технической реальностью и существуют уже значительное время.

Всё оборудование имеет прошивку

Прежде чем углубляться в особенности руткитов и буткитов UEFI, рассмотрим современное оборудование x86 и то, как внутри него хранятся различные виды прошивки. В наши дни всё оборудование поставляется с той или иной прошивкой; даже аккумуляторы ноутбуков имеют прошивку, которую обновляет операционная система, чтобы точнее измерять параметры аккумулятора и его использование.

> **Примечание.** Чарли Миллер (Charlie Miller) был первым исследователем, публично сосредоточившим внимание на аккумуляторах ноутбуков. На Black Hat 2011 он представил доклад «Battery Firmware Hacking»: https://media.blackhat.com/bh-us-11/Miller/BH_US_11_Miller_Battery_Firmware_Public_Slides.pdf.

Каждая единица прошивки представляет собой область, в которой злоумышленник может хранить и выполнять код, и тем самым создаёт возможность для вредоносного имплантата. Большинство современных настольных компьютеров и ноутбуков имеет следующие виды прошивки:

• прошивка UEFI (BIOS) и прошивка Manageability Engine (например, Intel ME);
• прошивка жёсткого диска (HDD/SSD);
• прошивка периферийных устройств (например, сетевых адаптеров);
• прошивка графической платы (GPU).

Несмотря на множество очевидных векторов атаки, атаки на прошивку не распространены среди киберпреступников, которые обычно предпочитают атаки, способные охватить широкий круг жертв. Поскольку прошивки разных систем обычно различаются, большинство известных случаев компрометации прошивки были целевыми атаками, а не PoC.

Например, первый обнаруженный в реальных условиях имплантат прошивки жёсткого диска был найден исследователями «Лаборатории Касперского» в начале 2015 года. «Лаборатория Касперского» назвала создателей этой вредоносной программы Equation Group и классифицировала их как субъекта угроз государственного уровня.

Согласно «Лаборатории Касперского», обнаруженная ими вредоносная программа могла заражать определённые модели жёстких дисков, включая некоторые весьма распространённые марки. Ни одна из целевых моделей накопителей не предъявляла требований аутентификации к обновлениям прошивки, что и сделало такую атаку осуществимой.

В этой атаке модуль заражения жёсткого диска `nls933w.dll`, обнаруживаемый «Лабораторией Касперского» как Trojan.Win32.EquationDrug.c, доставлял изменённую прошивку через интерфейс команд подключения накопителей Advanced Technology Attachment (ATA). Доступ к командам ATA позволял злоумышленникам перепрограммировать или обновлять прошивку HDD/SSD при наличии лишь слабой проверки или аутентификации обновления. Имплантат прошивки такого рода может подменять секторы диска на уровне прошивки или изменять потоки данных, перехватывая запросы чтения или записи, например для доставки изменённых версий MBR. Эти имплантаты прошивки жёсткого диска находятся низко в стеке прошивок, поэтому обнаружить их очень трудно.

Вредоносное ПО, нацеленное на прошивку, обычно доставляет имплантаты прошивки, перепрошивая вредоносные обновления прошивки посредством обычного процесса обновления ОС. Это означает, что главным образом оно воздействует на жёсткие диски, которые не поддерживают аутентификацию обновлений прошивки, а просто устанавливают новую прошивку в неизменном виде. В следующих разделах мы сосредоточимся на руткитах и имплантатах на основе UEFI, но полезно знать, что BIOS является не единственным местом для разработки постоянных имплантатов прошивки.

Уязвимости прошивки UEFI

В интернете имеется множество обсуждений и примеров различных видов уязвимостей современных операционных систем, но обсуждения уязвимостей прошивки UEFI встречаются гораздо реже. Здесь мы перечислим виды значимых для руткитов уязвимостей, публично раскрытых за последние несколько лет. Большинство из них представляют собой уязвимости повреждения памяти и внешних вызовов SMM, способные привести к выполнению произвольного кода, когда процессор находится в SMM. Злоумышленник может использовать уязвимости этих видов, чтобы обойти биты защиты BIOS и получить возможность произвольной записи в области SPI flash-памяти и чтения из них в некоторых системах. Подробнее мы рассмотрим это в главе 16, а пока приведём пару характерных примеров:

ThinkPwn (LEN-8324). Эксплойт выполнения произвольного кода SMM для BIOS нескольких поставщиков. Эта уязвимость позволяет злоумышленнику отключить защиту flash-памяти от записи и изменить прошивку платформы.

Aptiocalypsis (INTEL-SA-00057). Эксплойт выполнения произвольного кода SMM для прошивок на основе AMI, позволяющий злоумышленнику отключить биты защиты flash-памяти от записи и изменить прошивку платформы.

Любая из этих проблем может позволить злоумышленнику установить постоянные руткиты или имплантаты в оборудование жертвы. Многие уязвимости такого рода основаны либо на возможности злоумышленника обойти биты защиты памяти, либо на том, что эти биты не включены или неэффективны.

(Не)эффективность битов защиты памяти

Большинство распространённых технологий, защищающих SPI flash-память от произвольной записи, основано на битах защиты памяти - довольно старом защитном подходе, предложенном Intel десять лет назад. Биты защиты памяти являются единственным видом защиты, доступным для дешёвого оборудования на основе UEFI, используемого на рынке интернета вещей (Internet of Things, IoT). Уязвимость SMM, позволяющая злоумышленнику повысить привилегии, получить доступ к SMM и выполнить произвольный код, даст ему возможность изменить эти биты. Рассмотрим их подробнее:

BIOSWE. Бит разрешения записи BIOS (BIOS Write Enable), обычно установленный в `0` и изменяемый SMM на `1` для аутентификации прошивки или разрешения обновления.

BLE. Бит разрешения блокировки BIOS (BIOS Lock Enable), который по умолчанию должен быть установлен в `1`, чтобы защищать области BIOS в SPI flash-памяти от произвольной модификации. Злоумышленник с привилегиями SMM может изменить этот бит.

SMM_BWP. Бит защиты BIOS от записи в SMM (SMM BIOS Write Protection) должен быть установлен в `1`, чтобы защищать SPI flash-память от записи за пределами SMM. В 2015 году исследователи Кори Калленберг и Рафал Войтчук обнаружили уязвимость состояния гонки (VU#766164), при которой этот неустановленный бит мог привести к отключению бита BLE.

Защищённые диапазоны SPI PRx. Регистры PR (PR0-PR5) защищают от модификации не всю область BIOS, но предоставляют некоторую гибкость настройки политик чтения или записи для конкретных областей BIOS. Регистры PR защищены от произвольных изменений со стороны SMM. Если все биты безопасности установлены, а регистры PR настроены правильно, злоумышленнику может быть чрезвычайно трудно изменить SPI flash-память. Эти биты безопасности настраиваются

на этапе DXE, который мы рассматривали в главе 14. Если вам интересно, пример кода этапа инициализации платформы можно найти в репозитории Intel EDK2 на GitHub.

Проверка битов защиты

Проверить, включены и эффективны ли биты защиты BIOS, можно с помощью платформы оценки безопасности Chipsec, разработанной Intel Security Center of Excellence (ныне IPAS, Intel Product Assurance and Security) и выпущенной с открытым исходным кодом.

В главе 19 мы рассмотрим Chipsec с криминалистической точки зрения, а пока воспользуемся только модулем `bios_wp` (https://github.com/chipsec/chipsec/blob/master/chipsec/modules/common/bios_wp.py), который проверяет правильность настройки средств защиты и то, защищают ли они BIOS. Модуль `bios_wp` считывает фактические значения битов защиты и выводит состояние защиты SPI flash-памяти, предупреждая пользователя о неправильной настройке.

Чтобы использовать модуль `bios_wp`, установите Chipsec, а затем запустите его следующей командой:

```
chipsec_main.py -m common.bios_wp
```

В качестве примера мы выполнили эту проверку на уязвимой платформе MSI Cubi2 с процессором Intel седьмого поколения, которая на момент написания книги была довольно новым оборудованием. Результат проверки показан в листинге 15-1. Прошивка UEFI Cubi2 основана на каркасе AMI.

```
[x][ ================================================================================
[x][ Module: BIOS Region Write Protection
[x][ ================================================================================
[*] BC = 0x00000A88 << BIOS Control (b:d.f 00:31.5 + 0xDC)
[00] BIOSWE            = 0 << BIOS Write Enable
❶ [01] BLE             = 0 << BIOS Lock Enable
[02] SRC               = 2 << SPI Read Configuration
[04] TSS               = 0 << Top Swap Status
❷ [05] SMM_BWP         = 0 << SMM BIOS Write Protection
[06] BBS               = 0 << Boot BIOS Strap
[07] BILD              = 1 << BIOS Interface Lock Down
[-] BIOS region write protection is disabled!

[*] BIOS Region: Base = 0x00A00000, Limit = 0x00FFFFFF
SPI Protected Ranges
------------------------------------------------------------
❸ PRx (offset) | Value    | Base     | Limit    | WP? | RP?
------------------------------------------------------------
PR0 (84)       | 00000000 | 00000000 | 00000000 | 0   | 0
PR1 (88)       | 00000000 | 00000000 | 00000000 | 0   | 0
PR2 (8C)       | 00000000 | 00000000 | 00000000 | 0   | 0
PR3 (90)       | 00000000 | 00000000 | 00000000 | 0   | 0
PR4 (94)       | 00000000 | 00000000 | 00000000 | 0   | 0
[!] None of the SPI protected ranges write-protect BIOS region
[!] BIOS should enable all available SMM based write protection mechanisms or
configure SPI protected ranges to protect the entire BIOS region
[-] FAILED: BIOS is NOT protected completely
```

Листинг 15-1. Вывод инструмента Chipsec из модуля common.bios_wp

Вывод показывает, что BLE ❶ не включён, а значит, злоумышленник может изменять любую область памяти BIOS в микросхеме SPI flash непосредственно из режима ядра обычной ОС. Кроме того, SMM_BWP ❷ и PRx ❸ вообще не используются, что указывает на отсутствие на этой платформе какой-либо защиты SPI flash-памяти.

Если обновления BIOS для платформы, проверенной в листинге 15-1, не подписаны или поставщик оборудования не выполняет надлежащую аутентификацию обновлений, злоумышленник может легко изменить прошивку с помощью вредоносного обновления BIOS. Это может показаться аномалией, но такие простые ошибки на самом деле довольно распространены. Причины различаются: одни поставщики просто не заботятся о безопасности, а другие знают о проблемах безопасности, но не хотят разрабатывать сложные схемы обновления для дешёвого оборудования. Теперь рассмотрим другие способы заражения BIOS.

Способы заражения BIOS

В главе 14 мы изучили сложный и многогранный процесс загрузки UEFI. Для текущего обсуждения главный вывод из той главы состоит в том, что до того, как прошивка UEFI передаст управление загрузчику операционной системы и ОС начнёт загружаться, существует множество мест, где злоумышленник может спрятаться или заразить систему.

Фактически современная прошивка UEFI всё больше напоминает самостоятельную операционную систему. У неё есть собственный сетевой стек и планировщик задач, и она может напрямую взаимодействовать с физическими устройствами вне процесса загрузки: например, многие устройства взаимодействуют с ОС через драйверы UEFI DXE. На рисунке 15-3 показано, как может выглядеть заражение прошивки на разных этапах загрузки.

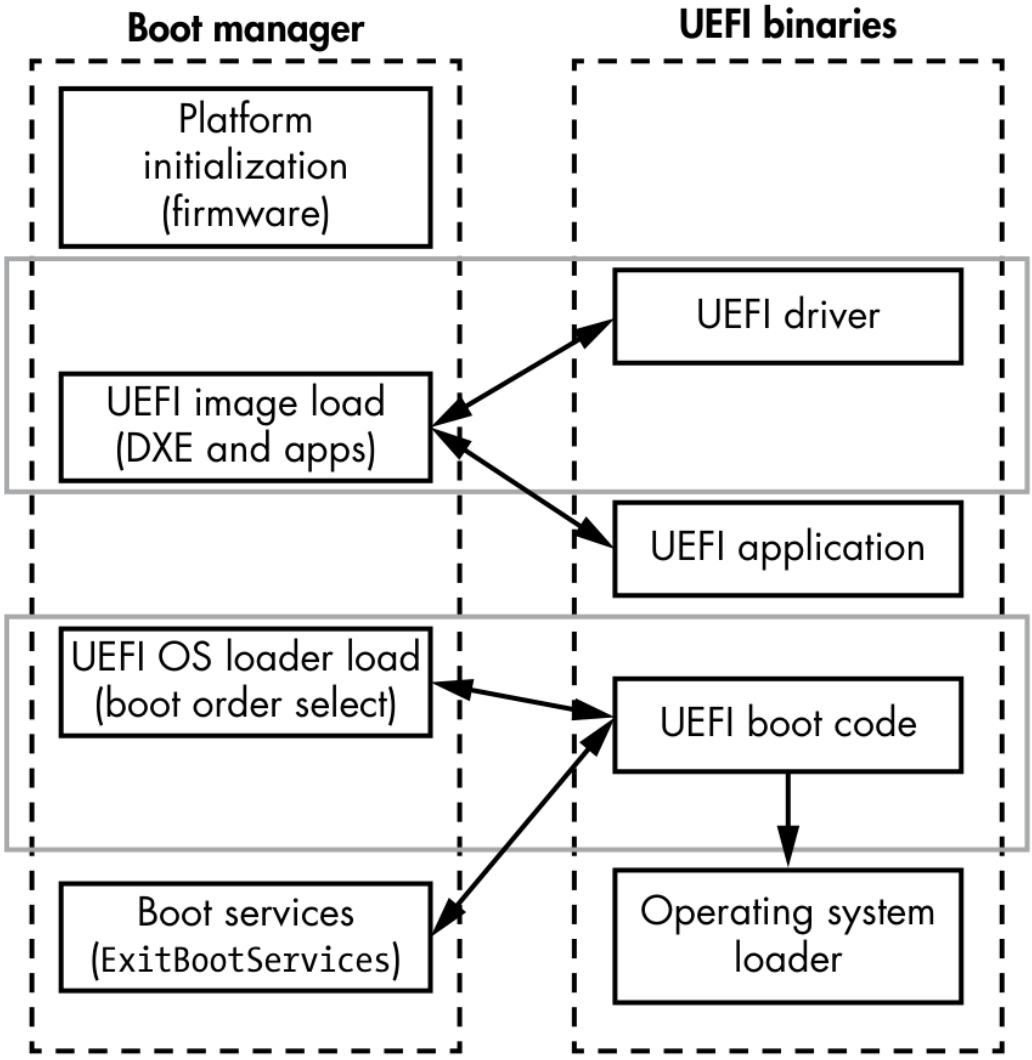


Рисунок 15-3. Процесс загрузки прошивки UEFI с точками атаки

За прошедшие годы исследователи безопасности обнаружили множество уязвимостей, позволяющих злоумышленнику изменить процесс загрузки дополнительным вредоносным кодом. К настоящему времени большинство из них исправлено, но некоторое оборудование, даже новое, всё ещё может быть уязвимо для этих старых проблем. Ниже перечислены различные способы заразить прошивку UEFI постоянным руткитом или имплантатом:

Модификация неподписанной UEFI Option ROM. Злоумышленник может изменить драйвер UEFI DXE в некоторых платах расширения, используемых для сетевых интерфейсов, накопителей и других целей, чтобы обеспечить выполнение вредоносного кода на этапе DXE.

Добавление или модификация драйвера DXE. Злоумышленник может изменить существующий драйвер DXE или добавить вредоносные драйверы DXE в образ прошивки UEFI. В результате добавленный или изменённый драйвер DXE будет выполнен на этапе DXE.

Замена Windows Boot Manager (резервного загрузчика). Злоумышленник может заменить диспетчер загрузки, то есть резервный загрузчик, в системном разделе EFI (ESP) жёсткого диска (`ESP\EFI\Microsoft\Boot\bootmgfw.efi` или `ESP\EFI\BOOT\bootx64.efi`), чтобы перехватить выполнение кода в момент, когда прошивка UEFI передаёт управление загрузчику ОС.

Добавление нового загрузчика (`bootkit.efi`). Злоумышленник может добавить ещё один загрузчик в список доступных загрузчиков, изменив переменные EFI `BootOrder`/`Boot####`, определяющие порядок загрузчиков ОС.

Из этих методов первые два наиболее интересны в контексте данной главы, поскольку они выполняют вредоносный код на этапе UEFI DXE; именно их мы рассмотрим подробнее. Последние два метода, хотя и связаны с процессом загрузки UEFI, сосредоточены на атаке загрузчиков ОС и выполнении вредоносного кода после завершения работы прошивки UEFI, поэтому далее мы их рассматривать не будем.

Модификация неподписанной UEFI Option ROM

Option ROM представляет собой прошивку расширения PCI/PCIe (ROM) с кодом x86, расположенную в PCI-совместимом устройстве. Option ROM загружается, настраивается и выполняется в процессе загрузки. Впервые Джон Хисман представил Option ROM как точку входа для скрытого заражения руткитом в 2007 году на конференции Black Hat (см. таблицу 15-1). Затем, в 2012 году, хакер, известный как Snare, представил различные методы заражения ноутбуков Apple, в том числе через Option ROM (http://ho.ax/downloads/De_Mysteriis_Dom_Jobsivs_Black_Hat_Slides.pdf). На Black Hat 2015 докладчики Траммелл Хадсон, Ксено Кова и Кори Калленберг продемонстрировали атаку Thunderstrike, проникавшую в Ethernet-адаптер Apple с изменённой прошивкой, которая загружала вредоносный код (https://www.blackhat.com/docs/us-15/materials/us-15-Hudson-Thunderstrike-2-Sith-Strike.pdf).

Option ROM содержит PE-образ, представляющий собой специальный драйвер DXE для устройства PCI. В комплекте Intel EDK2 с открытым исходным кодом (https://github.com/tianocore/edk2/) можно найти код, загружающий эти драйверы DXE; реализация загрузчика Option ROM находится в исходном файле `PciOptionRomSupport.h` в папке `PciBusDxe`. В листинге 15-2 показана функция `LoadOpRomImage()` из этого кода.

```
EFI_STATUS LoadOpRomImage (
       ❶ IN PCI_IO_DEVICE       *PciDevice,    // Экземпляр устройства PCI
       ❷ IN UINT64              RomBase        // Адрес Option ROM
);
```

Листинг 15-2. Процедура LoadOpRomImage() из EDK2

Мы видим, что функция `LoadOpRomImage()` получает два входных параметра: указатель на экземпляр устройства PCI ❶ и адрес образа Option ROM ❷. Из этого можно предположить, что функция отображает образ ROM в память и подготавливает его к выполнению. Следующая функция, `ProcessOpRomImage()`, показана в листинге 15-3.

```
EFI_STATUS ProcessOpRomImage (
     IN PCI_IO_DEVICE   *PciDevice    // Экземпляр устройства PCI
);
```

Листинг 15-3. Процедура ProcessOpRomImage() из EDK2

`ProcessOpRomImage()` отвечает за запуск процесса выполнения специального драйвера устройства, содержащегося в Option ROM. Создатели атаки Thunderstrike, использующей Option ROM как точку входа, построили свою атаку, изменив Ethernet-адаптер Thunderbolt так, чтобы он позволял подключать внешние периферийные устройства. Этот адаптер, разработанный Apple и Intel, основан на микросхеме GN2033 и предоставляет интерфейс Thunderbolt. Разобранный Ethernet-адаптер Thunderbolt, похожий на использованный в эксплойте Thunderstrike, показан на рисунке 15-4.

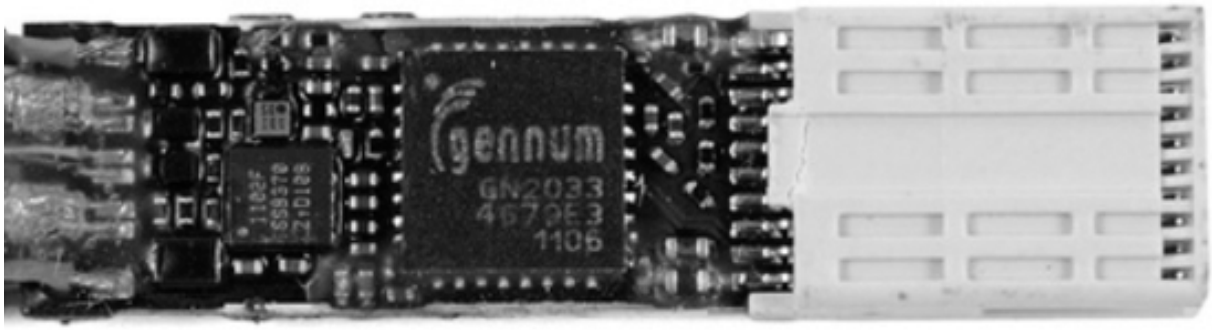


Рисунок 15-4. Разобранный Ethernet-адаптер Apple Thunderbolt

В частности, Thunderstrike загружал исходный драйвер Option ROM с дополнительным кодом, который затем выполнялся, поскольку прошивка не аутентифицировала драйвер расширения Option ROM в процессе загрузки. Эта атака была продемонстрирована на Apple MacBook, но применима и к другому оборудованию. Apple исправила проблему в своём оборудовании, однако многие другие поставщики всё ещё могут быть уязвимы для атак этого вида.

Многие уязвимости BIOS, перечисленные в таблице 15-1, исправлены в современных аппаратных средствах и операционных системах, например в новых версиях Windows, где Secure Boot по умолчанию активирована, если оборудование и прошивка способны её поддерживать. В главе 17 мы подробнее рассмотрим подходы к реализации и слабости Secure Boot, а пока достаточно сказать, что любая загружаемая прошивка или драйвер расширения без серьёзных требований к аутентификации может представлять проблему безопасности. На современном оборудовании корпоративного класса сторонние Option ROM обычно заблокированы по умолчанию, но их можно снова включить в интерфейсе управления BIOS, как показано на рисунке 15-5.

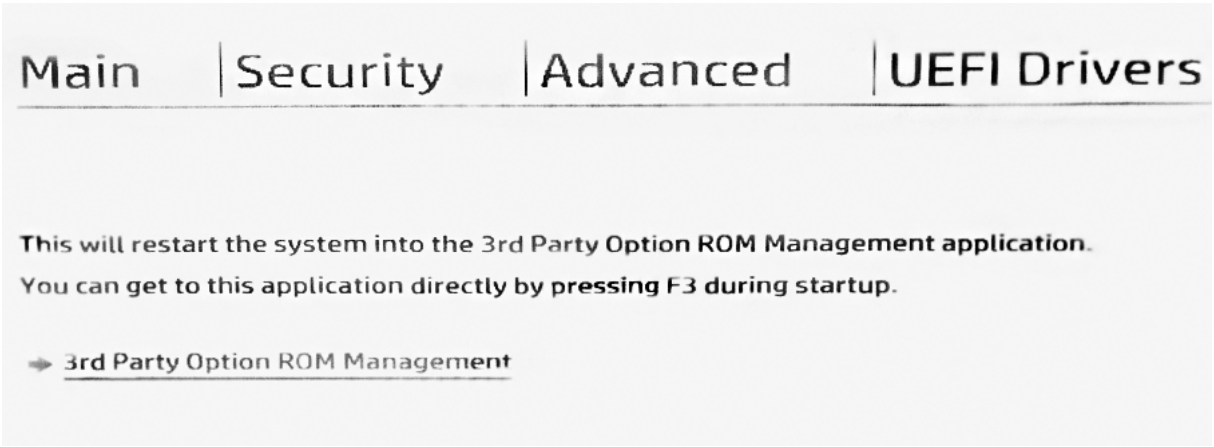


Рисунок 15-5. Блокировка сторонних Option ROM в интерфейсе управления BIOS

После выпуска PoC Thunderstrike некоторые поставщики, включая Apple, стали активнее блокировать все неподписанные или сторонние Option ROM. Мы считаем такую политику правильной: обстоятельства, в которых требуется загрузить стороннюю Option ROM, редки, а

блокирование всех Option ROM со сторонних устройств значительно снижает риски безопасности. Если вы используете расширения периферийных устройств со встроенными Option ROM, обязательно приобретайте их у того же поставщика, что и само устройство: покупка случайного устройства не стоит такого риска.

Добавление или модификация драйвера DXE

Теперь рассмотрим второй вид атаки из нашего списка: добавление или модификацию драйвера DXE в образе прошивки UEFI. По существу, эта атака довольно проста: изменив легитимный драйвер DXE в прошивке, злоумышленник может внедрить вредоносный код, который будет выполнен в предзагрузочной среде на этапе DXE. Однако самая интересная и, вероятно, самая сложная часть этой атаки - добавление или модификация драйвера DXE, включающая замысловатую цепочку эксплуатации уязвимостей, присутствующих в прошивке UEFI, операционной системе и приложениях пользовательского режима.

Один из способов изменить драйвер DXE в образе прошивки UEFI состоит в обходе рассмотренных ранее в этой главе битов защиты SPI flash посредством эксплуатации уязвимости повышения привилегий. Повышенные привилегии позволяют злоумышленнику отключить защиту SPI flash, сбросив биты защиты.

Другой способ состоит в эксплуатации уязвимости процесса обновления BIOS, позволяющей злоумышленнику обойти аутентификацию обновления и записать вредоносный код в SPI flash-память. Рассмотрим, как эти подходы применяются для заражения BIOS вредоносным кодом.

> **Примечание.** Эти два метода не являются единственными подходами к изменению защищённого содержимого SPI flash, но здесь мы сосредоточились на них, чтобы показать, как вредоносный код BIOS может постоянно сохраняться на компьютере жертвы. Более полный список уязвимостей прошивки UEFI приведён в главе 16.

Понимание внедрения руткита

Большинство пользовательских секретов и конфиденциальных сведений, интересующих злоумышленников, либо хранится на уровне ядра операционной системы, либо защищается кодом, работающим на этом уровне. Именно поэтому руткиты издавна стремились скомпрометировать режим ядра («кольцо 0»): с этого уровня руткит мог наблюдать за всей деятельностью пользователя или нацеливаться на конкретные приложения пользовательского режима («кольцо 3»), включая любые компоненты, загружаемые этими приложениями.

Однако в одном отношении руткит кольца 0 находится в невыгодном положении: у него отсутствует контекст пользовательского режима. Когда руткит, работающий из режима ядра, пытается украсть какие-либо данные приложения кольца 3, он видит эти данные не в самом естественном представлении, поскольку режим ядра по замыслу не должен знать об абстракциях данных пользовательского уровня. Поэтому руткиту режима ядра часто приходится тем или иным приёмом восстанавливать такие данные, особенно если они распределены по нескольким страницам памяти. Следовательно, руткитам режима ядра требуется умело повторно использовать код, реализующий абстракции пользовательского уровня. И всё же при наличии всего одного разделяющего уровня такое повторное использование кода не было особенно сложным.

SMM добавил ещё более привлекательную цель, но также добавил ещё один уровень отделения от абстракций пользовательского уровня. Руткит на основе SMM может управлять памятью как уровня ядра, так и пользовательского уровня, поскольку управляет любой страницей физической памяти.

Однако эта сильная сторона вредоносного кода уровня SMM одновременно является и слабостью: такой код должен надёжно повторно реализовать абстракции верхнего уровня, например виртуальную память, и справиться со всей сложностью этой задачи.

К счастью для злоумышленника, руткит SMM может внедрить вредоносный модуль руткита кольца 0 в ядро ОС подобно буткитам, причём не только во время загрузки. Затем он может полагаться на этот код для использования структур режима ядра в контексте режима ядра, одновременно защищая код от обнаружения средствами безопасности уровня ядра. Особенно важно, что код на основе SMM способен выбирать момент внедрения имплантата.

В частности, имплантаты прошивки могут обходить даже некоторые реализации Secure Boot, чего обычные буткиты сделать не могли, перемещая точку заражения на момент после завершения проверок целостности. На рисунке 15-6 показано, как методы доставки эволюционировали от простой схемы доставки с загрузчиком пользовательского режима (кольца 3), который эксплуатировал уязвимость для повышения своих привилегий и установки вредоносного драйвера режима ядра (кольца 0). Однако развитие защитных мер настигло эту схему. Политики Microsoft подписывания кода режима ядра сделали её неэффективной и положили начало эпохе буткитов, против которых, в свою очередь, была внедрена технология Secure Boot. Затем появились угрозы SMM, призванные подорвать Secure Boot.

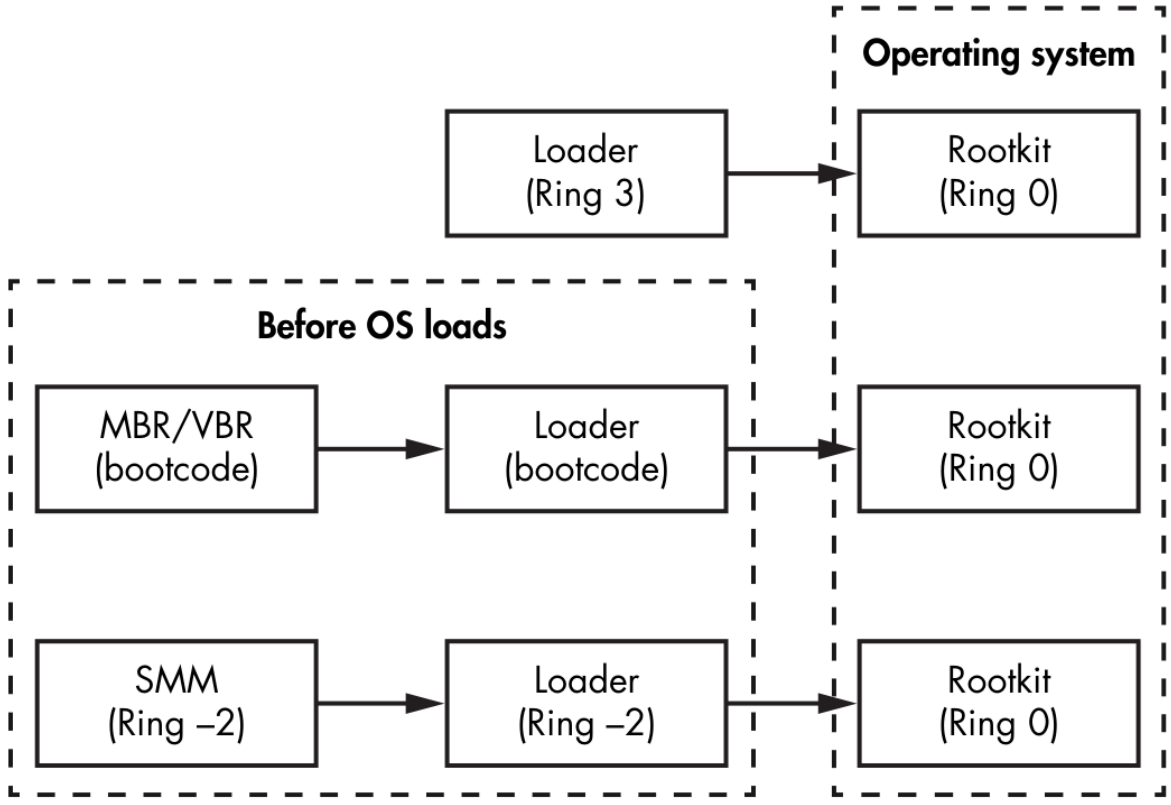


Рисунок 15-6. Возможные способы загрузки руткита кольца 0

На момент написания книги угрозам SMM удалось обойти Secure Boot на большинстве платформ на основе Intel. Руткиты и имплантаты SMM снова переместили границу безопасности ниже, ближе к физическому оборудованию.

По мере роста популярности угроз SMM криминалистический анализ прошивки становится новым и очень важным направлением исследований.

Внедрение вредоносного кода посредством повышения привилегий SMM

Чтобы повысить привилегии до уровня SMM и получить возможность изменять содержимое SPI flash, злоумышленник должен использовать интерфейсы обратного вызова для операционной системы, обрабатываемые обработчиками System Management Interrupt (подробнее обработчики SMI рассматриваются в главе 16). Обработчики SMI, отвечающие за аппаратные интерфейсы с операционной системой, выполняются в SMM, поэтому, если злоумышленник сможет эксплуатировать уязвимость внутри драйвера SMM, он может получить привилегии выполнения SMM. Вредоносный код, выполняемый с привилегиями SMM, способен на некоторых платформах отключить биты защиты SPI flash и изменить либо добавить драйвер DXE в прошивке UEFI.

Чтобы понять атаку такого рода, необходимо рассмотреть тактику атак для схем постоянного заражения с уровня операционной системы. Что требуется сделать злоумышленнику, чтобы изменить SPI flash-память? Необходимые этапы показаны на рисунке 15-7.

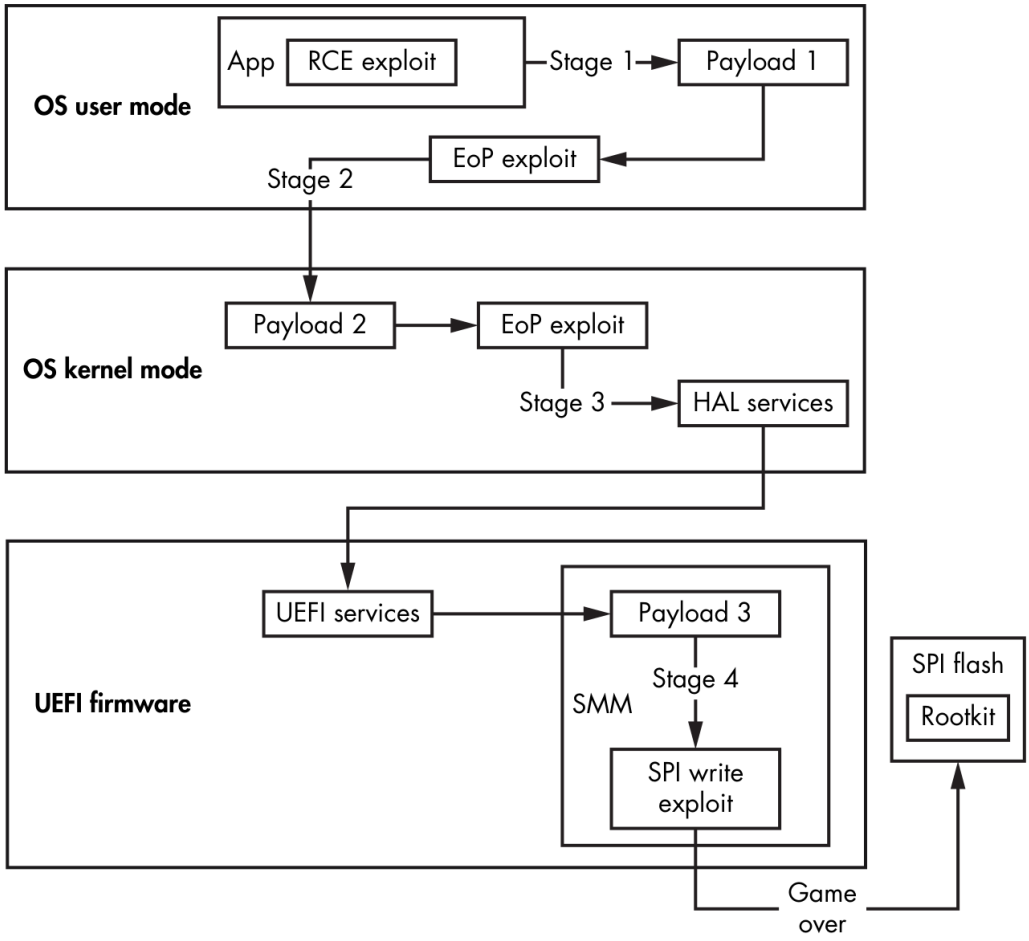


Рисунок 15-7. Обобщённая схема заражения руткитом UEFI

Как видим, путь эксплуатации довольно сложен и включает эксплойты на многих уровнях. Разделим этот процесс на этапы:

Этап 1, пользовательский режим. Клиентский эксплойт, например удалённое выполнение кода (remote code execution, RCE) в веб-браузере, помещает в систему вредоносный установщик. Затем установщик применяет эксплойт повышения привилегий, чтобы получить доступ `LOCALSYSTEM`, и продолжает выполнение с новыми привилегиями.

Этап 2, режим ядра. Установщик обходит политики подписывания кода, рассмотренные в главе 6, чтобы выполнить свой код в режиме ядра. Полезная нагрузка режима ядра, то есть драйвер, запускает эксплойт для получения привилегий SMM.

Этап 3, режим системного управления. Код SMM успешно выполняется, а привилегии повышаются до SMM. Полезная нагрузка SMM отключает защиту от модификации SPI flash-памяти.

Этап 4, SPI flash. Все средства защиты SPI flash отключены, и flash-память открыта для произвольной записи. Затем руткит или имплантат устанавливается в прошивку на микросхеме SPI flash. Благодаря этому эксплойту достигается очень высокий уровень постоянства в системе.

Обобщённая схема заражения на рисунке 15-8 фактически показывает реальный случай PoC программы-вымогателя SMM, который мы представили на Black Hat Asia 2017. Доклад называется «UEFI Firmware Rootkits: Myths and Reality»; рекомендуем ознакомиться с ним, если вы хотите узнать больше: https://www.blackhat.com/docs/asia-17/materials/asia-17-Matrosov-The-UEFI-Firmware-Rootkits-Myths-And-Reality.pdf.

Эксплуатация (не)безопасности процесса обновления BIOS

Ещё один способ внедрения вредоносного кода в BIOS состоит в злоупотреблении процессом аутентификации обновлений BIOS. Аутентификация обновления BIOS предназначена для предотвращения установки обновлений BIOS, подлинность которых невозможно проверить, и обеспечивает разрешение на установку только образов обновлений BIOS, выпущенных поставщиком платформы. Если злоумышленнику удастся эксплуатировать уязвимость этого механизма аутентификации, он сможет внедрить вредоносный код в образ обновления, который впоследствии будет записан в SPI flash.

В марте 2017 года один из авторов этой книги Алекс Матросов продемонстрировал PoC программы-вымогателя UEFI на Black Hat Asia (https://www.cylance.com/en_us/blog/gigabyte-brix-systems-vulnerabilities.html). Его PoC показывал, как можно эксплуатировать слабый процесс обновления, реализованный Gigabyte. Он использовал современную платформу Gigabyte на основе процессора Intel шестого поколения (Skylake) и Microsoft Windows 10 со всеми включёнными средствами защиты, включая Secure Boot и бит BLE. Несмотря на эту защиту, платформа Gigabyte Brix не аутентифицировала обновления, позволяя злоумышленнику установить любое обновление прошивки из ядра ОС (http://www.kb.cert.org/vuls/id/507496/). На рисунке 15-8 показан уязвимый процесс процедуры обновления BIOS на оборудовании Gigabyte Brix.

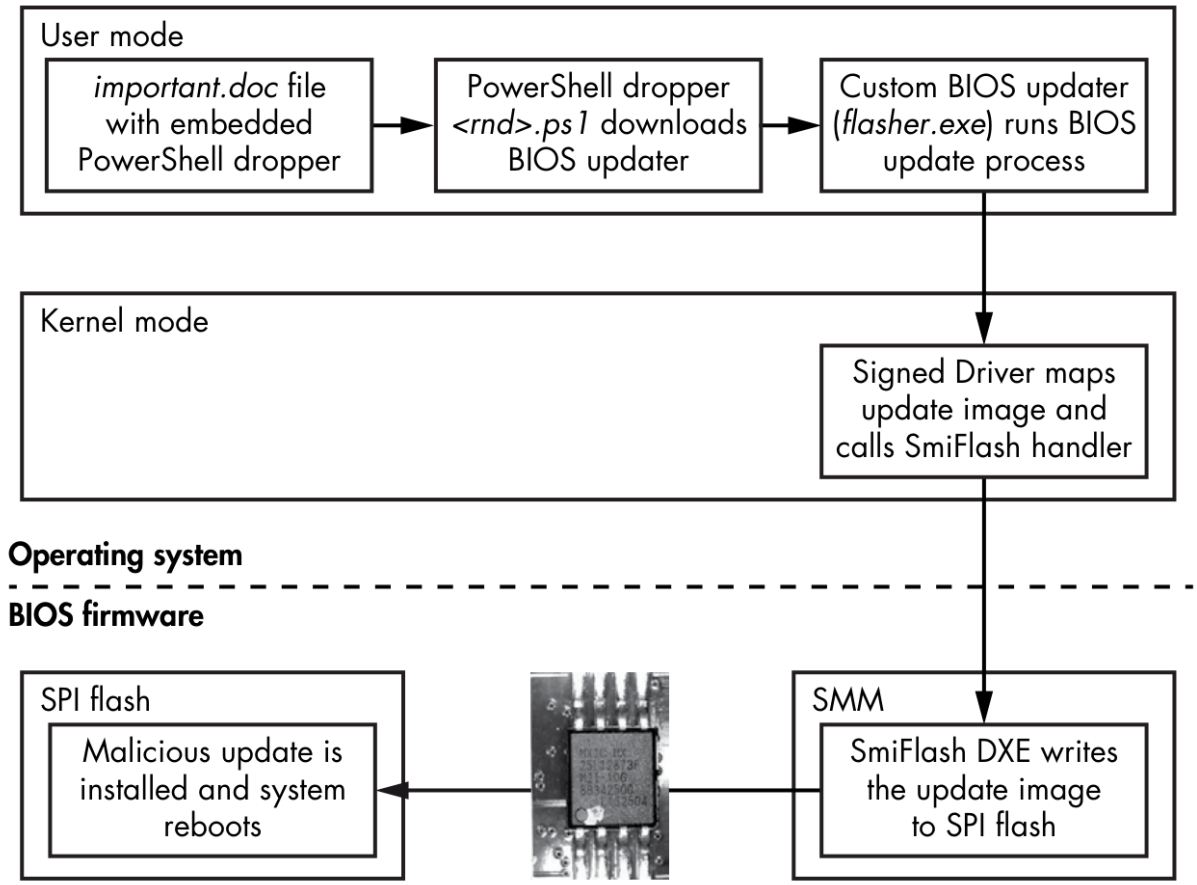


Рисунок 15-8. Алгоритм заражения программой-вымогателем UEFI

Как видим, злоумышленник может использовать исходный драйвер режима ядра из программного обеспечения обновления BIOS, предоставленный и подписанный поставщиком оборудования, для доставки вредоносного обновления BIOS. Драйвер взаимодействует с обработчиком SWSMI `SmiFlash`, имеющим интерфейсы записи в SPI flash-память и чтения из неё. Специально для этой презентации один из драйверов DXE был изменён и выполнен в SMM, чтобы продемонстрировать максимально возможный уровень постоянства в прошивке UEFI и управление процессом загрузки с самых ранних её этапов. Если заражение программой-вымогателем UEFI проходит успешно, целевая машина показывает сообщение с требованием выкупа, приведённое на рисунке 15-9.

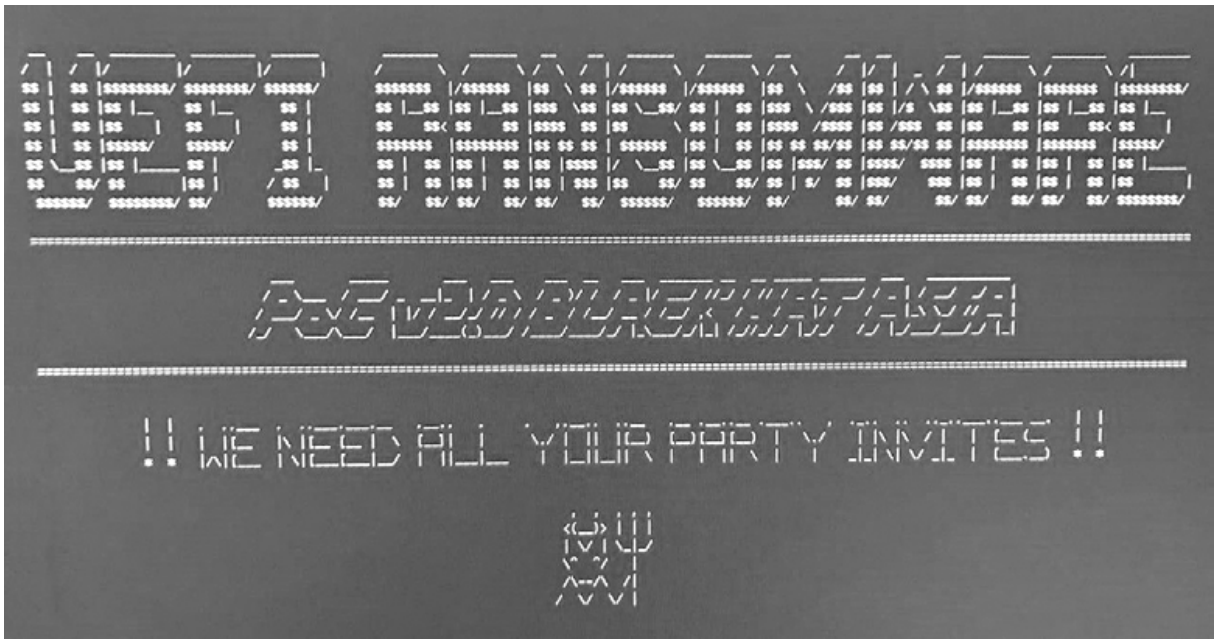


Рисунок 15-9. Экран активного заражения программой-вымогателем UEFI с Black Hat Asia 2017

В прошивках устаревшей BIOS, до того как UEFI стала отраслевым стандартом, основные поставщики оборудования не особенно задумывались о защите аутентификации обновлений прошивки. В результате системы были чрезвычайно уязвимы для вредоносных имплантатов BIOS; когда такие имплантаты начали появляться, поставщикам пришлось обратить на это внимание. В наши дни для противодействия таким атакам обновления прошивки UEFI имеют унифицированный формат Capsule Update, подробно описанный в спецификации UEFI. Capsule Update был разработан для внедрения более совершенного процесса доставки обновлений BIOS. Рассмотрим его подробнее на примере упомянутого ранее репозитория Intel EDK2.

Улучшение Capsule Update

Capsule Update имеет заголовок (`EFI_CAPSULE_HEADER` в обозначениях EDK2) и тело для хранения всех сведений об исполняемых модулях обновления, включая драйверы DXE и PEI. Образ Capsule Update содержит обязательную цифровую подпись данных обновления и код, используемый для аутентификации и защиты целостности.

Рассмотрим структуру образа Capsule Update с помощью утилиты UEFITool, разработанной Николаем Шлеем (https://github.com/LongSoft/UEFITool). Этот инструмент позволяет разбирать образы прошивки UEFI, включая предоставленные в обновлениях UEFI Capsule, и извлекать различные исполняемые модули DXE или PEI как отдельные двоичные файлы. Мы вернёмся к UEFITool в главе 19. На рисунке 15-10 показана структура UEFI Capsule Update в выводе UEFITool.

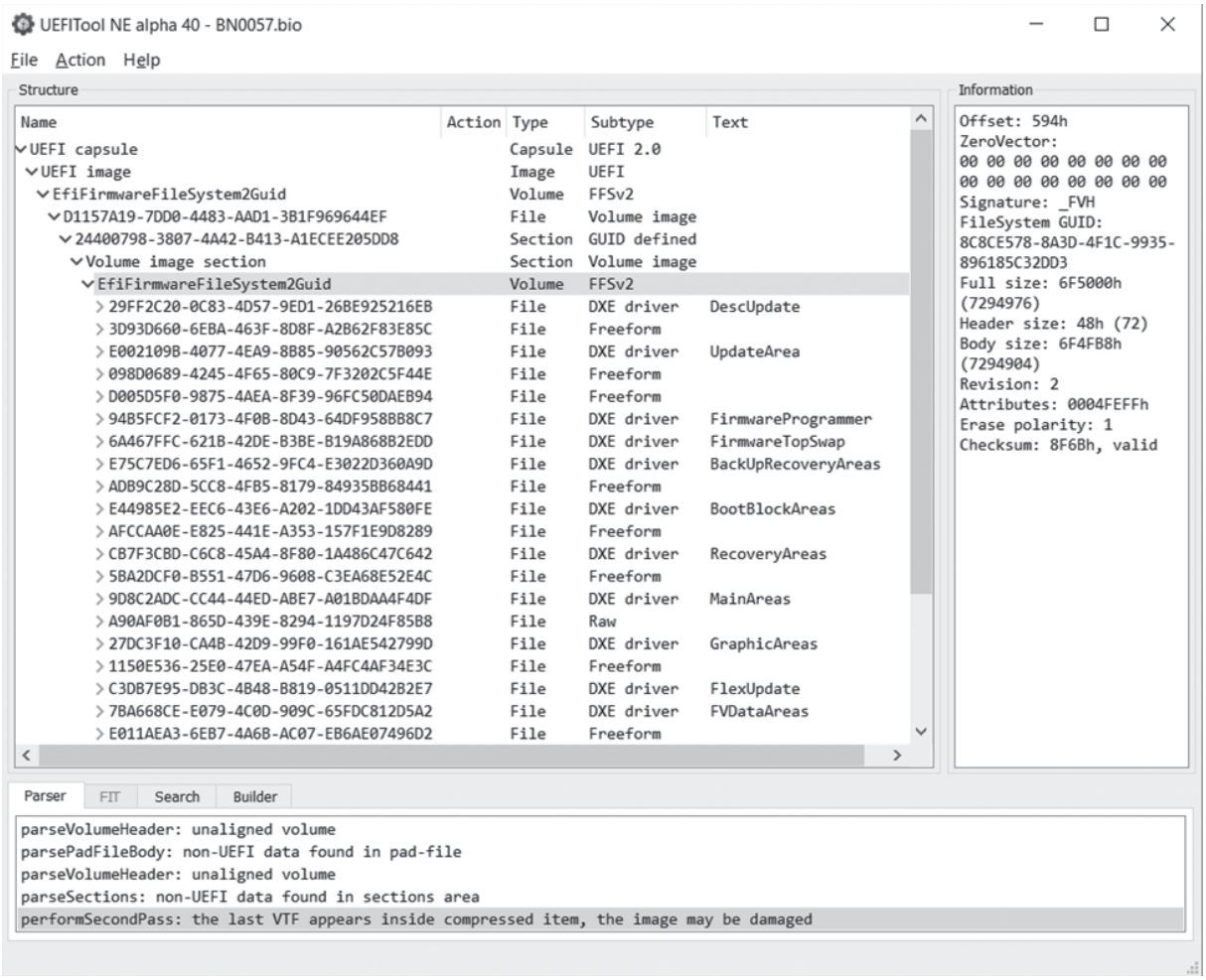


Рисунок 15-10. Интерфейс UEFITool

Образ капсулы начинается с заголовка ❶, описывающего общие параметры образа обновления, такие как размер заголовка и размер образа обновления. Затем следует тело капсулы, которое здесь состоит из одного тома прошивки ❷. Томы прошивки представляют собой объекты, определённые в спецификации инициализации платформы и используемые для хранения образов файлов прошивки, включая модули DXE и PEI. Подробнее мы рассмотрим их в главе 19. Этот том прошивки содержит фактические данные обновления BIOS, предназначенные для записи в SPI flash-память, в нескольких файлах прошивки: например, `BootBlockAreas` ❸ и `RecoveryAreas` ❹ содержат обновления для этапа PEI, а `MainAreas` ❺ содержит обновления для этапа DXE.

Важно, что содержимое тома прошивки, хранящего обновления BIOS, подписано, хотя UEFITool не отображает эти сведения на рисунке 15-11. В результате злоумышленник не может внести изменения в обновления, не сделав цифровую подпись недействительной. При правильной реализации Capsule Update противодействует использованию злоумышленниками неаутентифицированных обновлений прошивки.

Руткиты UEFI в реальных условиях

После вредоносной программы UEFI, обнаруженной «Лабораторией Касперского» в 2015 году, появилось множество сообщений СМИ о ещё более сложных руткитах, предположительно разработанных субъектами государственного уровня. В оставшейся части главы мы рассмотрим другие примеры руткитов UEFI, в том числе широко развёртывавшиеся коммерческими организациями, такие как Vector-EDK и Computrace.

Руткит Vector-EDK компании Hacking Team

В 2015 году была взломана итальянская компания Hacking Team, разрабатывавшая шпионское ПО для правоохранительных органов и других государственных клиентов, и значительная часть конфиденциальных сведений компании оказалась раскрыта, включая описания интересного проекта Vector-EDK. Анализ взлома показал, что Vector-EDK представлял собой руткит прошивки UEFI, устанавливавший и выполнявший свои вредоносные компоненты непосредственно в подсистеме NTFS пользовательского режима Windows.

Один из авторов этой книги Алекс Матросов, в то время входивший в группу Intel Advanced Threat Research (ATR), распознал атакующий потенциал Vector-EDK и опубликовал запись в блоге «Hacking Team's 'Bad BIOS': A Commercial Rootkit for UEFI Firmware?»: https://www.mcafee.com/enterprise/en-us/threat-center/advanced-threat-research/uefi-rootkit.html.

Обнаружение Vector-EDK

Наше расследование началось, когда мы обнаружили любопытный файл `Z5WE1X64.fd`, прикреплённый к одному из утекших писем Hacking Team внутри сжатого файла `Uefi_windows_persistent.zip` (см. рисунок 15-11).

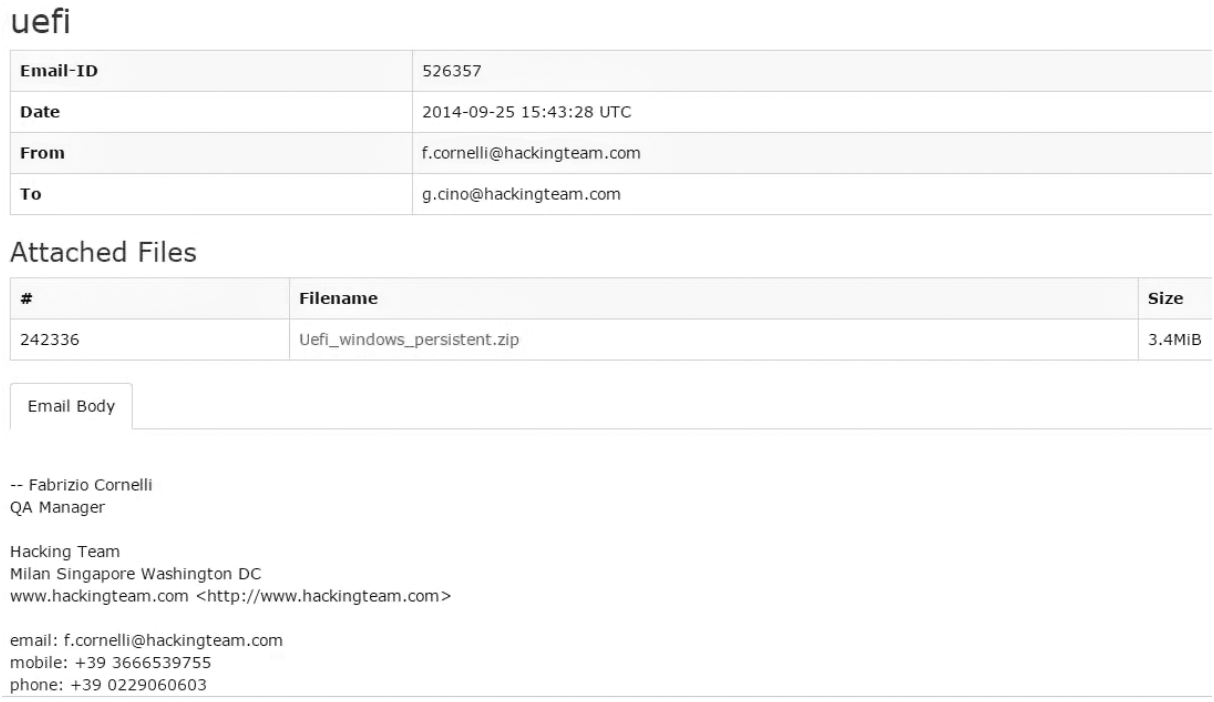


Рисунок 15-11. Одно из утекших писем из архива Hacking Team

Проанализировав вложение, мы поняли, что это образ прошивки UEFI, а прочитав ещё несколько утекших писем, увидели, что имеем дело с руткитом UEFI. Быстрое исследование с помощью UEFITool выявило многозначительное имя `rkloader`, подразумевающее загрузчик руткита, в списке драйверов DXE. Наш анализ показан на рисунке 15-12.

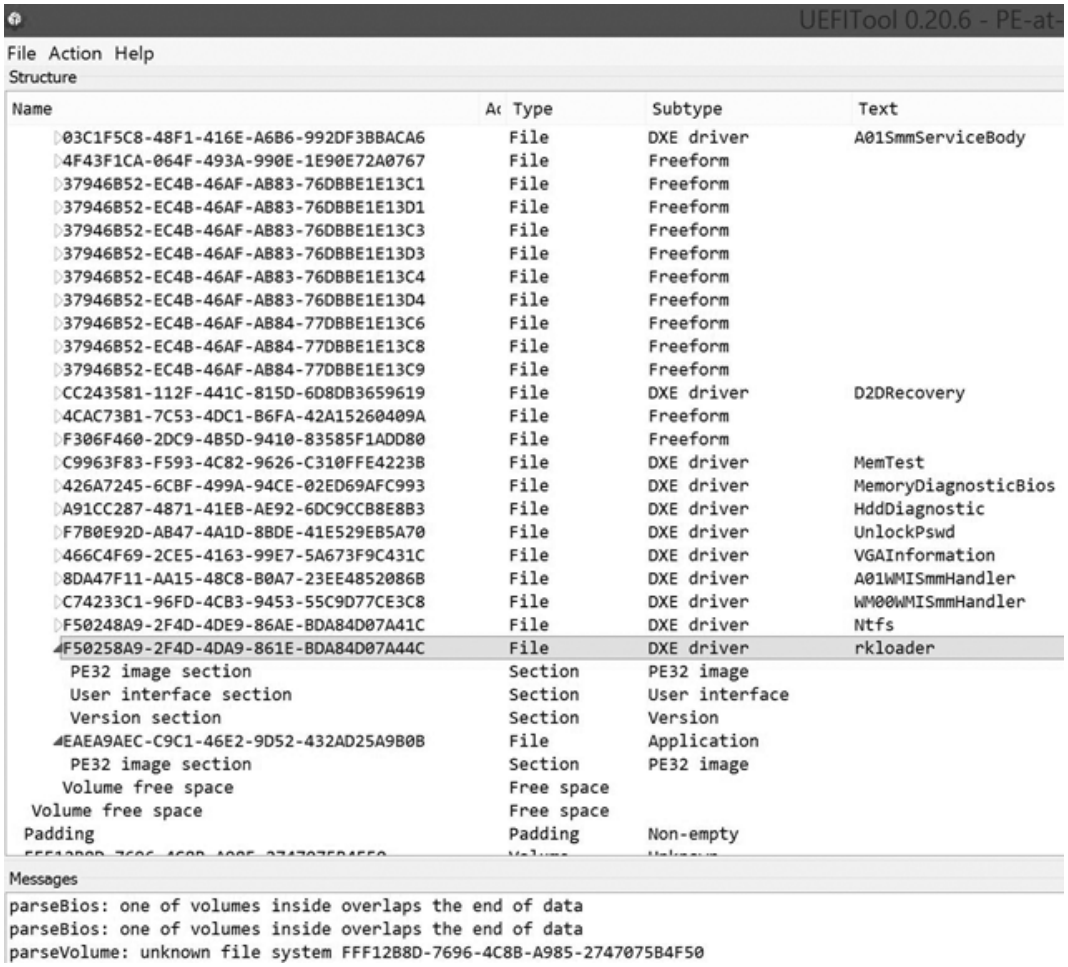


Рисунок 15-12. Обнаружение Vector-EDK компании Hacking Team с помощью UEFITool

Это привлекло наше внимание, поскольку прежде мы никогда не встречали драйвер DXE с таким именем. Мы внимательнее изучили утекший архив и обнаружили исходный код проекта Vector-EDK. Именно здесь по-настоящему началось наше техническое расследование.

Анализ Vector-EDK

Руткит Vector-EDK использует рассмотренные ранее методы доставки имплантата UEFI (`rkloader`). Однако этот руткит работает только на этапе DXE и не способен пережить обновление BIOS. Внутри заражённого образа BIOS `Z5WE1X64.fd` находились три основных модуля:

Синтаксический анализатор NTFS (`Ntfs.efi`). Драйвер DXE, содержащий полный синтаксический анализатор NTFS для операций чтения и записи.

Руткит (`rkloader.efi`). Драйвер DXE, регистрирующий обратный вызов для перехвата события `EFI_EVENT_GROUP_READY_TO_BOOT`, которое означает, что платформа готова выполнить загрузчик ОС, и загрузки приложения UEFI `fsbg.efi` до начала загрузки ОС.

Буткит (`fsbg.efi`). Приложение UEFI, запускаемое непосредственно перед тем, как BIOS передаёт управление загрузчикам ОС. Оно содержит основные функции буткита, которые разбирают NTFS с помощью `Ntfs.efi` и внедряют агенты вредоносного ПО в файловую систему.

Мы проанализировали утекший исходный код Vector-EDK и обнаружили, что компоненты `rkloader.efi` и `fsbg.efi` реализуют основную функциональность руткита.

Сначала рассмотрим `rkloader.efi`, запускающий `fsbg.efi`. В листинге 15-4 показана основная процедура `_ModuleEntryPoint()` драйвера UEFI DXE `rkloader`.

```
EFI_STATUS
EFIAPI
_ModuleEntryPoint (EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable)
{
    EFI_EVENT Event;
    DEBUG((EFI_D_INFO, "Running RK loader.\n"));
    InitializeLib(ImageHandle, SystemTable);
    gReceived = FALSE;    // Сбросить событие!

    //CpuBreakpoint();
    // Ожидать EFI EVENT GROUP READY TO BOOT
    ❶ gBootServices->CreateEventEx( 0x200, 0x10,
                      ❷ &CallbackSMI, NULL, &SMBIOS_TABLE_GUID, &Event );
    return EFI_SUCCESS;
}
```

Листинг 15-4. Процедура _ModuleEntryPoint() из компонента rkloader

Мы обнаружили, что процедура `_ModuleEntryPoint()` выполняет только две задачи. Первая состоит в создании триггера ❶ для группы событий `EFI_EVENT_GROUP_READY_TO_BOOT`. Вторая задача после поступления события - выполнить обработчик SMI ❷ посредством `CallbackSMI()`. Первый параметр процедуры `CreateEventEx()` указывает, что непосредственное значение `EFI_EVENT_GROUP_READY_TO_BOOT` равно `0x200`. Это событие происходит непосредственно перед тем, как загрузчик ОС получает управление в конце этапа BIOS DXE, позволяя вредоносной полезной нагрузке `fsbg.efi` перехватить выполнение раньше операционной системы.

Большая часть интересной логики содержится внутри процедуры `CallbackSMI()` из листинга 15-5. Код этой процедуры довольно длинный, поэтому здесь мы включили только наиболее важные части её потока выполнения.

```
VOID
EFIAPI
CallbackSMI (EFI_EVENT Event, VOID *Context)
{
   --snip--
   ❶ EFI_LOADED_IMAGE_PROTOCOL       *LoadedImage;
   EFI_FIRMWARE_VOLUME_PROTOCOL    *FirmwareProtocol;
   EFI_DEVICE_PATH_PROTOCOL        *DevicePathProtocol,
                                   *NewDevicePathProtocol,
                                   *NewFilePathProtocol,
                                   *NewDevicePathEnd;
   --snip--

   ❷ Status = gBootServices->HandleProtocol( gImageHandle,
                                        &LOADED_IMAGE_PROTOCOL_GUID,
                                        &LoadedImage);
   --snip--
   DeviceHandle = LoadedImage->DeviceHandle;
   ❸ Status = gBootServices->HandleProtocol( DeviceHandle,
                                        &FIRMWARE_VOLUME_PROTOCOL_GUID,
                                        &FirmwareProtocol);
   ❹ Status = gBootServices->HandleProtocol( DeviceHandle,
                                        &DEVICE_PATH_PROTOCOL_GUID,
                                        &DevicePathProtocol);
   --snip--
   // Скопировать дескриптор "VOLUME"
   ❺ gBootServices->CopyMem( NewDevicePathProtocol,
```

```
                          DevicePathProtocol,
                          DevicePathLength);
    --snip--
    ❻ gBootServices->CopyMem( ((CHAR8 *)(NewFilePathProtocol) + 4),
                          &LAUNCH_APP, sizeof(EFI_GUID));
    --snip--

    ❼ Status = gBootServices->LoadImage( FALSE,
                                         gImageHandle,
                                         NewDevicePathProtocol,
                                         NULL,
                                         0,
                                         &ImageLoadedHandle);
    --snip--
  done:
    return;
  }
```

Листинг 15-5. Процедура CallbackSMI() из компонента fsbg

Сначала мы видим инициализацию нескольких протоколов UEFI ❶:

EFI_LOADED_IMAGE_PROTOCOL. Предоставляет сведения о загруженных образах UEFI: базовый адрес образа, размер образа и расположение образа в прошивке UEFI.

EFI_FIRMWARE_VOLUME_PROTOCOL. Предоставляет интерфейс чтения из томов прошивки и записи в них.

EFI_DEVICE_PATH_PROTOCOL. Предоставляет интерфейс построения пути к устройству.

Интересующая нас часть начинается с инициализации нескольких объектов `EFI_DEVICE_PATH_PROTOCOL`; мы видим множество имён переменных с префиксом `New`, который обычно указывает, что они являются перехватчиками. Переменная `LoadedImage` инициализируется ❷ указателем на `EFI_LOADED_IMAGE_PROTOCOL`, после чего `LoadedImage` можно использовать для определения устройства, на котором расположен текущий модуль `rkloader`.

Затем код получает протоколы `EFI_FIRMWARE_VOLUME_PROTOCOL` ❸ и `EFI_DEVICE_PATH_PROTOCOL` ❹ для устройства, на котором расположен `rkloader`. Эти протоколы необходимы для построения пути к следующему вредоносному модулю, а именно `fsbg.efi`, чтобы загрузить его из тома прошивки.

После получения протоколов `rkloader` строит путь к модулю `fsbg.efi` для его загрузки из тома прошивки. Первая часть пути ❺ представляет собой путь к тому прошивки, в котором находится `rkloader`; `fsbg.efi` расположен в том же самом томе прошивки, что и `rkloader`. Вторая часть ❻ добавляет уникальный идентификатор модуля `fsbg.efi`: `LAUNCH_APP = {eaea9aec-c9c1-46e2-9d52432ad25a9b0b}`.

Последний этап представляет собой вызов процедуры `LoadImage()` ❼, передающий выполнение модулю `fsbg.efi`. Этот вредоносный компонент содержит основную полезную нагрузку с прямыми путями к файловой системе, которую он намерен изменить. В листинге 15-6 приведён список каталогов, в которые модуль `fsbg.efi` помещает вредоносный модуль уровня ОС.

```
#define FILE_NAME_SCOUT L"\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\
    \"
#define FILE_NAME_SOLDIER L"\\AppData\\Roaming\\Microsoft\\Windows\\Start Menu\\Programs\\
    Startup\\"
#define FILE_NAME_ELITE  L"\\AppData\\Local\\"
#define DIR_NAME_ELITE L"\\AppData\\Local\\Microsoft\\"
```

```
#ifdef FORCE_DEBUG
UINT16 g_NAME_SCOUT[] =   L"scoute.exe";
UINT16 g_NAME_SOLDIER[] = L"soldier.exe";
UINT16 g_NAME_ELITE[]   = L"elite";
#else
UINT16 g_NAME_SCOUT[] =   L"6To_60S7K_FU06yjEhjh5dpFw96549UU";
UINT16 g_NAME_SOLDIER[] = L"kdfas7835jfwe09j29FKFLDOR3r35fJR";
UINT16 g_NAME_ELITE[]   = L"eorpekf3904kLDKQO023iosdn93smMXK";
#endif
```

Листинг 15-6. Жёстко заданные пути к компонентам уровня ОС

На высоком уровне модуль `fsbg.efi` выполняет следующие действия:

1. Проверяет, не заражена ли система уже активно, с помощью предопределённой переменной UEFI с именем `fTA`.
2. Инициализирует протокол NTFS.
3. Ищет вредоносные исполняемые файлы в образе BIOS, просматривая предопределённые разделы.
4. Проверяет наличие существующих пользователей на машине, просматривая имена в домашнем каталоге для поиска конкретных целей.
5. Устанавливает исполняемые модули вредоносного ПО `scoute.exe` (бэкдор) и `soldier.exe` (агент RCS), записывая их непосредственно в NTFS.

Переменная UEFI `fTA` устанавливается `fsbg.efi` в момент первого заражения, а при каждой последующей загрузке проверяется её наличие: если переменная `fTA` присутствует, активное заражение уже имеется на жёстком диске и `fsbg.efi` не требуется доставлять вредоносный двоичный файл уровня ОС в файловую систему. Если вредоносные компоненты уровня ОС из листинга 15-6 не найдены по жёстко заданным путям, модуль `fsbg.efi` устанавливает их заново в процессе загрузки.

Vector-EDK компании Hacking Team является очень поучительным примером буткита UEFI. Для лучшего понимания принципа его работы мы настоятельно рекомендуем прочитать его исходный код целиком.

Computrace/LoJack компании Absolute Software

Следующий пример руткита UEFI, строго говоря, не является вредоносным. Computrace, также известный как LoJack, на самом деле представляет собой распространённую проприетарную противоугонную систему, разработанную Absolute Software и встречающуюся почти во всех популярных ноутбуках корпоративного класса. Computrace реализует систему отслеживания ноутбука через интернет и включает такие функции, как удалённая блокировка и удалённое стирание жёстких дисков в случае потери или кражи ноутбука.

Многие исследователи независимо друг от друга утверждали, что технически Computrace является руткитом, поскольку его поведение очень похоже на поведение руткита BIOS. Однако главное различие состоит в том, что Computrace не пытается скрываться. Его меню настройки можно найти даже в меню конфигурации BIOS (рисунок 15-13).

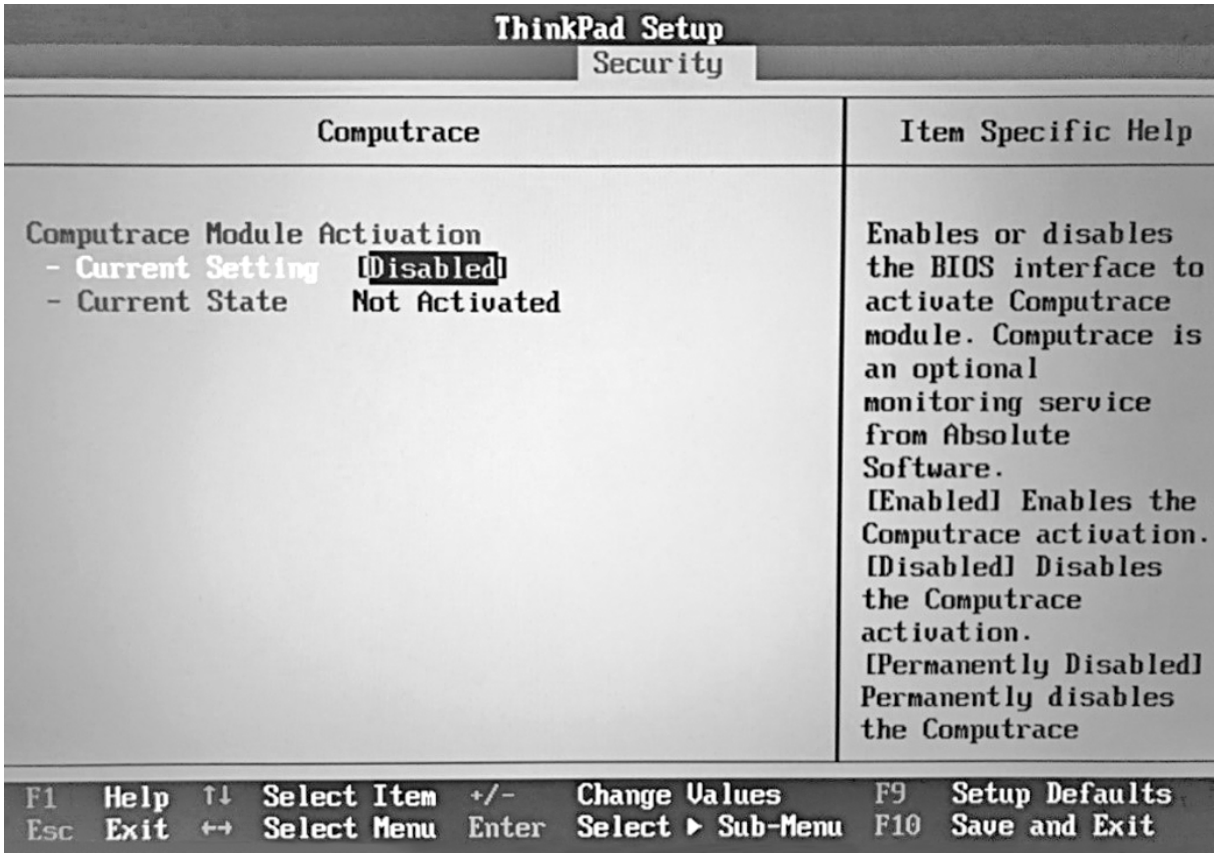


Рисунок 15-13. Меню Computrace в настройках BIOS Lenovo ThinkPad T540p

На новых компьютерах, не относящихся к корпоративному классу, Computrace обычно по умолчанию отключён в меню BIOS, как показано на рисунке 15-13. Также имеется возможность навсегда отключить Computrace, установив переменную NVRAM, которая запрещает повторную активацию Computrace и может быть запрограммирована в оборудовании только один раз.

Здесь мы проанализируем реализации Computrace на ноутбуках Lenovo T540p и P50. Наше концептуальное представление архитектуры Computrace показано на рисунке 15-14.

Computrace имеет сложную архитектуру с несколькими драйверами DXE, включающими компоненты, работающие в SMM. Кроме того, он содержит агент `rpcnetp.exe`, который выполняется в операционной системе и отвечает за весь сетевой обмен с облаком, то есть C&C-сервером.

`LenovoComputraceEnableDxe`. Драйвер DXE, отслеживающий параметры Computrace в меню BIOS, чтобы запустить этап установки `LenovoComputraceLoaderDxe`.

`LenovoComputraceLoaderDxe`. Драйвер DXE для проверки политик безопасности и загрузки `AbsoluteComputraceInstallerDxe`.

`AbsoluteComputraceInstallerDxe`. Драйвер DXE, устанавливающий агент Computrace в операционную систему посредством прямых модификаций файловой системы NTFS. Двоичный файл агента встроен в образ драйвера DXE, как показано на рисунке 15-15. На современном ноутбуке для установки агента используются таблицы ACPI.

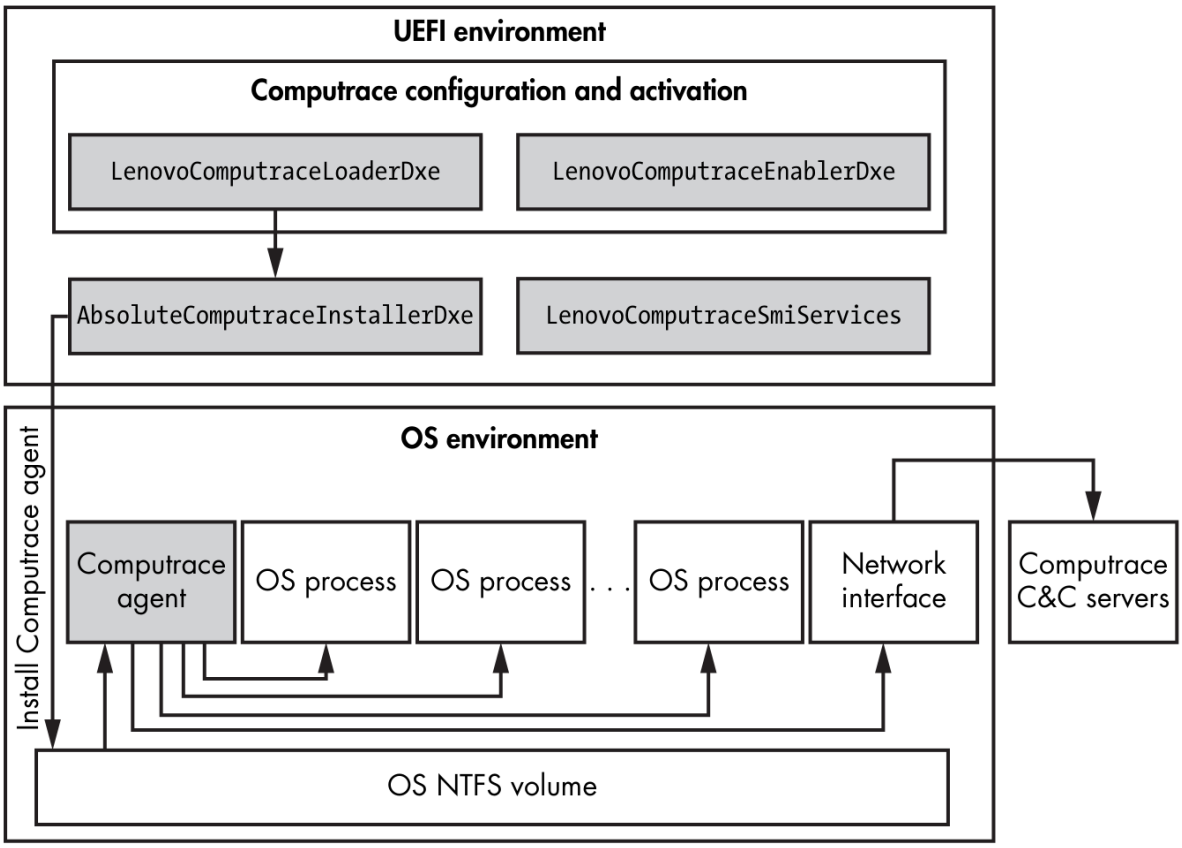


Рисунок 15-14. Высокоуровневая архитектура Computrace

Рисунок 15-15. Двоичный файл AbsoluteComputraceInstallerDxe в шестнадцатеричном редакторе Hiew

LenovoComputraceSmiServices. Драйвер DXE, выполняемый внутри SMM для поддержки обмена с агентом ОС и другими компонентами BIOS.

Агент Computrace (rpcnetp.exe). Исполняемый PE-образ, хранящийся внутри `AbsoluteComputraceInstallerDxe`. Агент Computrace выполняется после входа пользователя в операционную систему.

Основные функции агента Computrace `rpcnetp.exe` состоят в сборе геолокационных сведений и их отправке в облако Absolute Software. Это достигается внедрением компонента Computrace `rpcnetp.dll` в процессы `iexplore.exe` и `svchost.exe`, как показано на рисунке 15-16. Агент также получает команды из облака, например команду низкоуровневого стирания жёсткого диска для безопасного удаления файлов.

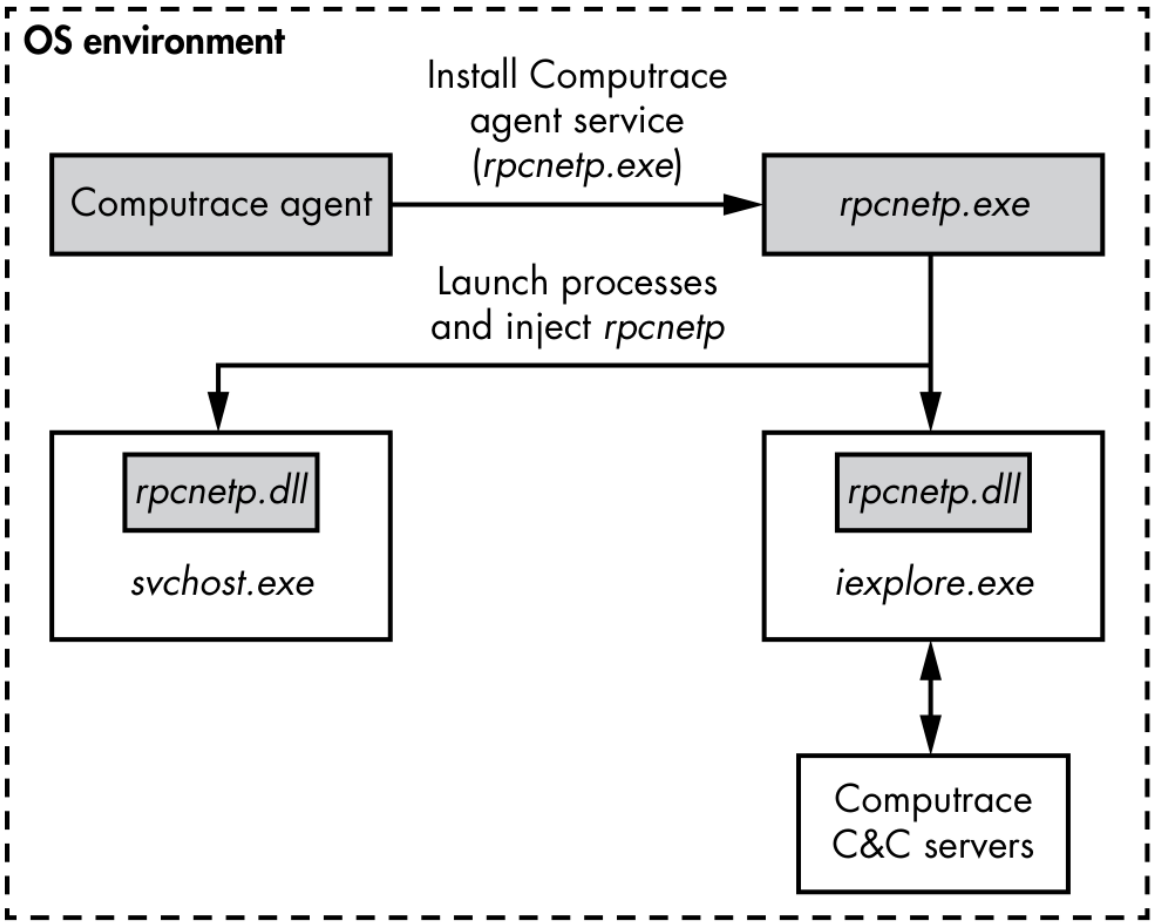


Рисунок 15-16. Схема внедрения процесса rpcnetp.exe

Computrace является хорошим примером технологии, которая явно выглядит как руткит BIOS, но обеспечивает постоянную функциональность для законных целей, например для возврата похищенного устройства. Такой вид постоянства позволяет основным компонентам Computrace работать независимо от ОС и глубоко интегрироваться с прошивкой UEFI. Для отключения Computrace злоумышленнику потребуется гораздо больше усилий, чем просто остановить его компонент агента ОС!

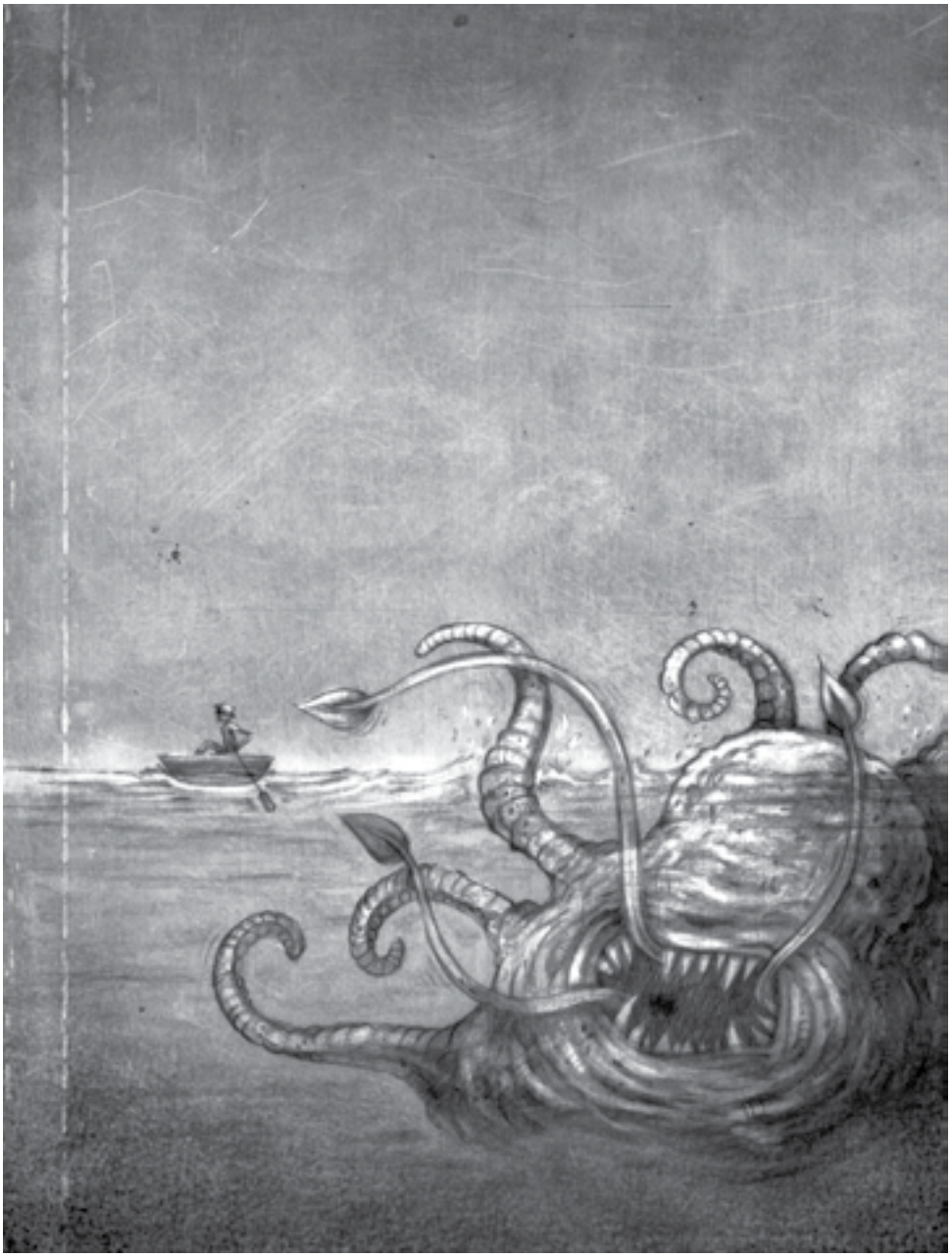
Заключение

Руткиты и имплантаты BIOS являются следующим этапом эволюции буткитов. Как мы увидели в этой главе, такая эволюция создаёт новый уровень постоянства в прошивке, который пока не учитывается антивирусным программным обеспечением, а значит, вредоносное ПО, использующее эти методы, может оставаться активным годами. Мы постарались дать подробный обзор руткитов BIOS, от первоначальных PoC и обнаруженных в реальных условиях образцов до передовых имплантатов UEFI. Однако эта тема сложна и для более глубокого освещения потребовала бы ещё множества глав. Мы рекомендуем перейти по приведённым ссылкам, продолжить самостоятельное чтение и следить за нашими блогами.

Подходы к противодействию вредоносному ПО такого рода всё ещё слабы, но верно и то, что поставщики оборудования продолжают внедрять всё более сложные реализации безопасной загрузки, в которых проверки целостности загрузки начинаются на более ранних этапах, ещё до запуска BIOS. В главе 17 мы подробнее рассмотрим современные реализации Secure Boot. На момент написания книги индустрия безопасности только начинает учиться проводить криминалистическое исследование прошивки, поскольку сведений о реальных случаях, к сожалению, мало. Более подробно криминалистику прошивки UEFI мы рассмотрим в последней главе.

Глава 16 посвящена уязвимостям UEFI. Насколько нам известно, ни одна другая книга до настоящего времени не освещала эту тему столь же подробно, так что держитесь крепче!

Глава 16. Уязвимости прошивки UEFI



В наши дни продукты безопасности обычно сосредоточены на угрозах, действующих на верхних уровнях программного стека, и достигают достаточно хороших результатов. Однако из-за этого они не способны видеть, что происходит в тёмных водах прошивки. Если злоумышленник уже получил привилегированный доступ к системе и установил имплантат прошивки, эти продукты бесполезны.

Лишь очень немногие продукты безопасности исследуют прошивку, а те, которые это делают, работают только с уровня операционной системы и обнаруживают имплантаты лишь после того, как они успешно установлены и скомпрометировали систему. Более сложные имплантаты также могут использовать своё привилегированное положение в системе, чтобы избегать обнаружения и подрывать работу продуктов безопасности уровня ОС.

По этим причинам руткиты и имплантаты прошивки относятся к наиболее опасным угрозам для ПК, а для современных облачных платформ они представляют ещё большую угрозу: одна неправильно настроенная или скомпрометированная гостевая операционная система подвергает опасности всех остальных гостей, открывая их память для вредоносных манипуляций.

Обнаружение аномалий прошивки является сложной технической задачей по многим причинам. Кодовые базы прошивки UEFI, предоставляемые разными поставщиками, различаются, и существующие методы обнаружения аномалий эффективны не во всех случаях. Злоумышленники также могут использовать в своих интересах как ложноположительные, так и ложноотрицательные результаты схемы обнаружения и даже перехватывать интерфейсы, через которые алгоритмы обнаружения уровня ОС получают доступ к прошивке и исследуют её.

Единственный жизнеспособный способ защиты от руткитов прошивки состоит в предотвращении их установки. Обнаружение и другие меры противодействия не работают; вместо этого необходимо блокировать возможные векторы заражения. Решения для обнаружения или предотвращения угроз прошивки работают только тогда, когда разработчик полностью контролирует как программный, так и аппаратный стек, как это делают Apple или Microsoft. У сторонних решений всегда будут слепые зоны.

В этой главе мы опишем большинство известных уязвимостей и векторов эксплуатации, применяемых для заражения прошивки UEFI. Сначала мы исследуем уязвимую прошивку, классифицируем виды слабостей и уязвимостей прошивки и проанализируем существующие меры её защиты. Затем опишем уязвимости Intel Boot Guard, модулей SMM, S3 Boot Script и Intel Management Engine.

Что делает прошивку уязвимой?

Начнём с обзора конкретных прошивок, на которые злоумышленники могут нацелить вредоносное обновление. Обновления являются наиболее эффективным методом заражения.

Поставщики обычно широко называют обновления прошивки UEFI обновлениями BIOS, поскольку BIOS является основной включённой прошивкой, однако типичное обновление также доставляет множество других видов встроенной прошивки различным аппаратным блокам внутри материнской платы и даже процессора. Скомпрометированное обновление BIOS уничтожает гарантии целостности для всех остальных обновлений прошивки, которыми управляет BIOS. Некоторые из этих обновлений, например микрокод Intel, имеют дополнительные методы аутентификации и не полагаются исключительно на BIOS. Поэтому любая уязвимость, обходящая аутентификацию образа обновления BIOS, также открывает путь для доставки вредоносных руткитов или имплантатов в любой из этих блоков.

На рисунке 16-1 показаны типичные блоки прошивки, которыми управляет BIOS; все они подвержены воздействию вредоносных обновлений BIOS.

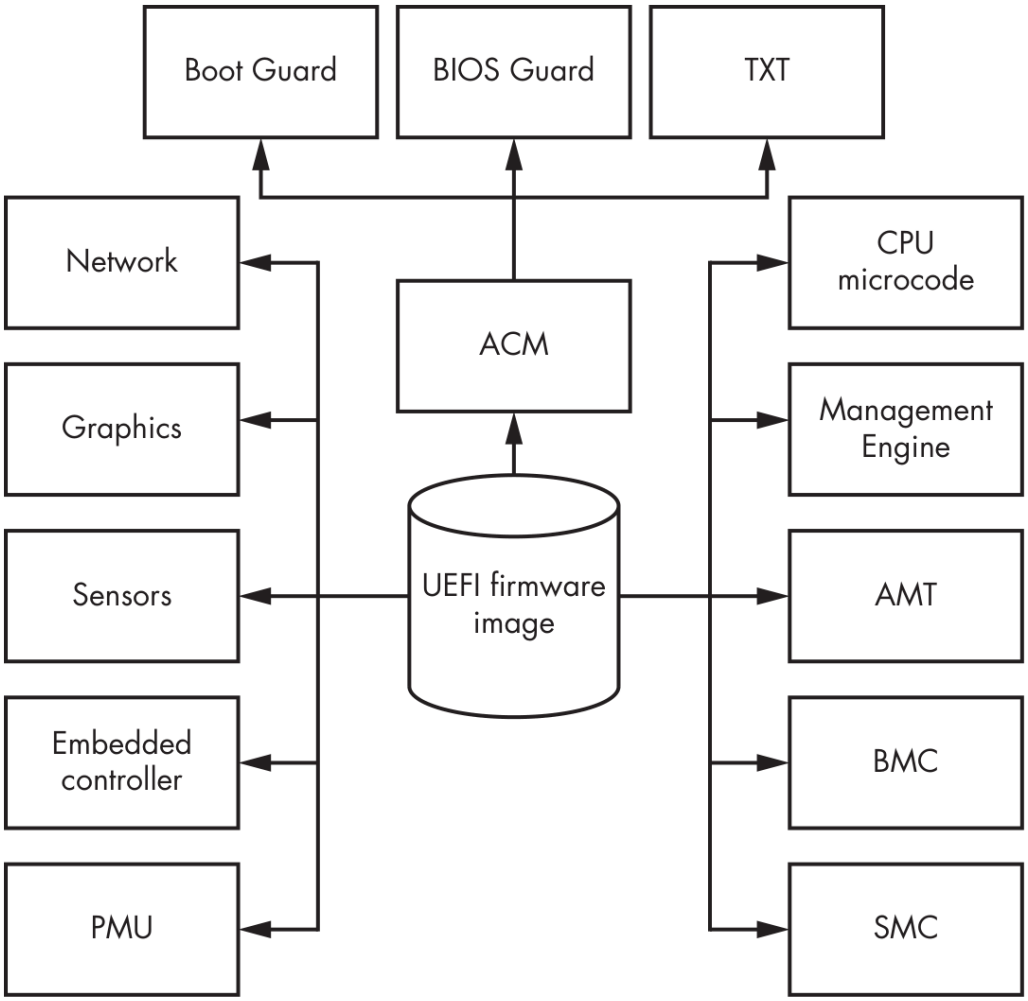


Рисунок 16-1. Обзор различных прошивок в современных компьютерах на основе x86

Ниже приведены краткие описания каждого вида прошивки.

Power Management Unit (PMU). Микроконтроллер, управляющий функциями питания и переходами ПК между различными состояниями питания, например сном и гибернацией. Он содержит собственную прошивку и маломощный процессор.

Intel Embedded Controller (EC). Постоянно включённый микроконтроллер. Он поддерживает множество функций, например включение и выключение компьютера, обработку сигналов с клавиатуры, расчёт температурных показателей и управление вентилятором. Он взаимодействует с основным процессором через ACPI, SMBus или общую память. EC вместе с описанным далее Intel Management Engine может выполнять роль аппаратного корня доверия, когда скомпрометирован режим системного управления. Например, технология Intel BIOS Guard в реализациях конкретных поставщиков использует EC для управления доступом на чтение и запись к SPI flash.

Intel Integrated Sensor Hub (ISH). Микроконтроллер, отвечающий за датчики, например датчики поворота устройства и автоматические регуляторы подсветки. Он также может отвечать за некоторые состояния сна с низким энергопотреблением для этих датчиков.

Graphics Processing Unit (GPU). Интегрированный графический процессор (iGPU), входящий в архитектуру Platform Controller Hub (PCH) в большинстве современных компьютеров Intel на основе x86. GPU имеют собственную развитую прошивку и вычислительные блоки, ориентированные на формирование графики, например шейдеры.

Intel Gigabit Network. Интегрированные Intel сетевые платы Ethernet для компьютеров на основе x86 представлены устройствами PCIe, подключёнными к PCH, и содержат собственную прошивку, доставляемую в образах обновлений BIOS.

Микрокод процессора Intel. Внутренняя прошивка процессора, представляющая собой интерпретирующий уровень, который интерпретирует ISA. Видимая программисту архитектура набора команд (instruction set architecture, ISA) является частью микрокода, но некоторые инструкции могут быть глубже интегрированы на аппаратном уровне. Микрокод Intel представляет собой уровень аппаратных инструкций, реализующих более высокоуровневые инструкции машинного кода и последовательность внутреннего конечного автомата во многих элементах цифровой обработки.

Authenticated Code Module (ACM). Подписанный двоичный объект, выполняемый в кэш-памяти. Микрокод Intel загружается и выполняется внутри защищённой внутренней памяти процессора, которая называется Authenticated Code RAM (ACRAM), или Cache-as-RAM (CAR). Эта быстрая память инициализируется на раннем этапе процесса загрузки. Она работает как обычная RAM до активации основной RAM и до выполнения кода вектора сброса для кода ACM ранней загрузки (Intel Boot Guard); её также можно загрузить позднее в процессе загрузки. Впоследствии она перепрофилируется под кэш общего назначения. ACM подписывается двоичным объектом RSA с заголовком, определяющим его точку входа. Современные компьютеры Intel могут иметь несколько ACM для разных целей, но главным образом они используются для поддержки дополнительных функций безопасности платформы.

Intel Management Engine (ME). Микроконтроллер, предоставляющий функциональность корня доверия для нескольких разработанных Intel средств безопасности, включая программный интерфейс к прошивочному Trusted Platform Module, или fTPM. Обычно TPM является специализированной микросхемой оконечного устройства для аппаратной аутентификации, также содержащей собственную отдельную прошивку. Начиная с шестого поколения процессоров Intel, Intel ME представляет собой микроконтроллер на основе x86.

Intel Active Management Technology (AMT). Аппаратная и программная платформа для удалённого управления персональными компьютерами и серверами. Она предоставляет удалённый доступ к мониторам, клавиатурам и другим устройствам. В её состав входит основанная на наборе микросхем Intel технология Baseboard Management Controller для клиентских платформ, рассматриваемая далее и интегрированная в Intel ME.

Baseboard Management Controller (BMC). Набор спецификаций компьютерных интерфейсов для автономной компьютерной подсистемы, предоставляющей возможности управления и мониторинга независимо от процессора основной системы, прошивки UEFI и операционной системы реального времени. BMC обычно реализуется на отдельной микросхеме с собственным сетевым интерфейсом Ethernet и прошивкой.

System Management Controller (SMC). Микроконтроллер на системной плате, управляющий функциями питания и датчиками. Чаще всего встречается в компьютерах производства Apple.

Каждый блок прошивки предоставляет злоумышленнику возможность хранить и выполнять код, а для сохранения целостности все блоки зависят друг от друга. Например, Алекс Матросов выявил проблему в современном оборудовании Gigabyte, где ME позволял BIOS записывать в свои области памяти и читать из них. В сочетании со слабой конфигурацией Intel Boot Guard эта проблема позволяла полностью обойти аппаратную реализацию Boot Guard. Дополнительные

сведения об этой уязвимости, впоследствии подтверждённой и исправленной поставщиком, см. в CVE-2017-11313 и CVE-2017-11314. Далее в этой главе мы рассмотрим реализации Boot Guard и возможные способы их обхода.

Основная цель руткита BIOS состоит в поддержании постоянного и скрытого заражения, как и у описанных ранее в книге руткитов режима ядра и буткитов MBR/VBR. Однако руткит BIOS может иметь и дополнительные интересные цели. Например, он может попытаться временно получить управление режимом системного управления (SMM) или непривилегированной средой выполнения драйверов (Driver Execution Environment, DXE), работающей вне SMM, чтобы проводить скрытые операции с памятью или файловой системой. Даже непостоянная атака, выполняемая из SMM, способна обходить границы безопасности современных систем Windows, включая безопасность на основе виртуализации (virtualization-based security, VBS) и экземпляры гостевых виртуальных машин.

Классификация уязвимостей прошивки UEFI

Прежде чем углубляться в уязвимости, классифицируем виды изъянов безопасности, на которые может быть нацелена установка имплантата BIOS. Все классы уязвимостей, показанные на рисунке 16-2, способны помочь злоумышленнику нарушить границы безопасности и установить постоянные имплантаты.

Исследователи Intel первыми попытались классифицировать уязвимости прошивки UEFI в соответствии с потенциальными последствиями атаки на такую уязвимость. Они представили свою классификацию на Black Hat USA 2017 в Лас-Вегасе в докладе «Firmware Is the New Black - Analyzing Past Three Years of BIOS/UEFI Security Vulnerabilities» (https://www.youtube.com/watch?v=SeZO5AYsBCw), который охватывал разные классы проблем безопасности и некоторые меры противодействия. Одним из важнейших его результатов стала статистика роста общего числа проблем безопасности, обработанных Intel PSIRT.

У нас другая классификация проблем безопасности, связанных с прошивкой UEFI, сосредоточенная на последствиях руткитов прошивки и показанная на рисунке 16-2.

> **Примечание.** Модель угроз, представленная на рисунке 16-2, охватывает только потоки, относящиеся к прошивке UEFI, но масштаб проблем безопасности Intel ME и AMT значительно растёт. Кроме того, за последние несколько лет BMC превратился в очень важный ресурс безопасности для платформ удалённого управления серверами и привлекает большое внимание исследователей.

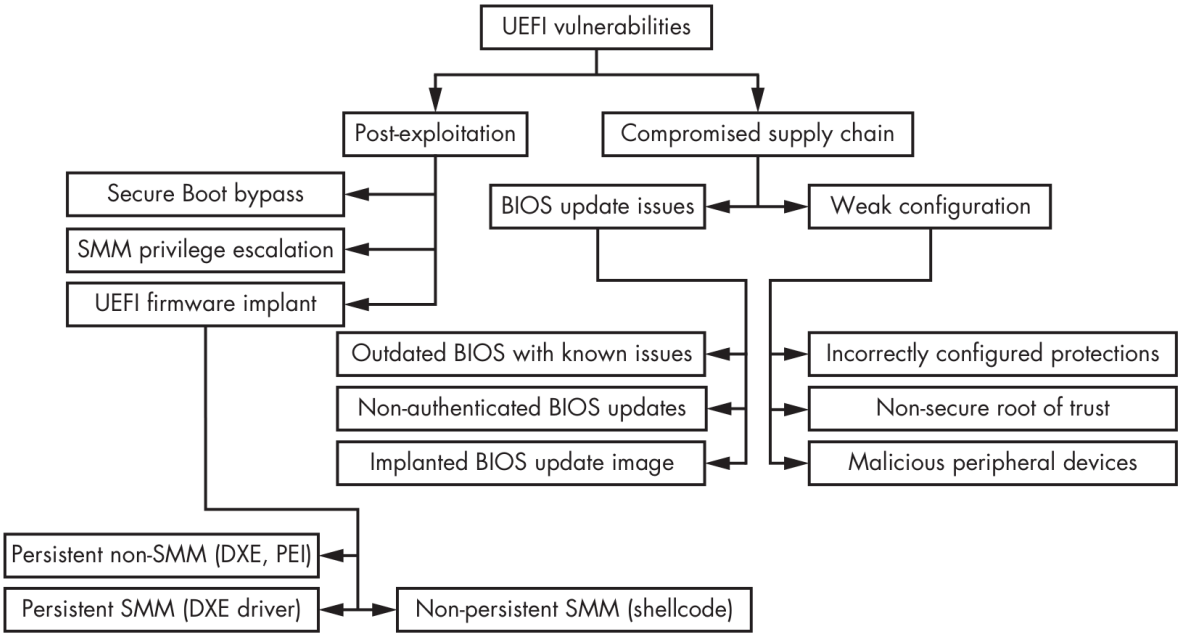


Рисунок 16-2. Классификация уязвимостей BIOS, полезных для установки имплантатов BIOS

Предложенные на рисунке 16-2 классы уязвимостей можно разделить по способу их использования на две основные группы: уязвимости после эксплуатации и уязвимости скомпрометированной цепочки поставок.

Уязвимости после эксплуатации

Уязвимости после эксплуатации обычно используются как второй этап доставки вредоносных полезных нагрузок; эта схема эксплуатации описана в главе 15. Это основная категория уязвимостей, которыми пользуются злоумышленники для установки как постоянных, так и непостоянных имплантатов после успешной эксплуатации предыдущих этапов атаки. Ниже приведены классы основных имплантатов, эксплойтов и уязвимостей этой категории.

Обход Secure Boot. Злоумышленники сосредоточены на компрометации процесса Secure Boot посредством эксплуатации корня доверия, то есть полной компрометации, или другой уязвимости на одном из этапов загрузки. Обходы Secure Boot могут происходить на разных этапах загрузки и использоваться злоумышленником против всех последующих уровней и их механизмов доверия.

Повышение привилегий SMM. SMM обладает большой властью над оборудованием x86, поскольку почти все проблемы повышения привилегий для SMM в итоге становятся проблемами выполнения кода. Повышение привилегий до SMM часто является одним из заключительных этапов установки имплантата BIOS.

Имплантат прошивки UEFI. Имплантат прошивки UEFI является заключительным этапом установки постоянного имплантата BIOS. Злоумышленник может установить имплантат на разных уровнях прошивки UEFI, либо как изменённый легитимный модуль, либо как самостоятельный драйвер, например DXE или PEI, которые мы рассмотрим далее.

Постоянный имплантат. Постоянный имплантат способен пережить полные циклы перезагрузки и выключения. В некоторых случаях, чтобы пережить процесс обновления, он может изменять образы обновлений BIOS до их установки.

Непостоянный имплантат. Непостоянный имплантат не переживает полные циклы перезагрузки и выключения. Такие имплантаты могут обеспечивать повышение привилегий и выполнение кода внутри ОС при наличии защищённой аппаратной виртуализации, например Intel VT-x, и уровней доверенного выполнения, например MS VBS. Они также могут использоваться как скрытые каналы для доставки вредоносных полезных нагрузок в режим ядра операционной системы.

Уязвимости скомпрометированной цепочки поставок

Атаки на скомпрометированную цепочку поставок используют ошибки, допущенные командой разработки BIOS или OEM-поставщиком оборудования, либо намеренные неправильные настройки целевого программного обеспечения, предоставляющие злоумышленникам допускающий правдоподобное отрицание обход средств безопасности платформы.

При атаках на цепочку поставок злоумышленник получает доступ к оборудованию во время его производства и изготовления и внедряет вредоносные изменения в прошивку либо устанавливает вредоносные периферийные устройства ещё до того, как оборудование попадёт к потребителю. Атаки на цепочку поставок могут происходить и удалённо: злоумышленник получает доступ к внутренней сети разработчика прошивки, а иногда к сайту поставщика, и доставляет вредоносные изменения непосредственно в репозиторий исходного кода или на сервер сборки.

Атаки на цепочку поставок с физическим доступом включают скрытое вмешательство в целевую платформу и иногда похожи на атаки «злой горничной», при которых злоумышленник имеет физический доступ в течение ограниченного времени и использует его для эксплуатации уязвимости цепочки поставок. Такие атаки используют ситуации, в которых владелец оборудования не может контролировать физический доступ к нему: например, оставляет ноутбук в сдаваемом багаже, передаёт его для досмотра иностранной таможне или просто забывает в гостиничном номере. Злоумышленник может использовать эти возможности, чтобы неправильно настроить оборудование и прошивку для доставки имплантатов BIOS или просто физически прошить вредоносную прошивку в микросхему SPI flash.

Большинство следующих проблем применимо к сценариям атак на цепочку поставок и атак «злой горничной».

Неправильно настроенные средства защиты. Атакуя оборудование или прошивку во время разработки либо на этапе после производства, злоумышленник может неправильно настроить технологические средства защиты так, чтобы впоследствии их было легко обойти.

Незащищённый корень доверия. Эта уязвимость включает компрометацию корня доверия из операционной системы через её интерфейсы взаимодействия с прошивкой, например SMM.

Вредоносные периферийные устройства. Атака такого рода включает внедрение периферийных устройств на этапах производства или доставки. Вредоносные устройства можно использовать несколькими способами, например для атак прямого доступа к памяти (Direct Memory Access, DMA).

Заражённые обновления BIOS. Злоумышленник может скомпрометировать сайт поставщика или другой механизм удалённого обновления и использовать его для доставки заражённого обновления BIOS. Точками компрометации могут быть серверы сборки поставщика, системы разработчиков или похищенные цифровые сертификаты с закрытыми ключами поставщика.

Неаутентифицированный процесс обновления BIOS. Поставщики могут намеренно или ненамеренно нарушить процесс аутентификации обновлений BIOS, позволив злоумышленникам вносить в образы обновлений любые желаемые изменения.

Устаревшая BIOS с известными проблемами безопасности. Разработчики BIOS могут продолжать использовать старые уязвимые версии кода прошивки BIOS даже после исправления

основной кодовой базы, что делает прошивку уязвимой для атак. Первоначально поставленная производителем оборудования устаревшая версия BIOS, вероятно, останется без обновлений на пользовательских ПК или серверах центра обработки данных. Это один из самых распространённых изъянов безопасности, связанных с прошивкой BIOS.

Противодействие уязвимостям цепочки поставок

Снизить риски, связанные с цепочками поставок, без радикальных изменений жизненных циклов разработки и производства очень трудно. Типичная выпускаемая клиентская или серверная платформа включает множество сторонних компонентов как в программном, так и в аппаратном обеспечении. Большинство компаний, которые не владеют всем производственным циклом, не слишком заботятся о безопасности и в действительности не могут себе этого позволить.

Ситуацию усугубляет общий недостаток информации и ресурсов, относящихся к настройке безопасности BIOS и набора микросхем. Публикации NIST 800-147 («BIOS Protection Guidelines») и NIST 800-147B («BIOS Protection Guidelines for Servers») служат полезной отправной точкой, но быстро устаревают после первоначального выпуска в 2011 году и обновления для серверов в 2014 году.

Чтобы заполнить некоторые из этих пробелов в широко распространённых знаниях, подробно рассмотрим ряд атак на прошивку UEFI.

История средств защиты прошивки UEFI

В этом разделе мы рассмотрим несколько классов уязвимостей, позволяющих злоумышленнику обойти Secure Boot; конкретные детали реализации Secure Boot будут обсуждаться в следующей главе.

Ранее любая проблема безопасности, позволявшая злоумышленнику выполнять код в среде SMM, могла обеспечить обход Secure Boot. Хотя некоторые современные аппаратные платформы даже с новыми аппаратными обновлениями всё ещё уязвимы для атак на Secure Boot на основе SMM, большинство поставщиков корпоративного оборудования перешло на новейшие средства безопасности Intel, усложняющие такие атаки.

Современные технологии Intel, например Intel Boot Guard и BIOS Guard, обе из которых будут рассмотрены далее в этой главе, перемещают корень доверия процесса загрузки из SMM в более защищённую среду: прошивку и оборудование Intel ME.

> **Корень доверия**
>
> Корень доверия представляет собой проверенный криптографический ключ, служащий якорем Secure Boot. Secure Boot создаёт проверенный оборудованием процесс загрузки, гарантирующий, что платформа может быть запущена только с доверенным кодом, который успешно проверен посредством корня доверия. В современных архитектурах платформ корень доверия блокируется в защищённом аппаратном хранилище, например в однократно программируемых предохранителях или отдельной микросхеме с постоянным хранилищем.

Первая версия UEFI Secure Boot была представлена в 2012 году. Её основные компоненты включали корень доверия, реализованный на этапе загрузки DXE, одном из последних этапов загрузки прошивки UEFI непосредственно перед передачей управления ОС. Это означало, что ранняя реализация Secure Boot фактически обеспечивала целостность только загрузчиков ОС, но не самой BIOS.

Вскоре слабости этой архитектуры стали очевидны, и в следующей реализации корень доверия был перемещён в PEI, ранний этап инициализации платформы, где блокировался до DXE. Эта граница безопасности также оказалась слабой. С 2013 года, после выпуска технологии Intel Boot Guard, корень доверия блокируется в оборудовании посредством микросхемы TPM или эквивалентной функциональности, реализованной в прошивке ME для снижения стоимости поддержки. Field-programmable fuses (FPF) расположены в наборе микросхем материнской платы, компоненте PCH, программируемом через прошивку ME.

Прежде чем углубляться в историю соответствующих атак, побудивших к этим переработкам, рассмотрим принцип работы основных технологий защиты BIOS.

Принцип работы средств защиты BIOS

На рисунке 16-3 приведено высокоуровневое представление технологий, используемых для защиты постоянного хранилища SPI flash. Изначально SMM был разрешён доступ как на чтение, так и на запись к хранилищу SPI flash для реализации обычных обновлений BIOS. Это означало, что целостность BIOS зависела от качества любого кода, выполняемого в SMM, поскольку любой такой код мог изменить BIOS в хранилище SPI. Следовательно, граница безопасности была настолько же слабой, насколько самый слабый код, когда-либо выполнявшийся в SMM и имевший доступ к области памяти за его пределами. Поэтому разработчики платформ предприняли меры для отделения обновлений BIOS от остальной функциональности SMM, внедрив ряд дополнительных средств контроля безопасности, например Intel BIOS Guard.

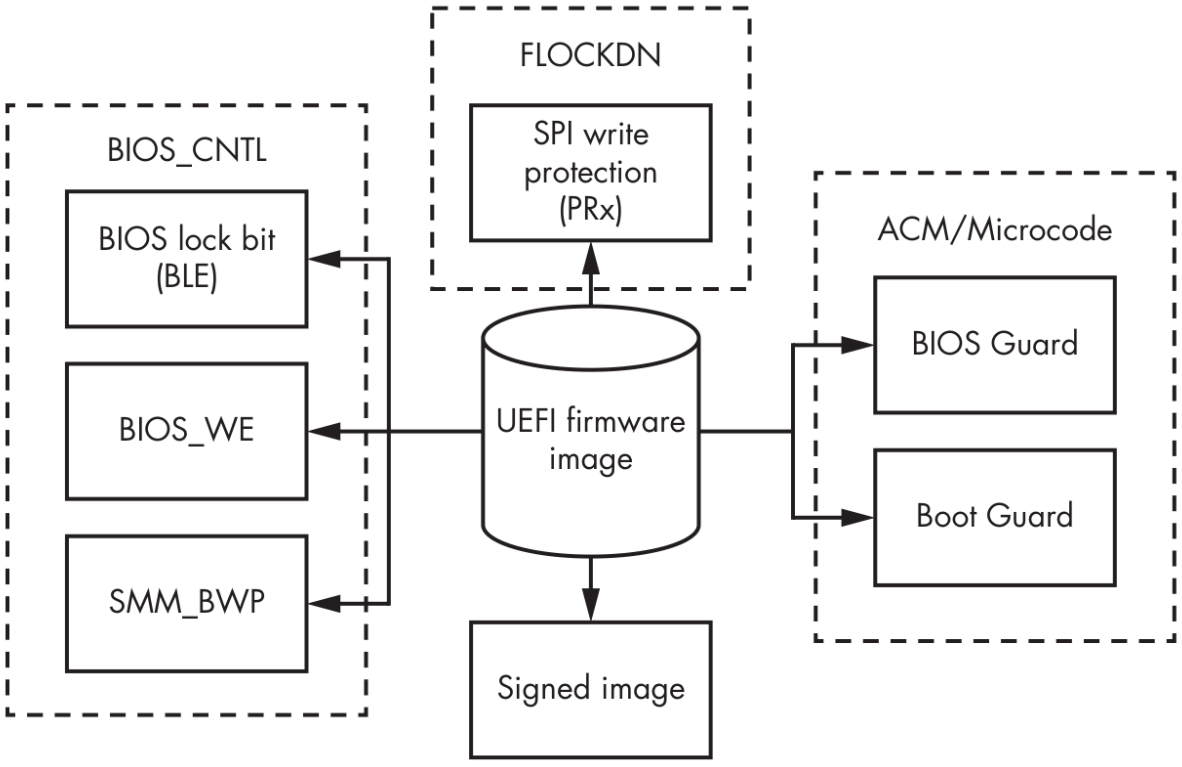


Рисунок 16-3. Высокоуровневое представление технологий безопасности BIOS

Средства защиты SPI flash и их уязвимости

Некоторые средства контроля, показанные на рисунке 16-3, мы рассматривали в разделе «(Не)эффективность битов защиты памяти» на странице 263: защиту управляющего бита BIOS (BIOS Control Bit Protection, BIOS_CNTL), блокировку конфигурации flash (Flash Configuration Lock-Down, FLOCKDN) и защиту SPI flash от записи (SPI flash Write Protection, PRx). Однако средства защиты BIOS_CNTL эффективны только против злоумышленника, пытающегося изменить BIOS из ОС, и могут быть обойдены любой уязвимостью выполнения кода из SMM, то есть через обработчики SMI, доступные извне, поскольку код SMM может свободно изменять эти биты защиты. По сути, BIOS_CNTL лишь создаёт иллюзию безопасности.

Изначально SMM имел доступ как на чтение, так и на запись к хранилищу SPI flash, чтобы реализовывать обычные обновления BIOS. Это делало целостность BIOS зависимой от качества любого кода, выполнявшегося в SMM с обращениями к внешним областям памяти, поскольку любой такой код мог изменить BIOS в хранилище SPI. Эта граница безопасности оказалась довольно слабой: настолько же слабой, насколько самый слабый код, когда-либо выполнявшийся в SMM.

В результате разработчики платформ предприняли меры для отделения обновлений BIOS от остальной функциональности SMM. Многие из этих средств контроля сами по себе были довольно слабыми. Примером служит защита управляющего бита BIOS (BIOS_CNTL), эффективная только против злоумышленника, пытающегося изменить BIOS из операционной системы; её можно обойти с помощью любой уязвимости выполнения кода из SMM, поскольку код SMM может свободно изменять эти биты защиты.

Средство контроля PRx эффективнее, поскольку его политики невозможно изменить из SMM. Однако, как мы вскоре обсудим, многие поставщики не используют защиту PRx, в том числе Apple и, как ни удивительно, Intel, создавшая эту технологию защиты.

В таблице 16-1 обобщено состояние активных технологий защиты на основе блокирующих битов безопасности в оборудовании на основе x86 популярных поставщиков по состоянию на январь 2018 года. Здесь RP означает защиту от чтения, а WP - защиту от записи.

Таблица 16-1. Уровень безопасности оборудования популярных поставщиков

Поставщик	BLE	SMM_BWP	PRx	Аутентифицированное обновление
ASUS	Активно	Активно	Неактивно	Неактивно
MSI	Неактивно	Неактивно	Неактивно	Неактивно
Gigabyte	Активно	Активно	Неактивно	Неактивно
Dell	Активно	Активно	RP/WP	Активно
Lenovo	Активно	Активно	RP	Активно
HP	Активно	Активно	RP/WP	Активно
Intel	Активно	Активно	Неактивно	Активно
Apple	Неактивно	Неактивно	WP	Активно

Как видим, подходы поставщиков к безопасности BIOS различаются чрезвычайно сильно. Некоторые из них даже не аутентифицируют обновления BIOS, создавая серьёзную проблему безопасности, поскольку это значительно облегчает установку имплантатов, если только поставщик не применяет политики Intel Boot Guard.

Кроме того, для эффективности защиту PRx необходимо настроить правильно. В листинге 16-1 показан пример неправильно настроенных областей flash, где все определения сегментов PRx установлены в ноль, что делает их бесполезными.

```
[*] BIOS Region: Base = 0x00800000, Limit = 0x00FFFFFF
SPI Protected Ranges
------------------------------------------------------------
PRx (offset) | Value    | Base     | Limit    | WP? | RP?
------------------------------------------------------------
PR0 (74)     | 00000000 | 00000000 | 00000000 | 0   | 0
PR1 (78)     | 00000000 | 00000000 | 00000000 | 0   | 0
PR2 (7C)     | 00000000 | 00000000 | 00000000 | 0   | 0
PR3 (80)     | 00000000 | 00000000 | 00000000 | 0   | 0
PR4 (84)     | 00000000 | 00000000 | 00000000 | 0   | 0
```

Листинг 16-1. Неправильно настроенные политики доступа PRx (дамп инструмента Chipsec)

Мы также видели, как некоторые поставщики настраивают политики только для защиты от чтения, что всё равно позволяет злоумышленнику изменять SPI flash. Кроме того, PRx не гарантирует каких-либо измерений целостности фактического содержимого SPI, поскольку лишь реализует побитовую блокировку прямого доступа на чтение и запись на самом раннем этапе PEI процесса загрузки.

Причина, по которой такие поставщики, как Apple и Intel, обычно отключают защиту PRx, состоит в том, что эта защита требует немедленной перезагрузки, делая обновление BIOS менее удобным. Без защиты PRx инструмент обновления BIOS поставщика может записать новый образ BIOS в свободную область физической памяти через API ОС, а затем вызвать прерывание SMI, чтобы вспомогательный код в SMM взял образ из этой области и записал его в SPI flash. Обновлённый образ SPI flash получает управление при следующей перезагрузке, но она может произойти позднее, когда будет удобно пользователю.

Когда PRx включён и правильно настроен для защиты соответствующих областей SPI от изменений кодом SMM, инструмент обновления BIOS больше не может использовать SMM для модификации BIOS. Вместо этого он должен сохранить образ обновления в динамической памяти с произвольным доступом (DRAM) и вызвать немедленную перезагрузку. Вспомогательный код для установки обновления должен быть частью специального драйвера раннего этапа загрузки, который выполняется до активации защиты PRx и переносит образ обновления из DRAM в SPI. Этот метод обновления иногда требует перезагрузки, либо прямого вызова обработчика SMI без перезагрузки, непосредственно при запуске инструмента, что гораздо менее удобно для пользователя.

Какой бы путь ни использовал инструмент обновления BIOS, крайне важно, чтобы вспомогательный код аутентифицировал образ обновления до его установки. В противном случае, с PRx или без PRx, с перезагрузкой или без неё, вспомогательный код охотно установит изменённый образ BIOS с имплантатом, если злоумышленнику удастся изменить его до запуска вспомогательного кода. Как показывает таблица 16-1, некоторые поставщики оборудования не аутентифицируют обновления прошивки, превращая задачу злоумышленника в простое вмешательство в образ обновления.

Первая публично известная атака на процесс обновления BIOS

Имейте в виду, что даже при правильной настройке PRx и аутентификации криптографических подписей обновлений BIOS система всё равно может быть подвержена атакам. Первая публично известная атака против аутентифицированного и подписанного процесса обновления BIOS с активными битами защиты SPI flash была представлена в докладе Рафала Войтчука и Алекса Терёшкина «Attacking Intel BIOS» на Black Hat Vegas в 2009 году. Авторы продемонстрировали уязвимость повреждения памяти внутри синтаксического анализатора файла образа обновления BIOS, которая приводила к выполнению произвольного кода и обходила аутентификацию подписи файла обновления.

Риски неаутентифицированного обновления BIOS

В сентябре 2018 года антивирусная компания ESET выпустила исследовательский отчёт о LOJAX, рутките, атаковавшем прошивку UEFI из ОС.[1] Все методы, использованные руткитом LOJAX, были хорошо известны на момент атаки и применялись в других обнаруженных вредоносных программах в течение предыдущих пяти лет. LOJAX использовал тактику, похожую на руткит UEFI компании Hacking Team: он злоупотреблял неаутентифицированными компонентами Computrace, хранившимися в NTFS, как мы рассматривали в главе 15. Таким образом, руткит LOJAX не использует новых уязвимостей; его единственная новизна состоит в способе заражения целей: он проверяет системы на наличие неаутентифицированного доступа к SPI flash и, обнаружив его, доставляет изменённый файл обновления BIOS.

Небрежные подходы к безопасности BIOS предоставляют множество возможностей для атак. Злоумышленник может сканировать систему во время выполнения, чтобы найти подходящие уязвимые цели и нужный вектор заражения, которых имеется предостаточно. Инфектор руткита LOJAX проверял несколько средств защиты, включая бит блокировки BIOS (BLE) и бит защиты BIOS от записи в SMM (SMM_BWP). Если прошивка не была аутентифицирована или не проверяла целостность образа обновления BIOS перед его переносом в хранилище SPI, злоумышленник мог доставлять изменённые обновления непосредственно из ОС. LOJAX использовал уязвимость Speed Racer (VU#766164, первоначально обнаруженную Кори Калленбергом в 2014 году), чтобы обойти биты защиты SPI flash посредством состояния гонки. Эту уязвимость и другие слабости, связанные с блокирующими битами защиты BIOS, можно обнаружить командой `chipsec_main -m common.bios_wp`.

Этот пример показывает, что граница безопасности сильна лишь настолько, насколько силён её самый слабый компонент. Независимо от прочих средств защиты платформы небрежная аутентификация кода в Computrace подрывала их, вновь открывая вектор атаки со стороны ОС, который другие средства защиты пытались устранить. Чтобы затопить равнину, достаточно одной бреши в морской стене.

Защита BIOS посредством Secure Boot

Как Secure Boot меняет этот ландшафт угроз? Короткий ответ: это зависит от её реализации. Старые версии, реализованные до 2016 года без технологий Intel Boot Guard и BIOS Guard, будут подвержены опасности, поскольку в этих старых реализациях корень доверия находится в SPI flash и может быть перезаписан.

Когда в 2012 году была представлена первая версия UEFI Secure Boot, её основные компоненты включали корень доверия, реализованный на этапе загрузки DXE, одном из последних этапов загрузки прошивки UEFI непосредственно перед передачей управления ОС. Поскольку корень доверия появлялся столь поздно

в процессе загрузки, эта ранняя реализация Secure Boot в действительности обеспечивала только целостность загрузчиков ОС, а не самой BIOS. Слабость архитектуры вскоре стала очевидна, и в следующей реализации корень доверия был перемещён в PEI, ранний этап инициализации платформы, чтобы заблокировать его до DXE. Эта граница безопасности также оказалась слабой.

Более новые дополнения к Secure Boot, Boot Guard и BIOS Guard, устраняют эту слабость: Boot Guard переместил корень доверия из SPI в оборудование, а BIOS Guard перенёс задачу обновления содержимого SPI flash из SMM в отдельную микросхему Intel Embedded Controller (EC) и удалил разрешения, позволявшие SMM записывать в SPI flash.

Ещё одна причина перемещения корня доверия на более ранний этап процесса загрузки и в оборудование состоит в сокращении времени загрузки доверенной платформы. Можно представить схему защиты загрузки, проверяющую цифровые подписи десятков отдельных доступных образов EFI вместо одного образа, включающего все драйверы. Однако для современного мира это было бы слишком медленно, поскольку поставщики платформ стремятся сэкономить миллисекунды времени загрузки.

В этот момент у вас может возникнуть вопрос: если в процессе Secure Boot участвует столько движущихся частей, как избежать ситуаций, в которых тривиальная ошибка разрушает все гарантии безопасности? Полный процесс Secure Boot рассматривается в главе 17. Лучший на сегодняшний день ответ состоит в использовании инструментов, проверяющих, что каждый компонент выполняет назначенную роль, а каждый этап процесса загрузки происходит строго в предусмотренном порядке. Иными словами, нужна формальная модель процесса, которую могут проверять автоматизированные средства анализа кода; чем проще модель, тем больше уверенности в правильности её проверки.

Secure Boot опирается на цепочку доверия: предусмотренный путь выполнения начинается с корня доверия, заблокированного в оборудовании или хранилище SPI flash, и проходит через этапы процесса Secure Boot, которые могут выполняться только в определённом порядке и лишь при соблюдении всех условий и политик на каждом этапе.

Формально эта модель называется конечным автоматом, где разные состояния представляют разные этапы процесса загрузки системы. Если какой-либо этап обладает недетерминированным поведением, например может переключить процесс загрузки в другой режим или иметь несколько выходов, процесс Secure Boot превращается в недетерминированный конечный автомат. Это значительно усложняет автоматическую проверку процесса Secure Boot, поскольку экспоненциально увеличивает число путей выполнения, которые необходимо проверить. По нашему мнению, недетерминированное поведение Secure Boot следует считать архитектурной ошибкой, способной привести к дорогостоящим уязвимостям, как в случае уязвимости S3 Boot Script, рассматриваемой далее в этой главе.

Intel Boot Guard

В этом разделе мы обсудим принцип работы технологии Intel Boot Guard, а затем исследуем некоторые её уязвимости. Хотя у Intel нет общедоступной официальной документации по Boot Guard, наши исследования и исследования других специалистов позволяют составить целостное представление об этой замечательной технологии.

Технология Intel Boot Guard

Boot Guard разделяет Secure Boot на два этапа: на первом Boot Guard аутентифицирует всё, что находится в разделе BIOS хранилища SPI, а на втором Secure Boot обрабатывает остальную часть процесса загрузки, включая аутентификацию загрузчика ОС (рисунок 16-4).

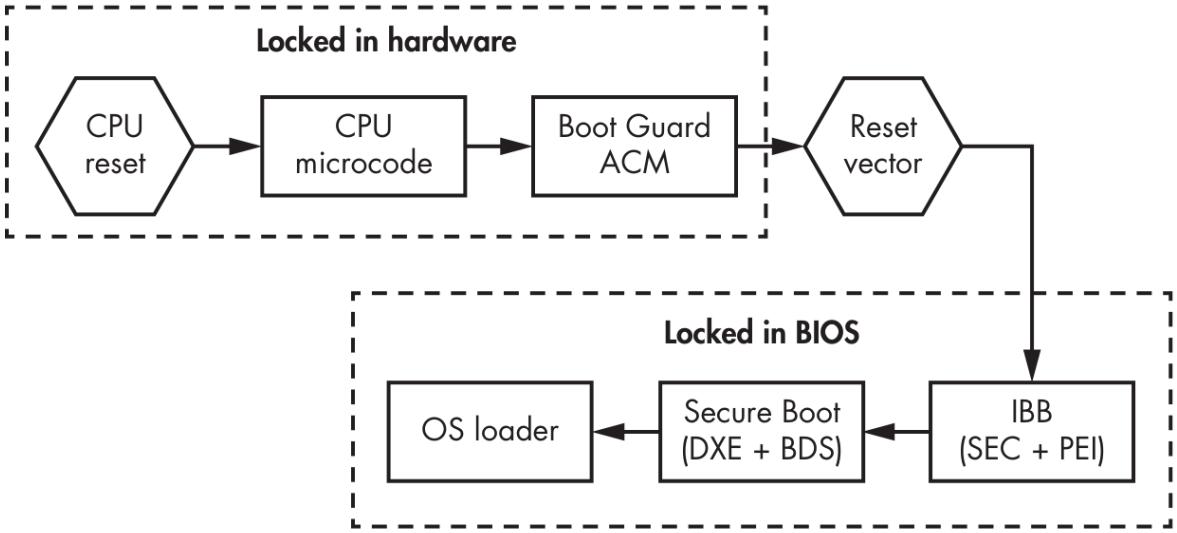


Рисунок 16-4. Процесс загрузки с активной технологией Intel Boot Guard

Технология Intel Boot Guard охватывает несколько уровней архитектуры процессора и связанных с ней абстракций. Одно из преимуществ состоит в отсутствии необходимости доверять хранилищу SPI, что позволяет избежать уязвимостей, рассмотренных ранее в этой главе. Boot Guard отделяет проверку целостности BIOS, хранящейся в SPI flash, от самой BIOS, используя подписанный Intel модуль Authenticated Code Module (ACM), чтобы проверить целостность образа BIOS до разрешения его выполнения. При активации Boot Guard на платформе корень доверия перемещается внутрь микроархитектуры Intel, где микрокод процессора разбирает содержимое ACM и проверяет процедуры проверки цифровой подписи, реализованные в ACM, которые, в свою очередь, проверяют подпись BIOS.

Напротив, корень доверия исходной UEFI Secure Boot находился на этапе UEFI DXE, почти последнем перед передачей управления загрузчику ОС, то есть, как мы уже отмечали, появлялся очень поздно. Если прошивка UEFI скомпрометирована на этапе DXE, злоумышленник может полностью обойти или отключить Secure Boot. Без аппаратной проверки невозможно гарантировать целостность этапов процесса загрузки, происходящих до этапа DXE, включая целостность самих драйверов DXE; реализация PEI также имеет подтверждённые слабости.

Boot Guard решает эту проблему, перемещая корень доверия Secure Boot из прошивки UEFI в само оборудование. Например, Verified Boot, новый вариант Boot Guard, представленный Intel в 2013 году и подробнее рассматриваемый в следующей главе, блокирует хеш открытого ключа OEM в хранилище field programmable fuse (FPF). FPF можно запрограммировать только один раз, и

поставщик оборудования блокирует конфигурацию к концу процесса производства. В некоторых случаях блокировку можно отозвать, но эти пограничные случаи мы здесь обсуждать не будем.

Уязвимости Boot Guard

Эффективность Boot Guard зависит от совместной работы всех компонентов без уязвимостей на каком-либо уровне, которые позволили бы злоумышленнику выполнить код или повысить привилегии для вмешательства в другие компоненты многоуровневой схемы Secure Boot. В докладе Алекса Матросова «Betraying the BIOS: Where the Guardians of the BIOS Are Failing» (https://www.youtube.com/watch?v=Dfl2JI2eLc8), представленном на Black Hat USA 2017, было показано, что злоумышленник может успешно атаковать схему, вмешиваясь в битовые флаги, устанавливаемые нижними уровнями для передачи верхним уровням сведений о состоянии их целостности.

Как было продемонстрировано, прошивке нельзя доверять, поскольку большинство атак SMM способно её скомпрометировать. Даже схема Measured Boot, полагающаяся на TPM как на корень доверия, может быть скомпрометирована, поскольку сам измеряющий код выполняется в SMM и во многих случаях может быть изменён из SMM, хотя ключ, хранящийся в оборудовании TPM, невозможно изменить из SMM. Несмотря на возможность некоторых атак на микросхему TPM, злоумышленникам с привилегиями SMM они не нужны, поскольку можно просто атаковать интерфейсы прошивки с TPM. В 2013 году Intel представила упомянутую ранее Verified Boot, чтобы устранить эту слабость Measured Boot.

Логика проверки ACM в Boot Guard измеряет начальный загрузочный блок (initial boot block, IBB) и проверяет его целостность перед передачей управления точке входа IBB. Если проверка IBB не проходит, процесс загрузки обычно прерывается в зависимости от политики. Часть IBB прошивки UEFI (BIOS) выполняется на обычном процессоре, без изоляции или аутентификации. Затем IBB продолжает процесс загрузки, следуя политикам Boot Guard в проверенном или измеренном режиме до этапа инициализации платформы. Драйвер PEI проверяет целостность драйверов DXE и переводит цепочку доверия на этап DXE. Затем этап DXE продолжает цепочку доверия до загрузчика операционной системы. В таблице 16-2 представлены исследовательские данные о состоянии безопасности каждого из этих этапов у разных поставщиков оборудования.

Таблица 16-2. Настройка безопасности разными поставщиками оборудования (по состоянию на январь 2018 года)

Поставщик	Доступ к ME	Доступ к EC	Отладка CPU (DCI)	Boot Guard	Принудительный ACM Boot Guard	FPF Boot Guard	BIOS Guard
ASUS VivoMini	Отключён	Отключён	Включена	Отключён	Отключён	Отключён	Отключён
MSI Cubi2	Отключён	Отключён	Включена	Отключён	Отключён	Отключён	Отключён
Gigabyte Brix	Чтение/запись включены	Чтение/запись включены	Включена	Измеренный/проверенный	Включён (FPF не установлен)	Не установлен	Отключён
Dell	Отключён	Отключён	Включена	Измеренный/проверенный	Включён	Включён	Включён
Lenovo ThinkCenter	Отключён	Отключён	Включена	Отключён	Отключён	Отключён	Отключён
HP Elitedesk	Отключён	Отключён	Включена	Отключён	Отключён	Отключён	Отключён
Intel NUC	Отключён	Отключён	Включена	Отключён	Отключён	Отключён	Отключён

Поставщик	Доступ к ME	Доступ к EC	Отладка CPU (DCI)	Boot Guard	Принудительный ACM Boot Guard	FPF Boot Guard	BIOS Guard
Apple	Чтение включено	Отключён	Отключена	Не поддерживается	Не поддерживается	Не поддерживается	Не поддерживается

Как видим, катастрофически неправильные настройки этих параметров безопасности существуют не только в теории. Например, некоторые поставщики не записали свои хеши в FPF либо записали, но после этого не отключили производственный режим, разрешающий такую запись. В результате злоумышленники могут записать в FPF собственные ключи, а затем заблокировать систему, навсегда привязав её к собственному корню и цепочке доверия. Однако если производитель оборудования разработал процесс отзыва, существует возможность перезаписи предохранителя для отзыва. Точнее, FPF может быть записан ME как его области памяти, пока ME остаётся в производственном режиме; в свою очередь, ME в этом режиме доступен из ОС как для чтения, так и для записи. Таким образом, злоумышленник действительно получает ключи от королевства.

Кроме того, в большинстве исследованных устройств на основе Intel была включена отладка процессора, поэтому злоумышленникам с физическим доступом к процессору были открыты все двери. Некоторые платформы поддерживали технологию Intel BIOS Guard, но она была отключена в процессе производства для упрощения обновлений BIOS.

Таким образом, таблица 16-2 содержит несколько превосходных примеров проблем безопасности цепочки поставок, при которых поставщики, стремясь упростить поддержку оборудования, создали критические бреши в безопасности.

Уязвимости модулей SMM

Теперь рассмотрим другой вектор эксплуатации прошивки UEFI из ОС: использование ошибок в модулях SMM.

Понимание SMM

Мы обсуждали SMM и обработчики SMI в предыдущих главах, но сейчас кратко повторим обе концепции.

SMM является высокопривилегированным режимом выполнения процессоров x86. Он был разработан для реализации функций управления, специфичных для платформы, независимо от ОС. Эти функции включают расширенное управление питанием, безопасные обновления прошивки и настройку переменных UEFI Secure Boot.

Ключевая особенность архитектуры SMM состоит в предоставлении отдельной среды выполнения, невидимой для ОС. Код и данные, используемые в SMM, хранятся в защищённой оборудованием области памяти SMRAM, доступной только коду, выполняемому внутри SMM. Для входа в SMM процессор генерирует System Management Interrupt (SMI), специальное прерывание, которое должно возбуждаться программным обеспечением ОС.

Обработчики SMI являются привилегированными службами и функциями прошивки платформы. SMI служит мостом между ОС и этими обработчиками SMI. После загрузки всего необходимого кода и данных в SMRAM прошивка блокирует область памяти так, чтобы доступ к ней имел только код, выполняемый в SMM, не позволяя ОС обращаться к ней.

Эксплуатация обработчиков SMI

Из-за высокого уровня привилегий SMM обработчики SMI представляют собой чрезвычайно интересную цель для имплантатов и руткитов. Любая уязвимость этих обработчиков может предоставить злоумышленнику возможность повысить привилегии до уровня SMM, так называемого кольца -2.

Как и в других многоуровневых моделях, например при разделении ядра и пользовательского пространства, лучший способ атаковать привилегированный код состоит в воздействии на данные, которые могут быть получены извне изолированной привилегированной области памяти. Для SMM это любая память вне SMRAM. В модели безопасности SMM злоумышленником является ОС или привилегированное программное обеспечение, например инструменты обновления BIOS; поэтому любое место в ОС вне SMRAM подозрительно, поскольку в некоторые моменты им может манипулировать злоумышленник, потенциально даже после выполнения какой-либо проверки. Возможные цели включают указатели на функции, используемые кодом SMM и способные направить выполнение в области вне SMRAM, а также любые буферы с данными, которые код SMM читает или разбирает.

В наши дни разработчики прошивки UEFI стараются уменьшить поверхность атаки, сводя к минимуму число обработчиков SMI, напрямую взаимодействующих с внешним миром, то есть кольцом 0, режимом ядра операционной системы, а также находя новые способы структурирования и проверки этих взаимодействий. Однако эта работа только началась, и проблемы безопасности обработчиков SMI, вероятно, сохранятся ещё довольно долго.

Разумеется, для полезной работы код SMM может получать некоторые данные от ОС. Однако для сохранения безопасности, как и в других многоуровневых моделях, код SMM никогда не должен действовать на основе внешних данных, если они не были скопированы и проверены внутри SMRAM. Любым данным, которые проверены, но оставлены за пределами SMRAM, доверять нельзя, поскольку злоумышленник потенциально может успеть изменить их между моментом проверки и моментом использования. Более того, любые скопированные внутрь данные не должны ссылаться на непроверенные и нескопированные внешние данные.

Это звучит просто, но языки вроде C изначально не помогают отслеживать области, на которые указывают указатели, поэтому важнейшее для безопасности различие между «внутренними» адресами памяти SMRAM и «внешней», управляемой злоумышленником памятью ОС не обязательно очевидно в коде. Таким образом, программисты в основном предоставлены сами себе. Если вам интересно, насколько эту проблему можно решить средствами статического анализа, читайте дальше: как выясняется, соглашение о вызовах SMI, рассматриваемое далее, значительно усложняет задачу.

Чтобы понять, как злоумышленники могут эксплуатировать обработчики SMI, необходимо знать их соглашение о вызовах. Хотя, как показано в листинге 16-2, вызовы обработчика SMI со стороны Python в каркасе Chipsec выглядят как обычные вызовы функций, фактическое двоичное соглашение о вызовах, показанное в листинге 16-3, отличается.

```
import chipsec.chipset
import chipsec.hal.interrupts
# Номер обработчика SW SMI
SMI_NUM = 0x25
# Инициализация CHIPSEC
cs = chipsec.chipset.cs()
cs.init(None, True)
# Создать экземпляры необходимых классов
ints = chipsec.hal.interrupts.Interrupts(cs)
# Вызвать обработчик SW SMI 0x25
cs.ints.send_SW_SMI(0, SMI_NUM, 0, 0, 0, 0, 0, 0, 0)
```

Листинг 16-2. Вызов обработчика SMI из Python с помощью каркаса Chipsec

Код листинга 16-2 вызывает обработчик SMI, обнулив все параметры, кроме `0x25`, номера вызываемого обработчика. Такой вызов действительно может не передавать параметров, но также возможно, что обработчик SMI извлекает их косвенно, например через переменные ACPI или UEFI, после получения управления. Когда операционная система возбуждает SMI, например как программное прерывание через порт ввода-вывода `0xB2`, она передаёт аргументы обработчику SMI через регистры общего назначения. В листинге 16-3 можно увидеть, как фактический вызов обработчика SMI выглядит на ассемблере и как передаются параметры. Разумеется, каркас Chipsec реализует это соглашение о вызовах внутри.

```
mov rax, rdx     ; rax_value
mov ax, cx       ; smi_code_data
mov rdx, r10     ; rdx_value
mov dx, 0B2h     ; Порт управления SMI (0xB2)
mov rbx, r8      ; rbx_value
mov rcx, r9      ; rcx_value
mov rsi, r11     ; rsi_value
mov rdi, r12     ; rdi_value
; Записать значение данных SMI в порты управления/данных SW SMI (0xB2/0xB3)
out dx, ax
```

Листинг 16-3. Вызов обработчика SMI на языке ассемблера

Проблемы внешних вызовов SMI и выполнение произвольного кода

Наиболее распространённые уязвимости обработчиков SMI, представляющие интерес для имплантатов BIOS, делятся на две основные группы: проблемы внешних вызовов SMI и выполнение произвольного кода, которому во многих случаях предшествуют проблемы внешних вызовов SMI. При проблемах внешних вызовов SMI код SMM по незнанию использует управляемый злоумышленником указатель на функцию, указывающий на полезную нагрузку имплантата вне SMM. При выполнении произвольного кода код SMM принимает данные извне SMRAM, способные влиять на поток управления и использоваться для получения дополнительного контроля. Такие адреса обычно находятся ниже первого мегабайта физической памяти, поскольку обработчики SMI рассчитывают использовать этот диапазон памяти, не занятый ОС. При проблемах внешних вызовов SMI злоумышленник может перезаписать адрес косвенного перехода или указатель на функцию, вызываемую из SMM; тогда произвольный код под управлением злоумышленника будет выполнен вне SMM, но с привилегиями SMM. Хорошим примером такой атаки служит VU#631788.

В новых версиях BIOS крупных поставщиков корпоративного оборудования такие уязвимости найти сложнее, однако проблемы доступа к указателям вне диапазона SMRAM сохраняются, несмотря на появление стандартной функции `SmmIsBufferOutsideSmmValid()`, проверяющей, находится ли указатель на буфер памяти в этом диапазоне. Реализация этой универсальной проверки была добавлена в репозиторий Intel EDK2 на GitHub (https://github.com/tianocore/edk2/blob/master/MdePkg/Library/SmmMemLib/SmmMemLib.c), а её объявление показано в листинге 16-4.

```
BOOLEAN
EFIAPI
SmmIsBufferOutsideSmmValid (
  IN EFI_PHYSICAL_ADDRESS  Buffer,
  IN UINT64                Length
  )
```

Листинг 16-4. Прототип функции SmmIsBufferOutsideSmmValid() из Intel EDK2

Функция `SmmIsBufferOutsideSmmValid()` точно обнаруживает указатели на буферы памяти за пределами диапазона SMRAM, за одним исключением: аргумент `Buffer` может быть структурой, одно из полей которой является указателем на другой буфер вне SMRAM. Если проверка

безопасности выполняется только для адреса самой структуры, код SMM всё равно может оставаться уязвимым, несмотря на проверку `SmmIsBufferOutsideSmmValid()`. Поэтому обработчики SMI должны проверять каждый адрес или указатель, включая смещения, полученный от ОС, до чтения из таких областей памяти или записи в них. Важно, что сюда входит возврат кодов состояния и ошибок. Любое арифметическое вычисление внутри SMM должно проверять все параметры, поступающие извне SMM или из менее привилегированных режимов.

Практические примеры эксплуатации обработчиков SMI

Обсудив опасность получения обработчиками SMI данных от ОС, перейдём к реальному случаю их эксплуатации. Мы рассмотрим распространённый рабочий процесс обновления прошивки UEFI, используемый, среди прочих операционных систем, Windows 10. В этой ситуации прошивка проверяется и аутентифицируется внутри SMM с помощью слабых драйверов среды выполнения DXE.

На рисунке 16-5 приведено высокоуровневое представление процесса обновления BIOS в этом сценарии.

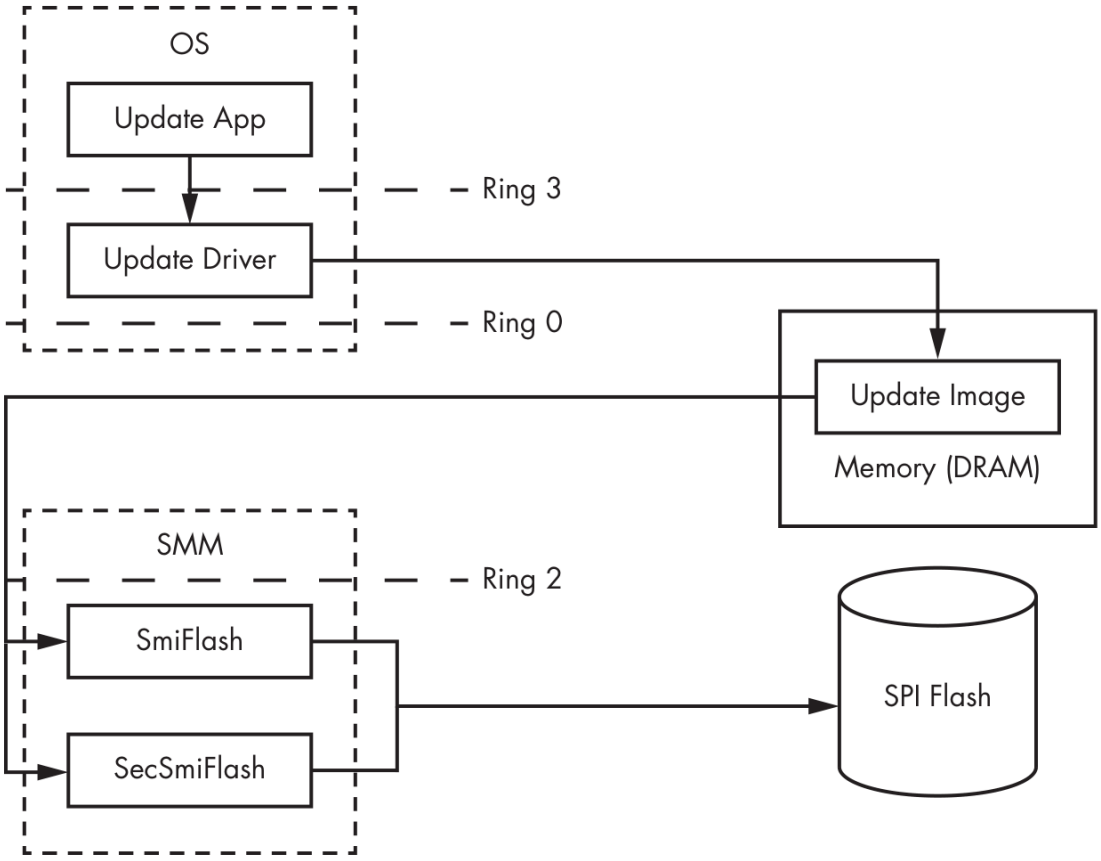


Рисунок 16-5. Высокоуровневое представление процесса обновления BIOS из ОС

Как видим, инструмент обновления BIOS пользовательского пространства (`Update App`) взаимодействует со своим драйвером режима ядра (`Update Driver`), который обычно имеет прямой доступ к устройству физической памяти через функцию API кольца 0 `MmMapIoSpace()`. Этот доступ позволяет потенциальным злоумышленникам изменять или отображать вредоносные данные в области памяти, используемые для взаимодействия с синтаксическими анализаторами обновлений обработчиков SMI BIOS (`SmiFlash` или `SecSmiFlash`). Обычно поток синтаксического анализа достаточно сложен, чтобы оставлять место для уязвимостей, особенно когда анализаторы

написаны на C, как это обычно бывает. Злоумышленник формирует вредоносный буфер данных и вызывает уязвимый обработчик SMI по его номеру, как показано в листинге 16-3, с помощью встроенных функций `__outbyte()`, доступных в компиляторе MS Visual C++.

Драйверы DXE, показанные на рисунке 16-5, `SmiFlash` и `SecSmiFlash`, встречаются во многих кодовых базах SMM. `SmiFlash` прошивает образ BIOS без какой-либо аутентификации. Используя основанный на этом драйвере инструмент обновления, злоумышленник может без лишних препятствий прошить злонамеренно изменённый образ обновления BIOS. Хорошим примером уязвимости этого вида служит VU#507496, обнаруженная Алексом Матросовым. Напротив, `SecSmiFlash` может аутентифицировать обновление, проверяя его цифровую подпись и блокируя такую атаку.

Уязвимости S3 Boot Script

В этом разделе мы дадим обзор уязвимостей S3 Boot Script, сценария, который BIOS использует для пробуждения из спящего режима. Хотя S3 Boot Script ускоряет процесс пробуждения, его неправильные реализации могут иметь серьёзные последствия для безопасности, которые мы здесь рассмотрим.

Понимание S3 Boot Script

Состояния перехода питания современного оборудования, например рабочий и спящий режимы, очень сложны и включают несколько этапов манипуляций с DRAM. Во время спящего режима, или S3, питание DRAM сохраняется, хотя процессор обесточен. Когда система пробуждается из спящего состояния, BIOS восстанавливает конфигурацию платформы, включая содержимое DRAM, а затем передаёт управление операционной системе. Хорошее обобщение этих состояний приведено на странице https://docs.microsoft.com/en-us/windows/desktop/power/system-power-states/.

Сценарий загрузки S3 хранится в DRAM, сохраняется в состоянии S3 и выполняется при возобновлении полной работы из S3. Хотя он называется «сценарием», на самом деле это последовательность кодов операций, интерпретируемых модулем прошивки Boot Script Executor (https://github.com/tianocore/edk2/blob/master/MdeModulePkg/Library/PiDxeS3BootScriptLib/BootScriptExecute.c). Boot Script Executor повторяет каждую операцию, определённую этими кодами операций, в конце этапа PEI, чтобы восстановить конфигурацию оборудования платформы и всё предзагрузочное состояние для ОС. После выполнения сценария загрузки S3 BIOS находит и выполняет вектор пробуждения ОС, чтобы вернуть выполнение её программного обеспечения в состояние, в котором оно остановилось. Это означает, что S3 Boot Script позволяет платформе пропустить этап DXE и сокращает время пробуждения из спящего состояния S3. Однако такая оптимизация сопряжена с определёнными рисками, которые мы рассмотрим далее.[2]

Атака на слабости S3 Boot Script

S3 Boot Script представляет собой ещё один вид программного кода, хранящегося в памяти. Злоумышленник, получивший доступ к нему и способный изменить код, может либо добавить скрытые действия в сам сценарий загрузки, оставаясь в рамках модели программирования S3, чтобы не вызвать тревогу, либо, если этого недостаточно, эксплуатировать интерпретатор сценария загрузки, выходя за пределы предусмотренной функциональности кодов операций.

S3 Boot Script имеет доступ к портам ввода-вывода для чтения и записи, чтению и записи конфигурации PCI, прямой доступ к физической памяти с привилегиями чтения и записи, а также к другим данным, критически важным для безопасности платформы. В частности, сценарий загрузки S3 может атаковать гипервизор, чтобы раскрыть изолированные в иных условиях области памяти. Всё это означает, что вредоносный сценарий S3 будет иметь последствия, сходные с уязвимостью выполнения кода внутри SMM, рассмотренной ранее в этой главе.

Поскольку сценарии S3 выполняются на раннем этапе процесса пробуждения до активации различных мер безопасности, злоумышленник может использовать их для обхода некоторых аппаратных настроек безопасности, которые обычно вступают в силу в процессе загрузки. Действительно, по замыслу большинство кодов операций S3 Boot Script заставляет системную прошивку восстанавливать содержимое различных регистров конфигурации оборудования. В основном этот процесс не отличается от записи в эти регистры во время работы операционной системы, за исключением того, что доступ на запись разрешён сценарию S3, но запрещён операционной системе.

Злоумышленники могут атаковать S3 Boot Script, изменяя структуру данных, называемую таблицей сценария загрузки UEFI, которая сохраняет состояние платформы во время этапа сна S3 по спецификации Advanced Configuration and Power Interface (ACPI), когда питание большинства компонентов платформы отключено.

Код UEFI создаёт таблицу сценария загрузки во время обычной загрузки и интерпретирует её записи при возобновлении из S3, когда платформа пробуждается. Злоумышленники, способные изменить содержимое текущей таблицы сценария загрузки из режима ядра ОС, а затем вызвать цикл приостановки и возобновления S3, могут добиться выполнения произвольного кода на раннем этапе пробуждения платформы, когда некоторые средства безопасности ещё не инициализированы или не заблокированы в памяти.

Обнаружение уязвимости S3 Boot Script

Первыми исследователями, публично описавшими вредоносное поведение S3 Boot Script, были Рафал Войтчук и Кори Калленберг. В декабре 2014 года в докладе «Attacks on UEFI Security, Inspired by Darth Venamis's Misery and Speed Racer» (https://bit.ly/2ucc2vU) на 31-м Chaos Communication Congress (31C3) они раскрыли связанную с S3 уязвимость CVE-2014-8274 (VU#976132). Через несколько недель исследователь безопасности Дмитрий Олексюк, также известный как Cr4sh, выпустил первый PoC-эксплойт этой уязвимости. Выпуск PoC повлёк множество открытий других исследователей. Через несколько месяцев Педро Виласа (Pedro Vilaca) обнаружил несколько связанных проблем в продуктах Apple на основе прошивки UEFI. Исследователи группы Intel Advanced Threat Research также выделили несколько потенциальных атак S3 на безопасность виртуализации в докладе «Attacking Hypervisors via Firmware and Hardware» (https://www.youtube.com/watch?v=nyW3eTobXAI), представленном на Black Hat Vegas в 2015 году. Если вы хотите больше узнать об уязвимостях S3 Boot Script, рекомендуем ознакомиться с некоторыми из этих докладов.

Эксплуатация уязвимости S3 Boot Script

Последствия эксплойта S3 Boot Script явно огромны. Но как именно работает атака? Сначала злоумышленник уже должен иметь возможность выполнять код в режиме ядра, кольце 0, операционной системы, как показано на рисунке 16-6.

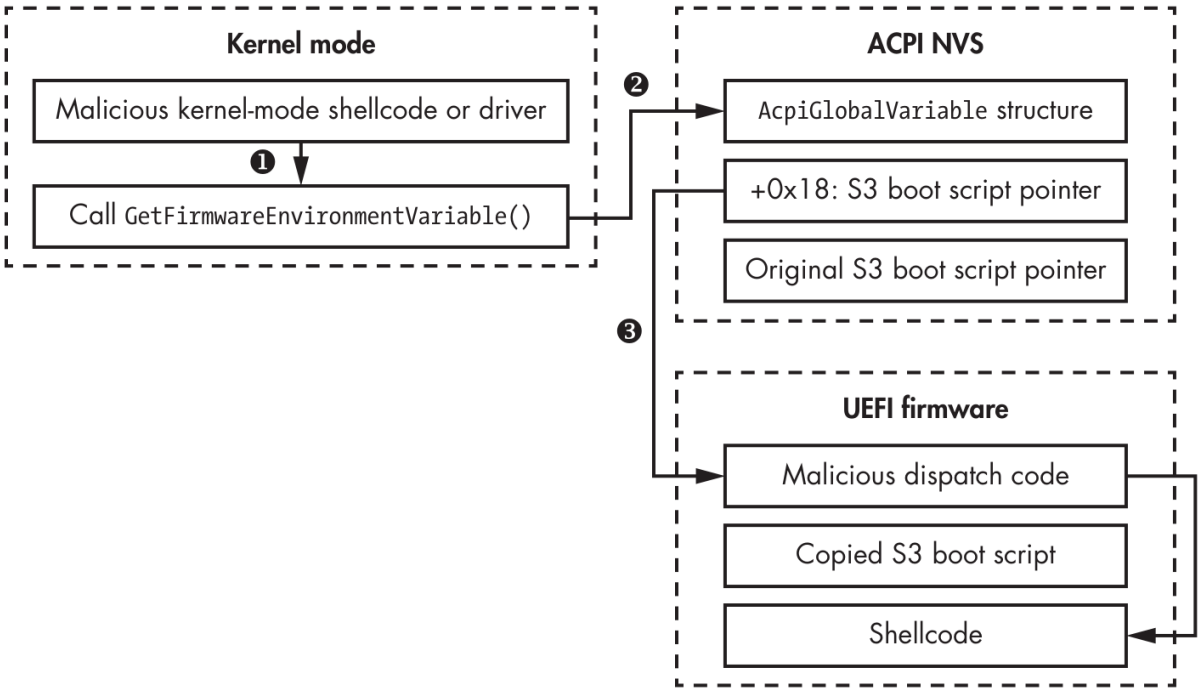


Рисунок 16-6. Пошаговая эксплуатация S3 Boot Script

Подробно рассмотрим каждый этап этого эксплойта.

1. **Первоначальная разведка.** На этапе разведки злоумышленник должен получить указатель, то есть адрес, S3 Boot Script из переменной UEFI `AcpiGlobalVariable`, указывающей на расположение сценария загрузки в незащищённой памяти DRAM. Затем необходимо скопировать исходный сценарий загрузки в область памяти, чтобы восстановить первоначальное состояние после эксплуатации. Наконец, необходимо убедиться, что система действительно подвержена уязвимости S3 Boot Script, используя код диспетчеризации модификации `EFI_BOOT_SCRIPT_DISPATCH_OPCODE`, который добавляет в указанную таблицу сценария загрузки запись для выполнения произвольного кода, как показано в листинге 16-5. Если изменение одного кода операции S3 проходит успешно, система, скорее всего, уязвима.
2. **Модификация S3 Boot Script.** Для изменения сценария загрузки злоумышленник вставляет вредоносную запись кода операции диспетчеризации в начало скопированного сценария, чтобы сделать её первой командой кода операции сценария загрузки. Затем он перезаписывает адрес сценария загрузки, устанавливая `AcpiGlobalVariable` в указатель на изменённую вредоносную версию сценария.

3. **Доставка полезной нагрузки.** Код диспетчеризации S3 Boot Script (`EFI_BOOT_SCRIPT_DISPATCH_OPCODE`) теперь должен указывать на вредоносный шелл-код. Содержимое полезной нагрузки зависит от цели злоумышленника. Она может служить нескольким целям, включая обход защиты памяти SMM или выполнение дополнительных этапов шелл-кода, отдельно отображённых в других местах памяти.
4. **Запуск уязвимости.** Вредоносный сценарий загрузки выполняется сразу после возвращения атакованной машины из спящего режима. Чтобы запустить эксплойт, пользователь или дополнительный вредоносный код внутри ОС должен активировать спящий режим S3. После начала выполнения сценарий загрузки переходит по адресу точки входа, определённому кодом диспетчеризации, где вредоносный шелл-код получает управление.

В листинге 16-5 перечислены все документированные Intel коды операций S3 Boot Script, включая выделенный `EFI_BOOT_SCRIPT_DISPATCH_OPCODE`, выполняющий вредоносный шелл-код.

```
EFI_BOOT_SCRIPT_IO_WRITE_OPCODE = 0x00
EFI_BOOT_SCRIPT_IO_READ_WRITE_OPCODE = 0x01
EFI_BOOT_SCRIPT_MEM_WRITE_OPCODE = 0x02
EFI_BOOT_SCRIPT_MEM_READ_WRITE_OPCODE = 0x03
EFI_BOOT_SCRIPT_PCI_CONFIG_WRITE_OPCODE = 0x04
EFI_BOOT_SCRIPT_PCI_CONFIG_READ_WRITE_OPCODE = 0x05
EFI_BOOT_SCRIPT_SMBUS_EXECUTE_OPCODE = 0x06
EFI_BOOT_SCRIPT_STALL_OPCODE = 0x07
EFI_BOOT_SCRIPT_DISPATCH_OPCODE = 0x08
EFI_BOOT_SCRIPT_MEM_POLL_OPCODE = 0x09
```

Листинг 16-5. Коды операций диспетчеризации S3 Boot Script

Эталонную реализацию S3 Boot Script, разработанную Intel, можно найти в репозитории EDKII на GitHub: https://github.com/tianocore/edk2/tree/master/MdeModulePkg/Library/PiDxeS3BootScriptLib/. Этот код полезен для понимания как внутреннего устройства поведения S3 Boot Script в системах x86, так и мер противодействия, реализованных для предотвращения только что рассмотренной уязвимости.

Чтобы проверить, подвержена ли система уязвимости S3 Boot Script, можно воспользоваться инструментом S3 Boot Script из Chipsec (`chipsec/modules/common/uefi/s3bootscript.py`). Однако этот инструмент нельзя использовать для эксплуатации уязвимости. Для доставки полезной нагрузки можно применить опубликованный на GitHub PoC эксплойта Дмитрия Олексюка (https://github.com/Cr4sh/UEFI_boot_script_expl/). В листинге 16-6 показан успешный результат эксплуатации этим PoC.

```
[x][ ================================================================================
[x][ Module: UEFI boot script table vulnerability exploit
[x][ ================================================================================
[*] AcpiGlobalVariable = 0x79078000
[*] UEFI boot script addr = 0x79078013
[*] Target function addr = 0x790780b6
8 bytes to patch
Found 79 zero bytes at 0x0x790780b3
Jump from 0x79078ffb to 0x79078074
Jump from 0x790780b6 to 0x790780b3
Going to S3 sleep for 10 seconds ...
rtcwake: wakeup from "mem" using /dev/rtc0 at Mon Jun 6 09:03:04 2018
[*] BIOS_CNTL = 0x28
[*] TSEGMB = 0xd7000000
[!] Bios lock enable bit is not set
[!] SMRAM is not locked
[!] Your system is VULNERABLE
```

Листинг 16-6. Результат успешной эксплуатации S3 Boot Script

Эта уязвимость и её эксплойт также полезны для отключения некоторых битов защиты BIOS, например BIOS Lock Enabled, BIOS Write Protection и некоторых других, настраиваемых в регистре FLOCKDN (Flash Lock-Down). Важно, что эксплойт S3 также может отключить защищённые диапазоны регистров PRx, изменив их конфигурацию. Кроме того, как упоминалось ранее, уязвимость S3 можно использовать для обхода технологий изоляции памяти виртуализации, например Intel VT-x. Фактически следующие коды операций S3 могут напрямую обращаться к памяти во время восстановления из спящего состояния:

```
EFI_BOOT_SCRIPT_IO_WRITE_OPCODE = 0x00
EFI_BOOT_SCRIPT_IO_READ_WRITE_OPCODE = 0x01
```

Эти коды операций могут от имени прошивки UEFI записать некоторое значение в указанную область памяти, что делает возможной атаку на гостевую виртуальную машину. Даже если архитектура включает гипервизор, более привилегированный, чем основная система, основная система может атаковать его через S3, а через него и всех гостей.

Исправление уязвимости S3 Boot Script

Уязвимость S3 Boot Script была одной из самых значительных уязвимостей безопасности прошивки UEFI. Её было легко эксплуатировать и трудно устранить, поскольку настоящее исправление требовало многочисленных архитектурных изменений прошивки.

Для устранения проблемы S3 Boot Script требовалась защита целостности от модификаций из кольца 0. Одним из способов добиться этого было перемещение S3 Boot Script в SMRAM, диапазон памяти SMM. Однако существует и другой способ: в методе, представленном в EDKII (`edk2/MdeModulePkg/Library/SmmLockBoxLib`), архитекторы Intel разработали механизм LockBox для защиты S3 Boot Script от любых модификаций извне SMM.[3]

Уязвимости Intel Management Engine

Intel Management Engine представляет интерес для злоумышленника. Эта технология привлекала исследователей безопасности оборудования с момента появления, поскольку почти не документирована и чрезвычайно мощна. Сегодня ME использует отдельный процессор на основе x86, а в прошлом использовала специализированный процессор ARC, и служит основой аппаратного корня доверия Intel и нескольких технологий безопасности, например Intel Boot Guard, Intel BIOS Guard и частично Intel Software Guard Extension (SGX). Таким образом, компрометация ME предоставляет способ обойти Secure Boot.

Получение контроля над ME является крайне желанной целью злоумышленников, поскольку ME обладает всей мощью SMM, но также может выполнять встроенную операционную систему реального времени на отдельном 32-разрядном микроконтроллере, работающем совершенно независимо от основного процессора. Рассмотрим некоторые его уязвимости.

История уязвимостей ME

В 2009 году исследователи безопасности Александр Терёшкин и Рафал Войтчук из Invisible Things Lab представили исследование злоупотребления ME в докладе «Introducing Ring -3 Rootkits» на конференции Black Hat USA в Лас-Вегасе.[4] Они поделились открытиями о внутреннем устройстве Intel ME и обсудили способы внедрения кода в контекст выполнения Intel AMT, например путём превращения ME в руткит.

Следующий шаг в понимании уязвимостей ME был сделан лишь через восемь лет. Исследователи Максим Горячий и Марк Ермолов из Positive Technologies обнаружили уязвимости выполнения кода в новой версии ME, присутствующей в шестом, седьмом и восьмом поколениях процессоров Intel. Эти уязвимости, соответственно CVE-2017-5705, CVE-2017-5706 и CVE-2017-5707, позволяли злоумышленнику выполнять произвольный код внутри контекста операционной системы ME, приводя к полной компрометации соответствующих платформ на самом высоком уровне привилегий. Горячий

и Ермолов представили эти открытия в докладе «How to Hack a Turned-Off Computer, or Running Unsigned Code in Intel Management Engine» на Black Hat Europe 2017,[5] где показали, как код руткита может обходить или отключать несколько средств безопасности, включая технологии Intel Boot Guard и BIOS Guard, посредством компрометации их корня доверия. Вопрос о том, существуют ли технологии безопасности, устойчивые к скомпрометированной ME, остаётся открытым для исследований. Помимо других возможностей, код руткита, выполняющийся в контексте Intel ME, позволяет злоумышленнику изменять образ BIOS и частично корень доверия Boot Guard непосредственно внутри микросхемы SPI flash, обходя тем самым большинство средств безопасности.

Атаки на код ME

Хотя код ME выполняется на собственной микросхеме, он взаимодействует с другими уровнями ОС и может быть атакован через эти каналы связи. Как всегда, граница взаимодействия является частью поверхности атаки любой вычислительной среды, независимо от степени её изоляции.

Intel создала специальный интерфейс Host-Embedded Controller Interface (HECI), чтобы приложения ME могли взаимодействовать с ядром операционной системы. Этот интерфейс можно использовать, например, для удалённого управления системой через сетевое соединение, завершающееся в ME, но способное захватывать графический интерфейс операционной системы, например через VNC, или для настройки платформы с помощью операционной системы во время производства. Его также можно использовать для реализации корпоративных служб управления Intel vPro, включая AMT, рассматриваемую в следующем разделе.

Обычно прошивка UEFI инициализирует HECI через прокси-драйвер SMM `HeciInitDxe`, расположенный внутри BIOS. Этот драйвер SMM передаёт сообщения между ME и специфичным для поставщика драйвером основной ОС через мост PCH, соединяющий процессор и микросхему ME.

Приложения, работающие внутри ME, могут регистрировать обработчики HECI для приёма сообщений от основной операционной системы; ME не должна доверять никаким входным данным от ОС. Если ядро ОС захвачено злоумышленником, эти интерфейсы становятся частью поверхности атаки ME. Например, чрезмерно доверчивый синтаксический анализатор внутри приложения ME, не выполняющий полную проверку сообщений со стороны ОС, может быть скомпрометирован специально сформированным сообщением, как и слабый сетевой сервер. Поэтому важно уменьшать поверхность атаки приложений ME, сводя к минимуму число обработчиков HECI. В частности, платформы Apple навсегда отключают интерфейсы HECI и

минимизируют число приложений ME как осознанное решение политики безопасности. Однако компрометация одного приложения ME не означает компрометации всей ME.

Практические примеры: атаки на Intel AMT и BMC

Теперь рассмотрим уязвимости двух технологий, использующих ME. Для управления крупными центрами обработки данных, а также огромными парками корпоративных рабочих станций,

которыми необходимо централизованно управлять, организации часто используют технологии, встраивающие конечную точку управления и его логику в основную плату платформы. Это позволяет удалённо управлять платформой, даже когда её основной процессор не работает. Эти технологии, в число которых входят Intel AMT и различные микросхемы baseboard management controller (BMC), неизбежно стали частью поверхности атаки своих платформ.

Полное обсуждение атак на AMT и BMC выходит за рамки этой главы. Однако мы всё же хотим дать несколько ориентиров, поскольку эксплуатация этих технологий напрямую связана с уязвимостями UEFI и в последнее время привлекает большое внимание из-за уязвимостей Intel AMT и BMC с серьёзными последствиями, раскрытых в 2017 и 2018 годах. Далее мы обсудим эти уязвимости.

Уязвимости AMT

Платформа Intel AMT реализована как приложение ME и потому непосредственно связана со средой выполнения Intel ME. AMT использует способность ME взаимодействовать с платформой по сети, даже когда основной процессор неактивен или полностью выключен. Она также использует ME для чтения и записи DRAM во время выполнения независимо от основного процессора. AMT является типичным примером приложения прошивки ME, предназначенного для обновления через механизм обновления BIOS. Для этого Intel AMT запускает собственный веб-сервер, служащий основной точкой входа для корпоративной консоли удалённого управления.

В 2017 году, после почти двух десятилетий без публично известных проблем безопасности, была раскрыта первая уязвимость AMT. Она оказалась ошеломляющей и, учитывая её природу, едва ли будет последней! Исследователи Embedi, частной компании в области безопасности, сообщили Intel о критической проблеме CVE-2017-5689 (INTEL-SA-00075), позволявшей удалённый доступ и обход аутентификации. Подвержены все системы Intel, выпущенные с 2008 года и поддерживающие ME. Сюда не входит значительное семейство Intel Atom, которое само по себе не включало ME, хотя все серверные продукты и рабочие станции, вероятно, были уязвимы, если содержали уязвимые компоненты ME. Официально AMT имеется только в системах Intel vPro. Масштаб этой уязвимости особенно интересен, поскольку главным образом она затрагивала системы, спроектированные для доступа через удалённую консоль управления AMT даже в выключенном состоянии, а значит, систему можно было атаковать и тогда, когда она выключена.

Обычно AMT продавалась как часть технологии Intel vPro, но в том же докладе исследователи Embedi продемонстрировали возможность включения AMT в системах без vPro. Они выпустили инструмент AMTactivator, который администратор операционной системы мог запустить для активации AMT, даже если официально она не являлась частью платформы. Исследователи показали, что AMT входила во все современные процессоры Intel с ME независимо от того, продавались ли они как поддерживающие vPro; в последнем случае AMT всё равно присутствовала и могла быть активирована во благо или во вред. Дополнительные сведения об уязвимости приведены на странице https://www.blackhat.com/docs/us-17/thursday/us-17-Evdokimov-Intel-AMT-Stealth-Breakthrough-wp.pdf.

Intel намеренно раскрыла очень мало сведений об AMT, создав значительные трудности для всех находящихся за пределами Intel исследователей недостатков безопасности этой технологии. Однако опытные злоумышленники приняли вызов и добились значительных успехов в анализе скрытых возможностей AMT. Защитников могут ожидать новые неприятные сюрпризы.

Руткит PLATINUM APT

С прошивкой Intel AMT непосредственно не связан, но также интересен тот факт, что субъект так называемой PLATINUM APT использовал канал Serial-over-LAN (SOL) AMT для сетевого обмена. Этот руткит был обнаружен исследовательской группой Microsoft Windows Defender летом 2017 года. Обмен через AMT SOL работал независимо от операционной системы и потому был невидим для межсетевого экрана уровня ОС и приложений сетевого мониторинга, выполнявшихся на основном устройстве. До этого происшествия не было известно ни одной вредоносной программы, злоупотреблявшей функцией AMT SOL как скрытым каналом связи. Дополнительные сведения см. в исходной статье и записи блога Microsoft: https://cloudblogs.microsoft.com/microsoftsecure/2017/06/07/platinum-continues-to-evolve-find-ways-to-maintain-invisibility/.
Существование этого канала было обнаружено исследователями LegbaCore, раскрывшими его до обнаружения в реальных условиях: http://legbacore.com/Research_files/HowManyMillionBIOSWouldYouLikeToInfect_Full.pdf.

Уязвимости микросхем BMC

В то же время, когда Intel разрабатывала предложения vPro на основе сред выполнения ME платформы AMT, другие поставщики занимались разработкой конкурирующих централизованных решений удалённого управления серверами: микросхем BMC, интегрированных в серверы. Как продукты параллельной эволюции, архитектуры BMC имеют множество тех же слабостей, что и AMT.

Реализации BMC, часто встречающиеся в серверном оборудовании, повсеместно распространены в центрах обработки данных. Крупные поставщики оборудования, например Intel, Dell и HP, имеют собственные реализации BMC, основанные главным образом на микроконтроллерах ARM с интегрированными сетевыми интерфейсами и flash-хранилищем. Это выделенное flash-хранилище содержит операционную систему реального времени (RTOS), обеспечивающую работу нескольких приложений, например веб-сервера, прослушивающего сетевой интерфейс микросхемы BMC, отдельный сетевой интерфейс управления.

Если вы читали внимательно, всё это должно кричать: «Поверхность атаки!» Действительно, встроенный веб-сервер BMC обычно написан на C, включая CGI, и поэтому является первоочередной целью для злоумышленников, ищущих уязвимости обработки входных данных. Хорошим примером служит уязвимость CVE-2017-12542 в HP iLO BMC, позволявшая обойти аутентификацию и удалённо выполнить код в веб-сервере соответствующей BMC. Эту проблему безопасности обнаружили исследователи Airbus Фабьен Периго (Fabien Périgaud), Александр Газе (Alexandre Gazet) и Жоффрей Чарни (Joffrey Czarny). Мы настоятельно рекомендуем их подробный аналитический отчёт «Subverting Your Server Through Its BMC: The HPE iLO4 Case»: https://bit.ly/2HxeCUS.

Уязвимости BMC подчёркивают тот факт, что независимо от применяемых методов аппаратного разделения общей мерой поверхности атаки платформы является граница её взаимодействия. Чем больше функциональности открыто на этой границе, тем выше риск для общей безопасности платформы. Платформа может иметь отдельный процессор с выполняющейся на нём отдельной прошивкой, но если эта прошивка включает богатую цель, например веб-сервер, злоумышленник

может использовать слабости платформы для установки имплантата. Например, процесс обновления прошивки на основе BMC, который не аутентифицирует образы обновлений, поступающие по сети, столь же уязвим, как любая схема установки программного обеспечения, полагающаяся на безопасность через неясность.

Заключение

Надёжность прошивки UEFI и других системных прошивок платформ на основе x86 сегодня является актуальной темой, достойной отдельной книги. В некотором смысле UEFI должна была заново изобрести BIOS, но сделала это со всеми недостатками подходов устаревшей BIOS, основанных на безопасности через неясность, и добавила ещё множество новых.

Нам пришлось принять трудные решения о том, какие уязвимости включить сюда, а какие рассмотреть подробнее, чтобы показать более масштабные архитектурные изъяны. В итоге мы надеемся, что эта глава охватила ровно столько исходных сведений, сколько нужно для более глубокого понимания современного состояния безопасности прошивки UEFI через призму распространённых архитектурных недостатков, а не просто развлекла вас бессвязной смесью печально известных уязвимостей.

В наши дни прошивка UEFI является краеугольным камнем безопасности платформы, хотя ещё несколько лет назад поставщики повсеместно пренебрегали ею. Совместные усилия исследовательского сообщества в области безопасности сделали эти изменения возможными, и мы надеемся, что наша книга отдаёт им должное и помогает дальнейшему прогрессу.

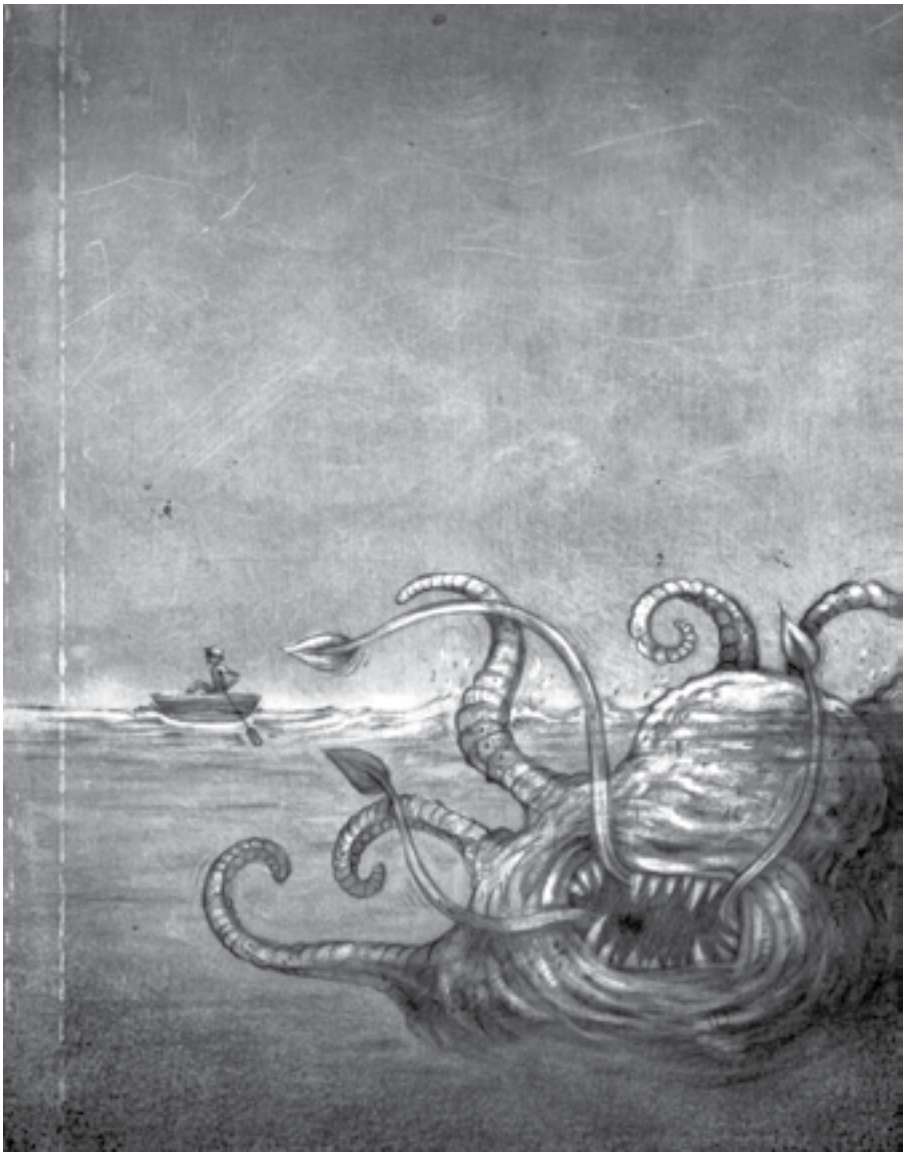
Эта страница в исходном издании не содержит основного текста.

Сноски

[1] [назад] ESET Research, «LOJAX: First UEFI Rootkit Found in the Wild, Courtesy of the Sednit Group» (аналитический отчёт), 27 сентября 2018 года, https://www.welivesecurity.com/wp-content/uploads/2018/09/ESET-LoJax.pdf.
[2] [назад] Подробное техническое объяснение реализации возобновления из S3 в рабочее состояние приведено в работе Дживэня Яо (Jiewen Yao) и Винсента Дж. Циммера (Vincent J. Zimmer) «A Tour Beyond BIOS Implementing S3 Resume with EDKII» (аналитический отчёт Intel), октябрь 2014 года: https://firmware.intel.com/sites/default/files/A_Tour_Beyond_BIOS_Implementing_S3_resume_with_EDKII.pdf.
[3] [назад] Дополнительные сведения приведены в упомянутой ранее работе «A Tour Beyond BIOS: Implementing S3 Resume with EDKII»: https://firmware.intel.com/sites/default/files/A_Tour_Beyond_BIOS_Implementing_S3_resume_with_EDKII.pdf.
[4] [назад] https://invisiblethingslab.com/resources/bh09usa/Ring%20-3%20Rootkits.pdf
[5] [назад] https://www.blackhat.com/docs/eu-17/materials/eu-17-Goryachy-How-To-Hack-A-Turned-Off-Computer-Or-Running-Unsigned-Code-In-Intel-Management-Engine.pdf

Часть III. Защита и криминалистические методы

Глава 17. Как работает UEFI Secure Boot



В предыдущих главах мы говорили о введении политики подписывания кода режима ядра, которая побудила разработчиков вредоносного ПО перейти от руткитов к буткитам, переместив вектор атаки из ядра ОС в незащищённые загрузочные компоненты. Вредоносное ПО такого рода выполняется до загрузки ОС, поэтому способно обходить или отключать механизмы безопасности ОС. Следовательно, для применения средств безопасности и обеспечения защищённости ОС должна иметь возможность загружаться в доверенную среду, компоненты которой не подвергались вмешательству.

Именно здесь в дело вступает технология UEFI Secure Boot, которой посвящена эта глава. Нацеленная главным образом на защиту загрузочных компонентов платформы от модификации и обеспечение загрузки и выполнения при запуске только доверенных модулей, UEFI Secure Boot может стать эффективным решением против угроз буткитов, если охватывает все направления атаки.

Однако средства защиты UEFI Secure Boot уязвимы для руткитов прошивки, новейшей и наиболее быстро развивающейся технологии вредоносного ПО. Поэтому необходим ещё один уровень безопасности, охватывающий весь процесс загрузки с самого начала. Этого можно добиться с помощью реализации Secure Boot под названием Verified and Measured Boot.

Эта глава познакомит вас с ядром данной технологии безопасности: сначала будет описано, как она способна защищать от руткитов прошивки при закреплении в оборудовании, а затем будут рассмотрены подробности её реализации и способы защиты жертв от буткитов.

Однако, как часто бывает в индустрии безопасности, очень немногие решения способны обеспечить окончательную защиту от атак: злоумышленники и защитники заперты в вечной гонке вооружений. В завершение главы мы обсудим недостатки UEFI Secure Boot, способы её обхода и защиту с помощью двух вариантов Verified and Measured Boot от Intel и ARM.

Что такое Secure Boot?

Основная цель Secure Boot состоит в предотвращении выполнения кем-либо неавторизованного кода в предзагрузочной среде; выполнять разрешается только код, соответствующий политике целостности платформы. Эта технология очень важна для платформ с высокими требованиями к надёжности, а также часто применяется во встроенных устройствах и мобильных платформах, поскольку позволяет поставщикам ограничивать платформы одобренным ими программным обеспечением, например iOS на iPhone или операционной системой Windows 10 S.

Secure Boot существует в трёх формах, зависящих от уровня иерархии процесса загрузки, на котором она применяется:

Secure Boot ОС. Реализуется на уровне загрузчика ОС. Проверяет компоненты, загружаемые загрузчиком ОС, например ядро ОС и драйверы, запускаемые при загрузке.

UEFI Secure Boot. Реализуется в прошивке UEFI. Проверяет драйверы и приложения UEFI DXE, Option ROM и загрузчики ОС.

Secure Boot платформы (Verified and Measured Secure Boot). Закрепляется в оборудовании. Проверяет прошивку инициализации платформы.

Secure Boot ОС мы рассматривали в главе 6, поэтому в этой главе сосредоточимся на UEFI Secure Boot и Verified and Measured Boot.

Подробности реализации UEFI Secure Boot

Начнём обсуждение с принципа работы UEFI Secure Boot. Прежде всего важно отметить, что UEFI Secure Boot является частью спецификации UEFI, которую можно найти по адресу http://www.uefi.org/sites/default/files/resources/UEFI_Spec_2_7.pdf. Мы будем ссылаться на спецификацию, то есть на описание того, как должна работать UEFI Secure Boot, хотя подробности реализации у разных производителей платформ могут различаться.

> **Примечание.** Далее в этом разделе под Secure Boot, если не указано иное, подразумевается UEFI Secure Boot.

Сначала мы рассмотрим последовательность загрузки, чтобы увидеть, где в неё вступает Secure Boot. Затем разберём, как Secure Boot аутентифицирует исполняемые файлы, и обсудим участвующие базы данных.

Последовательность загрузки

Кратко повторим последовательность загрузки UEFI, описанную в главе 14, чтобы увидеть, где в процесс входит Secure Boot. Если вы пропустили ту главу, сейчас стоит к ней вернуться.

Если обратиться к разделу «Как работает прошивка UEFI» на странице 242, можно увидеть, что первым кодом, выполняемым при выходе системы из сброса, является прошивка инициализации платформы (platform initialization, PI), выполняющая базовую инициализацию оборудования платформы. Во время выполнения PI набор микросхем и контроллер памяти всё ещё находятся в неинициализированном состоянии: DRAM ещё недоступна прошивке, а периферийные устройства на шине PCIe ещё не перечислены. Шина PCIe является стандартом высокоскоростной последовательной шины, используемым практически во всех современных ПК; подробнее мы обсудим её в последующих главах. В этот момент Secure Boot ещё не активна, поэтому часть PI системной прошивки пока не защищена.

После того как прошивка PI обнаруживает и настраивает RAM и выполняет базовую инициализацию платформы, она переходит к загрузке драйверов DXE и приложений UEFI, которые продолжают инициализацию оборудования платформы. Именно тогда вступает в действие Secure Boot. Закреплённая в прошивке PI, Secure Boot используется для аутентификации модулей UEFI, загружаемых из flash-памяти SPI (Serial Peripheral Interface) или Option ROM периферийных устройств.

Механизм аутентификации Secure Boot, по существу, является процессом проверки цифровой подписи. Выполняться разрешено только правильно аутентифицированным образам. Для управления ключами проверки подписей Secure Boot опирается на инфраструктуру открытых ключей (public key infrastructure, PKI).

В упрощённом виде реализация Secure Boot содержит открытый ключ, используемый для проверки цифровой подписи исполняемых образов, загружаемых при запуске. Образы должны содержать встроенную цифровую подпись, хотя, как будет показано далее в этой главе, у этого правила есть исключения. Если образ проходит проверку, он загружается и в конечном итоге выполняется. Если образ не имеет подписи и проверка завершается неудачей, запускается поведение исправления: действия, выполняемые при отказе Secure Boot. В зависимости от политики система может продолжить обычную загрузку либо прервать процесс загрузки и показать пользователю сообщение об ошибке.

Фактические реализации Secure Boot несколько сложнее описанной здесь. Для надлежащего установления доверия к коду, выполняемому во время загрузки, Secure Boot использует разные виды баз данных подписей, ключей и политик. Рассмотрим эти факторы по очереди и углубимся в подробности.

> **Практические реализации: компромиссы**
>
> В реальных реализациях прошивки UEFI производители платформ часто идут на компромисс между безопасностью и производительностью. Проверка цифровой подписи каждого образа UEFI, запрашивающего выполнение, занимает время. На средней современной платформе могут пытаться загрузиться несколько сотен образов UEFI, поэтому проверка цифровой подписи каждого исполняемого файла продлила бы процесс загрузки. В то же время производители вынуждены сокращать время загрузки, особенно во встроенных системах и автомобильной промышленности. Вместо проверки каждого образа UEFI поставщики прошивки часто предпочитают проверять образы UEFI по хешам для повышения производительности. Набор хешей разрешённых образов находится в хранилище, целостность и подлинность которого обеспечиваются цифровой подписью только один раз при обращении к хранилищу. Подробнее эти хеши будут рассмотрены далее в этой главе.

Аутентификация исполняемых файлов с помощью цифровых подписей

В качестве первого шага к пониманию Secure Boot рассмотрим, как фактически подписываются исполняемые файлы UEFI, то есть где цифровая подпись находится в исполняемом файле и какие виды подписей поддерживает Secure Boot.

В исполняемых файлах UEFI, являющихся образами Portable Executable (PE), цифровые подписи содержатся в специальных структурах данных, называемых сертификатами подписи. Расположение этих сертификатов в двоичном файле определяется специальным полем структуры данных заголовка PE, называемым Certificate Table Data Directory и показанным на рисунке 17-1. Стоит отметить, что один файл может иметь несколько цифровых подписей, созданных с помощью разных ключей подписывания для разных целей. По этому полю прошивка UEFI может найти сведения о подписи, используемые для аутентификации исполняемого файла.

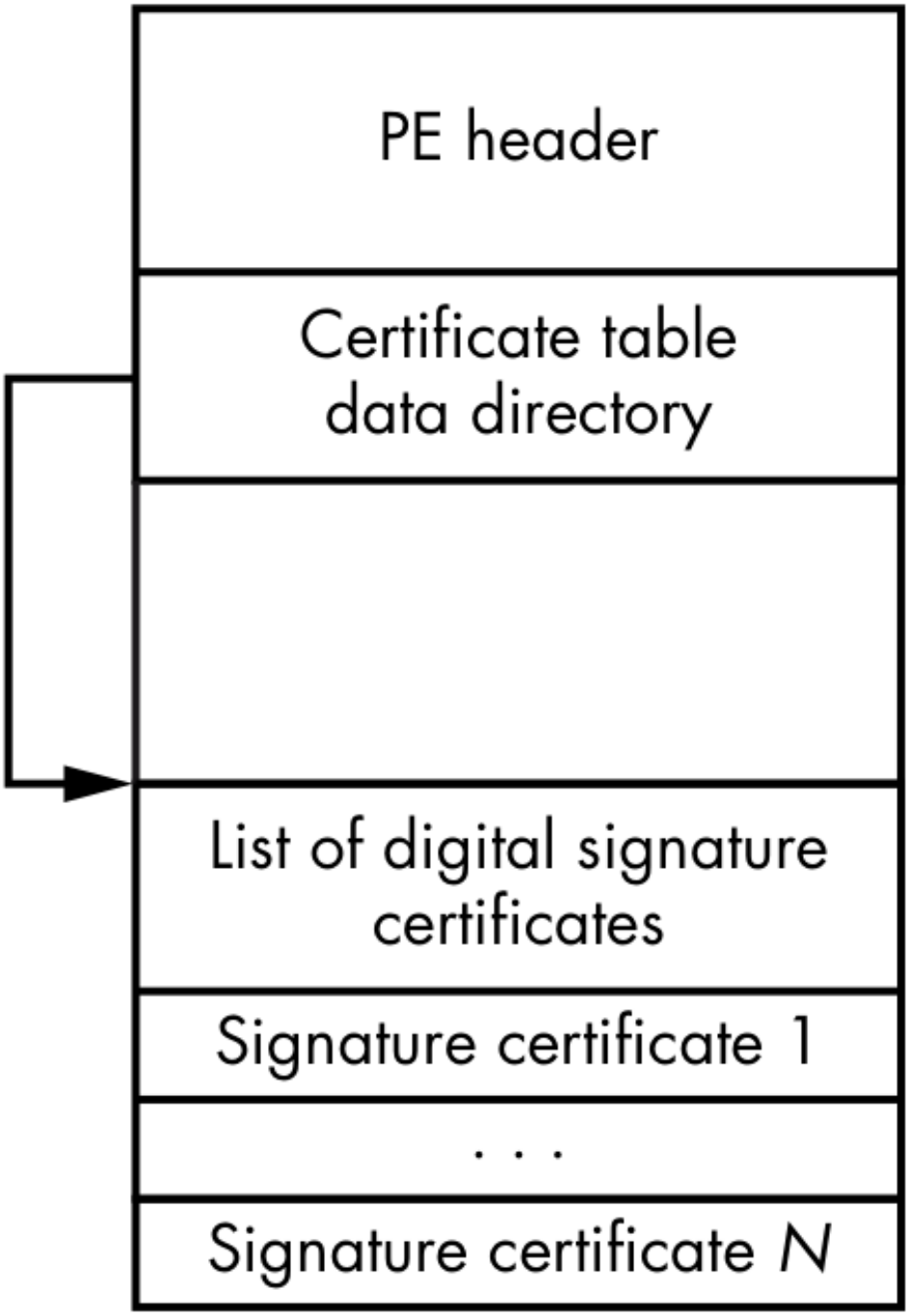


Рисунок 17-1. Расположение цифровых подписей в образах UEFI

Другие виды исполняемых образов UEFI, например образы Terse Executable (TE), не имеют встроенных цифровых подписей из-за особенностей своего исполняемого формата. Формат

образов TE был создан на основе PE/COFF в попытке уменьшить размер TE, чтобы он занимал меньше места. Поэтому образы TE содержат только поля формата PE, необходимые для выполнения образа в среде PI, и не имеют таких полей, как Certificate Table Data Directory. В результате прошивка UEFI не может напрямую аутентифицировать такие образы путём проверки их цифровой подписи. Однако Secure Boot предоставляет возможности аутентификации этих образов с помощью криптографических хешей; этот механизм подробнее описан в следующем разделе.

Структура встроенного сертификата подписи зависит от его вида. Мы не будем здесь углубляться в детали структуры, но дополнительные сведения можно найти в разделе «Расположение подписей драйверов» на странице 73.

Каждый вид сертификата подписи, используемого в Secure Boot, содержит как минимум следующие сведения: информацию о криптографических алгоритмах, применяемых для создания и проверки подписи, например криптографические хеш-функции и идентификаторы алгоритмов цифровой подписи; криптографический хеш рассматриваемого исполняемого файла; саму цифровую подпись; открытый ключ для проверки цифровой подписи.

Этих сведений достаточно, чтобы Secure Boot проверила подлинность исполняемого образа. Для этого прошивка UEFI находит и считывает сертификат подписи из исполняемого файла, вычисляет хеш исполняемого файла по заданному алгоритму, а затем сравнивает его с хешем, указанным в сертификате подписи. Если они совпадают, прошивка UEFI проверяет цифровую подпись хеша с помощью ключа, указанного в сертификате подписи. Если проверка подписи проходит успешно, прошивка UEFI принимает подпись. В любом другом случае, например при несовпадении хеша или неудаче проверки подписи, прошивка UEFI не аутентифицирует образ.

Однако одной проверки совпадения подписи недостаточно для установления доверия к исполняемому файлу UEFI. Прошивка UEFI также должна убедиться, что исполняемый файл подписан авторизованным ключом. В противном случае ничто не мешает любому человеку создать собственный ключ подписывания и подписать им вредоносный образ, чтобы пройти проверку Secure Boot.

Именно поэтому открытый ключ, используемый для проверки подписи, должен соответствовать доверенному закрытому ключу. Прошивка UEFI явно доверяет этим закрытым ключам, поэтому их можно использовать для установления доверия к образу. Список доверенных открытых ключей хранится в базе данных db, которую мы рассмотрим далее.

База данных db

База данных db содержит список сертификатов доверенных открытых ключей, авторизованных для аутентификации подписей. Всякий раз, когда Secure Boot проверяет подпись исполняемого файла, она сверяет открытый ключ подписи со списком ключей в базе данных db, чтобы определить, можно ли ему доверять. Во время загрузки на платформе будет выполняться только код, подписанный закрытыми ключами, соответствующими этим сертификатам.

Помимо списка сертификатов доверенных открытых ключей база данных db содержит хеши отдельных исполняемых файлов, которым разрешено выполняться на платформе независимо от наличия цифровой подписи. Этот механизм можно использовать для аутентификации файлов TE, не имеющих встроенных цифровых подписей.

Согласно спецификации UEFI, база данных подписей хранится в переменной энергонезависимой RAM (NVRAM), сохраняющейся после перезагрузок системы. Реализация переменных NVRAM зависит от платформы, и разные производители оригинального оборудования (OEM) могут реализовать её по-разному. Чаще всего эти переменные хранятся в той же SPI flash, которая

содержит прошивку платформы, например BIOS. Как будет показано в разделе «Модификация переменных UEFI для обхода проверок безопасности» на странице 337, это приводит к уязвимостям, которые можно использовать для обхода Secure Boot.

Рассмотрим содержимое базы данных db в вашей системе, выгрузив содержимое хранящей её переменной NVRAM. В качестве примера мы используем платформу Lenovo ThinkPad T540p, но вам следует использовать платформу, с которой вы работаете. Содержимое переменной NVRAM будет выгружено с помощью набора инструментов Chipsec с открытым исходным кодом, с которым вы познакомились в главе 15. Этот набор обладает богатой функциональностью, полезной для криминалистического анализа, и подробнее будет рассмотрен в главе 19.

Загрузите Chipsec с GitHub по адресу https://github.com/chipsec/chipsec/. Инструмент зависит от `winpy` (Python for Windows Extensions), который необходимо загрузить и установить до запуска Chipsec. После установки обоих компонентов откройте командную строку или другой интерпретатор командной строки и перейдите в каталог с загруженным Chipsec. Затем введите следующую команду, чтобы получить список переменных UEFI:

```
$ chipsec_util.py uefi var-list
```

Эта команда выгружает все переменные UEFI из текущего каталога в подкаталог `efi_variables.dir` и декодирует содержимое некоторых из них; Chipsec декодирует только содержимое известных переменных. Перейдите в этот каталог, где вы должны увидеть нечто похожее на рисунок 17-2.

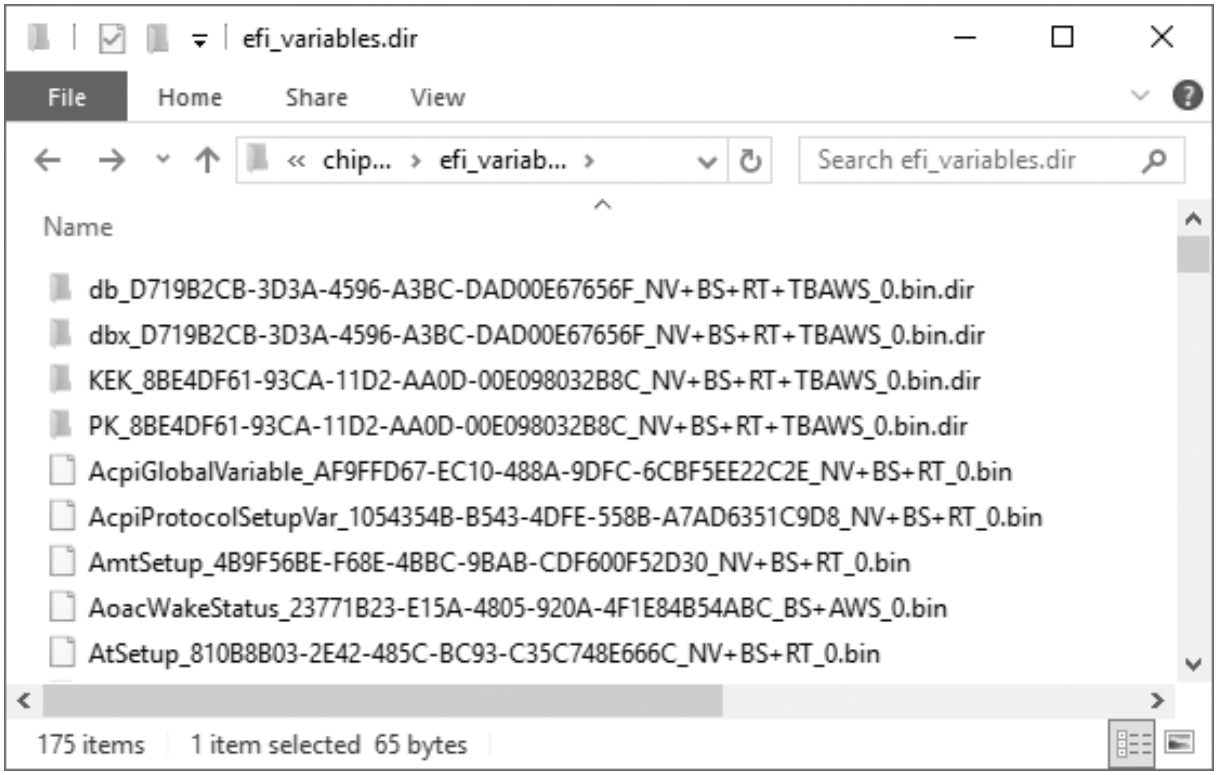


Рисунок 17-2. Переменные UEFI, выгруженные Chipsec

Каждая запись в этом каталоге соответствует отдельной переменной UEFI NVRAM. Имена переменных имеют структуру `VarName_VarGUID_VarAttributes.bin`, где `VarName` - имя переменной, `VarGUID` - 16-байтовый глобальный уникальный идентификатор (GUID) переменной, а `VarAttributes` - список атрибутов переменной в краткой форме. На основании спецификации UEFI ниже описаны некоторые атрибуты записей на рисунке 17-2.

NV. Энергонезависимая переменная, содержимое которой сохраняется после перезагрузки.

BS. Может быть доступна службам загрузки UEFI. Службы загрузки UEFI обычно доступны во время загрузки до выполнения загрузчика ОС. После запуска загрузчика ОС службы загрузки UEFI больше недоступны.

RT. Может быть доступна службам времени выполнения UEFI. В отличие от служб загрузки UEFI, службы времени выполнения сохраняются в течение загрузки ОС и во время её работы.

AWS. Аутентифицированная переменная на основе счётчика: любое новое содержимое переменной должно быть подписано авторизованным ключом, чтобы его можно было записать в переменную. Подписанные данные переменной включают счётчик для защиты от атак отката.

TBAWS. Аутентифицированная переменная на основе времени: любое новое содержимое переменной должно быть подписано авторизованным ключом, чтобы его можно было записать в переменную. Временная метка подписи отражает время подписывания данных. Она используется для подтверждения того, что подпись была создана до истечения срока действия соответствующего ключа подписывания. Дополнительные сведения об аутентификации на основе времени приведены в следующем разделе.

Если Secure Boot настроена и переменная db существует на платформе, в этом каталоге должен находиться подкаталог с именем, начинающимся с db_D719B2CB-3D3A-4596-A3BC-DAD00E67656F. Когда Chipsec выгружает переменную UEFI db, он автоматически декодирует содержимое переменной в этот подкаталог, который содержит файлы, соответствующие сертификатам открытых ключей и хешам образов UEFI, разрешённых к выполнению. В нашем случае имеется пять файлов: четыре сертификата и один хеш SHA256, как показано на рисунке 17-3.

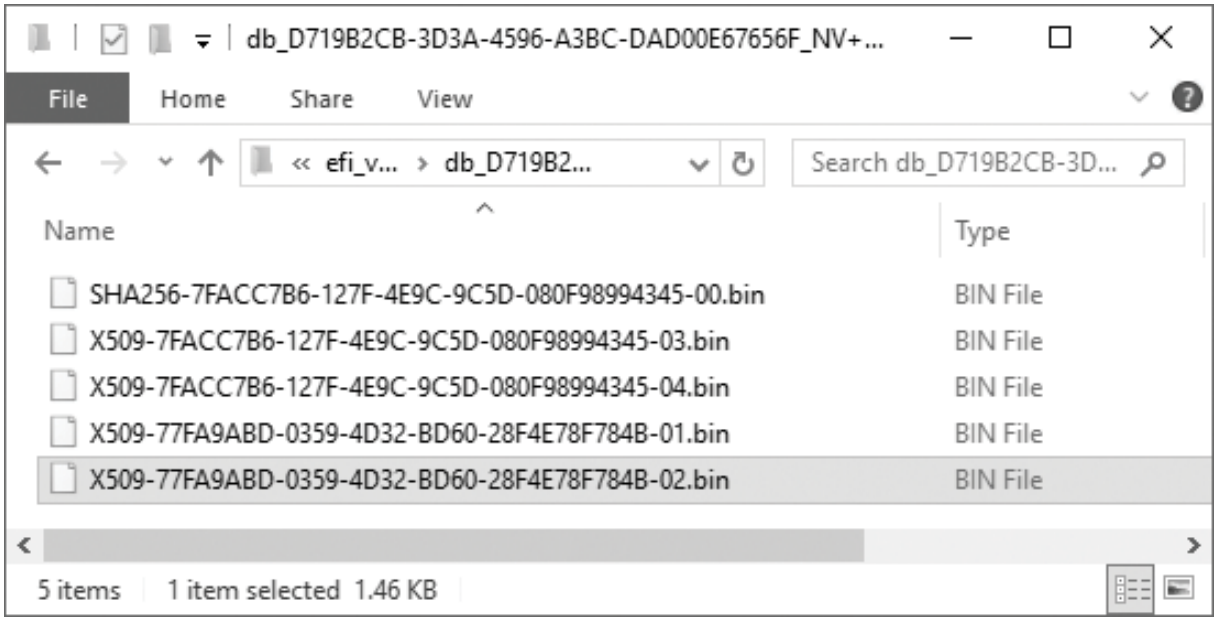


Рисунок 17-3. Содержимое переменной UEFI базы данных подписей

Эти сертификаты закодированы в X.509, криптографическом стандарте, определяющем формат сертификатов открытых ключей. Мы можем декодировать сертификаты и получить сведения об издателе, которые покажут, чья подпись пройдёт проверку Secure Boot. Для этого воспользуемся набором инструментов `openssl`, описанным во вставке «Набор инструментов OpenSSL». Установите инструмент со страницы https://github.com/openssl/openssl/, а затем запустите следующей командой, заменив `certificate_file_path` каталогом на вашем компьютере, содержащим `openssl`:

```
$ openssl x509 -in certificate_file_path
```

В операционной системе Windows достаточно изменить расширение файла сертификата X.509 с `bin` на `crt` и открыть файл в Проводнике, чтобы увидеть результат декодирования. В таблице 17-1 показаны полученные нами результаты с издателями и субъектами сертификатов.

Таблица 17-1. Декодированные сертификаты и хеши из переменной UEFI

Имя файла	Кому выдан	Кем выдан
`X509-7FACC7B6-127F-4E9C-9C5D-080F98994345-03.bin`	ThinkPad Product CA 2012	Lenovo Ltd. Root CA 2012
`X509-7FACC7B6-127F-4E9C-9C5D-080F98994345-04.bin`	Lenovo UEFI CA 2014	Lenovo UEFI CA 2014
`X509-77FA9ABD-0359-4D32-BD60-28F4E78F784B-01.bin`	Microsoft Corporation UEFI CA 2011	Microsoft Corporation Third-Party Marketplace Root
`X509-77FA9ABD-0359-4D32-BD60-28F4E78F784B-02.bin`	Microsoft Windows Production PCA 2011	Microsoft Root Certificate Authority 2010

Из таблицы видно, что проверки целостности кода UEFI Secure Boot пройдут только образы UEFI, подписанные Lenovo и Microsoft.

> **Набор инструментов OpenSSL**
>
> OpenSSL представляет собой библиотеку программного обеспечения с открытым исходным кодом, реализующую протоколы Secure Socket Layer и Transport Layer Security, а также криптографические примитивы общего назначения. OpenSSL распространяется по лицензии в стиле Apache и часто используется в коммерческих и некоммерческих приложениях. Библиотека предоставляет богатую функциональность для работы с сертификатами X.509, как для разбора существующих сертификатов, так и для создания новых. Сведения о проекте можно найти по адресу https://www.openssl.org/.

База данных dbx

В отличие от `db`, база данных `dbx` содержит сертификаты открытых ключей и хеши исполняемых файлов UEFI, выполнение которых во время загрузки запрещено. Эта база данных также называется базой отозванных подписей (Revoked Signature Database) и явно перечисляет образы, которые не пройдут проверку Secure Boot, предотвращая выполнение модуля с известной уязвимостью, способной скомпрометировать безопасность всей платформы.

Содержимое базы данных dbx мы исследуем тем же способом, что и базу подписей db. Среди папок, созданных при запуске Chipsec, находится папка `efi_variables.dir`, которая должна содержать подкаталог с именем, начинающимся с `dbx_D719B2CB-3D3A-4596-A3BC-DAD00E67656F`. В этой папке хранятся сертификаты и хеши запрещённых образов UEFI. В нашем случае папка содержит только 78 хешей и не содержит сертификатов, как показано на рисунке 17-4.

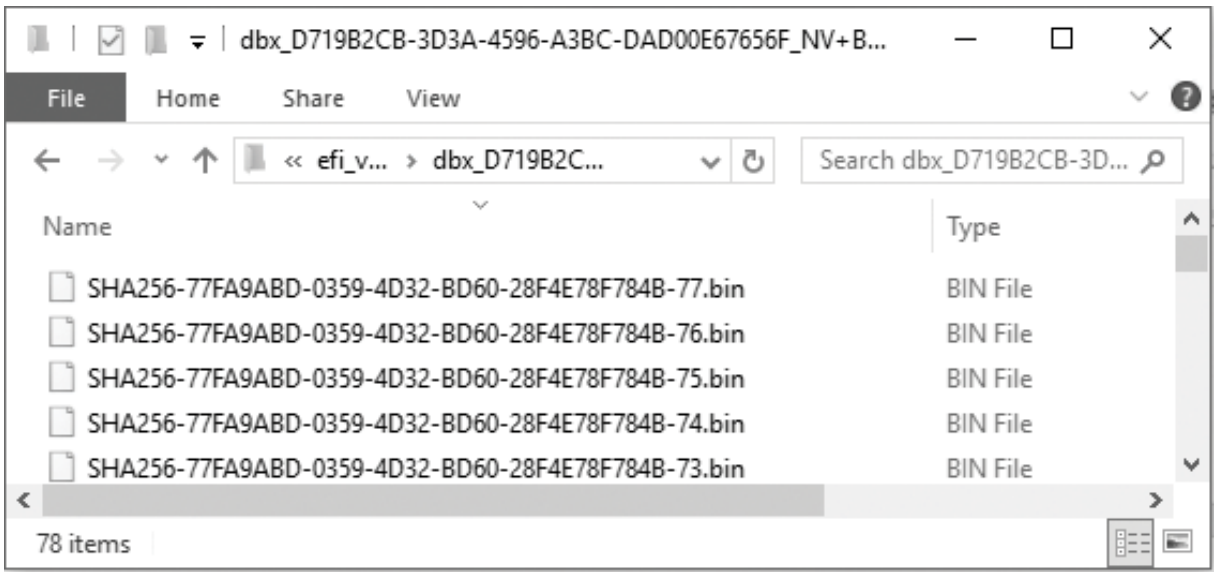


Рисунок 17-4. Содержимое базы данных dbx (базы отозванных подписей) в переменной UEFI

На рисунке 17-5 показан алгоритм проверки подписи образа с использованием обеих баз данных, db и dbx.

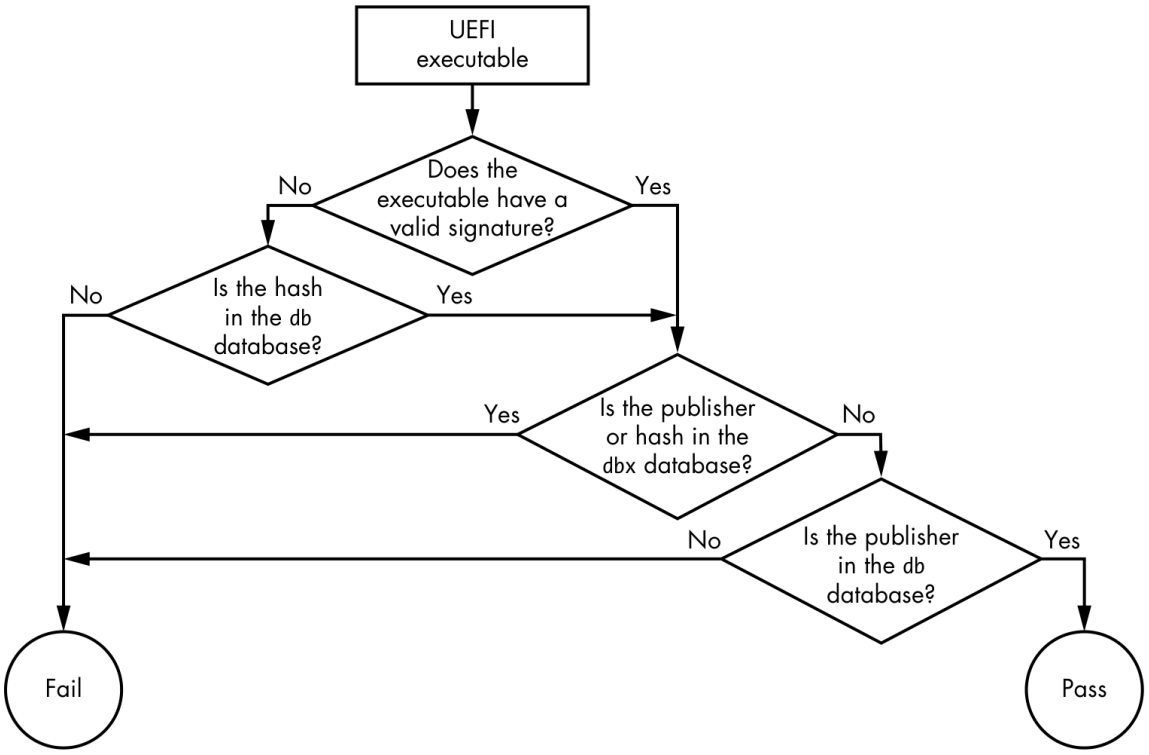


Рисунок 17-5. Алгоритм проверки образа UEFI Secure Boot

Из рисунка видно, что исполняемый файл UEFI проходит аутентификацию только в том случае, если его хеш или сертификат подписи считается доверенным согласно базе данных db и при этом он не указан в базе данных dbx. В противном случае образ не проходит проверку целостности Secure Boot.

Аутентификация на основе времени

Помимо баз данных db и dbx, Secure Boot использует ещё две базы данных, называемые dbt и dbr. Первая, dbr, содержит сертификаты открытых ключей, используемые для проверки подписей загрузчика восстановления ОС. Мы не будем подробно её обсуждать.

Вторая, dbt, содержит сертификаты меток времени, используемые для проверки временной метки цифровой подписи исполняемого файла UEFI, что позволяет применять аутентификацию на основе времени (TBAWS) в Secure Boot. С TBAWS вы уже встречались ранее в этой главе при рассмотрении атрибутов переменных UEFI.

Цифровая подпись исполняемого файла UEFI иногда содержит временную метку, выпущенную службой Time Stamping Authority (TSA). Временная метка подписи отражает момент её создания. Сравнивая временную метку подписи и метку истечения срока действия ключа подписывания, Secure Boot определяет, была ли подпись создана до или после истечения срока действия ключа. Обычно дата истечения срока действия ключа подписывания является датой, после которой ключ считается скомпрометированным. В результате временная метка подписи позволяет Secure Boot проверить, что подпись была создана в момент, когда ключ подписывания ещё не был скомпрометирован, обеспечивая легитимность подписи. Таким образом, аутентификация на основе времени снижает сложность PKI применительно к сертификатам db Secure Boot.

Аутентификация на основе времени также позволяет избежать повторного подписывания тех же образов UEFI. Временная метка подписи доказывает Secure Boot, что образ UEFI был подписан до истечения срока действия или отзыва соответствующего ключа подписывания. Поэтому подпись остаётся действительной даже после истечения срока действия ключа, поскольку была создана, когда ключ ещё был действителен и не был скомпрометирован.

Ключи Secure Boot

Теперь, когда вы знаете, откуда Secure Boot получает сведения о доверенных и отозванных сертификатах открытых ключей, поговорим о том, как эти базы данных хранятся и защищаются от несанкционированной модификации. Изменив базу данных db, злоумышленник мог бы легко обойти проверки Secure Boot, внедрив вредоносный сертификат и заменив загрузчик ОС вредоносным загрузчиком, подписанным закрытым ключом, соответствующим вредоносному сертификату. Поскольку вредоносный сертификат находится в базе подписей db, Secure Boot разрешила бы выполнить вредоносный загрузчик.

Поэтому для защиты баз данных db и dbx от несанкционированной модификации поставщик платформы или системы ОС должен подписывать эти базы данных. Когда прошивка UEFI собирается прочитать содержимое этих баз данных, она сначала аутентифицирует их, проверяя цифровую подпись с помощью открытого ключа, называемого ключом обмена ключами (key exchange key, KEK). Затем она аутентифицирует каждый KEK с помощью второго ключа, называемого ключом платформы (platform key, PK).

Ключи обмена ключами

Как и в случае баз данных db и dbx, список открытых KEK хранится в переменной UEFI NVRAM. Исследуем содержимое переменной KEK, используя результаты предыдущего выполнения команды Chipsec. Откройте каталог с результатами: там должен находиться подкаталог с именем наподобие KEK_8BE4DF61-93CA-11D2-AA0D-00E098032B8C, содержащий сертификаты открытых KEK (рисунок 17-6). Как будет показано далее, эта переменная UEFI также аутентифицируется.

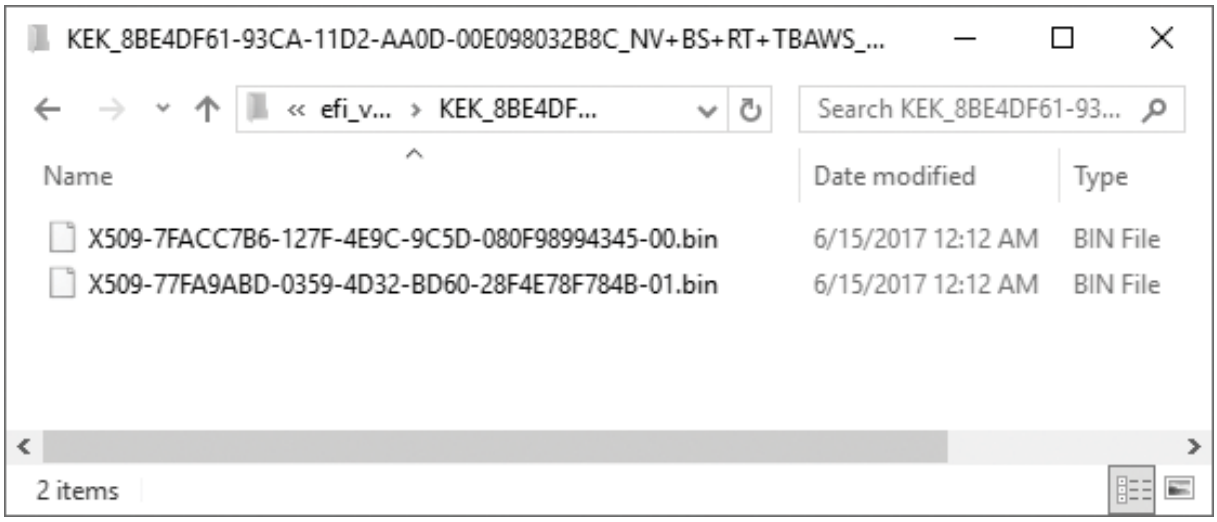


Рисунок 17-6. Содержимое переменной UEFI KEK

Изменить содержимое баз данных db и dbx может только владелец закрытого ключа, соответствующего одному из этих сертификатов. В данном примере имеется только два сертификата KEK, от Microsoft и Lenovo, как указано в таблице 17-2.

Таблица 17-2. Сертификаты в переменной UEFI KEK

Имя файла	Кому выдан	Кем выдан
X509-7FACC7B6-127F-4E9C-9C5D-080F98994345-00.bin	Lenovo Ltd. KEK CA 2012	Lenovo Ltd. KEK CA 2012
X509-77FA9ABD-0359-4D32-BD60-28F4E78F784B-01.bin	Microsoft Corporation KEK CA 2011	Microsoft Corporation Third-Party Marketplace Root

Владельцев закрытых ключей, соответствующих сертификатам KEK вашей системы, можно определить, выгрузив переменную KEK и выполнив использованную ранее команду openssl.

Ключ платформы

PK является последним ключом подписывания в иерархии ключей PKI Secure Boot. Как вы, вероятно, догадались, этот ключ используется для аутентификации KEK путём подписывания переменной UEFI KEK. Согласно спецификации UEFI, каждая платформа имеет единственный PK. Обычно этот ключ соответствует производителю платформы.

Вернитесь в подкаталог `PK_8BE4DF61-93CA-11D2-AA0D-00E098032B8C` каталога `efi_variables.dir`, созданного при выполнении Chipsec. Там можно найти сертификат открытого PK. Ваш сертификат будет соответствовать вашей платформе. Поскольку мы использовали платформу Lenovo ThinkPad T540p, ожидается, что сертификат PK будет соответствовать Lenovo (см. рисунок 17-7).

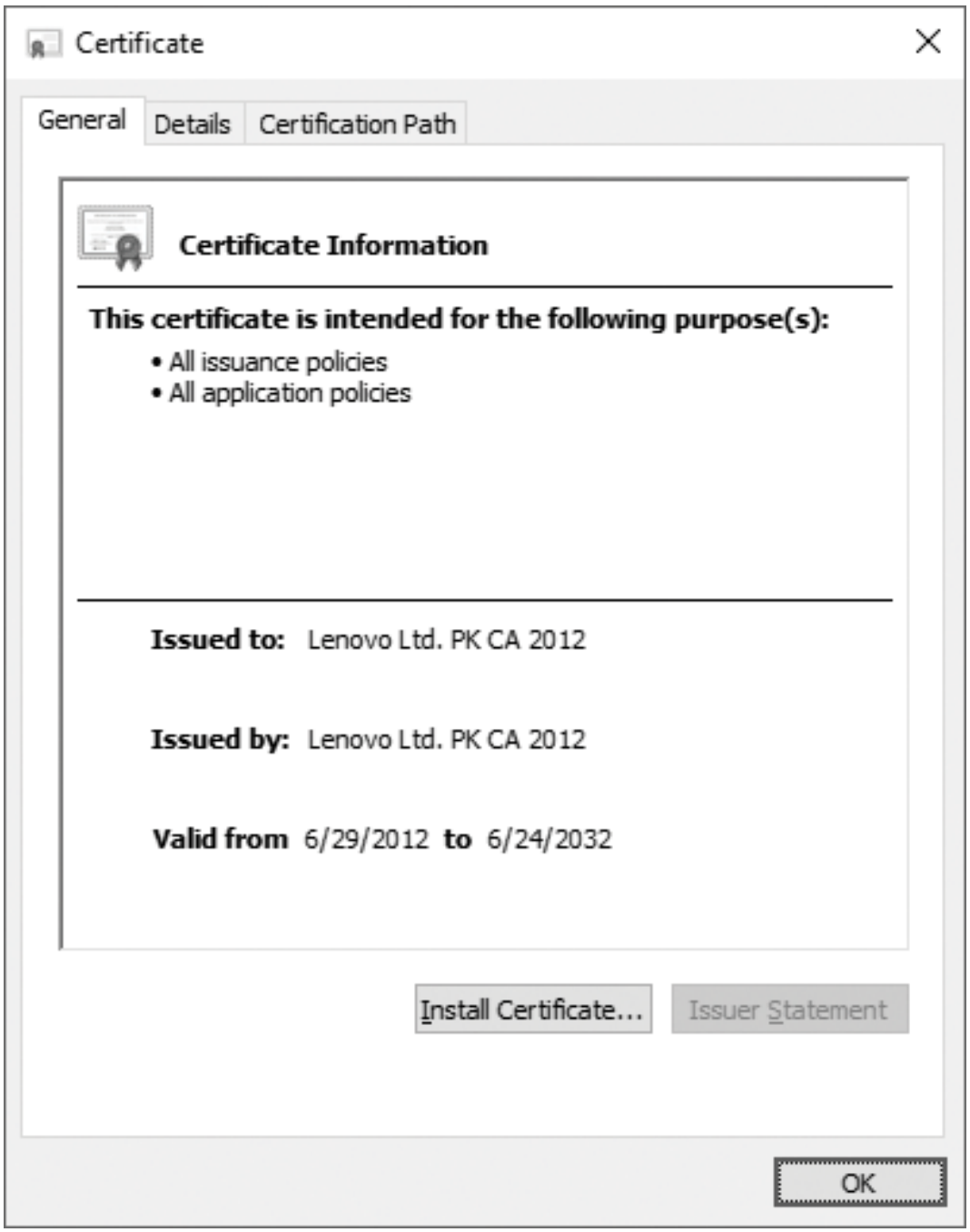


Рисунок 17-7. Сертификат PK

Как видно, наш сертификат действительно выпущен Lenovo. Переменная UEFI `PK` также аутентифицируется, и каждое обновление переменной должно быть подписано соответствующим закрытым ключом. Иными словами, если владелец платформы, или производитель платформы в терминологии UEFI, хочет обновить переменную `PK` новым сертификатом, буфер с новым сертификатом должен быть подписан закрытым ключом, соответствующим текущему сертификату, хранящемуся в переменной `PK`.

UEFI Secure Boot: полная картина

Теперь, исследовав полную иерархию инфраструктуры PKI, используемой в UEFI Secure Boot, объединим всё вместе и рассмотрим полную картину, показанную на рисунке 17-8.

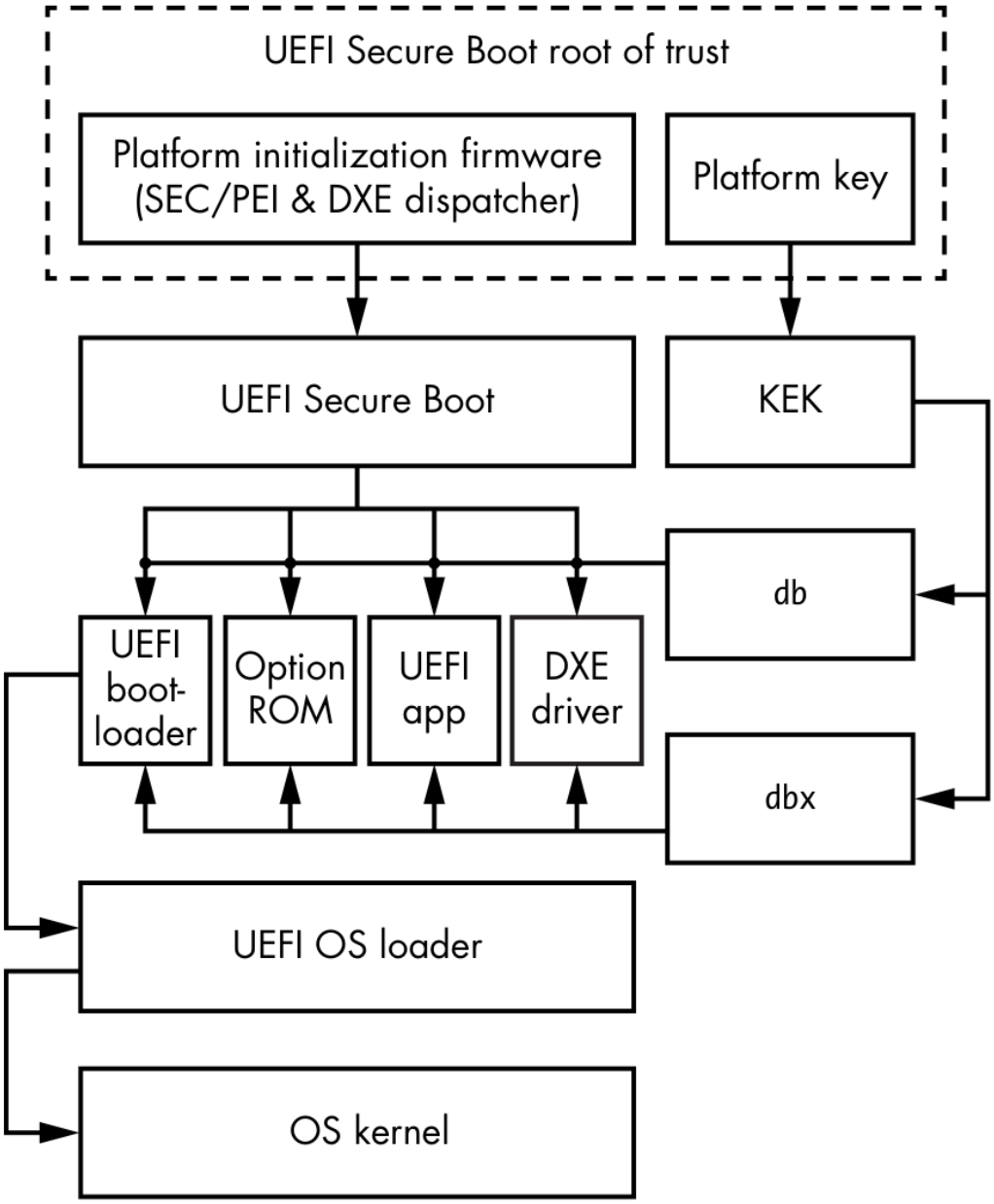


Рисунок 17-8. Поток проверки UEFI Secure Boot

В верхней части рисунка видно, что корнем доверия, то есть компонентами, которым UEFI Secure Boot доверяет изначально и на которых основывает все последующие проверки, являются прошивка инициализации платформы и ключ платформы. Прошивка инициализации платформы представляет собой самый первый код, выполняемый при выходе процессора из сброса, и UEFI Secure Boot безоговорочно доверяет ему. Если злоумышленник скомпрометирует прошивку PI, вся цепочка доверия, обеспечиваемая Secure Boot, будет нарушена. В таком случае злоумышленник может исправить любой модуль UEFI, реализующий процедуры проверки образов Secure Boot, так, чтобы он всегда возвращал успех и в результате разрешал каждому предоставленному образу UEFI пройти аутентификацию.

Именно поэтому модель доверия Secure Boot предполагает правильную реализацию механизма Firmware Secure Update, требующего, чтобы каждое обновление прошивки было подписано надлежащим ключом подписывания, который должен отличаться от PK. Благодаря этому выполняются только авторизованные обновления прошивки PI, а корень доверия остаётся нескомпрометированным.

Легко заметить, что эта модель доверия не защищает от злоумышленников с физическим доступом, способных физически перепрограммировать SPI flash вредоносным образом прошивки и скомпрометировать прошивку PI. О защите прошивки от физических атак мы поговорим в разделе «Защита Secure Boot с помощью Verified and Measured Boot» на странице 338.

В верхней части рисунка 17-8 видно, что ключ платформы, предоставленный производителем платформы, имеет тот же уровень изначального доверия, что и прошивка PI.

Этот ключ используется для установления доверия между прошивкой PI и производителем платформы. После предоставления ключа платформы прошивка платформы позволяет производителю обновлять KEK и тем самым управлять тем, какие образы проходят проверки Secure Boot, а какие нет.

На уровень ниже находятся KEK, устанавливающие доверие между прошивкой PI и ОС, работающей на платформе. После подготовки KEK платформы в переменной UEFI операционная система может указывать, какие образы способны пройти проверку Secure Boot. Например, поставщик ОС может использовать KEK, чтобы разрешить прошивке UEFI выполнить загрузчик ОС.

В нижней части модели доверия находятся базы данных `db` и `dbx`, подписанные KEK и содержащие хеши образов и сертификаты открытых ключей, непосредственно используемые в проверках целостности исполняемых файлов, обеспечиваемых Secure Boot.

Политика Secure Boot

Сама по себе Secure Boot использует переменные `PK`, `KEK`, `db`, `dbx` и `dbt`, чтобы сообщить платформе, является ли исполняемый образ доверенным. Однако способ интерпретации результата проверки Secure Boot, то есть решение о выполнении образа, в значительной степени зависит от действующей политики.

В этой главе мы уже несколько раз упоминали политики Secure Boot, не углубляясь в то, что они собой представляют. Теперь рассмотрим эту концепцию подробнее.

По существу, политика Secure Boot определяет действия, которые прошивка платформы должна предпринять после аутентификации образа. Прошивка может выполнить образ, запретить его выполнение, отложить выполнение или попросить пользователя принять решение.

Политика Secure Boot строго не определена в спецификации UEFI и потому зависит от конкретной реализации. В частности, политики могут различаться между реализациями прошивки UEFI разных поставщиков. В этом разделе мы исследуем некоторые элементы политики Secure Boot, реализованные в исходном коде Intel EDK2, который использовали в главе 15. Если вы ещё не сделали этого, загрузите или клонируйте исходный код EDK2 из репозитория https://github.com/tianocore/edk2/.

Один из элементов, учитываемых Secure Boot в реализации EDK2, - происхождение аутентифицируемых исполняемых образов. Образы могут поступать с разных устройств хранения, некоторым из которых можно доверять изначально. Например, если образ загружается из SPI flash, то есть находится на том же устройстве хранения, что и остальная прошивка UEFI, платформа может автоматически ему доверять. Однако если злоумышленник способен изменить образ в SPI flash, он также может вмешаться в остальную прошивку и полностью отключить Secure Boot. Эту

атаку мы обсудим далее в разделе «Исправление прошивки PI для отключения Secure Boot» на странице 335. С другой стороны, если образ загружается с внешнего устройства PCI, например из Option ROM, специальной прошивки, загружаемой с внешних периферийных устройств в предзагрузочной среде, он будет считаться недоверенным и подвергнется проверке Secure Boot.

Ниже приведены определения некоторых политик, определяющих обработку образов в зависимости от их происхождения. Эти политики находятся в файле `SecurityPkg\SecurityPkg.dec` репозитория EDK2. Каждая политика присваивает образам, удовлетворяющим критериям, значение по умолчанию.

`PcdOptionRomImageVerificationPolicy`. Определяет политику проверки образов, загруженных как Option ROM, например с устройств PCI. Значение по умолчанию: `0x00000004`.

`PcdRemovableMediaImageVerificationPolicy`. Определяет политику проверки образов, расположенных на съёмных носителях, включая CD-ROM, USB и сеть. Значение по умолчанию: `0x00000004`.

`PcdFixedMediaImageVerificationPolicy`. Определяет политику проверки образов на несъёмных носителях, например жёстких дисках. Значение по умолчанию: `0x00000004`.

Помимо этих политик существуют ещё две, которые явно не определены в файле `SecurityPkg\SecurityPkg.dec`, но используются в реализации Secure Boot EDK2.

Политика ROM в SPI flash. Определяет политику проверки образов, расположенных в SPI flash. Значение по умолчанию: `0x00000000`.

Другое происхождение. Определяет политику проверки всех образов, расположенных на устройствах, отличных от описанных выше. Значение по умолчанию: `0x00000004`.

> **Примечание.** Имейте в виду, что это не исчерпывающий список политик Secure Boot, используемых для аутентификации образов. Разные поставщики прошивки могут изменять или расширять его собственными политиками.

Ниже приведены описания значений политик по умолчанию.

`0x00000000`. Всегда доверять образу независимо от наличия подписи и от того, находится ли его хеш в базе данных `db` или `dbx`.

`0x00000001`. Никогда не доверять образу. Даже образы с действительными подписями будут отклонены.

`0x00000002`. Разрешить выполнение при нарушении безопасности. Образ будет выполнен, даже если подпись невозможно проверить или его хеш внесён в чёрный список базы данных `dbx`.

`0x00000003`. Отложить выполнение при нарушении безопасности. В таком случае образ не отклоняется немедленно и загружается в память. Однако его выполнение откладывается до повторной оценки состояния аутентификации.

`0x00000004`. Запретить выполнение, если Secure Boot не может аутентифицировать образ с помощью баз данных `db` и `dbx`.

`0x00000005`. Запросить решение пользователя при нарушении безопасности. Если Secure Boot не может аутентифицировать образ, авторизованный пользователь может решить, доверять ли ему. Например, во время загрузки пользователю может быть показано сообщение с запросом.

Из определений политики Secure Boot видно, что всем образам, загружаемым из SPI flash, доверяют изначально и вообще не подвергают проверке цифровой подписи. Во всех остальных случаях значение по умолчанию `0x00000004` требует проверки подписи и запрещает выполнение любого неаутентифицированного кода, поступающего как Option ROM либо находящегося на съёмном, несъёмном или любом другом носителе.

Защита от буткитов с помощью Secure Boot

Теперь, разобрав принцип работы Secure Boot, рассмотрим конкретный пример того, как она защищает от буткитов, нацеленных на поток загрузки ОС. Мы не будем обсуждать буткиты, нацеленные на MBR и VBR, поскольку, как объяснялось в главе 14, прошивка UEFI больше не использует такие объекты, как MBR и VBR, кроме режима совместимости UEFI, поэтому традиционные буткиты не могут компрометировать системы на основе UEFI.

Как упоминалось в главе 15, буткит DreamBoot был первым публичным доказательством концепции буткита, нацеленного на системы UEFI. В системе UEFI без Secure Boot этот буткит работает следующим образом:

1. Автор буткита заменяет исходный загрузчик UEFI Windows `bootmgfw.efi` вредоносным загрузчиком `bootx64.efi` в загрузочном разделе.
2. Вредоносный загрузчик загружает исходный `bootmgfw.efi`, исправляет его, чтобы получить управление загрузчиком Windows `winload.efi`, и выполняет его, как показано на рисунке 17-9.

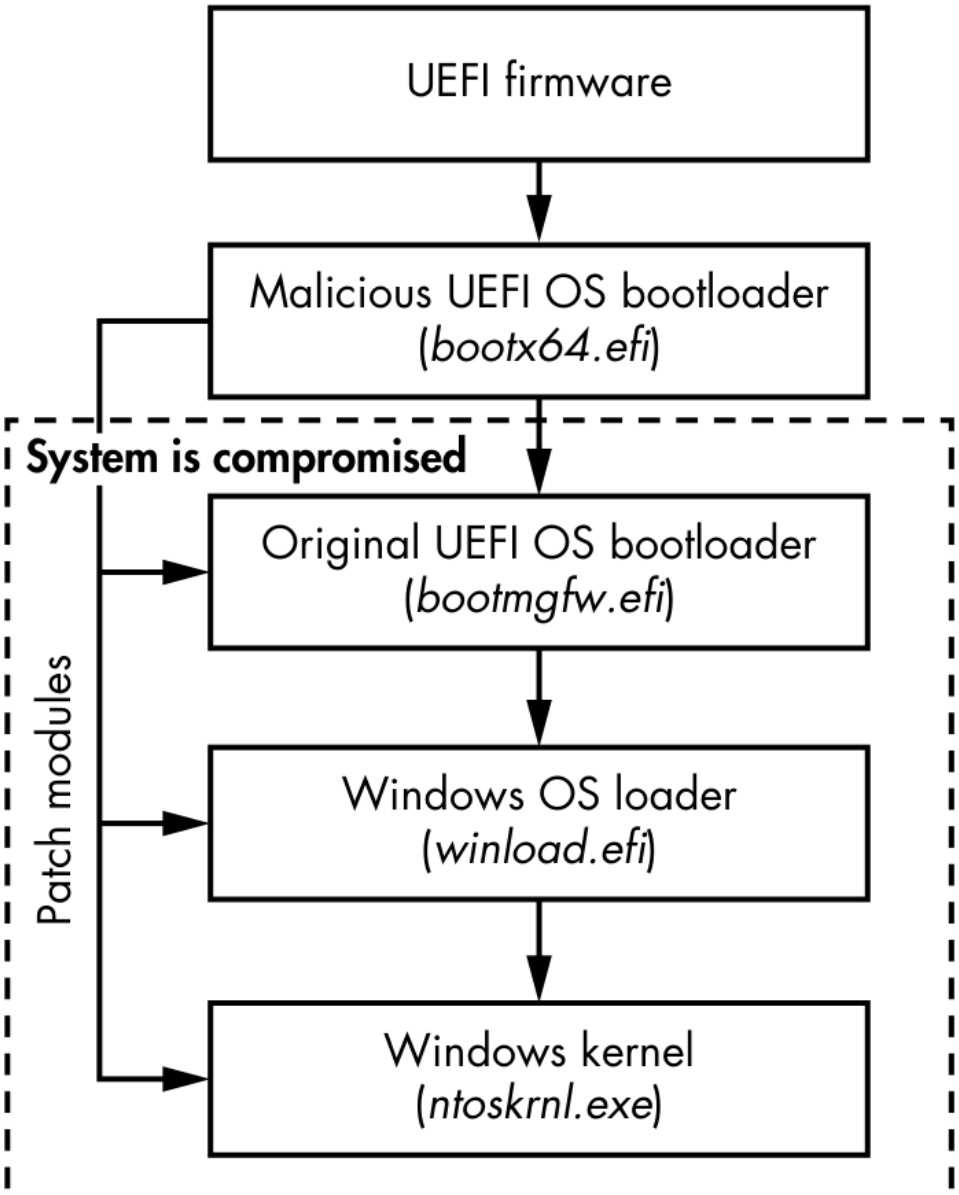


Рисунок 17-9. Поток атаки DreamBoot против загрузчика ОС

3. Вредоносный код продолжает исправлять системные модули, пока не достигнет ядра операционной системы, обходя механизмы защиты ядра, например политику подписывания кода режима ядра, предназначенные для предотвращения выполнения неавторизованного кода режима ядра.

Такая атака возможна, поскольку по умолчанию загрузчик ОС не аутентифицируется в процессе загрузки UEFI. Прошивка UEFI получает расположение загрузчика ОС из переменной UEFI; для платформ Microsoft Windows он находится по пути `\EFI\Microsoft\Boot\bootmgfw.efi` в загрузочном разделе. Злоумышленник с системными привилегиями может легко заменить или изменить загрузчик.

Однако при включённой Secure Boot такая атака больше невозможна. Поскольку Secure Boot проверяет целостность образов UEFI, выполняемых во время загрузки, а загрузчик ОС входит в число проверяемых исполняемых файлов, Secure Boot сверит подпись загрузчика с базами данных `db` и `dbx`. Вредоносный загрузчик не подписан надлежащим ключом подписывания, поэтому потенциально не пройдёт проверку и не будет выполнен, в зависимости от политики загрузки. Это один из способов защиты Secure Boot от буткитов.

Атака на Secure Boot

Теперь рассмотрим некоторые атаки, способные успешно действовать против UEFI Secure Boot. Поскольку Secure Boot опирается на прошивку PI и PK как на корень доверия, компрометация любого из этих компонентов делает всю цепочку проверок Secure Boot бесполезной. Мы рассмотрим как буткиты, так и руткиты, способные подорвать Secure Boot.

Рассматриваемый здесь класс буткитов главным образом опирается на модификацию содержимого SPI flash. В современных компьютерных системах SPI flash часто используется как основное хранилище прошивки. Практически каждый ноутбук и настольный компьютер хранит прошивку UEFI во flash-памяти, доступ к которой осуществляется через контроллер SPI.

В главе 15 были представлены различные атаки, устанавливающие постоянные руткиты UEFI во flash-прошивку, поэтому мы не будем снова вдаваться в подробности, хотя те же атаки, включая проблемы обработчиков SMI, S3 Boot Script и защиту BIOS от записи, можно использовать против Secure Boot. Для атак этого раздела предположим, что злоумышленник уже способен изменить содержимое flash-памяти с прошивкой UEFI. Посмотрим, что он может сделать далее.

Исправление прошивки PI для отключения Secure Boot

Получив возможность изменить содержимое SPI flash, злоумышленник может легко отключить Secure Boot, исправив прошивку PI. На рисунке 17-8 было показано, что UEFI Secure Boot закреплена в прошивке PI, поэтому, изменив модули прошивки PI, реализующие Secure Boot, можно фактически отключить её функциональность.

Для изучения этого процесса мы снова воспользуемся исходным кодом Intel EDK2 (https://github.com/tianocore/edk2/) как примером реализации UEFI. Мы выясним, где реализована функциональность проверки Secure Boot и как её можно повредить.

В папке `SecurityPkg/Library/DxeImageVerificationLib` репозитория находится файл исходного кода `DxeImageVerificationLib.c`, реализующий функциональность проверки целостности кода. В частности, этот файл реализует процедуру `DxeImageVerificationHandler`, которая решает, является ли исполняемый файл UEFI доверенным и должен ли он выполняться либо не проходит проверку. В листинге 17-1 показан прототип процедуры.

```
EFI_STATUS EFI_API DxeImageVerificationHandler (
  IN  UINT32                          AuthenticationStatus, ❶
  IN  CONST EFI_DEVICE_PATH_PROTOCOL  *File, ❷
  IN  VOID                            *FileBuffer, ❸
  IN  UINTN                           FileSize, ❹
  IN  BOOLEAN                         BootPolicy ❺
);
```

Листинг 17-1. Определение процедуры DxeImageVerificationHandler

Первым параметром процедура получает переменную `AuthenticationStatus` ❶, указывающую, подписан ли образ. Аргумент `File` ❷ является указателем на путь устройства к передаваемому файлу. Аргументы `FileBuffer` ❸ и `FileSize` ❹ предоставляют для проверки указатель на образ UEFI и его размер.

Наконец, `BootPolicy` ❺ является параметром, указывающим, поступил ли запрос на загрузку аутентифицируемого образа от диспетчера загрузки UEFI и является ли он выбором загрузки, то есть выбранным загрузчиком ОС. Подробнее диспетчер загрузки UEFI рассматривался в главе 14.

По завершении проверки эта процедура возвращает одно из следующих значений.

EFI_SUCCESS. Аутентификация успешно пройдена, и образ будет выполнен.

EFI_ACCESS_DENIED. Образ не аутентифицирован, поскольку политика платформы предписывает прошивке не использовать этот файл образа. Это может произойти, если прошивка пытается загрузить образ со съёмного носителя, а политика платформы запрещает выполнение со съёмных носителей во время загрузки независимо от наличия подписи. В этом случае процедура немедленно вернёт `EFI_ACCESS_DENIED` без проверки подписи.

EFI_SECURITY_VIOLATION. Аутентификация завершилась неудачей либо потому, что Secure Boot не смогла проверить цифровую подпись образа, либо потому, что значение хеша исполняемого файла найдено в базе запрещённых образов `dbx`. Это возвращаемое значение указывает, что образ не является доверенным, и платформа должна следовать политике Secure Boot, чтобы определить, можно ли его выполнить.

EFI_OUT_RESOURCE. Во время проверки произошла ошибка из-за нехватки системных ресурсов, обычно недостатка памяти, необходимых для аутентификации образа.

Чтобы обойти проверки Secure Boot, злоумышленник с доступом на запись к SPI flash может исправить эту процедуру так, чтобы она всегда возвращала `EFI_SUCCESS` для любого полученного исполняемого файла. В результате все образы UEFI будут проходить аутентификацию независимо от наличия подписи.

Модификация переменных UEFI для обхода проверок безопасности

Другой способ атаковать реализацию Secure Boot состоит в изменении переменных UEFI NVRAM. Как обсуждалось ранее в этой главе, Secure Boot использует определённые переменные для хранения параметров конфигурации: включена ли Secure Boot, PK и KEK, базы данных подписей и политики платформы. Если злоумышленник способен изменить эти переменные, он может отключить или обойти проверки Secure Boot.

Действительно, большинство реализаций Secure Boot хранит переменные UEFI NVRAM во flash-памяти SPI рядом с системной прошивкой. Хотя эти переменные аутентифицируются, а изменение их значений из режима ядра через API UEFI требует соответствующего закрытого ключа, злоумышленник, способный записывать в SPI flash, может изменить их содержимое.

Получив доступ к переменным UEFI NVRAM, злоумышленник может, например, вмешаться в PK, KEK, db и dbx, добавив собственные вредоносные сертификаты, что позволит вредоносному модулю обойти проверки безопасности. Другой вариант - добавить хеш вредоносного файла в базу db и удалить его из базы dbx, если изначально хеш там находился. Как показано на рисунке 17-10, изменив переменную PK так, чтобы она включала сертификат открытого ключа злоумышленника, он получает возможность добавлять и удалять KEK из переменной UEFI KEK, что, в свою очередь, даёт ему контроль над базами подписей db и dbx, нарушая защиту Secure Boot.

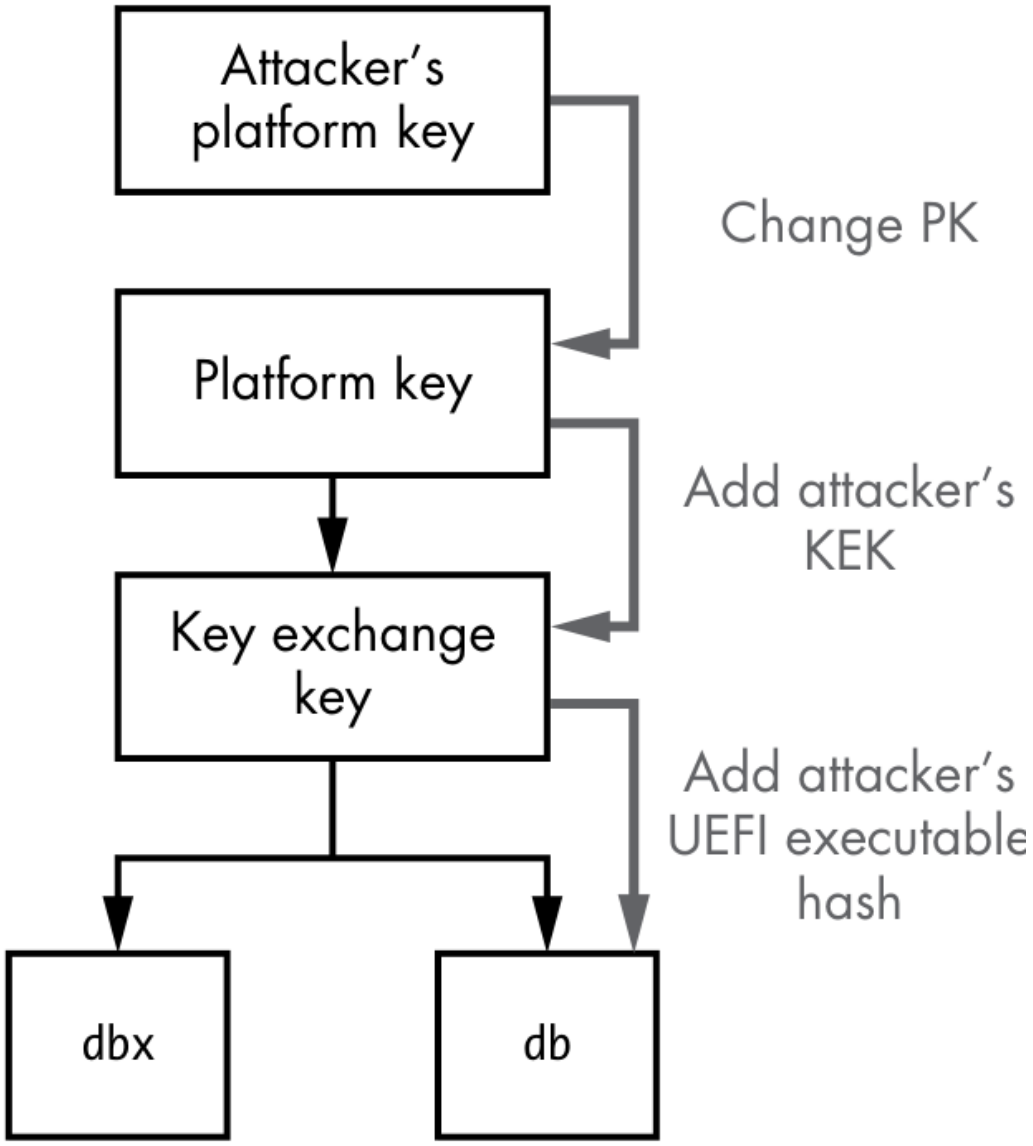


Рисунок 17-10. Атака на цепочку доверия UEFI Secure Boot

Третий вариант: вместо изменения PK и компрометации лежащей в основе иерархии PKI злоумышленник может просто повредить PK в переменной UEFI. Для работы Secure Boot требуется действительный PK, зарегистрированный в прошивке платформы; в противном случае защита отключается.

Если вам интересно узнать больше об этих атаках, следующие материалы конференций содержат всесторонний анализ технологии UEFI Secure Boot:

• Corey Kallenberg и др., «Setup for Failure: Defeating Secure Boot», LegbaCore, https://papers.put.as/papers/firmware/2014/SetupForFailure-syscan-v4.pdf;
• Yuriy Bulygin и др., «Summary of Attacks Against BIOS and Secure Boot», Intel Security, http://www.c7zero.info/stuff/DEFCON22-BIOSAttacks.pdf.

Защита Secure Boot с помощью Verified and Measured Boot

Как мы только что обсудили, одной Secure Boot недостаточно для защиты от атак, включающих изменения прошивки платформы. Существует ли защита самой технологии Secure Boot? Да. В этом разделе мы сосредоточимся на технологиях безопасности, предназначенных для защиты системной прошивки от несанкционированных изменений, а именно Verified and Measured Boot. Verified Boot проверяет, что прошивка платформы не подвергалась вмешательству или модификации, а Measured Boot вычисляет криптографические хеши определённых компонентов, участвующих в процессе загрузки, и сохраняет их в регистрах конфигурации платформы Trusted Platform Module (TPM PCR).

Verified Boot и Measured Boot работают независимо, и платформа может иметь только одну из этих технологий либо обе. Однако обе они являются частью одной цепочки доверия, как показано на рисунке 17-11.

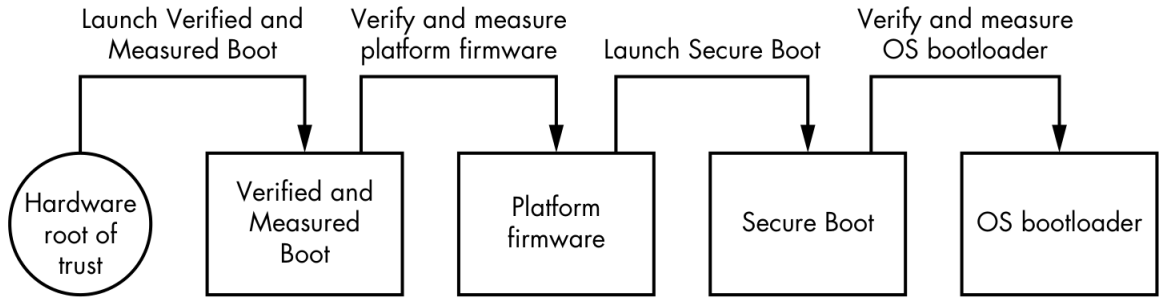


Рисунок 17-11. Поток Verified and Measured Boot

Как было показано на рисунке 17-8, прошивка PI является самым первым кодом, выполняемым после выхода процессора из сброса. UEFI Secure Boot безоговорочно доверяет прошивке PI, поэтому логично, что современные атаки на Secure Boot опираются на её несанкционированную модификацию.

Для защиты от таких атак системе необходим корень доверия вне прошивки PI. Именно здесь вступают в действие Verified and Measured Boot. Эти процессы выполняют механизмы защиты, корень доверия которых закреплён в оборудовании. Более того, они выполняются до системной прошивки, поэтому способны как аутентифицировать, так и измерять её. Значение измерения в этом контексте мы обсудим далее.

Verified Boot

При включении системы с Verified Boot аппаратная логика запускает функциональность проверки загрузки, реализованную в загрузочной ROM или микрокоде внутри процессора. Эта логика неизменяема, то есть программное обеспечение не может её изменить. Обычно Verified Boot выполняет модуль проверки целостности системы, обеспечивая выполнение системой подлинной прошивки без вредоносных модификаций. Для проверки прошивки Verified Boot опирается на криптографию открытых ключей: как и UEFI Secure Boot, она проверяет цифровую подпись прошивки платформы, чтобы убедиться в её подлинности. После успешной аутентификации прошивка платформы выполняется и переходит к проверке других компонентов прошивки, например Option ROM, драйверов DXE и загрузчиков ОС, чтобы поддерживать надлежащую цепочку доверия. Это проверяющая часть Verified and Measured Boot. Теперь перейдём к измеряющей части.

Measured Boot

Measured Boot работает посредством измерения прошивки платформы и загрузчиков ОС. Это означает вычисление криптографических хешей компонентов, участвующих в процессе загрузки. Хеши хранятся в наборе PCR модуля TPM. Сами значения хешей не показывают, являются ли измеренные компоненты доброкачественными или вредоносными, но позволяют определить, изменялись ли когда-либо конфигурация и загрузочные компоненты. Если загрузочный компонент был изменён, значение его хеша будет отличаться от вычисленного для исходной версии компонента. Таким образом, Measured Boot заметит любую модификацию загрузочного компонента.

Позднее системное программное обеспечение может использовать хеши в этих PCR TPM, чтобы убедиться, что система работает в известном надёжном состоянии без вредоносных модификаций. Система также может использовать эти хеши для удалённой аттестации, при которой одна система пытается доказать другой, что находится в доверенном состоянии.

Теперь, зная общий принцип работы Verified and Measured Boot, рассмотрим пару её реализаций, начиная с Intel BootGuard.

Intel BootGuard

Intel BootGuard является технологией Verified and Measured Boot компании Intel. На рисунке 17-12 показан поток загрузки на платформе с включённой Intel BootGuard.

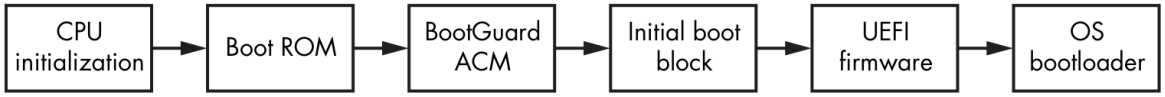


Рисунок 17-12. Поток Intel BootGuard

Во время инициализации, прежде чем процессор начнёт выполнять первый код, расположенный по вектору сброса, он выполняет код из загрузочной ROM. Этот код проводит необходимую инициализацию состояния процессора, а затем загружает и выполняет Authenticated Code Module (ACM) BootGuard.

ACM представляет собой особый вид модуля для выполнения операций, критичных для безопасности, и должен быть подписан Intel. Поэтому код загрузочной ROM, загружающий ACM, выполняет обязательную проверку подписи, не позволяя модулю запуститься, если он не подписан Intel. После успешной проверки подписи ACM выполняется в изолированной среде, чтобы вредоносное программное обеспечение не могло вмешаться в его работу.

ACM BootGuard реализует функциональность Verified and Measured Boot. Этот модуль загружает в память загрузчик прошивки первого этапа, называемый начальным загрузочным блоком (initial boot block, IBB), и в зависимости от действующей политики загрузки проверяет и/или измеряет его. IBB является частью прошивки, содержащей код, выполняемый по вектору сброса.

Строго говоря, на этом этапе процесса загрузки RAM отсутствует. Контроллер памяти ещё не инициализирован, и RAM недоступна. Однако процессор настраивает кэш последнего уровня так, чтобы его можно было использовать как RAM, переводя в режим Cache-as-RAM до момента, когда код обращения BIOS к памяти сможет настроить контроллер памяти и обнаружить RAM.

После успешной проверки и/или измерения IBB ACM передаёт ему управление. Если IBB не проходит проверку, ACM действует согласно действующей политике загрузки: система может немедленно выключиться либо разрешить восстановление прошивки по истечении определённого времени.

Затем IBB загружает остальную прошивку UEFI из SPI flash и проверяет и/или измеряет её. После получения IBB управления Intel BootGuard больше не отвечает за поддержание надлежащей цепочки доверия, поскольку его назначение состоит только в проверке и измерении IBB. IBB отвечает за продолжение цепочки доверия до момента, когда UEFI Secure Boot возьмёт на себя проверку и измерение образов прошивки.

Поиск ACM

Рассмотрим подробности реализации технологии Intel BootGuard для настольных платформ, начиная с ACM. Поскольку ACM является одним из первых компонентов Intel BootGuard, выполняемых при включении системы, первый вопрос звучит так: как процессор находит ACM при включении?

Точное расположение ACM указано в специальной структуре данных Firmware Interface Table (FIT), хранящейся в образе прошивки. FIT организована как массив записей FIT, каждая из которых описывает расположение определённого объекта в прошивке, например ACM или файлов обновления микрокода. На рисунке 17-13 показана структура FIT в системной памяти после сброса.

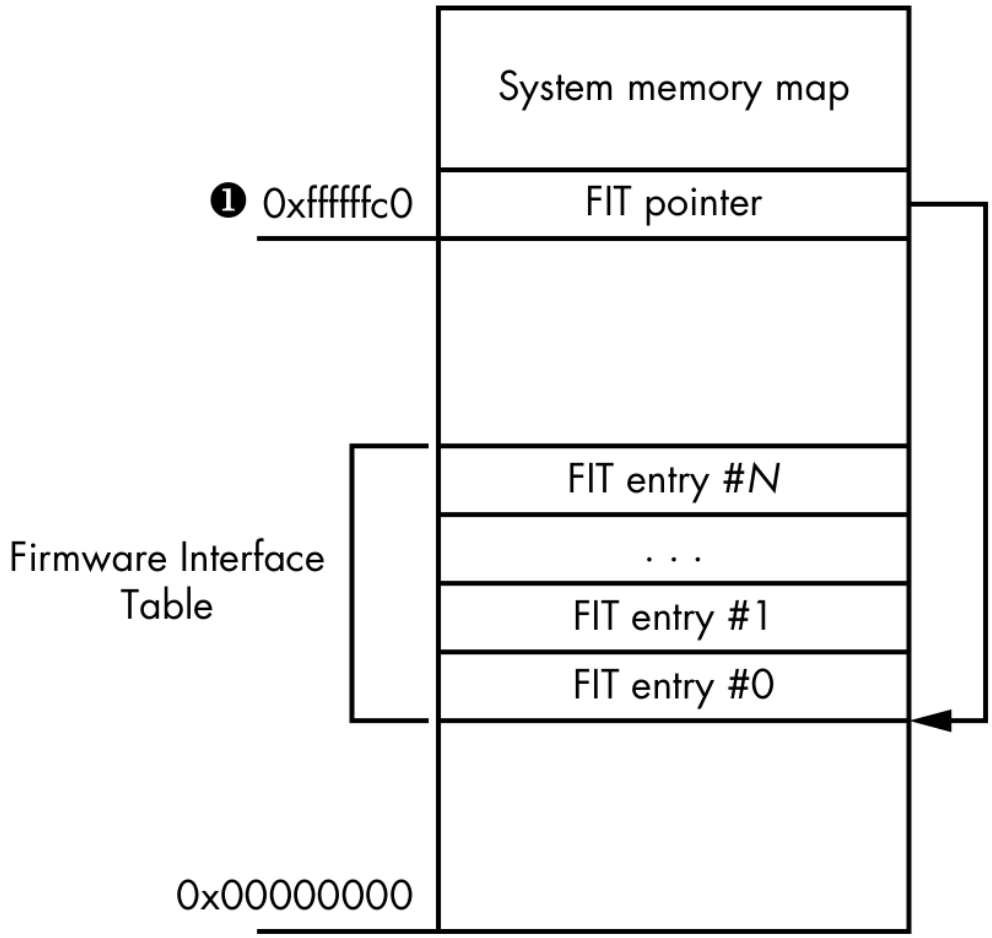


Рисунок 17-13. Расположение FIT в памяти

При включении процессор считывает адрес FIT из области памяти `0xFFFFFFC0` ❶. Поскольку RAM ещё отсутствует, когда процессор отправляет транзакцию чтения памяти для физического адреса `0xFFFFFFC0`, внутренняя логика набора микросхем распознаёт принадлежность адреса к специальному диапазону и декодирует его вместо отправки транзакции контроллеру памяти. Транзакции чтения памяти для таблицы FIT пересылаются контроллеру SPI flash, который считывает FIT из flash-памяти.

Подробно рассмотрим этот процесс, вернувшись к репозиторию EDK2. В каталоге `IntelSiliconPkg/Include/IndustryStandard/` находится заголовочный файл `FirmwareInterfaceTable.h`, содержащий несколько определений кода, связанных со структурой FIT. Структура записей FIT показана в листинге 17-2.

```
typedef struct {
  UINT64 Address; ❶
  UINT8  Size[3]; ❷
  UINT8  Reserved;
  UINT16 Version; ❸
  UINT8  Type : 7; ❹
  UINT8  C_V  : 1; ❺
  UINT8  Chksum; ❻
} FIRMWARE_INTERFACE_TABLE_ENTRY;
```

Листинг 17-2. Структура записей FIT

Как уже отмечалось, каждая запись FIT описывает определённый объект в образе прошивки. Природа каждого объекта закодирована в поле `Type` FIT. Такими объектами могут быть файлы обновления микрокода, ACM BootGuard или политика BootGuard. Поля `Address` ❶ и `Size` ❷ задают расположение объекта в памяти: `Address` содержит физический адрес объекта, а `Size` определяет размер в двойных словах, то есть 4-байтовых значениях. Поле `C_V` ❺ является признаком действительности контрольной суммы; если оно установлено в `1`, поле `Chksum` ❻ содержит действительную контрольную сумму объекта. Сумма всех байтов компонента по модулю `0xFF` и значения в поле `Chksum` должна равняться нулю. Поле `Version` ❸ содержит номер версии компонента в формате двоично-десятичного числа. Для записи заголовка FIT значение этого поля указывает номер редакции структуры данных FIT.

Заголовок `FirmwareInterfaceTable.h` содержит значения, которые может принимать поле `Type` ❹. Эти значения типов в основном не документированы и доступных сведений мало, но определения типов записей FIT достаточно выразительны, и их смысл можно вывести из контекста. Ниже приведены типы, относящиеся к BootGuard.

• Запись `FIT_TYPE_00_HEADER` предоставляет общее число записей в таблице FIT в поле `Size`. Её поле адреса содержит специальную 8-байтовую сигнатуру `'_FIT_ '`; после `_FIT_` идут три пробела.
• Запись вида `FIT_TYPE_02_STARTUP_ACM` предоставляет расположение ACM BootGuard, которое код загрузочной ROM разбирает, чтобы найти ACM в системной памяти.
• Записи видов `FIT_TYPE_0C_BOOT_POLICY_MANIFEST` (манифест политики загрузки BootGuard) и `FIT_TYPE_0B_KEY_MANIFEST` (манифест ключа BootGuard) предоставляют BootGuard действующую политику загрузки и сведения о конфигурации, которые мы вскоре рассмотрим в разделе «Настройка Intel BootGuard» на странице 343.

Имейте в виду, что политика загрузки Intel BootGuard и политика UEFI Secure Boot являются двумя разными понятиями. Первое относится к политике загрузки, используемой процедурами Verified and Measured Boot. То есть политика загрузки Intel BootGuard обеспечивается ACM и логикой набора микросхем и включает такие параметры, как необходимость выполнения BootGuard Verified и Measured Boot и действия BootGuard при неудачной аутентификации IBB. Второе понятие относится к UEFI Secure Boot, рассмотренной ранее в этой главе, и полностью обеспечивается прошивкой UEFI.

Исследование FIT

С помощью UEFITool, представленного в главе 15 и подробнее рассматриваемого в главе 19, можно исследовать некоторые записи FIT в образе прошивки и извлечь из образа ACM вместе с манифестами политики загрузки и ключа для дальнейшего анализа. Это может быть полезно, поскольку ACM можно использовать для сокрытия вредоносного кода. В следующем примере используется образ прошивки, полученный из системы с включённой технологией Intel BootGuard. Сведения о получении прошивки с платформы приведены в главе 19.

Сначала загрузите образ прошивки в UEFITool, выбрав **File > Open Image File**. После указания файла образа прошивки появится окно, похожее на показанное на рисунке 17-14.

Рисунок 17-14. Просмотр FIT в UEFITool

В нижней половине окна находится вкладка FIT со списком записей. В столбце Type вкладки FIT отображается вид записей FIT. Нас интересуют записи FIT видов BIOS ACM, BootGuard key manifest и BootGuard Boot Policy. По этим сведениям можно найти компоненты Intel BootGuard в образе прошивки и извлечь их для дальнейшего анализа. В данном примере запись FIT 6 указывает расположение BIOS ACM, начинающееся с адреса `0xfffc0000`. Записи FIT 7 и 8 указывают расположение манифестов ключа и политики загрузки, начинающихся соответственно с адресов `0xfffc9180` и `0xfffc8100`.

Настройка Intel BootGuard

При выполнении BIOS ACM BootGuard использует ключ BootGuard, а политика загрузки находит IBB в системной памяти, чтобы получить правильный открытый ключ для проверки подписи IBB.

Манифест ключа BootGuard содержит хеш манифеста политики загрузки (boot policy manifest, BPM), корневой открытый ключ OEM, цифровую подпись предыдущих полей, за исключением корневого открытого ключа, который не входит в подписанные данные, и номер версии безопасности, счётчик, увеличиваемый при каждом обновлении безопасности и предназначенный для предотвращения атак отката.

Сам BPM содержит номер версии безопасности, расположение и хеш IBB, открытый ключ BPM и цифровые подписи только что перечисленных полей BPM, опять же за исключением корневого открытого ключа, который можно проверить открытым ключом BPM. Расположение IBB задаёт его структуру в памяти. Это может быть не непрерывный блок памяти, а несколько несмежных областей. Хеш IBB содержит совокупное значение хеша всех областей памяти, занятых IBB. Таким образом, весь процесс проверки подписи IBB выглядит следующим образом:

1. BootGuard находит манифест ключа (KM) с помощью FIT и получает значение хеша манифеста политики загрузки и корневой ключ OEM, который назовём ключом 1. BootGuard проверяет цифровую подпись в KM с помощью ключа 1, чтобы убедиться в целостности значения хеша BPM. Если проверка не проходит, BootGuard сообщает об ошибке и запускает исправляющие действия.
2. Если проверка проходит успешно, BootGuard находит BPM с помощью FIT, вычисляет значение хеша BPM и сравнивает его с хешем BPM в KM. Если значения не равны, BootGuard сообщает об ошибке и запускает исправляющие действия; в противном случае получает значение хеша и расположение IBB из BPM.
3. BootGuard находит IBB в памяти, вычисляет его совокупный хеш и сравнивает с хешем IBB в BPM. Если хеши не равны, BootGuard сообщает об ошибке и запускает исправляющие действия.
4. В противном случае BootGuard сообщает об успешной проверке. Если включена Measured Boot, BootGuard также измеряет IBB, вычисляя его хеш, и сохраняет измерение в TPM. Затем BootGuard передаёт управление IBB.

KM является важнейшей структурой, поскольку содержит корневой открытый ключ OEM, используемый для проверки целостности IBB. У вас может возникнуть вопрос: если KM BootGuard хранится в незащищённой SPI flash вместе с образом прошивки, разве злоумышленник не может изменить его во flash-памяти, чтобы предоставить BootGuard поддельный ключ проверки? Для предотвращения такой атаки хеш корневого открытого ключа OEM хранится в однократно программируемых предохранителях набора микросхем. Эти предохранители можно запрограммировать только один раз при подготовке политики загрузки BootGuard. После записи предохранителей перезаписать их невозможно. Так ключ проверки BootGuard закрепляется в оборудовании, делая оборудование неизменяемым корнем доверия. Политика загрузки BootGuard также хранится в предохранителях набора микросхем, поэтому впоследствии изменить её невозможно.

Если злоумышленник изменит манифест ключа BootGuard, ACM обнаружит изменение ключа, вычислив его хеш и сравнив с «эталонным» значением, записанным в набор микросхем. Несовпадение хешей вызывает сообщение об ошибке и исправляющее поведение. На рисунке 17-15 показана цепочка доверия, обеспечиваемая BootGuard.

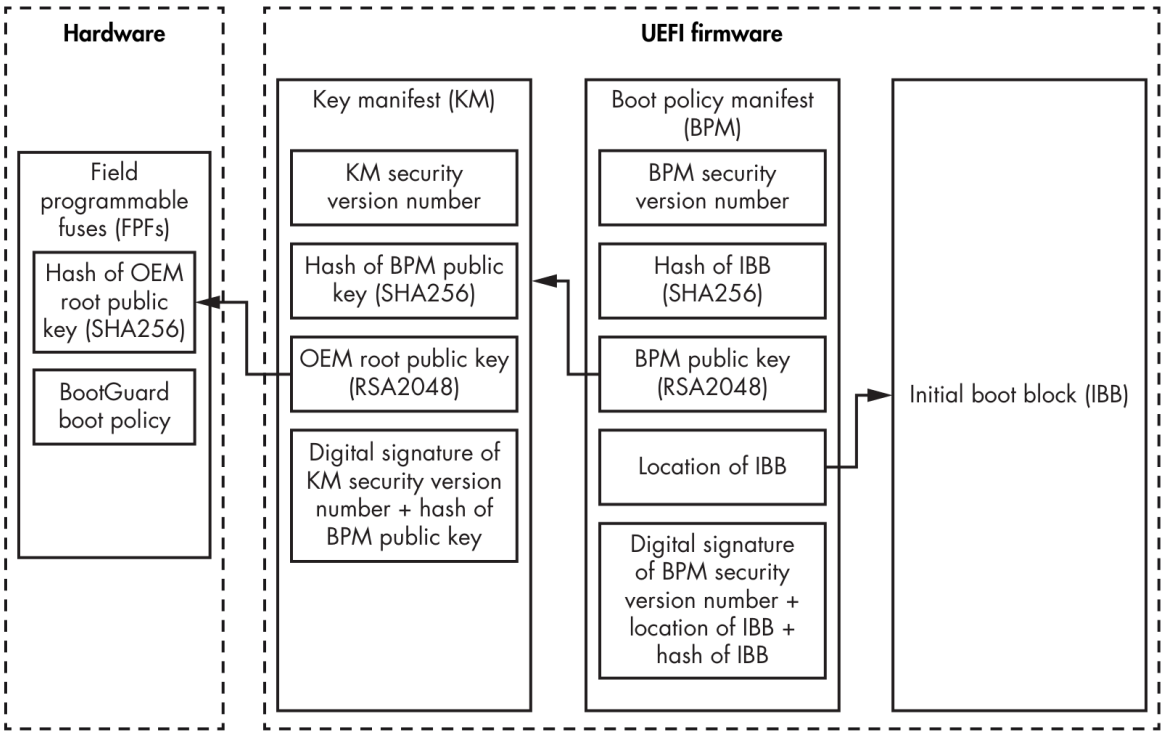


Рисунок 17-15. Цепочка доверия Intel BootGuard

После успешной проверки и, при необходимости, измерения IBB выполняется, проводит базовую инициализацию набора микросхем, а затем загружает прошивку UEFI. С этого момента IBB отвечает за аутентификацию прошивки UEFI до её загрузки и выполнения. В противном случае цепочка доверия будет нарушена.

Рисунок 17-16 завершает этот раздел представлением границ ответственности реализаций Secure Boot.

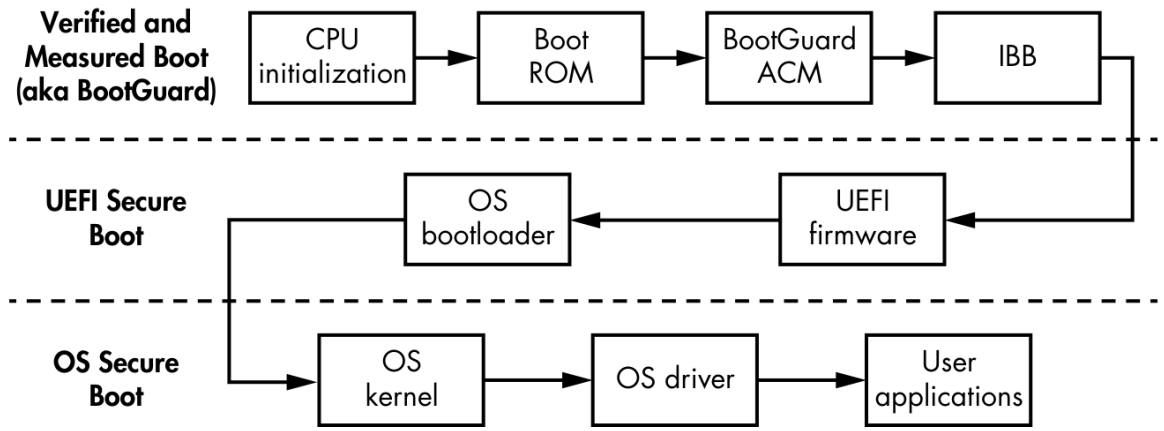


Рисунок 17-16. Границы ответственности реализации Secure Boot

ARM Trusted Boot Board

У ARM имеется собственная реализация технологии Verified and Measured Boot, называемая Trusted Boot Board (TBB), или просто Trusted Boot. В этом разделе мы рассмотрим архитектуру Trusted Boot. ARM имеет особую организацию, известную как технология безопасности TrustZone, которая разделяет среду выполнения на две части. Прежде чем перейти к процессу Verified and Measured Boot в ARM, необходимо описать принцип работы TrustZone.

ARM TrustZone

Технология безопасности TrustZone является аппаратно реализованным средством безопасности, разделяющим среду выполнения ARM на два мира: безопасный и обычный, или небезопасный, которые сосуществуют на одном физическом ядре, как показано на рисунке 17-17. Логика, реализованная в оборудовании и прошивке процессора, обеспечивает надлежащую изоляцию и защиту ресурсов безопасного мира от программного обеспечения, работающего в небезопасном мире.

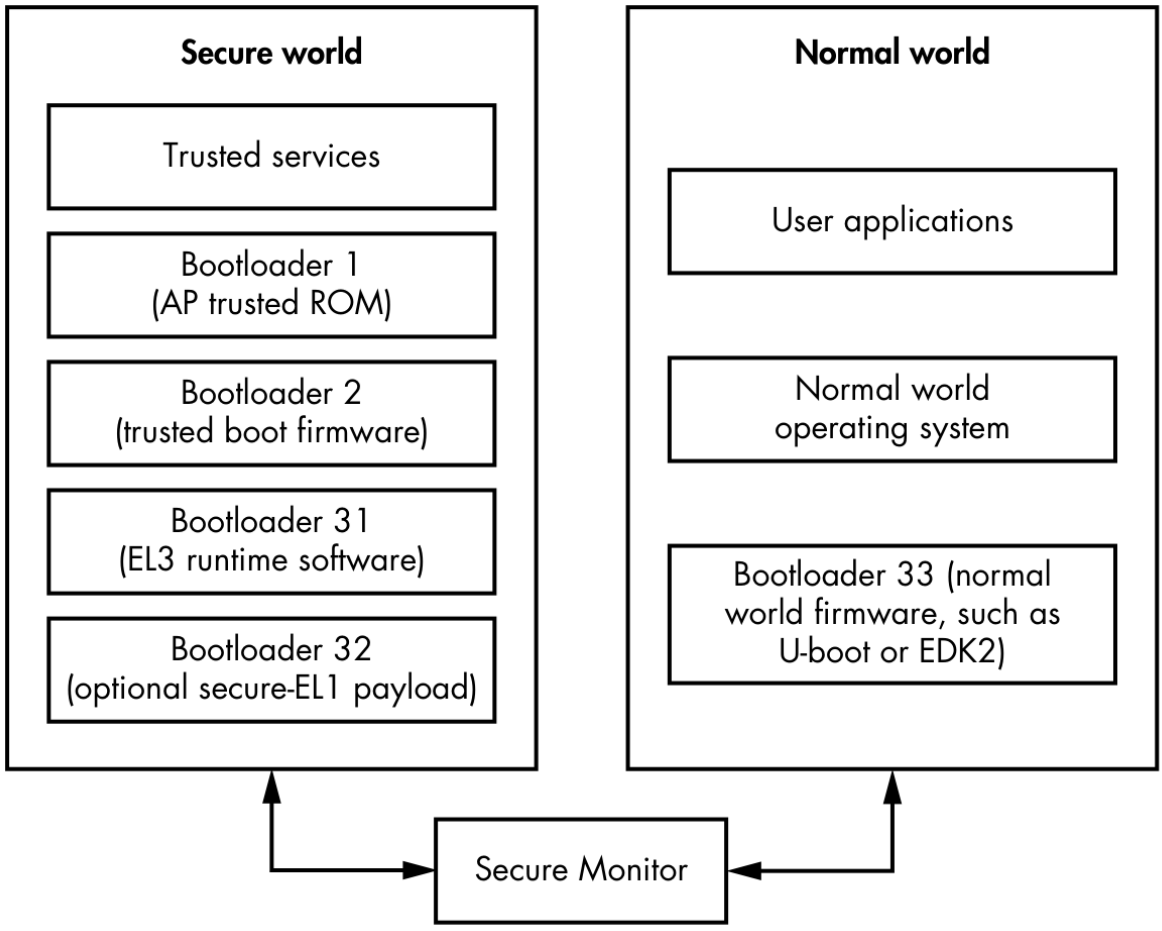


Рисунок 17-17. ARM TrustZone

Оба мира имеют собственные выделенные и разные стеки прошивки и программного обеспечения: обычный мир выполняет пользовательские приложения и ОС, а безопасный мир - безопасную ОС и доверенные службы. Прошивка этих миров состоит из разных загрузчиков, отвечающих за инициализацию мира и загрузку ОС, которые мы вскоре рассмотрим. Поэтому безопасный и обычный миры имеют разные образы прошивки.

Внутри процессора программное обеспечение, работающее в обычном мире, не может напрямую обращаться к коду и данным безопасного мира. Логика управления доступом, препятствующая этому, реализована в оборудовании, обычно в аппаратной части системы на кристалле. Однако программное обеспечение обычного мира может передавать управление программному обеспечению безопасного мира, например для выполнения доверенной службы, с помощью специального программного обеспечения Secure Monitor в ARM Cortex-A или логики ядра в ARM Cortex-M. Этот механизм гарантирует, что переключение между мирами не нарушает безопасность системы.

Вместе технологии Trusted Boot и TrustZone создают доверенную среду выполнения (Trusted Execution Environment), используемую для запуска программного обеспечения с высокими привилегиями и предоставления среды для таких технологий безопасности, как управление цифровыми правами, примитивы криптографии и аутентификации и другие приложения, критичные для безопасности. Таким образом, наиболее конфиденциальное программное обеспечение может размещаться в изолированной защищённой среде.

Загрузчики ARM

Поскольку безопасный и обычный миры разделены, каждому требуется собственный набор загрузчиков. Кроме того, процесс загрузки каждого мира состоит из нескольких этапов, поэтому на разных этапах процесса должны выполняться несколько загрузчиков. Здесь мы в общих чертах опишем поток Trusted Boot для прикладных процессоров ARM, начав со следующего списка загрузчиков, участвующих в Trusted Boot. Они уже были показаны на рисунке 17-17.

BL1. Загрузчик первого этапа, расположенный в загрузочной ROM и выполняемый в безопасном мире.

BL2. Загрузчик второго этапа, расположенный во flash-памяти, загружаемый и выполняемый BL1 в безопасном мире.

BL31. Прошивка времени выполнения безопасного мира, загружаемая и выполняемая BL2.

BL32. Необязательный загрузчик третьего этапа безопасного мира, загружаемый BL2.

BL33. Прошивка времени выполнения обычного мира, загружаемая и выполняемая BL2.

Этот список не является полным и совершенно точным для всех реальных реализаций ARM, поскольку некоторые производители добавляют дополнительные загрузчики или удаляют существующие. В некоторых случаях BL1 может быть не самым первым кодом, выполняемым прикладным процессором при выходе системы из сброса.

Для проверки целостности этих загрузочных компонентов Trusted Boot использует сертификаты открытых ключей X.509; напомним, что файлы в базе данных `db` UEFI Secure Boot были закодированы в X.509. Стоит отметить, что все сертификаты самоподписанные. Центр сертификации не нужен, поскольку цепочка доверия устанавливается не действительностью издателя сертификата, а содержимым расширений сертификата.

Trusted Boot использует два вида сертификатов: сертификаты ключей и содержимого. Сначала сертификаты ключей используются для проверки открытых ключей, которыми подписываются сертификаты содержимого. Затем сертификаты содержимого используются для хранения хешей образов загрузчиков. Эта связь показана на рисунке 17-18.

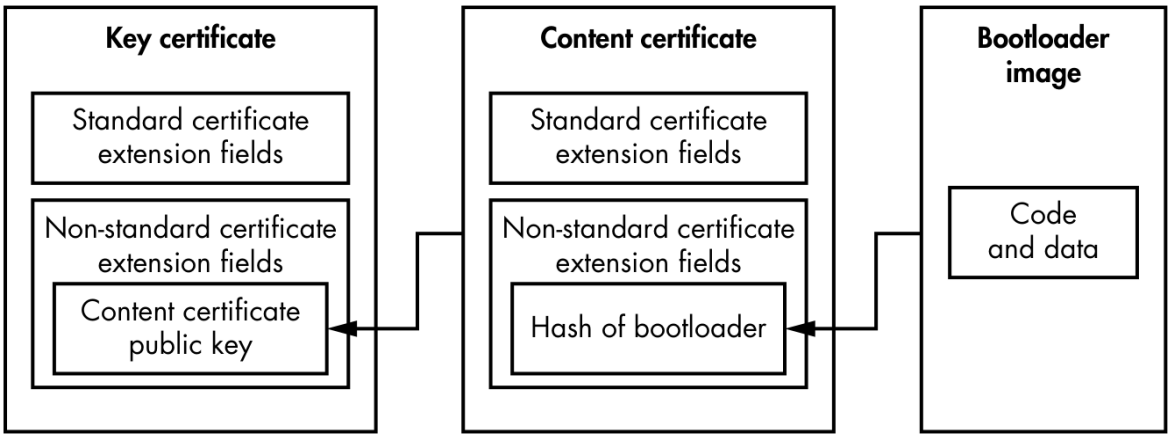


Рисунок 17-18. Сертификаты ключей и содержимого Trusted Boot

Trusted Boot аутентифицирует образ, вычисляя его хеш и сравнивая результат с хешем, извлечённым из сертификата содержимого.

Поток Trusted Boot

Теперь, познакомившись с основными концепциями Trusted Boot, рассмотрим поток Trusted Boot для прикладного процессора, показанный на рисунке 17-19. Это даст полную картину реализации Verified Boot в процессорах ARM и защиты платформ от выполнения недоверенного кода, включая руткиты прошивки.

На рисунке 17-19 сплошные стрелки обозначают передачу потока выполнения, а пунктирные - отношения доверия; иными словами, каждый элемент доверяет элементу, на который указывает его пунктирная стрелка.

После выхода процессора из сброса первым выполняется код загрузчика 1 (BL1) ❶. BL1 загружается из доступной только для чтения загрузочной ROM, поэтому хранящийся там код нельзя изменить. BL1 считывает сертификат содержимого загрузчика 2 (BL2) ❾ из flash-памяти и проверяет ключ его издателя. Затем BL1 вычисляет хеш издателя сертификата содержимого BL2 и сравнивает его с «эталонными» значениями, хранящимися в аппаратном регистре открытого ключа безопасного корня доверия (secure root of trust public key register, ROTPK) ❿. Регистр ROTPK и загрузочная ROM являются корнями доверия Trusted Boot, закреплёнными в оборудовании. Если хеши не равны или проверка подписи сертификата содержимого BL2 не проходит, система аварийно останавливается.

После проверки сертификата содержимого BL2 по ROTPK загрузчик BL1 загружает образ BL2 из flash-памяти ❷, вычисляет его криптографический хеш и сравнивает это значение с полученным из сертификата содержимого BL2 ❺.

После аутентификации BL1 передаёт управление BL2, который, в свою очередь, считывает сертификат доверенного ключа ❻ из flash-памяти. Этот сертификат содержит открытые ключи для проверки прошивки как безопасного мира ❼, так и обычного мира ❽. Ключ, выпустивший сертификат доверенного ключа, сверяется с регистром ROTPK ❿.

Затем BL2 аутентифицирует BL31 ❸, прошивку времени выполнения безопасного мира. Для аутентификации образа BL31 загрузчик BL2 использует сертификаты ключа и содержимого BL31 ❹. BL2 проверяет эти сертификаты, используя открытый ключ безопасного мира, полученный из сертификата доверенного ключа. Сертификат ключа BL31 содержит открытый ключ сертификата содержимого BL31, используемый для проверки подписи сертификата содержимого BL32.

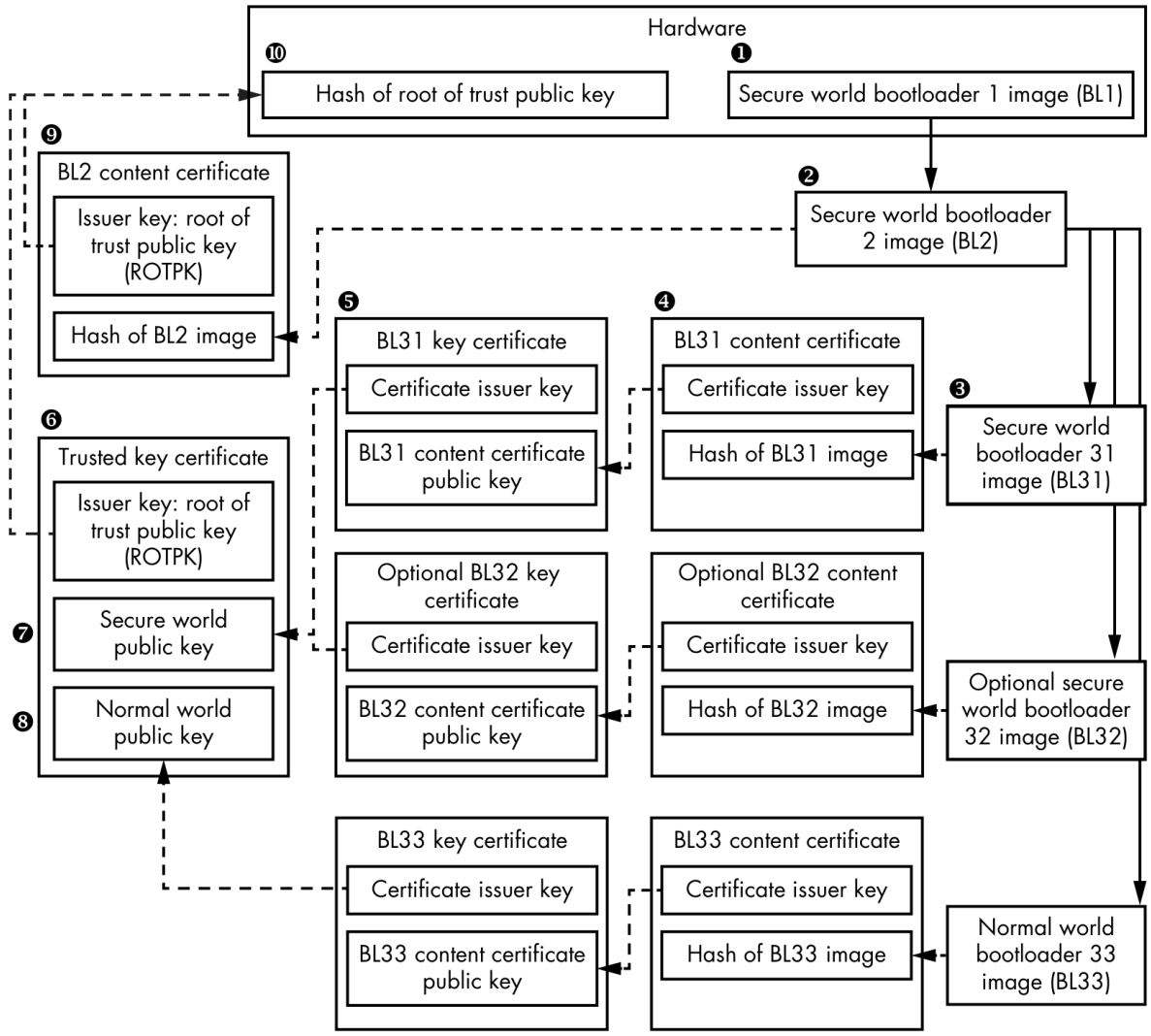


Рисунок 17-19. Поток Trusted Boot

После проверки сертификата содержимого BL31 значение хеша образа BL31, хранящееся в этом сертификате BL31, используется для проверки целостности образа BL31. И снова любая неудача приводит к аварийной остановке системы.

Подобным образом BL2 проверяет целостность необязательного образа BL32 безопасного мира с помощью сертификатов ключа и содержимого BL32.

Целостность образа прошивки BL33, выполняемого в обычном мире, проверяется с помощью сертификатов ключа и содержимого BL33. Сертификат ключа BL33 проверяется открытым ключом обычного мира, полученным из сертификата доверенного ключа.

Если все проверки проходят успешно, система продолжает работу, выполняя аутентифицированную прошивку как безопасного, так и обычного мира.

AMD Hardware Validated Boot

Хотя в этой главе она не рассматривается, у AMD имеется собственная реализация Verified and Measured Boot под названием Hardware Validated Boot (HVB). Эта технология реализует функциональность, сходную с Intel BootGuard. Она основана на технологии AMD Platform Security Processor и имеет выделенный для вычислений, связанных с безопасностью, микроконтроллер, работающий независимо от основного ядра системы.

Verified Boot против руткитов прошивки

Теперь, обладая всеми этими знаниями, наконец выясним, может ли Verified Boot защищать от руткитов прошивки.

Мы знаем, что Verified Boot выполняется до запуска любой прошивки в процессе загрузки. Это означает, что когда Verified Boot начинает проверять прошивку, заражающий её руткит ещё не активен и потому вредоносное ПО не может противодействовать процессу проверки. Verified Boot обнаружит любую вредоносную модификацию прошивки и предотвратит её выполнение.

Кроме того, корень доверия Verified Boot закреплён в оборудовании, поэтому злоумышленники не могут вмешаться в него. Корневой открытый ключ OEM Intel BootGuard записан в набор микросхем, а ключ корня доверия ARM хранится в защищённых регистрах. В обоих случаях загрузочный код, запускающий Verified Boot, загружается из памяти только для чтения, поэтому вредоносное ПО не может исправить или изменить его.

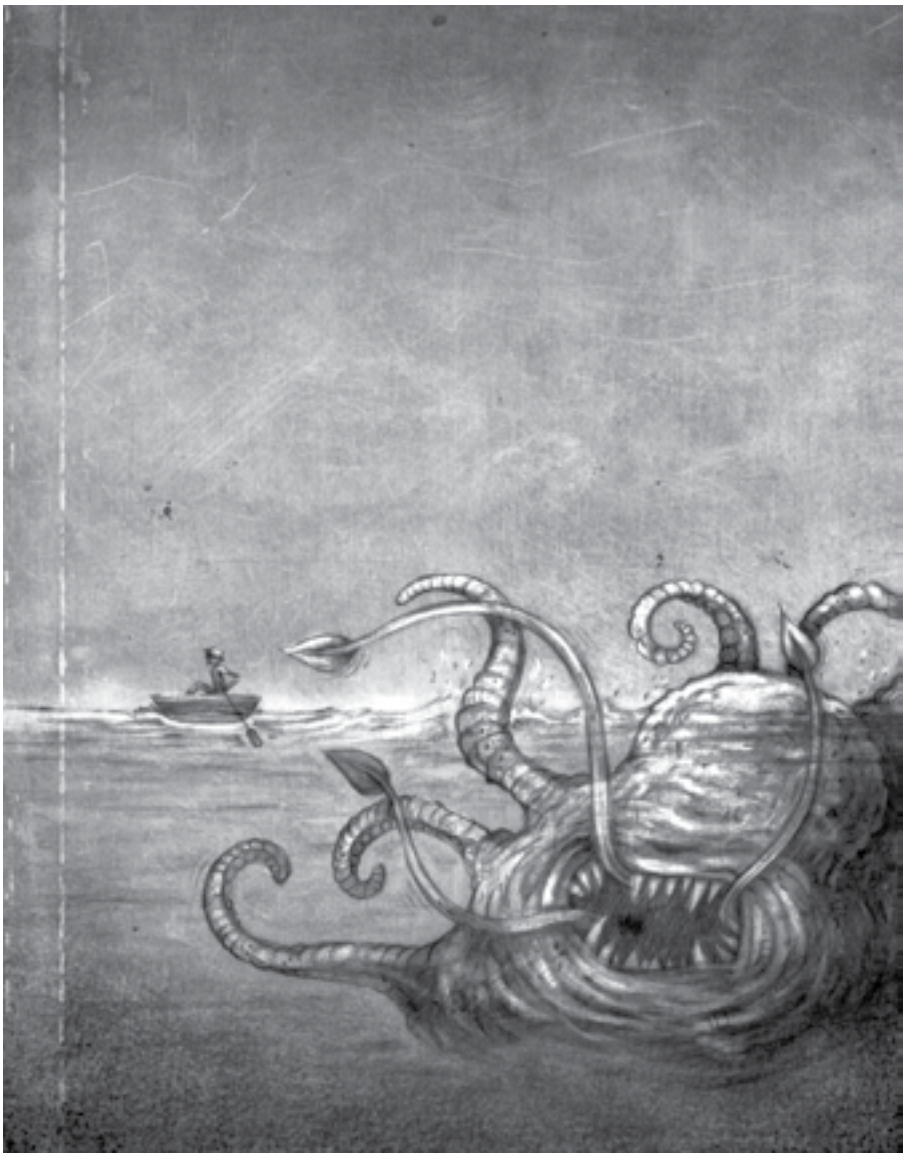
Таким образом, можно заключить, что Verified Boot способна противостоять атакам руткитов прошивки. Однако, как вы могли заметить, вся технология довольно сложна и имеет множество зависимостей, поэтому её легко реализовать неправильно. Технология безопасна лишь настолько, насколько безопасен её самый слабый компонент; одного изъяна в цепочке доверия достаточно для обхода. Это означает, что злоумышленники с большой вероятностью смогут найти в реализации Verified Boot уязвимости, пригодные для эксплуатации и установки руткитов прошивки.

Заключение

В этой главе мы исследовали три технологии Secure Boot: UEFI Secure Boot, Intel BootGuard и ARM Trusted Boot. Эти технологии опираются на цепочку доверия, обеспечиваемую с самого начала процесса загрузки до выполнения пользовательских приложений, и включают огромное число загрузочных модулей. При правильной настройке и реализации они защищают от постоянно растущего числа руткитов прошивки UEFI.

Именно поэтому системы с высокими требованиями к надёжности должны использовать Secure Boot, а многие потребительские системы в наши дни включают её по умолчанию. В следующей главе мы сосредоточимся на криминалистических подходах к анализу руткитов прошивки.

Глава 18. Подходы к анализу скрытых файловых систем



До сих пор в этой книге вы узнавали, как буткиты проникают на компьютер жертвы и сохраняются на нём, применяя сложные методы уклонения от обнаружения. Одной из общих особенностей этих передовых угроз является использование собственной скрытой системы хранения для модулей и конфигурационных сведений на скомпрометированной машине.

Многие скрытые файловые системы вредоносного ПО являются собственными или изменёнными версиями стандартных файловых систем, а значит, для криминалистического анализа компьютера, скомпрометированного руткитом или буткитом, часто требуется собственный набор инструментов. Чтобы разработать такие инструменты, исследователи должны изучить структуру скрытой файловой системы и алгоритмы шифрования данных посредством глубокого анализа и обратного проектирования.

В этой главе мы подробнее рассмотрим скрытые файловые системы и методы их анализа. Мы поделимся опытом длительного криминалистического анализа руткитов и буткитов, описанных в этой книге. Мы также обсудим подходы к извлечению данных из скрытого хранилища и поделимся решениями распространённых проблем, возникающих при таком анализе. Наконец, представим

разработанный нами инструмент HiddenFsReader, предназначенный для выгрузки содержимого скрытых файловых систем конкретных вредоносных программ.

Обзор скрытых файловых систем

На рисунке 18-1 приведён обзор типичной скрытой файловой системы. Видно, как вредоносная полезная нагрузка, взаимодействующая со скрытым хранилищем, внедряется в адресное пространство пользовательского режима процесса жертвы. Полезная нагрузка часто использует скрытое хранилище для чтения и обновления конфигурационных сведений либо хранения таких данных, как похищенные учётные данные.

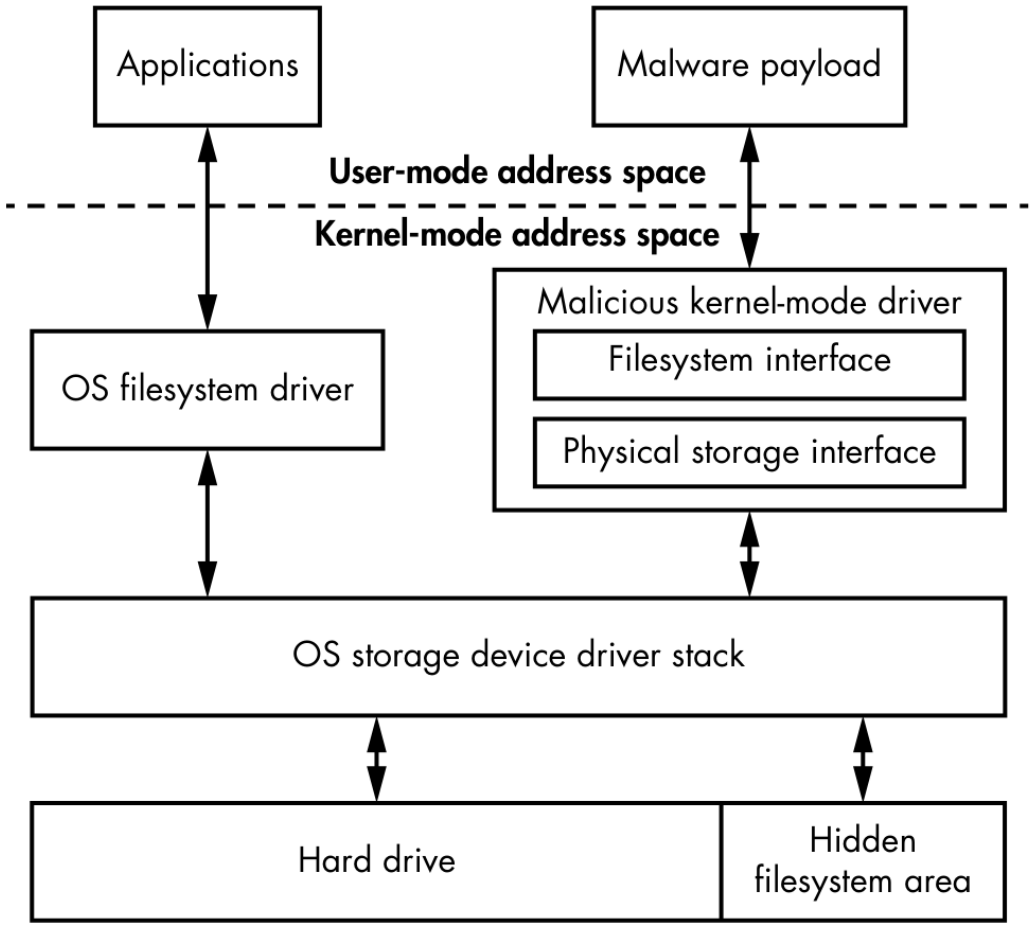


Рисунок 18-1. Типичная реализация вредоносной скрытой файловой системы

Служба скрытого хранилища предоставляется через модуль режима ядра, а открываемый вредоносной программой интерфейс виден только модулю полезной нагрузки. Обычно этот интерфейс недоступен другому программному обеспечению системы, и к нему нельзя обратиться стандартными способами, например через Проводник Windows.

Данные, сохранённые вредоносной программой в скрытой файловой системе, постоянно находятся в области жёсткого диска, которую не использует ОС, чтобы не создавать конфликтов. В большинстве случаев эта область расположена в конце жёсткого диска, где обычно имеется нераспределённое пространство. Однако иногда, как в случае буткита Rovnix, рассмотренного в главе 11, вредоносная программа хранит скрытую файловую систему в нераспределённом пространстве в начале жёсткого диска.

Основная цель любого исследователя, проводящего криминалистический анализ, состоит в извлечении этих скрытых сохранённых данных, поэтому далее мы обсудим несколько подходов к этой задаче.

Извлечение данных буткита из скрытой файловой системы

Криминалистические сведения с заражённого буткитом компьютера можно получить, извлекая данные, когда заражённая система выключена, либо считывая вредоносные данные из работающей заражённой системы.

У каждого подхода есть свои преимущества и недостатки, которые мы рассмотрим при обсуждении двух методов.

Извлечение данных из автономной системы

Начнём с получения данных с жёсткого диска, когда система отключена, то есть вредоносная программа неактивна. Этого можно добиться посредством автономного анализа жёсткого диска, но другой вариант состоит в загрузке незаражённого экземпляра операционной системы с live CD. Это гарантирует использование компьютером нескомпрометированного загрузчика, установленного на live CD, поэтому буткит не будет выполнен. Такой подход предполагает, что буткит не способен выполняться до легитимного загрузчика и не может обнаружить попытку загрузки с внешнего устройства, чтобы заранее стереть конфиденциальные данные.

Существенное преимущество этого метода перед анализом работающей системы состоит в отсутствии необходимости преодолевать механизмы самозащиты вредоносной программы, защищающие содержимое скрытого хранилища. Как будет показано в последующих разделах, обход защиты вредоносной программы является нетривиальной задачей и требует определённой квалификации.

> **Примечание.** Получив доступ к данным на жёстком диске, можно перейти к выгрузке образа вредоносной скрытой файловой системы, его расшифровке и разбору. Разные виды вредоносного ПО требуют разных подходов к расшифровке и разбору скрытых файловых систем, которые будут рассмотрены в разделе «Разбор образа скрытой файловой системы» на странице 360.

Однако недостаток этого метода состоит в необходимости как физического доступа к скомпрометированному компьютеру, так и технических знаний для загрузки компьютера с live CD и выгрузки скрытой файловой системы. Выполнить оба требования может быть проблематично.

Если анализ на неактивной машине невозможен, приходится использовать активный подход.

Чтение данных в работающей системе

В работающей системе с активным экземпляром буткита необходимо выгрузить содержимое вредоносной скрытой файловой системы.

Однако чтение вредоносного скрытого хранилища в системе с активно работающей вредоносной программой сопряжено с одной серьёзной трудностью: вредоносная программа может противодействовать попыткам чтения и подделывать данные, считываемые с жёсткого диска, чтобы мешать криминалистическому анализу. Большинство рассмотренных в книге руткитов,

включая TDL3, TDL4, Rovnix и Olmasco, отслеживают доступ к жёсткому диску и блокируют обращение к областям с вредоносными данными.

Чтобы прочитать вредоносные данные с жёсткого диска, необходимо преодолеть механизмы самозащиты вредоносной программы. Вскоре мы рассмотрим несколько подходов, но сначала исследуем стек драйверов устройств хранения в Windows и способ подключения к нему вредоносной программы, чтобы лучше понять, как она защищает вредоносные данные. Эти сведения также полезны для понимания некоторых подходов к удалению вредоносных перехватчиков.

Перехват минипортового драйвера хранилища

Архитектуру стека драйверов устройств хранения Microsoft Windows и способ его перехвата вредоносной программой мы затрагивали в главе 1. Этот метод пережил TDL3 и был заимствован последующими вредоносными программами, в том числе изученными в этой книге буткитами. Здесь мы рассмотрим его подробнее.

TDL3 перехватывал минипортовый драйвер хранилища, расположенный в самом низу стека драйверов устройств хранения, как показано на рисунке 18-2.

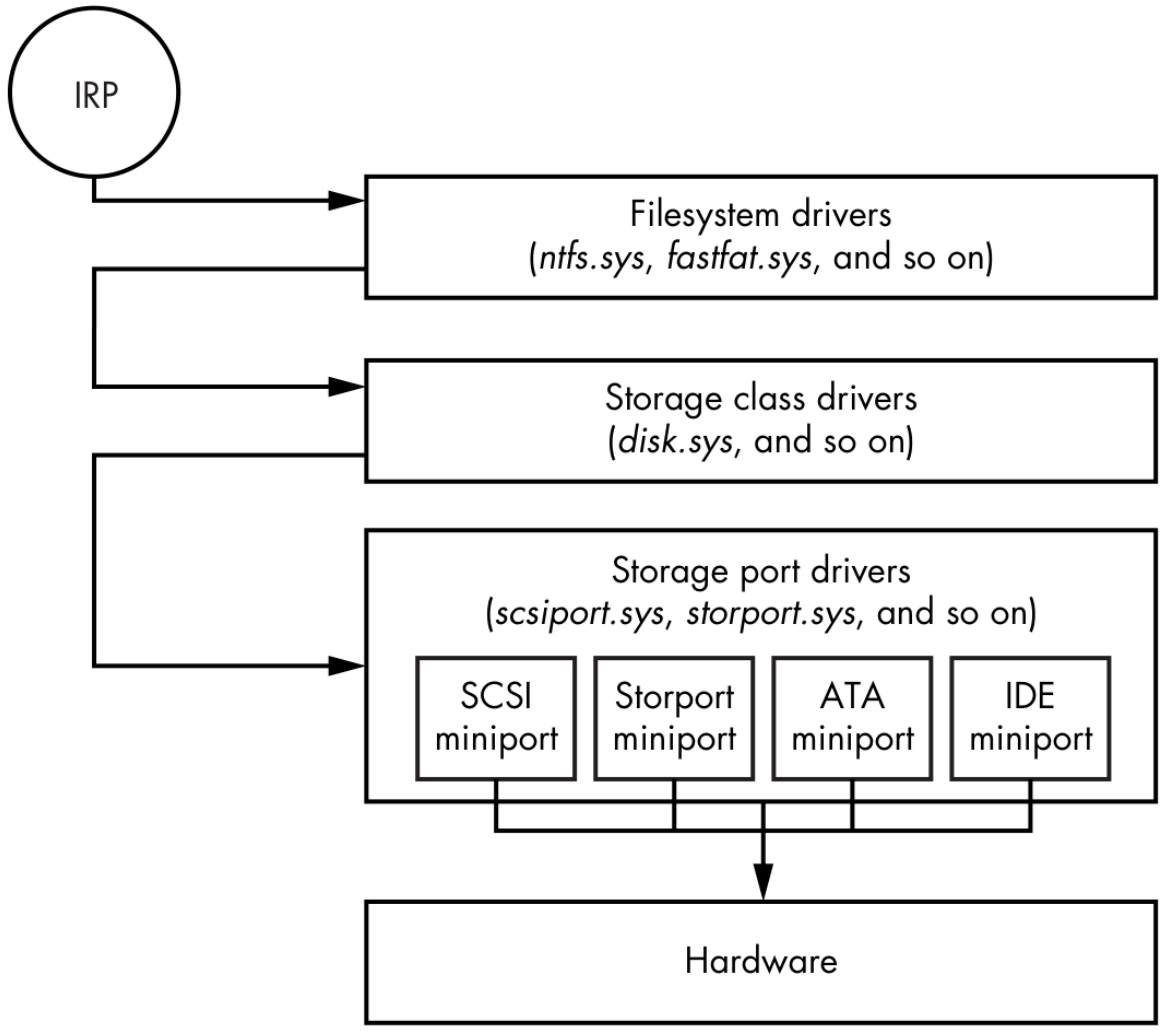


Рисунок 18-2. Стек драйверов устройств хранения

Перехват стека драйверов на этом уровне позволяет вредоносной программе отслеживать и изменять запросы ввода-вывода, направляемые к жёсткому диску и поступающие от него, предоставляя доступ к скрытому хранилищу.

Перехват в самом низу стека драйверов и непосредственное взаимодействие с оборудованием также позволяют вредоносной программе обходить программное обеспечение безопасности, работающее на уровне файловой системы или драйвера класса диска. Как было кратко рассмотрено в главе 1, при выполнении операции ввода-вывода на жёстком диске ОС создаёт пакет запроса ввода-вывода (input/output request packet, IRP), специальную структуру данных в ядре операционной системы, описывающую операцию ввода-вывода, которая проходит через весь стек устройств сверху вниз.

Модули программного обеспечения безопасности, отвечающие за отслеживание операций ввода-вывода жёсткого диска, могут исследовать и изменять пакеты IRP, но, поскольку вредоносные перехватчики установлены на уровне ниже программного обеспечения безопасности, они невидимы этим защитным инструментам.

Существует несколько других уровней, которые может перехватывать буткит, например API пользовательского режима, драйвер файловой системы и драйвер класса диска, но ни один из них не позволяет вредоносной программе быть столь же скрытной и мощной, как уровень минипорта хранилища.

Структура стека устройств хранения

В этом разделе мы не будем рассматривать все возможные методы перехвата минипорта хранилища. Вместо этого сосредоточимся на наиболее распространённых подходах, встречавшихся нам при анализе вредоносного ПО.

Сначала подробнее рассмотрим устройство хранения, показанное на рисунке 18-3.

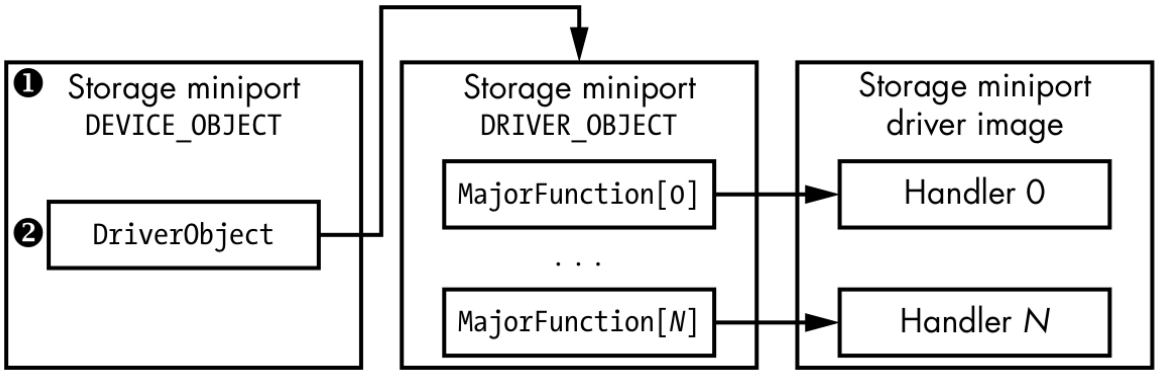


Рисунок 18-3. Организация минипортового устройства хранения

IRP проходит от вершины стека к его основанию. Каждое устройство в стеке может либо обработать и завершить запрос ввода-вывода, либо переслать его устройству на один уровень ниже.

`DEVICE_OBJECT` ❶ является системной структурой данных, используемой операционной системой для описания устройства в стеке, и содержит указатель ❷ на соответствующий `DRIVER_OBJECT`, другую системную структуру данных, описывающую загруженный в систему драйвер. В данном случае `DEVICE_OBJECT` содержит указатель на минипортовый драйвер хранилища.

Структура DRIVER_OBJECT показана в листинге 18-1.

```
typedef struct _DRIVER_OBJECT {
   SHORT Type;
   SHORT Size;
   ❶ PDEVICE_OBJECT DeviceObject;
   ULONG Flags;
   ❷ PVOID DriverStart;
   ❸ ULONG DriverSize;
   PVOID DriverSection;
   PDRIVER_EXTENSION DriverExtension;
   ❹ UNICODE_STRING DriverName;
   PUNICODE_STRING HardwareDatabase;
   PFAST_IO_DISPATCH FastIoDispatch;
   ❺ LONG * DriverInit;
   PVOID DriverStartIo;
   PVOID DriverUnload;

   ❻ LONG * MajorFunction[28];
} DRIVER_OBJECT, *PDRIVER_OBJECT;
```

Листинг 18-1. Структура DRIVER_OBJECT

Поле DriverName ❹ содержит имя драйвера, описываемого структурой; DriverStart ❷ и DriverSize ❸ содержат соответственно начальный адрес и размер драйвера в памяти; DriverInit ❺ содержит указатель на процедуру инициализации драйвера; DeviceObject ❶ содержит указатель на список структур DEVICE_OBJECT, относящихся к драйверу. С точки зрения вредоносной программы наиболее важным является поле MajorFunction ❻, расположенное в конце структуры и содержащее адреса реализованных в драйвере обработчиков различных операций ввода-вывода.

Когда пакет ввода-вывода поступает объекту устройства, операционная система проверяет поле DriverObject соответствующей структуры DEVICE_OBJECT, чтобы получить адрес DRIVER_OBJECT в памяти. После получения ядром структуры DRIVER_OBJECT оно извлекает из массива MajorFunction адрес соответствующего обработчика ввода-вывода для нужного вида операции. Обладая этими сведениями, можно определить части стека устройств хранения, которые вредоносная программа способна перехватить. Рассмотрим пару разных методов.

Прямое исправление образа минипортового драйвера хранилища

Один из способов перехвата минипортового драйвера хранилища состоит в непосредственной модификации образа драйвера в памяти. Получив адрес объекта минипортового устройства жёсткого диска, вредоносная программа обращается к DriverObject, чтобы найти соответствующую структуру DRIVER_OBJECT. Затем она извлекает адрес обработчика ввода-вывода жёсткого диска из массива MajorFunction и исправляет код по этому адресу, как показано на рисунке 18-4; серым цветом выделены разделы, изменённые вредоносной программой.

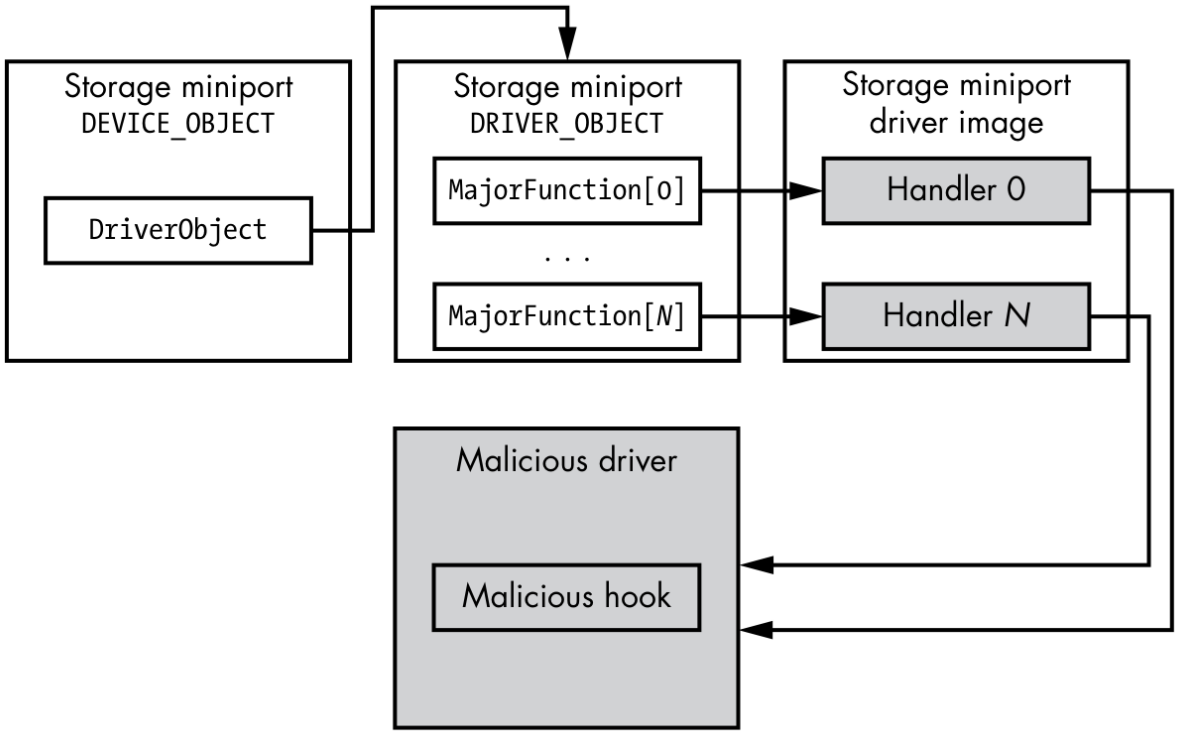


Рисунок 18-4. Перехват стека драйверов хранилища путём исправления минипортового драйвера

Когда объект устройства получает запрос ввода-вывода, выполняется вредоносная программа. Теперь вредоносный перехватчик может отклонять операции ввода-вывода, чтобы блокировать доступ к защищённой области жёсткого диска, либо изменять запросы ввода-вывода, чтобы возвращать поддельные данные и вводить защитное программное обеспечение в заблуждение.

Например, перехватчик этого типа используется буткитом Gapz, рассмотренным в главе 12. В случае Gapz вредоносная программа перехватывает две процедуры минипортового драйвера жёсткого диска, отвечающие за обработку запросов ввода-вывода `IRP_MJ_INTERNAL_DEVICE_CONTROL` и `IRP_MJ_DEVICE_CONTROL`, чтобы защитить их от чтения или перезаписи.

Однако этот подход не особенно скрытен. Защитное программное обеспечение может обнаружить и удалить перехватчики, найдя образ перехваченного драйвера в файловой системе и отобразив его в память. Затем оно сравнивает разделы кода драйвера, загруженного в ядро, с версией драйвера, вручную загруженной из файла, и отмечает все различия в разделах кода, которые могут указывать на наличие вредоносных перехватчиков в драйвере.

После этого защитное программное обеспечение может удалить вредоносные перехватчики и восстановить исходный код, перезаписав изменённый код кодом, взятым из файла. Этот метод предполагает, что драйвер в файловой системе является подлинным и не был изменён вредоносной программой.

Модификация DRIVER_OBJECT

Минипортовый драйвер жёсткого диска также можно перехватить посредством модификации структуры DRIVER_OBJECT. Как уже упоминалось, эта структура данных содержит местоположение образа драйвера в памяти и адреса диспетчерских процедур драйвера в массиве MajorFunction.

Следовательно, модификация массива MajorFunction позволяет вредоносной программе устанавливать свои перехватчики, не затрагивая образ драйвера в памяти. Например, вместо непосредственного исправления кода в образе, как в предыдущем методе, вредоносная программа может заменить элементы массива MajorFunction, относящиеся к запросам ввода-вывода IRP_MJ_INTERNAL_DEVICE_CONTROL и IRP_MJ_DEVICE_CONTROL, адресами вредоносных перехватчиков. В результате ядро операционной системы будет перенаправлено к вредоносному коду всякий раз, когда попытается разрешить адреса обработчиков в структуре DRIVER_OBJECT. Этот подход показан на рисунке 18-5.

Поскольку образ драйвера в памяти остаётся неизменным, этот подход более скрытен, чем предыдущий метод, однако и он не защищён от обнаружения. Защитное программное обеспечение всё равно может выявить перехватчики, найдя образ драйвера в памяти и проверив адреса обработчиков запросов ввода-вывода IRP_MJ_INTERNAL_DEVICE_CONTROL и IRP_MJ_DEVICE_CONTROL: если эти адреса не принадлежат диапазону адресов образа минипортового драйвера в памяти, это указывает на наличие перехватчиков в стеке устройств.

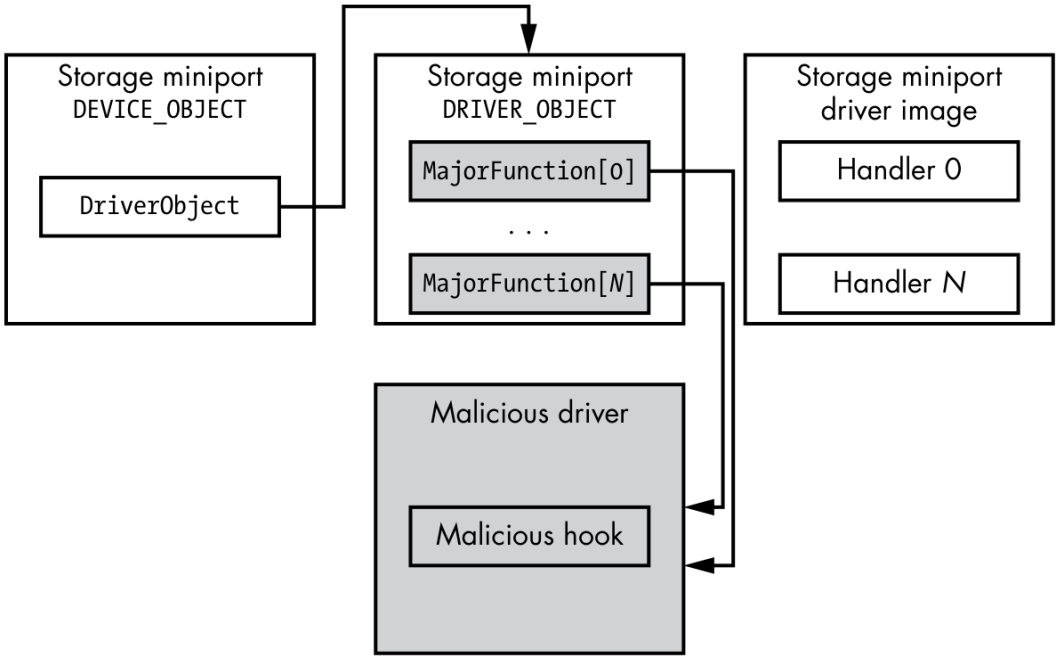


Рисунок 18-5. Перехват стека драйверов хранилища путём исправления DRIVER_OBJECT *минипорта*

С другой стороны, удалить эти перехватчики и восстановить исходные значения массива MajorFunction гораздо сложнее, чем при предыдущем методе. При таком подходе массив MajorFunction инициализируется самим драйвером во время выполнения его процедуры инициализации, которая получает в качестве входного параметра указатель на частично инициализированную соответствующую структуру DRIVER_OBJECT и завершает инициализацию, заполняя массив MajorFunction указателями на диспетчерские обработчики.

Адреса обработчиков известны только минипортовому драйверу. Защитное программное обеспечение ничего о них не знает, что значительно усложняет восстановление исходных адресов в структуре `DRIVER_OBJECT`.

Один из подходов, который защитное программное обеспечение может использовать для восстановления исходных данных, состоит в загрузке образа минипортового драйвера в эмулируемую среду, создании структуры `DRIVER_OBJECT` и выполнении точки входа драйвера (процедуры инициализации) со структурой `DRIVER_OBJECT`, переданной в качестве параметра. После выхода из процедуры инициализации структура `DRIVER_OBJECT` должна содержать корректные обработчики `MajorFunction`, и защитное программное обеспечение сможет воспользоваться этой информацией, чтобы вычислить адреса диспетчерских процедур ввода-вывода в образе драйвера и восстановить изменённую структуру `DRIVER_OBJECT`.

Однако эмуляция драйвера может оказаться непростой. Если процедура инициализации драйвера реализует простую функциональность (например, инициализирует структуру `DRIVER_OBJECT` корректными адресами обработчиков), этот подход сработает, но если она реализует сложную функциональность (например, вызывает системные службы или системный API, которые труднее эмулировать), эмуляция может завершиться с ошибкой до того, как драйвер инициализирует структуру данных. В таких случаях защитное программное обеспечение не сможет восстановить адреса исходных обработчиков и удалить вредоносные перехватчики.

Другой подход к этой проблеме состоит в создании базы данных исходных адресов обработчиков и её использовании для их восстановления. Однако этому решению не хватает универсальности. Оно может хорошо работать с наиболее часто используемыми минипортовыми драйверами, но давать сбой в случае редких или нестандартных драйверов, не включённых в базу данных.

Модификация DEVICE_OBJECT

Последний подход к перехвату минипортового драйвера, который мы рассмотрим в этой главе, представляет собой логическое продолжение предыдущего метода. Мы знаем, что для выполнения обработчика запроса ввода-вывода в минипортовом драйвере ядро ОС должно извлечь адрес структуры `DRIVER_OBJECT` из минипортового объекта `DEVICE_OBJECT`, затем извлечь адрес обработчика из массива `MajorFunction` и, наконец, выполнить обработчик.

Поэтому ещё один способ установки перехватчика состоит в модификации поля DriverObject в соответствующем DEVICE_OBJECT. Вредоносная программа должна создать поддельную структуру DRIVER_OBJECT и инициализировать её массив MajorFunction адресами вредоносных перехватчиков, после чего ядро операционной системы будет использовать вредоносную структуру DRIVER_OBJECT, чтобы получить адрес обработчика запроса ввода-вывода и выполнить вредоносный перехватчик (рисунок 18-6).

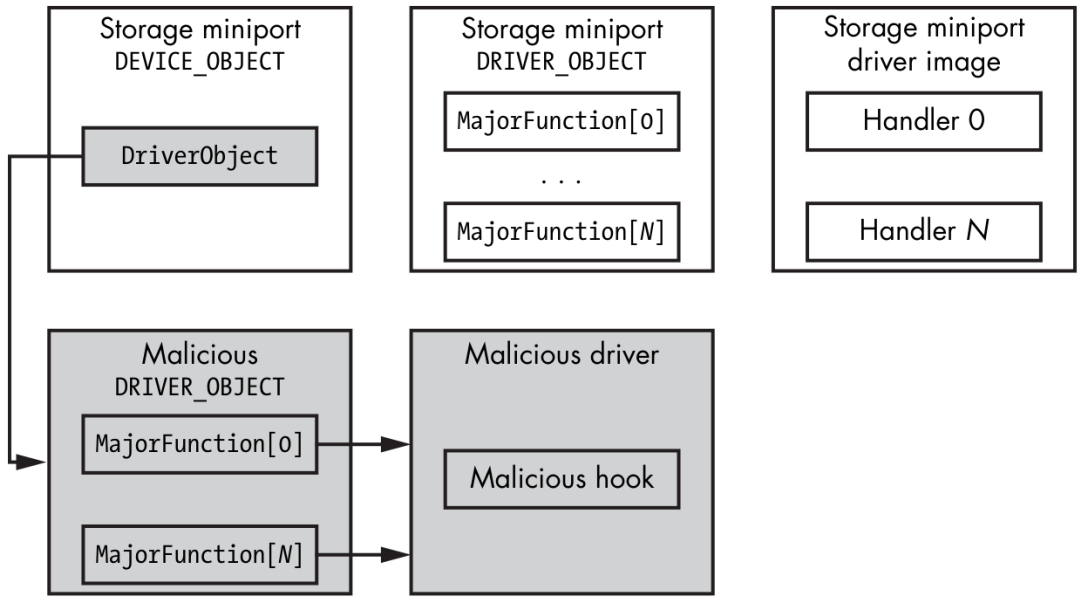


Рисунок 18-6. Перехват стека драйверов хранилища путём подмены DRIVER_OBJECT *минипорта*

Этот подход используется TDL3/TDL4, Rovnix и Olmasco и обладает такими же преимуществами и недостатками, как предыдущий подход. Однако его перехватчики удалить ещё труднее, поскольку отличается вся структура DRIVER_OBJECT, а значит, защитному программному обеспечению придётся приложить дополнительные усилия, чтобы найти исходную структуру DRIVER_OBJECT.

На этом мы завершаем обсуждение методов перехвата стека драйверов устройств. Как мы увидели, простого универсального решения для удаления вредоносных перехватчиков с целью чтения вредоносных данных из защищённых областей жёсткого диска заражённой машины не существует. Ещё одна причина этой сложности состоит в наличии множества различных реализаций минипортовых драйверов хранилища; поскольку они непосредственно взаимодействуют с аппаратным обеспечением, каждый производитель устройств хранения предоставляет для своего оборудования особые драйверы, поэтому подходы, работающие для определённого класса минипортовых драйверов, не сработают для других.

Разбор образа скрытой файловой системы

После деактивации защиты руткита мы можем прочитать данные из вредоносного скрытого хранилища, получив образ вредоносной файловой системы. Следующий логический шаг криминалистического анализа состоит в разборе скрытой файловой системы и извлечении значимой информации.

Чтобы разобрать выгруженную файловую систему, необходимо знать, какому типу вредоносной программы она соответствует. Каждая угроза имеет собственную реализацию скрытого хранилища, и единственный способ восстановить его структуру состоит в исследовании вредоносной программы для понимания кода, отвечающего за его обслуживание. В некоторых случаях структура

скрытого хранилища может меняться от одной версии к другой даже в пределах одного семейства вредоносных программ.

Вредоносная программа также может шифровать или обфусцировать своё скрытое хранилище, чтобы затруднить криминалистический анализ; в этом случае нам потребуется найти ключи шифрования.

В таблице 18-1 приведены сводные сведения о скрытых файловых системах, связанных с семействами вредоносных программ, которые мы рассматривали в предыдущих главах. В этой таблице учитываются только основные характеристики скрытой файловой системы, такие как тип структуры, используемое шифрование и наличие сжатия.

Таблица 18-1. Сравнение реализаций скрытых файловых систем

Функциональность / вредоносная программа	**TDL4**	**Rovnix**	**Olmasco**	**Gapz**
Тип файловой системы	Нестандартная	Модификация FAT16	Нестандартная	Нестандартная
Шифрование	XOR/RC4	Нестандартное (XOR+ROL)	Модификация RC6	RC4
Сжатие	Нет	Да	Нет	Да

Как мы видим, все реализации различаются, создавая трудности для криминалистов и следователей.

Инструмент HiddenFsReader

В ходе исследования сложных вредоносных угроз мы выполнили обратную разработку многих различных семейств вредоносных программ и сумели собрать обширные сведения о разнообразных реализациях скрытых файловых систем, которые могут оказаться весьма полезными для сообщества исследователей безопасности. Поэтому мы реализовали инструмент под названием HiddenFsReader (http://download.eset.com/special/ESETHfsReader.exe/), который автоматически ищет скрытые вредоносные контейнеры на компьютере и извлекает содержащуюся в них информацию. На рисунке 18-7 показана высокоуровневая архитектура HiddenFsReader.

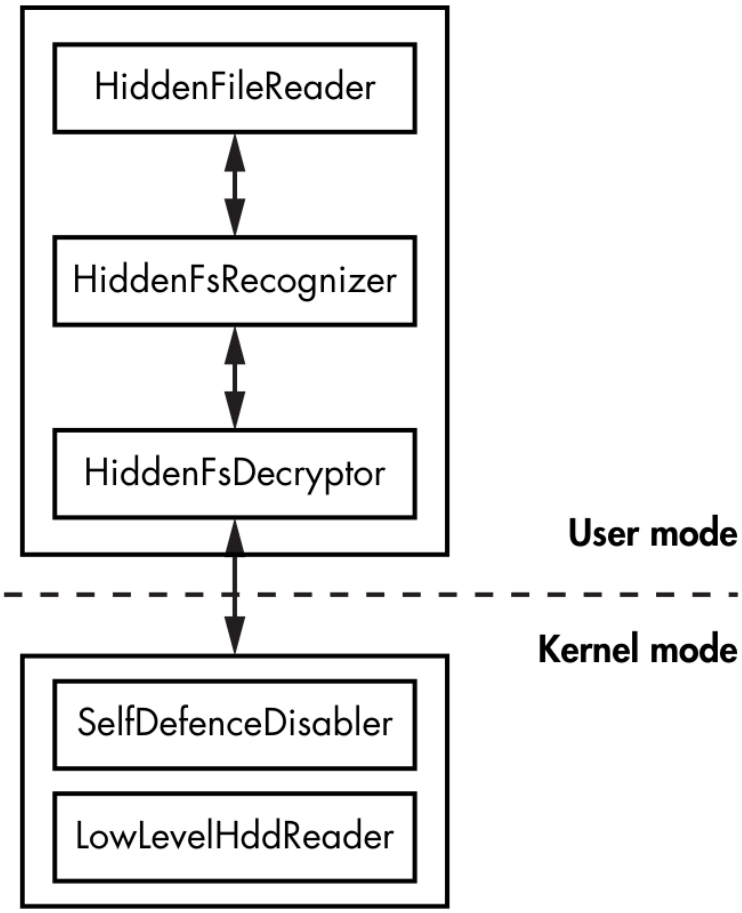


Рисунок 18-7. Высокоуровневая архитектура HiddenFsReader

HiddenFsReader состоит из двух компонентов: приложения пользовательского режима и драйвера режима ядра. Драйвер режима ядра, по существу, реализует функциональность отключения механизмов самозащиты руткита/буткита, а приложение пользовательского режима предоставляет пользователю интерфейс для получения низкоуровневого доступа к жёсткому диску. Приложение использует этот интерфейс для чтения фактических данных с жёсткого диска, даже если система заражена активным экземпляром вредоносной программы.

Само приложение пользовательского режима отвечает за идентификацию скрытых файловых систем, прочитанных с жёсткого диска, а также реализует функцию расшифрования, позволяющую получить открытые данные из зашифрованного скрытого хранилища.

В последней на момент написания книги версии HiddenFsReader поддерживаются следующие угрозы и соответствующие им скрытые файловые системы:

• Win32/Olmarik (TDL3/TDL3+/TDL4)
• Win32/Olmasco (MaxXSS)
• Win32/Sirefef (ZeroAccess)
• Win32/Rovnix
• Win32/Xpaj
• Win32/Gapz
• Win32/Flamer
• Win32/Urelas (GBPBoot)
• Win32/Avatar

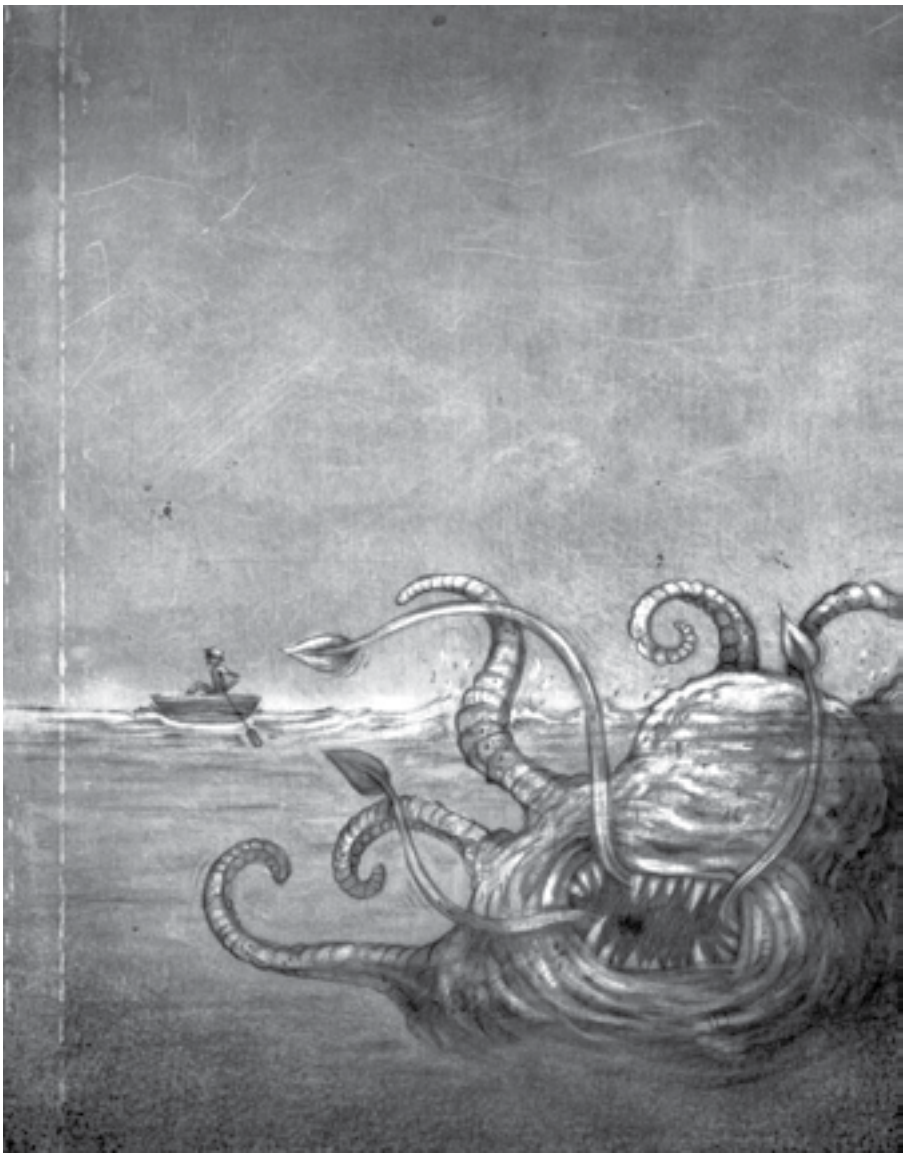
Эти угрозы используют нестандартные скрытые файловые системы для хранения полезной нагрузки и конфигурационных данных, лучше защищаясь от защитного программного обеспечения и затрудняя криминалистический анализ. Не все эти угрозы рассматриваются в данной книге, но сведения о них можно найти в списке ссылок по адресу https://nostarch.com/rootkits/.

Заключение

Реализация нестандартной скрытой файловой системы широко распространена среди сложных угроз, таких как руткиты и буткиты. Скрытое хранилище используется для тайного хранения конфигурационной информации и полезной нагрузки, делая традиционные подходы к криминалистическому анализу неэффективными.

Криминалисты должны отключить механизмы самозащиты угрозы и выполнить обратную разработку вредоносной программы. Таким образом они могут восстановить структуру скрытой файловой системы и определить схему шифрования и ключ, используемые для защиты вредоносных данных. Для каждой угрозы это требует дополнительных времени и усилий, однако в данной главе были рассмотрены некоторые возможные подходы к решению этих проблем. В главе 19 мы продолжим изучать криминалистический анализ вредоносных программ, сосредоточившись именно на UEFI-руткитах. Мы приведём сведения о получении и анализе прошивок UEFI применительно к вредоносным программам, нацеленным на прошивки UEFI.

Глава 19. Криминалистика BIOS/UEFI: получение прошивки и подходы к анализу



Недавнее появление руткитов, нацеленных на прошивки UEFI, вновь пробудило интерес к криминалистическому анализу прошивок UEFI. Утечки секретных сведений о спонсируемых государствами имплантатах BIOS, а также упомянутый в главе 15 взлом Hacking Team продемонстрировали всё более скрытные и мощные возможности вредоносных программ, нацеленных на BIOS, и побудили исследовательское сообщество глубже изучать прошивки. Некоторые технические подробности этих угроз BIOS уже обсуждались в предыдущих главах. Если вы не читали главы 15 и 16, настоятельно рекомендуем сделать это, прежде чем продолжить: в этих главах рассматриваются важные концепции безопасности прошивок, понимание которых предполагается в дальнейшем обсуждении.

> **Примечание.** В этой главе термины BIOS и прошивка UEFI используются как взаимозаменяемые.

Криминалистический анализ прошивок UEFI в настоящее время представляет собой формирующуюся область исследований, поэтому работающим в ней исследователям безопасности не хватает общепринятых инструментов и подходов. В этой главе мы рассмотрим некоторые методы анализа прошивок, включая различные подходы к получению прошивки, а также методы разбора и извлечения полезной информации.

Сначала мы сосредоточимся на получении прошивки, что обычно является первым шагом криминалистического анализа. Мы рассмотрим как программный, так и аппаратный подход к получению образа прошивки UEFI. Затем сравним эти подходы и обсудим преимущества и недостатки каждого. После этого мы рассмотрим внутреннюю структуру образа прошивки UEFI и способы его разбора для извлечения криминалистических артефактов. В рамках этого обсуждения будет показано, как использовать UEFITool, незаменимый инструмент с открытым исходным кодом для просмотра и изменения образов прошивок UEFI. Наконец, мы обсудим Chipsec, инструмент с чрезвычайно обширной и мощной функциональностью, и рассмотрим его применение в криминалистическом анализе. Оба инструмента были представлены в главе 15.

Ограничения наших криминалистических методов

Представленный здесь материал имеет некоторые ограничения. На современных платформах существует множество видов прошивок: прошивка UEFI, прошивка Intel ME, прошивка контроллера жёсткого диска и так далее. Эта глава посвящена именно анализу прошивки UEFI, которая составляет одну из крупнейших частей прошивки платформы.

Следует также учитывать, что прошивки в значительной степени зависят от платформы, то есть каждая платформа обладает собственными особенностями. В этой главе мы сосредоточимся на прошивках UEFI для систем Intel x86, составляющих большинство сегментов рынка настольных компьютеров, ноутбуков и серверов.

Почему криминалистический анализ прошивок важен

В главе 15 мы увидели, что современные прошивки, и в особенности BIOS, представляют собой удобное место для внедрения очень мощных бэкдоров или руткитов. Вредоносная программа такого типа способна пережить переустановку ОС или замену жёсткого диска и предоставляет злоумышленнику контроль над всей платформой. На момент написания книги большинство передовых средств защиты вообще не учитывают угрозы для прошивок UEFI, что делает их ещё опаснее. Это открывает злоумышленнику широкие возможности для внедрения вредоносной программы, которая незаметно сохраняется в целевой системе.

Далее мы обозначим несколько конкретных способов применения руткитов прошивки злоумышленниками.

Атака на цепочку поставок

Угрозы, нацеленные на прошивки UEFI, повышают риск атак на цепочку поставок, поскольку злоумышленники могут установить вредоносный имплантат на сервер до его доставки в центр обработки данных или на ноутбук до его поступления в ИТ-отдел. А поскольку такие угрозы способны затронуть большое число клиентов поставщика услуг, раскрыв все их секреты, крупные участники рынка облачных вычислений, такие как Google, недавно начали применять методы криминалистического анализа прошивок, чтобы удостовериться в отсутствии компрометации своих прошивок.

> **Микросхема Google Titan**
>
> В 2017 году Google публично представила Titan, микросхему, которая защищает прошивку платформы, создавая аппаратный корень доверия. Доверие к конфигурации оборудования важно, особенно для безопасности облачных систем, где последствия атаки умножаются на число затронутых клиентов.
>
> Компании, работающие с крупными облачными системами и данными, такие как Amazon, Google, Microsoft, Facebook и Apple, разрабатывают или уже разработали оборудование для управления корнем доверия платформы. Даже если злоумышленники скомпрометируют платформу при помощи руткита прошивки, наличие изолированного корня доверия предотвратит атаки на Secure Boot и механизм обновления прошивки.

Компрометация BIOS через уязвимость прошивки

Злоумышленники могут скомпрометировать прошивку платформы, воспользовавшись уязвимостью для обхода защиты от записи или аутентификации BIOS. Чтобы освежить в памяти эту атаку, вернитесь к главе 16, где обсуждаются различные классы уязвимостей, применяемых для атак на BIOS. Для выявления таких атак можно использовать рассмотренные в этой главе подходы к криминалистическому анализу прошивок, чтобы проверить целостность прошивки платформы или помочь обнаружить вредоносные модули прошивки.

Основы получения прошивки

Самый первый шаг криминалистического анализа BIOS состоит в получении образа прошивки BIOS для дальнейшего исследования. Чтобы лучше понять расположение прошивки BIOS на современных платформах, обратитесь к рисунку 19-1, на котором показана архитектура набора микросхем типичной системы ПК.

Набор микросхем включает два основных компонента: центральный процессор и концентратор контроллеров платформы (Platform Controller Hub, PCH), или южный мост. PCH обеспечивает соединение между контроллерами доступных на платформе периферийных устройств и центральным процессором. В большинстве современных систем на базе архитектуры Intel x86, включая 64-разрядные платформы, системная прошивка расположена во флеш-памяти на шине последовательного периферийного интерфейса (Serial Peripheral Interface, SPI) ❶, физически соединённой с PCH. Флеш-память SPI является основной целью криминалистического анализа, поскольку в ней хранится прошивка, которую требуется исследовать.

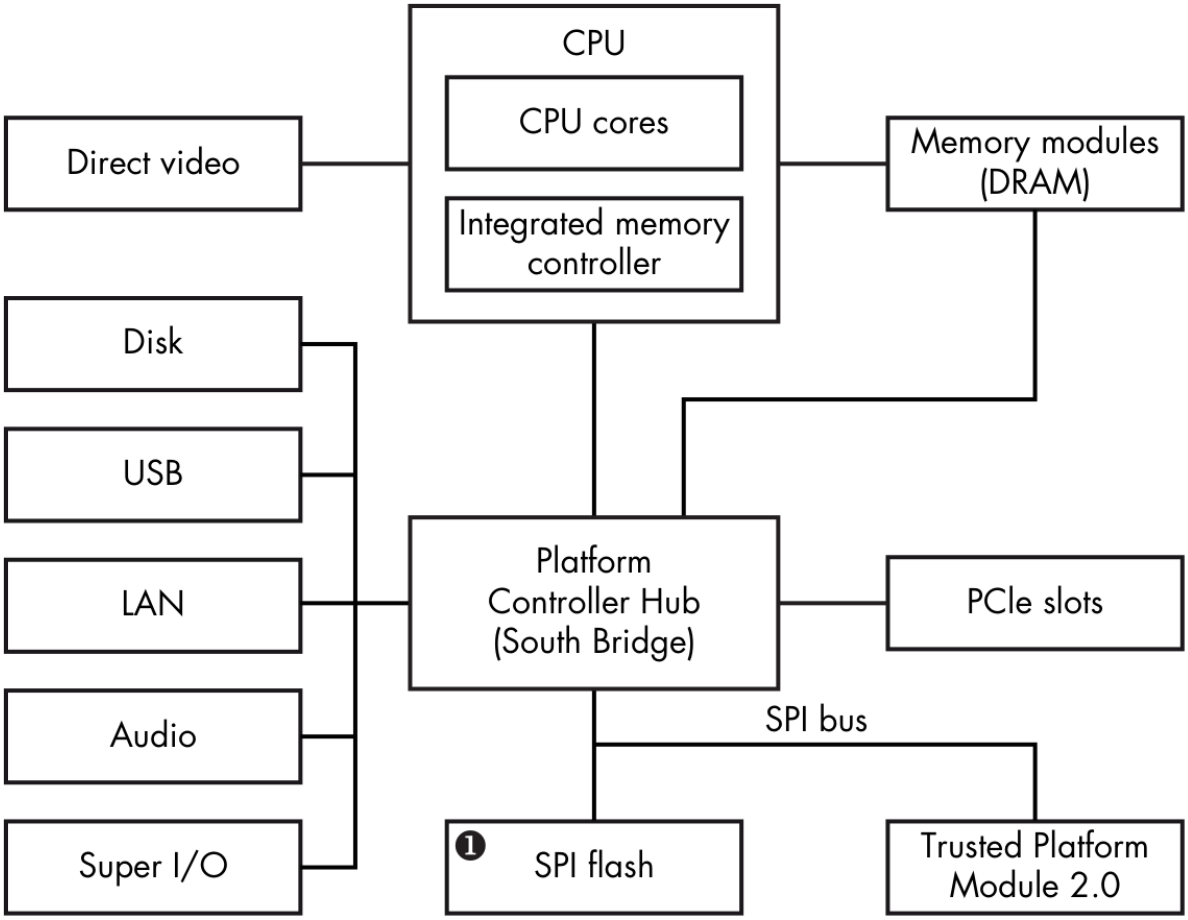


Рисунок 19-1. Блок-схема современного набора микросхем Intel

На материнской плате ПК обычно имеется одна отдельная физическая микросхема флеш-памяти SPI, припаянная к плате, но иногда встречаются системы с несколькими микросхемами флеш-памяти SPI. Это происходит, когда ёмкости одной микросхемы недостаточно для хранения всей системной прошивки; в таком случае поставщик платформы использует две микросхемы. Эта ситуация обсуждается далее в этой главе, в разделе «Определение местоположения микросхемы флеш-памяти SPI» на странице 376.

Чтобы получить образ прошивки, хранящийся во флеш-памяти SPI, необходимо уметь читать содержимое флеш-памяти. В общем случае прошивку можно считать при помощи программного либо аппаратного подхода. При программном подходе предпринимается попытка прочитать образ прошивки посредством взаимодействия с

Технология DualBIOS

Технология DualBIOS также использует несколько микросхем флеш-памяти SPI на материнской плате компьютера. Но в отличие от только что рассмотренного подхода, при котором несколько микросхем флеш-памяти SPI хранят единый образ прошивки, технология DualBIOS использует несколько микросхем для хранения разных образов прошивки или нескольких копий одного образа. Эта технология обеспечивает дополнительную защиту от повреждения прошивки: если прошивка в одной микросхеме повреждена, система может загрузиться со второй микросхемы, содержащей идентичный образ прошивки.

контроллером SPI при помощи программного обеспечения, выполняемого центральным процессором основной системы. При аппаратном подходе специальное устройство, называемое программатором SPI, физически подключается к флеш-памяти SPI, после чего образ прошивки считывается непосредственно из неё. Мы рассмотрим оба подхода, начав с программного метода.

Однако прежде чем перейти к описанию программного подхода, следует понять, что у каждого подхода есть свои преимущества и ограничения. Одно из преимуществ выгрузки прошивки UEFI программным методом состоит в возможности сделать это удалённо. Пользователь целевой системы может запустить приложение для выгрузки содержимого флеш-памяти SPI и отправить его криминалисту. Но у этого подхода есть и существенный недостаток: если злоумышленник уже скомпрометировал системную прошивку, он может вмешаться в процесс её получения, подделав данные, считываемые из флеш-памяти SPI. Это делает программный подход не вполне надёжным.

Аппаратный подход лишён этого недостатка. Хотя он требует физического присутствия и вскрытия корпуса целевой системы, этот метод напрямую считывает содержимое флеш-памяти SPI выключенной системы, не оставляя злоумышленнику возможности подделать данные, если только речь не идёт об аппаратном имплантате, который в этой книге не рассматривается.

Программный подход к получению прошивки

При программном подходе к выгрузке прошивки UEFI из целевой системы содержимое флеш-памяти SPI считывается из операционной системы. Доступ к контроллерам SPI современных систем можно получить через регистры в конфигурационном пространстве PCI, то есть блок регистров, задающих конфигурацию устройств на шине PCI. Эти регистры отображаются в память, и читать и записывать их можно при помощи обычных операций чтения и записи памяти. В этом разделе мы покажем, как найти эти регистры и взаимодействовать с контроллером SPI.

Прежде чем продолжить, следует знать, что расположение регистра SPI зависит от набора микросхем, поэтому для взаимодействия с контроллером SPI необходимо обратиться к документации по набору микросхем целевой платформы. В этой главе будет показано, как читать флеш-память SPI в наборах микросхем Intel 200 Series; расположение регистров SPI приведено в техническом описании набора микросхем Intel 200 Series PCH. На момент написания книги это были новейшие наборы микросхем для настольных систем.

Стоит также упомянуть, что области памяти, соответствующие регистрам, открываемым через конфигурационное пространство PCI, отображены в адресное пространство режима ядра и поэтому недоступны коду, выполняемому в адресном пространстве пользовательского режима. Для доступа к этому диапазону адресов потребуется разработать драйвер режима ядра. Инструмент Chipsec, рассматриваемый далее в этой главе, предоставляет собственный драйвер режима ядра для доступа к конфигурационному пространству PCI.

Определение местоположения регистров конфигурационного пространства PCI

Сначала необходимо найти диапазон памяти, в который отображены регистры контроллера SPI. Этот диапазон называется блоком регистров корневого комплекса (Root Complex Register Block, RCRB). По смещению 3800h в RCRB находится регистр базового адреса SPI (SPI Base Address Register, SPIBAR), содержащий базовый адрес отображённых в память регистров SPI (рисунок 19-2).

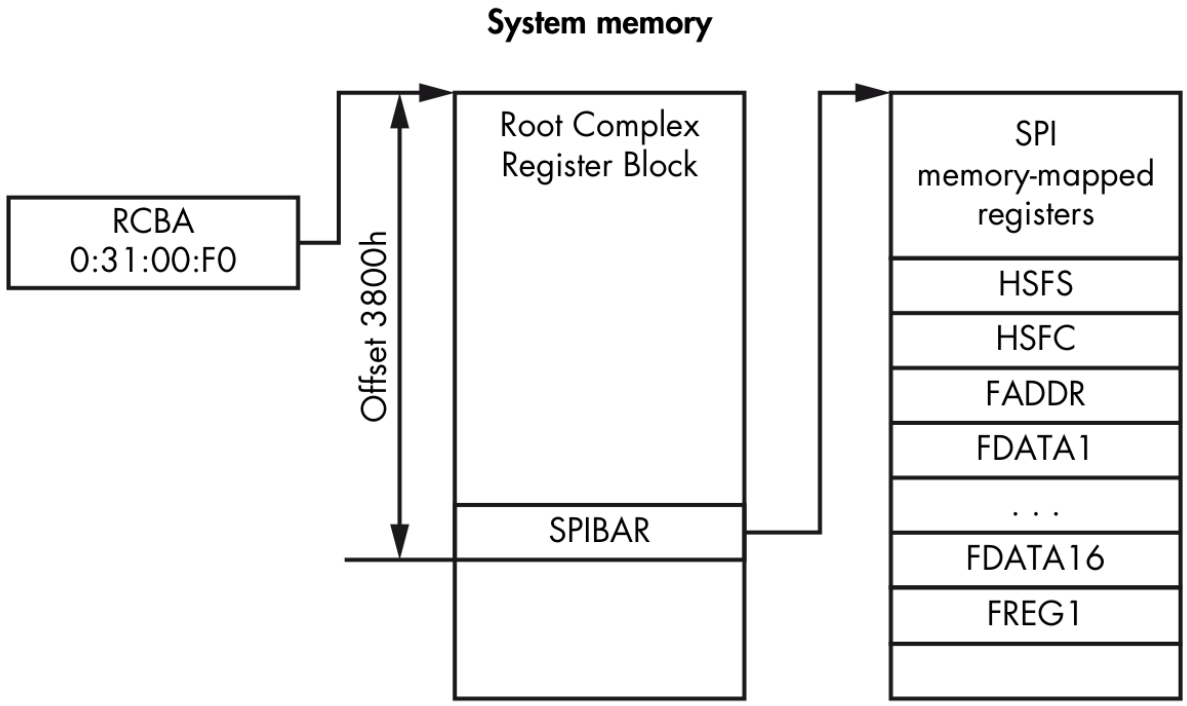


Рисунок 19-2. Расположение регистров управления и состояния SPI в системной памяти

> **Шина PCIe**
>
> Шина PCI Express (PCIe) представляет собой стандарт высокоскоростной последовательной шины, используемый практически во всех современных ПК разных сегментов рынка: потребительских ноутбуках и настольных компьютерах, серверах центров обработки данных и так далее. Шина PCIe служит соединением между различными компонентами и периферийными устройствами компьютера. Многие встроенные устройства набора микросхем, включая флеш-память SPI и контроллер памяти, представлены на этой шине как конечные точки PCIe.

Адрес RCRB хранится в PCI-регистре базового адреса корневого комплекса (Root Complex Base Address, RCBA), расположенном на шине 0, устройстве 31h, в функции 0. Это 32-разрядный регистр, а адрес RCRB задаётся битами 31:14. Мы предполагаем, что младшие 14 бит адреса RCRB равны нулю, поскольку RCRB выровнен по границе 16 Кбайт. Получив адрес RCRB, можно получить значение SPIBAR, прочитав память по смещению 3800h. В следующем разделе регистры SPI рассматриваются подробнее.

Прошивка во флеш-памяти SPI

Флеш-память SPI содержит не только прошивку BIOS, но и другие виды прошивок платформы, например прошивку Intel ME (Manageability Engine), прошивку контроллера Ethernet, а также прошивки и данные, зависящие от поставщика. Различные виды прошивок отличаются расположением и разрешениями управления доступом. Например, основная ОС не может обращаться к прошивке Intel ME, поэтому программный подход к получению прошивки для Intel ME не сработает.

Вычисление адресов конфигурационных регистров SPI

Получив значение SPIBAR, которое сообщает расположение регистров SPI в памяти, мы можем запрограммировать регистры для чтения содержимого флеш-памяти SPI. Смещения регистров SPI могут различаться в зависимости от платформы, поэтому лучший способ определить фактические значения для конкретной конфигурации оборудования состоит в поиске этих значений в документации к набору микросхем платформы. Например, для платформ, поддерживающих новейший на момент написания книги процессор Intel Kaby Lake, можно обратиться к техническому описанию концентратора контроллеров платформы семейства наборов микросхем Intel 200 Series, чтобы найти расположение отображённых в память регистров SPI. Эти сведения находятся в разделе «Serial Peripheral Interface». Для каждого регистра SPI в техническом описании приводятся его смещение относительно значения SPIBAR, имя регистра и значение регистра по умолчанию после сброса платформы. В этом разделе мы будем использовать данное техническое описание как справочник для определения адресов интересующих нас регистров SPI.

Использование регистров SPI

Теперь, зная, как найти адреса регистров SPI, можно определить, какие из них использовать для чтения содержимого флеш-памяти SPI. В таблице 19-1 перечислены все регистры, необходимые для получения образа флеш-памяти SPI.

Таблица 19-1. Регистры SPI для получения прошивки

Смещение относительно SPIBAR	Имя регистра	Описание регистра
04h-05h	HSFS	Состояние флеш-памяти при аппаратной последовательности
06h-07h	HSFC	Регистр управления флеш-памятью при аппаратной последовательности
08h-0Bh	FADDR	Адрес флеш-памяти
10h-4Fh	FDATAX	Массив данных флеш-памяти
58h-5Bh	FREG1	Область флеш-памяти 1 (дескриптор BIOS)

В последующих разделах мы обсудим каждый из этих регистров.

Регистр FREG1

Первым мы рассмотрим регистр области флеш-памяти 1 (flash region 1, FREG1). Он задаёт расположение области BIOS во флеш-памяти SPI. Структура этого 32-разрядного регистра показана на рисунке 19-3.

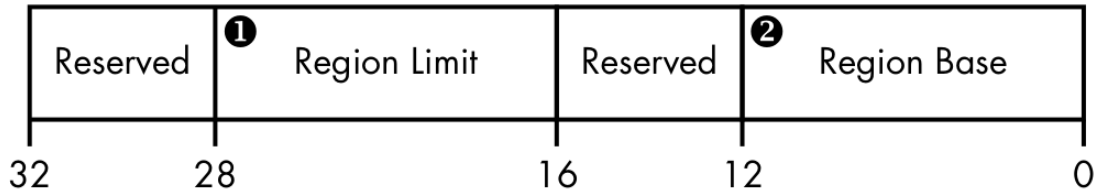


Рисунок 19-3. Структура регистра SPI FREG1

Поле `Region Base` ❷ задаёт биты 24:12 базового адреса области BIOS во флеш-памяти SPI. Поскольку область BIOS выровнена по границе 4 Кбайт, младшие 12 бит базового адреса области равны 0. Поле `Region Limit` ❶ задаёт биты 24:12 предельного адреса области BIOS во флеш-памяти SPI. Например, если поле `Region Base` содержит значение `0xaaa`, а `Region Limit` содержит значение `0xbbb`, то область BIOS во флеш-памяти SPI простирается от `0xaaa000` до `0xbbbfff`.

Регистр HSFC

Регистр управления флеш-памятью при аппаратной последовательности (hardware sequencing flash control, HSFC) позволяет отправлять команды контроллеру SPI. В спецификации эти команды называются циклами. Структура регистра HSFC показана на рисунке 19-4.

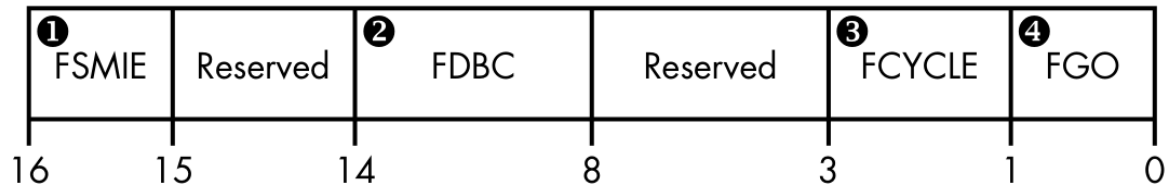


Рисунок 19-4. Структура регистра SPI HSFC

Регистр HSFC используется для отправки флеш-памяти SPI цикла чтения, записи или стирания. Двухбитовое поле `FCYCLE` ❸ кодирует выполняемую операцию следующим образом:

Значение	Операция
`00`	Чтение блока данных из флеш-памяти SPI
`01`	Запись блока данных во флеш-память SPI
`11`	Стирание блока данных во флеш-памяти SPI
`10`	Зарезервировано

Для циклов чтения и записи поле `FDBC` ❷ указывает число байтов, которые должны передаваться из флеш-памяти SPI или в неё. Содержимое этого поля отсчитывается от нуля: значение `000000b` представляет 1 байт, а значение `111111b` представляет 64 байта. Таким образом, число передаваемых байтов равно значению этого поля плюс 1.

Поле FGO ❹ используется для запуска операции с флеш-памятью SPI. Когда значение этого поля равно 1b, контроллер SPI будет читать, записывать или стирать данные в соответствии со значениями, записанными в поля FCYCLE и FDBC. Прежде чем устанавливать поле FGO, программное обеспечение должно задать все регистры, указывающие вид операции, объём данных и адрес во флеш-памяти SPI.

Последнее заслуживающее внимания поле HSFC называется разрешением SMI# флеш-памяти SPI (flash SPI SMI# enable, FSMIE) ❶. Когда это поле установлено, набор микросхем генерирует прерывание режима управления системой (System Management Interrupt, SMI), приводящее к выполнению кода SMM. Как будет показано в разделе «Рассмотрение недостатков программного подхода» на странице 373, FSMIE можно использовать для противодействия получению образа прошивки.

Взаимодействие с контроллером SPI

Использование регистра HSFC не является единственным способом отправки команд контроллеру SPI. В общем случае взаимодействовать с флеш-памятью SPI можно двумя способами: посредством аппаратной и программной последовательности. В показанном здесь методе аппаратной последовательности выбор команд SPI, отправляемых для операций чтения и записи, поручается оборудованию, и именно для этого используется регистр HSFC. Программная последовательность предоставляет больше возможностей для выбора конкретных команд, отправляемых при операциях чтения и записи. В этом разделе мы используем аппаратную последовательность через регистр HSFC, поскольку она проста и предоставляет функциональность, необходимую для чтения прошивки BIOS.

Регистр FADDR

Регистр адреса флеш-памяти (flash address, FADDR) используется для указания линейного адреса во флеш-памяти SPI при операциях чтения, записи и стирания. Этот регистр имеет длину 32 бита, но для указания линейного адреса операции используются только младшие 24 бита. Старшие 8 бит регистра зарезервированы и не используются.

Регистр HSFS

После запуска цикла SPI путём установки поля FGO регистра HSFC определить момент завершения цикла можно по регистру состояния флеш-памяти при аппаратной последовательности (hardware sequencing flash status, HSFS). Этот регистр состоит из нескольких полей, предоставляющих сведения о состоянии запрошенной операции. В таблице 19-2 показаны поля HSFS, используемые для чтения образа SPI.

Таблица 19-2. Поля регистра SPI HSFS

Смещение поля	Размер поля	Имя поля	Описание поля
0h	1	FDONE	Цикл флеш-памяти завершён
1h	1	FCERR	Ошибка цикла флеш-памяти
2h	1	AEL	Журнал ошибок доступа
5h	1	SCIP	Выполняется цикл SPI

Бит `FDONE` устанавливается набором микросхем, когда завершён предыдущий цикл флеш-памяти, инициированный полем `FGO` регистра HSFC. Биты `FCERR` и `AEL` указывают соответственно на возникновение ошибки во время цикла флеш-памяти SPI и на то, что возвращённые данные могут содержать недопустимые значения. Бит `SCIP` указывает, что цикл флеш-памяти выполняется. Мы устанавливаем `SCIP`, задавая бит `FGO`, а `SCIP` сбрасывается, когда значение `FDONE` становится равным 1. На основании этих сведений можно определить, что инициированная операция успешно завершена, когда истинно следующее выражение:

```
(FDONE == 1) && (FCERR == 0) && (AEL == 0) && (SCIP == 0)
```

Регистры FDATAX

Массив регистров данных флеш-памяти (`FDATAX`) содержит данные, предназначенные для чтения из флеш-памяти SPI или записи в неё. Каждый регистр имеет длину 32 бита, а общее число используемых регистров FDATAX зависит от количества передаваемых байтов, задаваемого в поле `FDBC` регистра HSFC.

Чтение данных из флеш-памяти SPI

Теперь соберём все эти сведения воедино и посмотрим, как читать данные из флеш-памяти SPI при помощи данных регистров. Сначала мы находим блок регистров корневого комплекса, по которому можно определить базовый адрес отображённых в память регистров SPI и получить доступ к этим регистрам. Прочитав регистр SPI FREG1, можно определить расположение области BIOS во флеш-памяти, то есть начальный и предельный адреса BIOS.

Затем область BIOS считывается при помощи только что описанных регистров SPI. Этот шаг показан на рисунке 19-5.

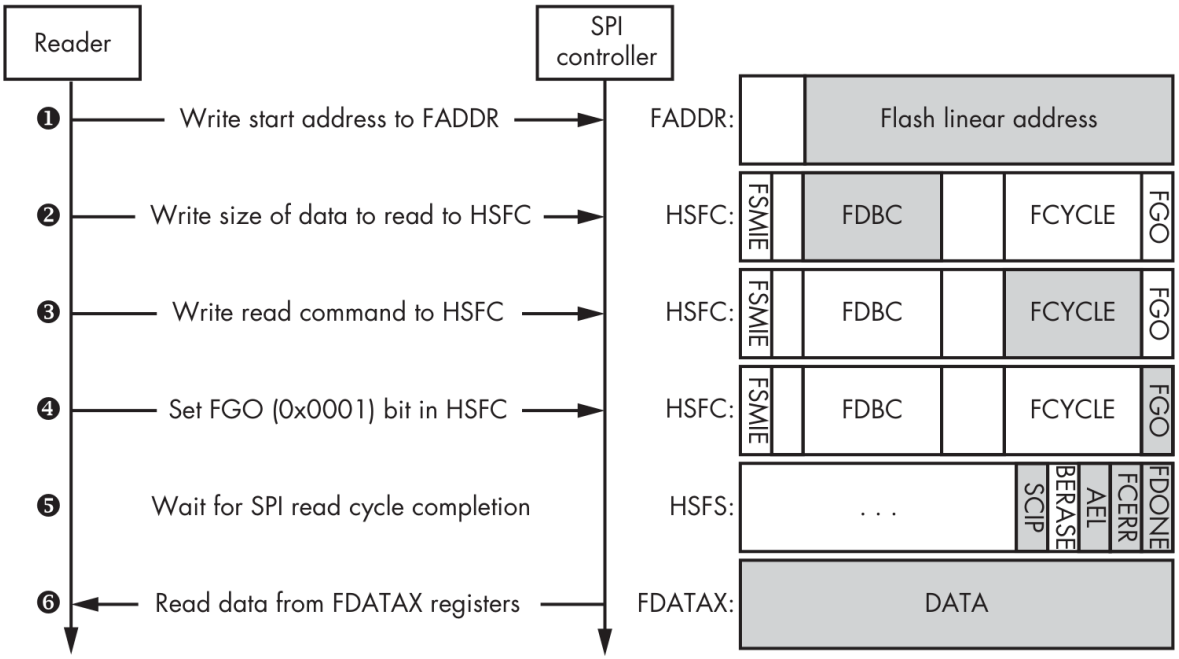


Рисунок 19-5. Чтение данных из флеш-памяти SPI

Сначала мы задаём в `FADDR` линейный адрес области флеш-памяти, которую требуется прочитать ❶. Затем указываем общее число байтов, считываемых из флеш-памяти, задавая поле `FDBC` ❷ регистра управления флеш-памятью. Значение `111111b` позволяет считывать 64 байта за цикл. Далее в поле `FCYCLE` ❸ записывается значение `00b`, обозначающее цикл чтения, и устанавливается бит `FGO` ❹, запускающий операцию чтения флеш-памяти.

После установки бита `FGO` необходимо отслеживать регистр состояния флеш-памяти, чтобы узнать о завершении операции. Это можно сделать, проверяя поля `FDONE`, `FCERR`, `AEL` и `SCIP` ❺. По окончании операции чтения данные флеш-памяти считываются из регистров `FDATAX` ❻. Регистр `FDATAX[1]` предоставляет первые 4 байта флеш-памяти по целевому адресу, заданному в регистре `FADDR`; `FDATAX[2]` предоставляет следующие 4 байта флеш-памяти и так далее. Повторяя эти шаги и увеличивая значение `FADDR` на 64 байта при каждой итерации, мы считываем из флеш-памяти SPI всю область BIOS.

Рассмотрение недостатков программного подхода

Программный подход к выгрузке прошивки BIOS удобен тем, что не требует физического присутствия: он позволяет удалённо читать содержимое флеш-памяти SPI. Но этот метод неустойчив к действиям злоумышленника, который уже скомпрометировал системную прошивку и способен выполнять вредоносный код в SMM.

Как уже отмечалось, регистр HSFC содержит бит FSMIE, который запускает SMI по завершении цикла флеш-памяти. Если злоумышленник уже скомпрометировал SMM и способен установить бит FSMIE до того, как программное обеспечение получения прошивки установит бит FGO, то при генерации SMI злоумышленник получит управление и сможет изменить содержимое регистров FDATAX. В результате программное обеспечение получения прошивки прочитает из FDATAX поддельные значения и не сможет получить исходный образ области BIOS. Эта атака показана на рисунке 19-6.

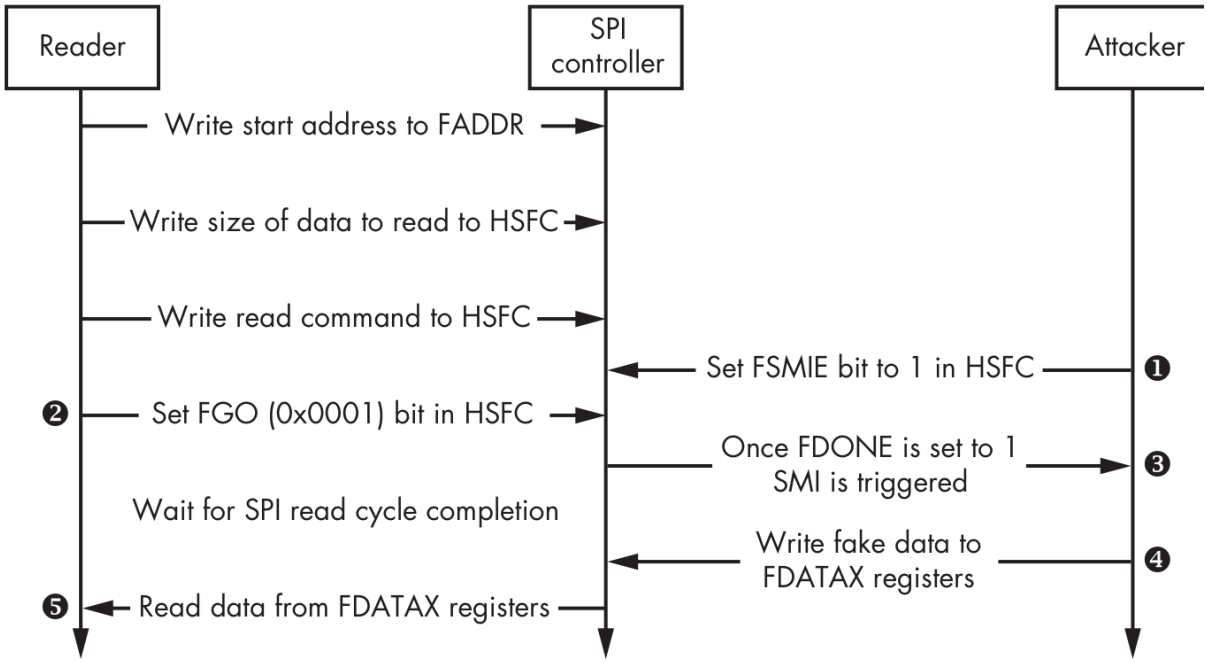


Рисунок 19-6. Нарушение программного получения BIOS посредством SMI

Прежде чем считывающее программное обеспечение установит бит FGO ❷ в регистре управления флеш-памятью, злоумышленник записывает 1 в бит FSMIE ❶ этого регистра. По завершении цикла и обратной записи данных в регистры FDATAX запускается SMI, и злоумышленник получает управление ❸. Затем злоумышленник изменяет содержимое регистров FDATAX ❹, чтобы скрыть атаку на прошивку BIOS. Вернув управление, считывающее программное обеспечение получит поддельные данные ❺ и не обнаружит скомпрометированную прошивку.

Эта атака показывает, что программный подход не предоставляет абсолютно надёжного решения для получения прошивки. В следующем разделе мы обсудим аппаратный подход к получению системной прошивки для криминалистического анализа. Криминалистический анализ посредством физического подключения устройства к флеш-памяти SPI исключает возможность атаки, показанной на рисунке 19-6.

Аппаратный подход к получению прошивки

Чтобы гарантированно получить фактический образ BIOS, хранящийся во флеш-памяти SPI, а не уже скомпрометированный злоумышленником образ, можно использовать аппаратный подход. При этом подходе устройство физически подключается к флеш-памяти SPI, и её содержимое считывается напрямую. Это наилучшее решение, поскольку оно заслуживает большего доверия, чем программный подход. Дополнительное преимущество состоит в возможности получить другие хранящиеся во флеш-памяти SPI прошивки, такие как прошивки ME и GBE, которые могут быть недоступны при программном подходе из-за ограничений, обеспечиваемых контроллером SPI.

Шина SPI современных систем позволяет нескольким ведущим устройствам взаимодействовать с флеш-памятью SPI. Например, в системах на базе набора микросхем Intel обычно имеются три ведущих устройства: основной центральный процессор, Intel ME и GBE. Эти три ведущих устройства обладают разными правами доступа к различным областям флеш-памяти SPI. На большинстве современных платформ основной центральный процессор не может читать и записывать область флеш-памяти SPI, содержащую прошивки Intel ME и GBE.

На рисунке 19-7 показана типичная конфигурация для получения образа прошивки BIOS посредством чтения флеш-памяти SPI.

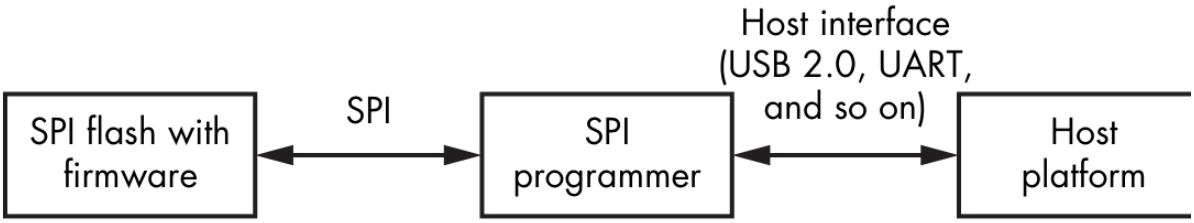


Рисунок 19-7. Типичная конфигурация для выгрузки образа флеш-памяти SPI

Для чтения данных из флеш-памяти необходимо дополнительное устройство, называемое программатором SPI, которое физически подключается к микросхеме флеш-памяти SPI целевой системы. Программатор SPI также подключается через интерфейс USB или UART к основной системе, используемой для получения образа прошивки BIOS. Затем на программаторе запускается специальное программное обеспечение, заставляющее его считывать данные из микросхемы флеш-памяти и передавать их на компьютер аналитика. Это может быть проприетарное программное обеспечение, поставляемое с конкретным программатором SPI, либо решение с открытым исходным кодом, такое как инструмент Flashrom, который рассматривается далее в разделе «Чтение флеш-памяти SPI при помощи мини-модуля FT2232» на странице 377.

Практический пример Lenovo ThinkPad T540p

Аппаратный подход ещё сильнее зависит от конкретной системы, чем программный. Он требует обращения к документации платформы, чтобы выяснить, флеш-память какого типа используется для хранения прошивки и где она физически расположена в системе. Кроме того, для чтения содержимого флеш-памяти можно использовать множество устройств программирования флеш-памяти, предназначенных для конкретного оборудования. Мы не будем обсуждать различные аппаратные и программные варианты получения системной прошивки, поскольку их слишком много. Вместо этого рассмотрим один из возможных способов выгрузки прошивки из Lenovo ThinkPad T540p при помощи программатора SPI FT2232.

Мы выбрали этот программатор SPI из-за его сравнительно низкой цены, около 30 долларов, гибкости и собственного опыта работы с ним. Как уже отмечалось, существует множество решений, каждое из которых обладает уникальными особенностями, преимуществами и недостатками.

Программатор Dediprog SF100 ISP IC

Ещё одно устройство, которое мы хотели бы упомянуть, это программатор Dediprog SF100 ISP IC, показанный на рисунке 19-8. Он популярен среди исследователей безопасности, поддерживает множество видов флеш-памяти SPI и предлагает обширную функциональность. На сайте Minnowboard, открытой эталонной платы для разработчиков аппаратного обеспечения и прошивок, имеется хорошее руководство по использованию Dediprog для обновления прошивки.

Рисунок 19-8. Программатор Dediprog SF100 ISP IC

Определение местоположения микросхемы флеш-памяти SPI

Начнём с физического считывания образа прошивки платформы Lenovo ThinkPad T540p. Сначала, чтобы выгрузить системную прошивку из целевой системы, необходимо найти на системной плате микросхемы флеш-памяти SPI. Для этого мы обратились к руководству по обслуживанию оборудования данной модели ноутбука и разобрали оборудование целевой системы. На рисунках 19-9 и 19-10 показано расположение двух микросхем флеш-памяти. На рисунке 19-9 представлена системная плата целиком. Микросхемы флеш-памяти SPI расположены в выделенной области.

Предупреждение. Не повторяйте описанные в этом разделе действия, если вы не уверены на 100 процентов в том, что делаете. Недопустимая или неправильная конфигурация инструментов может привести целевую систему в неработоспособное состояние.

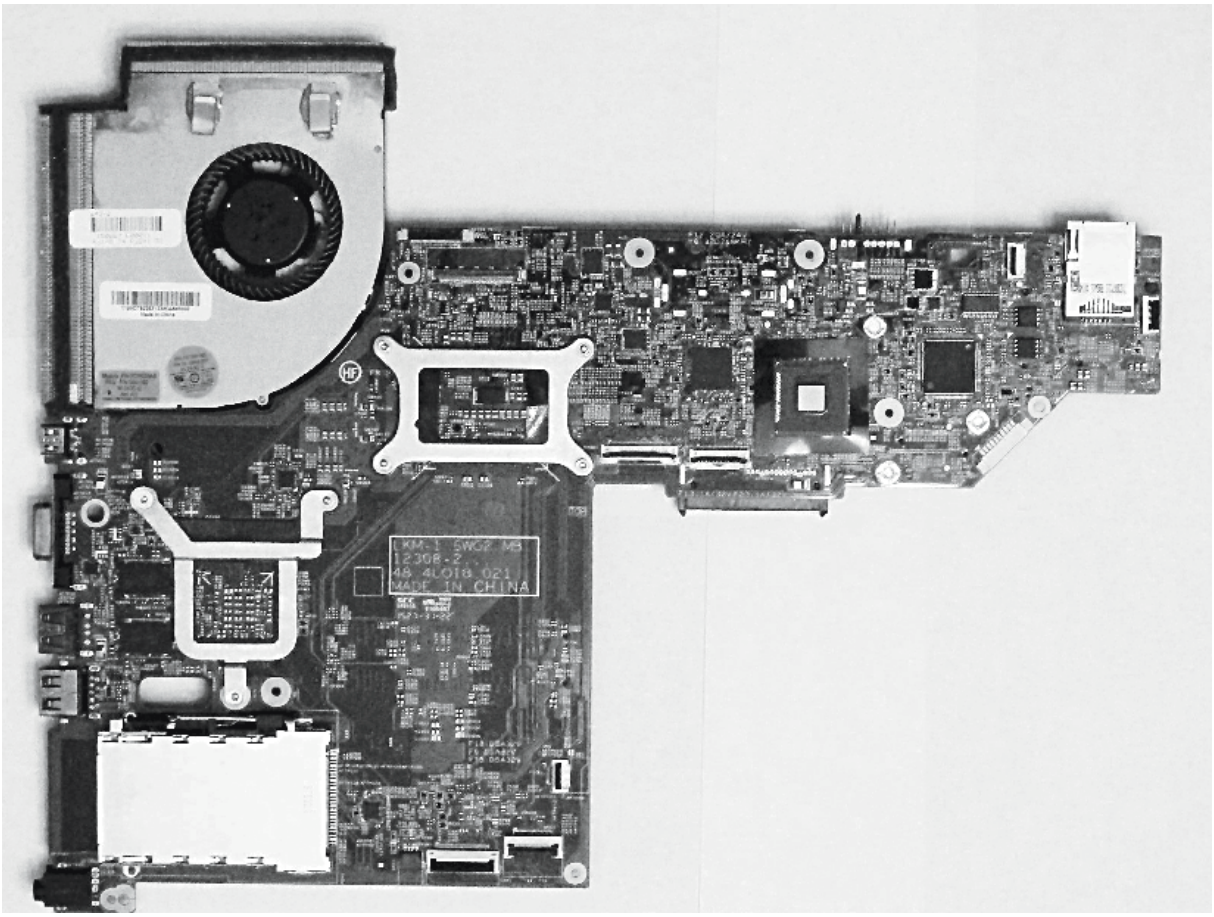


Рисунок 19-9. Системная плата Lenovo ThinkPad T540p с модулями флеш-памяти SPI

На рисунке 19-10 увеличена область, выделенная на рисунке 19-9, благодаря чему микросхемы флеш-памяти SPI видны более отчётливо. В этой модели ноутбука для хранения прошивки используются два модуля флеш-памяти SOIC-8: один объёмом 64 Мбит (8 Мбайт) и другой объёмом 32 Мбит (4 Мбайт). Это очень распространённое решение во многих современных настольных компьютерах и ноутбуках.

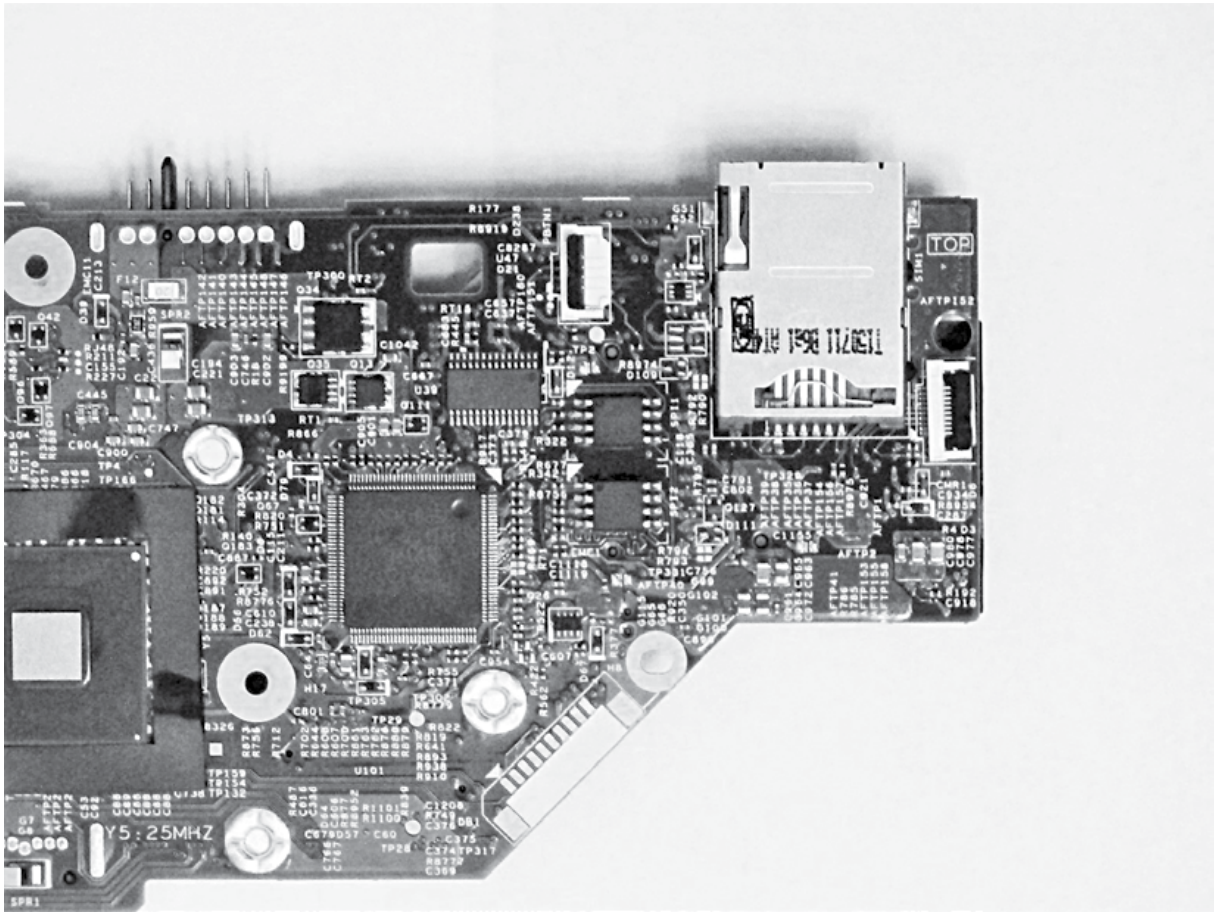


Рисунок 19-10. Расположение модулей флеш-памяти SPI на системной плате ноутбука

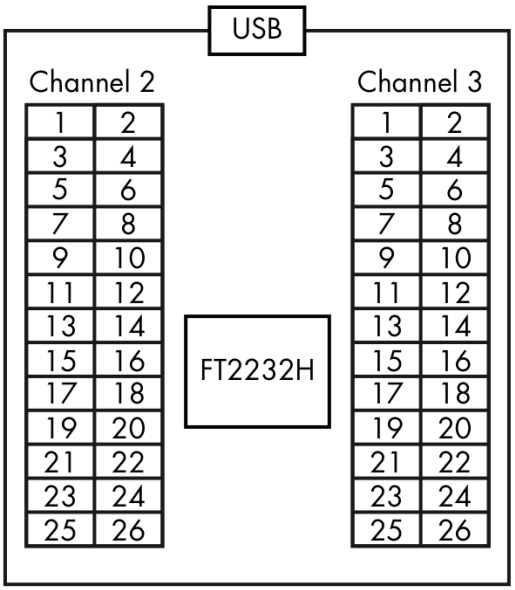
Поскольку для хранения системной прошивки используются две отдельные микросхемы, потребуется выгрузить содержимое обеих. Окончательный образ прошивки будет получен путём объединения образов двух микросхем флеш-памяти в один файл.

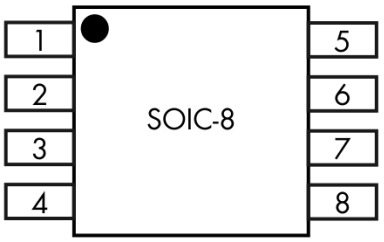
Чтение флеш-памяти SPI при помощи мини-модуля FT2232

Определив физическое расположение микросхем, можно подключить выводы программатора SPI к модулю флеш-памяти на системной плате. В техническом описании мини-модуля FT2232H показано, какие выводы следует использовать для подключения устройства к микросхеме памяти. На рисунке 19-11 представлено расположение выводов как мини-модуля FT2232H, так и микросхемы флеш-памяти SPI.

FT2232H имеет два набора выводов, соответствующих двум каналам: каналу 2 и каналу 3. Для чтения содержимого флеш-памяти SPI можно использовать любой канал. В нашем эксперименте для подключения FT2232H к микросхеме флеш-памяти SPI используется канал 3. На рисунке 19-11 показано, какие выводы FT2232H мы соединили с соответствующими выводами микросхемы флеш-памяти SPI.

Помимо подключения FT2232H к микросхеме памяти, необходимо настроить его для работы с питанием от шины USB. Мини-модуль FT2232H поддерживает два режима работы: с питанием от шины USB и с автономным питанием. В режиме питания от шины мини-модуль получает питание от шины USB, к которой он подключён, а в режиме автономного питания питание подаётся независимо от подключения к шине USB.





Channel 3 pin	SPI flash chip pin
23	1
24	2
21	3
04	4
25	5
26	6
20	7
01	8

Рисунок 19-11. Расположение выводов мини-модуля FT2232H и микросхемы флеш-памяти SPI

Чтобы облегчить подключение программатора SPI к модулю микросхемы SPI, мы используем зажим SOIC-8, показанный на рисунке 19-12. Этот зажим позволяет легко соединить выводы мини-модуля с соответствующими выводами микросхемы флеш-памяти.

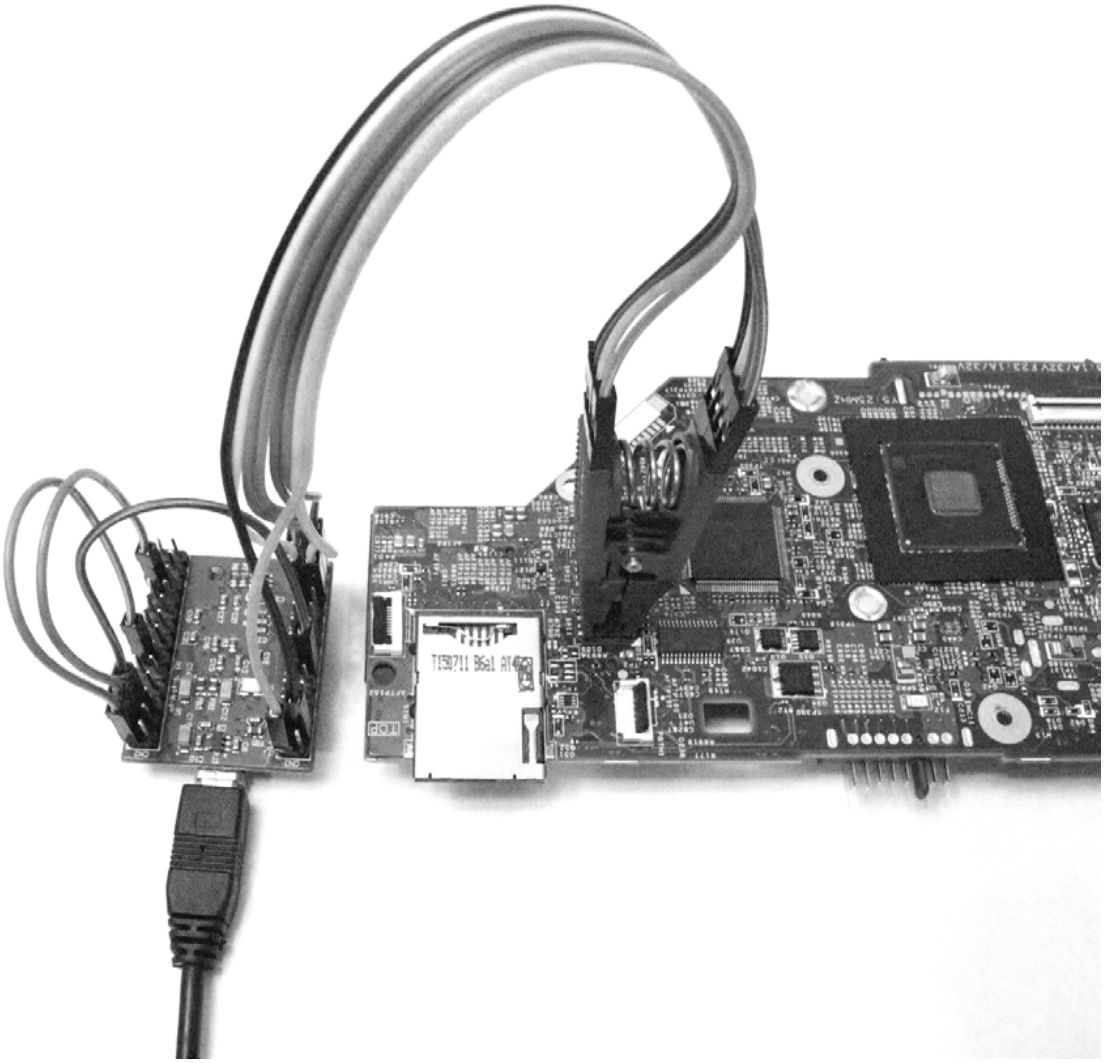


Рисунок 19-12. Подключение мини-модуля FT2232H к микросхеме флеш-памяти SPI

После соединения всех компонентов можно прочитать содержимое микросхемы флеш-памяти SPI. Для этого мы используем инструмент с открытым исходным кодом под названием Flashrom. Этот инструмент был разработан специально для идентификации, чтения, записи, проверки и стирания микросхем флеш-памяти. Он поддерживает множество микросхем флеш-памяти и работает с большим числом различных программаторов SPI, включая мини-модуль FT2232H.

В листинге 19-1 показаны результаты запуска Flashrom для чтения содержимого обеих микросхем флеш-памяти SPI на платформе Lenovo ThinkPad T540p.

```
❶ user@host: flashrom -p ft2232_spi:type=2232H,port=B --read dump_1.bin
flashrom v0.9.9-r1955 on Linux 4.8.0-36-generic (x86_64)
flashrom is free software, get the source code at https://flashrom.org
Calibrating delay loop... OK.
❷ Found Macronix flash chip "MX25L6436E/MX25L6445E/MX25L6465E/MX25L6473E"
(8192 kB, SPI) on ft2232_spi.
❸ Reading flash... done.
user@host: flashrom -p ft2232_spi:type=2232H,port=B --read dump_2.bin
flashrom v0.9.9-r1955 on Linux 4.8.0-36-generic (x86_64)
flashrom is free software, get the source code at https://flashrom.org
Calibrating delay loop... OK.
Found Macronix flash chip "MX25L3273E" (4096 kB, SPI) on ft2232_spi.
Reading flash... done.
❹ user@host: cat dump_2.bin >> dump_1.bin
```

Листинг 19-1. Выгрузка образов флеш-памяти SPI при помощи инструмента Flashrom

Сначала мы запускаем Flashrom для выгрузки содержимого первой микросхемы флеш-памяти SPI, передавая ему в качестве параметров тип программатора и номер порта ❶. Указанный тип `2232H` соответствует нашему мини-модулю FT2232H, а порт `B` соответствует каналу 3, используемому для подключения к микросхеме флеш-памяти SPI. Параметр `--read` предписывает Flashrom прочитать содержимое флеш-памяти SPI в файл `dump_1.bin`. После запуска инструмент выводит тип обнаруженной микросхемы флеш-памяти SPI, в нашем случае Macronix MX25L6473E ❷. Завершив чтение флеш-памяти, Flashrom выводит подтверждение ❸.

После чтения первой микросхемы флеш-памяти мы переставляем зажим на вторую микросхему и снова запускаем Flashrom, чтобы выгрузить содержимое второй микросхемы в файл `dump_2.bin`. По завершении этой операции создаётся полный образ прошивки путём объединения двух выгруженных образов ❹.

Теперь мы выгрузили полный, заслуживающий доверия образ прошивки. Даже если BIOS уже заражена и злоумышленник попытается помешать получению прошивки, мы всё равно получим фактический код и данные прошивки. Далее мы проанализируем этот образ.

Анализ образа прошивки при помощи UEFITool

Получив образ прошивки из флеш-памяти SPI целевой системы, мы можем его проанализировать. В этом разделе будут рассмотрены основные компоненты прошивки платформы, такие как тома прошивки, файлы томов и разделы, которые необходимо понимать для изучения структуры прошивки UEFI в образе флеш-памяти. Затем мы сосредоточимся на важнейших этапах криминалистического анализа прошивки.

> **Примечание.** В этом разделе мы приведём высокоуровневые описания, а не подробные определения используемых структур данных, поскольку эта тема слишком обширна и её глубокое рассмотрение выходит за рамки главы. Однако для желающих получить дополнительные сведения мы приведём ссылки на документацию, содержащую определения и структуру данных.

Мы вновь обратимся к UEFITool, представленному в главе 15 инструменту с открытым исходным кодом для разбора, извлечения и изменения образов прошивок UEFI, чтобы продемонстрировать теоретические концепции на реальном образе прошивки, полученном в предыдущем разделе. Возможность заглянуть внутрь образа прошивки, просмотреть и извлечь различные компоненты чрезвычайно полезна для криминалистического анализа. Этот инструмент не требует установки: после загрузки приложение сразу готово к запуску.

Знакомство с областями флеш-памяти SPI

Прежде чем рассматривать образ прошивки, необходимо понять, как организованы сведения, хранящиеся во флеш-памяти SPI. В общем случае флеш-память SPI современных платформ на базе набора микросхем Intel состоит из нескольких областей. Каждая область предназначена для хранения прошивки определённого устройства платформы; например, прошивка UEFI BIOS, прошивка Intel ME и прошивка Intel GBE, то есть встроенного устройства локальной сети, хранятся каждая в собственной области. На рисунке 19-13 показано расположение нескольких областей флеш-памяти SPI.

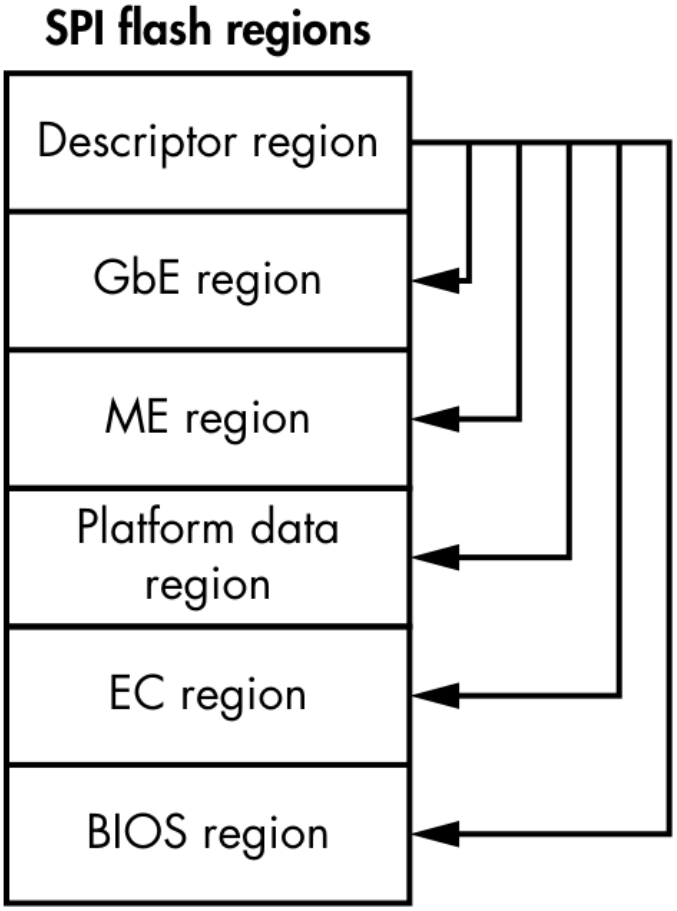


Рисунок 19-13. Области образа флеш-памяти SPI

Флеш-память SPI современных систем поддерживает до шести областей, включая область дескриптора, с которой всегда начинаются образы флеш-памяти. Область дескриптора содержит сведения о структуре флеш-памяти SPI, то есть предоставляет набору микросхем сведения о других имеющихся во флеш-памяти SPI областях, включая их расположение и права доступа. Область дескриптора также определяет права доступа каждого ведущего устройства системы, способного взаимодействовать с контроллером флеш-памяти SPI. Несколько ведущих устройств могут одновременно взаимодействовать с контроллером. Полную структуру области дескриптора, включая определения всех расположенных в ней структур данных, можно найти в спецификации набора микросхем целевой платформы.

В этой главе нас прежде всего интересует область BIOS, содержащая прошивку, которую центральный процессор выполняет по вектору сброса. Расположение области BIOS можно извлечь из области дескриптора. Обычно BIOS является последней областью во флеш-памяти SPI и представляет собой главную цель криминалистического анализа.

Рассмотрим различные области образа SPI, полученного при помощи аппаратного подхода.

Просмотр областей флеш-памяти SPI при помощи UEFITool

Сначала запустите UEFITool и выберите `File` > `Open image file`. Затем выберите файл с образом SPI, который требуется проанализировать; образ для работы доступен среди ресурсов книги. Результат этой операции показан на рисунке 19-14.

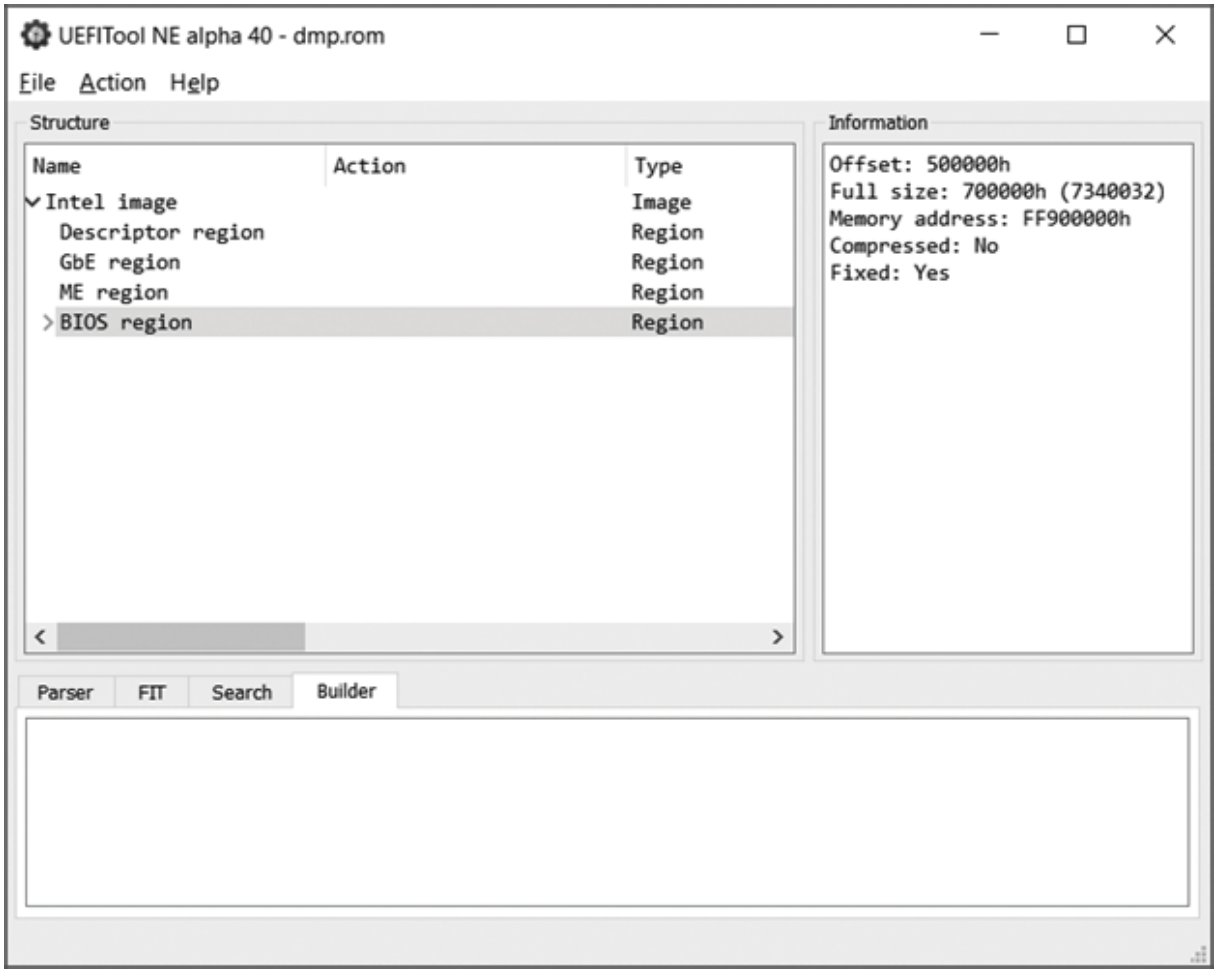


Рисунок 19-14. Просмотр областей флеш-памяти SPI в UEFITool

После загрузки образа прошивки UEFITool автоматически разбирает его и представляет эти сведения в древовидной структуре. На рисунке 19-14 инструмент определил, что образ прошивки получен из системы на базе набора микросхем Intel ❶ и содержит лишь четыре области SPI: дескриптор, ME, GbE и BIOS. Если выбрать область BIOS в окне `Structure`, сведения о ней отобразятся в окне `Information`. UEFITool показывает следующие элементы, описывающие область:

• `Offset` ❷ - смещение области от начала образа флеш-памяти SPI.
• `Full size` ❸ - размер области в байтах.
• `Memory address` ❹ - адрес области, отображённой в физическую память.
• `Compressed` ❺ - признак того, что область содержит сжатые данные.

Инструмент предоставляет удобный способ извлекать отдельные области, а также любой другой объект, показанный в окне структуры, из образа SPI и сохранять их в отдельном файле, как показано на рисунке 19-15.

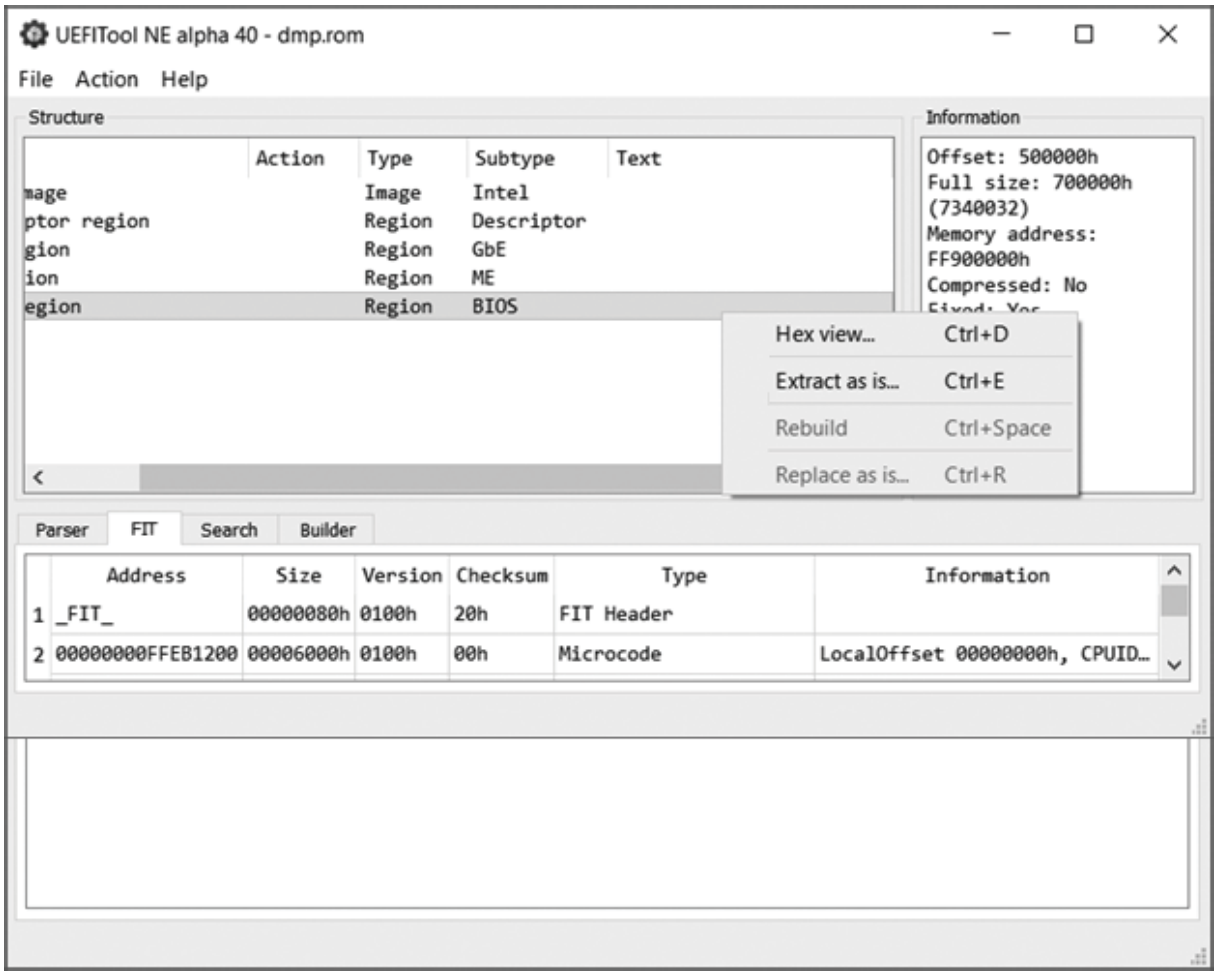


Рисунок 19-15. Извлечение области BIOS и её сохранение в отдельном файле

Чтобы извлечь и сохранить область, щёлкните по ней правой кнопкой мыши и выберите в контекстном меню `Extract as is...`. После этого инструмент покажет обычное диалоговое окно, позволяющее выбрать место сохранения нового файла. Выполнив эту операцию, проверьте выбранное расположение и убедитесь, что она завершилась успешно.

Анализ области BIOS

Определив расположение области BIOS, можно перейти к её анализу. На высоком уровне область BIOS организована в тома прошивки, которые представляют собой основные хранилища данных и кода. Точное определение тома прошивки приведено в спецификации EFI Firmware Volume. Каждый том начинается с заголовка, содержащего необходимые атрибуты тома, такие как тип файловой системы тома, размер тома и контрольная сумма.

Рассмотрим тома прошивки, имеющиеся в полученной BIOS. Если дважды щёлкнуть левой кнопкой мыши по области BIOS в окне UEFITool, как на рисунке 19-15, будет показан список доступных томов прошивки, представленный на рисунке 19-16.

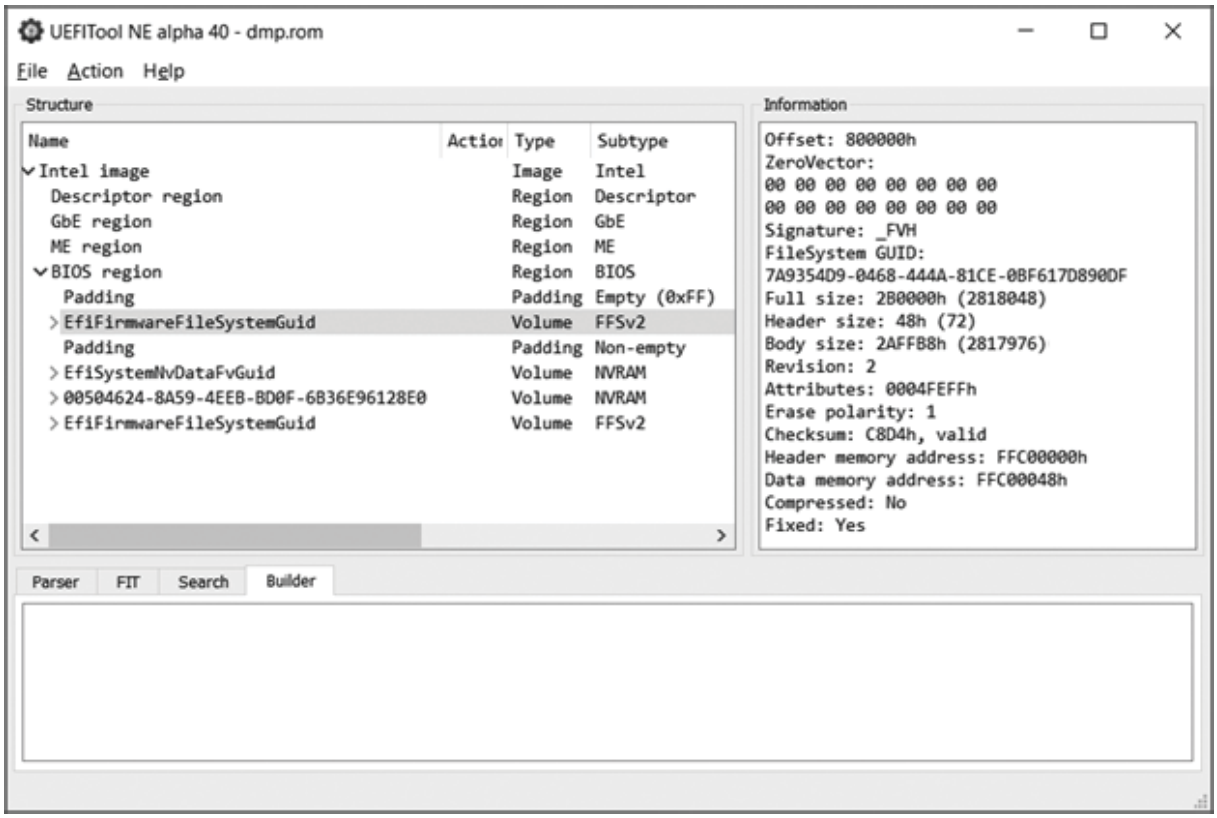


Рисунок 19-16. Просмотр томов прошивки, имеющихся в области BIOS

В нашей области BIOS имеется четыре тома прошивки, а также две области с пометкой `Padding`. Области заполнения не принадлежат ни одному тому прошивки, а представляют собой пустое пространство между ними, заполненное значениями `0x00` или `0xff` в зависимости от полярности стирания флеш-памяти SPI. Полярность стирания определяет значения, записываемые во флеш-память при операции стирания. Если полярность стирания равна 1, стёртые байты флеш-памяти получают значения `0xff`; если полярность стирания равна 0, стёртые байты получают значения `0x00`. Следовательно, когда полярность стирания равна 1, области заполнения, то есть пустое пространство, состоят из значений `0xff`.

На вкладке сведений справа от томов на рисунке 19-16 можно увидеть атрибуты выбранного тома. Ниже перечислены некоторые важные поля:

• `Offset` ❶ - смещение тома прошивки от начала образа SPI.
• `Signature` ❷ - сигнатура тома прошивки в заголовке. Это поле используется для идентификации томов в областях BIOS.

• `Filesystem GUID` ❸ - идентификатор файловой системы, используемой в томе прошивки. Этот глобально уникальный идентификатор (Globally Unique Identifier, GUID) отображается в окне структуры как имя тома. Если GUID документирован, UEFITool вместо шестнадцатеричного значения показывает его удобочитаемое имя, например `EfiFirmwareFileSystemGuid` на рисунке 19-16.
• `Header size` ❹ - размер заголовка тома прошивки. Данные тома следуют за заголовком.
• `Body size` ❺ - размер тела тома прошивки, то есть размер данных, хранящихся в томе.

Знакомство с файловой системой прошивки

Тома прошивки организованы как файловая система, тип которой указывается GUID файловой системы в заголовке прошивки. Чаще всего в томах прошивки используется файловая система прошивки (firmware filesystem, FFS), определённая в спецификации EFI FFS, но в томах прошивки применяются и другие файловые системы, такие как FAT32 или NTFS. Мы сосредоточимся на FFS как наиболее распространённой.

FFS хранит все файлы в корневом каталоге и не предусматривает никакой иерархии каталогов. Согласно спецификации EFI FFS, с каждым файлом связан расположенный в его заголовке тип, описывающий данные, которые хранятся в этом файле. Ниже перечислены некоторые часто встречающиеся типы файлов, которые могут быть полезны при криминалистическом анализе:

• `EFI_FV_FILETYPE_RAW` - необработанный файл; о хранящихся в нём данных не следует делать никаких предположений.
• `EFI_FV_FILETYPE_FIRMWARE_VOLUME_IMAGE` - файл, содержащий инкапсулированный том прошивки. Хотя FFS не предусматривает иерархии каталогов, этот тип файла можно использовать для создания древовидной структуры путём инкапсуляции модулей прошивки в файлах.
• `EFI_FV_FILETYPE_SECURITY_CORE` - файл с кодом и данными, выполняемыми на этапе Security (SEC) процесса загрузки. Этап SEC является самым первым этапом процесса загрузки UEFI.
• `EFI_FV_FILETYPE_PEI_CORE` - исполняемый файл, запускающий этап инициализации до EFI (Pre-EFI Initialization, PEI) процесса загрузки. Этап PEI следует за этапом SEC.
• `EFI_FV_FILETYPE_PEIM` - модули PEI, то есть файлы с кодом и данными, выполняемыми на этапе PEI.
• `EFI_FV_FILETYPE_DXE_CORE` - исполняемый файл, запускающий этап среды выполнения драйверов (Driver Execution Environment, DXE) процесса загрузки. Этап DXE следует за этапом PEI.
• `EFI_FV_FILETYPE_DRIVER` - исполняемый файл, запускаемый на этапе DXE.
• `EFI_FV_FILETYPE_COMBINED_PEIM_DRIVER` - файл с кодом и данными, которые могут выполняться как на этапе PEI, так и на этапе DXE.
• `EFI_FV_FILETYPE_APPLICATION` - приложение UEFI, представляющее собой исполняемый файл, который можно запустить на этапе DXE.
• `EFI_FV_FILETYPE_FFS_PAD` - файл заполнения.

В отличие от типичных файловых систем, используемых в операционных системах, где файлы имеют удобочитаемые имена, файлы FFS идентифицируются посредством GUID.

Знакомство с разделами файлов

Большинство файлов прошивки, хранящихся в FFS, состоят из одной или нескольких отдельных частей, называемых разделами, хотя некоторые файлы, например файлы `EFI_FV_FILETYPE_RAW`, не содержат никаких разделов.

Существуют два типа разделов: конечные разделы и разделы инкапсуляции. Конечные разделы непосредственно содержат данные, тип которых определяется атрибутом типа раздела в заголовке раздела. Разделы инкапсуляции содержат разделы файлов, которые могут быть как конечными разделами, так и разделами инкапсуляции. Это означает, что один раздел инкапсуляции может содержать вложенный раздел инкапсуляции.

В следующем списке описаны некоторые типы конечных разделов:

• `EFI_SECTION_PE32` - содержит образ PE.
• `EFI_SECTION_PIC` - содержит позиционно-независимый код (position-independent code, PIC).
• `EFI_SECTION_TE` - содержит образ Terse Executable (TE).
• `EFI_SECTION_USER_INTERFACE` - содержит строку пользовательского интерфейса. Обычно она используется для хранения удобочитаемого имени файла в дополнение к GUID файла.
• `EFI_SECTION_FIRMWARE_VOLUME_IMAGE` - содержит инкапсулированный образ прошивки.

А ниже перечислена пара разделов инкапсуляции, определённых в спецификации FFS:

• `EFI_SECTION_COMPRESSION` - содержит сжатые разделы файлов.
• `EFI_SECTION_GUID_DEFINED` - инкапсулирует другие разделы в соответствии с алгоритмом, идентифицируемым GUID раздела. Например, этот тип используется для подписанных разделов.

Эти объекты составляют содержимое прошивки UEFI на современных платформах. Криминалист должен учитывать каждый компонент прошивки, будь то раздел с исполняемым кодом, таким как PE32, TE или PIC, либо файл данных с энергонезависимыми переменными.

Чтобы лучше понять представленные здесь концепции, обратитесь к рисунку 19-17, на котором показано расположение драйвера CpuInitDxe в томе прошивки. Этот драйвер отвечает за инициализацию центрального процессора на этапе DXE. Чтобы описать его расположение в образе прошивки, мы пройдём по иерархии FFS снизу вверх.

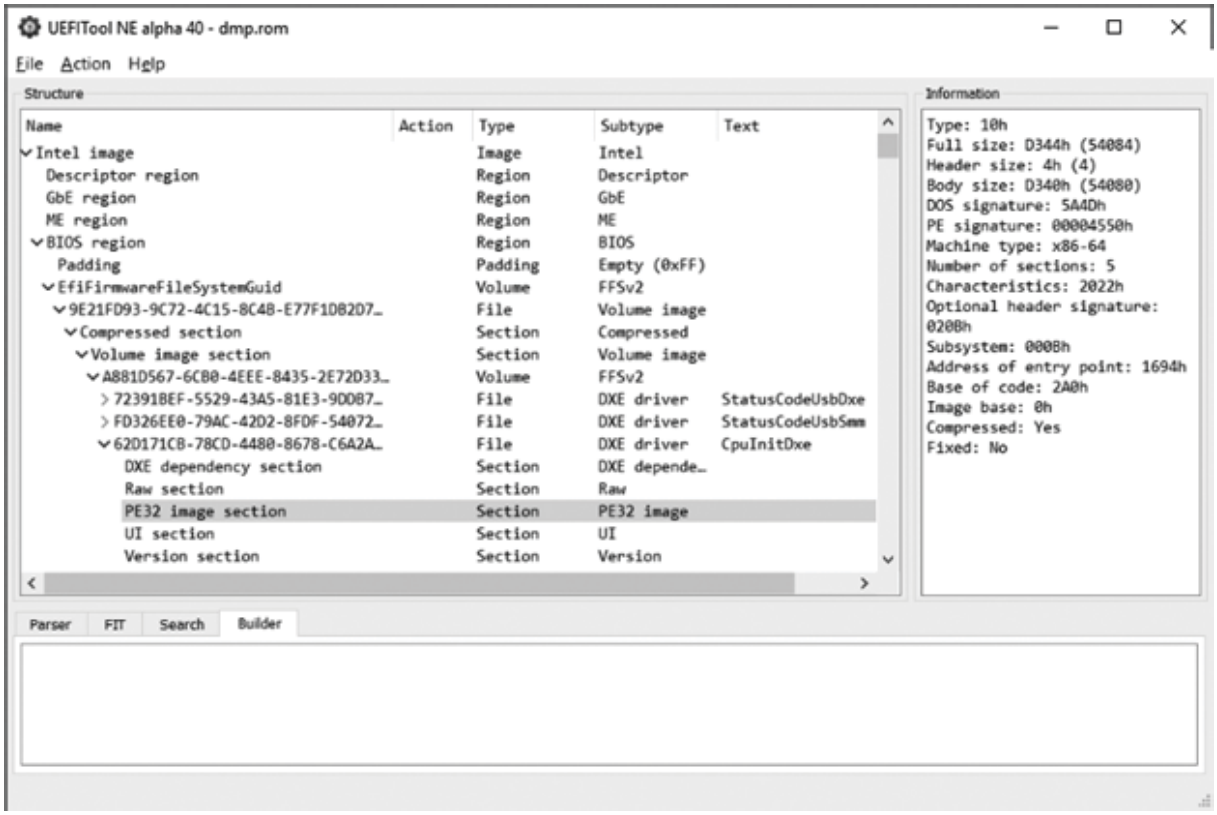


Рисунок 19-17. Расположение драйвера CpuInitDxe в области BIOS

Исполняемый образ драйвера расположен в разделе образа PE32 ❼. Этот раздел вместе с другими разделами, содержащими имя драйвера ❽, версию ❾ и зависимости ❻, находится в файле с GUID {62D171CB-78CD-4480-8678-C6A2A797A8DE} ❺. Файл является частью инкапсулированного тома прошивки ❹, хранящегося в сжатом разделе ❸. Сжатый раздел расположен в файле {9E21FD93-9C72-4C15-8C4B-E77F1DB2D792} ❷ типа образа тома прошивки, который хранится в томе прошивки верхнего уровня ❶.

Этот пример прежде всего предназначен для демонстрации иерархии объектов, составляющих прошивку UEFI, но это лишь один из возможных подходов к её разбору.

Теперь, когда нам известна организация области BIOS, мы сможем перемещаться по её иерархии и искать различные объекты, хранящиеся в прошивке BIOS.

Анализ образа прошивки при помощи Chipsec

В этом разделе мы обсудим криминалистический анализ прошивки при помощи представленной в главе 15 среды оценки безопасности платформы Chipsec. Мы подробнее изучим архитектуру инструмента, а затем проанализируем прошивку и приведём несколько примеров, демонстрирующих функциональность и полезность Chipsec.

Инструмент предоставляет ряд интерфейсов для доступа к аппаратным ресурсам платформы, таким как физическая память, регистры PCI, переменные NVRAM и флеш-память SPI. Эти интерфейсы очень полезны криминалистам, и далее в этом разделе мы рассмотрим их внимательнее.

Чтобы установить и настроить инструмент, следуйте руководству по установке в документации Chipsec. В документации также описано множество доступных функций, но в этом разделе мы сосредоточимся только на возможностях Chipsec для криминалистического анализа.

Знакомство с архитектурой Chipsec

На рисунке 19-18 показана высокоуровневая архитектура инструмента.

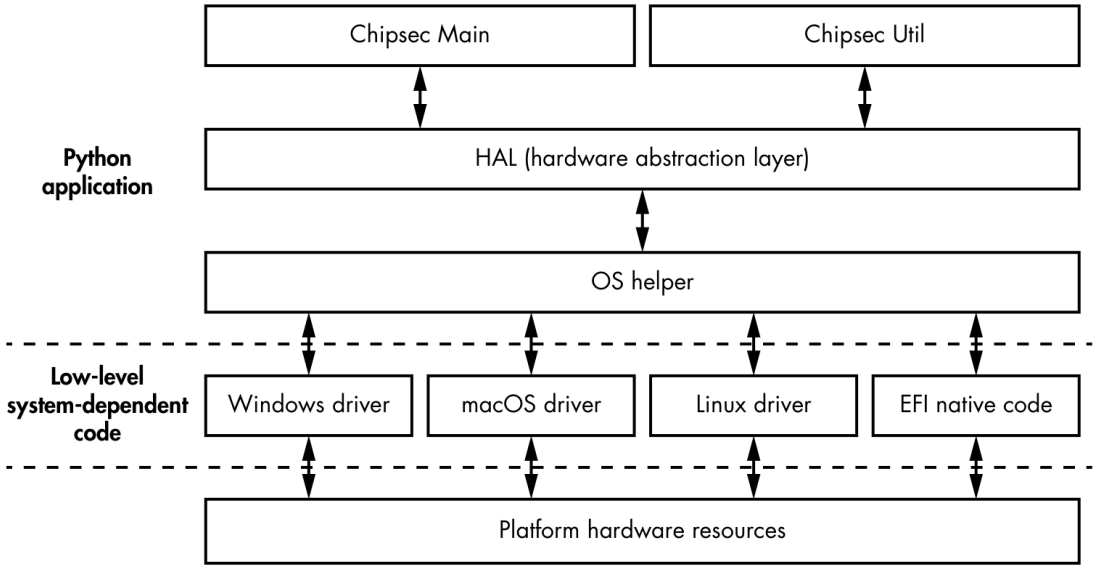


Рисунок 19-18. Архитектура инструмента Chipsec

В нижней части расположены модули, предоставляющие доступ к системным ресурсам, таким как диапазоны адресов ввода-вывода, отображённые в память, регистры конфигурационного пространства PCI и физическая память. Эти зависящие от платформы модули реализованы как драйверы режима ядра и собственный код EFI. В настоящее время Chipsec предоставляет драйверы режима ядра для Windows, Linux и macOS. Большинство модулей написаны на C и предназначены для выполнения в режиме ядра или в оболочке EFI.

> **Примечание.** UEFI Shell представляет собой приложение UEFI, предоставляющее интерфейс командной строки для прошивки и позволяющее запускать приложения UEFI и выполнять команды. Оболочку UEFI можно использовать для получения сведений о платформе, просмотра и изменения переменных диспетчера загрузки, загрузки драйверов UEFI и многого другого.

Над этими низкоуровневыми, зависящими от ОС компонентами находится независимый от ОС уровень абстракции под названием OS Helper, состоящий из нескольких модулей, которые скрывают от остальной части приложения зависящий от ОС API для взаимодействия с компонентами режима ядра. Модули, расположенные на этом уровне, реализованы на Python. Снизу эти модули взаимодействуют с компонентами режима ядра, а сверху предоставляют независимый от ОС интерфейс другому компоненту, уровню аппаратных абстракций (hardware abstraction layer, HAL).

HAL дополнительно абстрагирует низкоуровневые понятия платформы, такие как конфигурационные регистры PCI и регистры, зависящие от модели (model-specific registers, MSR), и предоставляет интерфейс компонентам Chipsec, расположенным непосредственно над ним: Chipsec Main и Chipsec Util. HAL также написан на Python и использует OSHelper для доступа к зависящим от платформы аппаратным ресурсам.

Два оставшихся компонента, расположенные на верхнем уровне архитектуры, предоставляют основную функциональность, доступную пользователям. Первый интерфейс, Chipsec Main, доступен через сценарий Python `chipsec_main.py` в корневой папке инструмента. Он позволяет выполнять тесты, проверяющие конфигурацию безопасности определённых аспектов платформы, запускать PoC для проверки наличия уязвимостей в системной прошивке и многое другое. Второй интерфейс, Chipsec Util, доступен через сценарий `chipsec_util.py`. Его можно использовать для выполнения отдельных команд и доступа к аппаратным ресурсам платформы, чтобы прочитать образ флеш-памяти SPI, выгрузить переменные NVRAM UEFI и так далее.

Нас прежде всего интересует интерфейс Chipsec Util, поскольку он предоставляет обширную функциональность для работы с прошивкой UEFI.

Анализ прошивки при помощи Chipsec Util

Чтобы узнать, какие команды предоставляет Chipsec Util, можно запустить расположенный в корневом каталоге репозитория инструмента сценарий `chipsec_util.py`, не указывая никаких аргументов. Как правило, команды сгруппированы в модули в соответствии с аппаратными ресурсами платформы, с которыми они работают. Ниже перечислены некоторые из наиболее полезных модулей:

- `acpi` - реализует команды для работы с таблицами Advanced Configuration and Power Interface.
- `cpu` - реализует команды, связанные с центральным процессором, например чтение конфигурационных регистров и получение сведений о центральном процессоре.
- `spi` - реализует ряд команд для работы с флеш-памятью SPI, включая чтение, запись и стирание данных. Также имеется возможность отключить защиту BIOS от записи в системах с разблокированной защитой от записи, как обсуждалось в главе 16.
- `uefi` - реализует команды для разбора прошивки UEFI, то есть области BIOS флеш-памяти SPI, с целью извлечения исполняемых файлов, переменных NVRAM и других объектов.

Можно выполнить `chipsec_util.py command_name`, где `command_name` обозначает команду, сведения о которой требуется получить, чтобы вывести её описание и информацию об использовании. Например, в листинге 19-2 показан вывод `chipsec_util.py spi`.

```
################################################################
##                                                            ##
##  CHIPSEC: Platform Hardware Security Assessment Framework  ##
##                                                            ##

################################################################
[CHIPSEC] Version 1.3.3h
[CHIPSEC] API mode: using OS native API (not using CHIPSEC kernel module)
[CHIPSEC] Executing command 'spi' with args []
❶ >>> chipsec_util spi info|dump|read|write|erase|disable-wp
[flash_address] [length] [file]
    Examples:
    >>> chipsec_util spi info
    >>> chipsec_util spi dump rom.bin
    >>> chipsec_util spi read 0x700000 0x100000 bios.bin
    >>> chipsec_util spi write 0x0 flash_descriptor.bin
    >>> chipsec_util spi disable-wp
```

Листинг 19-2. Описание модуля `spi` *и сведения о его использовании*

Это полезно, когда требуется узнать поддерживаемые параметры команд с говорящими именами, таких как `info`, `read`, `write`, `erase` или `disable-wp` ❶. В следующих примерах мы главным образом будем использовать команды `spi` и `uefi` для получения и распаковки образа прошивки.

Выгрузка и разбор образа флеш-памяти SPI

Сначала рассмотрим `spi`, позволяющий получать прошивку. Эта команда применяет программный подход к выгрузке содержимого флеш-памяти SPI. Для получения образа флеш-памяти SPI можно выполнить следующую команду:

```
chipsec_util.py spi dump path_to_file
```

где `path_to_file` обозначает путь к расположению, в котором требуется сохранить образ SPI. После успешного выполнения этой команды файл будет содержать образ флеш-памяти.

Теперь, располагая образом флеш-памяти SPI, мы можем разобрать его и извлечь полезную информацию при помощи команды `decode`. Стоит упомянуть, что саму команду `decode` можно использовать для разбора образа флеш-памяти SPI, полученного аппаратным методом. Команда выглядит следующим образом:

```
chipsec_util.py decode path_to_file
```

Здесь `path_to_file` указывает на файл с образом флеш-памяти SPI. Chipsec разберёт и извлечёт данные, хранящиеся в образе флеш-памяти, и сохранит их в каталоге. Эту задачу также можно выполнить командой `uefi` с параметром `decode`:

```
chipsec_util.py uefi decode path_to_file
```

После успешного выполнения команды мы получим набор объектов, извлечённых из образа, таких как исполняемые файлы, файлы данных с переменными NVRAM и разделы файлов.

Выгрузка переменных NVRAM UEFI

Теперь мы используем Chipsec для перечисления и извлечения переменных UEFI из образа флеш-памяти SPI. В главе 17 мы кратко рассматривали использование `chipsec uefi var-list` для извлечения переменных NVRAM. UEFI Secure Boot использует переменные NVRAM для хранения конфигурационных данных, таких как значение политики Secure Boot, ключ платформы, ключи обмена ключами и данные `db` и `dbx`. Выполнение этой команды создаёт список всех переменных NVRAM UEFI, хранящихся в образе прошивки, вместе с их содержимым и атрибутами.

Это лишь несколько команд из богатого арсенала инструмента Chipsec. Для полного списка всех вариантов использования Chipsec потребовалась бы отдельная книга, но если вас заинтересовал этот инструмент, рекомендуем обратиться к его документации.

На этом анализ образа прошивки при помощи Chipsec завершён. После выполнения этих команд мы получаем извлечённое содержимое образа прошивки. Следующим этапом криминалистического анализа будет отдельный анализ извлечённых компонентов с использованием инструментов, соответствующих типу извлечённого объекта. Например, модули PEI и DXE можно анализировать при помощи дизассемблера IDA Pro, а переменные NVRAM UEFI просматривать в шестнадцатеричном редакторе.

Этот список команд Chipsec служит хорошей отправной точкой для дальнейшего изучения прошивки UEFI. Мы призываем вас поэкспериментировать с этим инструментом и обратиться к руководству, чтобы узнать о других его возможностях и функциях и углубить свои знания о криминалистическом анализе прошивок.

Заключение

В этой главе мы обсудили важные подходы к криминалистическому анализу прошивок UEFI: получение прошивки, а также разбор и извлечение информации из образа прошивки UEFI.

Мы рассмотрели два различных способа получения прошивки: программный и аппаратный подходы. Программный подход удобен, но не предоставляет полностью заслуживающего доверия способа получить образ прошивки целевой системы. Поэтому, несмотря на большую сложность, мы рекомендуем аппаратный подход.

Мы также продемонстрировали использование двух инструментов с открытым исходным кодом, незаменимых для анализа и обратной разработки образов флеш-памяти SPI: UEFITool и Chipsec. UEFITool предоставляет функциональность для просмотра, изменения и извлечения криминалистических данных из образа флеш-памяти SPI, а Chipsec полезен для выполнения множества операций, необходимых при криминалистическом анализе. Использование Chipsec также показывает, насколько легко злоумышленник может изменить образ прошивки вредоносной полезной нагрузкой, поэтому мы ожидаем значительного роста интереса к криминалистическому анализу прошивок в индустрии безопасности.